

Golang เล่ม 1

ครอบคลุมทุกมิติ ตั้งแต่พื้นฐานสู่สถาปัตยกรรมระดับองค์กร พร้อมแผนภาพและโค้ดตัวอย่างที่รันได้จริง

Version 3.0 – เมษายน 2026

สารบัญ

- ภาคที่ 1: ปฐมบทกับการเขียนโปรแกรม
 - บทที่ 1: ความรู้เบื้องต้นเกี่ยวกับการเขียนโปรแกรมคอมพิวเตอร์
 - บทที่ 2: รู้จักกับภาษา Go
 - บทที่ 3: พื้นฐานการใช้งาน Terminal
 - บทที่ 4: เตรียมสภาพแวดล้อมสำหรับพัฒนา
 - บทที่ 5: สร้างแอปพลิเคชันแรกของคุณ
- ภาคที่ 2: พื้นฐานภาษาและโครงสร้างข้อมูล
 - บทที่ 6: ระบบเลขฐานสองและฐานสิบ
 - บทที่ 7: เลขฐานสิบหก, ฐานแปด, ASCII, UTF8, Unicode และ Runes
 - บทที่ 8: ตัวแปร, ค่าคงที่ และชนิดข้อมูลพื้นฐาน
 - บทที่ 9: คำสั่งควบคุมการทำงาน
 - บทที่ 10: ฟังก์ชัน
 - บทที่ 11: แพคเกจและการนำเข้า
 - บทที่ 12: การเริ่มต้นทำงานของแพคเกจ
 - บทที่ 13: การสร้างชนิดข้อมูลใหม่ (Types)
 - บทที่ 14: เมธอด (Methods)
 - บทที่ 15: พอยน์เตอร์ (Pointer)
 - บทที่ 16: อินเทอร์เฟซ (Interfaces)
- ภาคที่ 3: การจัดการโปรเจกต์และโครงสร้างข้อมูลขั้นสูง
 - บทที่ 17: Go Modules - การจัดการโปรเจกต์สมัยใหม่
 - บทที่ 18: Go Module Proxies
 - บทที่ 19: การทดสอบหน่วย (Unit Tests)
 - บทที่ 20: อาร์เรย์ (Arrays)
 - บทที่ 21: สไลซ์ (Slices)
 - บทที่ 22: แมพ (Maps)
 - บทที่ 23: การจัดการข้อผิดพลาด (Errors)
- ภาคที่ 4: การพัฒนาแอปพลิเคชันเชิงปฏิบัติ
 - บทที่ 24: ฟังก์ชันนิรนาม (Anonymous functions) และ Closure
 - บทที่ 25: การจัดการข้อมูล JSON และ XML
 - บทที่ 26: พื้นฐานการสร้าง HTTP Server

- บทที่ 27: Enum, iota และ Bitmask
- บทที่ 28: วันที่และเวลา
- บทที่ 29: การจัดเก็บข้อมูล: ไฟล์และฐานข้อมูล
- บทที่ 30: การทำงานพร้อมกัน (Concurrency)
- บทที่ 31: การบันทึกเหตุการณ์ (Logging)
- บทที่ 32: เทมเพลต (Templates)
- บทที่ 33: การจัดการค่า Configuration
- ภาคที่ 5: สู่การเป็นนักพัฒนา Go มืออาชีพ
 - บทที่ 34: การวัดประสิทธิภาพ (Benchmarks)
 - บทที่ 35: สร้าง HTTP Client
 - บทที่ 36: การวิเคราะห์โปรไฟล์ (Program Profiling)
 - บทที่ 37: การจัดการ Context
 - บทที่ 38: Generics - การเขียนโค้ดแบบยืดหยุ่น
 - บทที่ 39: Go กับกระบวนการทัศน์ OOP?
 - บทที่ 40: การอัปเดตหรือดาวน์เกรดเวอร์ชัน Go
 - บทที่ 41: คำแนะนำในการออกแบบโค้ดที่ดี
 - บทที่ 42: ชีทสรุป (Cheatsheet)
- ภาคที่ 6: เครื่องมือและไลบรารียอดนิยม
 - บทที่ 43: chi, viper, cobra, zap และเครื่องมือสำคัญ
 - บทที่ 44: GORM – ORM ทรงพลังสำหรับ Go
 - บทที่ 45: การส่งอีเมลด้วย gmail และ hermes
- ภาคที่ 7: การออกแบบสถาปัตยกรรมและ Workflow
 - บทที่ 46: Clean Architecture และโครงสร้างโปรเจกต์
 - บทที่ 47: Blueprint สำหรับโปรเจกต์ Go ระดับ Production
 - บทที่ 48: การออกแบบ Workflow และ Task Management
- ภาคที่ 8: Domain-Driven Design (DDD) กับ Go
 - บทที่ 49: หลักการ DDD และการนำไปใช้ใน Go
 - บทที่ 50: Aggregates, Event Storming และ CQRS
 - บทที่ 51: การออกแบบบริการด้วย Go-DDD
- ภาคที่ 9: การผสานระบบภายนอกและคุณลักษณะเสริม (Advanced Integrations)
 - บทที่ 52: Redis สำหรับ Cache และ Message Queue
 - บทที่ 53: RabbitMQ – Message Broker มาตรฐานองค์กร
 - บทที่ 54: MQTT สำหรับ IoT และระบบเรียลไทม์
 - บทที่ 55: InfluxDB – Time-Series Database
 - บทที่ 56: WebSocket และ Socket.IO
 - บทที่ 57: การส่ง SMS และ LINE Notify
 - บทที่ 58: Discord Webhook สำหรับแจ้งเตือน
- ภาคที่ 10: เทมเพลต กระบวนการพัฒนา และตัวอย่างโค้ด

- บทที่ 59: ตัวอย่างโค้ดครบวงจร (Full-stack Example)
- บทที่ 60: Task List Template
- บทที่ 61: Checklist Template
- บทที่ 62: แผนภาพการทำงาน (Workflow Diagram)
- บทที่ 63: mop Config – การจัดการ Configuration

ภาคผนวก

- GORM CRUD กับฐานข้อมูลหลายประเภท (PostgreSQL, MySQL, MongoDB, InfluxDB)
- คู่มือภาษา Go ฉบับ นำไปทำงาน
- Golang Live-reload เมื่อ code เปลี่ยนแปลง

ภาคที่ 1: ปฐมบทกับการเขียนโปรแกรม

บทที่ 1: ความรู้เบื้องต้นเกี่ยวกับการเขียนโปรแกรมคอมพิวเตอร์

1.1 การเขียนโปรแกรมคืออะไร?

การเขียนโปรแกรม (Programming) คือกระบวนการสร้างชุดคำสั่งที่ใช้ควบคุมการทำงานของคอมพิวเตอร์ให้ทำงานตามที่เราต้องการ โดยใช้ภาษาเฉพาะที่คอมพิวเตอร์สามารถเข้าใจได้ ภาษาที่มนุษย์ใช้เขียนเรียกว่า "ภาษาคอมพิวเตอร์ระดับสูง" (High-Level Language) เช่น Go, Python, Java ซึ่งจากนั้นจะถูกแปลงเป็นภาษาเครื่อง (Machine Language) ที่เป็นเลขฐานสอง (0 และ 1) ที่ซีพียูสามารถประมวลผลได้

1.2 โครงสร้างพื้นฐานของโปรแกรม

โปรแกรมคอมพิวเตอร์โดยทั่วไปประกอบด้วย:

- **ข้อมูล (Data)** : ตัวเลข, ข้อความ, รายการต่างๆ
- **การประมวลผล (Processing)** : การดำเนินการกับข้อมูล เช่น การคำนวณ การเปรียบเทียบ
- **การควบคุมการทำงาน (Control Flow)** : การตัดสินใจ (if-else), การวนซ้ำ (loop)
- **การจัดเก็บ (Storage)** : หน่วยความจำ, ไฟล์, ฐานข้อมูล
- **อินพุต/เอาต์พุต (I/O)** : การรับข้อมูลจากผู้ใช้ หรือแสดงผล

1.3 ตัวแปลภาษาและคอมไพเลอร์

- **คอมไพเลอร์ (Compiler)** : แปลงซอร์สโค้ดทั้งหมดเป็นไฟล์ไบนารีได้ก่อนรัน (เช่น Go, C)
- **อินเทอร์พรีเตอร์ (Interpreter)** : แปลงและรันทีละคำสั่ง (เช่น Python, JavaScript)

Go เป็นภาษาแบบคอมไพล์ (compiled) ซึ่งมีข้อดีคือทำงานเร็วและสร้างไฟล์ binary ที่รันได้ทันทีโดยไม่ต้องพึ่งพาสิ่งแวดล้อมอื่น (ยกเว้นระบบปฏิบัติการ)

1.4 กระบวนการเขียนโปรแกรม

Go รองรับวิธีการเขียนโปรแกรมแบบ:

- **Procedural** : ใช้ฟังก์ชันและลำดับขั้นตอน
- **Concurrent** : ทำงานพร้อมกันด้วย goroutine

- **Functional** (บางส่วน) : ฟังก์ชันเป็น first-class citizen
 - **ไม่ใช่ OOP แบบคลาสสิก** : ใช้ struct และ interface แทน inheritance
-

1.5 Golang Procedural คืออะไร

Procedural หรือการเขียนโปรแกรมแบบเชิงกระบวนการ เป็นรูปแบบที่เน้นการเขียนฟังก์ชัน (function) และเรียกใช้ตามลำดับขั้นตอน โกลแลง (Go) รองรับการเขียนแบบนี้โดยใช้ฟังก์ชันเป็นหน่วยหลัก โค้ดจะถูกแบ่งออกเป็นฟังก์ชันย่อย ๆ ที่ทำงานเฉพาะอย่าง และเรียกใช้ตามลำดับที่กำหนด ถึงแม้ Go จะไม่ใช่ภาษา procedural ล้วน ๆ แต่ก็สามารถเขียนในสไตล์นี้ได้อย่างดี

1.6 Concurrent คืออะไร

Concurrency (การทำงานร่วมกัน) คือความสามารถของโปรแกรมในการจัดการงานหลาย ๆ งานพร้อมกัน โดยงานเหล่านี้อาจเริ่มทำงาน สลับการทำงาน หรือจบลงในเวลาที่ไม่เหมือนกัน โดยไม่จำเป็นต้องทำงานพร้อมกันจริง ๆ (parallel) Go มีเครื่องมือสำหรับ concurrency โดยตรง ได้แก่ **goroutine** และ **channel** ทำให้การเขียนโปรแกรม concurrent ง่ายและมีประสิทธิภาพ

1.7 goroutine คืออะไร

goroutine คือเธรด (thread) ขนาดเบาที่ถูกจัดการโดย runtime ของ Go ใช้สำหรับทำงานแบบ concurrent การสร้าง goroutine ทำได้ง่ายโดยใช้คำยี่ห้อ **go** หน้าฟังก์ชัน:

```
go myFunction()
```

Goroutine มีน้ำหนักเบา ใช้สแต็กเริ่มต้นเพียงไม่กี่กิโลไบต์ และสามารถทำงานหลายหมื่นตัวพร้อมกันได้โปรแกรมเดียว

1.8 Functional คืออะไร

Functional programming (การเขียนโปรแกรมเชิงฟังก์ชัน) เป็นกระบวนทัศน์ที่เน้นการใช้ฟังก์ชันบริสุทธิ์ (pure function) ไม่มีการเปลี่ยนแปลงสถานะภายนอก และใช้ฟังก์ชันเป็นพลเมืองชั้นหนึ่ง (first-class citizen) Go รองรับคุณสมบัติบางอย่างของ functional เช่น การส่งฟังก์ชันเป็นค่าพารามิเตอร์ การคืนค่าฟังก์ชัน แต่ไม่มีการรับประกันความบริสุทธิ์ของฟังก์ชันหรือโครงสร้างข้อมูลแบบ immutable โดยตรง

1.9 first-class citizen คืออะไร

First-class citizen (พลเมืองชั้นหนึ่ง) หมายถึงสิ่งที่สามารถทำได้เช่นเดียวกับค่าอื่น ๆ ในภาษา เช่น ถูกกำหนดให้กับตัวแปร ถูกส่งเป็นอาร์กิวเมนต์ให้ฟังก์ชัน ถูกคืนค่าจากฟังก์ชัน หรือถูกเก็บในโครงสร้างข้อมูล ใน Go **ฟังก์ชัน** ถือเป็น first-class citizen:

```
var fn func(int) int = func(x int) int { return x * x }
```

นอกจากนี้ ชนิดข้อมูลต่าง ๆ เช่น struct, interface ก็ถือเป็น first-class citizen เช่นกัน

1.10 OOP คืออะไร

OOP (Object-Oriented Programming) เป็นกระบวนทัศน์ที่เน้นการสร้างวัตถุ (object) ซึ่งรวมข้อมูลและพฤติกรรมเข้าด้วยกัน หลักการสำคัญคือ encapsulation, inheritance, และ polymorphism Go ไม่ใช่ภาษา OOP แบบดั้งเดิมเพราะไม่มี class แต่ใช้ **struct** เป็นตัวเก็บข้อมูล และใช้ **method** ที่ผูกกับ struct รวมถึง **interface** เพื่อสร้าง polymorphism การสืบทอด (inheritance) ทำได้โดยการ **composition** (การแทรก struct) แทน

1.11 struct คืออะไร

struct เป็นชนิดข้อมูลที่ใช้รวมฟิลด์ (field) หลาย ๆ ชนิดเข้าด้วยกัน คล้ายกับ class ในภาษาอื่น แต่ไม่มี method ในตัว เราสามารถกำหนด method ให้กับ struct ได้โดยประกาศฟังก์ชันที่มี **receiver** เป็น struct นั้น

```
type Person struct {
    Name string
    Age  int
}

func (p Person) Greet() string {
    return "Hello, " + p.Name
}
```

1.12 interface คืออะไร

interface คือชนิดข้อมูลที่กำหนดชุดของ method signature (ชื่อฟังก์ชัน, พารามิเตอร์, ผลลัพธ์) โดยไม่ระบุการนำไปใช้ ถ้าชนิดใด (เช่น struct) มี method ครบตามที่ interface กำหนด ชนิดนั้นจะถูกพิจารณาว่า **implement** interface นั้นโดยอัตโนมัติ (implicit implementation) ทำให้เกิด polymorphism แบบหนึ่งใน Go

```
type Greeter interface {
    Greet() string
}

// Person มี method Greet() string ดังนั้น Person จึงเป็น Greeter โดยอัตโนมัติ
```

1.13 inheritance คืออะไร

Inheritance (การสืบทอด) คือกลไกที่ class หนึ่งสามารถสืบทอดสมาชิก (ฟิลด์และ method) จากอีก class หนึ่ง เพื่อนำมาใช้หรือขยายความสามารถ Go **ไม่มี** inheritance แบบ class-based แต่นิยมใช้ **composition** โดยการฝัง struct (embedded struct) เพื่อให้ได้ผลลัพธ์คล้ายการสืบทอด แต่ยังคงความยืดหยุ่นและหลีกเลี่ยงปัญหาที่เกิดจาก inheritance ที่ซับซ้อน

```
type Animal struct {
    Name string
}
```

```
type Dog struct {  
    Animal // ผัง struct Animal เข้ามา  
    Breed string  
}
```

Dog จะสามารถเข้าถึงฟิลด์ Name และ method (ถ้ามี) ของ Animal ได้โดยตรง

1. Procedural Programming

Procedural คืออะไร?

Procedural programming (การเขียนโปรแกรมแบบกระบวนการ) เป็นกระบวนการที่เน้นการเขียนโค้ดเป็นชุดของคำสั่งหรือฟังก์ชัน (procedures / routines) ที่ทำงานตามลำดับขั้นตอน (step-by-step) โดยใช้ตัวแปร โครงสร้างควบคุม (if, for, switch) และฟังก์ชันที่เรียกซ้ำกันได้

Procedural มีกี่แบบ?

ในทางปฏิบัติ Procedural programming ไม่ได้แบ่งเป็น “แบบ” อย่างเป็นทางการ แต่สามารถมองได้ตาม โครงสร้างของโค้ด:

- **Linear / Sequential** – คำสั่งทำงานเรียงตามลำดับ
- **Modular** – แบ่งโค้ดออกเป็นฟังก์ชัน/โมดูล
- **Structured** – ใช้โครงสร้างควบคุม (sequence, selection, iteration) หลีกเลี่ยง goto

ภาษา Go มีพื้นฐานเป็น procedural โดยมีการจัดระเบียบผ่านฟังก์ชันและแพ็คเกจ

ใช้อย่างไร ในกรณีไหน?

- เขียนโปรแกรมที่มีขั้นตอนชัดเจน (linear flow)
- งานที่ต้องการประสิทธิภาพสูง ควบคุมทรัพยากรใกล้เคียงฮาร์ดแวร์
- โค้ดขนาดเล็กถึงกลาง ไม่ต้องการความซับซ้อนของ OOP
- Go มักใช้ procedural ร่วมกับ concurrent และ OOP (ผ่าน struct & interface)

หลักการทำงาน

1. โปรแกรมเริ่มทำงานจากฟังก์ชัน `main()`
2. เรียกฟังก์ชันตามลำดับ อาจมีการส่งค่าและรับค่ากลับ
3. ตัวแปรมีขอบเขตตาม block หรือ package
4. การทำงานเป็นแบบ imperative: เปลี่ยนแปลงสถานะของตัวแปรโดยตรง

Dataflow Diagram (Flowchart TB)

```
graph TB  
    Start([เริ่ม main]) --> FuncA[เรียก Function A]  
    FuncA --> FuncB[เรียก Function B]  
    FuncB --> Cond{เงื่อนไข?}
```

```
Cond -- true --> Action1[ทำ Action 1]
Cond -- false --> Action2[ทำ Action 2]
Action1 --> End([จบ])
Action2 --> End
```

ตัวอย่างการใช้งานจริง: โปรแกรมคำนวณราคาสินค้า

```
package main

import "fmt"

// ฟังก์ชันคำนวณราคารวม
func calculateTotal(price float64, quantity int) float64 {
    return price * float64(quantity)
}

// ฟังก์ชันคำนวณส่วนลด
func applyDiscount(total float64, discountPercent float64) float64 {
    return total * (1 - discountPercent/100)
}

// ฟังก์ชันหลัก
func main() {
    price := 250.0
    qty := 3
    discount := 10.0

    total := calculateTotal(price, qty)
    fmt.Printf("Total: %.2f\n", total)

    finalPrice := applyDiscount(total, discount)
    fmt.Printf("After discount: %.2f\n", finalPrice)
}
```

เทมเพลตโครงสร้างโปรเจกต์แบบ Procedural ใน Go

```
project/
├─ main.go
├─ handlers/      # ฟังก์ชันจัดการ request
├─ services/      # ฟังก์ชัน business logic
├─ repositories/  # ฟังก์ชันติดต่อฐานข้อมูล
└─ utils/         # ฟังก์ชันช่วยเหลือ
```

โค้ดในแต่ละ package จะเป็นชุดฟังก์ชันที่ทำงานตามลำดับที่เรียกใช้

2. Concurrent Programming

Concurrent คืออะไร?

Concurrent programming คือการเขียนโปรแกรมให้สามารถทำงานหลายอย่าง **พร้อมกัน** (overlap in time) โดยไม่จำเป็นต้องทำพร้อมกันจริง (parallel) แต่เป็นการจัดการหลาย tasks สลับกันเพื่อเพิ่มประสิทธิภาพและความสามารถในการตอบสนอง

Go มี **goroutine** (เธรดน้ำหนักเบา) และ **channel** สำหรับการสื่อสารระหว่าง goroutines ทำให้การเขียน concurrent โปรแกรมง่ายและปลอดภัย

Concurrent มีกี่แบบ?

รูปแบบหลักในการเขียน concurrent code:

- **Goroutines + Channels** – CSP (Communicating Sequential Processes) แบบ Go
- **Mutex / Atomic** – ใช้การล็อกเพื่อป้องกัน race condition
- **Worker Pool** – ใช้ goroutines จำนวนคงที่ทำงานจากคิวงาน
- **Pipeline** – ข้อมูลไหลผ่านหลาย goroutines ที่เชื่อมต่อกันด้วย channel

ใช้อย่างไร ในกรณีไหน?

- งานที่ต้องการทำหลายอย่างพร้อมกัน (web server, API calls)
- งาน I/O-bound (อ่านไฟล์, เรียกฐานข้อมูล, HTTP request)
- ปรับปรุงประสิทธิภาพการทำงานของระบบ
- Queue processor, background jobs

หลักการทำงาน

1. สร้าง goroutine ด้วย `go function()`
2. Goroutine ทำงาน concurrent กับ main goroutine
3. ใช้ `channel` สำหรับส่งข้อมูลระหว่าง goroutines (synchronization)
4. ใช้ `sync.WaitGroup` หรือ `context` เพื่อควบคุม lifecycle

Dataflow Diagram (Flowchart TB) - Worker Pool

```
graph TB
    subgraph Main
        A[main] --> B[สร้าง Jobs Channel]
        B --> C[สร้าง Workers n ตัว]
        C --> D[ส่งงานเข้า Jobs Channel]
    end

    subgraph Worker1
        E[for job := range jobs] --> F[ประมวลผล job]
    end

    subgraph Worker2
        G[for job := range jobs] --> H[ประมวลผล job]
```

```

end

D --> E
D --> G
F --> I[ส่งผลลัพธ์]
H --> I
I --> J[main รวบรวมผล]

```

ตัวอย่างการใช้งานจริง: Download เว็บไซต์หลายๆ ที่พร้อมกัน

```

package main

import (
    "fmt"
    "net/http"
    "sync"
    "time"
)

func fetch(url string, wg *sync.WaitGroup, results chan<- string) {
    defer wg.Done()
    start := time.Now()
    resp, err := http.Get(url)
    if err != nil {
        results <- fmt.Sprintf("%s -> error: %v", url, err)
        return
    }
    defer resp.Body.Close()
    elapsed := time.Since(start)
    results <- fmt.Sprintf("%s -> %d ms", url, elapsed.Milliseconds())
}

func main() {
    urls := []string{
        "https://google.com",
        "https://github.com",
        "https://stackoverflow.com",
    }

    var wg sync.WaitGroup
    results := make(chan string, len(urls))

    for _, url := range urls {
        wg.Add(1)
        go fetch(url, &wg, results)
    }

    wg.Wait()
    close(results)
}

```

```

    for res := range results {
        fmt.Println(res)
    }
}

```

เทมเพลต Worker Pool

```

type Job struct { ID int; Data interface{} }
type Result struct { JobID int; Output interface{}; Err error }

func worker(jobs <-chan Job, results chan<- Result, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs {
        // process job
        results <- Result{JobID: job.ID, Output: processedData}
    }
}

func main() {
    jobs := make(chan Job, 100)
    results := make(chan Result, 100)

    // start workers
    var wg sync.WaitGroup
    for w := 0; w < 5; w++ {
        wg.Add(1)
        go worker(jobs, results, &wg)
    }

    // send jobs
    for i := 0; i < 100; i++ {
        jobs <- Job{ID: i, Data: ...}
    }
    close(jobs)
    wg.Wait()
    close(results)

    // collect results
    for res := range results { ... }
}

```

3. Functional Programming

Functional คืออะไร?

Functional programming (การเขียนโปรแกรมเชิงฟังก์ชัน) เป็นกระบวนทัศน์ที่มองการคำนวณเป็นการประเมินค่าของฟังก์ชัน โดยเน้น:

- **Pure functions** – ผลลัพธ์ขึ้นอยู่กับอินพุตเท่านั้น ไม่มี side effect
- **Immutability** – ข้อมูลไม่เปลี่ยนแปลงหลังสร้าง
- **First-class functions** – ฟังก์ชันเป็นค่าที่สามารถส่งต่อได้
- **Higher-order functions** – ฟังก์ชันรับฟังก์ชันเป็นพารามิเตอร์หรือคืนฟังก์ชัน

Go รองรับฟังก์ชัน first-class, closures, และ higher-order functions แต่ไม่มี immutability บังคับ (ใช้แนวทาง pragmatic functional)

Functional มีกี่แบบ?

- **Pure FP** – ใช้เฉพาะ pure functions, immutable data (Haskell, Elm)
- **Impure FP** – อนุญาต side effect บางส่วน (Go, JavaScript, Scala)
- **Higher-order functions** – map, filter, reduce
- **Function composition** – นำฟังก์ชันเล็ก ๆ มาต่อกัน

ใช้อย่างไร ในกรณีไหน?

- การแปลงข้อมูล (transform) ด้วย pipeline
- เขียนโค้ดที่คาดเดาได้ง่าย ทดสอบง่าย
- ลดความซับซ้อนในการจัดการสถานะ
- ใช้ร่วมกับ concurrent (pure functions ปลอดภัยต่อ race condition)

หลักการทำงาน

1. กำหนดฟังก์ชันเล็ก ๆ ที่ทำหน้าที่เฉพาะ
2. ใช้ higher-order functions (map, filter, reduce) กับ slice
3. หลีกเลี่ยงการเปลี่ยนแปลงตัวแปรภายนอก (immutability เท่าที่ทำได้)
4. สร้าง pipeline โดยการต่อฟังก์ชันเข้าด้วยกัน

Dataflow Diagram (Flowchart TB) - Pipeline

```
graph LR
  A[Input Data] --> B[Filter]
  B --> C[Map]
  C --> D[Reduce]
  D --> E[Output Result]
```

ตัวอย่างการใช้งานจริง: ประมวลผลข้อมูลผู้ใช้

```
package main

import (
    "fmt"
```

```

    "strings"
)

// Pure function
func toUpper(s string) string { return strings.ToUpper(s) }

// Higher-order filter
func filter(users []string, predicate func(string) bool) []string {
    var result []string
    for _, u := range users {
        if predicate(u) {
            result = append(result, u)
        }
    }
    return result
}

// Higher-order map
func mapFunc(users []string, fn func(string) string) []string {
    result := make([]string, len(users))
    for i, u := range users {
        result[i] = fn(u)
    }
    return result
}

func main() {
    users := []string{"alice", "bob", "charlie", "admin"}

    // Pipeline: filter user with length > 3, then convert to upper
    filtered := filter(users, func(u string) bool { return len(u) > 3 })
    upper := mapFunc(filtered, toUpper)

    fmt.Println(upper) // [CHARLIE ADMIN]
}

```

เทมเพลต: generic map/filter/reduce (Go 1.18+)

```

type Slice[T any] []T

func (s Slice[T]) Map(fn func(T) T) Slice[T] {
    result := make(Slice[T], len(s))
    for i, v := range s {
        result[i] = fn(v)
    }
    return result
}

func (s Slice[T]) Filter(pred func(T) bool) Slice[T] {
    result := Slice[T]{}

```



```

    for _, v := range s {
        if pred(v) {
            result = append(result, v)
        }
    }
    return result
}

```

4. Object-Oriented Programming (OOP)

OOP คืออะไร?

Object-Oriented Programming (การเขียนโปรแกรมเชิงวัตถุ) เป็นกระบวนทัศน์ที่จัดระเบียบโค้ดเป็น "วัตถุ" (objects) ซึ่งรวมข้อมูล (fields) และพฤติกรรม (methods) เข้าด้วยกัน เน้นหลักการ:

- **Encapsulation** – ซ่อนรายละเอียดภายใน
- **Inheritance** – สืบทอดคุณสมบัติจากคลาสแม่ (Go ใช้ composition แทน)
- **Polymorphism** – หลายรูปแบบผ่าน interface

ภาษา Go ไม่มีคลาสแบบดั้งเดิม แต่ใช้ **struct** แทนข้อมูล และ **interface** แทนพฤติกรรมร่วมกัน ทำให้ได้แนวคิด OOP แบบ lightweight

OOP มีกี่แบบ?

ตามกระบวนทัศน์ OOP แบ่งเป็น:

- **Class-based** (Java, C++, Python) – มีคลาสเป็นต้นแบบ
- **Prototype-based** (JavaScript) – วัตถุสืบทอดจากวัตถุอื่น
- Go ใช้ **struct + interface** เข้าถึง OOP ในรูปแบบ composition over inheritance

ใช้อย่างไร ในกรณีไหน?

- โปรแกรมที่ต้องการจำลองสิ่งของในโลกจริง (entity modeling)
- ต้องการความสามารถในการขยาย (extension) ผ่าน interface
- ต้องการ polymorphism โดยไม่ต้องใช้ inheritance ซับซ้อน
- โค้ดขนาดใหญ่ที่ต้องการ modularity

หลักการทำงาน

1. กำหนด **struct** สำหรับเก็บข้อมูล
2. เพิ่ม **method** ให้ struct ด้วย `func (r Receiver) MethodName()`
3. กำหนด **interface** เพื่อประกาศชุดของ method ที่ต้องมี
4. ใช้งาน polymorphism ผ่าน interface (dependency injection, mock)

Dataflow Diagram (Flowchart TB) - Polymorphism

```

graph TB
    A[Interface Speaker] --> B1[Dog.Speak]
    A --> B2[Cat.Speak]
    A --> B3[Robot.Speak]
    C[Function makeSound] --> A
    C --> D[เรียก s.Speak]

```

ตัวอย่างการใช้งานจริง: สัตว์เลี้ยงและหุ่นยนต์

```

package main

import "fmt"

// Interface
type Speaker interface {
    Speak() string
}

// Struct Dog
type Dog struct{ Name string }

func (d Dog) Speak() string {
    return fmt.Sprintf("%s says Woof!", d.Name)
}

// Struct Cat
type Cat struct{ Name string }

func (c Cat) Speak() string {
    return fmt.Sprintf("%s says Meow!", c.Name)
}

// Function ที่ใช้ polymorphism
func makeSound(s Speaker) {
    fmt.Println(s.Speak())
}

func main() {
    dog := Dog{Name: "Rex"}
    cat := Cat{Name: "Luna"}

    makeSound(dog) // Rex says Woof!
    makeSound(cat) // Luna says Meow!
}

```

เทมเพลต: โครงสร้าง OOP ใน Go

```
// entity.go
type Entity struct {
    ID    int
    Name  string
}

func (e Entity) GetID() int { return e.ID }

// repository.go
type Repository interface {
    Save(entity Entity) error
    Find(id int) (Entity, error)
}

type MySQLRepository struct {
    db *sql.DB
}

func (r MySQLRepository) Save(entity Entity) error { ... }
func (r MySQLRepository) Find(id int) (Entity, error) { ... }
```

ขยายความ เพื่อให้เห็นภาพชัด

- Encapsulation คืออะไร
- Encapsulation มีกี่แบบ
- Encapsulation ใช้อย่างไร ในกรณีไหน ใช้อย่างไร แสดงรูปการทำงาน

หลักการ OOP ในภาษา Go (Golang)

หมายเหตุสำคัญ: Go ไม่ใช่ภาษา OOP เต็มรูปแบบ (ไม่มี class, ไม่มี inheritance แบบดั้งเดิม) แต่มีแนวคิดที่คล้ายคลึงกันผ่าน **struct**, **interface**, และ **composition (การซ้อน struct)** โดย Go สนับสนุน Encapsulation และ Polymorphism ได้ดี ส่วน Inheritance ใช้ **Embedding** แทน

1. Encapsulation (การห่อหุ้มข้อมูล) ใน Go

Encapsulation คืออะไรใน Go?

Encapsulation ใน Go ทำได้โดยควบคุม **การมองเห็น (visibility)** ผ่าน **ตัวพิมพ์ใหญ่/เล็ก** ของชื่อฟิลด์หรือเมธอด:

- **ตัวพิมพ์ใหญ่ (Exported)** → เข้าถึงได้จากแพ็คเกจอื่น (public)
- **ตัวพิมพ์เล็ก (Unexported)** → เข้าถึงได้เฉพาะภายในแพ็คเกจเดียวกัน (private)

Go ไม่มี getter/setter อัตโนมัติ แต่เราสร้างฟังก์ชันหรือเมธอดเพื่อควบคุมการเข้าถึง

Encapsulation มีกี่แบบใน Go?

แบ่งตามระดับการมองเห็นได้ **2 แบบ** (เรียบง่ายกว่า Java):

ระดับ	การตั้งชื่อ	การเข้าถึง
Exported (public)	ขึ้นต้นด้วยตัวพิมพ์ใหญ่ เช่น <code>Balance</code> , <code>GetBalance()</code>	ทุกแพ็คเกจ
Unexported (private)	ขึ้นต้นด้วยตัวพิมพ์เล็ก เช่น <code>balance</code> , <code>setBalance()</code>	เฉพาะแพ็คเกจเดียวกัน

🌀 ใช้อย่างไร? ในกรณีไหน?

- **ใช้เมื่อ** ต้องการควบคุมการแก้ไขข้อมูล ป้องกันสถานะไม่ถูกต้อง
- **กรณีใช้งานจริง:**
 - ระบบบัญชีธนาคาร (ห้ามให้ยอดติดลบ)
 - ระบบผู้ใช้ (ห้ามตั้งรหัสผ่านสั้นเกินไป)
 - ข้อมูลการตั้งค่า (config) ที่อ่านอย่างเดียว

📐 แผนภาพ Dataflow (Flowchart TB) – Encapsulation ใน Go

```
flowchart TB
    subgraph External["แพ็คเกจอื่น (client)"]
        A[เรียกใช้เมธอด SetBalance / GetBalance]
    end

    subgraph Internal["แพ็คเกจเดียวกัน (account)"]
        B["เมธอด Exported (พิมพ์ใหญ่)<br>func (a *Account) SetBalance(amt float64)"]
        C["ตรวจสอบความถูกต้อง<br>amt >= 0 ?"]
        D["ฟิลต์ unexported (พิมพ์เล็ก)<br>balance float64"]
        E["เมธอด GetBalance() float64"]
    end

    F[ส่งค่ากลับ / แสดงผล]

    A --> B
    B --> C
    C -- ผ่าน --> D
    C -- ไม่ผ่าน --> G[คืน error หรือ log]
    D --> E
    E --> F
    G --> F
```

คำอธิบายแผนภาพ:

- ฟิลต์ `balance` เป็นตัวพิมพ์เล็ก → มองไม่เห็นจากนอกแพ็คเกจ
- เมธอด `SetBalance` และ `GetBalance` เป็นตัวพิมพ์ใหญ่ → เรียกใช้จากภายนอกได้
- การเปลี่ยนแปลงค่าทำผ่านเมธอดที่มี validation เท่านั้น

```
// bank/account.go (แพ็คเกจ bank)
package bank

import "errors"

// Account คือ struct ที่มี encapsulation (การห่อหุ้มข้อมูล)
// ฟังก์ชันทั้งหมดเป็น unexported (ขึ้นต้นด้วยตัวพิมพ์เล็ก) ทำให้เข้าถึงได้เฉพาะภายในแพ็คเกจ bank เท่านั้น
type Account struct {
    // unexported field (private) - ไม่สามารถเข้าถึงโดยตรงจากภายนอกแพ็คเกจ
    accountNumber string
    balance        float64
}

// Constructor (ฟังก์ชัน exported - ขึ้นต้นด้วยตัวพิมพ์ใหญ่)
// ใช้สำหรับสร้าง instance ของ Account อย่างถูกต้อง (เหมือนการสร้าง object ใน OOP)
func NewAccount(number string, initialBalance float64) *Account {
    // ตรวจสอบเงื่อนไขเบื้องต้น: ยอดเริ่มต้นต้องไม่ติดลบ
    if initialBalance < 0 {
        initialBalance = 0
    }
    // คืนค่า pointer ของ Account ที่สร้างขึ้นใหม่
    return &Account{
        accountNumber: number,
        balance:        initialBalance,
    }
}

// Exported method: Getter (อ่านค่าฟิลด์ balance)
// เนื่องจาก balance เป็น unexported จึงต้องมีเมธอดนี้เพื่อให้ภายนอกอ่านค่าได้
func (a *Account) GetBalance() float64 {
    return a.balance
}

// Exported method: Setter พร้อม validation
// ควบคุมการเปลี่ยนแปลงค่า balance โดยเพิ่มกฎทางธุรกิจ (ห้ามติดลบ)
func (a *Account) SetBalance(amount float64) error {
    if amount < 0 {
        return errors.New("ยอดเงินต้องไม่ติดลบ")
    }
    a.balance = amount
    return nil
}

// เมธอดอื่นๆ: ฝากเงิน (Deposit)
// มีการตรวจสอบจำนวนเงินก่อนดำเนินการ
func (a *Account) Deposit(amount float64) error {
    if amount <= 0 {
        return errors.New("จำนวนเงินฝากต้องมากกว่า 0")
    }
}
```

```

    a.balance += amount
    return nil
}

// เมธอดอื่นๆ: ถอนเงิน (Withdraw)
// ตรวจสอบทั้งจำนวนเงินที่ถอนและยอดคงเหลือเพียงพอหรือไม่
func (a *Account) Withdraw(amount float64) error {
    if amount <= 0 || amount > a.balance {
        return errors.New("จำนวนเงินไม่ถูกต้องหรือยอดไม่พอ")
    }
    a.balance -= amount
    return nil
}

```

```

// main.go (ต่างแพ็คเกจ - ไม่ใช่แพ็คเกจ bank)
package main

import (
    "fmt"
    "yourmodule/bank" // import แพ็คเกจ bank ที่เราสร้าง
)

func main() {
    // สร้าง instance ของ Account โดยใช้ Constructor (NewAccount)
    // ไม่สามารถสร้างโดยใช้ Account{} โดยตรงได้เพราะฟิลด์เป็น unexported
    acc := bank.NewAccount("123-456", 1000)

    // อ่านยอดเงินผ่าน Getter (เมธอด exported)
    fmt.Println("ยอดคงเหลือ:", acc.GetBalance()) // 1000

    // ฝากเงิน 500 บาท (ผ่านเมธอด Deposit ที่มี validation)
    acc.Deposit(500)
    fmt.Println("หลังฝาก:", acc.GetBalance()) // 1500

    // พยายามตั้งยอดเป็น -100 (ติดลบ) ผ่าน Setter
    // Setter จะตรวจสอบและปฏิเสธ พร้อมคืน error
    err := acc.SetBalance(-100)
    if err != nil {
        fmt.Println("Error:", err) // ผลลัพธ์: ยอดเงินต้องไม่ติดลบ
    }

    // บรรทัดต่อไปนี้จะคอมไพล์ไม่ผ่าน เพราะ balance เป็น unexported
    // ไม่สามารถเข้าถึงโดยตรงจากแพ็คเกจ main ได้
    // acc.balance = 2000 // ✗ compile error: balance unexported
}

```

🔒 อธิบายหลักการ Encapsulation ใน Go (จากตัวอย่าง)

1. การควบคุมการมองเห็น (Visibility) ด้วยตัวพิมพ์ใหญ่/เล็ก

- **Unexported (private)** : ฟิลด์ `accountNumber`, `balance` ขึ้นต้นด้วยตัวพิมพ์เล็ก → มองเห็นได้เฉพาะภายในแพ็คเกจ `bank` เท่านั้น
- **Exported (public)** : เมธอด `GetBalance`, `SetBalance`, `Deposit`, `Withdraw` ขึ้นต้นด้วยตัวพิมพ์ใหญ่ → มองเห็นได้จากทุกแพ็คเกจ (รวมถึง `main`)

2. การเข้าถึงข้อมูลทางอ้อมผ่านเมธอด (Getter/Setter)

- ภายนอกไม่สามารถแก้ไข `balance` ได้โดยตรง (`acc.balance = 2000` ✗)
- ต้องใช้เมธอด `SetBalance(amount)` ซึ่งมีการตรวจสอบเงื่อนไข (`amount >= 0`) ก่อนเปลี่ยนค่า
- การอ่านค่าก็ต้องใช้ `GetBalance()` แทนการเข้าถึงโดยตรง

3. Validation (การตรวจสอบความถูกต้อง) ใน Setter และเมธอดอื่นๆ

- `SetBalance` : ห้ามตั้งยอดติดลบ
- `Deposit` : จำนวนเงินฝากต้องมากกว่า 0
- `Withdraw` : จำนวนเงินถอนต้องมากกว่า 0 และไม่เกินยอดคงเหลือ

4. Constructor (ฟังก์ชันสำหรับสร้าง object)

- Go ไม่มี constructor ในตัว แต่ใช้ฟังก์ชัน `NewAccount` ซึ่งเป็น exported (พิมพ์ใหญ่) เพื่อสร้าง instance
- ภายใน constructor สามารถตั้งค่าเริ่มต้นและตรวจสอบความถูกต้องได้ (เช่น ถ้า `initialBalance` ติดลบ ให้ตั้งเป็น 0)

5. ประโยชน์ของ Encapsulation ในตัวอย่างนี้

- **ป้องกันสถานะไม่ถูกต้อง** : ไม่มียอดคงเหลือติดลบเกิดขึ้นได้
- **ซ่อนรายละเอียดภายใน** : การเปลี่ยนแปลงฟیلด์ `balance` ต้องผ่านกฎทางธุรกิจเท่านั้น
- **ปรับเปลี่ยนง่าย** : ถ้าอยากเพิ่ม validation อื่น (เช่น ห้ามยอดเกิน 1,000,000) ก็แค่แก้ไขในเมธอด Setter โดยไม่ต้องแก้ไขโค้ดภายนอก

6. ความแตกต่างจากภาษา OOP แบบคลาสสิก (Java/C++)

- Go ใช้ **package** เป็นหน่วยของการ encapsulate (ไม่ใช่ class)
- ไม่มีคีย์เวิร์ด `private/public` แต่ใช้ **การตั้งชื่อ** แทน
- ไม่มี `this` โดยปริยาย แต่ใช้ **receiver** (`a *Account`) ซึ่งชัดเจนกว่า

ตัวอย่างการใช้งานจริงเพิ่มเติม

```
// สมมติว่ามีการเรียกใช้ในระบบธนาคาร
func Transfer(from, to *bank.Account, amount float64) error {
    if err := from.Withdraw(amount); err != nil {
        return err
    }
    if err := to.Deposit(amount); err != nil {
        // ถ้าโอนเข้า failed ให้คืนเงิน (Rollback)
        from.Deposit(amount)
        return err
    }
}
```

```
    return nil
}
```

จะเห็นว่า **Transfer** ไม่สามารถเข้าถึง **balance** โดยตรงได้ ต้องใช้เมธอดที่ปลอดภัยเท่านั้น → ช่วยป้องกันความผิดพลาดและรักษาความถูกต้องของข้อมูล

สรุป

Encapsulation ใน Go ทำได้โดย:

- **Unexported fields** (พิมพ์เล็ก) → ห่อหุ้มข้อมูลไว้ภายในแพ็คเกจ
- **Exported methods** (พิมพ์ใหญ่) → เปิดช่องทางเข้าถึงที่ปลอดภัย พร้อม validation
- ทำให้โค้ด ปลอดภัย, บำรุงรักษาง่าย และ ลดการพึ่งพากัน (low coupling)

2. Inheritance (การสืบทอด) ใน Go

 Inheritance คืออะไรใน Go?

Go ไม่มีการสืบทอดแบบคลาส (class-based inheritance) แต่ใช้ **Composition** และ **Embedding** (การฝัง struct) เพื่อให้ได้ความสามารถคล้ายกัน โดย struct ลูกสามารถเข้าถึงฟิลด์และเมธอดของ struct แม่ได้เสมือน "is-a" บางส่วน

 Inheritance มีกี่แบบใน Go?

Go มีรูปแบบการ "สืบทอด" ผ่าน Composition/Embedding ได้ 3 แบบ:

แบบ	คำอธิบาย	ตัวอย่าง
1. Struct Embedding	ฝัง struct แม่ลงใน struct ลูก (คล้าย single inheritance)	<code>type Dog struct { Animal }</code>
2. Multiple Embedding	ฝังหลาย struct (คล้าย multiple inheritance แต่ไม่มีความขัดแย้ง)	<code>type Bird struct { Animal, Flyer }</code>
3. Interface Embedding	ฝัง interface เพื่อสร้าง interface ใหม่	<code>type ReaderWriter interface { Reader; Writer }</code>

ข้อควรรู้: Go ไม่มี method overriding แบบอัตโนมัติ แต่เราสามารถ "override" ได้โดยการนิยามเมธอดชื่อเดียวกันใน struct ลูก (shadowing) และสามารถเรียกเมธอดของ struct แม่ผ่าน `Animal.Eat()` ได้

 ใช้อย่างไร? ในกรณีไหน?

- **ใช้เมื่อ** ต้องการนำโค้ดมาใช้ซ้ำ (reuse) โดยไม่ต้องเขียนซ้ำ
- **กรณีใช้งานจริง:**
 - ระบบยานพาหนะ: `Vehicle` มี `Start()`, `Car` ฝัง `Vehicle` และเพิ่ม `OpenSunroof()`
 - ระบบ HTTP handler: ฝัง `http.ServeMux`
 - ระบบ Logger: ฝัง `log.Logger`

แผนภาพ Dataflow (Flowchart TB) – Struct Embedding ใน Go

```
flowchart TB
    subgraph Base["struct แม่ (Animal)"]
        A["ฟิลด์: Name string<br>เมธอด: Eat(), Sleep()"]
    end

    subgraph Derived["struct ลูก (Dog)"]
        B["ฝัง Animal<br>type Dog struct { Animal }"]
        C["เพิ่มฟิลด์ Breed string<br>เพิ่มเมธอด Bark()"]
        D["ต้องการเปลี่ยนเมธอด Eat?"]
        E["กำหนด func (d Dog) Eat() {<br>    fmt.Println(d.Name, \"กินกระดูก\")<br>}"]
    end

    F["d := Dog{Animal{Name: \"ทองคำ\"}, \"โกลเด้น\"}<br>d.Eat()"]
    G["เรียกเมธอดของ Animal ที่ถูกฝัง<br>d.Animal.Sleep()"]

    A -- ฝังเข้า --> B
    B --> C
    C --> D
    D -- ใช่ --> E
    D -- ไม่ใช่ --> C
    E --> F
    F --> G
```

คำอธิบายแผนภาพ:

- Dog ฝัง Animal → สามารถเรียก `d.Name`, `d.Eat()` ได้โดยตรง
- ถ้า Dog ไม่มีเมธอด `Eat()` จะใช้ของ Animal แทน
- ถ้าต้องการ "override" ก็แค่เขียนเมธอด `Eat()` ใน Dog
- การเข้าถึงเมธอดของแม่ทำได้โดย `d.Animal.Eat()`

ตัวอย่างโค้ดจริง (Go)

```
package main

import "fmt"

// ===== struct แม่ (Base) =====
type Animal struct {
    Name string
}

func (a Animal) Eat() {
    fmt.Printf("%s กำลังกินอาหาร\n", a.Name)
}
```

```

func (a Animal) Sleep() {
    fmt.Printf("%s กำลังนอน\n", a.Name)
}

// ===== struct ลูก ใช้ Embedding =====
type Dog struct {
    Animal    // ผีง struct Animal (ไม่มีชื่อฟิลด์)
    Breed string
}

// "Override" เมธอด Eat (shadowing)
func (d Dog) Eat() {
    fmt.Printf("%s (พันธุ์%s) กำลังกินกระดูก\n", d.Name, d.Breed)
}

// เมธอดเพิ่มเติมเฉพาะ Dog
func (d Dog) Bark() {
    fmt.Printf("%s เห่า โห้งๆ\n", d.Name)
}

// ===== Multiple Embedding =====
type Flyer struct {
    MaxSpeed int
}

func (f Flyer) Fly() {
    fmt.Printf("บินได้ด้วยความเร็วสูงสุด %d km/h\n", f.MaxSpeed)
}

// Bird ผีงทั้ง Animal และ Flyer
type Bird struct {
    Animal
    Flyer
    WingSpan float64
}

// ===== การใช้งานจริง =====
func main() {
    // Single embedding
    dog := Dog{
        Animal: Animal{Name: "ทองคำ"},
        Breed:  "โกลเด้น",
    }

    dog.Eat()    // ทองคำ (พันธุ์โกลเด้น) กำลังกินกระดูก (ใช้ของ Dog)
    dog.Sleep()  // ทองคำ กำลังนอน (ใช้ของ Animal เพราะ Dog ไม่มี Sleep)
    dog.Bark()   // ทองคำ เห่า โห้งๆ

    // เรียกเมธอดของ Animal โดยตรง (ถ้าถูก shadow)
    dog.Animal.Eat() // ทองคำ กำลังกินอาหาร

    // Multiple embedding

```

```

bird := Bird{
    Animal: Animal{Name: "นกอินทรี"},
    Flyer: Flyer{MaxSpeed: 160},
    WingSpan: 2.5,
}
bird.Eat() // นกอินทรี กำลังกินอาหาร
bird.Fly() // บินได้ด้วยความเร็วสูงสุด 160 km/h
}

```

🐦 3. Polymorphism (พหุสัณฐาน) ใน Go

🔍 Polymorphism คืออะไรใน Go?

Polymorphism ใน Go ทำผ่าน **Interface** เป็นหลัก โดย interface กำหนดชุดของเมธอดที่ประเภทต่างๆ (struct) ต้อง implement จากนั้นตัวแปรชนิด interface สามารถเก็บค่าได้หลายประเภท และเรียกเมธอดเดียวกันโดยได้พฤติกรรมต่างกันตามประเภทจริง (runtime polymorphism)

นอกจากนี้ยังมี **compile-time polymorphism** ผ่าน **Generic functions** (ตั้งแต่ Go 1.18)

📊 Polymorphism มีกี่แบบใน Go?

แบ่งเป็น 2 แบบ:

แบบ	เวลาทำงาน	กลไก	ตัวอย่าง
1. Runtime Polymorphism	ตอน run	Interface + Dynamic dispatch	<code>var a Animal; a = Dog{}; a.Speak()</code>
2. Compile-time Polymorphism	ตอน compile	Generics (type parameters)	<code>func Print[T any](val T)</code>

หมายเหตุ: Method Overloading (ชื่อเมธอดเดียวกันแต่พารามิเตอร์ต่างกัน) **ไม่มีใน Go** – ใช้วิธีตั้งชื่อต่างกันหรือใช้ variadic parameters แทน

🔗 ใช้อย่างไร? ในกรณีไหน?

- **ใช้เมื่อ** ต้องการเขียนโค้ดที่ยืดหยุ่น รับค่าได้หลายประเภทผ่าน interface เดียว
- **กรณีใช้งานจริง:**
 - ระบบการจ่ายเงิน: `PaymentMethod` interface มี `Pay(amount)` – `CreditCard`, `PayPal`, `Cash`
 - ระบบ Logging: `Logger` interface – `FileLogger`, `ConsoleLogger`, `RemoteLogger`
 - HTTP handlers: `http.Handler` interface

📄 แผนภาพ Dataflow (Flowchart TB) – Polymorphism ใน Go

```

flowchart TB
    subgraph Interface["Interface (Shape)"]
        A["type Shape interface {<br>    Area() float64<br>}"]
    end

    subgraph Implement["Structs ที่ implement Shape"]
        B1["type Circle struct { Radius float64 }<br>func (c Circle) Area() float64 { return  $\pi * r^2$  }"]
        B2["type Rectangle struct { W, H float64 }<br>func (r Rectangle) Area() float64 { return w*h }"]
    end

    subgraph Runtime["ขณะรันโปรแกรม"]
        C["var s Shape"]
        D["s เก็บ object ประเภทใด?"]
        E1["s = Circle{5} → s.Area() =  $\pi * 25$ "]
        E2["s = Rectangle{3,4} → s.Area() = 12"]
    end

    A -- implement โดย --> B1
    A -- implement โดย --> B2
    C --> D
    D -- ถ้าเป็น Circle --> E1
    D -- ถ้าเป็น Rectangle --> E2

```

คำอธิบายแผนภาพ:

- **Shape** interface กำหนดเมธอด **Area()**
- **Circle** และ **Rectangle** implement **Area()** ตามรูปแบบของตน
- ตัวแปร **s** ชนิด **Shape** สามารถชี้ไปที่ **Circle** หรือ **Rectangle** ก็ได้
- เมื่อเรียก **s.Area()** Go จะเรียกเมธอดของประเภทจริงโดยอัตโนมัติ

ตัวอย่างโค้ดจริง (Go)

```

package main

import (
    "fmt"
    "math"
)

// ===== Runtime Polymorphism ผ่าน Interface =====
// 1. ประกาศ interface
type Shape interface {
    Area() float64
    Perimeter() float64
}

```

```
// 2. Struct ที่ implement Shape
type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func (c Circle) Perimeter() float64 {
    return 2 * math.Pi * c.Radius
}

type Rectangle struct {
    Width, Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

func (r Rectangle) Perimeter() float64 {
    return 2 * (r.Width + r.Height)
}

// ฟังก์ชันที่ใช้ Polymorphism (รับ interface)
func PrintShapeInfo(s Shape) {
    fmt.Printf("พื้นที่: %.2f, เส้นรอบรูป: %.2f\n", s.Area(), s.Perimeter())
}

// ===== Compile-time Polymorphism ผ่าน Generics (Go 1.18+) =====
func Sum[T int | float64](a, b T) T {
    return a + b
}

// Generic function กับ slice
func PrintSlice[T any](s []T) {
    for _, v := range s {
        fmt.Print(v, " ")
    }
    fmt.Println()
}

// ===== การใช้งานจริง =====
func main() {
    // Runtime Polymorphism
    var s Shape

    s = Circle{Radius: 5}
    fmt.Println("Circle:")
    fmt.Printf("Area: %.2f\n", s.Area())           // 78.54
    fmt.Printf("Perimeter: %.2f\n", s.Perimeter()) // 31.42
    PrintShapeInfo(s) // พื้นที่: 78.54, เส้นรอบรูป: 31.42
}
```

```

s = Rectangle{Width: 4, Height: 6}
fmt.Println("\nRectangle:")
fmt.Printf("Area: %.2f\n", s.Area()) // 24.00
PrintShapeInfo(s) // พื้นที่: 24.00, เส้นรอบรูป: 20.00

// ใช้ slice ของ interface
shapes := []Shape{
    Circle{Radius: 2},
    Rectangle{Width: 3, Height: 4},
    Circle{Radius: 7},
}

fmt.Println("\nAll shapes:")
for _, sh := range shapes {
    fmt.Printf("- Area = %.2f\n", sh.Area())
}

// Compile-time Polymorphism (Generics)
fmt.Println("\nGenerics:")
fmt.Println(Sum(10, 20)) // 30 (int)
fmt.Println(Sum(3.5, 2.7)) // 6.2 (float64)

PrintSlice([]int{1, 2, 3})
PrintSlice([]string{"a", "b", "c"})
}

```

สรุปเปรียบเทียบหลักการ OOP ใน Go เทียบกับภาษา OOP ดั้งเดิม

หลักการ	ใน Java/C++	ใน Go
Encapsulation	<code>private</code> , <code>protected</code> , <code>public</code>	ตัวพิมพ์เล็ก (unexported) vs ตัวพิมพ์ใหญ่ (exported)
Inheritance	<code>extends</code> (class-based)	Struct embedding (composition แทน)
Polymorphism	Method overloading + overriding + interface	Interface (runtime) + Generics (compile-time)
คอนสตรักเตอร์	Constructor ชื่อเดียวกับคลาส	ฟังก์ชัน <code>NewXxx()</code> (convention)
this/super	<code>this</code> , <code>super</code>	รับเป็น receiver (<code>func (t T) method()</code>)

ข้อควรจำสำหรับ Go

1. ไม่มี `class` → ใช้ `struct` และ `interface`
2. ไม่มี `inheritance` แบบ `is-a` → ใช้ `embedding` เพื่อ reuse โค้ด (ควรใช้ `composition` มากกว่า)
3. การควบคุมการเข้าถึง → ดูที่ ตัวอักษรตัวแรก (พิมพ์ใหญ่/เล็ก)
4. **Polymorphism** → **interface** เป็นหัวใจสำคัญ (implement โดยนัย ไม่ต้องประกาศ `implements`)

5. Method overloading ไม่มี → ใช้ชื่อเมธอดต่างกัน หรือใช้ variadic/empty interface ร่วมกับ type switch

```
// ตัวอย่าง: Polymorphism ที่เป็นเอกลักษณ์ของ Go (empty interface + type switch)
func Describe(v interface{}) {
    switch val := v.(type) {
    case int:
        fmt.Printf("int: %d\n", val)
    case string:
        fmt.Printf("string: %s\n", val)
    default:
        fmt.Printf("unknown type: %T\n", val)
    }
}
```

ตัวอย่างเพิ่มเติมในบริบทเฉพาะ (Web API, การทดสอบ unit test ด้วย interface)

ตัวอย่างเพิ่มเติม: Web API + Unit Testing ด้วย Interface ใน Go

ต่อไปนี้เป็นตัวอย่างการนำหลักการทั้งสาม (Encapsulation, Inheritance/Embedding, Polymorphism) มาใช้ใน **Web API (REST API)** และ **Unit Testing** โดยใช้ **Interface** เพื่อให้ทดสอบได้ง่าย (Dependency Injection + Mocking)

โครงสร้างโปรเจกต์ตัวอย่าง

```
userapi/
├── go.mod
├── main.go
├── internal/
│   ├── domain/
│   │   ├── user.go          # struct และ interface
│   │   └── repository/
│   │       ├── user_repo.go  # interface + real implementation
│   │       └── mock_user_repo.go # mock สำหรับ test
│   ├── service/
│   │   └── user_service.go    # service layer
│   └── handler/
│       └── user_handler.go    # HTTP handler
└── service_test.go           # unit test
```

1 Encapsulation ใน Web API (domain/user.go)

```

// internal/domain/user.go
package domain

import "errors"

// User struct: ฟิลด์เป็น unexported (encapsulation)
type User struct {
    id      int
    username string
    email   string
    isActive bool
}

// Constructor (exported)
func NewUser(id int, username, email string) (*User, error) {
    if username == "" || email == "" {
        return nil, errors.New("username และ email ห้ามว่าง")
    }
    return &User{
        id:      id,
        username: username,
        email:   email,
        isActive: true,
    }, nil
}

// Getters (อ่านค่าได้อย่างเดียว)
func (u *User) ID() int      { return u.id }
func (u *User) Username() string { return u.username }
func (u *User) Email() string { return u.email }
func (u *User) IsActive() bool { return u.isActive }

// Setter พร้อม validation (encapsulation ควบคุมการแก้ไข)
func (u *User) Deactivate() {
    u.isActive = false
}

func (u *User) UpdateEmail(newEmail string) error {
    if newEmail == "" {
        return errors.New("email ห้ามว่าง")
    }
    u.email = newEmail
    return nil
}

```

☑ Encapsulation ใน API:

- ฟิลด์ `id`, `username` ไม่สามารถแก้ไขได้โดยตรงจาก handler
- การแก้ไขต้องผ่านเมธอดที่มี validation (`UpdateEmail`, `Deactivate`)

2 Inheritance (Struct Embedding) – reusable base model

```
// internal/domain/base.go (ฝังใน repository หรือ service)
package domain

import "time"

// BaseModel ฝังใน struct อื่นเพื่อ reuse ฟังก์ชันทั่วไป
type BaseModel struct {
    CreatedAt time.Time
    UpdatedAt time.Time
}

func (b *BaseModel) SetTimestamps() {
    now := time.Now()
    if b.CreatedAt.IsZero() {
        b.CreatedAt = now
    }
    b.UpdatedAt = now
}

// ตัวอย่างฝังใน UserEntity (สำหรับ database layer)
type UserEntity struct {
    BaseModel // embedding
    ID         int
    Username   string
    Email      string
    IsActive   bool
}
```

☑ Inheritance ใน API:

- `UserEntity` มีฟิลด์ `CreatedAt`, `UpdatedAt` และเมธอด `SetTimestamps()` โดยอัตโนมัติ
- ช่วยลดโค้ดซ้ำในหลาย struct (เช่น `Product`, `Order`)

3 Polymorphism ผ่าน Interface – เพื่อการทดสอบ (Testable API)

◇ ขั้นที่ 1: กำหนด interface ของ Repository

```
// internal/repository/user_repo.go
package repository

import (
    "context"
    "yourmodule/internal/domain"
)

// UserRepository interface (polymorphism)
```

```

type UserRepository interface {
    GetByID(ctx context.Context, id int) (*domain.User, error)
    Save(ctx context.Context, user *domain.User) error
}

```

◊ ขั้นที่ 2: Implement จริง (ใช้ database)

```

// internal/repository/user_repo.go (ต่อ)
type userRepositoryImpl struct {
    // db *sql.DB หรือ gorm.DB
}

func NewUserRepository() UserRepository {
    return &userRepositoryImpl{}
}

func (r *userRepositoryImpl) GetByID(ctx context.Context, id int)
(*domain.User, error) {
    // จำลองการดึงจาก DB
    // ในของจริง: row := db.QueryRow("SELECT ...")
    return domain.NewUser(id, "john_doe", "john@example.com")
}

func (r *userRepositoryImpl) Save(ctx context.Context, user *domain.User) error
{
    // INSERT หรือ UPDATE
    return nil
}

```

◊ ขั้นที่ 3: Service layer ที่ใช้ interface (Polymorphism)

```

// internal/service/user_service.go
package service

import (
    "context"
    "errors"
    "yourmodule/internal/domain"
    "yourmodule/internal/repository"
)

type UserService struct {
    repo repository.UserRepository // รับ interface ไม่ใช่ concrete type
}

// Constructor รับ interface (Dependency Injection)
func NewUserService(repo repository.UserRepository) *UserService {
    return &UserService{repo: repo}
}

```

```

}

func (s *UserService) GetUser(ctx context.Context, id int) (*domain.User,
error) {
    if id <= 0 {
        return nil, errors.New("invalid user id")
    }
    return s.repo.GetByID(ctx, id)
}

func (s *UserService) DeactivateUser(ctx context.Context, id int) error {
    user, err := s.repo.GetByID(ctx, id)
    if err != nil {
        return err
    }
    user.Deactivate() // เรียกเมธอด encapsulated
    return s.repo.Save(ctx, user)
}

```

◇ ขั้นที่ 4: HTTP Handler (ใช้ Service)

```

// internal/handler/user_handler.go
package handler

import (
    "encoding/json"
    "net/http"
    "strconv"
    "yourmodule/internal/service"
)

type UserHandler struct {
    userService *service.UserService
}

func NewUserHandler(svc *service.UserService) *UserHandler {
    return &UserHandler{userService: svc}
}

func (h *UserHandler) GetUser(w http.ResponseWriter, r *http.Request) {
    idStr := r.URL.Query().Get("id")
    id, _ := strconv.Atoi(idStr)

    user, err := h.userService.GetUser(r.Context(), id)
    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }

    // ใช้ getter (encapsulation) สร้าง response

```

```

    resp := map[string]interface{}{
        "id":      user.ID(),
        "username": user.Username(),
        "email":   user.Email(),
        "active":  user.IsActive(),
    }
    json.NewEncoder(w).Encode(resp)
}

```

◇ ชั้นที่ 5: main.go – ประกอบ Dependency

```

// main.go
package main

import (
    "log"
    "net/http"
    "yourmodule/internal/handler"
    "yourmodule/internal/repository"
    "yourmodule/internal/service"
)

func main() {
    // real repository
    userRepo := repository.NewUserRepository()
    userSvc := service.NewUserService(userRepo)
    userHandler := handler.NewUserHandler(userSvc)

    http.HandleFunc("/user", userHandler.GetUser)
    log.Fatal(http.ListenAndServe(":8080", nil))
}

```

Unit Testing ด้วย Mock Interface (Polymorphism ช่วยให้ทดสอบได้)

เราสร้าง **Mock** ที่ implement **UserRepository** เดียวกัน แล้ว inject เข้า service

สร้าง Mock (manual หรือใช้ testify/mock)

```

// internal/repository/mock_user_repo.go
package repository

import (
    "context"
    "yourmodule/internal/domain"
)

type MockUserRepository struct {

```

```

    GetByIDFunc func(ctx context.Context, id int) (*domain.User, error)
    SaveFunc     func(ctx context.Context, user *domain.User) error
}

func (m *MockUserRepository) GetByID(ctx context.Context, id int)
(*domain.User, error) {
    if m.GetByIDFunc != nil {
        return m.GetByIDFunc(ctx, id)
    }
    return nil, nil
}

func (m *MockUserRepository) Save(ctx context.Context, user *domain.User) error
{
    if m.SaveFunc != nil {
        return m.SaveFunc(ctx, user)
    }
    return nil
}

```

เขียน Unit Test (service_test.go)

```

// service_test.go
package service

import (
    "context"
    "errors"
    "testing"
    "yourmodule/internal/domain"
    "yourmodule/internal/repository"
)

func TestUserService_GetUser_Success(t *testing.T) {
    // 1. สร้าง mock
    mockRepo := &repository.MockUserRepository{
        GetByIDFunc: func(ctx context.Context, id int) (*domain.User, error) {
            // จำลองการคืน user
            return domain.NewUser(id, "testuser", "test@example.com")
        },
    }

    // 2. inject mock เข้า service
    svc := NewUserService(mockRepo)

    // 3. เรียกใช้
    user, err := svc.GetUser(context.Background(), 123)

    // 4. ตรวจสอบผลลัพธ์
    if err != nil {

```

```

        t.Fatalf("unexpected error: %v", err)
    }
    if user.ID() != 123 {
        t.Errorf("expected ID 123, got %d", user.ID())
    }
    if user.Username() != "testuser" {
        t.Errorf("expected username testuser, got %s", user.Username())
    }
}

func TestUserService_GetUser_InvalidID(t *testing.T) {
    mockRepo := &repository.MockUserRepository{}
    svc := NewUserService(mockRepo)

    _, err := svc.GetUser(context.Background(), -5)
    if err == nil {
        t.Error("expected error for invalid id, got nil")
    }
}

func TestUserService_DeactivateUser_CallsSave(t *testing.T) {
    saved := false
    mockRepo := &repository.MockUserRepository{
        GetByIDFunc: func(ctx context.Context, id int) (*domain.User, error) {
            return domain.NewUser(id, "active", "a@b.com")
        },
        SaveFunc: func(ctx context.Context, user *domain.User) error {
            saved = true
            if user.IsActive() {
                t.Error("user should be deactivated before save")
            }
            return nil
        },
    }

    svc := NewUserService(mockRepo)
    err := svc.DeactivateUser(context.Background(), 1)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if !saved {
        t.Error("expected Save to be called")
    }
}

```

ใช้ testify/mock (สะดวกขึ้น)

```

// go get github.com/stretchr/testify/mock

// internal/repository/mock_user_repo_testify.go

```

```

package repository

import (
    "context"
    "github.com/stretchr/testify/mock"
    "yourmodule/internal/domain"
)

type MockUserRepoTestify struct {
    mock.Mock
}

func (m *MockUserRepoTestify) GetByID(ctx context.Context, id int)
(*domain.User, error) {
    args := m.Called(ctx, id)
    if args.Get(0) == nil {
        return nil, args.Error(1)
    }
    return args.Get(0).(*domain.User), args.Error(1)
}

func (m *MockUserRepoTestify) Save(ctx context.Context, user *domain.User)
error {
    args := m.Called(ctx, user)
    return args.Error(0)
}

// ใช้ใน test
func TestWithTestify(t *testing.T) {
    mockRepo := new(repository.MockUserRepoTestify)
    mockRepo.On("GetByID", mock.Anything, 100).Return(domain.NewUser(100,
"john", "j@k.com"))
    mockRepo.On("Save", mock.Anything, mock.Anything).Return(nil)

    svc := NewUserService(mockRepo)
    user, _ := svc.GetUser(context.Background(), 100)
    if user.Username() != "john" {
        t.Error("fail")
    }
    mockRepo.AssertExpectations(t)
}

```

แผนภาพ Dataflow ของ Web API + Unit Test (Flowchart TB)

```

flowchart TB
    subgraph Production["โหม่งจริง (Production)"]
        Client[HTTP Client] --> Handler[Handler]
        Handler --> Service[Service]
        Service --> RealRepo["RealRepository<br>(implement interface)"]
    end

```

```

    RealRepo --> DB[(Database)]
end

subgraph Test["โหมดทดสอบ (Unit Test)"]
    TestFunc[Test Function] --> Service2[UserService เดียวกัน]
    Service2 --> MockRepo["MockUserRepository<br>(implement interface เดียว)"]
    MockRepo --> Assertion[ตรวจสอบผลลัพธ์<br>+ ตรวจสอบว่า mock ถูกเรียก]
end

subgraph InterfaceDef["Interface ที่เป็นหัวใจ"]
    Interface["UserRepository interface<br>{ GetByID, Save }"]
end

RealRepo -. implement .-> Interface
MockRepo -. implement .-> Interface
Service --> Interface
Service2 --> Interface

```

อธิบาย:

- **UserService** ทำงานกับ **UserRepository interface** ไม่รู้ว่าข้างหลังเป็นของจริงหรือ mock
- ใน production ส่ง **RealRepository**
- ใน test ส่ง **MockRepository** ทำให้ test เร็วและไม่ต้องพึ่ง database จริง
- นี่คือพลังของ **Polymorphism ผ่าน Interface** ช่วยให้โค้ด **testable**

☑ สรุปประโยชน์ของ Interface ในการทดสอบ Web API

สิ่งที่ได้	คำอธิบาย
Decoupling	Handler/Service ไม่ผูกติดกับ DB เฉพาะ สามารถเปลี่ยน implementation ได้
Mocking	สร้าง Mock ที่ implement interface เดิม แล้ว inject ผ่าน constructor
Test Coverage	ทดสอบ business logic ได้โดยไม่ต้องรัน database
Parallel Test	mock ไม่มี shared state ทำให้ test รันขนานกันได้
Fast	ไม่มี I/O จริง (network, disk)

📦 ตัวอย่างคำสั่งรัน

```

# รัน web server
go run main.go

# ทดสอบ unit test
go test -v ./...

```



```
# ทดสอบ API (อีก terminal)
curl "http://localhost:8080/user?id=1"
```

☞ ข้อคิดสำหรับ Go Web API

1. **Always accept interfaces, return structs** – รับ interface ในฟังก์ชัน/constructor
2. **Keep interfaces small** – 1-2 เมธอดต่อ interface (เช่น `UserRepository`)
3. **Use embedding** สำหรับ middleware หรือ base handler
4. **Encapsulation** ใช้ unexported fields + exported methods เพื่อควบคุม state
5. **Test with interfaces** – สร้าง mock ได้ง่ายมาก

ตัวอย่างเพิ่มเติม การ test HTTP handler โดยใช้ `httptest` หรือการ test middleware

🔧 การทดสอบ HTTP Handler และ Middleware ใน Go ด้วย `httptest`

ต่อไปนี้เป็นตัวอย่างการทดสอบ HTTP handler และ middleware โดยใช้แพ็คเกจ `net/http/httptest` ควบคู่กับหลักการ **Dependency Injection** ผ่าน **interface** เพื่อให้ทดสอบได้ง่าย (Unit Test + Integration Test)

📦 1. การทดสอบ HTTP Handler แบบหน่วย (Unit Test)

- ◊ โครงสร้าง Handler ที่รองรับการทดสอบ (ใช้ Interface)

```
// internal/service/user_service.go (สมมติมี interface)
package service

import "context"

type UserService interface {
    GetUser(ctx context.Context, id int) (*User, error)
}

type User struct {
    ID    int
    Name string
}

// internal/handler/user_handler.go
package handler

import (
    "encoding/json"
    "net/http"
    "strconv"
    "yourmodule/internal/service"
)
```

```

)

type UserHandler struct {
    userService service.UserService // รับ interface
}

func NewUserHandler(svc service.UserService) *UserHandler {
    return &UserHandler{userService: svc}
}

func (h *UserHandler) GetUser(w http.ResponseWriter, r *http.Request) {
    idStr := r.URL.Query().Get("id")
    id, err := strconv.Atoi(idStr)
    if err != nil || id <= 0 {
        http.Error(w, "invalid id", http.StatusBadRequest)
        return
    }

    user, err := h.userService.GetUser(r.Context(), id)
    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(user)
}

```

◇ สร้าง Mock Service (implement interface)

```

// internal/service/mock_user_service.go
package service

import "context"

type MockUserService struct {
    GetUserFunc func(ctx context.Context, id int) (*User, error)
}

func (m *MockUserService) GetUser(ctx context.Context, id int) (*User, error) {
    if m.GetUserFunc != nil {
        return m.GetUserFunc(ctx, id)
    }
    return nil, nil
}

```

◇ เขียน Unit Test โดยใช้ httptest

```
// handler/user_handler_test.go
package handler

import (
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "testing"
    "yourmodule/internal/service"
)

func TestUserHandler_GetUser_Success(t *testing.T) {
    // 1. สร้าง mock service
    mockSvc := &service.MockUserService{
        GetUserFunc: func(ctx context.Context, id int) (*service.User, error) {
            return &service.User{ID: id, Name: "John Doe"}, nil
        },
    }

    // 2. สร้าง handler พร้อม inject mock
    handler := NewUserHandler(mockSvc)

    // 3. สร้าง request และ response recorder
    req := httptest.NewRequest(http.MethodGet, "/user?id=123", nil)
    rr := httptest.NewRecorder()

    // 4. เรียก handler
    handler.GetUser(rr, req)

    // 5. ตรวจสอบผลลัพธ์
    if status := rr.Code; status != http.StatusOK {
        t.Errorf("expected status OK, got %v", status)
    }

    var user service.User
    if err := json.NewDecoder(rr.Body).Decode(&user); err != nil {
        t.Fatal("failed to decode response")
    }
    if user.ID != 123 || user.Name != "John Doe" {
        t.Errorf("unexpected user: %v", user)
    }
}

func TestUserHandler_GetUser_InvalidID(t *testing.T) {
    mockSvc := &service.MockUserService{}
    handler := NewUserHandler(mockSvc)

    req := httptest.NewRequest(http.MethodGet, "/user?id=invalid", nil)
    rr := httptest.NewRecorder()

    handler.GetUser(rr, req)
}
```

```

    if rr.Code != http.StatusBadRequest {
        t.Errorf("expected BadRequest, got %v", rr.Code)
    }
}

func TestUserHandler_GetUser_NotFound(t *testing.T) {
    mockSvc := &service.MockUserService{
        GetUserFunc: func(ctx context.Context, id int) (*service.User, error) {
            return nil, errors.New("user not found")
        },
    }
    handler := NewUserHandler(mockSvc)

    req := httptest.NewRequest(http.MethodGet, "/user?id=999", nil)
    rr := httptest.NewRecorder()

    handler.GetUser(rr, req)

    if rr.Code != http.StatusNotFound {
        t.Errorf("expected NotFound, got %v", rr.Code)
    }
}

```

🔗 2. การทดสอบ Middleware

◇ ตัวอย่าง Middleware: Logging และ Recovery

```

// internal/middleware/logging.go
package middleware

import (
    "log"
    "net/http"
    "time"
)

func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        // ใช้ ResponseRecorder แบบกำหนดเอง เพื่อ capture status code
        rr := &responseRecorder{ResponseWriter: w, statusCode: http.StatusOK}
        next.ServeHTTP(rr, r)
        log.Printf("%s %s %d %v", r.Method, r.URL.Path, rr.statusCode,
            time.Since(start))
    })
}

type responseRecorder struct {
    http.ResponseWriter
}

```

```

    statusCode int
}

func (rr *responseRecorder) WriteHeader(code int) {
    rr.statusCode = code
    rr.ResponseWriter.WriteHeader(code)
}

```

```

// internal/middleware/recovery.go
package middleware

import (
    "log"
    "net/http"
)

func RecoveryMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                log.Printf("panic recovered: %v", err)
                http.Error(w, "Internal Server Error",
                    http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}

```

◊ ทดสอบ Middleware ด้วย `httptest`

```

// middleware/logging_test.go
package middleware

import (
    "net/http"
    "net/http/httptest"
    "testing"
)

func TestLoggingMiddleware(t *testing.T) {
    // สร้าง handler ปลายทาง (dummy)
    finalHandler := http.HandlerFunc(func(w http.ResponseWriter, r
    *http.Request) {
        w.WriteHeader(http.StatusOK)
        w.Write([]byte("ok"))
    })
}

```

```
// ห่อด้วย middleware
handler := LoggingMiddleware(finalHandler)

// สร้าง request
req := httptest.NewRequest(http.MethodGet, "/test", nil)
rr := httptest.NewRecorder()

// เรียก middleware
handler.ServeHTTP(rr, req)

// ตรวจสอบ response
if rr.Code != http.StatusOK {
    t.Errorf("expected status OK, got %d", rr.Code)
}
if rr.Body.String() != "ok" {
    t.Errorf("expected body 'ok', got %q", rr.Body.String())
}
}
```

◇ ทดสอบ Middleware ที่มีการ panic

```
func TestRecoveryMiddleware_HandlesPanic(t *testing.T) {
    // handler ที่ panic
    panicHandler := http.HandlerFunc(func(w http.ResponseWriter, r
    *http.Request) {
        panic("something went wrong")
    })

    handler := RecoveryMiddleware(panicHandler)

    req := httptest.NewRequest(http.MethodGet, "/panic", nil)
    rr := httptest.NewRecorder()

    // ควรไม่ทำให้ test panic
    handler.ServeHTTP(rr, req)

    if rr.Code != http.StatusInternalServerError {
        t.Errorf("expected 500, got %d", rr.Code)
    }
}
```

◇ ทดสอบ chain ของ middleware (หลายตัว)

```
func TestMiddlewareChain(t *testing.T) {
    // สร้าง handler ที่บันทึกการเรียก
    var called bool
    finalHandler := http.HandlerFunc(func(w http.ResponseWriter, r
    *http.Request) {
```

```

        called = true
        w.WriteHeader(http.StatusTeapot)
    })

    // ต่อ middleware: recovery -> logging -> final
    handler := RecoveryMiddleware(LoggingMiddleware(finalHandler))

    req := httptest.NewRequest(http.MethodGet, "/chain", nil)
    rr := httptest.NewRecorder()

    handler.ServeHTTP(rr, req)

    if !called {
        t.Error("final handler not called")
    }
    if rr.Code != http.StatusTeapot {
        t.Errorf("expected 418, got %d", rr.Code)
    }
}

```

แผนภาพ Flowchart การทดสอบ HTTP Handler และ Middleware

```

flowchart TB
    subgraph TestFunc["ฟังก์ชันทดสอบ (TestXXX)"]
        A1[สร้าง Mock Service / Mock Dependency]
        A2[สร้าง Handler พร้อม Inject Mock]
        A3[สร้าง httptest.NewRequest]
        A4[สร้าง httptest.NewRecorder]
        A5[เรียก handler.ServeHTTP หรือ handler โดยตรง]
    end

    subgraph HandlerExecution["การทำงานของ Handler (ใน test)"]
        B1[Handler รับ request]
        B2{เรียก service interface}
        B3[Mock Service คืนค่าตามที่กำหนด]
        B4[Handler สร้าง response]
    end

    subgraph Assertions["ตรวจสอบผลลัพธ์"]
        C1[ตรวจสอบ Status Code]
        C2[ตรวจสอบ Response Body]
        C3[ตรวจสอบ Header]
        C4[ตรวจสอบว่า Mock ถูกเรียกตามที่คาด]
    end

    A1 --> A2 --> A3 --> A4 --> A5
    A5 --> B1 --> B2 --> B3 --> B4 --> C1
    C1 --> C2 --> C3 --> C4

```

```
flowchart LR
    Request[HTTP Request] --> MW1[Middleware 1<br>Logging]
    MW1 --> MW2[Middleware 2<br>Recovery]
    MW2 --> Handler[Final Handler]
    Handler --> Response[Response]

    subgraph Test["การจำลองใน httptest"]
        TestReq[httptest.NewRequest] --> CallHandler[handler.ServeHTTP]
        CallHandler --> Capture[httptest.ResponseRecorder]
        Capture --> Assert[ตรวจสอบ status/body]
    end

    end

    MW1 -.-> Test
    MW2 -.-> Test
```

3. ตัวอย่างการทดสอบทั้งระบบ (Integration Test) ด้วย `httptest.Server`

บางครั้งเราต้องการทดสอบทั้ง routing และ middleware chain แบบ end-to-end โดยไม่ต้องเปิดพอร์ตจริง

```
// integration_test.go
package integration

import (
    "io"
    "net/http"
    "net/http/httptest"
    "testing"
    "yourmodule/internal/handler"
    "yourmodule/internal/middleware"
    "yourmodule/internal/service"
)

func TestIntegration_GetUserFlow(t *testing.T) {
    // 1. สร้าง service จริงหรือ mock ก็ได้ (ที่นี่ใช้ mock)
    mockSvc := &service.MockUserService{
        GetUserFunc: func(ctx context.Context, id int) (*service.User, error) {
            return &service.User{ID: id, Name: "Integration User"}, nil
        },
    }

    // 2. สร้าง handler
    userHandler := handler.NewUserHandler(mockSvc)

    // 3. สร้าง router พร้อม middleware chain
    mux := http.NewServeMux()
    mux.HandleFunc("/user", userHandler.GetUser)
```



```

// ใส่ middleware ทัวไป
var handler http.Handler = mux
handler = middleware.LoggingMiddleware(handler)
handler = middleware.RecoveryMiddleware(handler)

// 4. สร้าง test server
ts := httptest.NewServer(handler)
defer ts.Close()

// 5. ส่ง request จริงผ่าน HTTP client
resp, err := http.Get(ts.URL + "/user?id=999")
if err != nil {
    t.Fatal(err)
}
defer resp.Body.Close()

if resp.StatusCode != http.StatusOK {
    t.Errorf("expected 200, got %d", resp.StatusCode)
}

body, _ := io.ReadAll(resp.Body)
if !strings.Contains(string(body), "Integration User") {
    t.Errorf("unexpected body: %s", body)
}
}

```

☞ สรุปแนวทางปฏิบัติที่ดี

หัวข้อ	คำแนะนำ
Handler Testing	ใช้ <code>httptest.NewRecorder</code> + <code>httptest.NewRequest</code> เสมอ
Dependency Injection	ทำให้ handler รับ interface แทน concrete type เพื่อให้ mock ได้ง่าย
Middleware Testing	สร้าง dummy handler แล้วห่อด้วย middleware ที่ต้องการทดสอบ
Panic Recovery	ใช้ <code>defer</code> ใน middleware และทดสอบด้วย handler ที่ panic
Integration Test	ใช้ <code>httptest.NewServer</code> สร้าง server จริง แต่ไม่ต้อง bind port
Assertions	ตรวจสอบ status code, body, header, และ verify ว่า mock ถูกเรียก

📦 ตัวอย่างโค้ดเพิ่มเติม: การตรวจสอบ Header หรือ Cookie ใน middleware

```

// middleware/auth.go
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
        if token != "secret-token" {

```

```

        http.Error(w, "unauthorized", http.StatusUnauthorized)
        return
    }
    next.ServeHTTP(w, r)
})
}

// ทดสอบ AuthMiddleware
func TestAuthMiddleware_ValidToken(t *testing.T) {
    final := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    })
    handler := AuthMiddleware(final)

    req := httptest.NewRequest(http.MethodGet, "/", nil)
    req.Header.Set("Authorization", "secret-token")
    rr := httptest.NewRecorder()

    handler.ServeHTTP(rr, req)
    if rr.Code != http.StatusOK {
        t.Errorf("expected 200, got %d", rr.Code)
    }
}

func TestAuthMiddleware_InvalidToken(t *testing.T) {
    final := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        t.Error("handler should not be called")
    })
    handler := AuthMiddleware(final)

    req := httptest.NewRequest(http.MethodGet, "/", nil)
    req.Header.Set("Authorization", "wrong")
    rr := httptest.NewRecorder()

    handler.ServeHTTP(rr, req)
    if rr.Code != http.StatusUnauthorized {
        t.Errorf("expected 401, got %d", rr.Code)
    }
}

```

☒ สรุปภาพรวมการทดสอบใน Go Web API

flowchart TD

```

A[เขียน Handler / Middleware] --> B[กำหนด Interface สำหรับ Dependency]
B --> C[สร้าง Mock ของ Dependency]
C --> D[เขียน Unit Test ด้วย httptest]
D --> E[ทดสอบ Handler แต่ละกรณี]
D --> F[ทดสอบ Middleware แต่ละตัว]
D --> G[ทดสอบ Chain Middleware]

```

```
G --> H[ทดสอบ Integration ด้วย httptest.Server]
H --> I[รัน go test -cover]
```

ด้วยแนวทางนี้ คุณจะได้อะไรที่ **ทดสอบได้**, **ยืดหยุ่น** และ **บำรุงรักษาง่าย** ตามหลักการ OOP ที่ Go รองรับผ่าน interface และ composition

การทดสอบ WebSocket หรือการทดสอบ graceful shutdown

This chapter dives into two advanced but vital topics for building robust Go applications: testing WebSocket handlers and ensuring your servers shut down gracefully.

1. การทดสอบ WebSocket

การทดสอบ WebSocket นั้นซับซ้อนกว่า HTTP ทั่วไปเพราะต้องจัดการกับการเชื่อมต่อแบบสองทาง (full-duplex) แต่ใน Go เรามีเครื่องมือและเทคนิคที่ช่วยให้ทดสอบได้อย่างมีประสิทธิภาพ

วิธีการทดสอบ WebSocket

มีแนวทางหลักๆ 3 แบบที่ใช้กัน:

1. **Integration Test** ด้วย **httptest.Server** วิธีที่ตรงไปตรงมาและเหมาะกับกรณีส่วนใหญ่ โดยจะสร้าง WebSocket server จริงใน test และทำการเชื่อมต่อไปทดสอบ[reference:0]。
2. **Unit Test** ด้วย **Mock Connection** ออกแบบโค้ดให้พึ่งพา interface (**WSConn**) แทน concrete type (***websocket.Conn**) เพื่อให้สามารถใช้ mock connection ทดสอบ business logic แบบแยกส่วน[reference:1]。
3. **ใช้ Library** ช่วยในการทดสอบ มี library ที่ออกแบบมาสำหรับการทดสอบ WebSocket โดยเฉพาะ เช่น **wsmock** หรือ **wstest** ซึ่งช่วยลดความซับซ้อนได้มาก[reference:2]。

◇ วิธีที่ 1: Integration Test ด้วย **httptest.Server**

นี่เป็นวิธีพื้นฐานและตรงไปตรงมาที่สุด โดยใช้ **httptest.NewServer** สร้าง test server จากนั้น client ก็ dial ไปหา test server เหมือนเชื่อมต่อจริงๆ เลย[reference:3]。

```
// handler/websocket_handler.go
package handler

import (
    "net/http"
    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { return true },
}

func WebSocketHandler(w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
```

```

        http.Error(w, "Could not upgrade", http.StatusBadRequest)
        return
    }
    defer conn.Close()

    for {
        msgType, msg, err := conn.ReadMessage()
        if err != nil {
            break
        }
        // Echo ข้อความกลับไป
        conn.WriteMessage(msgType, msg)
    }
}

```

```

// handler/websocket_handler_test.go
package handler

import (
    "net/http/httptest"
    "testing"
    "github.com/gorilla/websocket"
)

func TestWebSocketHandler_Echo(t *testing.T) {
    // 1. สร้าง test server
    server := httptest.NewServer(http.HandlerFunc(WebSocketHandler))
    defer server.Close()

    // 2. แปลง http URL เป็น ws URL
    wsURL := "ws" + server.URL[4:] + "/ws"

    // 3. Dial ไปยัง test server
    conn, _, err := websocket.DefaultDialer.Dial(wsURL, nil)
    if err != nil {
        t.Fatalf("Dial error: %v", err)
    }
    defer conn.Close()

    // 4. ส่งข้อความ
    testMsg := "Hello, WebSocket!"
    err = conn.WriteMessage(websocket.TextMessage, []byte(testMsg))
    if err != nil {
        t.Fatalf("Write error: %v", err)
    }

    // 5. อ่าน response
    _, msg, err := conn.ReadMessage()
    if err != nil {
        t.Fatalf("Read error: %v", err)
    }
}

```

```

    }

    if string(msg) != testMsg {
        t.Errorf("Expected %q, got %q", testMsg, msg)
    }
}

```

◊ วิธีที่ 2: Unit Test แบบ Mock Connection (Best Practice)

นี่เป็นวิธีที่ถูกหลักการออกแบบซอฟต์แวร์มากที่สุด เพราะช่วยให้เราทดสอบ business logic ได้โดยไม่ต้องพึ่งพา network เลย ขั้นตอนคือกำหนด **interface** สำหรับ connection จากนั้นสร้าง **mock** ที่ implement interface เดียวกันเพื่อใช้ใน unit test[reference:4]。

1. กำหนด **WSConn** Interface เพื่อใช้ **Dependency Injection**

```

// websocket/wsconn.go
package websocket

import "time"

// WSConn interface (กำหนดเมธอดที่ business logic ต้องการ)
type WSConn interface {
    ReadMessage() (messageType int, data []byte, err error)
    WriteMessage(messageType int, data []byte) error
    Close() error
    SetReadDeadline(t time.Time) error
}

```

2. สร้าง **Wrapper** สำหรับ **Gorilla's websocket.Conn** (ใช้ใน production)

```

// websocket/gorilla_wrapper.go
package websocket

import (
    "github.com/gorilla/websocket"
)

type GorillaConn struct {
    conn *websocket.Conn
}

func NewGorillaConn(conn *websocket.Conn) *GorillaConn {
    return &GorillaConn{conn: conn}
}

func (g *GorillaConn) ReadMessage() (int, []byte, error) {
    return g.conn.ReadMessage()
}

```

```

}

func (g *GorillaConn) WriteMessage(messageType int, data []byte) error {
    return g.conn.WriteMessage(messageType, data)
}

func (g *GorillaConn) Close() error {
    return g.conn.Close()
}

func (g *GorillaConn) SetReadDeadline(t time.Time) error {
    return g.conn.SetReadDeadline(t)
}

```

3. สร้าง Mock Connection สำหรับใช้ใน Unit Test

```

// websocket/mock_conn.go
package websocket

import (
    "errors"
    "sync"
    "time"
)

type MockConn struct {
    mu          sync.Mutex
    sentMessages [][]byte
    messagesToRead [][]byte
    closed      bool
}

func NewMockConn() *MockConn {
    return &MockConn{
        messagesToRead: make([][]byte, 0),
        sentMessages:   make([][]byte, 0),
    }
}

// QueueMessage กำหนด message ที่จะถูกอ่านใน ReadMessage()
func (m *MockConn) QueueMessage(data []byte) {
    m.mu.Lock()
    defer m.mu.Unlock()
    m.messagesToRead = append(m.messagesToRead, data)
}

func (m *MockConn) ReadMessage() (int, []byte, error) {
    m.mu.Lock()
    defer m.mu.Unlock()

```

```

    if m.closed {
        return 0, nil, errors.New("connection closed")
    }

    if len(m.messagesToRead) == 0 {
        return 0, nil, errors.New("no messages queued")
    }

    msg := m.messagesToRead[0]
    m.messagesToRead = m.messagesToRead[1:]
    return 1, msg, nil
}

func (m *MockConn) WriteMessage(messageType int, data []byte) error {
    m.mu.Lock()
    defer m.mu.Unlock()

    if m.closed {
        return errors.New("connection closed")
    }

    m.sentMessages = append(m.sentMessages, data)
    return nil
}

func (m *MockConn) Close() error {
    m.mu.Lock()
    defer m.mu.Unlock()
    m.closed = true
    return nil
}

func (m *MockConn) SetReadDeadline(t time.Time) error {
    return nil // Mock ไม่มี deadline
}

func (m *MockConn) GetSentMessages() [][]byte {
    m.mu.Lock()
    defer m.mu.Unlock()
    return m.sentMessages
}

```

4. Business Logic หรือ WConn Interface

```

// service/chat_service.go
package service

import (
    "encoding/json"
    "log"

```

```

    "yourmodule/websocket"
)

type Message struct {
    Username string `json:"username"`
    Content  string `json:"content"`
}

type ChatService struct {
    // ไม่ต้องพึ่งพา concrete type!
}

func (s *ChatService) HandleConnection(conn websocket.WSConn) {
    defer conn.Close()

    for {
        _, data, err := conn.ReadMessage()
        if err != nil {
            log.Printf("Read error: %v", err)
            break
        }

        var msg Message
        if err := json.Unmarshal(data, &msg); err != nil {
            // ส่ง error กลับไปที่ client
            conn.WriteMessage(1, []byte("invalid message format"))
            continue
        }

        // Business logic: broadcast หรือ process ข้อความ
        response, _ := json.Marshal(map[string]string{
            "status": "received",
            "from":   msg.Username,
        })
        conn.WriteMessage(1, response)
    }
}

```

5. Unit Test โดยใช้ Mock Connection

```

// service/chat_service_test.go
package service

import (
    "encoding/json"
    "testing"
    "yourmodule/websocket"
)

func TestChatService_HandleValidMessage(t *testing.T) {

```



```
// 1. สร้าง mock connection
mockConn := websocket.NewMockConn()

// 2. จำลอง incoming message
msg := Message{Username: "Alice", Content: "Hello"}
data, _ := json.Marshal(msg)
mockConn.QueueMessage(data)

// 3. เรียกใช้ service (inject mock)
svc := &ChatService{}
svc.HandleConnection(mockConn)

// 4. ตรวจสอบว่า service ส่ง response อะไรกลับไป
sent := mockConn.GetSentMessages()
if len(sent) == 0 {
    t.Fatal("Expected response, got none")
}

var response map[string]string
json.Unmarshal(sent[0], &response)
if response["status"] != "received" {
    t.Errorf("Expected status 'received', got %v", response["status"])
}
}

func TestChatService_HandleInvalidMessage(t *testing.T) {
    mockConn := websocket.NewMockConn()
    mockConn.QueueMessage([]byte("invalid json"))

    svc := &ChatService{}
    svc.HandleConnection(mockConn)

    sent := mockConn.GetSentMessages()
    if len(sent) == 0 {
        t.Fatal("Expected error response, got none")
    }

    if string(sent[0]) != "invalid message format" {
        t.Errorf("Expected error message, got %s", sent[0])
    }
}
}
```

◊ วิธีที่ 3: ใช้ Library ช่วยในการทดสอบ

มี library ที่ออกแบบมาเพื่อการทดสอบ WebSocket โดยเฉพาะ ลองดู 2 ตัวนี้:

ตัวที่ 1: wstest - ช่วยให้ทดสอบ WebSocket handler ได้โดยไม่ต้องเปิดพอร์ตจริง[reference:5]。

```
import (
    "testing"
```

```

    "github.com/gorilla/websocket"
    "github.com/posener/wstest"
)

func TestWithWSTest(t *testing.T) {
    handler := http.HandlerFunc(WebSocketHandler)
    dialer := wstest.NewDialer(handler)

    conn, _, err := dialer.Dial("ws://whatever/ws", nil)
    if err != nil {
        t.Fatal(err)
    }
    defer conn.Close()

    conn.WriteMessage(websocket.TextMessage, []byte("ping"))
    _, msg, _ := conn.ReadMessage()
    // ตรวจสอบ response...
}

```

ตัวที่ 2: wsmock - ให้ Gorilla mocks สำหรับ unit testing[reference:6]。

```

import (
    "testing"
    "github.com/silently/wsmock"
)

func TestWithWSMock(t *testing.T) {
    conn, rec := wsmock.NewGorillaMockAndRecorder(t)

    // ส่งข้อความจาก client ไปยัง handler
    conn.Send("Hello")

    // ประกาศ assertion ว่าควรได้รับ response อะไร
    rec.AssertReceived("Hello, World!")

    wsmock.Run(t, 100*time.Millisecond)
}

```

แผนภาพการทดสอบ WebSocket

```

flowchart TB
    subgraph Production["โหมด Production"]
        Client[Client] --> Upgrader[HTTP Handler<br>Upgrade to WebSocket]
        Upgrader --> RealConn["*websocket.Conn"]
        RealConn --> BusinessLogic[Business Logic]
    end

    subgraph Test["Integration Test (httptest)"]

```

```

    TestClient[Test Client] --> TestServer["httpTest.Server"]
    TestServer --> RealConn2["*websocket.Conn"]
    RealConn2 --> BusinessLogic2[Business Logic]
end

subgraph UnitTest["Unit Test (Mock Connection)"]
    MockConn[MockConn<br>implement WSConn interface] -->
BusinessLogic3[Business Logic]
    Assertions[ตรวจสอบ sentMessages]
end

RealConn --> WSConnInterface["WSConn Interface"]
MockConn --> WSConnInterface
BusinessLogic2 --> WSConnInterface
BusinessLogic3 --> WSConnInterface

```

2. การทดสอบ Graceful Shutdown

Graceful shutdown คือการที่เซิร์ฟเวอร์หยุดรับ connection ใหม่ แต่ยังคงรอให้ requests ที่กำลังประมวลผลอยู่เสร็จก่อนที่จะปิดตัวเองลง[reference:7]。การทดสอบ graceful shutdown สามารถทำได้หลายระดับ

◇ วิธีที่ 1: Acceptance Test (Build + Run + Send Signal)

นี่เป็นวิธีที่ใช้ใน production จริง วิธีคือสร้าง binary ของเซิร์ฟเวอร์, รันมันขึ้นมา, ส่ง request ที่ใช้เวลานาน, จากนั้นส่งสัญญาณ **SIGINT** (Ctrl+C) เพื่อดูว่าเซิร์ฟเวอร์ shutdown แบบ graceful หรือไม่[reference:8]。

```

// graceful_test.go
package graceful_test

import (
    "bytes"
    "fmt"
    "net/http"
    "os/exec"
    "testing"
    "time"
)

func TestGracefulShutdown_WithSlowRequest(t *testing.T) {
    // 1. Build binary
    cmd := exec.Command("go", "build", "-o", "testserver", "./cmd/server")
    if err := cmd.Run(); err != nil {
        t.Fatalf("Build failed: %v", err)
    }
    defer exec.Command("rm", "-f", "testserver").Run()

    // 2. Run server in background
    serverCmd := exec.Command("./testserver")
    if err := serverCmd.Start(); err != nil {

```

```

        t.Fatalf("Start server failed: %v", err)
    }
    defer serverCmd.Process.Kill()

    // 3. Wait for server to be ready
    time.Sleep(500 * time.Millisecond)

    // 4. Send a slow request (ใน goroutine)
    respChan := make(chan *http.Response, 1)
    errChan := make(chan error, 1)

    go func() {
        // GET request ที่ใช้เวลานาน 10 วินาที
        resp, err := http.Get("http://localhost:8080/slow")
        if err != nil {
            errChan <- err
            return
        }
        respChan <- resp
    }()

    // 5. Give the request some time to start
    time.Sleep(500 * time.Millisecond)

    // 6. Send SIGINT to the server
    if err := serverCmd.Process.Signal(os.Interrupt); err != nil {
        t.Fatalf("Failed to send interrupt: %v", err)
    }

    // 7. Wait for result (should succeed if graceful shutdown works)
    select {
    case resp := <-respChan:
        defer resp.Body.Close()
        if resp.StatusCode != http.StatusOK {
            t.Errorf("Expected 200, got %d", resp.StatusCode)
        }
        buf := new(bytes.Buffer)
        buf.ReadFrom(resp.Body)
        if buf.String() != "slow request finished" {
            t.Errorf("Unexpected response: %s", buf.String())
        }
    case err := <-errChan:
        t.Errorf("Request failed: %v", err)
    case <-time.After(15 * time.Second):
        t.Error("Timeout waiting for response")
    }
}

```

◊ วิธีที่ 2: Unit Test การทำงานของ Shutdown Logic

เราสามารถทดสอบการทำงานของ `Shutdown()` method ได้โดยตรงด้วย `httptest.Server` ที่รันอยู่ใน `test process[reference:9]`。

```
// graceful_test.go
package graceful

import (
    "context"
    "net/http"
    "net/http/httptest"
    "testing"
    "time"
)

func TestServer_ShutdownCompletesInflightRequests(t *testing.T) {
    // 1. สร้าง handler ที่ทำงานช้า
    handler := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        time.Sleep(2 * time.Second)
        w.WriteHeader(http.StatusOK)
        w.Write([]byte("done"))
    })

    // 2. สร้าง server พร้อม graceful shutdown wrapper
    srv := &http.Server{
        Addr:    ":0",
        Handler: handler,
    }

    // 3. เริ่ม server ใน goroutine
    errChan := make(chan error, 1)
    go func() {
        if err := srv.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
            errChan <- err
        }
    }()
    defer srv.Close()

    // 4. สร้าง request channel
    respChan := make(chan *http.Response, 1)
    req, _ := http.NewRequest("GET", "http://localhost"+srv.Addr+"/", nil)

    go func() {
        client := &http.Client{}
        resp, err := client.Do(req)
        if err != nil {
            return
        }
        respChan <- resp
    }()
}
```

```

// 5. รอให้ request เริ่มทำงาน
time.Sleep(500 * time.Millisecond)

// 6. เรียก Shutdown (graceful shutdown)
ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()

shutdownDone := make(chan struct{})
go func() {
    if err := srv.Shutdown(ctx); err != nil {
        t.Errorf("Shutdown error: %v", err)
    }
    close(shutdownDone)
}()

// 7. รอ response จาก request ที่กำลังทำงานอยู่
select {
case <-shutdownDone:
    t.Error("Shutdown completed too early, before the slow request
finished")
case <-time.After(3 * time.Second):
    // คาดว่า request จะเสร็จภายใน 3 วินาที
}

select {
case resp := <-respChan:
    if resp.StatusCode != http.StatusOK {
        t.Errorf("Expected 200, got %d", resp.StatusCode)
    }
case <-time.After(2 * time.Second):
    t.Error("Request did not complete after shutdown")
}

// 8. รอให้ shutdown เสร็จ
<-shutdownDone
}

```

◇ วิธีที่ 3: ทดสอบ Shutdown Context Cancellation (Signal.NotifyContext)

ใช้ `signal.NotifyContext` เพื่อทดสอบการตอบสนองต่อสัญญาณ interrupt[reference:10]。

```

// main_test.go
package main

import (
    "context"
    "os"
    "os/signal"
    "syscall"
    "testing"

```

```

    "time"
)

func TestMain_ShutdownOnSignal(t *testing.T) {
    // สร้าง context ที่จะถูก cancel เมื่อได้รับ signal
    ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGINT,
    syscall.SIGTERM)
    defer stop()

    // จำลองการส่ง signal ไปยัง context
    done := make(chan struct{})
    go func() {
        <-ctx.Done()
        close(done)
    }()

    // ส่ง signal เอง
    p, err := os.FindProcess(os.Getpid())
    if err != nil {
        t.Fatalf("Failed to find process: %v", err)
    }
    if err := p.Signal(syscall.SIGINT); err != nil {
        t.Fatalf("Failed to send signal: %v", err)
    }

    // รอให้ context ถูก cancel
    select {
    case <-done:
        // success
    case <-time.After(1 * time.Second):
        t.Error("Context was not canceled after signal")
    }
}

```

แผนภาพการทำงานของ Graceful Shutdown

```

sequenceDiagram
    participant U as User
    participant K as Kubernetes/System
    participant S as Go HTTP Server
    participant H as HTTP Handler

    U->>S: Request 1 (slow, 5 sec)
    activate S
    S->>H: Process Request 1
    activate H

    K->>S: SIGTERM
    S->>S: Stop accepting new connections

```

```

U->>S: Request 2 (new)
S-->>U: Connection refused

Note over S,H: Request 1 still processing
H-->>S: Response completed
deactivate H
S-->>U: Response sent

S->>S: Shutdown complete
deactivate S

```

☑ สรุปแนวทางปฏิบัติที่ดี

หัวข้อ	คำแนะนำ
WebSocket Testing	ใช้ interface (<code>WSConn</code>) เพื่อ dependency injection จะทำให้ทดสอบ business logic ได้ง่ายที่สุด
Integration Test	ใช้ <code>httptest.NewServer</code> + Gorilla WebSocket client สำหรับ end-to-end testing
Mock Library	พิจารณาใช้ <code>wstest</code> (ไม่ต้องเปิดพอร์ต) หรือ <code>wsmock</code> (มี assertion helper)
Graceful Shutdown	ใช้ <code>http.Server.Shutdown()</code> ร่วมกับ <code>signal.NotifyContext</code> ใน production
Acceptance Test	สร้าง binary → รัน → ส่ง request เข้า → ส่ง SIGTERM → รอ response
Unit Test Shutdown	เรียก <code>Shutdown()</code> โดยตรงใน test และตรวจสอบว่า in-flight requests เสร็จสมบูรณ์

- ด้วยเทคนิคเหล่านี้ คุณสามารถมั่นใจได้ว่า WebSocket service ของคุณทำงานได้อย่างถูกต้องและสามารถ shutdown แบบไม่กระทบต่อผู้ใช้งานจริงใน production ได้

5. Struct

Struct คืออะไร?

Struct (structure) เป็นชนิดข้อมูล (type) ที่กำหนดขึ้นเอง โดยรวมฟิลด์ (fields) หลายชนิดเข้าเป็นหน่วยเดียวกัน ใช้แทน "record" หรือ "object" ในภาษา Go

Struct มีกี่แบบ?

- **Named struct** – ประกาศด้วย `type Name struct { fields }`
- **Anonymous struct** – ประกาศตรงจุดโดยไม่ตั้งชื่อ
- **Embedded struct** – ผัง struct อื่นโดยไม่ระบุชื่อฟิลด์ (ใช้แทน inheritance)
- **Empty struct** – `struct{}` ไม่มีฟิลด์ ใช้เป็นเซตหรือ signalling

ใช้อย่างไร ในกรณีไหน?

- จำลองข้อมูลที่มีโครงสร้าง (User, Product, Order)
- จัดกลุ่มข้อมูลที่เกี่ยวข้องกัน
- ผัง struct เพื่อ reuse โค้ด (composition)
- สร้าง method ให้ struct (รับ receiver)

หลักการทำงาน

1. ประกาศ struct type พร้อมฟิลด์พร้อมชนิด
2. สร้าง instance ด้วย `var, new, &StructName{...}`
3. เข้าถึงฟิลด์ด้วย dot (`.`)
4. สามารถเพิ่ม method ให้ struct ด้วย receiver (value หรือ pointer)

Dataflow Diagram (Flowchart TB) - Struct with Methods

```
graph TB
    A[ประกาศ type User struct] --> B[สร้าง instance u]
    B --> C[เรียก u.GetFullName]
    C --> D[รับค่า firstName + lastName]
```

ตัวอย่างการใช้งานจริง: ระบบจัดการผู้ใช้

```
package main

import "fmt"

type User struct {
    ID          int
    FirstName   string
    LastName    string
    Email       string
}

// Method with value receiver
func (u User) FullName() string {
    return u.FirstName + " " + u.LastName
}

// Method with pointer receiver (modify)
func (u *User) UpdateEmail(newEmail string) {
    u.Email = newEmail
}

func main() {
    // create struct
    user := User{
        ID:          1,
        FirstName:   "John",
        LastName:    "Doe",
    }
```

```

    Email:      "john@example.com",
}

fmt.Println(user.FullName()) // John Doe

user.UpdateEmail("john.doe@example.com")
fmt.Println(user.Email) // john.doe@example.com
}

```

เทมเพลต: Embedded struct (Composition)

```

type Base struct {
    CreatedAt time.Time
    UpdatedAt time.Time
}

type Product struct {
    Base      // embedded
    ID        int
    Name      string
    Price     float64
}

```

6. Inheritance

Inheritance คืออะไร?

Inheritance (การสืบทอด) เป็นกลไกใน OOP ที่คลาสลูกสามารถสืบทอดฟิลด์และเมธอดจากคลาสแม่ ทำให้สามารถ reuse โค้ดและสร้างความสัมพันธ์แบบ **is-a** Go **ไม่มี inheritance** แบบคลาส แต่ใช้ **embedding** (struct ผึง struct) และ **interface** เพื่อให้ได้ผลคล้ายกันโดยเน้น **composition** (has-a) มากกว่า

Inheritance มีกี่แบบ?

ในภาษา OOP ทั่วไป:

- **Single inheritance** – สืบทอดจากคลาสแม่เดียว (Java, C#)
- **Multiple inheritance** – สืบทอดจากหลายคลาส (C++) – เกิดปัญหา diamond problem
- **Multilevel inheritance** – A -> B -> C

Go ใช้ **embedding** ซึ่งไม่ใช่ inheritance แท้ แต่ให้ความสามารถในการ reuse โค้ดและ method promotion

ใช้อย่างไร ในกรณีไหน?

- ต้องการ reuse โค้ดโดยไม่ต้องเขียนซ้ำ (ใช้ embedding)
- ต้องการสร้าง "type hierarchy" แบบง่ายผ่าน interface

- หลีกเลี่ยงปัญหาความซับซ้อนของ deep inheritance
- ใช้ในไลบรารีเช่น GORM (ฝัง `gorm.Model`)

หลักการทำงาน (ใน Go ด้วย embedding)

1. ประกาศ struct `Parent` ที่มีฟิลด์และเมธอด
2. ประกาศ struct `Child` ที่ฝัง `Parent` (anonymous field)
3. `Child` สามารถเข้าถึงฟิลด์และเมธอดของ `Parent` โดยตรง (promotion)
4. `Child` สามารถ override เมธอดได้โดยการประกาศเมธอดชื่อเดียวกัน
5. ไม่สามารถแปลงจาก `Child` เป็น `Parent` ได้โดยอัตโนมัติ (ต้องใช้ interface)

Dataflow Diagram (Flowchart TB) - Embedding

```
graph TB
    A["A[type Animal struct]"] --> B["B[type Dog struct<br/>Animal]"]
    B --> C["C[dog := Dog{...}]"]
    C --> D["D[เรียก dog.Speak()<br/>พบเมธอดใน Dog?]"]
    D -- มี --> E["E[เรียก Dog.Speak]"]
    D -- ไม่มี --> F["F[เรียก Animal.Speak]"]
```

ตัวอย่างการใช้งานจริง: ระบบพาหนะ

```
package main

import "fmt"

// Base struct
type Vehicle struct {
    Brand string
    Year  int
}

func (v Vehicle) Info() string {
    return fmt.Sprintf("%s (%d)", v.Brand, v.Year)
}

func (v Vehicle) Start() string {
    return "Engine started"
}

// Car embeds Vehicle
type Car struct {
    Vehicle
    Doors int
}

// Override Start method
func (c Car) Start() string {
```

```

    return "Car engine started with key"
}

// Motorcycle embeds Vehicle
type Motorcycle struct {
    Vehicle
    HasSidecar bool
}

func main() {
    car := Car{
        Vehicle: Vehicle{Brand: "Toyota", Year: 2020},
        Doors: 4,
    }

    bike := Motorcycle{
        Vehicle: Vehicle{Brand: "Harley", Year: 2019},
        HasSidecar: false,
    }

    fmt.Println(car.Info()) // Toyota (2020) (promoted)
    fmt.Println(car.Start()) // Car engine started with key (overridden)
    fmt.Println(bike.Start()) // Engine started (inherited)
}

```

เทมเพลต: Embedding แบบ GORM

```

import "gorm.io/gorm"

type BaseModel struct {
    ID          uint          `gorm:"primaryKey"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
    DeletedAt   gorm.DeletedAt `gorm:"index"`
}

type User struct {
    BaseModel // embedding
    Name      string
    Email     string
}

// User จะมีฟิลด์ ID, CreatedAt, UpdatedAt, DeletedAt โดยอัตโนมัติ

```

สรุปเปรียบเทียบ

กระบวน
ทัศน์

จุดเน้น

Go รองรับอย่างไร

กระบวนทัศน์	จุดเน้น	Go รองรับอย่างไร
Procedural	ลำดับขั้นตอน, ฟังก์ชัน	พื้นฐานของ Go (main, functions, packages)
Concurrent	การทำงานหลายอย่างพร้อมกัน	goroutine, channel, sync
Functional	Pure functions, immutability, higher-order	first-class functions, closures, generics
OOP	วัตถุ, encapsulation, polymorphism	struct (data) + interface (behavior), composition
Struct	การรวมข้อมูล	type struct, methods, embedding
Inheritance	สืบทอดคุณสมบัติ (is-a)	ไม่มี direct inheritance แต่ใช้ embedding และ interface

แต่ละกระบวนทัศน์มีข้อดีและเหมาะกับงานต่างกัน Go ออกแบบให้เรียบง่ายและสามารถผสมผสานหลายกระบวนทัศน์ได้อย่างลงตัว โดยยึดหลัก "less is more" และ "composition over inheritance"

แหล่งอ้างอิง

- [The Go Programming Language Specification](#)
- [Effective Go](#)
- [Go by Example](#)
- [GORM Documentation](#)

1.14 ขั้นตอนการพัฒนาโปรแกรม

1. เขียนซอร์สโค้ด (.go)
2. คอมไพล์ (go build)
3. ทดสอบรัน (go run)
4. แก้ไขข้อผิดพลาด (debug)
5. จัดการแพ็คเกจ (go mod)
6. ทดสอบหน่วย (go test)

Heap, Worker, Pool คืออะไร?

1. Heap (ฮีบ)

Heap มี 2 ความหมายในบริบทการเขียนโปรแกรม:

1. โครงสร้างข้อมูล **Heap** (Priority Queue)

- เป็นต้นไม้ไบนารีที่สมบูรณ์ (complete binary tree) ซึ่งมีคุณสมบัติ **heap property**
 - **Max-heap:** โหนดพ่มีค่ามากกว่าหรือเท่ากับลูก
 - **Min-heap:** โหนดพ่มีค่าน้อยกว่าหรือเท่ากับลูก
- ใช้สำหรับสร้าง **priority queue** (คือลำดับความสำคัญ)

- การดำเนินการหลัก: **Push** (แทรก) และ **Pop** (ดึงค่าสูงสุด/ต่ำสุด) ด้วยเวลา $O(\log n)$

2. Heap Memory (หน่วยความจำฮีบ)

- พื้นที่หน่วยความจำที่ใช้สำหรับ allocate วัตถุที่มีอายุการใช้งานยาวนาน (เช่น ตัวแปรที่สร้างด้วย **new, make** ใน Go)
- ต่างจาก stack memory ซึ่งเป็นหน่วยความจำเฉพาะฟังก์ชัน

ในบริบทของ Queue Processor ที่พูดถึงก่อนหน้านี้ **heap** หมายถึง priority queue ที่ใช้จัดลำดับงานตามความสำคัญ

2. Worker (เวิร์กเกอร์)

Worker คือหน่วยงาน (goroutine, thread, หรือ process) ที่ทำหน้าที่ **ดึงงานจากคิวและประมวลผล**

- โดยทั่วไป Worker จะทำงานแบบไม่มีที่สิ้นสุด (loop) รอรับงาน
 - ช่วยให้ระบบสามารถทำงานหลายอย่างพร้อมกัน (concurrency)
 - การมีหลาย Workers เรียกว่า **Worker Pool**
-

3. Pool (พูล)

Pool คือกลุ่มของทรัพยากรที่เตรียมไว้ล่วงหน้าเพื่อนำมาใช้ซ้ำ ลดค่าใช้จ่ายในการสร้างและทำลายทรัพยากรบ่อย ๆ

- **Worker Pool**: กลุ่มของ Workers ที่พร้อมรับงาน
 - **Connection Pool**: กลุ่มของ connection ฐานข้อมูล
 - **Memory Pool**: จัดสรรบล็อกหน่วยความจำล่วงหน้า
-

แผนภาพประกอบ (Flowchart + Worker Pool + Priority Queue)

```
graph TD
    subgraph Producer
        A[Client / API] --> B[Create Job with Priority]
    end

    subgraph PriorityQueue[Heap-based Priority Queue]
        B --> C["(Min-Heap / Max-Heap)"]
    end

    subgraph WorkerPool[Worker Pool]
        D[Worker 1] --> E[Process Job]
        F[Worker 2] --> E
        G[Worker 3] --> E
        H[...]
    end

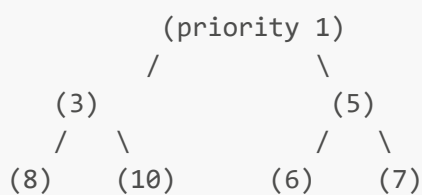
    C -- "Job with highest priority" --> D
    C -- "Job with highest priority" --> F
    C -- "Job with highest priority" --> G
```

```
E --> I[Database / External Service]
I --> J[Result / Acknowledge]
```

คำอธิบาย:

- **Producer** สร้างงานและกำหนด priority
- งานจะถูกแทรกเข้าไปใน **Priority Queue** (implemented ด้วย heap) ซึ่งจะจัดเรียงตาม priority
- **Worker Pool** มี Workers หลายตัว พร้อมดึงงานจากคิว (pop ค่าที่ priority สูงสุด)
- แต่ละ Worker ประมวลผลงานและส่งผลลัพธ์กลับ

ตัวอย่างการทำงานของ Heap ใน Priority Queue (Min-Heap)



- เมื่อ **Push** งาน priority 2 → แทรกที่ตำแหน่งท้าย แล้วปรับโครงสร้าง (heapify up)
- เมื่อ **Pop** งาน priority ต่ำสุด → ดึงราก (priority 1) แล้วนำโหนดสุดท้ายมาเป็นรากใหม่ แล้วปรับโครงสร้าง (heapify down)

ตัวอย่าง Worker Pool ใน Go

```
type Job struct {
    ID      int
    Priority int
    Payload interface{}
}

func worker(jobs <-chan Job, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs {
        fmt.Printf("Worker processing job %d (priority %d)\n", job.ID,
job.Priority)
        // simulate work
        time.Sleep(100 * time.Millisecond)
    }
}

func main() {
    const numWorkers = 3
    jobs := make(chan Job, 100)
```

```

var wg sync.WaitGroup
for i := 0; i < numWorkers; i++ {
    wg.Add(1)
    go worker(jobs, &wg)
}

// ส่งงานเข้า queue (ในตัวอย่างใช้ channel ธรรมดา แต่ priority queue ต้อง implement
เอง)
for i := 0; i < 10; i++ {
    jobs <- Job{ID: i, Priority: i % 3}
}
close(jobs)
wg.Wait()
}

```

สรุป

คำศัพท์	ความหมาย	การประยุกต์ใน Queue Processor
Heap	โครงสร้างข้อมูลแบบ priority queue	ใช้จัดเรียงงานตามลำดับความสำคัญก่อนดึงไปประมวลผล
Worker	หน่วยประมวลผล (goroutine/thread)	ดึงงานจากคิวและทำงานจริง
Pool	กลุ่มทรัพยากรที่เตรียมไว้ล่วงหน้า	Worker Pool ช่วยควบคุม concurrency, reuse goroutines

การรวมกันของ **Heap + Worker Pool** ช่วยให้ระบบสามารถจัดการงานที่มี priority ต่างกันได้อย่างมีประสิทธิภาพ โดยไม่ต้องสร้าง goroutine ใหม่ทุกครั้ง และประมวลผลงานสำคัญก่อนเสมอ

บทที่ 2: รู้จักกับภาษา Go

2.1 ประวัติความเป็นมา

ภาษา Go (หรือ Golang) ถูกพัฒนาโดย Google เริ่มต้นในปี 2007 โดย Robert Griesemer, Rob Pike, และ Ken Thompson เปิดตัวเป็นโอเพนซอร์สในปี 2009 จุดประสงค์เพื่อแก้ปัญหาที่เกิดขึ้นในภาษา C++ และ Java ในระบบขนาดใหญ่ของ Google เช่น การคอมไพล์ที่ช้า, การจัดการการทำงานพร้อมกันที่ซับซ้อน, และความยุ่งยากในการบำรุงรักษา

2.2 จุดเด่นของภาษา Go

- **เรียบง่ายและอ่านง่าย** : ไวยากรณ์กระชับ ไม่มีฟิเจอร์ที่ซับซ้อนเกินจำเป็น
- **คอมไพล์เร็ว** : สามารถคอมไพล์โปรเจกต์ขนาดใหญ่ได้ในไม่กี่วินาที
- **การจัดการหน่วยความจำอัตโนมัติ** : มี garbage collector ที่มีประสิทธิภาพ
- **Concurrency ระดับภาษา** : goroutine และ channel ทำให้เขียนโปรแกรม concurrent ได้ง่าย
- **Static typing** : ตรวจสอบชนิดข้อมูลตั้งแต่ตอนคอมไพล์ ป้องกันข้อผิดพลาด
- **เครื่องมือที่ครบครัน** : go fmt, go test, go mod, go vet, go doc
- **สามารถคอมไพล์ข้ามแพลตฟอร์ม** (cross-compile) ไปยัง Windows, Linux, macOS, ARM เป็นต้น

2.3 โครงสร้างภาษา Go

- ไม่มี class แต่ใช้ struct และ method

- ไม่มี inheritance แต่ใช้ composition และ interface
- ไม่มี exception handling แต่ใช้ error return value
- มี garbage collection
- มี pointer แต่ไม่มี pointer arithmetic

2.4 ตัวอย่าง Hello World ใน Go

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

ข้อดีของ Golang

1. การทำงานพร้อมกัน (Concurrency) ที่ยอดเยี่ยม

มี goroutine ซึ่งเบากว่า thread ทั่วไปมาก และ channel สำหรับสื่อสารระหว่าง goroutine ทำให้เขียนโปรแกรม concurrent ได้ง่ายและมีประสิทธิภาพ

2. คอมไพล์เร็วมาก

โค้ด Go คอมไพล์เป็น binary ไฟล์เดียว ใช้เวลาไม่กี่วินาที แม้โปรเจกต์ใหญ่ ช่วยให้ development cycle รวดเร็ว

3. ไวยากรณ์เรียบง่าย อ่านง่าย

ไม่มีฟีเจอร์ซับซ้อนอย่าง inheritance, generics (แต่ปัจจุบันมีแล้วแบบจำกัด), ternary operator ทำให้โค้ดตรงไปตรงมา เรียนรู้ได้ไว

4. ประสิทธิภาพดี

เป็น compiled language + garbage collection ที่ปรับแต่งมาดี ทำงานเร็วใกล้เคียง C/C++ ในหลายงาน

5. Standard library ครบครัน

โดยเฉพาะด้าน networking, HTTP, encoding, crypto ช่วยให้เขียน backend, web server, API ได้โดยไม่ต้องพึ่งพาไลบรารีภายนอกมาก

6. เครื่องมือพัฒนาคุณภาพสูง

มี `go fmt` จัดรูปแบบโค้ดอัตโนมัติ, `go test` สำหรับเทสต์, `go mod` สำหรับจัดการ dependency, `go vet` ตรวจสอบข้อผิดพลาด

7. Cross-platform compile ง่าย

ตั้งค่า `GOOS` และ `GOARCH` ก็คอมไพล์ข้ามระบบปฏิบัติการและสถาปัตยกรรมได้ทันที

ข้อเสียของ Golang

1. Error handling ซ้ำซาก

ต้องตรวจสอบ error ทุกครั้งผ่าน multiple return values ทำให้โค้ดยาวและมี `if err != nil` เยอะ ขาดกลไก exception หรือ structured error handling ที่กระชับ

2. Generics มาช้าและยังมีข้อจำกัด

แม้เวอร์ชัน 1.18+ จะเพิ่ม generics แล้ว แต่ก็ซับซ้อนกว่าและรองรับน้อยกว่าภาษาอื่น (ไม่มี method generics, type constraints ยังไม่สมบูรณ์)

3. ขาดการสืบทอด (inheritance)

ใช้ composition แทน ซึ่งอาจไม่ถนัดสำหรับนักพัฒนาที่คุ้นเคย OOP อย่าง Java หรือ C++ และต้องปรับ mindset

4. Package management ในอดีตยุ่งยาก

ปัจจุบัน go modules ใช้ได้ดีแล้ว แต่ก็ยังมีปัญหาเรื่อง dependency และ versioning บางกรณี โดยเฉพาะถ้ามี replace หรือ indirect dependencies

5. Garbage collection อาจกระทบ real-time

แม้ GC จะเร็วขึ้นมาก แต่ก็ยังมี stop-the-world ช่วงสั้นๆ ไม่เหมาะกับระบบที่ต้องการ latency ต่ำมากๆ เช่น trading, เกม

6. ระบบ GUI และมีมือถือน้อย

ไม่มี native binding สำหรับ GUI บน desktop หรือ mobile ที่แข็งแรง ไลบรารีก็ยังไม่ มักใช้ Go สำหรับ backend หรือ CLI เป็นหลัก

7. ขาดฟีเจอร์ syntactic sugar บางอย่าง

เช่น ternary operator, while loop (ใช้ for แทน), function overloading ทำให้บางคนรู้สึกว่าจะต้องเขียนโค้ดยืดเยื้อกว่าภาษาอื่น

8. ชุมชนและ ecosystem น้อยกว่า Java, Python, JS

แม้จะเติบโตเร็ว แต่ไลบรารีในบาง domain (data science, ML, GUI, game) ยังมีให้เลือกน้อย

สรุป: Go เหมาะสำหรับงานระบบ backend, microservice, CLI tool, network programming ที่ต้องการประสิทธิภาพสูงและ concurrent ดี แต่ถ้าต้องการ OOP เต็มรูปแบบ, GUI, หรือ rapid prototyping ที่ error handling กระชับ อาจต้องพิจารณาภาษาอื่นประกอบ

ภาษา Go (Golang) เป็นภาษาเอนกประสงค์ที่ถูกออกแบบมาให้เรียบง่าย ทำงานได้เร็ว และจัดการงานพร้อมกัน (Concurrency) ได้ดีเยี่ยม ทำให้ปัจจุบัน Go ถูกนำไปใช้พัฒนาซอฟต์แวร์หลากหลายประเภท ตั้งแต่ระบบระดับองค์กรไปจนถึงเครื่องมือ CLI เล็ก ๆ

นี่คือสิ่งที่คุณสามารถทำได้กับ Go ในโลกจริง:

1. พัฒนา Web Backend / REST API / Microservices

Go คือตัวเลือกอันดับต้น ๆ สำหรับการสร้าง API และบริการบนเซิร์ฟเวอร์ ด้วยความเร็วสูงและการใช้หน่วยความจำน้อย

- **Web Framework ยอดนิยม:** Gin, Echo, Fiber (เร็วระดับใกล้เคียง Node.js/Express แต่ใช้ทรัพยากรน้อยกว่า)
- **Microservices:** Go มีน้ำหนักเบา, compile เป็นไฟล์ไบนารีเดียว ทำให้ deploy ด้วย Docker ได้ง่าย เหมาะกับสถาปัตยกรรมแบบ Distributed Systems
- **GraphQL:** มี library เช่น **gqlgen** สำหรับสร้าง GraphQL server

2. เครื่องมือ Command Line (CLI) และ DevOps

Go ถูกใช้สร้างเครื่องมือ CLI ระดับโลกมากมาย เพราะ compile เป็นไฟล์ **执行** ไฟล์เดียว ไม่ต้องพึ่งพา runtime อะไรเพิ่ม

- **ตัวอย่างเครื่องมือที่เขียนด้วย Go:** Docker, Kubernetes, Terraform, Prometheus, Hugo

- **ไลบรารียอดนิยม:** **cobra** (สร้าง CLI แบบ professional), **urfave/cli**

3. System Programming และ Network Software

Go มี standard library ที่แข็งแกร่งเรื่อง net/http, tcp/udp, และ low-level I/O ทำให้เหมาะกับ:

- **Proxy, Load Balancer, API Gateway** (เช่น Traefik, Caddy)
- **เซิร์ฟเวอร์ TCP/UDP ความเร็วสูง**
- **WebSocket server**
- **DNS server, VPN tool**

4. การประมวลผลแบบ Concurrent และ Parallel

Go มี goroutine (lightweight thread) และ channel ทำให้เขียนโปรแกรมที่ทำงานพร้อมกันหลายล้านงานได้ง่ายและปลอดภัยกว่า threading แบบดั้งเดิม

- **Real-time data processing**
- **Web crawler / scraper**
- **ระบบ Chat, Gaming server**
- **ETL pipeline**

5. Cloud Native, Distributed Systems

Go คือภาษาหลักของระบบคลาวด์ยุคใหม่

- **พัฒนา Operator สำหรับ Kubernetes** (ใช้ client-go)
- **Serverless function** (AWS Lambda, GCP Cloud Functions รองรับ Go)
- **Service Mesh** (เช่น Istio เขียนด้วย Go)

6. เขียน Desktop Application (น้อย แต่ทำได้)

แม้จะไม่ใชเป้าหมายหลัก แต่ก็มีส่วนช่วย:

- **Fyne** หรือ **Gio** สำหรับ GUI ข้ามแพลตฟอร์ม
- **Wails** – ใช้ Go เป็น backend และ HTML/JS เป็น frontend (คล้าย Electron แต่เบากว่า)

7. เกมและกราฟิก (เฉพาะส่วน server หรือ game logic)

- เกมแนวเรียลไทม์ที่ต้องการความเร็วสูงและจัดการผู้เล่นพร้อมกันจำนวนมาก (Game server)
- มี library เช่น **ebiten** สำหรับทำเกม 2D ได้พอสมควร

8. Blockchain และ Cryptocurrency

- **Ethereum (Geth)** – โคลเอนต์หลักของ Ethereum เขียนด้วย Go
- **Hyperledger Fabric** – blockchain สำหรับองค์กร
- กระเป๋าเงิน, เครื่องมือเกี่ยวกับ crypto

9. ประมวลผลข้อมูล, Machine Learning (ระดับต้น)

- Go ไม่โดดเด่นเท่า Python แต่ก็มี ecosystem พอสมควร: **gonum** (matrix, statistics), **gorgonia** (deep learning คล้าย TensorFlow)
- เหมาะกับการนำโมเดลที่เทรนแล้วไปใช้งานเป็น service (inference)

10. เขียน C/C++ extension และระบบฝังตัว (Embedded)

- ผ่าน **cgo** เรียกใช้ไลบรารี C ได้
- รันบน Raspberry Pi, ARM, หรือ IoT device ขนาดเล็ก (แม้จะกิน resource มากกว่า Rust/C เล็กน้อย)

จุดเด่นที่ทำให้ Go ทำสิ่งเหล่านี้ได้ดี

- **ง่าย เรียนรู้เร็ว:** syntax คล้าย C แต่ปลอดภัยกว่า มี garbage collection
- **compile เร็วมาก:** แม้โปรเจกต์ใหญ่ก็ไม่กี่วินาที
- **ออกแบบมาสำหรับ multicore:** concurrency เป็นส่วนหนึ่งของภาษา
- **Standard library มหาศาล:** มี net/http, json, crypto, template, testing ในตัว
- **Deploy ง่ายสุด ๆ:** compile เป็นไฟล์เดียว (static binary) ลงบนเซิร์ฟเวอร์ได้เลย

สรุป: Go เหมาะกับใคร?

- นักพัฒนาระบบคลาวด์, DevOps, SRE
- คนที่เขียน backend API ต้องการ performance สูง
- ทีมที่ต้องการภาษาเรียนง่าย แต่แข็งแกร่งระดับ production
- โปรเจกต์ที่ต้องขยาย scale แนวนอน (horizontal scaling) หลายล้าน concurrent connection

ถ้าคุณกำลังมองหาภาษาที่ **ทำงานเร็วเท่า C#/Java** แต่ **เขียนง่ายเท่า Python** และ **จัดการ concurrency ได้เป็นระบบ** Go คือตัวเลือกที่ไม่ควรพลาด

Go (Golang) กับงาน IoT – เป็นคู่ที่ลงตัวมากในหลายระดับ โดยเฉพาะในส่วนของ **Edge Gateway**, **เซิร์ฟเวอร์กลาง (Cloud/Backend)** และ **อุปกรณ์ที่มีทรัพยากรพอสมควร** (เช่น Raspberry Pi, BeagleBone, อุปกรณ์ Linux ARM)

1. Go ใช้ทำอะไรใน IoT บ้าง?

ระดับ	ตัวอย่างงาน	ความเหมาะสมของ Go
Edge Gateway / Hub	รวมข้อมูลจากเซ็นเซอร์หลายตัว, ประมวลผลเบื้องต้น, ส่งขึ้นคลาวด์	★★★★★ ยอดเยี่ยม
Backend / Cloud	รับข้อมูลจากอุปกรณ์นับล้าน, API, Dashboard, MQTT broker	★★★★★ ยอดเยี่ยม
อุปกรณ์ปลายทาง (Device)	เซ็นเซอร์ธรรมดา, แอคทูเอเตอร์, ไมโครคอนโทรลเลอร์	★★ พอใช้ (ถ้าใช้ TinyGo)
Real-time control	ควบคุมหุ่นยนต์, ตอบสนองระดับมิลลิวินาที	★★ (มี GC อาจไม่เหมาะกับ hard real-time)

2. จุดเด่นของ Go สำหรับ IoT

☑ **Cross-compile** ง่ายมาก

GOOS=linux GOARCH=arm GOARM=7 go build

ได้ binary ไฟล์เดียว วางรันบน Raspberry Pi, Orange Pi, หรือ router ที่ใช้ MIPS ได้ทันที

☑ **Concurrency** ระดับสูง

ใช้ goroutine จัดการเซ็นเซอร์หลายตัว, MQTT topic หลายช่อง, HTTP request พร้อมกัน โดยไม่ต้องมานั่งเขียน threading ให้ปวดหัว

☑ **ประสิทธิภาพดี** หน่วยความจำน้อย

binary เล็ก (ประมาณ 5-20 MB) ใช้ RAM น้อยกว่า Java หรือ Python มาก เหมาะกับ gateway ที่มี RAM 256-512 MB

☑ **Standard library** ครบคลุม

มี net/http, mqtt (มี library ชั้่นนำ), serial port, json, websocket ในตัว

☑ **เสถียรภาพระดับ production**

Docker, Kubernetes, Prometheus เขียนด้วย Go – พิสูจน์แล้วว่ารับภาระหนักได้

3. ข้อควรระวัง / ข้อจำกัด

⚠ **Garbage Collection (GC)**

แม้ Go จะ GC เร็วมาก แต่ก็ยังเกิด latency spike เป็นบางช่วง (ปกติ 10-100 μ s ถึง ms) ถ้าต้องการควบคุมหุ่นยนต์หรือเซ็นเซอร์ที่ต้องการความแม่นยำระดับไมโครวินาที ควรใช้ C หรือ Rust

⚠ **ขนาด binary**

เทียบกับ C แล้ว Go ใหญ่กว่า (~1-2 MB ขั้นต่ำ) อาจไม่เหมาะกับไมโครคอนโทรลเลอร์หน่วยความจำ 32KB

⚠ **Driver** รองรับฮาร์ดแวร์น้อยกว่า Python/C

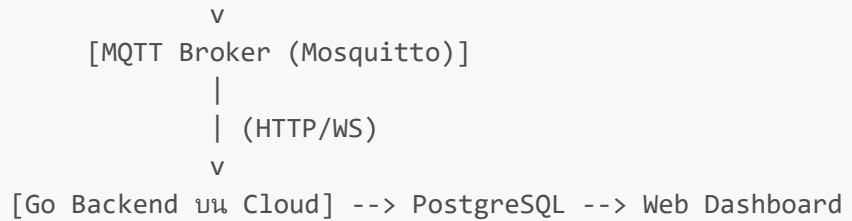
การติดต่อ GPIO, I2C, SPI, ADC ใน Go มี library แต่ไม่หนาเท่า **RPi.GPIO** ของ Python หรือ libc ของ C

4. เครื่องมือและไลบรารีเด่น

ชื่อ	ใช้ทำอะไร
TinyGo	คอมไพล์ Go ไปรันบน Arduino, ESP32, STM32, RP2040 (ใช้ LLVM แทน GC ปกติ)
Gobot	เฟรมเวิร์ก IoT คล้าย Johnny-Five รองรับ drone, robot, เซ็นเซอร์หลายร้อยตัว
Periph.io	ชั้น GPIO, I2C, SPI บน Linux embedded (Raspberry Pi, Jetson Nano, etc.)
Eclipse Paho MQTT	MQTT client สำหรับ Go
Serial	go.bug.st/serial สำหรับอ่านค่าเซ็นเซอร์ผ่าน USB/COM port

5. ตัวอย่างโครงสร้างระบบ IoT ด้วย Go

```
[เซ็นเซอร์ DS18B20] --(1-wire)--> [Raspberry Pi (Go)]
                                |
                                (MQTT)
```



หรือสำหรับอุปกรณ์ขนาดเล็ก:

Arduino + TinyGo → อ่านค่า → ส่ง serial → Edge Gateway (Go) → ประมวลผล → ส่งขึ้นคลาวด์

6. ภาษาไหนดีที่สุดสำหรับ IoT? (เทียบคร่าวๆ)

ภาษา	เหมาะกับ
C	Firmware, microcontroller หน่วยความจำต่ำ, hard real-time
C++	Arduino, ESP32, Robotics
Python	Prototyping ด่วน, Data analysis, บน Raspberry Pi ที่มีพลัง
Go	Gateway, Backend, ระบบที่ต้องการความเสถียรและ concurrency สูง
Rust	ปลอดภัย, ไร้ GC, กำลังมาแรงใน embedded สมัยใหม่

สรุป: ใช้ Go ใน IoT เมื่อใด?

☑️ **การใช้ Go ถ้า:**

- คุณทำ Edge Gateway, MQTT aggregator, API proxy
- ต้องการ performance สูง แต่ไม่อยากเขียน C
- มีอุปกรณ์ Linux ARM (Raspberry Pi ขึ้นไป)
- ต้องการ deploy เป็น container หรือ binary ง่ายๆ

❌ **ไม่เหมาะถ้า:**

- ใช้ MCU ราคาถูก (ATmega, ESP8266 แรม 80KB)
- ต้องการ hard real-time ระดับ <10 μs
- มี driver เฉพาะที่เขียนใน Python หรือ C อยู่แล้ว และไม่มี wrapper Go

การจะเริ่มต้นโปรเจกต์ IoT ด้วย Go ให้สำเร็จได้นั้น หัวใจสำคัญคือการเลือกเส้นทางให้ตรงกับโปรเจกต์ เพราะ Go มีความสามารถสองแบบหลัก ๆ คือ การพัฒนา **"ระบบปฏิบัติการ" (Edge Gateway/Backend)** ด้วย Go เต็มรูปแบบ และการพัฒนา **"ตัวอุปกรณ์" (Microcontroller)** โดยตรงด้วย TinyGo

1. เลือกฮาร์ดแวร์ให้เหมาะกับงาน

สิ่งแรกที่ต้องทำคือเลือก "ฮาร์ดแวร์" ให้สอดคล้องกับบทบาทของ Go ในโปรเจกต์คุณ

- **Raspberry Pi (Linux SBC):** เหมาะสำหรับทำ Edge Gateway ที่ต้องประมวลผลข้อมูลจากเซ็นเซอร์หลายตัว หรือเป็น Backend ขนาดเล็ก สามารถรัน Go ได้โดยตรง[reference:0]
- **ESP32 / Arduino (Microcontroller):** หากต้องการควบคุมเซ็นเซอร์หรือแอคทูเอเตอร์โดยตรง แนะนำให้ใช้ **TinyGo** ซึ่งเป็น Go เวอร์ชันสำหรับไมโครคอนโทรลเลอร์[reference:1][reference:2]
- **Computer ปกติ (Windows/macOS/Linux):** เหมาะสำหรับจำลอง (Simulation) หรือพัฒนา Cloud Backend ก่อนนำไป Deploy จริง[reference:3]

2. ติดตั้งเครื่องมือที่จำเป็น

พอเลือกฮาร์ดแวร์ได้แล้ว ขั้นตอนต่อไปคือการติดตั้งเครื่องมือ (Toolchain) ให้พร้อม

- **ติดตั้ง Go ปกติ:** ดาวน์โหลดและติดตั้งจากเว็บไซต์ทางการของ Go[reference:4]
- **ติดตั้ง TinyGo (สำหรับ Microcontroller):** ไปที่เว็บไซต์ TinyGo.org ดาวน์โหลดตัวติดตั้งที่ตรงกับระบบปฏิบัติการของคุณ หรือคอมไพล์จาก source code ก็ได้[reference:5][reference:6]
- **ติดตั้ง Cross-compilation Tools (สำหรับ Linux ARM):** การจะข้าม platform (Cross-compile) โปรเจกต์ Go ของคุณบนเครื่อง PC ให้ไปรันบน Raspberry Pi ได้นั้น สิ่งที่ต้องทำมีแค่เปิดใช้งาน Go Modules และใช้ Environment Variables เท่านั้น[reference:7][reference:8]

3. เขียนโปรแกรม Go สำหรับ IoT (Hands-on)

เมื่อเครื่องมือพร้อม ก็ถึงเวลาเขียนโปรแกรมเพื่อสื่อสารกับฮาร์ดแวร์จริง ๆ กันเลย

ตัวอย่างที่ 1: เขียนบน Raspberry Pi โดยตรง (ใช้ Go ปกติ + periph.io)

หากคุณใช้ Raspberry Pi เป็น Edge Gateway ให้เริ่มจากการควบคุมอุปกรณ์พื้นฐานอย่าง LED ก่อน โดยใช้ **periph.io** library

```
package main

import (
    "log"
    "time"
    "periph.io/x/conn/v3/gpio"
    "periph.io/x/conn/v3/gpio/gpioreg"
    "periph.io/x/host/v3"
)

func main() {
    // 1. Initialize host
    if _, err := host.Init(); err != nil {
        log.Fatal(err)
    }

    // 2. Get pin by name (BCM 18)
    pin := gpioregByName("GPIO18")
    if pin == nil {
        log.Fatal("Pin not found")
    }

    // 3. Configure as output
```

```

if err := pin.Out(gpio.High); err != nil {
    log.Fatal(err)
}

// 4. Blink LED
for i := 0; i < 5; i++ {
    pin.Set(gpio.High)
    time.Sleep(500 * time.Millisecond)
    pin.Set(gpio.Low)
    time.Sleep(500 * time.Millisecond)
}
}

```

[14+L11-L17]

วิธีรัน: `go run main.go` (อาจต้องใช้ `sudo` หากเจอ permission error)[reference:9]

ตัวเลือกที่ 2: เขียนบน ESP32 โดยตรง (ใช้ TinyGo + machine package)

หากคุณใช้ ESP32 เป็นอุปกรณ์ปลายทาง ให้เริ่มจากโปรแกรมกะพริบ LED บนบอร์ด (Blink) ก่อน

```

package main

import (
    "machine"
    "time"
)

func main() {
    led := machine.LED // กำหนด pin ของ LED
    led.Configure(machine.PinConfig{Mode: machine.PinOutput})

    for {
        led.Toggle()
        time.Sleep(time.Second)
    }
}

```

[11+L17-L19]

วิธีรัน: `tinygo flash -target=esp32c3-supermini blink.go`[reference:10]

4. เชื่อมต่ออุปกรณ์กับระบบ IoT (MQTT)

เมื่อควบคุมฮาร์ดแวร์ได้แล้ว ขั้นตอนต่อไปคือการส่งข้อมูลขึ้น Cloud หรือระบบอื่น ๆ โดยใช้ MQTT

- **ติดตั้ง MQTT Client:** ในโปรเจกต์ Go ของคุณ ให้ใช้คำสั่ง `go get github.com/eclipse/paho.mqtt.golang`[reference:11]

- เขียน **Publisher (อุปกรณ์)**: ตัวอุปกรณ์จะทำหน้าที่อ่านค่าจากเซ็นเซอร์แล้วส่งข้อมูล JSON ออกไปยัง Topic `sensor/temperature`[reference:12]

5. เริ่มต้นสร้าง Full-stack IoT Project

เมื่อเข้าใจพื้นฐานครบถ้วนแล้ว ลองนำความรู้ทั้งหมดมาประกอบเป็นโปรเจกต์เต็มรูปแบบดู

- **โครงสร้าง 3 ชั้น**: สร้างระบบ IoT แบบมีอาชีวะด้วยสถาปัตยกรรม 三层架构[reference:13]
 - **** (Perception Layer)****: เขียนโปรแกรมด้วย **TinyGo** รันบน ESP32 เพื่ออ่านค่าจากเซ็นเซอร์ DHT11[reference:14]
 - **** (Network Layer)****: ตัว ESP32 จะส่งข้อมูลขึ้น Cloud ผ่าน MQTT protocol[reference:15]
 - **** (Application Layer)****: สร้าง Web Dashboard ด้วย Go (ใช้ Gin/Echo) เพื่อแสดงผลข้อมูลแบบ Real-time[reference:16]
- **Gobot Framework**: สำหรับโปรเจกต์ที่ซับซ้อนขึ้น เช่น การควบคุมหุ่นยนต์หรือ Drone แนะนำให้ใช้ **Gobot framework** ซึ่งเป็นเครื่องมือระดับสูงสำหรับ Go โดยเฉพาะ[reference:17]

 แหล่งเรียนรู้เพิ่มเติม

- **Official Documentation**: golang.org/doc, tinygo.org
- **GitHub Repositories**: [TinyGo](#), [Gobot](#), [Periph.io](#)
- **Tutorials**: [datasea.cn Go IoT tutorials](#)[reference:18], [MQTT with Go](#)[reference:19]

แนะนำให้เริ่มจากโปรเจกต์ **"Environmental Monitoring Node"** ที่ใช้ ESP32 อ่านค่าจากเซ็นเซอร์ DHT11 ส่งข้อมูลผ่าน MQTT ขึ้นไปยัง Broker สาธารณะ (เช่น EMQX) จากนั้นใช้ Go เขียน Service เพื่อ Subscribe ข้อมูลและแสดงผลผ่าน Web Dashboard วิธีนี้จะช่วยให้คุณเข้าใจภาพรวมของระบบ IoT ได้อย่างเป็นระบบ

Remark

Nil ในภาษา Go (Golang)

Nil คืออะไร?

Nil คือค่าศูนย์ (zero value) ของประเภทข้อมูลที่มีความสามารถในการอ้างอิงถึงพื้นที่หน่วยความจำอื่นๆ (reference types) ในภาษา Go ซึ่งได้แก่:

- **Pointer (*T)** → ตัวชี้ไปยังค่าของชนิด **T**
- **Slice ([]T)** → ส่วนตัดตอนของอาร์เรย์ (ขนาดปรับเปลี่ยนได้)
- **Map (map[K]V)** → แผนที่จับคู่คีย์ **K** กับค่า **V**
- **Channel (chan T)** → ช่องทางรับส่งข้อมูลชนิด **T** ระหว่าง goroutine
- **Function (func)** → ฟังก์ชัน
- **Interface (interface{})** → ชุดของเมธอด (ข้อกำหนด) ที่ชนิดอื่นต้องทำให้ได้

nil หมายถึง "ไม่มีค่า" หรือ "ไม่ได้ชี้ไปที่สิ่งใด" เหมือน **null** ในภาษาอื่นๆ แต่มีความปลอดภัยและพฤติกรรมที่แตกต่างกันในแต่ละประเภท

Nil มีกี่แบบ?

ใน Go **nil** ไม่ใช่ประเภทในตัวของมันเอง แต่เป็นค่าที่กำหนดไว้ล่วงหน้า (predeclared identifier) ซึ่งสามารถกำหนดให้กับตัวแปรของ **6 ประเภท** ดังนี้:

ลำดับ	ประเภทข้อมูล	ตัวอย่างการประกาศ nil	หมายเหตุ
1	Pointer	<code>var p *int = nil</code>	ชี้ไปที่ nowhere
2	Slice	<code>var s []int = nil</code>	ไม่มี underlying array
3	Map	<code>var m map[string]int = nil</code>	ไม่มี hash table
4	Channel	<code>var ch chan int = nil</code>	ไม่มีการเชื่อมต่อ
5	Function	<code>var f func() = nil</code>	ไม่มีโค้ดให้เรียก
6	Interface	<code>var i interface{} = nil</code>	ทั้ง dynamic type และ dynamic value เป็น nil

ข้อสำคัญ: แม้ nil จะเป็นค่าเดียวกัน แต่การใช้งานกับแต่ละประเภทให้ผลลัพธ์ต่างกัน เช่น การอ่านจาก nil map ได้ค่า zero value แต่การเขียนลง nil map จะ panic

Panic ในภาษา Go คือ **ภาวะที่โปรแกรมทำงานต่อไปไม่ได้** เนื่องจากเกิดข้อผิดพลาดร้ายแรง (runtime error) หรือถูกเรียกโดยตรง (`panic()`) ส่งผลให้:

- ฟังก์ชันปัจจุบันหยุดทำงานทันที
- แทนเรียก (stack) ถูกคลี่กลับ (unwind) พร้อมรัน `defer` ตามลำดับ
- หากไม่มี `recover()` มาจับไว้ โปรแกรมจะ **จบการทำงานทันที** พร้อมแสดง stack trace

เปรียบเทียบง่ายๆ:

- **error** = ปัญหาที่คาดเดาได้ (เช่น หาไฟล์ไม่เจอ) → จัดการได้
- **panic** = ปัญหาที่ไม่ควรเกิดขึ้น (เช่น อ่านค่า nil pointer) → โปรแกรมพัง

🌀 Nil ใช้อย่างไร? ในกรณีไหน?

1. Nil Pointer

- **ใช้เมื่อ:** ต้องการแสดงว่า pointer ยังไม่ได้ชี้ไปที่ struct หรือตัวแปรใดๆ
- **การใช้งาน:** ต้องตรวจสอบ nil ก่อน dereference (`*p`) มิฉะนั้นจะ panic

```
var p *int
fmt.Println(p == nil) // true
// *p = 10 // panic: runtime error: invalid memory address
```

2. Nil Slice

- **ใช้เป็น:** ค่าเริ่มต้นของ slice ที่ยังไม่มี element (เทียบเท่ากับ empty slice แต่ต่างที่ nil slice มี length=0, capacity=0)
- **การใช้งาน:** สามารถใช้ `append` กับ nil slice ได้ (Go จะสร้าง underlying array ให้อัตโนมัติ)

```
var s []int // nil slice
s = append(s, 1) // ทำงานได้ดี
fmt.Println(s) // [1]
```

3. Nil Map

- **ใช้เมื่อ:** ประกาศ map โดยไม่ใช้ **make** หรือ literal
- **ข้อควรระวัง:** อ่านได้ (คืนค่า zero value) แต่ **เขียนไม่ได้** (จะ panic)

```
var m map[string]int
fmt.Println(m["key"]) // 0 (ไม่ panic)
// m["key"] = 1 // panic: assignment to entry in nil map
```

4. Nil Channel

- **ใช้เมื่อ:** ต้องการปิดการสื่อสาร หรือยังไม่ได้ initialize channel
- **พฤติกรรม:** การส่ง (**ch <- v**) และรับ (**<-ch**) จะ **block** ตลอดไป (deadlock ถ้าไม่มี goroutine อื่น)

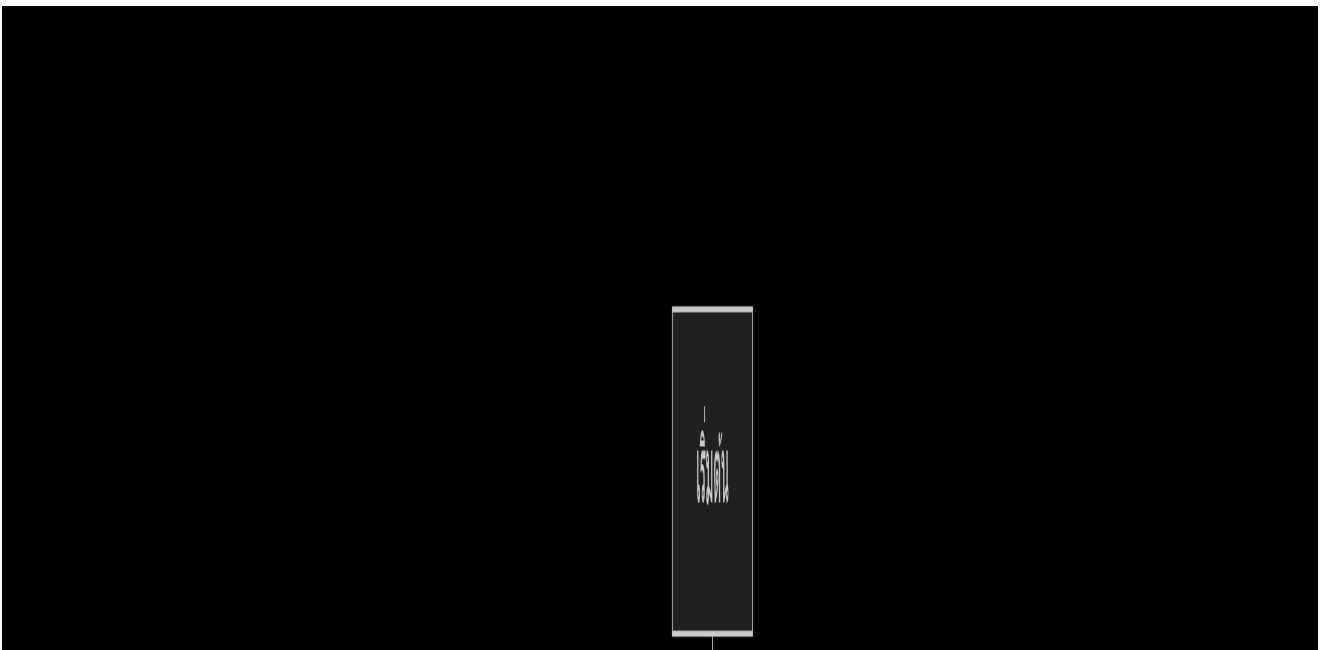
5. Nil Function

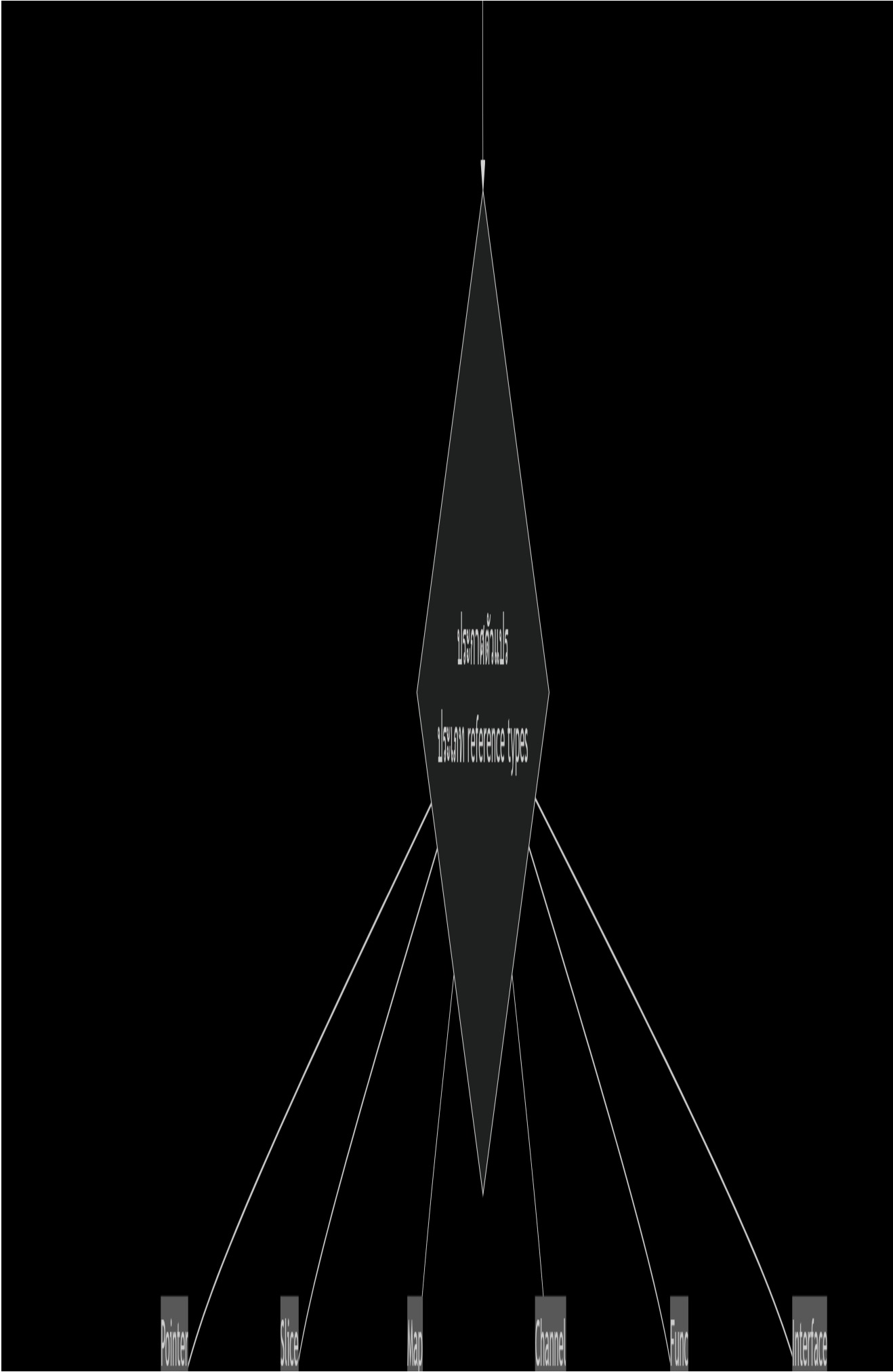
- **ใช้เมื่อ:** ตัวแปรฟังก์ชันยังไม่ได้ถูกกำหนด (ใช้กับ strategy pattern หรือ callback)
- **การใช้งาน:** ต้องตรวจสอบ nil ก่อนเรียก มิฉะนั้นจะ panic

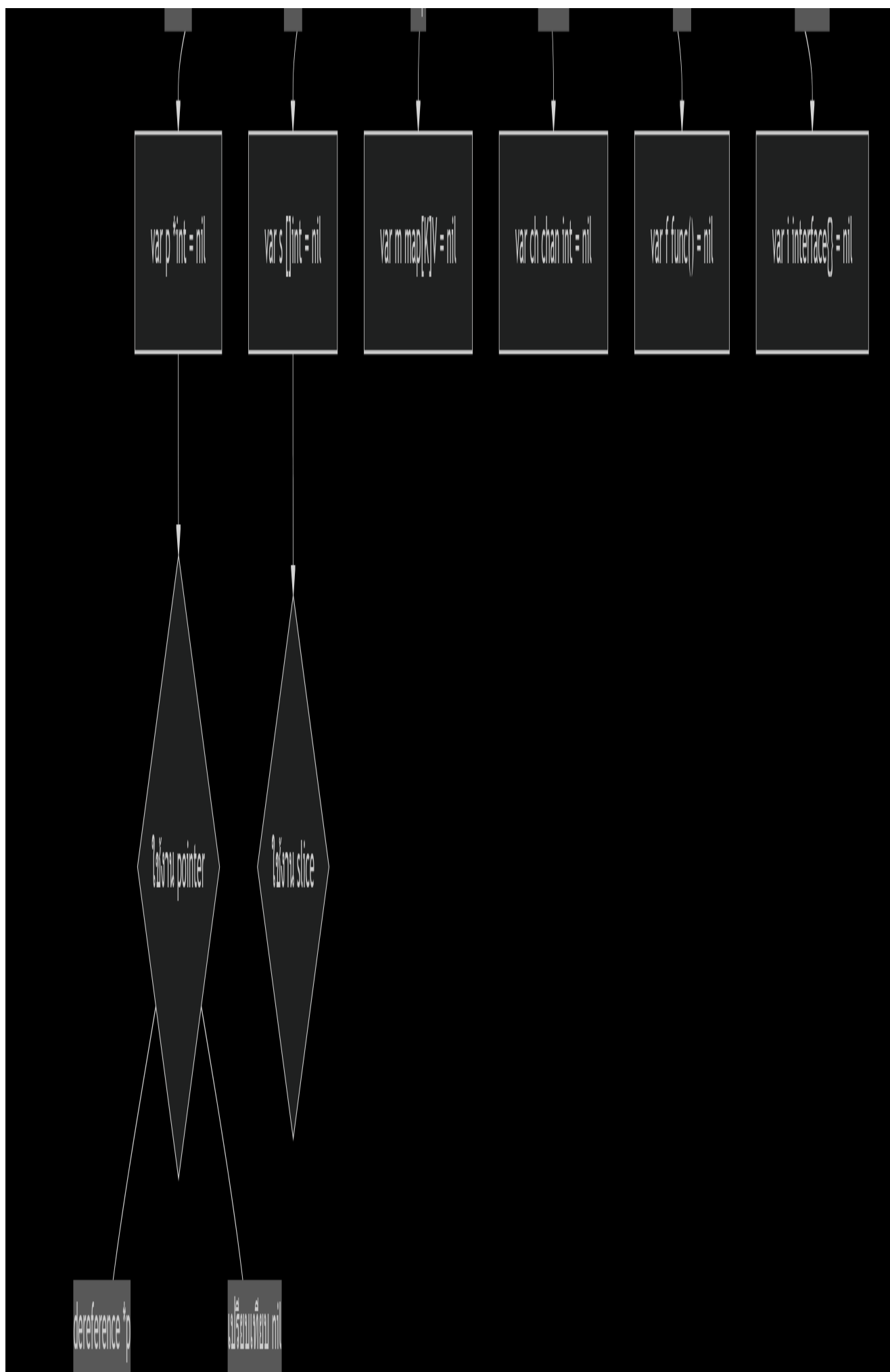
6. Nil Interface

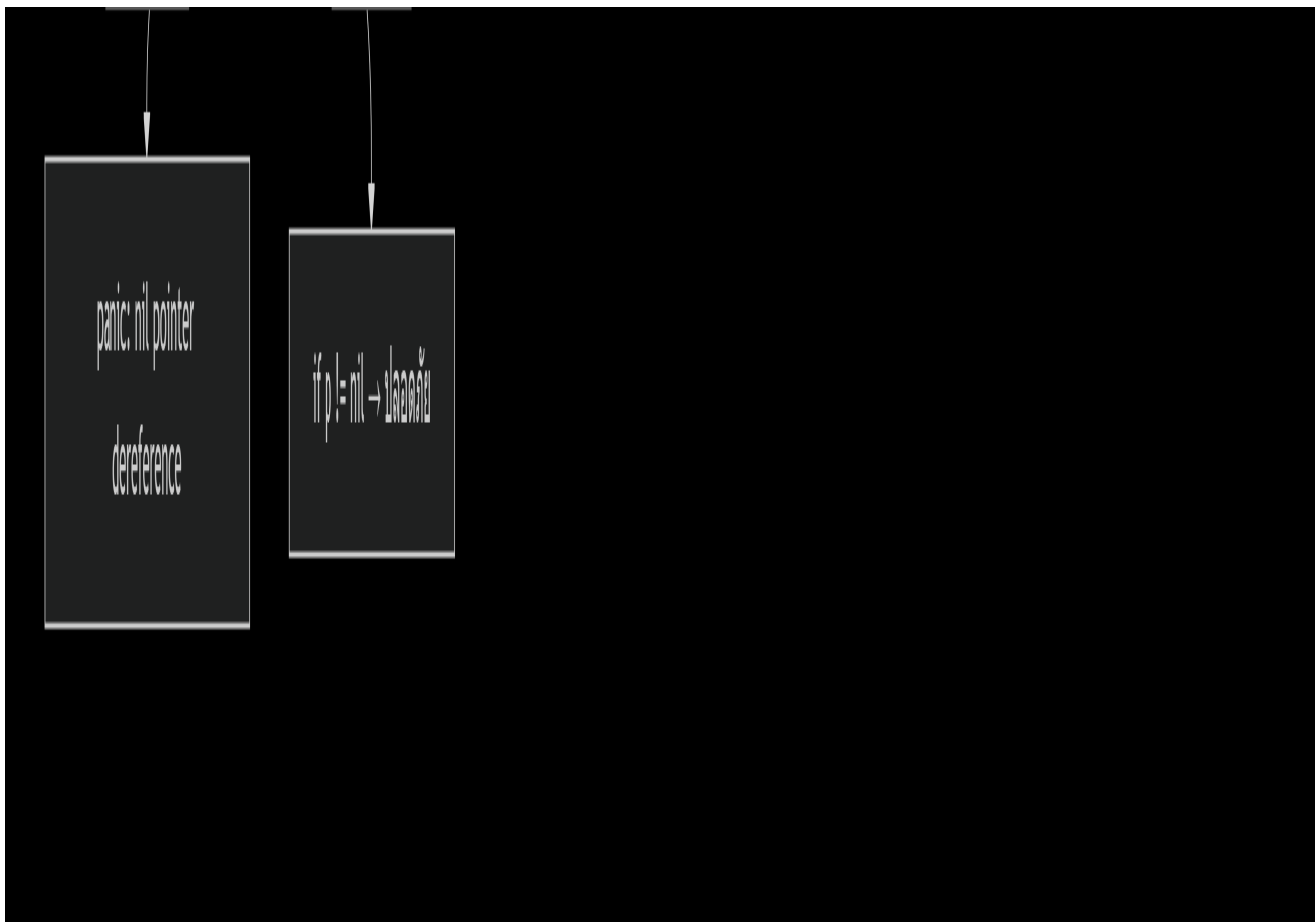
- **กรณีซับซ้อน:** interface ที่มี dynamic type = nil แต่ dynamic value ≠ nil จะไม่เท่ากับ nil โดยตรง
- **การใช้:** **if err == nil** หรือ **if v == nil** เฉพาะเมื่อทราบว่า interface ไม่ได้เก็บ concrete type ไว้

แผนภาพ Dataflow (Flowchart TB) – การทำงานกับ Nil ใน Go









ตัวอย่างโค้ดจริง พร้อมคำอธิบายแต่ละจุด

ตัวอย่างที่ 1: Nil Pointer และการป้องกัน panic

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func main() {
    var p *Person // p เป็น nil pointer
    fmt.Println(p == nil) // true

    // ✗ บรรทัดต่อไปนี้จะ panic เพราะ dereference nil pointer
    // fmt.Println(p.Name) // panic: runtime error: invalid memory address

    // ☑ ตรวจสอบ nil ก่อนใช้งาน
    if p != nil {
        fmt.Println(p.Name)
    } else {
        fmt.Println("p is nil, cannot access") // จะพิมพ์这句
    }
}
```

```

}

// ☑ สร้าง instance ก่อนใช้งาน
p = &Person{Name: "Alice", Age: 30}
fmt.Println(p.Name) // Alice (ปลอดภัย)
}

```

ตัวอย่างที่ 2: Nil Slice – สามารถ append ได้

```

package main

import "fmt"

func main() {
    var s []int // nil slice
    fmt.Println(s == nil) // true
    fmt.Println(len(s)) // 0
    fmt.Println(cap(s)) // 0

    // ☑ append กับ nil slice ได้ (Go สร้าง underlying array ให้)
    s = append(s, 10, 20, 30)
    fmt.Println(s) // [10 20 30]
    fmt.Println(s == nil) // false (ตอนนี้ไม่ใช่ nil แล้ว)

    // ⚠ การเข้าถึง index ที่ไม่มีจะ panic
    // fmt.Println(s[10]) // panic: index out of range

    // ☑ เปรียบเทียบ nil slice กับ empty slice ที่สร้างด้วย make
    var s2 []int = make([]int, 0)
    fmt.Println(s2 == nil) // false (empty slice ไม่ใช่ nil)

    // nil slice มักใช้เป็นค่าเริ่มต้นในฟังก์ชันที่ return slice
    // เช่น return nil เมื่อไม่มีข้อมูล
}

```

ตัวอย่างที่ 3: Nil Map – อ่านได้ แต่เขียนไม่ได้

```

package main

import "fmt"

func main() {
    var m map[string]int // nil map
    fmt.Println(m == nil) // true

    // ☑ อ่านค่าได้ (คืนค่า zero value ของ value type)
    value := m["key"]
    fmt.Println(value) // 0 (int zero value)
}

```

```

// ✗ เขียนค่าลง nil map ทำให้ panic
// m["key"] = 100           // panic: assignment to entry in nil map

// ☑ ต้อง initialize map ก่อนเขียน
m = make(map[string]int)
m["key"] = 100
fmt.Println(m["key"]) // 100

// ☑ หรือใช้ literal
m2 := map[string]int{}
m2["a"] = 1
}

```

ตัวอย่างที่ 4: Nil Channel – block ตลอดไป

```

package main

import "fmt"

func main() {
    var ch chan int           // nil channel

    // ✗ การส่งหรือรับจะ block ตลอดกาล (deadlock ถ้าไม่มี goroutine อื่น)
    // ch <- 10                // fatal error: all goroutines are asleep -
    deadlock!
    // <-ch                    // deadlock เช่นกัน

    // ☑ ควร initialize channel ก่อนใช้งาน
    ch = make(chan int)

    // ส่งข้อมูลใน goroutine เพื่อไม่ให้ block main
    go func() {
        ch <- 42
    }()

    fmt.Println(<-ch)         // 42

    // nil channel มีประโยชน์ในการปิดการทำงานของ select
    var noData chan int
    select {
    case v := <-noData:       // จะไม่ถูกเลือกเพราะ nil channel block
        fmt.Println(v)
    default:
        fmt.Println("no data") // จะไปที่ default
    }
}

```

ตัวอย่างที่ 5: Nil Function – ต้องตรวจสอบก่อนเรียก


```

package main

import "fmt"

type Processor func(data int) int

func main() {
    var proc Processor // nil function

    // ✗ การเรียก nil function จะ panic
    // result := proc(10) // panic: runtime error: invalid memory address

    // ☑ ตรวจสอบ nil ก่อนเรียก
    if proc != nil {
        result := proc(10)
        fmt.Println(result)
    } else {
        fmt.Println("processor is nil, skip") // จะพิมพ์这句
    }

    // ☑ กำหนด function ให้ตัวแปร
    proc = func(data int) int {
        return data * 2
    }

    fmt.Println(proc(10)) // 20
}

```

ตัวอย่างที่ 6: Nil Interface – กับดักที่พบบ่อย

```

package main

import "fmt"

type MyError struct {
    Msg string
}

func (e *MyError) Error() string {
    return e.Msg
}

func main() {
    var err error // interface nil (ทั้ง type และ value เป็น nil)
    fmt.Println(err == nil) // true

    var p *MyError = nil // pointer nil
    err = p // err มี dynamic type = *MyError, dynamic value =
    nil
}

```

```

fmt.Println(err == nil) // false! (เพราะ type ไม่ใช่ nil)

// ⚠️ กับดัก: การ return nil concrete type จากฟังก์ชันที่คืนค่า interface
// ทำให้ caller ตรวจสอบ err == nil ไม่ได้

// ✅ วิธีที่ถูกต้อง: return nil interface โดยตรง
funcReturnsNil := func() error {
    // var err *MyError = nil
    // return err // ❌ ทำให้ interface ไม่เป็น nil
    return nil // ✅ คืนค่า nil interface โดยตรง
}

err = funcReturnsNil()
fmt.Println(err == nil) // true
}

```

ตัวอย่างการใช้งานจริงในระบบ Web API

```

package main

import (
    "encoding/json"
    "fmt"
    "net/http"
)

type User struct {
    ID    int
    Name string
}

// Repository interface
type UserRepository interface {
    GetUser(id int) (*User, error)
}

type InMemoryRepo struct {
    users map[int]*User
}

func (r *InMemoryRepo) GetUser(id int) (*User, error) {
    // nil map จะ panic ถ้าไม่ได้ initialize
    if r.users == nil {
        return nil, fmt.Errorf("repository not initialized")
    }
    user, ok := r.users[id]
    if !ok {
        return nil, nil // return nil user, ไม่ใช่ error
    }
}

```

```

    return user, nil
}

func main() {
    var repo UserRepository    // nil interface
    // repo.GetUser(1)         // panic! nil interface method call

    // ต้อง initialize ก่อน
    repo = &InMemoryRepo{
        users: make(map[int]*User), // สำคัญ: อย่าให้เป็น nil map
    }

    // handler
    http.HandleFunc("/user", func(w http.ResponseWriter, r *http.Request) {
        user, err := repo.GetUser(1)
        if err != nil {
            http.Error(w, err.Error(), http.StatusInternalServerError)
            return
        }
        // nil user หมายถึงไม่พบ
        if user == nil {
            http.NotFound(w, r)
            return
        }
        json.NewEncoder(w).Encode(user)
    })

    http.ListenAndServe(":8080", nil)
}

```

🔗 สรุปการใช้งาน Nil ในแต่ละประเภท

ประเภท	nil เริ่มต้น	อ่านค่า	เขียน/ เปลี่ยนค่า	append	len/cap	เรียกใช้	หมายเหตุ
Pointer	nil	✗ panic (dereference)	☑ กำหนดค่า ใหม่ได้	N/A	N/A	N/A	ต้องตรวจสอบก่อน dereference
Slice	nil	✗ panic (index)	☑ append ใช้ได้	☑ ใช้ได้	☑ 0	N/A	nil slice เทียบเท่า empty slice ในแง่ append
Map	nil	☑ zero value	✗ panic	N/A	☑ 0	N/A	ต้อง make ก่อน เขียน
Channel	nil	✗ block ตลอดไป	✗ block ตลอดไป	N/A	N/A	close: panic	ใช้ nil channel เพื่อ ปิด select case

ประเภท	nil เริ่ม ต้น	อ่านค่า	เขียน/ เปลี่ยนค่า	append	len/cap	เรียกใช้	หมายเหตุ
Function	nil	N/A	N/A	N/A	N/A	✗ panic	ต้องตรวจสอบก่อนเรียก
Interface	nil	N/A	N/A	N/A	N/A	✗ panic (method call)	ระวัง interface ที่มี concrete nil

🛡️ หลักการป้องกัน panic จาก nil

1. ตรวจสอบ **nil** ก่อนเสมอ สำหรับ pointer, function, interface
2. ใช้ **make** หรือ **literal** สำหรับ map และ channel ก่อนเขียน/ส่ง
3. **nil slice** ปลอดภัยสำหรับ **append**, **len**, **cap** (ใช้เป็นค่าเริ่มต้นได้)
4. **nil map** อ่านได้ แต่ห้ามเขียน
5. **nil channel** ใช้ใน select เพื่อ disable case ได้
6. ระวัง **nil interface** ที่ซ่อน **concrete nil type** – ควร return nil โดยตรง

```
// เหมเพลตฟังก์ชันที่คืนค่า interface อย่างปลอดภัย
func SafeGet() error {
    // ✗ ไม่ดี
    // var err *MyError = nil
    // return err

    // ☑ ดี
    return nil
}
```

ตัวอย่างการใช้งาน nil ใน concurrent pattern (เช่น nil channel ใน select)

🏗️ การใช้งาน nil ใน Concurrent Pattern (nil channel ใน select)

🎯 แนวคิดหลัก

ในภาษา Go **channel** ที่เป็น **nil** มีพฤติกรรมพิเศษเมื่อใช้ใน **select**:

- การส่งค่า (**ch <- v**) ไปยัง nil channel → **block** ตลอดไป
- การรับค่า (**<-ch**) จาก nil channel → **block** ตลอดไป
- กรณีนี้ **select** จะ **ไม่เลือก case นั้น** (เพราะมัน block ตลอด) → มีผลเหมือน disable case ชั่วคราว

คุณสมบัตินี้มีประโยชน์มากในการ **เปิด/ปิดการทำงานของ case แบบไดนามิก** โดยไม่ต้องใช้ flag หรือ restructuring โค้ด

🔍 nil channel ใน select ทำงานอย่างไร?

```
flowchart TB
    subgraph SelectBlock["select { ... }"]
        A["case 1: ch1 ที่เป็น nil"]
        B["case 2: ch2 ที่เป็น non-nil"]
        C["default: optional"]
    end
    end

    A -->|block ตลอด| Skip1[ถูกข้ามไป]
    B -->|มีข้อมูลหรือพร้อม| Exec[ทำงาน case นั้น]
    C -->|ถ้าไม่มี case อื่นพร้อม| ExecDefault[ทำงาน default]

    Skip1 --> Next[พิจารณา case ถัดไป]
```

กฎสำคัญ:

- **select** จะเลือก case ที่ **ไม่ block** แบบสุ่ม (ถ้าหลาย case พร้อม)
- ถ้า nil channel ถูกใช้ใน case → case นั้นจะ **block เสมอ** → จึงไม่มีทางถูกเลือก
- ถ้าทุก case block (รวมถึง nil channel) และไม่มี default → **select** จะ block ตลอด (deadlock ถ้าไม่มี goroutine อื่น)

💻 ตัวอย่างการใช้งานจริง

ตัวอย่างที่ 1: เปิด/ปิดการรับข้อมูลจาก channel แบบไดนามิก

```
package main

import (
    "fmt"
    "time"
)

func main() {
    // สร้าง channel สำหรับรับงาน
    workCh := make(chan string)

    // channel สำหรับควบคุมการเปิด/ปิด (toggle)
    toggleCh := make(chan bool)

    // nil channel เริ่มต้น (ปิดอยู่)
    var activeWorkCh chan string = nil

    go func() {
        for {
            select {
                case work, ok := <-activeWorkCh:
```

```

        if !ok {
            fmt.Println("work channel closed, exiting")
            return
        }
        fmt.Printf("✅ รับงาน: %s\n", work)

        case toggle := <-toggleCh:
            if toggle {
                activeWorkCh = workCh // เปิด: ใช้ channel จริง
                fmt.Println("🟢 เปิดรับงาน")
            } else {
                activeWorkCh = nil // ปิด: ใช้ nil channel (ไม่รับข้อมูล)
                fmt.Println("🟢 ปิดรับงาน")
            }
        }
    }
}()

// ส่งงานไปเรื่อยๆ
go func() {
    i := 1
    for {
        workCh <- fmt.Sprintf("งานที่ %d", i)
        i++
        time.Sleep(500 * time.Millisecond)
    }
}()

// ตัวอย่างการ toggle
toggleCh <- true // เปิดรับ
time.Sleep(2 * time.Second)
toggleCh <- false // ปิดรับ (งานที่ส่งมาจะไม่ถูกประมวลผล)
time.Sleep(2 * time.Second)
toggleCh <- true // เปิดอีกครั้ง
time.Sleep(2 * time.Second)

close(workCh)
close(toggleCh)
}

```

คำอธิบาย:

- `activeWorkCh` เป็นตัวแปรที่เปลี่ยนระหว่าง `workCh` (channel จริง) และ `nil`
- เมื่อ `activeWorkCh = nil` → `case <-activeWorkCh` จะ block ตลอด → ไม่ถูกเลือก
- เมื่อ `activeWorkCh = workCh` → `case` พร้อมรับข้อมูลทันที
- วิธีนี้ **ไม่ต้องใช้ flag** และ **ไม่ต้อง restart loop** แค่เปลี่ยนค่าตัวแปร channel

ตัวอย่างที่ 2: การทำ rate limiting + timeout แบบยืดหยุ่น

```

package main

import (
    "fmt"
    "time"
)

func main() {
    requests := make(chan int, 5)

    // เตรียม request
    for i := 1; i <= 10; i++ {
        requests <- i
    }
    close(requests)

    // rate limiter: ทุก 1 วินาที
    ticker := time.NewTicker(1 * time.Second)
    defer ticker.Stop()

    // ชั่วโมงเร่งด่วน: เปิดใช้งาน rate limit ปกติ
    var rateLimitChan <-chan time.Time = ticker.C

    // channel สำหรับสลับโหมด (rush hour = ไม่จำกัด rate)
    rushHourCh := make(chan bool)

    go func() {
        time.Sleep(3 * time.Second)
        rushHourCh <- true // เปิดโหมด rush hour (ไม่จำกัด)
        time.Sleep(3 * time.Second)
        rushHourCh <- false // กลับมา rate limit ปกติ
    }()

    for req := range requests {
        select {
        case <-rateLimitChan:
            fmt.Printf("📦 ประมวลผล request %d (rate limited)\n", req)
        case <-rushHourCh:
            // เมื่อรับ rush hour signal ให้เปลี่ยน rateLimitChan เป็น nil
            // (case นั้นจะถูก disable)
            fmt.Println("🚧 เข้าโหมด rush hour! ไม่จำกัด rate")
            rateLimitChan = nil
            // ทำงานต่อโดยไม่รอ ticker
            fmt.Printf("📦 ประมวลผล request %d ทันที\n", req)
        default:
            // ถ้าไม่มี rate limit (rateLimitChan == nil) และไม่มี signal
            fmt.Printf("⚡ ประมวลผล request %d ทันที (ไม่มี rate limit)\n", req)
        }
    }
}

```

อธิบาย:

- `rateLimitChan` เปลี่ยนระหว่าง `ticker.C` (มีค่า) และ `nil`
- เมื่อเป็น `nil` → case `<-rateLimitChan` จะถูกข้าม → ไปที่ default ทันที (ประมวลผลทันที)
- การสลับทำได้ง่ายโดยไม่ต้องมี if-else ช่างนอก

ในการเขียนโปรแกรมด้วย **Golang** (โดยเฉพาะฝั่ง Backend/Microservices) สองคำนี้คือ "หัวใจ" ของการทำให้ระบบมีความเสถียร (Resilience) และไม่ล่มง่าย ๆ เมื่อเจอกับ Traffic มหาศาลครับ

1. Rate Limiting (การจำกัดอัตราการเรียกใช้)

Rate Limiting คือการควบคุมว่า "ใน 1 วินาที/นาที ยอมให้เรียกใช้งานได้กี่ครั้ง" เพื่อป้องกันไม่ให้ Server ของเราทำงานหนักเกินไป หรือป้องกันการโดน Spam (เช่น การยิง Brute-force password)

วิธีการทำงาน (Concept)

วิธีที่นิยมที่สุดคือ **Token Bucket**: เปรียบเสมือนเรามีถังใส่เหรียญ (Token) ถ้าจะใช้งานต้องหยิบเหรียญออกไป 1 เหรียญ ถ้าเหรียญหมดก็ต้องรอให้ระบบเติมเหรียญใหม่ตามเวลาที่กำหนด

ตัวอย่างการเขียนใน Go

ใน Go เรามักจะใช้ Package มาตรฐานอย่าง golang.org/x/time/rate

```
import (
    "context"
    "fmt"
    "golang.org/x/time/rate"
    "time"
)

func main() {
    // ยอมให้ทำงาน 2 ครั้งต่อวินาที (Limit) และเก็บสะสมได้สูงสุด 5 ครั้ง (Burst)
    limiter := rate.NewLimiter(2, 5)
    ctx := context.Background()

    for i := 0; i < 10; i++ {
        // Wait จะรอจนกว่าจะมี Token วางให้ใช้งาน
        err := limiter.Wait(ctx)
        if err != nil {
            fmt.Println("Error:", err)
            return
        }
        fmt.Println("ทำขั้นตอนที่:", i, "เวลา:", time.Now().Format("15:04:05"))
    }
}
```

2. Timeout (การกำหนดเวลาหมดเขต)

Timeout คือการกำหนดว่า "งานนี้ห้ามใช้เวลานานเกินเท่านั้น" ถ้าทำไม่เสร็จตามกำหนด ให้ยกเลิกทันที (Cancel) เพื่อคืนทรัพยากร (Memory/CPU) ให้งานอื่นมาทำต่อ ไม่ให้ระบบยืนค้างจนเต็ม

วิธีการทำงาน (Concept)

ใน Go เราจะใช้ **context** เป็นตัวจัดการเรื่อง Timeout เป็นหลักครับ

ตัวอย่างการเขียนใน Go

```
import (  
    "context"  
    "fmt"  
    "time"  
)  
  
func main() {  
    // ตั้งเวลา Timeout ไว้ที่ 2 วินาที  
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)  
    defer cancel() // คืนทรัพยากรเมื่อจบฟังก์ชัน  
  
    select {  
    case <-time.After(3 * time.Second):  
        // จำลองงานที่ใช้เวลา 3 วินาที (ซึ่งนานกว่า Timeout)  
        fmt.Println("งานเสร็จแล้ว!")  
    case <-ctx.Done():  
        // ถ้าหมดเวลา 2 วินาทีก่อน งานจะมาถึงที่เคสนี้  
        fmt.Println("ขอโทษที่ เข้าไปนะ! (Timeout):", ctx.Err())  
    }  
}
```

สรุปความต่างและการใช้งานคู่กัน

หัวข้อ	Rate Limiting	Timeout
เป้าหมาย	ป้องกัน Server โดนถล่มจาก "ภายนอก"	ป้องกันงานค้างจนกินทรัพยากร "ภายใน"
ศัพท์เวิร์ด	<code>rate.Limiter</code> , Requests per second	<code>context.WithTimeout</code> , <code>http.Client.Timeout</code>
ตัวอย่าง	ยอมให้ API นี้โดนเรียกได้แค่ 100 ครั้ง/นาที	ถ้า Database ไม่ตอบกลับใน 5 วินาที ให้ตัดการเชื่อมต่อ

Pro Tip: ในระบบที่ Production-ready จริงๆ คุณควรใช้ทั้งสองอย่างร่วมกันครับ! Rate Limiting จะคัดกรองแขกที่มากเกินไป ส่วน Timeout จะคอยเตะแขกที่อยู่นานเกินไปออกไปนั่นเอง

ตัวอย่างที่ 3: การทำ worker pool ที่สามารถปิด worker แบบเลือกได้

```
package main  
  
import (  
    "context"
```

```

    "fmt"
    "sync"
    "time"
)

type Worker struct {
    id      int
    taskCh  chan int
    stopCh  chan struct{}
    disabled bool
}

func (w *Worker) Start(wg *sync.WaitGroup) {
    wg.Add(1)
    go func() {
        defer wg.Done()

        // ตัวแปร channel ที่อาจเป็น nil (disabled)
        var activeTaskCh chan int = w.taskCh

        for {
            select {
            case task, ok := <-activeTaskCh:
                if !ok {
                    fmt.Printf("Worker %d: task channel closed\n", w.id)
                    return
                }
                fmt.Printf("Worker %d: กำลังทำงาน %d\n", w.id, task)
                time.Sleep(500 * time.Millisecond)

            case <-w.stopCh:
                // ปิด worker แบบถาวร
                fmt.Printf("Worker %d: หยุดทำงาน\n", w.id)
                return

            case <-time.After(2 * time.Second):
                // ถ้า activeTaskCh == nil จะมาที่นี่ทุก 2 วินาที
                if activeTaskCh == nil {
                    fmt.Printf("Worker %d: ถูก disable อยู่ กำลังรอ enable...\n",
w.id)

                } else {
                    fmt.Printf("Worker %d: idle\n", w.id)
                }
            }
        }
    }()
}

func (w *Worker) Enable() {
    w.disabled = false
    // เปลี่ยนเป็นใช้ task channel จริง
    // จริงๆต้องมีวิธีเข้าถึงตัวแปร activeTaskCh ข้างใน goroutine
    // ซึ่งต้องใช้ channel ส่ง หรือ redesign

```

```

// วิธีง่าย: ส่ง signal ไปอีก channel เพื่อบอกให้เปลี่ยน activeTaskCh
}

func main() {
    taskCh := make(chan int, 10)
    stopCh := make(chan struct{})

    // สร้าง worker
    go func() {
        // ตัวแปร active channel สามารถเปลี่ยนค่า nil/non-nil ได้
        var activeCh chan int = taskCh // เริ่มต้น active

        for {
            select {
            case task, ok := <-activeCh:
                if !ok {
                    return
                }
                fmt.Printf("Worker: ทำงาน %d\n", task)

            case <-stopCh:
                return

            case <-time.After(3 * time.Second):
                // toggle activeCh ทุก 3 วินาที (disable/enable)
                if activeCh == nil {
                    activeCh = taskCh
                    fmt.Println("🌀 Enable worker")
                } else {
                    activeCh = nil
                    fmt.Println("🌀 Disable worker (ไม่รับงานใหม่)")
                }
            }
        }
    }()

    // ส่งงาน
    go func() {
        for i := 1; i <= 20; i++ {
            taskCh <- i
            time.Sleep(500 * time.Millisecond)
        }
        close(taskCh)
    }()

    time.Sleep(15 * time.Second)
    close(stopCh)
}

```

หมายเหตุ: ตัวอย่างนี้แสดงการ toggle โดยใช้ timer จริงๆ worker จะรับงานเฉพาะเมื่อ activeCh ไม่เป็น nil

ตัวอย่างที่ 4: ระบบ Fan-in ที่สามารถปิด source ชั่วครา

```
package main

import (
    "fmt"
    "time"
)

func source(name string, interval time.Duration, toggleCh <-chan bool) <-chan string {
    out := make(chan string)
    go func() {
        var activeCh chan string = out
        for {
            select {
            case <-time.After(interval):
                if activeCh != nil {
                    activeCh <- fmt.Sprintf("%s: data", name)
                }
            case toggle := <-toggleCh:
                if toggle {
                    activeCh = out // เปิด
                    fmt.Printf("[%s] เปิดการส่งข้อมูล\n", name)
                } else {
                    activeCh = nil // ปิด
                    fmt.Printf("[%s] ปิดการส่งข้อมูล\n", name)
                }
            }
        }
    }()
    return out
}

func fanIn(channels ...<-chan string) <-chan string {
    out := make(chan string)
    for _, ch := range channels {
        go func(c <-chan string) {
            for v := range c {
                out <- v
            }
        }(ch)
    }
    return out
}

func main() {
    toggleA := make(chan bool)
    toggleB := make(chan bool)

    srcA := source("A", 1*time.Second, toggleA)
```

```

srcB := source("B", 1*time.Second, toggleB)

merged := fanIn(srcA, srcB)

// สลับเปิด-ปิด source A และ B
go func() {
    time.Sleep(3 * time.Second)
    toggleA <- false // ปิด A
    time.Sleep(3 * time.Second)
    toggleA <- true  // เปิด A
    time.Sleep(2 * time.Second)
    toggleB <- false // ปิด B
    time.Sleep(3 * time.Second)
    toggleB <- true  // เปิด B
    time.Sleep(5 * time.Second)
    close(toggleA)
    close(toggleB)
}()

for msg := range merged {
    fmt.Println("ได้รับ:", msg)
}
}

```

แผนภาพ Dataflow – Nil Channel ใน Select

```

flowchart TB
    subgraph Runtime["การทำงานขณะรัน"]
        Start[เริ่ม select] --> CheckCases{มี case ไหน<br>ไม่ block?}

        CheckCases -->|มีหลาย case| Random[เลือกแบบสุ่ม]
        CheckCases -->|มี case เดียว| Execute[ทำงาน case นั้น]
        CheckCases -->|ไม่มีเลย + มี default| RunDefault[ทำงาน default]
        CheckCases -->|ไม่มีเลย + ไม่มี default| Block[block ตลอดไป]
    end

    subgraph NilChannelCase["กรณี case ที่ใช้ nil channel"]
        N1["case <-nilCh (receive)"]
        N2["case nilCh <- v (send)"]

        N1 --> AlwaysBlock1[block เสมอ]
        N2 --> AlwaysBlock2[block เสมอ]

        AlwaysBlock1 --> Skip[select จะข้าม case นี้<br>เหมือนไม่มี case นี้]
        AlwaysBlock2 --> Skip
    end

    subgraph Benefit["ประโยชน์"]
        B1[เปิด/ปิด case แบบไดนามิก]
        B2[ลด complexity ของโค้ด]
    end

```

```

B3[ไม่ต้องใช้ flag + lock]
B4[ทำงานร่วมกับ select ได้ดี]
end

NilChannelCase --> Benefit
CheckCases -.-> NilChannelCase

```

☑ สรุปแนวทางการใช้ nil channel ใน concurrent pattern

รูปแบบ	การใช้งาน	ข้อดี
Toggle receive case	เปลี่ยนตัวแปร channel ระหว่าง nil ↔ real channel	เปิด/ปิดการรับข้อมูลแบบเรียลไทม์
Toggle send case	เหมือนกันแต่กับ send	ป้องกันการส่งข้อมูลเมื่อไม่ต้องการ
Disable case ชั่วคราว	ตั้ง channel = nil แล้วคืนค่าทีหลัง	ไม่ต้องลบ case ออกจาก select
Rate limiting แบบปรับได้	เปลี่ยน ticker channel เป็น nil ตอนไม่ต้องการ limit	โศด
Graceful shutdown บาง component	เปลี่ยน channel เป็น nil แทนการ close	ปลอดภัยกว่า เพราะ close แล้วใช้ต่อไม่ได้

☹ ข้อควรระวัง

1. **nil channel** ที่อยู่ใน **select** โดยไม่มี **case** อื่น → จะ deadlock
2. การใช้ **nil channel** ร่วมกับ **for loop** → ต้องมีวิธีออกจาก loop (ใช้ quit channel หรือ break)
3. อย่ยาสับสนระหว่าง **nil channel** กับ **closed channel**:
 - closed channel: receive ได้ zero value ทันที, send panic
 - nil channel: receive/send block ตลอดไป

```

// เปรียบเทียบ
var nilCh chan int          // nil channel
closedCh := make(chan int)
close(closedCh)

select {
case <-nilCh:               // block ตลอด
case <-closedCh:           // ได้ zero value ทันที (0)
}

```

Deadlock หรือ **การติดตาย** คือสถานการณ์ที่กระบวนการ (Process) หรือโปรแกรมอย่างน้อยสองตัว ต่างก็หยุดทำงานและไม่สามารถดำเนินการต่อได้ เนื่องจากแต่ละฝ่ายต่างรอให้อีกฝ่ายปล่อยทรัพยากรที่ตนเองต้องการ**[reference:0][reference:1]** ส่งผลให้ระบบหยุดชะงักและไม่สามารถทำงานต่อไปได้

🔍 Deadlock เกิดจากอะไร

Deadlock จะเกิดขึ้นได้ก็ต่อเมื่อมี **4 เงื่อนไข** ต่อไปนี้เกิดขึ้นพร้อมกัน**[reference:2]**:

1. **Mutual Exclusion** – ทรัพยากรที่ใช้ไม่สามารถใช้ร่วมกันได้ มีเพียงกระบวนการเดียวเท่านั้นที่สามารถใช้ทรัพยากรนั้นได้ในแต่ละช่วงเวลา**[reference:3]**
2. **Hold and Wait** – กระบวนการหนึ่งครอบครองทรัพยากรบางตัวไว้และกำลังรอทรัพยากรอื่นที่ถูกครอบครองโดยกระบวนการอื่น**[reference:4]**
3. **No Preemption** – ทรัพยากรที่ถูกครอบครองอยู่จะถูกปล่อยก็ต่อเมื่อกระบวนการที่ครอบครองใช้เสร็จแล้วเท่านั้น ไม่สามารถแย่งชิงได้**[reference:5]**
4. **Circular Wait** – กระบวนการหลายตัวต่างรอทรัพยากรจากกันและกันเป็นลูกโซ่แบบวงกลม เช่น P1 รอ P2, P2 รอ P3, P3 รอ P1**[reference:6]**

หากทั้ง 4 เงื่อนไขนี้เกิดขึ้นพร้อมกัน ก็จะทำให้เกิด deadlock ขึ้นในระบบ

ตัวอย่าง Deadlock ในชีวิตจริง

ตัวอย่างที่ 1: ระบบปฏิบัติการ

สมมติว่ามี 2 โปรเซส:

- โปรเซส A ครอบครองเครื่องพิมพ์ และกำลังรอแสกนเนอร์
 - โปรเซส B ครอบครองแสกนเนอร์ และกำลังรอเครื่องพิมพ์
- ทั้งคู่ต่างรอซึ่งกันและกัน ส่งผลให้ไม่สามารถทำงานต่อไปได้

ตัวอย่างที่ 2: การจราจร

รถ 4 คันมาถึงทางแยกพร้อมกันและต่างปิดกันกันจนไม่สามารถเคลื่อนที่ได้**[reference:7]**

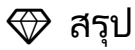
การจัดการ Deadlock

แนวทางจัดการ deadlock มี 3 วิธีหลัก:

แนวทาง	หลักการ	ข้อดี	ข้อเสีย
Prevention (การป้องกัน)	ป้องกันไม่ให้เงื่อนไขใดเงื่อนไขหนึ่งใน 4 ข้อเกิดขึ้น**[reference:8]**	มั่นใจได้ว่า deadlock จะไม่เกิด	อาจทำให้ประสิทธิภาพของระบบลดลง
Avoidance (การหลีกเลี่ยง)	ระบบจะตรวจสอบก่อนจัดสรรทรัพยากรว่าทำให้เกิด deadlock หรือไม่	หลีกเลี่ยง deadlock ได้โดยไม่ต้องป้องกันทุกเงื่อนไข	ต้องการข้อมูลล่วงหน้าว่าโปรเซสจะใช้ทรัพยากรอะไรบ้าง
Detection & Recovery (การตรวจจับและกู้คืน)	ระบบยอมให้ deadlock เกิดขึ้น แล้วค่อยตรวจสอบและแก้ไขภายหลัง	ไม่ลดประสิทธิภาพของระบบระหว่างการทำงานปกติ	อาจทำให้ระบบต้องเสียเวลา กู้คืนและสูญเสียข้อมูลบางส่วน

วิธีการกู้คืนเมื่อเกิด deadlock อาจทำได้โดย:

- **Terminate process** – ยุติกระบวนการที่ติด deadlock ทิ้ง**[reference:9]**
- **Rollback** – ย้อนกลับกระบวนการไปยังสถานะที่ปลอดภัย**[reference:10]**



สรุป

Deadlock (การติดตาย) คือภาวะที่ระบบไม่สามารถทำงานต่อได้เนื่องจากกระบวนการต่างๆ ต่างรอทรัพยากรซึ่งกันและกัน โดยเกิดจาก 4 เงื่อนไขพร้อมกัน การจัดการ deadlock ทำได้ทั้งการป้องกัน การหลีกเลี่ยง และการตรวจจับพร้อมกัน เรื่อง Deadlock ใน Go เป็นอีกเรื่องที่开发者หลายคน (รวมถึงตัวผมเองด้วย) เคยปวดหัวไม่น้อยครับ แต่ Go ก็มีเครื่องมือและแนวทางช่วยจัดการได้ดีพอสมควร

❶ 1. Deadlock จาก Mutex (การ Lock ที่สลับซับซ้อน)

สาเหตุหลักเกิดจากการถือครอง Mutex พร้อมกันหลายตัว โดยไม่ได้กำหนดลำดับการ Lock ที่แน่นอน[reference:0] [reference:1] หรือการ Lock ซ้ำ (Re-entrant Lock) โดยไม่ได้ Unlock ซึ่ง Mutex ของ Go (`sync.Mutex`) ไม่รองรับการ Lock ซ้ำโดยตรง[reference:2][reference:3]

- ตัวอย่าง: `Transfer` เงินระหว่างสองบัญชี (`wallet`) ที่ต้อง Lock ทั้งผู้โอน (`from`) และผู้รับ (`to`) ผิดลำดับ[reference:4]

```
package main

import (
    "fmt"
    "sync"
    "time"
)

type Wallet struct {
    ID      int
    mu      sync.Mutex
    Balance int
}

// BUG! Transfer นี้มีโอกาส Deadlock สูง เพราะไม่ได้กำหนดลำดับการ Lock
func TransferBad(from, to *Wallet, amount int) {
    // Goroutine 1: Transfer(A, B) จะ Lock A -> B
    // Goroutine 2: Transfer(B, A) จะ Lock B -> A
    from.mu.Lock()
    fmt.Printf("🔒 Lock %d\n", from.ID)
    // จำลองการทำงานระหว่าง Lock ทั้งสองตัว เพิ่มโอกาส Deadlock
    time.Sleep(1 * time.Millisecond)
    to.mu.Lock()
    fmt.Printf("🔒 Lock %d\n", to.ID)

    // Critical Section
    from.Balance -= amount
    to.Balance += amount

    to.mu.Unlock()
    from.mu.Unlock()
    fmt.Printf("✅ Transfer %d -> %d success\n", from.ID, to.ID)
}

func main() {
```



```
// สร้าง Wallet สองใบ
walletA := &Wallet{ID: 1, Balance: 100}
walletB := &Wallet{ID: 2, Balance: 100}

// พยายามโอนเงินพร้อมกันคนละทิศทาง
go TransferBad(walletA, walletB, 10) // A -> B
go TransferBad(walletB, walletA, 20) // B -> A

// รอให้ Goroutine ทั้งคู่ทำงานเสร็จ
time.Sleep(2 * time.Second)
fmt.Printf("Final Balance: A=%d, B=%d\n", walletA.Balance, walletB.Balance)
}
```

ผลลัพธ์ที่ได้ (อาจจะไม่เหมือนกันทุกครั้ง แต่จะติด Deadlock สักครั้งหนึ่ง):

```
🔒 Lock 1
🔒 Lock 2
🔒 Lock 2
fatal error: all goroutines are asleep - deadlock!
```

สาเหตุ: Goroutine แรก Lock walletA ไว้ รอ walletB, Goroutine ที่สอง Lock walletB ไว้ รอ walletA กลายเป็นวงจรที่รอกันและกันตาย[reference:5]

แนวทางแก้ไข: กำหนดลำดับการ **Lock** ให้แน่นอน (Consistent Lock Ordering) เช่น ใช้ ID ของ Wallet เป็นตัวเปรียบเทียบ[reference:6]

```
func TransferGood(from, to *Wallet, amount int) {
    // สร้าง slice ของ Wallet
    wallets := []*Wallet{from, to}
    // เรียงลำดับตาม ID ก่อนเสมอ
    for i := 0; i < len(wallets)-1; i++ {
        for j := i + 1; j < len(wallets); j++ {
            if wallets[i].ID > wallets[j].ID {
                wallets[i], wallets[j] = wallets[j], wallets[i]
            }
        }
    }
    // Lock ตามลำดับที่เรียงแล้ว (ID น้อย -> มาก)
    for _, w := range wallets {
        w.mu.Lock()
        defer w.mu.Unlock()
    }
    from.Balance -= amount
    to.Balance += amount
}
```

⚠ ปัญหาที่พบบ่อยอื่นๆ ของ Mutex

- **Deadlock ข้าม Goroutine:** การ Lock Mutex ตัวหนึ่งไว้ แล้ว Spawn Goroutine ใหม่ไป Lock อีกตัว[reference:7]
- **Duplicate Locking:** การ Lock Mutex ตัวเดิมซ้ำใน Goroutine เดียวกัน โดยไม่ได้ Unlock[reference:8]
- **RW-Mutex Starvation:** การที่ RLock ข้อนั้นจน Writer (Lock) รอแบบไม่มีที่สิ้นสุด[reference:9]

📖 2. Deadlock จาก Channel

เกิดขึ้นเมื่อ Goroutine ถูกบล็อกค้างไว้อย่างถาวร ขณะรอการส่ง (`ch <-`) หรือรับ (`<-ch`) ข้อมูลจาก Channel ที่ไม่มีคู่สื่อสาร[reference:10]

- **Unbuffered Channel Blocking** (Blocking due to Unbuffered Channels): การส่งข้อมูลเข้า Unbuffered Channel จะถูกบล็อกจนกว่าจะมี Goroutine อื่นเข้ามารับ[reference:11]
- **Nested Channel Dependency:** Goroutine หนึ่งกำลังรอรับจาก Channel A แต่ Channel A ก็รอข้อมูลจาก Channel B ซึ่ง B ก็รอ A อีกที
- **Missing Channel Close:** Goroutine กำลังวนลูปรอรับค่าจาก Channel (`for v := range ch`) แต่ Channel นั้นไม่ถูกปิด ทำให้รอรับค่าไปตลอด[reference:12]
- **Goroutine Leak:** Goroutine ส่งค่าไปยัง Channel ที่ไม่มีใครรับอีกแล้ว

แนวทางแก้ไข:

1. ใช้ **Buffered Channel** เพื่อไม่ให้ Sender ติดบล็อกถ้า Buffer ยังไม่เต็ม[reference:13][reference:14]
2. ใช้ **select** เพื่อจัดการ Channel พร้อมกัน และเพิ่ม **default** Branch เพื่อให้ไม่ต้องรอ[reference:15]
3. ปิด **Channel** เมื่อไม่ใช้แล้ว เพื่อให้ Receiver หยุดรอ[reference:16][reference:17]
4. ใช้ **context** สำหรับ Timeout หรือ Cancel[reference:18]

🔍 3. การตรวจสอบ Deadlock

Go มีเครื่องมือทั้งในตัวและของชุมชนที่ช่วยตรวจจับและป้องกัน Deadlock ได้อย่างมีประสิทธิภาพ

🔧 Built-in Runtime Detection

- **fatal error: all goroutines are asleep - deadlock!**: ทำงานเมื่อ **Goroutine** ทุกตัว ถูกบล็อก[reference:19] แต่ถ้ามี Goroutine ใดทำงานอยู่ (แม้ไม่เกี่ยวข้อง) ระบบจะไม่แจ้งเตือน[reference:20]

🔗 Third-party Tools & Libraries

- **go-deadlock:** ใช้แทน `sync.Mutex` ได้ทันที ตรวจจับ Inconsistent Lock Ordering และ Potential Deadlock[reference:21][reference:22]
- **-race flag:** `go run -race main.go` ใช้ตรวจจับ Race Condition ซึ่งเป็นต้นตอของ Deadlock ได้บางส่วน[reference:23]
- **Static Analysis Tools:**
 - **go vet:** วิเคราะห์โค้ดเบื้องต้น หา Potential Deadlock[reference:24]
 - **dingo-hunter:** Static Analyzer สำหรับ Deadlock โดยเฉพาะ[reference:25]
- **Runtime Detection** (Go 1.26+): Uber พัฒนา Dynamic Deadlock Detection บน GC[reference:26] แต่ ยัง **ไม่ Stable** และ **Support เฉพาะ Mutex** เดียว เท่านั้น

💡 4. Best Practices

- **Minimize Lock Scope:** ถือ Mutex ให้น้อยที่สุด และใช้ `defer mu.Unlock()` เสมอ[reference:27]

- **Use `sync.Map`:** สำหรับ Cache หรือข้อมูลที่อ่านเยอะ เขียนน้อย[reference:28]
- **ใช้ `select`:** หลีกเลี่ยงการ Block โดยใช้ `select` และ `default` Branch[reference:29]
- **Avoid Mixed Primitives:** ไม่ผสม Mutex และ Channel จนเกินไป เพราะสร้าง Deadlock ที่ตีบตันยาก[reference:30]

เทมเพลตสำหรับใช้ nil channel ใน select

```
type Worker struct {
    taskCh    chan Task
    enabled   bool
    // ไม่ต้องมี mutex
}

func (w *Worker) Run() {
    var activeCh chan Task = w.taskCh
    if !w.enabled {
        activeCh = nil
    }

    for {
        select {
        case task, ok := <-activeCh:
            if !ok {
                return
            }
            process(task)
        case <-w.toggleCh:
            // รับ signal เพื่อสลับสถานะ
            w.enabled = !w.enabled
            if w.enabled {
                activeCh = w.taskCh
            } else {
                activeCh = nil
            }
        case <-w.quit:
            return
        }
    }
}
```

การใช้ nil channel ใน select เป็นเทคนิคระดับสูงที่ช่วยลดความซับซ้อนของโค้ด concurrent ได้มาก โดยเฉพาะในระบบที่มีการเปิด/ปิดการทำงานของ worker หรือ source แบบพลวัต

การใช้ nil channel กับ context หรือ pattern อื่นๆ

การใช้ nil channel ร่วมกับ context และ Pattern อื่นๆ ใน Go

การรวม nil channel เข้ากับ `context.Context` เป็นเทคนิคที่มีประสิทธิภาพในการควบคุมการทำงานของ goroutine แบบไดนามิก โดยเฉพาะเมื่อต้องการยกเลิก (cancel), กำหนดเวลา (timeout), หรือปรับเปลี่ยนพฤติกรรมของ select cases โดยไม่ต้องเขียนโค้ดซับซ้อน

🔍 หลักการพื้นฐาน

- `context.Done()` คืนค่า channel (`<-chan struct{}`) ที่ถูกปิดเมื่อ context ถูก cancel หรือ timeout
- nil channel ไม่เคยพร้อม (block ตลอด) → เมื่อนำมาใช้ใน select จะถูกข้าม
- การสลับระหว่าง `context.Done()` กับ nil channel ช่วยให้เราสามารถ เปิด/ปิดการตอบสนองต่อ context cancellation แบบไดนามิก

💻 ตัวอย่างที่ 1: เปิด/ปิดการรับ signal จาก context

```
package main

import (
    "context"
    "fmt"
    "time"
)

func worker(ctx context.Context, enableCancel bool) {
    var cancelCh <-chan struct{} = ctx.Done()
    if !enableCancel {
        cancelCh = nil // ปิดการรับ signal cancel
    }

    ticker := time.NewTicker(500 * time.Millisecond)
    defer ticker.Stop()

    for {
        select {
        case <-cancelCh:
            fmt.Println("🛑 รับ signal cancel -> หยุดทำงาน")
            return
        case t := <-ticker.C:
            fmt.Printf("✅ ทำงานที่เวลา %s\n", t.Format("15:04:05"))
        }
    }
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())

    // เปิดใช้งานการรับ cancel
    go worker(ctx, true)

    time.Sleep(2 * time.Second)
```

```

fmt.Println("🔔 ส่ง cancel signal")
cancel() // worker จะหยุด

time.Sleep(1 * time.Second)

// สร้าง context ใหม่
ctx2, cancel2 := context.WithCancel(context.Background())
// ปิดการรับ cancel
go worker(ctx2, false)

time.Sleep(2 * time.Second)
fmt.Println("🔔 ส่ง cancel signal (worker จะไม่หยุด)")
cancel2() // worker จะไม่หยุดเพราะ cancelCh = nil

time.Sleep(2 * time.Second)
fmt.Println("จบโปรแกรม")
}

```

คำอธิบาย:

- `cancelCh` จะเป็น `ctx.Done()` หรือ `nil` ตามพารามิเตอร์ `enableCancel`
- เมื่อ `cancelCh = nil` → `case <-cancelCh` จะไม่ถูกเลือก ทำให้ worker ทำงานต่อไปแม้ context จะถูก cancel
- มีประโยชน์ในการทดสอบหรือระบบที่ต้องการ ignore cancellation ในบางช่วง

ตัวอย่างที่ 2: การปรับ timeout แบบไดนามิกด้วย nil channel

```

package main

import (
    "context"
    "fmt"
    "time"
)

type DynamicTimeoutWorker struct {
    timeout time.Duration
}

func (w *DynamicTimeoutWorker) Run(ctx context.Context, taskCh <-chan string) {
    var timeoutCh <-chan time.Time = nil // เริ่มต้นไม่มี timeout

    for {
        select {
        case task, ok := <-taskCh:
            if !ok {
                fmt.Println("task channel ปิดแล้ว")
                return
            }
            fmt.Printf("📦 รับงาน: %s\n", task)

```

```

        // จำลองการทำงาน
        time.Sleep(500 * time.Millisecond)

        case <-timeoutCh:
            fmt.Println("🕒 timeout! ทำงานไม่ทัน")
            return

        case <-ctx.Done():
            fmt.Println("✗ context cancelled")
            return
    }
}

func (w *DynamicTimeoutWorker) SetTimeout(d time.Duration) {
    w.timeout = d
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    worker := &DynamicTimeoutWorker{}
    taskCh := make(chan string, 10)

    // เริ่ม worker (ยังไม่มี timeout)
    go worker.Run(ctx, taskCh)

    // ส่งงาน
    go func() {
        for i := 1; i <= 5; i++ {
            taskCh <- fmt.Sprintf("งาน %d", i)
            time.Sleep(800 * time.Millisecond)
        }
        close(taskCh)
    }()

    // จำลองการปรับ timeout หลังจาก 2 วินาที
    go func() {
        time.Sleep(2 * time.Second)
        fmt.Println("🕒 เปิดใช้งาน timeout 1.5 วินาที")
        worker.SetTimeout(1500 * time.Millisecond)
        // จริงๆ ต้องมีวิธีเปลี่ยน timeoutCh ใน worker
        // แต่ตัวอย่างนี้แสดงแนวคิด: เราต้องส่ง signal ไปที่ worker
        // วิธีทำจริงใช้ channel ส่ง หรือ redesign
    }()

    time.Sleep(6 * time.Second)
}

```

หมายเหตุ: ตัวอย่างข้างต้นไม่สามารถเปลี่ยน `timeoutCh` ได้โดยตรง เพราะมันอยู่ใน goroutine การแก้ไขต้องใช้ channel ส่งงาน (ดูตัวอย่างถัดไป)

ตัวอย่าง 2.1: การปรับ timeout แบบถูกต้อง (ใช้ channel ส่ง + nil channel)

```
package main

import (
    "context"
    "fmt"
    "time"
)

type Worker struct {
    timeoutUpdateCh chan time.Duration // รับค่า timeout ใหม่
}

func NewWorker() *Worker {
    return &Worker{
        timeoutUpdateCh: make(chan time.Duration),
    }
}

func (w *Worker) Run(ctx context.Context, taskCh <-chan string) {
    var timeoutCh <-chan time.Time = nil
    var currentTimeout time.Duration = 0

    for {
        select {
            case task, ok := <-taskCh:
                if !ok {
                    return
                }
                fmt.Printf("ทำงาน: %s\n", task)
                time.Sleep(500 * time.Millisecond)

            case <-timeoutCh:
                fmt.Println("Timeout! หยุด")
                return

            case newTimeout := <-w.timeoutUpdateCh:
                if newTimeout > 0 {
                    currentTimeout = newTimeout
                    timeoutCh = time.After(currentTimeout) // non-nil
                    fmt.Printf("ตั้ง timeout = %v\n", currentTimeout)
                } else {
                    timeoutCh = nil // ปิด timeout
                    fmt.Println("ปิด timeout")
                }
            }

            case <-ctx.Done():
        }
    }
}
```

```

        fmt.Println("Cancel")
        return
    }
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    w := NewWorker()
    taskCh := make(chan string)

    go w.Run(ctx, taskCh)

    // ส่งงาน
    go func() {
        for i := 1; i <= 10; i++ {
            taskCh <- fmt.Sprintf("งาน%d", i)
            time.Sleep(400 * time.Millisecond)
        }
        close(taskCh)
    }()

    // ปรับ timeout
    time.Sleep(1 * time.Second)
    w.timeoutUpdateCh <- 2 * time.Second // ตั้ง timeout 2 วิ
    time.Sleep(3 * time.Second)
    w.timeoutUpdateCh <- 0 // ปิด timeout
    time.Sleep(2 * time.Second)

    fmt.Println("จบ")
}

```

ตัวอย่างที่ 3: Graceful shutdown + ปิดการรับงานใหม่ (nil channel + context)

```

package main

import (
    "context"
    "fmt"
    "os"
    "os/signal"
    "sync"
    "syscall"
    "time"
)

```



```

type Server struct {
    workers    int
    taskCh     chan int
    wg         sync.WaitGroup
    shutdown   chan struct{}
}

func NewServer(workers int) *Server {
    return &Server{
        workers: workers,
        taskCh:  make(chan int, 100),
        shutdown: make(chan struct{}),
    }
}

func (s *Server) Start(ctx context.Context) {
    for i := 0; i < s.workers; i++ {
        s.wg.Add(1)
        go s.worker(ctx, i)
    }
}

func (s *Server) worker(ctx context.Context, id int) {
    defer s.wg.Done()
    var taskCh <-chan int = s.taskCh // เริ่มต้นรับงาน

    for {
        select {
        case task, ok := <-taskCh:
            if !ok {
                return
            }
            fmt.Printf("Worker %d: ทำงาน %d\n", id, task)
            time.Sleep(500 * time.Millisecond)

        case <-s.shutdown:
            // ส่ง shutdown: หยุดรับงานใหม่ทันที (เปลี่ยน taskCh เป็น nil)
            fmt.Printf("Worker %d: ได้รับ shutdown signal -> หยุดรับงานใหม่\n", id)
            taskCh = nil // สำคัญ: ปิดการรับงานใหม่
            // แต่ยังทำงานที่ค้างอยู่? ในที่นี้ไม่มีการค้างเพราะ taskCh ถูกเปลี่ยน

        case <-ctx.Done():
            fmt.Printf("Worker %d: context cancelled\n", id)
            return
        }
    }
}

func (s *Server) Stop() {
    close(s.shutdown) // ส่ง signal ให้ worker ทั้งหมดหยุดรับงานใหม่
    close(s.taskCh)   // ปิด task channel
    s.wg.Wait()
    fmt.Println("Server stopped gracefully")
}

```

```

}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    srv := NewServer(3)
    srv.Start(ctx)

    // ส่งงาน
    go func() {
        for i := 1; i <= 20; i++ {
            select {
            case srv.taskCh <- i:
                fmt.Printf("ส่งงาน %d\n", i)
            case <-time.After(100 * time.Millisecond):
                fmt.Println("ส่งงานช้าเกินไป")
            }
            time.Sleep(200 * time.Millisecond)
        }
    }()

    // รับ signal SIGTERM
    sigCh := make(chan os.Signal, 1)
    signal.Notify(sigCh, syscall.SIGINT, syscall.SIGTERM)

    <-sigCh
    fmt.Println("\nรับ signal ปิดระบบ")
    srv.Stop()
}

```

อธิบายการทำงาน:

- เมื่อ `s.shutdown` ถูก close (จาก `Stop()`) → worker จะเปลี่ยน `taskCh` จาก channel จริงเป็น `nil`
- การเปลี่ยนเป็น `nil` ทำให้ case `<-taskCh` ถูกข้าม → worker จะไม่รับงานใหม่
- แต่ worker ยังไม่ exit จนกว่า context จะถูก cancel หรือมี shutdown เพิ่มเติม (ในตัวอย่างนี้ worker จะรออยู่นาน เพราะ select มีแค่ shutdown กับ taskCh ที่เป็น nil → block ตลอด ต้องเพิ่ม quit channel)
- เพื่อให้ worker ออกได้ ควรเพิ่ม `case <-ctx.Done()` หรือ close quit channel

ตัวอย่างที่ 4: Fan-out pattern – หยุดส่งงานให้ worker ที่ทำงานช้า (nil channel)

```

package main

import (
    "context"
    "fmt"
    "sync"
    "time"

```

```

)

type Worker struct {
    id      int
    input    chan int
    slow     bool
    activeCh chan int // ตัวแปรที่ใช้ใน select (อาจเป็น nil)
}

func NewWorker(id int, slow bool) *Worker {
    return &Worker{
        id:      id,
        input:    make(chan int, 10),
        slow:     slow,
        activeCh: nil, // เริ่มต้นไม่ active
    }
}

func (w *Worker) Start(ctx context.Context, wg *sync.WaitGroup) {
    wg.Add(1)
    go func() {
        defer wg.Done()
        w.activeCh = w.input // เริ่ม active

        for {
            select {
            case task, ok := <-w.activeCh:
                if !ok {
                    return
                }
                fmt.Printf("Worker %d: เริ่มงาน %d\n", w.id, task)
                if w.slow {
                    time.Sleep(2 * time.Second) // ช้า
                } else {
                    time.Sleep(300 * time.Millisecond)
                }
                fmt.Printf("Worker %d: เสร็จงาน %d\n", w.id, task)

            case <-ctx.Done():
                fmt.Printf("Worker %d: exit\n", w.id)
                return
            }
        }
    }()
}

// ฟังก์ชันปิด worker (เปลี่ยน activeCh เป็น nil)
func (w *Worker) Disable() {
    w.activeCh = nil
    fmt.Printf("Worker %d: ถูก disable (ไม่รับงานใหม่)\n", w.id)
}

func (w *Worker) Enable() {

```

```

w.activeCh = w.input
fmt.Printf("Worker %d: ถูก enable (รับงานได้อีก)\n", w.id)
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    w1 := NewWorker(1, false) // เร็ว
    w2 := NewWorker(2, true)  // ช้า

    var wg sync.WaitGroup
    w1.Start(ctx, &wg)
    w2.Start(ctx, &wg)

    // ส่งงานไปยังทั้งสอง worker ผ่าน fan-out
    taskSource := make(chan int)
    go func() {
        for i := 1; i <= 20; i++ {
            taskSource <- i
            time.Sleep(200 * time.Millisecond)
        }
        close(taskSource)
    }()

    // Fan-out: ส่งงานไปยัง worker ที่ active
    go func() {
        for task := range taskSource {
            // ส่งไป w1
            select {
            case w1.input <- task:
                fmt.Printf("ส่งงาน %d ไป worker1\n", task)
            default:
                fmt.Printf("worker1 เต็ม ช้าม\n")
            }
            // ส่งไป w2 ถ้า active (activeCh != nil)
            if w2.activeCh != nil {
                select {
                case w2.input <- task:
                    fmt.Printf("ส่งงาน %d ไป worker2\n", task)
                default:
                    fmt.Printf("worker2 เต็ม ช้าม\n")
                }
            } else {
                fmt.Printf("worker2 ถูก disable ไม่ส่งงาน %d\n", task)
            }
        }
    }()

    // จำลองการ disable worker2 หลังจาก 3 วินาที
    time.Sleep(3 * time.Second)
    w2.Disable()
    time.Sleep(3 * time.Second)
}

```

```

w2.Enable()
time.Sleep(3 * time.Second)

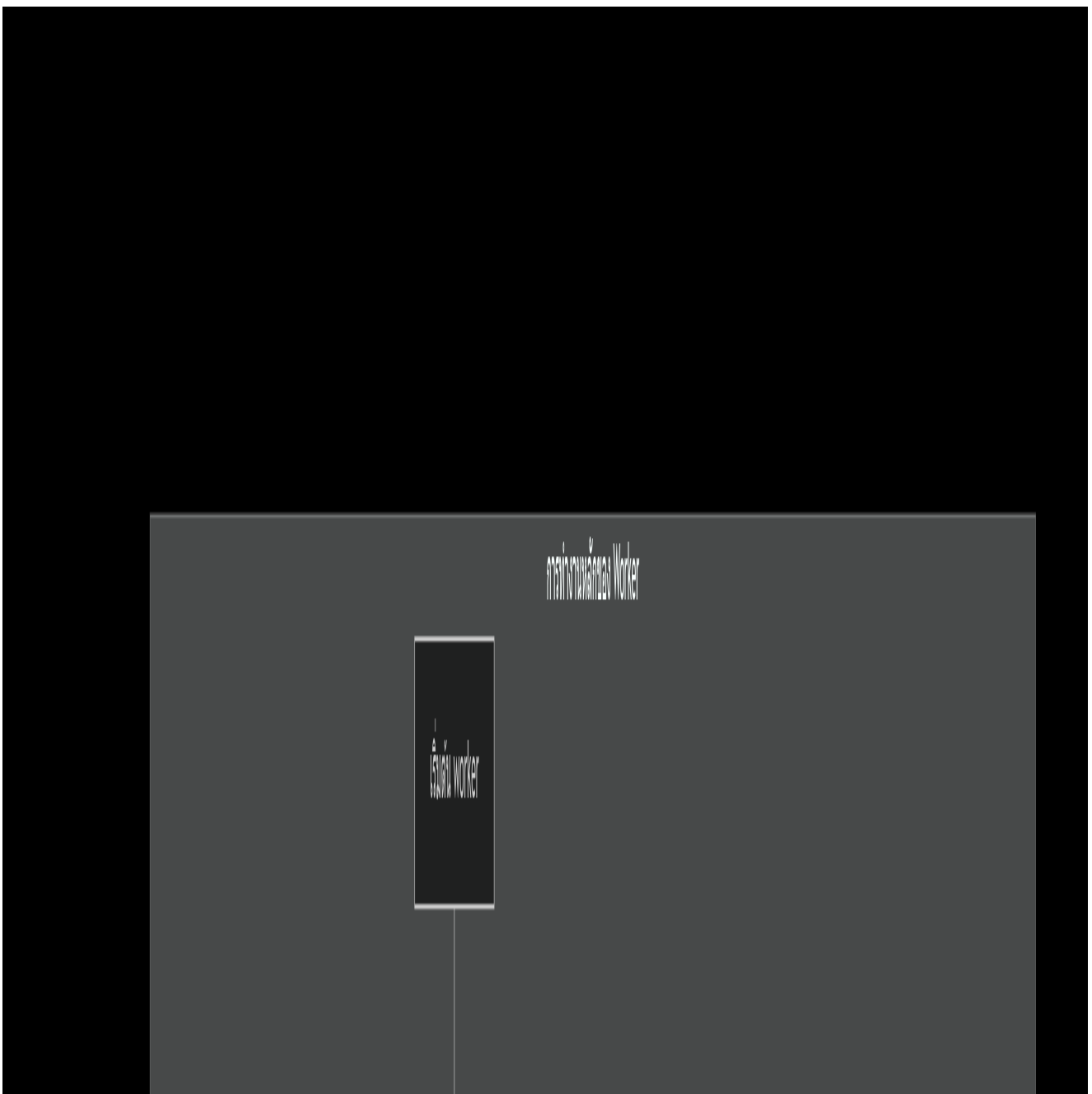
cancel()
wg.Wait()
fmt.Println("จบ")
}

```

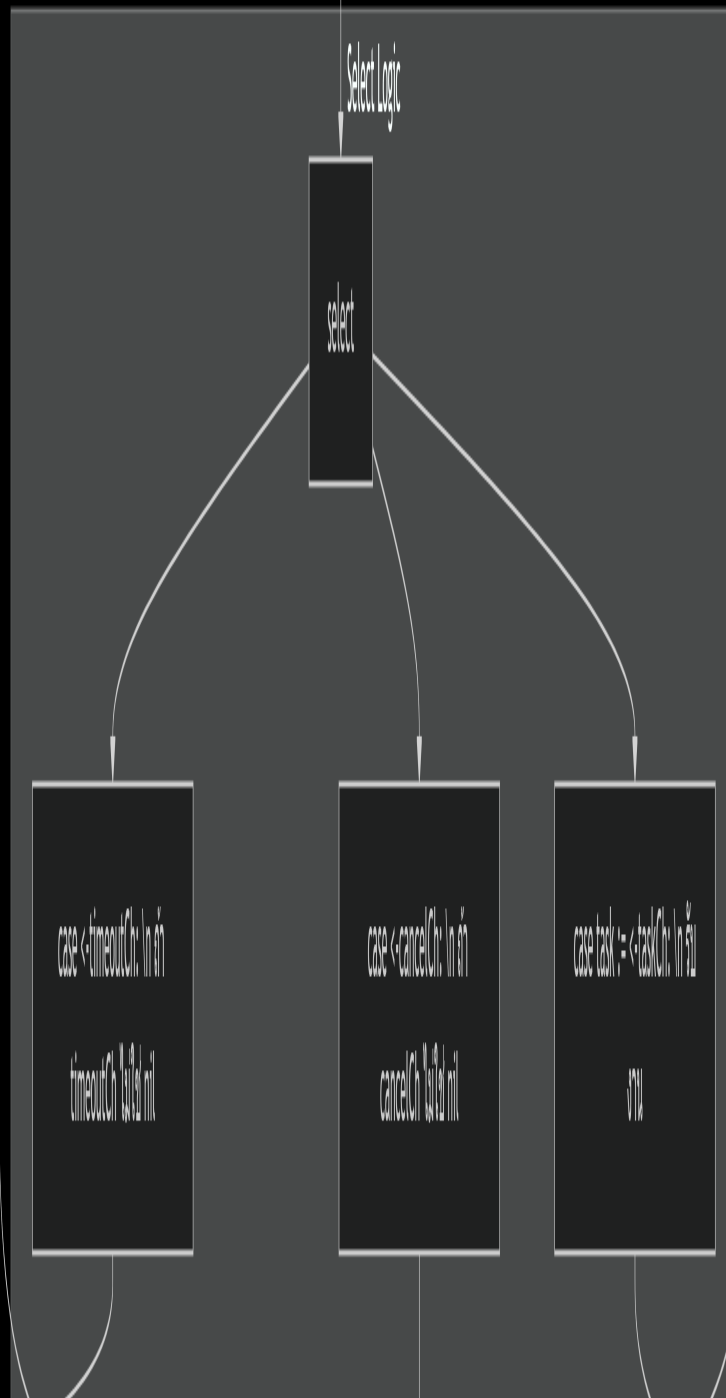
อธิบาย:

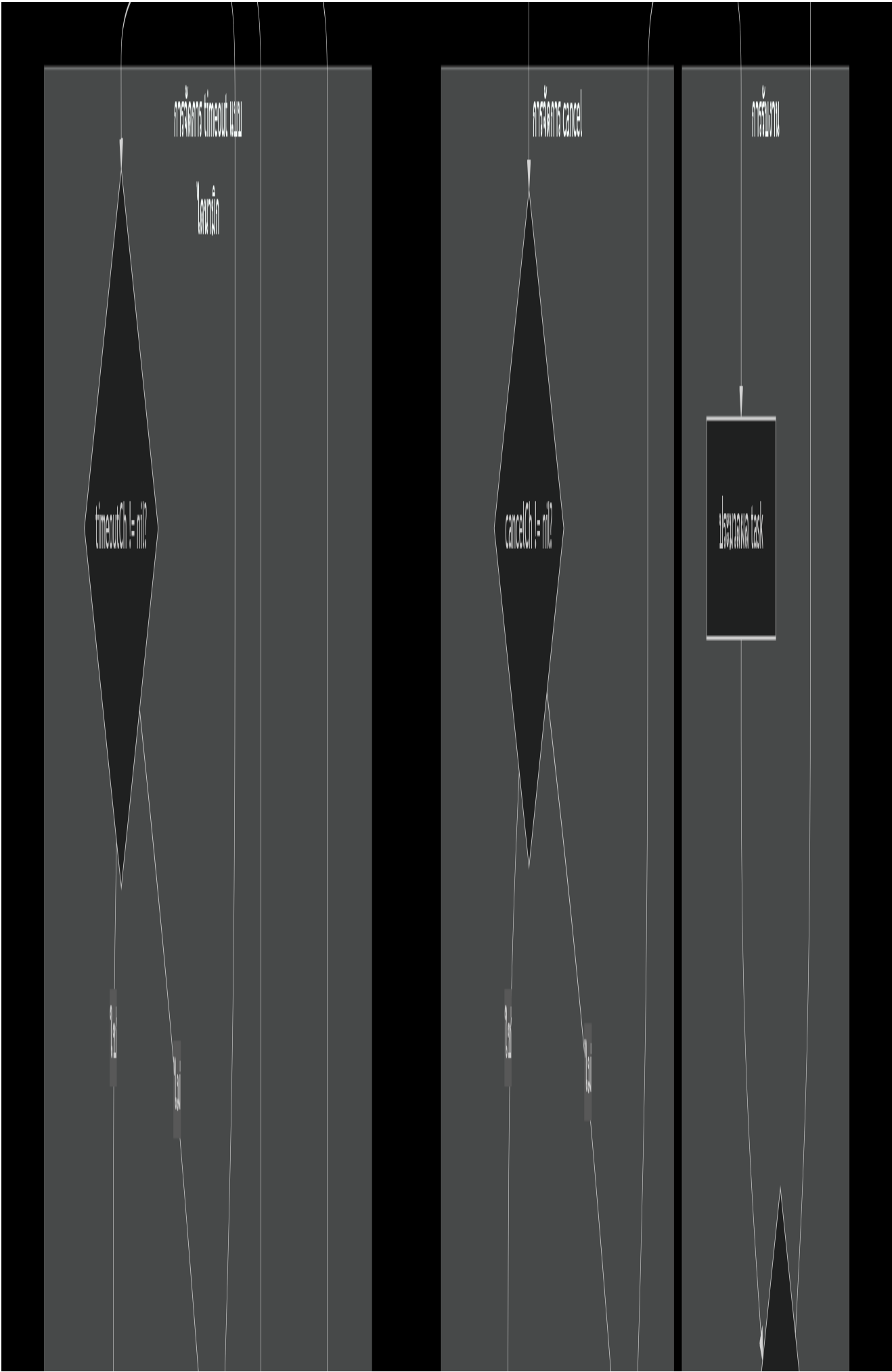
- `w2.activeCh` เป็นตัวแปรที่ใช้ใน select loop ของ worker ถ้าเป็น `nil` worker จะไม่รับงาน
- ภายนอก (fan-out) ก็สามารถตรวจสอบ `w2.activeCh != nil` ก่อนส่งงานได้
- ช่วยลดการส่งงานไปยัง worker ที่ไม่พร้อม

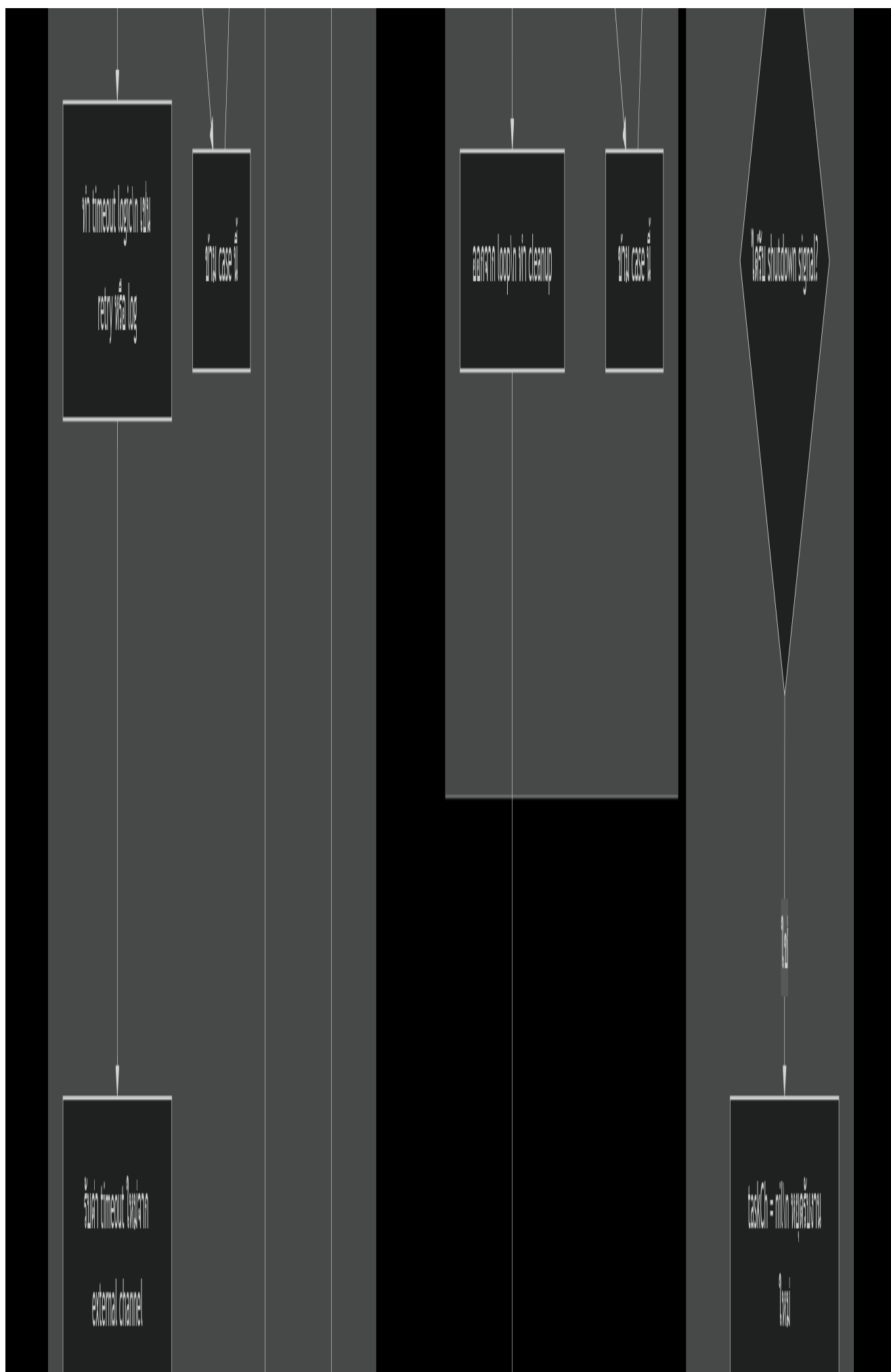
แผนภาพ Flowchart – Nil Channel + Context

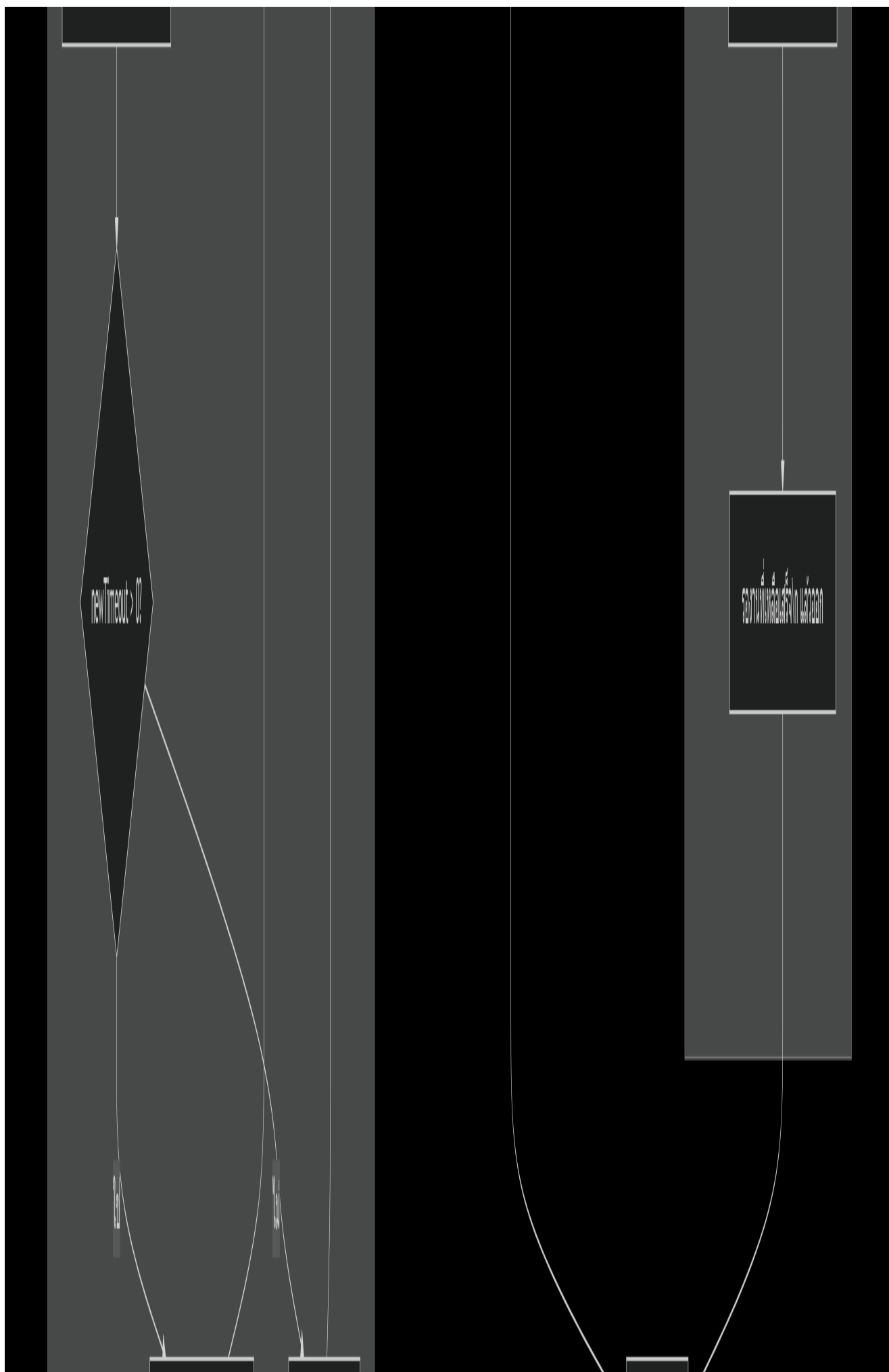


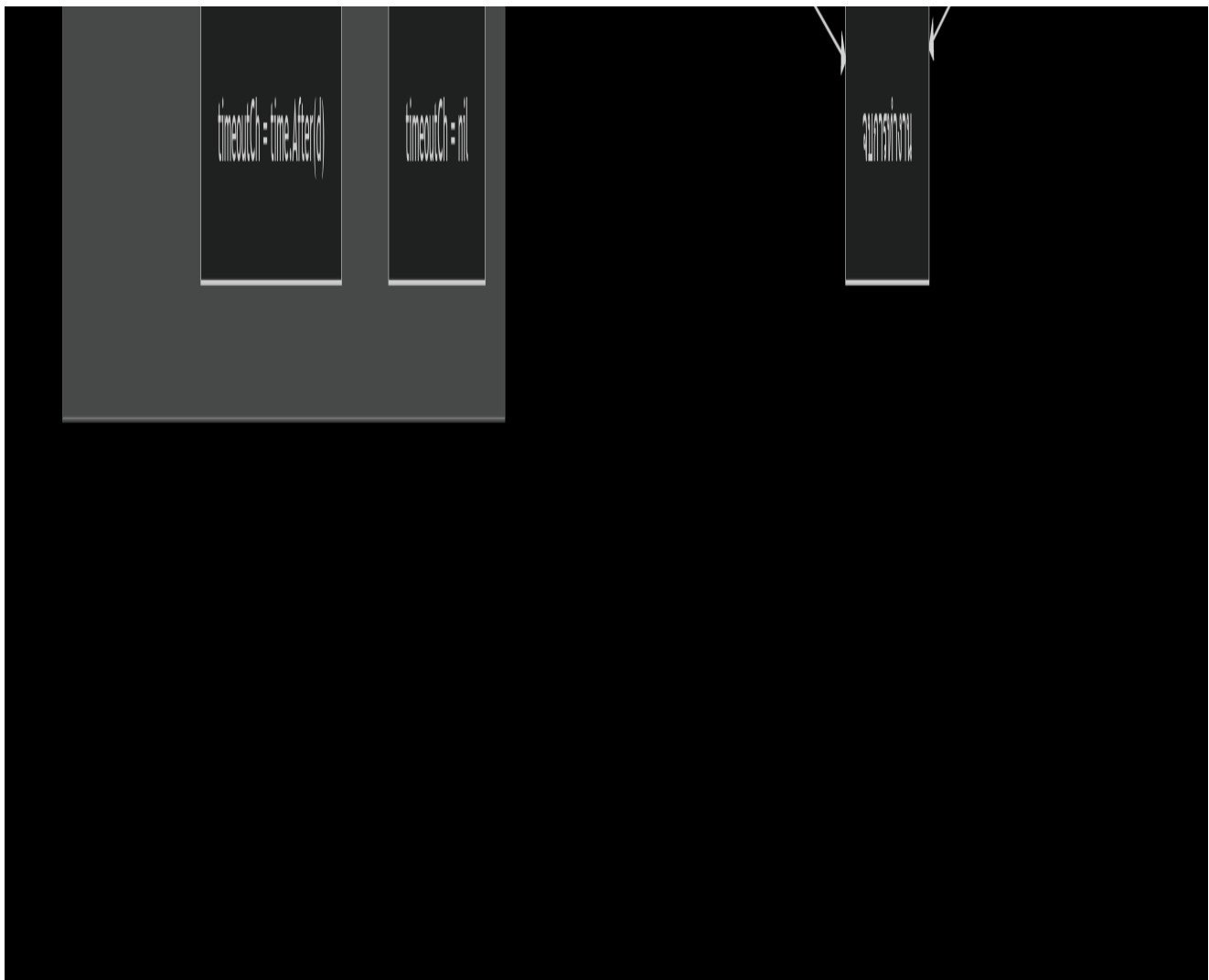












🌀 Pattern อื่นๆ ที่ใช้ nil channel + context

1. Heartbeat pattern ที่สามารถปิดได้

```
func heartbeat(ctx context.Context, interval time.Duration) {  
    ticker := time.NewTicker(interval)  
    defer ticker.Stop()  
    var heartbeatCh <-chan time.Time = ticker.C  
  
    for {  
        select {  
        case <-heartbeatCh:  
            fmt.Println("heartbeat")  
        case <-ctx.Done():  
            heartbeatCh = nil // หยุด heartbeat  
            fmt.Println("heartbeat disabled")  
            // แต่ยังไม่ exit รอ signal อื่น  
        case <-time.After(10 * time.Second):  
            if heartbeatCh == nil {  
                // ถ้า heartbeat ถูกปิดนานเกินไป อาจ exit  
                return  
            }  
        }  
    }  
}
```

```

    }
  }
}

```

2. Rate limiter แบบปรับอัตราตาม context

```

type AdaptiveRateLimiter struct {
    ctx      context.Context
    ticker    *time.Ticker
    tickerCh  <-chan time.Time
    rateCh    chan time.Duration // รับอัตราใหม่
}

func (rl *AdaptiveRateLimiter) Run() {
    for {
        select {
        case <-rl.tickerCh:
            // ทำบางอย่าง
        case newRate := <-rl.rateCh:
            rl.ticker.Stop()
            rl.ticker = time.NewTicker(newRate)
            rl.tickerCh = rl.ticker.C
        case <-rl.ctx.Done():
            rl.tickerCh = nil // ปิด rate limiting
        }
    }
}

```

3. Pipeline ที่สามารถ skip stage ได้

```

func stage(ctx context.Context, in <-chan int) <-chan int {
    out := make(chan int)
    var inCh <-chan int = in
    go func() {
        defer close(out)
        for {
            select {
            case v, ok := <-inCh:
                if !ok {
                    return
                }
                out <- v * 2
            case <-ctx.Done():
                inCh = nil // หยุดรับ input
                // แต่ยังไม่ให้ output ถูกอ่านจนหมด
            }
        }
    }
}

```

```

}()
return out
}

```

☑ สรุปแนวทางปฏิบัติ

Pattern	การใช้ nil channel	ร่วมกับ context
Toggle cancellation	เปลี่ยน <code>ctx.Done()</code> ↔ <code>nil</code>	ควบคุมว่า worker จะตอบสนองต่อ cancel หรือไม่
Dynamic timeout	เปลี่ยน <code>time.After(d)</code> ↔ <code>nil</code>	ปรับ timeout ตาม context value หรือ external signal
Graceful shutdown	เปลี่ยน input channel ↔ <code>nil</code>	หยุดรับงานใหม่ แต่ให้ context cancel เพื่อออก
Adaptive heartbeat	เปลี่ยน ticker channel ↔ <code>nil</code>	ปิด heartbeat เมื่อ context ยกเลิก
Skip pipeline stage	เปลี่ยน input channel ↔ <code>nil</code>	หยุดรับข้อมูลจาก stage ก่อนหน้า

☹ ข้อควรจำ

- nil channel ใน select **ไม่ทำให้เกิด deadlock** ถ้ามี case อื่นที่ non-nil และพร้อม
- การใช้ nil channel ร่วมกับ context ต้องระวัง: ถ้าต้องการออกจาก goroutine ควรมี `case <-ctx.Done()` หรือ quit channel ที่ไม่เป็น nil เสมอ
- nil channel เหมาะสำหรับการ **ปิดการทำงานของ case ชั่วคราว** โดยไม่ต้อง restructuring โค้ด

การใช้ nil channel กับ `sync.Cond` หรือ `atomic.Value`

📚 การใช้ nil channel ร่วมกับ sync.Cond และ atomic.Value ใน Go

การรวม nil channel เข้ากับ `sync.Cond` (Condition Variable) และ `atomic.Value` เป็นเทคนิคขั้นสูงที่ช่วยให้การประสานงานระหว่าง goroutine มีประสิทธิภาพและยืดหยุ่นมากขึ้น โดยเฉพาะเมื่อต้องการ **เปิด/ปิดการแจ้งเตือน** หรือ **สลับ channel แบบ thread-safe** โดยไม่ต้องใช้ mutex มากมาย

🔍 ทบทวน nil channel กับ select

- nil channel ใน select จะถูก **ข้าม (skip)** เพราะ block ตลอด
- nil channel จึงใช้เป็น **สวิตช์ปิด case** แบบไดนามิก

```
var ch chan int = nil // ปิดการรับ/ส่ง
select {
case <-ch:           // case นี้จะไม่มีทางถูกเลือก
default:
    fmt.Println("nil channel case skipped")
}
```

sync.Cond – Condition Variable

`sync.Cond` ใช้สำหรับให้ goroutine รอ (wait) จนกว่าเงื่อนไขบางอย่างจะเป็นจริง โดยมี `Broadcast()` และ `Signal()` เพื่อปลุก

🧩 nil channel + sync.Cond มีประโยชน์อย่างไร?

แม้ `sync.Cond` จะใช้ `Wait()` ซึ่งไม่เกี่ยวข้องกับ channel โดยตรง แต่เราสามารถ ใช้ **nil channel** ใน **select** เพื่อควบคุมการรอแบบมีเงื่อนไข ร่วมกับ Cond ได้ เช่น:

- รอจากหลายแหล่ง: channel หรือ Cond ก็ได้
- ใช้ nil channel เพื่อปิดการรอจาก channel ชั่วคราว

```
type Event struct {
    cond *sync.Cond
    // channel ที่อาจเป็น nil สำหรับ external trigger
    triggerCh chan struct{}
}

func (e *Event) WaitWithTimeout(ctx context.Context, timeout time.Duration) bool {
    var timerCh <-chan time.Time = time.After(timeout)
    var triggerCh <-chan struct{} = e.triggerCh // อาจเป็น nil ได้

    done := make(chan bool, 1)
    go func() {
        e.cond.L.Lock()
        e.cond.Wait() // รอ Cond signal
        e.cond.L.Unlock()
        done <- true
    }()

    select {
    case <-done:
        return true // ได้รับ signal จาก Cond
    case <-triggerCh:
        return true // ได้รับ signal จาก channel (ถ้าไม่เป็น nil)
    case <-timerCh:
        return false // timeout
    case <-ctx.Done():
        return false
    }
```

```
}  
}
```

 ตัวอย่างจริง: ระบบ Queue ที่สามารถปิดการแจ้งเตือนผ่าน channel ได้

```
package main  
  
import (  
    "fmt"  
    "sync"  
    "time"  
)  
  
type Queue struct {  
    items []int  
    mu     sync.Mutex  
    cond   *sync.Cond  
    // channel สำหรับ external trigger (อาจเป็น nil)  
    notifyCh chan struct{}  
}  
  
func NewQueue() *Queue {  
    q := &Queue{  
        notifyCh: make(chan struct{}, 1),  
    }  
    q.cond = sync.NewCond(&q.mu)  
    return q  
}  
  
// EnableNotify เปิดการแจ้งเตือนผ่าน channel (ตั้ง notifyCh ให้ไม่เป็น nil)  
func (q *Queue) EnableNotify() {  
    q.mu.Lock()  
    defer q.mu.Unlock()  
    if q.notifyCh == nil {  
        q.notifyCh = make(chan struct{}, 1)  
    }  
}  
  
// DisableNotify ปิดการแจ้งเตือนผ่าน channel (ตั้ง notifyCh = nil)  
func (q *Queue) DisableNotify() {  
    q.mu.Lock()  
    defer q.mu.Unlock()  
    q.notifyCh = nil  
}  
  
// Push เพิ่มข้อมูลและแจ้งเตือนทั้ง Cond และ channel (ถ้า channel ไม่เป็น nil)  
func (q *Queue) Push(v int) {  
    q.mu.Lock()  
    q.items = append(q.items, v)  
    q.mu.Unlock()
```

```

q.cond.Signal() // ปลุก Wait

// ส่ง signal ผ่าน channel ถ้ามี
q.mu.Lock()
ch := q.notifyCh
q.mu.Unlock()
if ch != nil {
    select {
        case ch <- struct{}{}:
        default:
    }
}
}

// PopWithChannel รอรับข้อมูลผ่าน channel หรือ Cond
func (q *Queue) PopWithChannel(ctx <-chan struct{}) (int, error) {
    // ตัวแปร channel ที่อาจเป็น nil
    var notifyCh <-chan struct{} = q.notifyCh

    for {
        q.mu.Lock()
        if len(q.items) > 0 {
            v := q.items[0]
            q.items = q.items[1:]
            q.mu.Unlock()
            return v, nil
        }
        q.mu.Unlock()

        // รอจากหลายแหล่ง
        select {
            case <-ctx:
                return 0, fmt.Errorf("cancelled")
            case <-notifyCh:
                // ได้รับ signal จาก channel (ถ้าไม่เป็น nil)
                continue
            default:
                // ใช้ Cond Wait แทน
                q.mu.Lock()
                q.cond.Wait()
                q.mu.Unlock()
        }
    }
}

func main() {
    q := NewQueue()
    q.EnableNotify() // เปิด channel notification

    // ตัวรับ
    go func() {
        for i := 0; i < 5; i++ {

```



```

        val, _ := q.PopWithChannel(nil)
        fmt.Println("received:", val)
    }
}()

// ตัวส่ง
for i := 1; i <= 5; i++ {
    q.Push(i)
    time.Sleep(500 * time.Millisecond)
}

// ปิด channel notification แล้วลองอีกครั้ง
q.DisableNotify()
fmt.Println("disable notify, push 6")
q.Push(6)
time.Sleep(1 * time.Second) // จะยังไม่มีใครรับ เพราะ channel ปิดและ Cond ก็ไม่ถูก
Signal? (ต้อง Signal ด้วย)
// แต่ cond ถูก Signal ใน Push อยู่แล้ว ดังนั้นยังรับได้
}

```

อธิบาย:

- `notifyCh` เป็น channel ที่สามารถเป็น nil ได้ (`DisableNotify`)
- ใน `PopWithChannel` ถ้า `notifyCh` เป็น nil → case `<-notifyCh` จะถูกละเลย
- ทำให้เราสามารถเปิด/ปิดการรับ signal ผ่าน channel ได้โดยไม่ต้องเปลี่ยนโค้ดภายใน loop

atomic.Value – การเก็บ nil channel แบบ thread-safe

`atomic.Value` ใช้สำหรับจัดเก็บค่าแบบ atomic (race-free) และสามารถเก็บค่า nil ได้ (ถ้าประเภทของ value นั้นรองรับ nil) เช่น `chan int, *T, interface{}`

การใช้ nil channel ร่วมกับ atomic.Value

ประโยชน์:

- สลับ **channel** ระหว่าง **nil** กับ **non-nil** โดยไม่ต้อง lock
- หลาย goroutine อ่านค่า channel ปัจจุบันได้อย่างปลอดภัย
- ใช้ใน pattern **dynamic channel switching** (เปลี่ยน channel ที่ worker ใช้รับข้อมูล)

 ตัวอย่าง: Worker ที่เปลี่ยน channel ปลายทางได้แบบ atomic

```

package main

import (
    "fmt"
    "sync/atomic"
    "time"
    "unsafe"
)

```

```

type DynamicWorker struct {
    // เก็บ channel (เป็น unsafe.Pointer เพราะ atomic.Value เก็บ interface{})
    // แต่ใช้ atomic.Value ดีกว่า
    ch atomic.Value // stores chan int หรือ nil
}

func NewDynamicWorker() *DynamicWorker {
    w := &DynamicWorker{}
    w.ch.Store(chan int(nil)) // เริ่มต้นเป็น nil channel
    return w
}

// SetChannel เปลี่ยน channel ปลายทาง (thread-safe)
func (w *DynamicWorker) SetChannel(ch chan int) {
    w.ch.Store(ch)
}

// Disable ปิดการรับข้อมูล (set channel = nil)
func (w *DynamicWorker) Disable() {
    w.ch.Store(chan int(nil))
}

// Run ทำงาน: อ่านจาก channel ปัจจุบัน
func (w *DynamicWorker) Run(stop <-chan struct{}) {
    for {
        select {
        case <-stop:
            return
        default:
            // โหลด channel ปัจจุบัน (atomic)
            val := w.ch.Load()
            var ch chan int
            if val != nil {
                ch = val.(chan int)
            } else {
                ch = nil
            }

            if ch == nil {
                // ไม่มี channel ให้รับ -> รอสักครู่แล้วลองใหม่
                time.Sleep(100 * time.Millisecond)
                continue
            }

            select {
            case data, ok := <-ch:
                if !ok {
                    fmt.Println("channel closed, disabling...")
                    w.Disable()
                    continue
                }
            }
            fmt.Printf("received: %d\n", data)
        }
    }
}

```

```

        case <- stop:
            return
        }
    }
}

func main() {
    worker := NewDynamicWorker()
    stop := make(chan struct{})
    go worker.Run(stop)

    // สร้าง channel แรก
    ch1 := make(chan int)
    worker.SetChannel(ch1)

    // ส่งข้อมูล
    go func() {
        for i := 1; i <= 3; i++ {
            ch1 <- i
            time.Sleep(500 * time.Millisecond)
        }
        close(ch1)
    }()

    time.Sleep(2 * time.Second)

    // เปลี่ยนไปใช้ channel ที่สอง
    ch2 := make(chan int)
    worker.SetChannel(ch2)

    go func() {
        for i := 10; i <= 12; i++ {
            ch2 <- i
            time.Sleep(500 * time.Millisecond)
        }
        close(ch2)
    }()

    time.Sleep(3 * time.Second)

    // ปิด worker (set channel = nil)
    worker.Disable()
    time.Sleep(1 * time.Second)

    close(stop)
    fmt.Println("done")
}

```

 ตัวอย่างที่ซับซ้อน: atomic.Value เก็บ struct ที่มี channel

```

type Config struct {
    InCh  chan string
    OutCh chan string
}

type ConfigManager struct {
    cfg atomic.Value // stores *Config หรือ nil
}

func (m *ConfigManager) Update(cfg *Config) {
    m.cfg.Store(cfg)
}

func (m *ConfigManager) Disable() {
    m.cfg.Store((*Config)(nil))
}

func (m *ConfigManager) Get() *Config {
    val := m.cfg.Load()
    if val == nil {
        return nil
    }
    return val.(*Config)
}

// Worker ที่ใช้ config
func process(m *ConfigManager, stop <-chan struct{}) {
    for {
        cfg := m.Get()
        if cfg == nil {
            time.Sleep(200 * time.Millisecond)
            continue
        }

        select {
        case data, ok := <-cfg.InCh:
            if !ok {
                // channel ปิด -> ปิด config นี้
                m.Disable()
                continue
            }
            result := data + " processed"
            cfg.OutCh <- result
        case <-stop:
            return
        }
    }
}

```



แผนภาพ Flowchart – nil channel + sync.Cond + atomic.Value

flowchart TB

```
subgraph SyncCond["sync.Cond + nil channel"]
    A[Cond.Wait] --> B{ต้องการรอหลายแหล่ง?}
    B -->|ใช่| C[สร้าง select case จาก channel + Cond signal channel]
    C --> D{channel เป็น nil?}
    D -->|ใช่| E[case ข้าม ใช้ Cond อย่างเดียว]
    D -->|ไม่ใช่| F[case รอ channel signal]
end

subgraph AtomicValue["atomic.Value + nil channel"]
    G[atomic.Value เก็บ chan int หรือ nil]
    H[goroutine1: เปลี่ยน channel ด้วย Store]
    I[goroutine2: Load channel และใช้ใน select]
    J{Loaded channel == nil?}
    J -->|ใช่| K[ข้าม case รับข้อมูล]
    J -->|ไม่ใช่| L[รับข้อมูลจาก channel]
end

subgraph Combined["นำมารวมกัน"]
    M[ใช้ atomic.Value เพื่อเปลี่ยน channel ที่ sync.Cond จะรอ]
    N[ใช้ Cond เพื่อแจ้งเตือนเมื่อ atomic channel เปลี่ยน]
end
```

ตัวอย่างการนำไปใช้จริง: Pub-Sub ที่สามารถปิด topic ได้

```
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
    "time"
)

type Topic struct {
    name    string
    ch       atomic.Value // chan string หรือ nil
    mu       sync.Mutex
    cond     *sync.Cond
    closed   bool
}

func NewTopic(name string) *Topic {
    t := &Topic{name: name}
    t.cond = sync.NewCond(&t.mu)
    t.ch.Store(chan string(nil))
    return t
}
```

```

// Enable เปิด topic: สร้าง channel ใหม่
func (t *Topic) Enable() {
    t.mu.Lock()
    defer t.mu.Unlock()
    if t.closed {
        return
    }
    ch := make(chan string, 10)
    t.ch.Store(ch)
    t.cond.Broadcast() // ปลุก subscriber ที่รอ
}

// Disable ปิด topic: เปลี่ยน channel เป็น nil
func (t *Topic) Disable() {
    old := t.ch.Swap(chan string(nil))
    if old != nil {
        close(old.(chan string))
    }
}

// Publish ส่งข้อความ (ถ้า topic เปิด)
func (t *Topic) Publish(msg string) {
    val := t.ch.Load()
    if val == nil {
        fmt.Printf("Topic %s is disabled, drop msg\n", t.name)
        return
    }
    ch := val.(chan string)
    select {
    case ch <- msg:
    default:
        fmt.Println("buffer full, drop")
    }
}

// Subscribe รับข้อความ (ใช้ channel ของ topic โดยตรง หรือ nil)
func (t *Topic) Subscribe() <-chan string {
    val := t.ch.Load()
    if val == nil {
        return nil
    }
    return val.(chan string)
}

// SubscribeWithCond รอจนกว่าจะมี channel (ใช้ Cond)
func (t *Topic) SubscribeWithCond(timeout time.Duration) (<-chan string, error) {
    t.mu.Lock()
    defer t.mu.Unlock()

    // ตรวจสอบ channel ปัจจุบัน
    val := t.ch.Load()

```

```

if val != nil {
    return val.(chan string), nil
}

// รอ signal จาก cond (Enable จะ broadcast)
done := make(chan struct{})
go func() {
    time.Sleep(timeout)
    close(done)
}()

go func() {
    t.cond.Wait()
    close(done) // จริง ๆ ต้องออกแบบให้ถูกต้องกว่านี้
}()
// (ตัวอย่างย่อ)
return nil, nil
}

func main() {
    topic := NewTopic("news")
    topic.Enable()

    // subscriber
    go func() {
        ch := topic.Subscribe()
        for msg := range ch {
            fmt.Println("sub1:", msg)
        }
    }()

    // publisher
    topic.Publish("hello")
    topic.Publish("world")

    time.Sleep(1 * time.Second)
    topic.Disable()
    topic.Publish("this will be dropped")
    time.Sleep(1 * time.Second)
    topic.Enable()
    topic.Publish("after re-enable")
    time.Sleep(1 * time.Second)
}

```

สรุปแนวทางปฏิบัติ

เทคนิค	การใช้ nil channel	ทำงานร่วมกับ
sync.Cond	ใช้ nil channel ใน select ขนานกับ Cond.Wait	ทำให้รอจากหลายแหล่ง (channel + Cond)

เทคนิค	การใช้ nil channel	ทำงานร่วมกับ
atomic.Value	เก็บ channel (nil หรือ non-nil) แบบ thread-safe	สลับ channel ที่ worker ใช้โดยไม่ต้อง lock
Cond + atomic	ใช้ Cond เพื่อแจ้งเตือนเมื่อ atomic channel เปลี่ยน	ป้องกัน race และลด busy loop

☹️ ข้อดีและข้อควรระวัง

- **nil channel + atomic.Value** ช่วยให้เปลี่ยน channel ปลายทางได้โดยไม่ต้อง restart goroutine
- **nil channel + sync.Cond** ช่วยลดการ busy wait และให้รอหลายเงื่อนไข
- ระวัง: atomic.Value เก็บ nil channel ได้ แต่ต้อง cast กลับเป็น **chan T** และตรวจสอบ nil ก่อนใช้
- ถ้าใช้ nil channel ใน select ร่วมกับ Cond.Wait ต้องแน่ใจว่ามี case อื่นที่ non-nil และพร้อม ไม่เช่นนั้นจะ deadlock

📦 เทมเพลตสำเร็จรูป

```
// Template: dynamic channel with atomic.Value
type DynamicReceiver struct {
    ch atomic.Value // chan Data
}

func (d *DynamicReceiver) SetChannel(ch chan Data) {
    d.ch.Store(ch)
}

func (d *DynamicReceiver) Disable() {
    d.ch.Store(chan Data(nil))
}

func (d *DynamicReceiver) Run() {
    for {
        val := d.ch.Load()
        var ch chan Data
        if val != nil {
            ch = val.(chan Data)
        }
        if ch == nil {
            time.Sleep(100 * time.Millisecond)
            continue
        }
        select {
        case data := <-ch:
            process(data)
        case <-ctx.Done():
            return
        }
    }
}
```


การใช้ `nil channel` กับ `sync.Map` หรือ `context` ร่วมกับ `atomic.Value`

การใช้ `nil channel` ร่วมกับ `sync.Map` และ `context` + `atomic.Value`

ในหัวข้อนี้เราจะมาดูการประยุกต์ใช้ `nil channel` ร่วมกับโครงสร้างข้อมูลและเครื่องมือ concurrent อื่นๆ ของ Go อย่าง `sync.Map` และ `context` ที่ร่วมกับ `atomic.Value` เพื่อสร้างระบบที่ยืดหยุ่น ปลอดภัย และสามารถเปลี่ยนแปลงพฤติกรรมขณะรันได้โดยไม่ต้อง lock แบบหยาบ

ทบทวน `nil channel`

- `nil channel` ไม่เคยพร้อม → ใช้ใน `select` แล้ว case นั้นจะถูกข้าม (เหมือน `disable case`)
- `nil channel` สามารถเก็บไว้ใน `sync.Map` หรือ `atomic.Value` ได้ เพราะเป็นค่า zero value ของ type `chan T`
- `nil channel` ไม่เหมือน `closed channel` – `closed channel` พร้อมอ่านได้ทันที (zero value) และส่งไม่ได้ (panic)

`nil channel` + `sync.Map`

แนวคิด

`sync.Map` เป็น map ที่ปลอดภัยสำหรับ concurrent access (multiple goroutines อ่าน/เขียนพร้อมกัน) โดยไม่ต้อง lock เอง เราสามารถเก็บค่า channel (หรือ `nil`) ไว้ใน `sync.Map` เพื่อใช้เป็น ตัวเปิด/ปิดการสื่อสารแบบต่อ `client` หรือ `worker`

 ตัวอย่าง: ระบบ Pub-Sub ที่สามารถปิด subscriber แต่ละรายได้

```
package main

import (
    "fmt"
    "sync"
    "time"
)

type Broker struct {
    subs sync.Map // key: subscriber ID, value: chan string (หรือ nil)
}

func NewBroker() *Broker {
    return &Broker{}
}

// Subscribe ลงทะเบียน subscriber พร้อมสร้าง channel
func (b *Broker) Subscribe(id int) <-chan string {
    ch := make(chan string, 10)
```

```

    b.subs.Store(id, ch)
    return ch
}

// Unsubscribe ยกเลิก subscriber (ปิด channel และเก็บ nil แทน)
func (b *Broker) Unsubscribe(id int) {
    val, ok := b.subs.LoadAndDelete(id)
    if ok {
        if ch, ok := val.(chan string); ok {
            close(ch)
        }
    }
    // เก็บ nil ก็ได้ แต่เราลบออกเลย
}

// DisableSubscriber ปิดการรับข่าวของ subscriber ชั่วคราว (เปลี่ยน channel เป็น nil)
func (b *Broker) DisableSubscriber(id int) {
    val, ok := b.subs.Load(id)
    if ok {
        b.subs.Store(id, nil) // เก็บ nil แทน channel จริง
    }
}

// EnableSubscriber เปิดการรับข่าวอีกครั้ง (ต้องมี channel เดิม)
func (b *Broker) EnableSubscriber(id int, ch chan string) {
    b.subs.Store(id, ch)
}

// Publish ส่งข้อความไปยัง subscriber ทั้งหมดที่ยัง active (channel != nil)
func (b *Broker) Publish(msg string) {
    b.subs.Range(func(key, value interface{}) bool {
        ch, ok := value.(chan string)
        if !ok || ch == nil {
            return true // ข้าม subscriber ที่ถูก disable
        }
        select {
        case ch <- msg:
        default:
            fmt.Printf("subscriber %v buffer full, drop\n", key)
        }
        return true
    })
}

func main() {
    broker := NewBroker()

    // subscriber 1
    ch1 := broker.Subscribe(1)
    go func() {
        for msg := range ch1 {
            fmt.Println("sub1 got:", msg)
        }
    }
}

```

```

}()

// subscriber 2
ch2 := broker.Subscribe(2)
go func() {
    for msg := range ch2 {
        fmt.Println("sub2 got:", msg)
    }
}()

broker.Publish("hello all")

// ปิด sub2 ชั่วคราว
broker.DisableSubscriber(2)
fmt.Println("sub2 disabled")
broker.Publish("this message only sub1 sees")

time.Sleep(1 * time.Second)
// เปิด sub2 ใหม่ (ต้องสร้าง channel ใหม่หรือใช้ของเก่า)
newCh2 := make(chan string, 10)
broker.EnableSubscriber(2, newCh2)
go func() {
    for msg := range newCh2 {
        fmt.Println("sub2 (re-enabled) got:", msg)
    }
}()
broker.Publish("sub2 is back")

time.Sleep(1 * time.Second)
}

```

อธิบาย:

- `sync.Map` เก็บ `chan string` หรือ `nil` สำหรับ subscriber แต่ละคน
- `DisableSubscriber` เปลี่ยนค่าเป็น `nil` → `Publish` จะข้าม subscriber นั้น (ไม่ส่ง)
- `EnableSubscriber` ต้องคืน channel จริงกลับไป
- วิธีนี้ **ไม่ต้อง lock map เอง** และเปลี่ยนสถานะได้ทันทีโดยไม่กระทบ goroutine อื่น

2 nil channel + context + atomic.Value

แนวคิด

`atomic.Value` ใช้เก็บค่าแบบ atomic (race-free) เราสามารถเก็บ **channel (หรือ nil)** ไว้ แล้วให้ goroutine อ่าน channel นั้นผ่าน `Load()` และใช้ใน `select` ร่วมกับ `ctx.Done()` ได้

 ตัวอย่าง: Rate Limiter ที่สามารถเปิด/ปิดการจำกัดอัตราแบบไดนามิก

```
package main
```

```

import (
    "context"
    "fmt"
    "sync/atomic"
    "time"
)

type DynamicRateLimiter struct {
    tickerCh atomic.Value // stores <-chan time.Time หรือ nil
}

func NewDynamicRateLimiter() *DynamicRateLimiter {
    drl := &DynamicRateLimiter{}
    drl.tickerCh.Store(<-chan time.Time(nil)) // เริ่มต้นเปิด rate limit
    return drl
}

// EnableRateLimit เปิดใช้งาน rate limit (ทุก interval)
func (drl *DynamicRateLimiter) EnableRateLimit(interval time.Duration) {
    ticker := time.NewTicker(interval)
    drl.tickerCh.Store(ticker.C)
}

// DisableRateLimit ปิด rate limit (ไม่จำกัด)
func (drl *DynamicRateLimiter) DisableRateLimit() {
    old := drl.tickerCh.Swap(<-chan time.Time(nil))
    if old != nil {
        // หยุด ticker เก่าเพื่อไม่ให้รัว
        if t, ok := old.(*time.Ticker); ok {
            // เราไม่มีวิธีหยุด ticker โดยตรง เพราะเรามีแค่ channel
            // ต้อง redesign ให้เก็บ ticker ด้วย แต่เพื่อความง่ายเราไม่ทำ
        }
    }
}

// Process จำลองการทำงาน โดยจะ rate limit ถ้าเปิดอยู่
func (drl *DynamicRateLimiter) Process(ctx context.Context, id int) {
    for {
        // โหลด channel rate limit ปัจจุบัน
        val := drl.tickerCh.Load()
        var rateLimitCh <-chan time.Time
        if val != nil {
            rateLimitCh = val.(*time.Ticker).C
        } // ถ้า nil จะไม่ใช่ rate limit

        select {
        case <-ctx.Done():
            fmt.Printf("worker %d: cancelled\n", id)
            return
        case <-rateLimitCh:
            // มี rate limit และถึงเวลา
            fmt.Printf("worker %d: rate-limited processing\n", id)
            doWork(id)
        }
    }
}

```

```

    default:
        if rateLimitCh == nil {
            // ไม่มี rate limit → ทำทันที
            fmt.Printf("worker %d: unlimited processing\n", id)
            doWork(id)
        } else {
            // มี rate limit แต่ยังไม่ถึงเวลา → รอ
            time.Sleep(10 * time.Millisecond)
        }
    }
}

func doWork(id int) {
    fmt.Printf("worker %d: working...\n", id)
    time.Sleep(200 * time.Millisecond)
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    limiter := NewDynamicRateLimiter()

    // เปิด rate limit ทุก 1 วินาที
    limiter.EnableRateLimit(1 * time.Second)

    go limiter.Process(ctx, 1)

    time.Sleep(3 * time.Second)

    // ปิด rate limit (เปลี่ยน tickerCh เป็น nil)
    limiter.DisableRateLimit()
    fmt.Println("rate limit disabled")

    time.Sleep(3 * time.Second)

    cancel()
}

```

อธิบาย:

- `tickerCh` เก็บ `<-chan time.Time` (หรือ `nil`) โดยใช้ `atomic.Value`
- ใน `Process()` ถ้า `rateLimitCh == nil` จะไม่ใช่ `select` case นั้น (ไป default ทันที) ทำให้ทำงานไม่จำกัด
- การเปลี่ยน `tickerCh` เป็น `nil` ทำให้ `rate limit` ถูกปิดทันทีโดยไม่ต้อง restart goroutine

3] รวมทั้ง 3 อย่าง: `sync.Map` + `atomic.Value` + `context` + `nil channel`

 ตัวอย่าง: ระบบจัดการ Client Connection แบบไดนามิก

```

package main

import (
    "context"
    "fmt"
    "sync"
    "sync/atomic"
    "time"
)

// ClientManager จัดการ client แต่ละราย
type ClientManager struct {
    clients sync.Map // key: clientID, value: chan string (หรือ nil)
    config  atomic.Value // stores *Config
    ctx     context.Context
    cancel  context.CancelFunc
}

type Config struct {
    GlobalEnabled bool // เปิด/ปิดส่งข้อความทั้งหมด
    RateLimit     time.Duration // 0 = ไม่จำกัด
}

func NewClientManager() *ClientManager {
    ctx, cancel := context.WithCancel(context.Background())
    cm := &ClientManager{
        ctx:    ctx,
        cancel: cancel,
    }
    // config เริ่มต้น
    cm.config.Store(&Config{GlobalEnabled: true, RateLimit: 0})
    return cm
}

// AddClient เพิ่ม client พร้อม channel
func (cm *ClientManager) AddClient(id int) <-chan string {
    ch := make(chan string, 10)
    cm.clients.Store(id, ch)
    return ch
}

// RemoveClient ลบ client (ปิด channel)
func (cm *ClientManager) RemoveClient(id int) {
    val, ok := cm.clients.LoadAndDelete(id)
    if ok {
        if ch, ok := val.(chan string); ok {
            close(ch)
        }
    }
}

// DisableClient ปิด clientชั่วคราว (เปลี่ยน channel เป็น nil)

```

```

func (cm *ClientManager) DisableClient(id int) {
    cm.clients.Store(id, nil)
}

// EnableClient เปิด client ใหม่ (ต้องสร้าง channel ใหม่)
func (cm *ClientManager) EnableClient(id int) <-chan string {
    ch := make(chan string, 10)
    cm.clients.Store(id, ch)
    return ch
}

// UpdateConfig อัปเดต configuration แบบ atomic
func (cm *ClientManager) UpdateConfig(cfg *Config) {
    cm.config.Store(cfg)
}

// Broadcast ส่งข้อความไปยัง client ทั้งหมดที่ active และ global enabled
func (cm *ClientManager) Broadcast(msg string) {
    cfg := cm.config.Load().(*Config)
    if !cfg.GlobalEnabled {
        fmt.Println("global disabled, drop broadcast")
        return
    }

    // rate limit ถ้า cfg.RateLimit > 0
    var rateCh <-chan time.Time
    if cfg.RateLimit > 0 {
        ticker := time.NewTicker(cfg.RateLimit)
        defer ticker.Stop()
        rateCh = ticker.C
    } else {
        rateCh = nil // no rate limit
    }

    // รอ rate limit ถ้ามี
    if rateCh != nil {
        <-rateCh
    }

    cm.clients.Range(func(key, value interface{}) bool {
        ch, ok := value.(chan string)
        if !ok || ch == nil {
            return true // ข้าม client ที่ disabled หรือไม่มี channel
        }
        select {
        case ch <- msg:
        default:
            fmt.Printf("client %v buffer full\n", key)
        }
        return true
    })
}

```

```

// RunHeartbeat ส่ง heartbeat ให้ client ที่ active ทุก 2 วินาที
func (cm *ClientManager) RunHeartbeat(ctx context.Context) {
    ticker := time.NewTicker(2 * time.Second)
    defer ticker.Stop()
    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            cm.Broadcast("heartbeat")
        }
    }
}

// Shutdown ปิด manager ทั้งหมด
func (cm *ClientManager) Shutdown() {
    cm.cancel()
    // ปิด channel ทั้งหมด
    cm.clients.Range(func(key, value interface{}) bool {
        if ch, ok := value.(chan string); ok && ch != nil {
            close(ch)
        }
        cm.clients.Delete(key)
        return true
    })
}

func main() {
    cm := NewClientManager()

    // เพิ่ม client 1 และ 2
    ch1 := cm.AddClient(1)
    ch2 := cm.AddClient(2)

    go func() {
        for msg := range ch1 {
            fmt.Println("client1:", msg)
        }
    }()
    go func() {
        for msg := range ch2 {
            fmt.Println("client2:", msg)
        }
    }()

    // เริ่ม heartbeat
    go cm.RunHeartbeat(cm.ctx)

    time.Sleep(3 * time.Second)

    // ปิด client2 ชั่วคราว
    cm.DisableClient(2)
    fmt.Println("client2 disabled")
}

```



```

time.Sleep(3 * time.Second)

// เปิด client2 ใหม่
newCh2 := cm.EnableClient(2)
go func() {
    for msg := range newCh2 {
        fmt.Println("client2 (re-enabled):", msg)
    }
}()

time.Sleep(2 * time.Second)

// เปลี่ยน config: ปิด global broadcast
cm.UpdateConfig(&Config{GlobalEnabled: false, RateLimit: 0})
fmt.Println("global broadcast disabled")

time.Sleep(3 * time.Second)

cm.Shutdown()
}

```

อธิบาย:

- `sync.Map` เก็บ `chan string` หรือ `nil` สำหรับ client แต่ละคน → เปิด/ปิดได้ทีละ client
- `atomic.Value` เก็บ `*Config` ที่มี `GlobalEnabled` และ `RateLimit` → เปลี่ยนพฤติกรรมรวมแบบเรียลไทม์
- `Broadcast` ใช้ `rateCh` ซึ่งอาจเป็น `nil` (ไม่จำกัด) หรือ `ticker.C`
- `context` ใช้ปิด `RunHeartbeat` และ `Shutdown` ทั้งระบบ

แผนภาพ Flowchart รวม nil channel + sync.Map + atomic.Value + context

```

flowchart TB
    subgraph Storage["การเก็บ nil channel"]
        SM[sync.Map] -->|Store key, nil| DisableClient
        SM -->|Store key, non-nil| EnableClient
        AV[atomic.Value] -->|Store nil| DisableRateLimit
        AV -->|Store ticker.C| EnableRateLimit
    end

    subgraph Readers["Goroutine ที่อ่าน"]
        R1[Client goroutine] -->|Load from sync.Map| Ch{ch == nil?}
        Ch -->|ใช่| Skip[ไม่รับข้อมูล]
        Ch -->|ไม่ใช่| Recv[รับจาก channel]
        R2[Processor goroutine] -->|Load from atomic.Value| Tick{tickerCh == nil?}
        Tick -->|ใช่| Unlimited[ไม่จำกัด rate]
        Tick -->|ไม่ใช่| Limited[รอ ticker]
    end

```

```

subgraph Context["Context integration"]
    Ctx[context.Context] -->|Done channel| Cancel[ยกเลิก goroutine]
    Cancel -->|ปิดการทำงาน| Exit
end

DisableClient --> Skip
EnableClient --> Recv
DisableRateLimit --> Unlimited
EnableRateLimit --> Limited
Recv --> Exit
Unlimited --> Exit
Limited --> Exit

```

☑ สรุปแนวทางปฏิบัติ

เครื่องมือ	ใช้ nil channel เพื่อ	ข้อดี
sync.Map	ปิด/เปิด subscriber หรือ worker แต่ละราย	ไม่ต้อง lock map, เปลี่ยนสถานะได้ทันที
atomic.Value	สลับ channel ที่ใช้ใน rate limiting หรือ dynamic routing	ปลอดภัยสำหรับ concurrent, ไม่ต้อง restart goroutine
context	ส่ง signal ยกเลิกให้ goroutine ที่กำลังรอ nil channel (จะได้ออก)	nil channel ทำให้ block ตลอด → ต้องมี ctx.Done() เพื่อ exit
ทั้งสามรวมกัน	สร้างระบบที่ยืดหยุ่น ปรับเปลี่ยนพฤติกรรมขณะรัน	ลด complexity, ไม่ต้องใช้ mutex มาก, ง่ายต่อการ test

☹ ข้อควรระวัง

- **nil channel ใน select ที่ไม่มี case อื่นที่พร้อม** → deadlock ต้องมี `case <-ctx.Done()` หรือ default
- การเก็บ nil channel ใน **sync.Map** แล้ว load มาใช้: ต้อง type assertion และตรวจสอบ nil ก่อนใช้
- **atomic.Value** เก็บ nil channel ได้ แต่ต้องรู้ type ตอนโหลด: `val.(<-chan time.Time)` จะ panic ถ้า val ไม่ใช่ type นั้น (nil channel ก็เป็น type นั้นได้)
- การเปลี่ยน **atomic.Value** บ่อยๆ อาจทำให้เกิด race ในการใช้ ticker เก่า (ควรหยุด ticker ด้วย)

📦 เทมเพลตสำเร็จรูป

```

// template: dynamic feature flag with nil channel + atomic.Value
type Feature struct {
    ch atomic.Value // chan struct{} or nil
}

func (f *Feature) Enable() {
    f.ch.Store(make(chan struct{}))
}

```

```
func (f *Feature) Disable() {
    f.ch.Store(chan struct{})(nil))
}

func (f *Feature) Wait(ctx context.Context) bool {
    val := f.ch.Load()
    var ch chan struct{}
    if val != nil {
        ch = val.(chan struct{})
    }
    select {
    case <-ch:
        return true
    case <-ctx.Done():
        return false
    }
}
```

การผสมผสาน nil channel กับ `sync.Map`, `atomic.Value`, และ `context` จะทำให้คุณได้ระบบที่ **concurrent-safe**, **dynamic**, และ **responsive** โดยไม่ต้องเขียนล็อกซับซ้อน ลองนำไปปรับใช้กับงานจริงดูครับ

2.5 ใครใช้ Go บ้าง?

- **Google** : ระบบ backend, Kubernetes, Docker
- **Uber** : ระบบการจับคู่การเดินทาง
- **Dropbox** : เปลี่ยนจาก Python มาเป็น Go สำหรับระบบจัดเก็บไฟล์
- **Netflix** : ส่วนของ proxy และ caching
- **Cloudflare** : เครื่องมือโครงสร้างพื้นฐาน

บทที่ 3: พื้นฐานการใช้งาน Terminal

3.1 Terminal คืออะไร?

Terminal (หรือ command line, console) เป็นเครื่องมือที่เราสั่งงานคอมพิวเตอร์ผ่านข้อความ แทนการใช้ GUI การพัฒนา Go มักใช้ terminal ในการรันคำสั่งต่างๆ เช่น `go build`, `go run`, `go test`, `git` เป็นต้น

3.2 คำสั่งพื้นฐาน (Unix/Linux/macOS)

- `pwd` : แสดงไดเรกทอรีปัจจุบัน
- `ls` : แสดงรายการไฟล์ (ใช้ `ls -la` แสดงรายละเอียด)
- `cd <path>` : เปลี่ยนไดเรกทอรี
- `mkdir <name>` : สร้างโฟลเดอร์
- `touch <file>` : สร้างไฟล์
- `rm <file>` : ลบไฟล์ (ใช้ `rm -rf` ลบโฟลเดอร์)
- `cp <source> <dest>` : คัดลอกไฟล์
- `mv <source> <dest>` : ย้ายหรือเปลี่ยนชื่อ
- `cat <file>` : แสดงเนื้อหาไฟล์
- `echo <text>` : แสดงข้อความ

- `grep <pattern> <file>` : ค้นหาข้อความ

3.3 คำสั่งพื้นฐานสำหรับ Windows (Command Prompt หรือ PowerShell)

- `cd` : เปลี่ยนไดเรกทอรี
- `dir` : แสดงรายการไฟล์
- `mkdir` : สร้างโฟลเดอร์
- `del` : ลบไฟล์
- `copy` : คัดลอก
- `move` : ย้าย
- `type` : แสดงเนื้อหาไฟล์

3.4 การตั้งค่า PATH

หลังจากติดตั้ง Go แล้ว เราต้องให้ terminal สามารถเรียกใช้คำสั่ง `go` ได้ โดยเพิ่มไดเรกทอรีของ Go (เช่น `/usr/local/go/bin` หรือ `C:\Go\bin`) ลงในตัวแปรสภาพแวดล้อม PATH

3.5 การใช้ go command

- `go version` : ตรวจสอบเวอร์ชัน Go
- `go env` : แสดงตัวแปรสภาพแวดล้อมของ Go
- `go run <file>` : คอมไพล์และรันโปรแกรม
- `go build` : คอมไพล์เป็น binary
- `go mod init` : เริ่มต้น go module
- `go get` : ดาวน์โหลดแพ็คเกจ (deprecated ใช้ `go install` หรือ `go mod download`)
- `go test` : รัน test

บทที่ 4: เตรียมสภาพแวดล้อมสำหรับพัฒนา

4.1 การติดตั้ง Go

1. ดาวน์โหลดจาก <https://go.dev/dl/>
2. เลือกเวอร์ชันที่เหมาะสมกับ OS ของคุณ
3. ติดตั้งตามขั้นตอน
4. ตรวจสอบการติดตั้ง: เปิด terminal แล้วพิมพ์ `go version`

4.2 ตัวแปรสภาพแวดล้อมที่สำคัญ

- `GOROOT` : ตำแหน่งที่ติดตั้ง Go (มักตั้งค่าให้อัตโนมัติ)
- `GOPATH` : ตำแหน่ง workspace ของ Go (เลิกใช้แล้ว ถ้าใช้ modules)
- `GOBIN` : ตำแหน่งที่ติดตั้ง binaries ที่ `go install`
- `GOOS/GOARCH` : ใช้ cross-compile

4.3 การเลือก Editor/IDE

- **Visual Studio Code** : มี extension "Go" โดย官方 ให้ linting, autocomplete, debugging
- **GoLand** : IDE เฉพาะของ JetBrains
- **Vim/Neovim** : ใช้ plugin vim-go

- **Sublime Text** : มี GoSublime

4.4 การตั้งค่า Go Modules

Go Modules เป็นระบบจัดการ dependencies เริ่มใช้ตั้งแต่ Go 1.11 โดยค่าเริ่มต้นใช้งานได้ตั้งแต่ Go 1.16 เป็นต้นไป

```
mkdir myproject
cd myproject
go mod init example.com/myproject
```

ไฟล์ **go.mod** จะถูกสร้างขึ้น

4.5 workspace structure (แบบเดิมกับแบบใหม่)

- แบบ **GOPATH** (ก่อน Go 1.11): ต้องวางโค้ดใน **`$GOPATH/src`**
- แบบ **Modules** (ปัจจุบัน): สามารถสร้างโปรเจกต์ได้ทุกที่ โดยมี go.mod กำกับ

4.6 การติดตั้งเครื่องมือเสริม

- **gopls** : language server (ติดตั้งโดยอัตโนมัติใน VS Code)
- **dlv** : delve debugger (`go install github.com/go-delve/delve/cmd/dlv@latest`)
- **golangci-lint** : linter (`go install github.com/golangci/golangci-lint/cmd/golangci-lint@latest`)

บทที่ 5: สร้างแอปพลิเคชันแรกของคุณ

5.1 สร้างโปรเจกต์

```
mkdir hello
cd hello
go mod init hello
```

5.2 เขียนโค้ด

สร้างไฟล์ **main.go**:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Go!")
}
```

5.3 รันโปรแกรม

```
go run main.go
```

หรือคอมไพล์แล้วรัน:

```
go build -o hello
./hello # (Linux/macOS) หรือ hello.exe (Windows)
```

5.4 เพิ่มฟังก์ชัน

```
package main

import "fmt"

func greet(name string) string {
    return "Hello, " + name
}

func main() {
    message := greet("Gopher")
    fmt.Println(message)
}
```

ทดสอบ `go run main.go` ควรได้ "Hello, Gopher"

5.5 การอ่านค่าจาก command line

```
package main

import (
    "fmt"
    "os"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Println("Please provide a name")
        return
    }
    name := os.Args[1]
    fmt.Printf("Hello, %s!\n", name)
}
```

รัน: `go run main.go Somchai`

5.6 การจัดรูปแบบโค้ด

ใช้คำสั่ง `go fmt` เพื่อจัดรูปแบบโค้ดให้เป็นมาตรฐาน:

```
go fmt ./...
```

5.7 ข้อผิดพลาดที่พบบ่อย

- ใช้ตัวแปรแต่ไม่ได้ใช้ -> Go จะคอมไพล์ไม่ผ่าน (ยกเว้นตัวแปร _)
- import แพคเกจแต่ไม่ได้ใช้ -> ต้องลบออกหรือใช้ _
- ใส่เครื่องหมาย , หรือ ; ไม่ถูกต้อง

ภาคที่ 2: พื้นฐานภาษาและโครงสร้างข้อมูล

(บทที่ 6-16)

แผนภาพแสดงความสัมพันธ์ของเนื้อหา

```
flowchart TD
    subgraph DataRepresentation["1. การแทนข้อมูล (บทที่ 6-7)"]
        direction TB
        A1["เลขฐานสอง (Binary)<br/>0 และ 1<br/>หน่วยเก็บข้อมูลพื้นฐาน"]
        A2["เลขฐานสิบ (Decimal)<br/>ที่มนุษย์ใช้"]
        A3["เลขฐานสิบหก (Hex)<br/>0x0-0xF<br/>แสดงข้อมูลขนาดใหญ่"]
        A4["เลขฐานแปด (Octal)<br/>0o0-0o7<br/>สิทธิ์ไฟล์ Unix"]
        A5["ASCII / UTF-8 / Unicode<br/>การแทนข้อความ<br/>Rune: ตัวอักษรเดียว"]

        A1 --> A3
        A2 --> A3
        A3 --> A5
        A4 --> A5
    end

    subgraph DataStorage["2. การจัดเก็บข้อมูล (บทที่ 8-9)"]
        direction TB
        B1["ตัวแปร (Variables)<br/>var, :=<br/>เก็บค่าที่เปลี่ยนได้"]
        B2["ค่าคงที่ (Constants)<br/>const, iota<br/>ค่าที่ไม่เปลี่ยนแปลง"]
        B3["ชนิดข้อมูลพื้นฐาน<br/>int, float, string, bool<br/>byte, rune"]
        B4["โครงสร้างควบคุม<br/>if-else, for, switch<br/>ควบคุมการทำงาน"]

        B1 --> B4
        B2 --> B4
        B3 --> B4
    end
```

```

subgraph CodeOrganization["3. การจัดระเบียบโค้ด (บทที่ 10-12)"]
    direction TB
    C1["ฟังก์ชัน (Functions)<br/>func, return<br/>multiple returns<br/>defer, panic/recover"]
    C2["แพ็คเกจ (Packages)<br/>package, import<br/>exported/unexported"]
    C3["การเริ่มต้นแพ็คเกจ<br/>init()<br/>ลำดับการทำงาน"]

    C1 --> C2
    C2 --> C3
end

subgraph AdvancedTypes["4. ชนิดข้อมูลขั้นสูง (บทที่ 13-16)"]
    direction TB
    D1["การสร้างชนิดใหม่<br/>type, struct<br/>type alias"]
    D2["เมธอด (Methods)<br/>func (t T) method()<br/>value receiver<br/>pointer receiver"]
    D3["พอยน์เตอร์ (Pointers)<br/>*, &<br/>การอ้างอิงที่อยู่<br/>ไม่มี pointer arithmetic"]
    D4["อินเทอร์เฟซ (Interfaces)<br/>interface{}<br/>implicit implementation<br/>polymorphism"]

    D1 --> D2
    D2 --> D3
    D3 --> D4
end

subgraph Flow["ลำดับการเรียนรู้"]
    direction LR
    F1[เข้าใจการแทนข้อมูล] --> F2[รู้จักการจัดเก็บ] --> F3[จัดระเบียบโค้ด] --> F4[สร้างโครงสร้างขั้นสูง]
end

DataRepresentation --> DataStorage
DataStorage --> CodeOrganization
CodeOrganization --> AdvancedTypes
Flow --> DataRepresentation

```

คำอธิบายภาษาไทย (แบบละเอียด)

ภาคที่ 2 ครอบคลุมบทที่ 6–16 โดยมีเนื้อหาแบ่งเป็น 4 ช่วงหลัก ตามแผนภาพด้านบน ซึ่งแสดงลำดับการเรียนรู้ที่ต่อเนื่องกัน

1. การแทนข้อมูล (บทที่ 6–7)

เป้าหมาย: เข้าใจว่าคอมพิวเตอร์เก็บข้อมูลอย่างไร

บทที่ 6: ระบบเลขฐานสองและฐานสิบ

คอมพิวเตอร์ทำงานด้วยไฟฟ้า รู้จักแค่สถานะ "เปิด" (1) และ "ปิด" (0) ดังนั้นข้อมูลทุกชนิดจึงถูกแปลงเป็นเลขฐานสอง (Binary)

ฐาน	คำอธิบาย	ตัวอย่าง
ฐานสอง (Binary)	ตัวเลข 0 และ 1 เท่านั้น ใช้แสดงข้อมูลระดับเครื่อง	$1010_2 = 10_{10}$
ฐานสิบ (Decimal)	ตัวเลข 0-9 ตามที่มนุษย์ใช้	10_{10}
การแปลง	ฐานสอง → ฐานสิบ: นำแต่ละหลักคูณ $2^{\text{(ตำแหน่ง)}}$ ฐานสิบ → ฐานสอง: หาร 2 ซ้ำๆ เอาเศษ	$1010_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 = 10$

Bit, Byte, และหน่วยความจำ

- **Bit:** หน่วยย่อยที่สุด (0 หรือ 1)
- **Byte:** 8 bits = 1 byte (ใช้แทนตัวอักษร 1 ตัวใน ASCII)
- หน่วยที่ใหญ่กว่า: KB (1024 bytes), MB (1024 KB), GB (1024 MB)

บทที่ 7: เลขฐานสิบหก, ฐานแปด, ASCII, UTF-8, Unicode, Rune

หัวข้อ	คำอธิบาย	ตัวอย่าง
เลขฐานสิบหก (Hexadecimal)	ใช้เลข 0-9 และตัวอักษร A-F (10-15) นิยมเขียนนำหน้าด้วย 0x 1 hex digit = 4 bits (ครึ่ง byte)	$0xFF = 255_{10}$ $0x1A = 26_{10}$
เลขฐานแปด (Octal)	ใช้เลข 0-7 นิยมเขียนนำหน้าด้วย 0o ใช้กำหนดสิทธิ์ไฟล์ใน Unix	$0o755$ = รหัสสิทธิ์ไฟล์
ASCII	ตารางรหัส 7 bits (0-127) ครอบคลุมตัวอักษรอังกฤษ, ตัวเลข, เครื่องหมาย	A = 65, a = 97, 0 = 48
Unicode	มาตรฐานสากลสำหรับตัวอักษรทุกภาษา มีมากกว่า 140,000 ตัวอักษร	U+0E01 = "ก" (ไทย) U+4E2D = "中" (จีน)
UTF-8	การเข้ารหัส Unicode แบบ variable-length ใช้ 1-4 bytes ต่อตัวอักษร เข้ากันได้กับ ASCII	ภาษาอังกฤษ: 1 byte ภาษาไทย: 3 bytes
Rune	ชนิดข้อมูลใน Go สำหรับแทน Unicode code point rune = int32 alias ใช้ 'a' (single quote)	'ก' = U+0E01 (3585) 'A' = 65

ตัวอย่างใน Go

```
package main

import "fmt"
```

```
func main() {
    // ตัวเลขฐานต่างๆ
    binary := 0b1010    // 10
    octal  := 0o12       // 10
    hex    := 0xA        // 10

    fmt.Println(binary, octal, hex)

    // Rune และ UTF-8
    text := "Hello, โลก"
    fmt.Println("Length in bytes:", len(text))           // 11 bytes
    fmt.Println("Number of runes:", len([]rune(text)))   // 8 runes

    // แสดงแต่ละ rune
    for i, r := range text {
        fmt.Printf("Position %d: %c (U+%04X)\n", i, r, r)
    }
}
```

ความสำคัญของ rune ใน Go

- `string` ใน Go เป็น sequence ของ bytes (UTF-8)
- การวนลูปด้วย `for range` จะได้ rune อัตโนมัติ
- ใช้ `[]rune(str)` เพื่อจัดการทีละตัวอักษร

2. การจัดเก็บข้อมูล (บทที่ 8–9)

เป้าหมาย: เรียนรู้การประกาศตัวแปรและควบคุมการทำงาน

บทที่ 8: ตัวแปร, ค่าคงที่, และชนิดข้อมูลพื้นฐาน

ตัวแปร (Variables)

```
// วิธีประกาศ
var name string = "Go"    // ระบุชนิด
var version = 1.23        // type inference
lang := "Golang"          // short declaration (ใช้ภายในฟังก์ชัน)

// ประกาศหลายตัว
var x, y int = 10, 20
a, b := "hello", true

// zero value (ค่าเริ่มต้น)
var i int    // 0
var s string // "" (empty string)
var b bool   // false
var p *int   // nil
```

ค่าคงที่ (Constants)

```
const Pi = 3.14159
const (
    StatusOK      = 200
    StatusNotFound = 404
)

// iota: ตัวนับอัตโนมัติ (เริ่มที่ 0)
const (
    Sunday = iota    // 0
    Monday    // 1
    Tuesday    // 2
    // ... ไปเรื่อยๆ
)
```

ชนิดข้อมูลพื้นฐาน

ชนิด	ขนาด (bit)	ช่วงค่า	ตัวอย่าง
bool	1	true/false	var ok bool = true
int	32/64	ขึ้นอยู่กับระบบ	var age int = 30
int8	8	-128 ถึง 127	var score int8 = 100
int16	16	-32768 ถึง 32767	-
int32 (rune)	32	-2^{31} ถึง $2^{31}-1$	var char rune = 'A'
int64	64	-9×10^{18} ถึง 9×10^{18}	-
uint	32/64	0 ถึง 2^n-1	var count uint = 100
float32	32	$\sim 1.4 \times 10^{-45}$ ถึง 3.4×10^{38}	var pi float32 = 3.14
float64	64	$\sim 4.9 \times 10^{-324}$ ถึง 1.8×10^{308}	var price float64 = 99.99
string	-	sequence of bytes	var name string = "Go"
byte	8	alias of uint8	var b byte = 'A'
rune	32	alias of int32	var r rune = 'ก'

บทที่ 9: คำสั่งควบคุมการทำงาน

if-else

```
// if พื้นฐาน
if x > 0 {
    fmt.Println("positive")
} else if x < 0 {
```

```

    fmt.Println("negative")
} else {
    fmt.Println("zero")
}

// if พร้อม short statement
if err := doSomething(); err != nil {
    fmt.Println("Error:", err)
    return
}

```

for loop (Go มีแค่ for ไม่มี while)

```

// แบบคลาสสิก (for i := 0; i < 10; i++)
for i := 0; i < 10; i++ {
    fmt.Println(i)
}

// while-style (for condition {})
count := 0
for count < 10 {
    count++
}

// infinite loop
for {
    // ใช้ break ออก
}

// range (iterating over slices, maps, strings)
numbers := []int{1, 2, 3}
for index, value := range numbers {
    fmt.Printf("index=%d, value=%d\n", index, value)
}

// range บน string ได้ rune
for i, r := range "Go" {
    fmt.Printf("%d: %c\n", i, r)
}

```

switch (ไม่ต้องใช้ break)

```

// switch กับค่า
switch day := time.Now().Weekday(); day {
case time.Saturday, time.Sunday:
    fmt.Println("Weekend")
default:
    fmt.Println("Weekday")
}

```

```

}

// switch แบบไม่มี expression (ใช้แทน if-else chain)
score := 85
switch {
case score >= 90:
    fmt.Println("A")
case score >= 80:
    fmt.Println("B")
case score >= 70:
    fmt.Println("C")
default:
    fmt.Println("F")
}

// fallthrough (ไปตรวจ case ถัดไป)
switch 2 {
case 1:
    fmt.Println("1")
case 2:
    fmt.Println("2")
    fallthrough
case 3:
    fmt.Println("3") // จะพิมพ์ "2" และ "3"
}

```

3. การจัดระเบียบโค้ด (บทที่ 10–12)

เป้าหมาย: เขียนโค้ดที่เป็นระเบียบ แบ่งเป็นฟังก์ชันและแพ็คเกจ

บทที่ 10: ฟังก์ชัน (Functions)

```

// ฟังก์ชันพื้นฐาน
func add(x int, y int) int {
    return x + y
}

// การประกาศแบบสั้น
func add(x, y int) int {
    return x + y
}

// คำนวณหลายค่า
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("division by zero")
    }
    return a / b, nil
}

```

```
// named return values
func split(sum int) (x, y int) {
    x = sum * 4 / 9
    y = sum - x
    return // naked return
}

// variadic parameters (รับพารามิเตอร์จำนวนไม่แน่นอน)
func sum(numbers ...int) int {
    total := 0
    for _, n := range numbers {
        total += n
    }
    return total
}
// เรียกใช้: sum(1, 2, 3) หรือ sum([]int{1,2,3}...)

// defer: ทำงานก่อนออกจากฟังก์ชัน
func readFile(filename string) error {
    f, err := os.Open(filename)
    if err != nil {
        return err
    }
    defer f.Close() // จะถูกเรียกเมื่อฟังก์ชันจบ

    // อ่านไฟล์...
    return nil
}

// panic และ recover
func safeDivide(a, b int) (result int) {
    defer func() {
        if r := recover(); r != nil {
            fmt.Println("Recovered from panic:", r)
            result = 0
        }
    }()

    if b == 0 {
        panic("division by zero")
    }
    return a / b
}
```

บทที่ 11: แพคเกจและการนำเข้า (Packages and Imports)

หลักการ

- โค้ด Go จัดกลุ่มเป็นแพคเกจ (package)
- `package main` คือ executable program

- แพคเกจอื่นๆ คือ reusable libraries

การตั้งชื่อและ visibility

- ตัวพิมพ์ใหญ่: **exported** (สามารถเข้าถึงจากภายนอกแพคเกจได้)
- ตัวพิมพ์เล็ก: **unexported** (ใช้ภายในแพคเกจเท่านั้น)

ตัวอย่างโครงสร้างโปรเจกต์

```
myproject/  
├── go.mod  
├── main.go  
└── math/  
    └── math.go
```

math/math.go

```
package math  
  
// Exported function (ขึ้นต้นด้วยตัวพิมพ์ใหญ่)  
func Add(a, b int) int {  
    return a + b  
}  
  
// Unexported function (ใช้ภายในแพคเกจ)  
func subtract(a, b int) int {  
    return a - b  
}
```

main.go

```
package main  
  
import (  
    "fmt"  
    "myproject/math" // import แพคเกจ custom  
)  
  
func main() {  
    result := math.Add(5, 3)  
    fmt.Println(result) // 8  
  
    // math.subtract(5, 3) // ERROR: ไม่สามารถเข้าถึงได้  
}
```

บทที่ 12: การเริ่มต้นทำงานของแพคเกจ (Package Initialization)

ลำดับการทำงาน

1. ตัวแปรระดับแพ็คเกจถูกกำหนดค่า
2. ฟังก์ชัน `init()` ถูกเรียก (เรียงตามลำดับในไฟล์ และข้ามแพ็คเกจ)
3. ฟังก์ชัน `main()` ถูกเรียก (เฉพาะแพ็คเกจ `main`)

```
package main

import "fmt"

var globalVar = initGlobal()

func initGlobal() string {
    fmt.Println("1. Initializing global variable")
    return "ready"
}

func init() {
    fmt.Println("2. First init function")
}

func init() {
    fmt.Println("3. Second init function")
}

func main() {
    fmt.Println("4. Main function")
}

// ผลลัพธ์:
// 1. Initializing global variable
// 2. First init function
// 3. Second init function
// 4. Main function
```

การใช้ `init()` เพื่อตั้งค่า

```
package database

import (
    "database/sql"
    _ "github.com/lib/pq" // blank import: เรียกเฉพาะ init()
)

var DB *sql.DB

func init() {
    // ตั้งค่า connection pool
    var err error
```



```

    DB, err = sql.Open("postgres", "connection string")
    if err != nil {
        panic(err)
    }
}

```

4. ชนิดข้อมูลขั้นสูง (บทที่ 13–16)

เป้าหมาย: สร้างโครงสร้างข้อมูลที่ซับซ้อนและเขียนโค้ดที่ยืดหยุ่น

บทที่ 13: การสร้างชนิดข้อมูลใหม่ (Types)

```

// type alias
type MyInt int
type UserID string

// struct (โครงสร้างข้อมูล)
type Person struct {
    Name    string
    Age     int
    Address string
}

// struct พร้อม tags (ใช้กับ JSON, validation)
type User struct {
    ID        uint   `json:"id"`
    Email     string `json:"email" validate:"required,email"`
    Password  string `json:"-"` // ไม่ถูกส่งใน JSON
    CreatedAt time.Time `json:"created_at"`
}

// embedded struct
type Employee struct {
    Person    // embedded field (ไม่ต้องมีชื่อ)
    Position string
    Salary    float64
}

func main() {
    var id MyInt = 100
    var uid UserID = "user-123"

    // สร้าง struct
    p1 := Person{Name: "Alice", Age: 30}
    p2 := Person{
        Name: "Bob",
        Age: 25,
    }
}

```

```
// เข้าถึง field
fmt.Println(p1.Name)
p1.Age = 31

// embedded struct
emp := Employee{
    Person:  Person{Name: "Charlie", Age: 35},
    Position: "Developer",
    Salary:  75000,
}
fmt.Println(emp.Name) // เข้าถึง field จาก Person โดยตรง
}
```

บทที่ 14: เมธอด (Methods)

เมธอดคือฟังก์ชันที่ผูกกับ type (receiver)

```
// กำหนด type
type Rectangle struct {
    Width, Height float64
}

// Value receiver (รับสำเนา)
func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

// Pointer receiver (รับ reference)
func (r *Rectangle) Scale(factor float64) {
    r.Width *= factor
    r.Height *= factor
}

// ใช้ pointer receiver เมื่อต้องการแก้ไข struct
func (r *Rectangle) SetWidth(w float64) {
    r.Width = w
}

// เมธอดบนชนิดอื่นๆ (ไม่ใช่แค่ struct)
type Counter int

func (c *Counter) Increment() {
    *c++
}

func (c Counter) Value() int {
    return int(c)
}

func main() {
```

```

rect := Rectangle{Width: 10, Height: 5}

// value receiver (ไม่แก้ไขต้นฉบับ)
area := rect.Area() // 50

// pointer receiver (แก้ไขต้นฉบับ)
rect.Scale(2) // rect เปลี่ยนเป็น Width=20, Height=10

// counter
var cnt Counter = 5
cnt.Increment()
fmt.Println(cnt.Value()) // 6
}

```

เมื่อใช้ value receiver vs pointer receiver

Value Receiver	Pointer Receiver
รับสำเนา ไม่แก้ไขต้นฉบับ	รับ reference แก้ไขต้นฉบับได้
เหมาะกับ struct เล็ก	เหมาะกับ struct ใหญ่
เรียกใช้กับ value หรือ pointer ก็ได้	เรียกใช้กับ value หรือ pointer ก็ได้
ไม่เปลี่ยนค่าในเมธอด	เปลี่ยนค่าในเมธอดได้

บทที่ 15: พอยน์เตอร์ (Pointers)

พอยน์เตอร์คือตัวแปรที่เก็บ ที่อยู่หน่วยความจำ (memory address)

```

func main() {
    x := 10
    p := &x // p ชี้ไปที่ x (เก็บ address)

    fmt.Println(x) // 10
    fmt.Println(p) // 0xc0000120a0 (address)
    fmt.Println(*p) // 10 (dereference: อ่านค่า)

    // เปลี่ยนค่าผ่าน pointer
    *p = 20
    fmt.Println(x) // 20 (เปลี่ยนแล้ว)

    // zero value ของ pointer คือ nil
    var q *int
    fmt.Println(q) // nil

    // สร้าง pointer ใหม่ด้วย new
    r := new(int)
    *r = 100
    fmt.Println(*r) // 100
}

```

```
// ฟังก์ชันที่รับ pointer
func increment(p *int) {
    *p++
}

// ฟังก์ชันที่คืน pointer
func newInt(value int) *int {
    return &value
}

// เปรียบเทียบ: pass by value vs pass by pointer
func byValue(x int) {
    x = 100 // ไม่มีผลนอกฟังก์ชัน
}

func byPointer(x *int) {
    *x = 100 // มีผลนอกฟังก์ชัน
}
```

ข้อควรรู้เกี่ยวกับ pointer ใน Go

- Go มี pointer แต่ **ไม่มี pointer arithmetic** (ต่างจาก C)
- ใช้ **&** เพื่อ get address, ***** เพื่อ dereference
- **nil** pointer dereference ทำให้เกิด panic
- ใช้ pointer กับ struct ขนาดใหญ่เพื่อประหยัดหน่วยความจำ

บทที่ 16: อินเทอร์เฟซ (Interfaces)

Interface กำหนด **พฤติกรรม** (method set) ที่ type ต้องมี

```
// ประกาศ interface
type Speaker interface {
    Speak() string
}

type Animal interface {
    Speak() string
    Move() string
}

// type ที่ implement interface
type Dog struct {
    Name string
}

func (d Dog) Speak() string {
    return "Woof!"
}
```

```

func (d Dog) Move() string {
    return "Running on 4 legs"
}

type Cat struct {
    Name string
}

func (c Cat) Speak() string {
    return "Meow!"
}

func (c Cat) Move() string {
    return "Walking gracefully"
}

// ฟังก์ชันที่รับ interface
func MakeSound(s Speaker) {
    fmt.Println(s.Speak())
}

func main() {
    dog := Dog{Name: "Max"}
    cat := Cat{Name: "Luna"}

    // Dog และ Cat implement Speaker
    MakeSound(dog) // Woof!
    MakeSound(cat) // Meow!

    // empty interface (interface{}) สามารถเก็บค่าใดก็ได้
    var anything interface{}
    anything = 42
    anything = "hello"
    anything = Dog{Name: "Buddy"}

    // type assertion (ตรวจสอบชนิด)
    val, ok := anything.(Dog)
    if ok {
        fmt.Println(val.Speak())
    }

    // type switch
    switch v := anything.(type) {
    case int:
        fmt.Println("int:", v)
    case string:
        fmt.Println("string:", v)
    case Dog:
        fmt.Println("Dog:", v.Name)
    default:
        fmt.Println("unknown type")
    }
}

```

Interface Composition (การรวม interface)

```
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}

// ประกอบ interface
type ReadWriter interface {
    Reader
    Writer
}
```

ข้อดีของ interface

- **Polymorphism**: เขียนโค้ดที่ทำงานกับ type ต่างๆ ที่มีพฤติกรรมเดียวกัน
- **Decoupling**: แยก implementation ออกจาก contract
- **Testability**: สร้าง mock ได้ง่าย

Interface ใน Go ต่างจากภาษา OOP อื่น

- Go interface **implement implicitly** (ไม่ต้องประกาศ implements)
- Interface เป็น **ค่าที่มี 2 ส่วน**: dynamic type + dynamic value
- Interface สามารถเป็น **nil** (ถ้า type เป็น nil และ value เป็น nil)

สรุปภาคที่ 2: ความสัมพันธ์ของเนื้อหา

```
flowchart LR
    subgraph Knowledge["องค์ความรู้"]
        K1["การแทนข้อมูล<br/>บทที่ 6-7"] --> K2["การจัดเก็บ<br/>บทที่ 8-9"]
        K2 --> K3["การจัดระเบียบ<br/>บทที่ 10-12"]
        K3 --> K4["ชนิดข้อมูลขั้นสูง<br/>บทที่ 13-16"]
    end

    subgraph Skills["ทักษะที่ได้"]
        S1["เข้าใจการทำงาน<br/>ระดับบิต/ไบต์"]
        S2["เขียนโปรแกรม<br/>แบบมีเงื่อนไข"]
        S3["แบ่งโค้ดเป็น<br/>ฟังก์ชัน/แพ็คเกจ"]
        S4["สร้าง struct,<br/>methods, interfaces"]
    end

    K1 --> S1
    K2 --> S2
```

K3 --> S3
K4 --> S4

สิ่งที่ได้เรียนรู้

1. บทที่ 6-7: รู้ว่าคอมพิวเตอร์จัดเก็บข้อมูลอย่างไร (Binary, Hex, UTF-8, Rune)
2. บทที่ 8-9: ประกาศตัวแปรและควบคุมการทำงานด้วย if, for, switch
3. บทที่ 10-12: จัดระเบียบโค้ดด้วยฟังก์ชันและแพ็คเกจ
4. บทที่ 13-16: สร้าง struct, กำหนดเมธอด, ใช้ pointer, และออกแบบ interface

พื้นฐานเหล่านี้จะนำไปใช้ใน ภาคที่ 3 เพื่อจัดการโปรเจกต์, สร้าง test, และใช้ slice/map ในการจัดการข้อมูลจำนวนมาก

บทที่ 6: ระบบเลขฐานสองและฐานสิบ

6.1 ความสำคัญของระบบเลข

คอมพิวเตอร์ทำงานด้วยสัญญาณไฟฟ้า 2 สถานะ คือ ON (1) และ OFF (0) ดังนั้นการแทนค่าทั้งหมดในคอมพิวเตอร์จึงใช้เลขฐานสอง

6.2 เลขฐานสิบ (Decimal)

เป็นระบบที่มนุษย์ใช้ประจำ มีตัวเลข 0-9 (ฐาน 10) เช่น $123 = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$

6.3 เลขฐานสอง (Binary)

ใช้ตัวเลข 0 และ 1 (ฐาน 2) เช่น $1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$

6.4 การแปลงเลขฐานสองเป็นฐานสิบ

วิธี: คูณแต่ละหลักด้วย 2 ยกกำลังตำแหน่ง (เริ่มจากขวาเป็น 0)

- $1010_2 = (1 \times 8) + (0 \times 4) + (1 \times 2) + (0 \times 1) = 10_{10}$

6.5 การแปลงเลขฐานสิบเป็นฐานสอง

วิธี: หารด้วย 2 ไปเรื่อยๆ เก็บเศษ แล้วอ่านเศษจากล่างขึ้นบน

- $13 \div 2 = 6$ เศษ 1
 - $6 \div 2 = 3$ เศษ 0
 - $3 \div 2 = 1$ เศษ 1
 - $1 \div 2 = 0$ เศษ 1
- อ่านจากล่างขึ้นบน: 1101_2

6.6 การแสดงเลขฐานสองใน Go

Go รองรับ literal ของเลขฐานสองโดยใช้ prefix `0b` หรือ `0B`:

```
var x int = 0b1101 // 13
fmt.Printf("%b\n", x) // แสดงเป็น binary: 1101
```

6.7 หน่วยของข้อมูล

- 1 bit = 0 หรือ 1
- 1 byte = 8 bits
- 1 kilobyte (KB) = 1024 bytes
- 1 megabyte (MB) = 1024 KB
- Go มีชนิดข้อมูลตามขนาด: uint8 (1 byte), uint16, uint32, uint64

6.8 ระบบฐานอื่นๆ

- ฐานแปด (octal): ใช้ prefix `0o` หรือ `00` (เช่น `0o17` = 15)
- ฐานสิบหก (hexadecimal): ใช้ prefix `0x` (เช่น `0xFF` = 255)

บทที่ 7: เลขฐานสิบหก, ฐานแปด, ASCII, UTF8, Unicode และ Runes

7.1 เลขฐานสิบหก (Hexadecimal)

ใช้ตัวเลข 0-9 และตัวอักษร A-F (10-15) ฐาน 16 นิยมใช้แทนสี, address ในหน่วยความจำ

- `0xFF` = 255, `0x10` = 16
- ใน Go: `color := 0xFF00FF`

7.2 เลขฐานแปด (Octal)

ใช้ตัวเลข 0-7 ฐาน 8

- `0o10` = 8, `0o777` = 511

7.3 ASCII

American Standard Code for Information Interchange เป็นรหัส 7 bits แทนตัวอักษรภาษาอังกฤษ, ตัวเลข, เครื่องหมาย 128 ตัว

- `'A'` = 65, `'a'` = 97, `'0'` = 48

7.4 Unicode

เป็นมาตรฐานสากลที่กำหนดรหัสให้กับอักขระทุกภาษา (มากกว่า 1 ล้านรหัส) โดยแต่ละรหัสเรียกว่า "code point" เช่น `U+0041` คือ `'A'`

7.5 UTF-8

UTF-8 เป็นการเข้ารหัส Unicode แบบ variable-length (1-4 bytes) ที่ Go ใช้เป็นค่าเริ่มต้นสำหรับสตริง

- ตัวอักษร ASCII ใช้ 1 byte
- ตัวอักษรไทย 3 bytes

7.6 Rune ใน Go

`rune` เป็น alias ของ `int32` ใช้แทน Unicode code point ใน Go

```
var ch rune = 'C' // รหัส Unicode 0x0043
fmt.Printf("%c %U\n", ch, ch) // C U+0043
```

การวนลูปผ่าน string แบบ rune:

```
s := "hello"
for i, r := range s {
    fmt.Printf("%d: %c (%U)\n", i, r, r)
}
```

7.7 การจัดการ string และ bytes

String ใน Go เป็น read-only slice of bytes (ไม่ใช่ runes) การนับจำนวน runes ใช้ `utf8.RuneCountInString`

```
import "unicode/utf8"
s := "สวัสดี"
fmt.Println(len(s)) // 12 bytes
fmt.Println(utf8.RuneCountInString(s)) // 4 runes
```

7.8 การแปลงระหว่าง string และ []rune

```
s := "ไทย"
runes := []rune(s) // แปลงเป็น slice of runes
s2 := string(runes) // แปลงกลับ
```

บทที่ 8: ตัวแปร, ค่าคงที่ และชนิดข้อมูลพื้นฐาน

8.1 ตัวแปร (Variables)

ตัวแปรคือที่เก็บข้อมูลในหน่วยความจำ มีชื่อและชนิดข้อมูล

```
var name string = "John"
var age int = 30
var isActive bool = true
```

รูปแบบ: `var <ชื่อ> <ชนิด> = <ค่าเริ่มต้น>`

การประกาศแบบสั้นด้วย `:=` (ใช้ภายในฟังก์ชันเท่านั้น):

```
name := "John" // Go infer type เป็น string
age := 30
```

การประกาศหลายตัวแปร:

```
var x, y int = 1, 2
var (
    a string = "hello"
    b int = 42
)
```

8.2 ค่าคงที่ (Constants)

ค่าคงที่ประกาศด้วย `const` ไม่สามารถเปลี่ยนแปลงค่าได้

```
const Pi = 3.14159
const (
    StatusOK = 200
    StatusNotFound = 404
)
```

ค่าคงที่สามารถเป็นตัวเลข, สตริง, หรือ boolean เท่านั้น

8.3 ชนิดข้อมูลพื้นฐาน

Numeric types:

- Integers: `int`, `int8`, `int16`, `int32`, `int64` (int ขนาดตามระบบ 32/64 bit)
- Unsigned: `uint`, `uint8` (byte), `uint16`, `uint32`, `uint64`
- Floating point: `float32`, `float64`
- Complex: `complex64`, `complex128`

Boolean: `bool` (true/false)

String: `string` (immutable sequence of bytes)

Rune: `rune` (int32) ใช้แทน Unicode code point

Byte: `byte` (uint8) ใช้แทน byte

8.4 ค่าเริ่มต้น (Zero values)

ตัวแปรทุกตัวเมื่อประกาศโดยไม่กำหนดค่า จะมีค่าเริ่มต้น:

- ตัวเลข: 0
- bool: false

- string: "" (empty string)
- pointer, slice, map, channel, interface, function: nil

8.5 การแปลงชนิดข้อมูล (Type conversion)

Go ต้องแปลงชนิดอย่างชัดเจน ไม่มี implicit conversion

```
var i int = 42
var f float64 = float64(i) // แปลง int -> float64
var u uint = uint(f)       // แปลง float64 -> uint
```

8.6 การกำหนดชื่อตัวแปร

- เริ่มต้นด้วยตัวอักษรหรือ _
- ตามด้วยตัวอักษร ตัวเลข หรือ _
- case-sensitive
- นิยมใช้ camelCase (เช่น userName)
- ตัวพิมพ์ใหญ่ต้น = exported (public) ถ้าอยู่ในแพ็คเกจ

8.7 ตัวดำเนินการพื้นฐาน

- คณิตศาสตร์: +, -, *, /, %
- การเปรียบเทียบ: ==, !=, <, >, <=, >=
- ตรรกะ: &&, ||, !
- Bitwise: &, |, ^, &^, <<, >>

บทที่ 9: คำสั่งควบคุมการทำงาน

9.1 if-else

```
if score >= 80 {
    fmt.Println("A")
} else if score >= 70 {
    fmt.Println("B")
} else {
    fmt.Println("C")
}
```

สามารถมีคำสั่งสั้น ๆ ก่อนเงื่อนไข:

```
if val := getValue(); val > 10 {
    fmt.Println("big")
}
```

9.2 switch

Go มี switch ที่ยืดหยุ่น:

```
switch day {
case "Monday":
    fmt.Println("Start of week")
case "Friday":
    fmt.Println("TGIF")
default:
    fmt.Println("Other day")
}
```

switch แบบไม่มี expression (เทียบกับ true):

```
switch {
case score >= 80:
    fmt.Println("A")
case score >= 70:
    fmt.Println("B")
default:
    fmt.Println("C")
}
```

fallthrough (ใช้ในกรณีต้องการให้ทำงาน case ถัดไป):

```
switch num {
case 1:
    fmt.Println("one")
    fallthrough
case 2:
    fmt.Println("two")
}
// ถ้า num = 1 จะพิมพ์ one และ two
```

9.3 for loop

Go มีแค่ for (ไม่มี while, do-while)

```
// for แบบคลาสสิก
for i := 0; i < 10; i++ {
    fmt.Println(i)
}

// while-like
```

```

x := 0
for x < 10 {
    fmt.Println(x)
    x++
}

// infinite loop
for {
    // ใช้ break เพื่อออก
}

```

9.4 range

ใช้กับ array, slice, map, string:

```

numbers := []int{2,4,6}
for index, value := range numbers {
    fmt.Println(index, value)
}

// ถ้าไม่ต้องการ index ใช้ _
for _, value := range numbers {
    fmt.Println(value)
}

```

9.5 break, continue, goto

- **break** : ออกจาก loop
- **continue** : ข้ามไป iteration ถัดไป
- **goto** : กระโดดไปยัง label (ไม่แนะนำให้ใช้มาก)

```

for i := 0; i < 5; i++ {
    if i == 2 {
        continue
    }
    if i == 4 {
        break
    }
    fmt.Println(i)
}
// output: 0 1 3

```

9.6 label และการ break ออกจาก outer loop

Outer loop (หรือ loop ภายนอก) คือลูปที่อยู่ชั้นนอกสุดในโครงสร้างของลูปซ้อนกัน (nested loops) หมายถึงลูปที่มีลูปอื่นอยู่ข้างใน ใช้สำหรับควบคุมการวนรอบที่มีหลายมิติ เช่น การวนสมาชิกในอาร์เรย์ 2 มิติ หรือการทำงานที่ต้องมีลูปย่อยหลายรอบ

ตัวอย่างในภาษา Go

```
package main

import "fmt"

func main() {
    // outer loop วนแถว (row)
    for i := 0; i < 3; i++ {
        // inner loop วนคอลัมน์ (column)
        for j := 0; j < 3; j++ {
            fmt.Printf("(%d,%d) ", i, j)
        }
        fmt.Println()
    }
}
```

ผลลัพธ์:

```
(0,0) (0,1) (0,2)
(1,0) (1,1) (1,2)
(2,0) (2,1) (2,2)
```

การควบคุม outer loop ด้วย **break** และ **continue**

ใน Go สามารถใช้ **label** เพื่อระบุว่าการ **break** หรือ **continue** ที่ outer loop แทนที่จะเป็นแค่ inner loop:

```
package main

import "fmt"

func main() {
    // ตั้ง label ชื่อ "outer"
    outer:
    for i := 0; i < 3; i++ {
        for j := 0; j < 3; j++ {
            if i == 1 && j == 1 {
                break outer // ออกจาก outer loop ทั้งหมด
            }
            fmt.Printf("(%d,%d) ", i, j)
        }
        fmt.Println()
    }
}
```

ผลลัพธ์:

```
(0,0) (0,1) (0,2)
(1,0)
```

(เมื่อเจอ (1,1) จะออกจาก outer loop ทันที)

Queue Processor

Queue Processor คืออะไร?

Queue Processor คือระบบหรือคอมโพเนนต์ที่ทำหน้าที่ประมวลผลงาน (jobs/tasks) ที่ถูกส่งเข้ามาในคิว (queue) ตามลำดับ FIFO (First-In-First-Out) หรือตามลำดับความสำคัญ โดยจะดึงงานจากคิวมาประมวลผลแบบ asynchronous ช่วยลดภาระของ main thread, ป้องกันการ overload, และทำให้ระบบสามารถขยายขนาดได้ง่าย

Queue Processor มีกี่แบบ?

แบ่งตามสถาปัตยกรรมและการจัดการได้หลายรูปแบบ:

1. **Simple In-Memory Queue** – ใช้ slice + channel ใน Go เหมาะกับงานภายในแอปพลิเคชันเดียว
2. **Worker Pool** – มี worker หลายตัวดึงงานจาก queue เดียวกัน ใช้ concurrency
3. **Priority Queue** – จัดเรียงตาม priority (heap)
4. **Message Broker** – Redis (List, Stream, Pub/Sub), RabbitMQ, Kafka, AWS SQS เป็นระบบคิวภายนอก
5. **Delayed / Scheduled Queue** – รองรับการประมวลผลตามเวลาที่กำหนด
6. **Transactional Queue** – รองรับการ commit/rollback เช่น Redis Streams + consumer group

ใช้อย่างไร ในกรณีไหน?

- ลดภาระ **synchronous API** – รับ request แล้วส่งเข้า queue ตอบกลับทันที
- งานที่ใช้เวลานาน – ส่งอีเมล, ประมวลผลรูปภาพ, อัปเดตข้อมูลจำนวนมาก
- **Retry & Backoff** – ทำงานที่อาจล้มเหลวแล้วลองใหม่
- กระจายโหลด – มี worker หลายตัวบนหลายเครื่อง
- ร่วมกับ **GORM + Redis Transaction** – เช่น รับ order เข้า queue, เมื่อประมวลผลสำเร็จจะอัปเดตฐานข้อมูล (INSERT/UPDATE/DELETE) ภายใต้อัตราการ commit และ rollback หากสำเร็จ หรือ rollback หาก error

หลักการทำงาน

1. **Producer** สร้าง job และส่งเข้า queue (push)
2. **Queue** จัดเก็บ job (อาจเป็น Redis List, SQS, etc.)
3. **Processor (Consumer)** ดึง job จาก queue (pop) ตามกลไก (polling, push, pub/sub)
4. **Processor** ประมวลผล job (เรียก business logic, เรียก API, ทำงาน DB)
5. **Acknowledge / Remove** เมื่อประมวลผลสำเร็จ, ถ้าล้มเหลวอาจ retry หรือส่งไป dead letter queue

Dataflow Diagram (Flowchart TB)

```

graph TB
    subgraph Producer
        A[API / Service] --> B[Create Job]
        B --> C[Push to Queue]
    end

    subgraph Queue Storage
        D[(Redis List / SQS / Kafka)]
    end

    subgraph Consumer
        E[Queue Processor] --> F[Pop Job]
        F --> G[Process Job]
        G --> H{Success?}
        H -- Yes --> I[Acknowledge / Remove]
        H -- No --> J{Retry?}
        J -- Yes --> K[Requeue or Delay]
        J -- No --> L[Dead Letter Queue]
    end

    C --> D
    D --> E

```

ตัวอย่างการใช้งานจริง (Go + Redis + GORM)

โครงสร้าง Job

```

type Job struct {
    ID          string    `json:"id"`
    Type        string    `json:"type"          // "email", "order_processing"
    Payload     []byte    `json:"payload"`
    RetryCount  int       `json:"retry_count"`
    CreatedAt   time.Time `json:"created_at"`
}

```

Producer: สร้าง job และ push ไป Redis (LPUSH หรือ RPUSH)

```

func EnqueueJob(ctx context.Context, rdb *redis.Client, queueName string, job
Job) error {
    data, _ := json.Marshal(job)
    return rdb.LPush(ctx, queueName, data).Err()
}

```

Consumer: รวน loop ดึง job และประมวลผล


```

func ProcessQueue(ctx context.Context, rdb *redis.Client, db *gorm.DB,
queueName string) {
    for {
        // ใช้ BRPOP เพื่อดึง job (blocking)
        result, err := rdb.BRPop(ctx, 0, queueName).Result()
        if err != nil {
            log.Printf("BRPop error: %v", err)
            continue
        }
        // result[0] = queue name, result[1] = job data
        var job Job
        if err := json.Unmarshal([]byte(result[1]), &job); err != nil {
            log.Printf("Unmarshal error: %v", err)
            continue
        }

        // ประมวลผล job โดยใช้ transaction
        if err := processJob(ctx, db, job); err != nil {
            log.Printf("Job %s failed: %v", job.ID, err)
            // handle retry logic
        }
    }
}

```

processJob พร้อม transaction (GORM) และ Redis

```

func processJob(ctx context.Context, db *gorm.DB, job Job) error {
    // เริ่ม transaction
    tx := db.Begin()
    defer func() {
        if r := recover(); r != nil {
            tx.Rollback()
        }
    }()

    switch job.Type {
    case "order_processing":
        var order Order
        if err := json.Unmarshal(job.Payload, &order); err != nil {
            tx.Rollback()
            return err
        }

        // INSERT order
        if err := tx.Create(&order).Error; err != nil {
            tx.Rollback()
            return err
        }
    }
}

```

```

// UPDATE inventory
for _, item := range order.Items {
    if err := tx.Model(&Product{}).Where("id = ?", item.ProductID).
        Update("stock", gorm.Expr("stock - ?", item.Quantity)).Error;
err != nil {
    tx.Rollback()
    return err
}

// DELETE temporary data
if err := tx.Where("order_id = ?",
order.ID).Delete(&TempOrder{}).Error; err != nil {
    tx.Rollback()
    return err
}

// ใช้ Redis เพื่อบันทึก log
rdb := redis.NewClient(&redis.Options{Addr: "localhost:6379"})
if err := rdb.Set(ctx, fmt.Sprintf("order:%d:processed", order.ID),
time.Now().Unix(), 0).Err(); err != nil {
    tx.Rollback()
    return err
}

// commit transaction
if err := tx.Commit().Error; err != nil {
    return err
}

default:
    return fmt.Errorf("unknown job type: %s", job.Type)
}
return nil
}

```

9.1 if-else

if-else คืออะไร?

if-else เป็นคำสั่งควบคุมเงื่อนไขที่ใช้ตัดสินใจเลือกบล็อกโค้ดที่จะทำงานตามค่าความจริง (boolean) ของเงื่อนไข

if-else มีกี่แบบ?

- if (แบบเดี่ยว)
- if-else (สองทางเลือก)
- if-else if-else (หลายทางเลือก)
- if พร้อม short statement (ประกาศตัวแปรภายในเงื่อนไข)

ใช้อย่างไร ในกรณีไหน?

- ตรวจสอบ error หลังเรียกฟังก์ชัน
- ตรวจสอบค่าก่อนทำ SQL (INSERT/UPDATE/DELETE)
- ตรวจสอบผลลัพธ์จาก Redis ก่อน commit/rollback
- ควบคุม logic ใน Queue Processor (เช่น retry, fallback)

หลักการทำงาน

1. ประเมิน boolean expression
2. ถ้า true → ทำบล็อก if
3. ถ้า false → ทำบล็อก else (ถ้ามี)
4. ตัวแปรใน short statement มีขอบเขตภายในบล็อก

Dataflow Diagram (Flowchart TB)

```
graph TB
    Start([เริ่ม]) --> Cond{เงื่อนไข true?}
    Cond -- true --> IfBlock[ทำ if block]
    Cond -- false --> ElseBlock[ทำ else block]
    IfBlock --> End1([จบ])
    ElseBlock --> End2([จบ])
```

ตัวอย่างกับ GORM / Redis / Queue Processor

ตรวจสอบ error ก่อน commit

```
func UpdateOrderStatus(db *gorm.DB, orderID uint, status string) error {
    tx := db.Begin()
    defer func() {
        if r := recover(); r != nil {
            tx.Rollback()
        }
    }()

    if err := tx.Model(&Order{}).Where("id = ?", orderID).Update("status",
status).Error; err != nil {
        tx.Rollback()
        return err
    }

    // ถ้า update สำเร็จ commit
    if err := tx.Commit().Error; err != nil {
        return err
    }
    return nil
}
```

ใช้ใน Queue Processor: ตรวจสอบ retry count

```
func processJobWithRetry(job Job) error {
    err := processJob(job)
    if err != nil {
        if job.RetryCount < 3 {
            job.RetryCount++
            // re-queue with delay
            requeueWithDelay(job, time.Second * time.Duration(job.RetryCount))
            return nil // not considered error now
        } else {
            // move to dead letter queue
            sendToDLQ(job)
            return fmt.Errorf("max retries exceeded: %v", err)
        }
    }
    return nil
}
```

9.2 switch

switch คืออะไร?

switch คือคำสั่งเลือกทำงานตามค่าของ expression หรือประเภทของ interface (type switch) เหมาะกับหลายทางเลือก

switch มีกี่แบบ?

- Expression switch
- Type switch
- Switch without expression (ใช้แทน if-else chain)
- สามารถใช้ fallthrough, multiple case

ใช้อย่างไร ในกรณีไหน?

- เปลี่ยนการทำงานตาม status, type ของงาน (Queue Processor)
- Type switch สำหรับ interface{} ที่รับ payload ต่างกัน
- ควบคุมการทำ SQL หลายประเภท (INSERT/UPDATE/DELETE)

หลักการทำงาน

1. ประเมิน expression (หรือใช้ true)
2. ตรวจสอบ case ตามลำดับ
3. ถ้า case ตรง → ทำบล็อก ถ้าไม่มี fallthrough จะออกทันที
4. Type switch จะทำ type assertion

Dataflow Diagram (Flowchart TB)

```
graph TB
    Start([เริ่ม]) --> Eval{Expression / Type}
    Eval --> Case1{case 1?}
    Case1 -- true --> Block1[block 1]
    Case1 -- false --> Case2{case 2?}
    Case2 -- true --> Block2
    Case2 -- false --> Default[default block]
    Block1 --> End([จบ])
    Block2 --> End
    Default --> End
```

ตัวอย่างกับ GORM / Redis / Queue Processor

Expression switch สำหรับประมวลผล job หลายประเภท

```
func handleJob(job Job) error {
    switch job.Type {
    case "email":
        return sendEmail(job.Payload)
    case "order_processing":
        return processOrder(job.Payload)
    case "report_generation":
        return generateReport(job.Payload)
    default:
        return fmt.Errorf("unknown job type: %s", job.Type)
    }
}
```

Type switch สำหรับ payload ที่หลากหลาย

```
func processPayload(payload interface{}) error {
    switch v := payload.(type) {
    case Order:
        // insert order
        return db.Create(&v).Error
    case EmailData:
        // send email
        return emailService.Send(v)
    case string:
        // treat as log message
        log.Println(v)
        return nil
    default:
        return fmt.Errorf("unsupported payload type: %T", v)
    }
}
```

ใช้กับ Redis transaction: commit/rollback ตามผลลัพธ์

```
func executeRedisPipeline(rdb *redis.Client, cmds []redis.Cmder) error {
    pipe := rdb.Pipeline()
    for _, cmd := range cmds {
        switch cmd.Name() {
            case "set":
                args := cmd.Args()
                pipe.Set(ctx, args[1].(string), args[2], 0)
            case "incr":
                pipe.Incr(ctx, cmd.Args()[1].(string))
            default:
                return fmt.Errorf("unsupported command")
        }
    }
    _, err := pipe.Exec(ctx)
    return err
}
```

9.3 for loop

for loop คืออะไร?

คำสั่งวนซ้ำเพียงหนึ่งใน Go ใช้สำหรับทำซ้ำบล็อกโค้ดตามเงื่อนไข

for loop มีกี่แบบ?

- Classic: `for init; condition; post { }`
- While-style: `for condition { }`
- Infinite: `for { }`
- Range loop (แยกใน 9.4)

ใช้อย่างไร ในกรณีไหน?

- Batch processing ข้อมูลในฐานข้อมูล
- Retry loop สำหรับ Redis transaction ที่ล้มเหลว (WATCH)
- Poll queue (Queue Processor) อย่างต่อเนื่อง
- วน slice เพื่อทำ INSERT/UPDATE/DELETE หลายรายการ

หลักการทำงาน

1. init ทำงานครั้งแรก
2. ตรวจสอบ condition; ถ้า true → เข้า body
3. หลัง body ทำงาน post แล้วกลับไปตรวจสอบ condition อีกครั้ง

Dataflow Diagram (Flowchart TB) - Classic

```
graph TB
    Start([เริ่ม]) --> Init[init]
    Init --> Cond{condition true?}
    Cond -- true --> Body[body]
    Body --> Post[post]
    Post --> Cond
    Cond -- false --> End([จบ])
```

ตัวอย่างกับ GORM / Redis / Queue Processor

Batch INSERT หลายรายการ

```
func BatchInsertOrders(db *gorm.DB, orders []Order, batchSize int) error {
    for i := 0; i < len(orders); i += batchSize {
        end := i + batchSize
        if end > len(orders) {
            end = len(orders)
        }
        batch := orders[i:end]
        if err := db.Create(&batch).Error; err != nil {
            return err
        }
    }
    return nil
}
```

Retry loop สำหรับ Redis transaction

```
func IncrementWithRetry(ctx context.Context, rdb *redis.Client, key string)
error {
    for retries := 0; retries < 10; retries++ {
        err := rdb.Watch(ctx, func(tx *redis.Tx) error {
            val, err := tx.Get(ctx, key).Int64()
            if err != nil && err != redis.Nil {
                return err
            }
            _, err = tx.TxPipelined(ctx, func(pipe redis.Pipeliner) error {
                pipe.Set(ctx, key, val+1, 0)
                return nil
            })
            return err
        }, key)
        if err == nil {
            return nil
        }
    }
}
```

```

    }
    if err == redis.TxFailedErr {
        continue // retry
    }
    return err
}
return fmt.Errorf("max retries exceeded")
}

```

Queue Processor worker loop

```

func worker(rdb *redis.Client, queueName string) {
    for {
        // blocking pop
        result, err := rdb.BRPop(ctx, 0, queueName).Result()
        if err != nil {
            log.Println("BRPop error:", err)
            time.Sleep(time.Second)
            continue
        }
        jobData := result[1]
        // process job...
    }
}

```

9.4 range

range คืออะไร?

range ใช้ร่วมกับ for เพื่อวนซ้ำผ่าน elements ของ array, slice, map, string, channel

range มีกี่แบบ?

- Slice/array: `for i, v := range slice`
- Map: `for k, v := range m`
- String: `for i, r := range str` (วนทีละ rune)
- Channel: `for v := range ch`
- สามารถใช้ `_` ละทิ้งค่าใดค่าหนึ่ง

ใช้อย่างไร ในกรณีไหน?

- วนผ่านผลลัพธ์จาก GORM (slice of models)
- อ่านข้อมูลจาก Redis hash / map
- ดึงข้อมูลจาก channel ใน queue processor
- แปลง struct slice เป็น map

หลักการทำงาน

1. range จะคืนค่า sequence ของ elements
2. แต่ละรอบกำหนดค่าลงตัวแปร
3. กับ map ไม่รับประกันลำดับ
4. กับ string วนทีละ rune (ไม่ใช่ byte)
5. กับ channel วนจนกว่า channel ปิด

Dataflow Diagram (Flowchart TB) - range with slice

```
graph TB
    Start([เริ่ม]) --> Init[for index, value := range collection]
    Init --> Next{มี element ถัดไป?}
    Next -- true --> Assign[กำหนด index, value]
    Assign --> Body[ทำ body]
    Body --> Next
    Next -- false --> End([จบ])
```

ตัวอย่างกับ GORM / Redis / Queue Processor

วน slice ของ model เพื่อ UPDATE

```
func UpdateProducts(db *gorm.DB, products []Product) error {
    for _, p := range products {
        if err := db.Model(&Product{}).Where("id = ?", p.ID).Updates(p).Error;
        err != nil {
            return err
        }
    }
    return nil
}
```

วน map เพื่อ INSERT ลง Redis hash

```
func StoreUserAttrs(ctx context.Context, rdb *redis.Client, userID string,
attrs map[string]interface{}) error {
    key := fmt.Sprintf("user:%s:attrs", userID)
    for field, value := range attrs {
        if err := rdb.HSet(ctx, key, field, value).Err(); err != nil {
            return err
        }
    }
    return nil
}
```

วน channel ใน Queue Processor (รับจาก Redis Pub/Sub)

```
func SubscribeQueue(ctx context.Context, pubsub *redis.PubSub) {
    ch := pubsub.Channel()
    for msg := range ch {
        // msg.Payload contains job data
        var job Job
        json.Unmarshal([]byte(msg.Payload), &job)
        processJob(job)
    }
}
```

9.6 label และการ break ออกจาก outer loop

label และการ break ออกจาก outer loop คืออะไร?

label คือการกำหนดชื่อให้กับ statement (เช่น for, switch, select) เพื่อใช้ `break label` หรือ `continue label` ออกจากหลายชั้นหรือข้าม iteration ของ outer loop

มีกี่แบบ?

- `break label` – ออกจากลูปที่ label ระบุ
- `continue label` – ข้ามไป iteration ถัดไปของลูปที่ label ระบุ
- `goto label` – (ไม่แนะนำ) กระโดดไปตำแหน่ง label

ใช้อย่างไร ในกรณีไหน?

- ค้นหาข้อมูลใน nested loops แล้วต้องการหยุดทั้งหมดเมื่อเจอ
- ประมวลผล nested struct (เช่น order -> items) และต้องการยกเลิกการประมวลผลทั้งหมดเมื่อ error
- ในการทำ SQL transaction ซ้อน loop: ถ้า error ต้อง rollback ทั้ง transaction

หลักการทำงาน

1. ประกาศ label ก่อน loop ที่ต้องการควบคุม
2. ภายใน loop ซ้อน ใช้ `break label` เพื่อออกจาก loop ที่มี label นั้นที่
3. `continue label` จะข้าม iteration ของ loop ที่มี label ไป iteration ถัดไป

Dataflow Diagram (Flowchart TB)

```
graph TB
    Start([เริ่ม]) --> OuterStart[OuterLoop: for ...]
    OuterStart --> InnerStart[InnerLoop: for ...]
    InnerStart --> Cond{พบเงื่อนไข break?}
    Cond -- ใช่ --> BreakLabel[break OuterLoop]
    Cond -- ไม่ใช่ --> ContinueInner[ทำ inner ต่อไป]
```

```
ContinueInner --> InnerStart
BreakLabel --> End([ออกจาก outer loop])
```

ตัวอย่างกับ GORM / Redis / Queue Processor

ค้นหา user ในหลายกลุ่ม แล้วหยุดเมื่อพบ (nested loops)

```
func FindUserInGroups(groups [][]User, targetID uint) (*User, bool) {
OuterLoop:
    for _, group := range groups {
        for _, user := range group {
            if user.ID == targetID {
                return &user, true
            }
            // ถ้ามีเงื่อนไขอื่นเช่น user disabled
            if user.Disabled {
                // ข้าม group นี้ทั้งหมด
                continue OuterLoop
            }
        }
    }
    return nil, false
}
```

ใน Queue Processor: ตรวจสอบข้อมูลหลายชั้นและ rollback transaction เมื่อ error

```
func ProcessOrderWithItems(order Order, items []Item) error {
    tx := db.Begin()
    defer func() {
        if r := recover(); r != nil {
            tx.Rollback()
        }
    }()

    // INSERT order
    if err := tx.Create(&order).Error; err != nil {
        tx.Rollback()
        return err
    }

MainLoop:
    for _, item := range items {
        // ตรวจสอบ stock
        var product Product
        if err := tx.First(&product, item.ProductID).Error; err != nil {
            tx.Rollback()
            break MainLoop
        }
    }
}
```

```

    if product.Stock < item.Quantity {
        // ไม่พอ stock -> rollback ทั้งหมด
        tx.Rollback()
        return fmt.Errorf("insufficient stock for product %d", product.ID)
    }
    // UPDATE stock
    if err := tx.Model(&Product{}).Where("id = ?", item.ProductID).
        Update("stock", gorm.Expr("stock - ?", item.Quantity)).Error; err
!= nil {
        tx.Rollback()
        break MainLoop
    }
    // INSERT order item
    if err := tx.Create(&item).Error; err != nil {
        tx.Rollback()
        break MainLoop
    }
}

// ถ้าผ่านทุก loop ให้ commit
return tx.Commit().Error
}

```

ใช้ label break ใน Redis pipeline loop

```

func UpdateManyAccounts(ctx context.Context, rdb *redis.Client, updates
map[string]int64) error {
MainLoop:
    for acc, delta := range updates {
        err := rdb.Watch(ctx, func(tx *redis.Tx) error {
            bal, err := tx.Get(ctx, acc).Int64()
            if err != nil && err != redis.Nil {
                return err
            }
            newBal := bal + delta
            if newBal < 0 {
                return fmt.Errorf("insufficient balance for %s", acc)
            }
            _, err = tx.TxPipelined(ctx, func(pipe redis.Pipeliner) error {
                pipe.Set(ctx, acc, newBal, 0)
                return nil
            })
            return err
        }, acc)
        if err != nil {
            if err == redis.TxFailedErr {
                // conflict, retry outer loop
                // ในที่นี้อาจทำ continue MainLoop เพื่อ retry
                continue MainLoop
            }
        }
    }
}

```

```
        return err
    }
}
return nil
}
```

สรุป

- **Queue Processor** ช่วยให้ระบบทำงานแบบ asynchronous, เพิ่มความเสถียรและ scalability
- **if-else, switch** ใช้ควบคุม logic ตามเงื่อนไขและประเภทของงาน
- **for loop, range** ใช้สำหรับ batch processing, retry และการวนผ่านข้อมูล
- **label** มีประโยชน์เมื่อต้องการควบคุม nested loops อย่างแม่นยำ โดยเฉพาะใน transaction ที่ต้อง rollback ทั้งชุด

สามารถนำตัวอย่างโค้ดไปปรับใช้กับ GORM และ Redis ร่วมกับ Queue Processor เพื่อสร้างระบบที่ robust และรองรับการทำงานที่มีความซับซ้อนได้

แหล่งอ้างอิง

- [Effective Go - Control structures](#)
- [GORM Documentation](#)
- [go-redis Documentation](#)
- [Redis Streams](#)

บทที่ 10: ฟังก์ชัน

10.1 การประกาศฟังก์ชัน

```
func add(x int, y int) int {
    return x + y
}
```

ถ้าพารามิเตอร์ชนิดเดียวกัน สามารถย่อ:

```
func add(x, y int) int {
    return x + y
}
```

10.2 การคืนค่าหลายค่า

Go รองรับการคืนค่าหลายค่าได้โดยตรง:

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}
```

เรียกใช้:

```
result, err := divide(10, 2)
if err != nil {
    fmt.Println("Error:", err)
} else {
    fmt.Println("Result:", result)
}
```

10.3 Named return values

สามารถตั้งชื่อค่าที่คืนได้:

```
func split(sum int) (x, y int) {
    x = sum * 4 / 9
    y = sum - x
    return // naked return
}
```

10.4 ฟังก์ชันแบบ variadic (รับพารามิเตอร์ไม่จำกัด)

```
func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}
// เรียกใช้: sum(1,2,3,4)
```

10.5 ฟังก์ชันเป็นค่า (first-class functions)

ฟังก์ชันสามารถถูกกำหนดให้กับตัวแปร, ส่งเป็นพารามิเตอร์, คืนค่าเป็นผลลัพธ์

```
var fn func(int) int = func(x int) int { return x * 2 }
result := fn(5) // 10
```

10.6 defer

defer คือคำสั่งในภาษา Go ที่ใช้ **เลื่อนการทำงานของฟังก์ชัน** ออกไปจนกว่าฟังก์ชันรอบนอก (ฟังก์ชันที่ประกาศ **defer**) จะจบการทำงาน ไม่ว่าจะจบแบบปกติ (return) หรือเกิด panic

defer ทำให้ฟังก์ชันถูกเรียกหลังจากฟังก์ชัน enclosing จบการทำงาน (ใช้สำหรับ cleanup)

หลักการทำงาน

- คำสั่งที่อยู่ภายใต้ **defer** จะถูกเก็บไว้ในสแต็ก (stack) และทำงานในลำดับ **LIFO** (Last In, First Out) – ตัวที่ประกาศทีหลังจะถูกเรียกก่อน
- นิยมใช้เพื่อปิดทรัพยากร (ไฟล์, database connection, mutex unlock) เพื่อป้องกันการรั่วไหล

ตัวอย่าง

```
func readFile() {
    f, err := os.Open("data.txt")
    if err != nil {
        return
    }
    defer f.Close() // ปิดไฟล์เมื่อฟังก์ชันจบ

    // อ่านข้อมูล...
}
```

panic และ recover

- panic** : หยุดการทำงานปกติและเริ่ม unwind stack
- recover** : ใช้ใน defer เพื่อจับ panic และควบคุมการทำงานต่อ

```
func safeDivide(a, b int) {
    defer func() {
        if r := recover(); r != nil {
            fmt.Println("Recovered from panic:", r)
        }
    }()
    if b == 0 {
        panic("division by zero")
    }
    fmt.Println(a / b)
}
```

ข้อควรระวัง: panic ควรใช้ในกรณีผิดพลาดรุนแรงเท่านั้น ไม่ใช่แทน error handling

10.7 ฟังก์ชัน init

แต่ละแพ็คเกจ ได้รับ `init()` ฟังก์ชัน ซึ่งจะถูกเรียกอัตโนมัติเมื่อแพ็คเกจถูกโหลด (ก่อน main)

```
func init() {  
    fmt.Println("initializing package")  
}
```

บทที่ 11: แพ็คเกจและการนำเข้า

11.1 แพ็คเกจ (Packages)

Go จัดระเบียบโค้ดเป็นแพ็คเกจ แต่ละไฟล์ .go ต้องขึ้นต้นด้วย `package <name>` ชื่อแพ็คเกจควรเป็นตัวพิมพ์เล็ก

- `package main` : สำหรับโปรแกรมที่รันได้ (executable)
- แพ็คเกจอื่นๆ : สำหรับ library

11.2 การนำเข้า (import)

```
import "fmt"  
import "math/rand"
```

หรือแบบ grouped:

```
import (  
    "fmt"  
    "math/rand"  
)
```

11.3 การตั้งชื่อให้กับ import (alias)

```
import (  
    "fmt"  
    r "math/rand" // alias  
)
```

11.4 Blank import

ใช้ `_` เพื่อนำเข้าเฉพาะ side-effect (เช่น เรียก init) โดยไม่ต้องใช้ฟังก์ชัน

```
import _ "image/png" // ลงทะเบียนตัวถอดรหัส PNG
```


11.5 การเข้าถึงสมาชิก (exported vs unexported)

- ตัวพิมพ์ใหญ่: exported (public) สามารถเข้าถึงได้จากแพ็คเกจอื่น
- ตัวพิมพ์เล็ก: unexported (private) เข้าถึงได้เฉพาะภายในแพ็คเกจ

```
// mymath/mymath.go
package mymath

func Add(a, b int) int { // Exported
    return a + b
}

func sub(a, b int) int { // Unexported
    return a - b
}
```

11.6 การจัดโครงสร้างโปรเจกต์ด้วยแพ็คเกจ

```
myproject/
├── go.mod
├── main.go
├── utils/
│   └── stringutils.go
└── models/
    └── user.go
```

ใน `main.go`:

```
import (
    "myproject/utils"
    "myproject/models"
)
```

11.7 การจัดระเบียบแพ็คเกจมาตรฐาน

- `fmt` : การจัดรูปแบบ I/O
- `io / os` : การอ่านเขียนไฟล์
- `net/http` : HTTP client/server
- `encoding/json` : JSON
- `sync` : concurrency primitives
- `time` : เวลา
- `context` : context management

11.8 การสร้างแพ็คเกจของตัวเอง

บทที่ 12: การเริ่มต้นทำงานของแพ็คเกจ

12.1 ลำดับการเริ่มต้น

1. แพ็คเกจที่ถูก import จะถูกเริ่มต้นก่อน (แบบ recursive)
2. ตัวแปรระดับแพ็คเกจถูกกำหนดค่า (initialized)
3. ฟังก์ชัน `init()` ถูกเรียก (เรียงตามลำดับการประกาศในไฟล์)
4. ถ้าเป็น `package main` ฟังก์ชัน `main()` จะถูกเรียก

12.2 ฟังก์ชัน init

สามารถมีหลาย init ในแพ็คเกจเดียวกัน หรือแม้แต่หลาย init ในไฟล์เดียวกัน

```
var config string

func init() {
    config = loadConfig()
    fmt.Println("init called")
}
```

12.3 การควบคุมลำดับ init

ถ้าต้องการให้ init ทำงานก่อน ควรนำเข้าแพ็คเกจนั้นโดย blank import

12.4 ตัวอย่างการเริ่มต้น

ไฟล์ `a.go`:

```
package main

import "fmt"

var a = b + 1
var b = 1

func init() {
    fmt.Println("init 1", a, b)
}

func init() {
    fmt.Println("init 2")
}

func main() {
    fmt.Println("main", a, b)
}
```

ผลลัพธ์:

```
init 1 2 1
init 2
main 2 1
```

ตัวแปรจะถูกกำหนดค่าตามลำดับการประกาศในไฟล์

12.5 การใช้ init เพื่อตั้งค่าแพ็คเกจ

เหมาะสำหรับการตั้งค่า connection pools, การลงทะเบียน driver, การโหลด configuration

```
package database

import "database/sql"

var db *sql.DB

func init() {
    // เปิด connection pool
    db, _ = sql.Open("mysql", "user:pass@/dbname")
}
```

12.6 การหลีกเลี่ยงการใช้งาน init มากเกินไป

init ทำให้ยากต่อการทดสอบและการควบคุมลำดับ หากเป็นไปได้ควรใช้ฟังก์ชันเริ่มต้นที่เรียกอย่างชัดเจน (เช่น New) แทน

บทที่ 13: การสร้างชนิดข้อมูลใหม่ (Types)

13.1 type keyword

ใช้สร้างชนิดข้อมูลใหม่จากชนิดเดิม

```
type Celsius float64
type Fahrenheit float64

func (c Celsius) ToFahrenheit() Fahrenheit {
    return Fahrenheit(c*9/5 + 32)
}
```

Celsius และ Fahrenheit เป็นคนละชนิดกัน แม้ underlying type เหมือนกัน ต้องแปลงอย่างชัดเจน

13.2 struct

struct คือการรวมตัวแปรหลายๆ ตัวเข้าด้วยกัน

```
type Person struct {  
    Name string  
    Age  int  
    Email string  
}
```

การสร้าง instance:

```
p1 := Person{"John", 30, "john@example.com"} // ตามลำดับ field  
p2 := Person{Name: "Jane", Age: 25}          // ระบุชื่อ field  
p3 := new(Person) // สร้าง pointer  
p3.Name = "Bob"
```

13.3 struct fields และ tags

Tags ใช้สำหรับ metadata เช่น การ encode/decode JSON

```
type User struct {  
    ID      int    `json:"id"`  
    Username string `json:"username"`  
    Password string `json:"-"` // ไม่แสดงใน JSON  
}
```

13.4 การสืบทอด (embedding) แทน inheritance

Go ไม่มี inheritance แต่ใช้ composition ผ่าน embedding

```
type Animal struct {  
    Name string  
}  
  
func (a Animal) Speak() string {  
    return "???"  
}  
  
type Dog struct {  
    Animal // embedded field  
    Breed string  
}  
  
func (d Dog) Speak() string {  
    return "Woof!"  
}
```

Dog จะมี field Name และ method Speak (สามารถ override ได้)

13.5 ชนิดข้อมูลแบบ type alias

Alias (Go 1.9+) ทำให้มีชื่อใหม่ที่เทียบเท่ากับชนิดเดิม ใช้ในกรณี refactoring

```
type MyInt = int // alias, MyInt และ int ใช้แทนกันได้
```

ความแตกต่าง: type definition (type MyInt int) สร้างชนิดใหม่, alias แค่ชื่ออื่น

Dataflow Diagram: Embedding in Go

Embedding (also called composition) in Go allows a struct to include another struct as an anonymous field, making its fields and methods directly accessible on the outer struct. This promotes code reuse without classical inheritance.

Below is a **dataflow diagram** (flowchart TB) that illustrates how embedding works when a method is called on the outer struct.

Diagram: Method Promotion with Embedding

```
graph TD
    Start([Start: dog.Speak()]) --> CheckOuter{Does Dog have its own Speak method?}
    CheckOuter -- Yes --> CallOuter[Call Dog.Speak]
    CheckOuter -- No --> Promote[Promote Animal.Speak]
    Promote --> CallAnimal[Call Animal.Speak]
    CallOuter --> Return[Return result]
    CallAnimal --> Return
    Return --> End([End])
```

Explanation

1. **Outer struct** (**Dog**) embeds an inner struct (**Animal**) as an anonymous field.
2. When a method is called on the outer struct (**dog.Speak()**), Go first checks if the outer struct defines that method directly.
 - If **yes**, that method is invoked (overriding).
 - If **no**, Go looks for the method in the embedded struct(s) and promotes it.
3. The promoted method is called as if it were part of the outer struct.
4. The result is returned to the caller.

Example Code

```

type Animal struct {
    Name string
}

func (a Animal) Speak() string {
    return "Animal sound"
}

type Dog struct {
    Animal // embedded
    Breed string
}

func (d Dog) Speak() string {
    return "Woof!" // overrides Animal.Speak
}

func main() {
    dog := Dog{Animal: Animal{Name: "Max"}, Breed: "Golden"}
    fmt.Println(dog.Speak()) // Woof!
}

```

Field Access Flow

For fields, the same promotion applies. If a field is not found in the outer struct, Go looks into embedded structs.

```

graph TB
    StartField([dog.Name]) --> CheckOuterField{Does Dog have Name field?}
    CheckOuterField -- No --> LookEmbedded[Look in embedded Animal]
    LookEmbedded --> Found[Return Animal.Name]
    CheckOuterField -- Yes --> ReturnOuter[Return Dog.Name]
    Found --> EndField([End])
    ReturnOuter --> EndField

```

บทที่ 14: เมธอด (Methods)

14.1 การกำหนดเมธอด

เมธอดคือฟังก์ชันที่มี receiver (ตัวรับ)

```

type Rectangle struct {
    Width, Height float64
}

func (r Rectangle) Area() float64 {

```

```

    return r.Width * r.Height
}

func (r *Rectangle) Scale(f float64) {
    r.Width *= f
    r.Height *= f
}

```

14.2 Pointer receiver vs Value receiver

- **Value receiver:** ทำงานกับ copy เปลี่ยนแปลงไม่กระทบตัวเดิม
- **Pointer receiver:** ทำงานกับตัวแปรเดิม เปลี่ยนแปลงได้ และประหยัดหน่วยความจำสำหรับ struct ใหญ่

กฎ: ถ้าเมธอดต้องแก้ไข receiver ให้ใช้ pointer receiver

14.3 เมธอดกับ non-struct

สามารถกำหนดเมธอดให้กับชนิดใดก็ได้ (ยกเว้นชนิดจากแพ็คเกจอื่นและชนิดพื้นฐานที่ไม่ใช่ alias)

```

type MyInt int

func (m MyInt) Double() MyInt {
    return m * 2
}

```

14.4 การเรียกเมธอด

```

r := Rectangle{10, 5}
area := r.Area()      // value receiver
r.Scale(2)             // pointer receiver

```

14.5 การแปลงระหว่าง value และ pointer อัตโนมัติ

Go จะแปลงให้อัตโนมัติเมื่อเรียกเมธอด:

- ถ้ามี pointer receiver แต่เรียกจาก value: Go จะส่ง address ให้ (ถ้า value addressable)
- ถ้ามี value receiver แต่เรียกจาก pointer: Go จะ dereference ให้

14.6 เมธอดและการสืบทอด (embedding)

เมื่อ embed struct ไปอีก struct เมธอดของ struct ที่ถูก embed จะถูกเลื่อนขึ้นมา

```

type Animal struct{}

func (a Animal) Speak() string {

```

```

    return "..."
```

```

}

type Dog struct {
    Animal
}

d := Dog{}
d.Speak() // เรียก Speak ของ Animal
```

14.7 เมธอดและ interface

เมธอดเป็นสิ่งที่ทำให้ type implement interface ได้

บทที่ 15: พอยน์เตอร์ (Pointer)

15.1 พอยน์เตอร์คืออะไร?

พอยน์เตอร์คือตัวแปรที่เก็บ address ของตัวแปรอื่น

```

var x int = 10
var p *int = &x // p ชี้ไปที่ x
fmt.Println(*p) // 10 (dereference)
*p = 20         // เปลี่ยนค่า x ผ่าน pointer
```

15.2 การประกาศ pointer

- `&` : address-of operator
- `*` : dereference operator (และใช้ใน type declaration)

15.3 Zero value ของ pointer

pointer ที่ไม่ได้กำหนดค่าจะเป็น `nil`

15.4 การใช้ pointer กับฟังก์ชัน

เพื่อให้ฟังก์ชันสามารถแก้ไขตัวแปรนอกฟังก์ชันได้

```

func zeroVal(val int) {
    val = 0
}

func zeroPtr(ptr *int) {
    *ptr = 0
}

x := 5
```



```
zeroVal(x)      // x ยังเป็น 5
zeroPtr(&x)     // x กลายเป็น 0
```

15.5 การใช้ pointer กับ struct

เพื่อประสิทธิภาพ (ไม่ต้อง copy struct ทั้งตัว) และการแก้ไข

```
func (p *Person) UpdateAge(age int) {
    p.Age = age
}
```

15.6 new() และ make()

- `new(T)` : จัดสรรหน่วยความจำสำหรับ type T, คืน pointer *T, zero value
- `make(T, ...)` : ใช้กับ slice, map, channel เท่านั้น; คืน initialized (ไม่ใช่ zero) value

15.7 Pointer arithmetic

Go ไม่สนับสนุน pointer arithmetic (ต่างจาก C) เพื่อความปลอดภัย

15.8 ข้อควรระวัง

- nil pointer dereference จะทำให้ panic
- การใช้ pointer เมื่อจำเป็นเท่านั้น (การแก้ไข, ประสิทธิภาพ)

บทที่ 16: อินเทอร์เฟซ (Interfaces)

16.1 อินเทอร์เฟซคืออะไร?

อินเทอร์เฟซคือชุดของ method signatures ชนิดข้อมูลใดๆ ที่ implement method ครบทุก method ในอินเทอร์เฟซ จะถือว่าอินสแตนซ์นั้น implement อินเทอร์เฟซนั้นโดยปริยาย (implicit implementation)

```
type Speaker interface {
    Speak() string
}

type Dog struct{}

func (d Dog) Speak() string {
    return "Woof!"
}

var s Speaker = Dog{}
fmt.Println(s.Speak())
```

16.2 Empty interface

`interface{}` (หรือ `any` ใน Go 1.18+) เป็นอินเทอร์เฟซที่ไม่มี method ใดๆ ทุกชนิด implement มันได้

```
var anything interface{}
anything = 42
anything = "hello"
anything = Dog{}
```

ใช้เมื่อต้องการรับค่าทุกชนิด (คล้าย dynamic type)

16.3 Type assertion

ใช้เพื่อดึงค่า concrete type ออกจาก interface

```
var s interface{} = "hello"
str, ok := s.(string)
if ok {
    fmt.Println(str)
}
```

หากไม่ตรวจสอบ ok จะ panic ถ้าชนิดไม่ตรง

16.4 Type switch

ตรวจสอบชนิดของ interface

```
switch v := s.(type) {
case string:
    fmt.Println("string:", v)
case int:
    fmt.Println("int:", v)
default:
    fmt.Println("unknown")
}
```

16.5 การ embed interface

```
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}
```

```
type ReadWriter interface {
    Reader
    Writer
}
```

16.6 Interface และ nil

ตัวแปร interface มีค่า nil เมื่อทั้ง type และ value เป็น nil

```
var s Speaker // nil interface
fmt.Println(s == nil) // true
```

แต่ถ้าเก็บ pointer ที่เป็น nil ไว้:

```
var d *Dog = nil
var s Speaker = d // s ไม่เป็น nil เพราะ type เป็น *Dog
```

16.7 ข้อดีของ interface

- ช่วยให้โค้ดยืดหยุ่น เปลี่ยน implementation ได้ง่าย
- สนับสนุน dependency injection
- เขียน test ได้ง่าย (ใช้ mock)

16.8 Interface ที่ใช้บ่อย

- `io.Reader`, `io.Writer`
- `http.Handler`
- `error` (มี method `Error() string`)

ภาคที่ 3: การจัดการโปรเจกต์และโครงสร้างข้อมูลขั้นสูง

(บทที่ 17–23)

แผนภาพแสดงความสัมพันธ์ของเนื้อหา

```
flowchart TD
    subgraph ProjectManagement["1. การจัดการโปรเจกต์ (บทที่ 17-18)"]
        direction TB
        A1["Go Modules (บทที่ 17)<br/>go.mod, go.sum<br/>module, require, replace<br/>go mod init/tidy/vendor"]
        A2["Go Module Proxies (บทที่ 18)<br/>GOPROXY, GOSUMDB<br/>GOPRIVATE, Athens<br/>ความปลอดภัยและความเร็ว"]
    end
```

```

    A1 --> A2
end

subgraph Testing["2. การทดสอบ (บทที่ 19)"]
    direction TB
    B1["Unit Tests<br/>_test.go, TestXxx(t *testing.T)<br/>t.Run(), table-  
driven tests<br/>t.Helper(), t.Parallel()"]
    B2["Test Coverage<br/>go test -cover<br/>go test -coverprofile<br/>go  
tool cover -html"]
    B3["Test Doubles<br/>Mock, Stub, Fake<br/>interface-based mocking"]

    B1 --> B2
    B2 --> B3
end

subgraph DataStructures["3. โครงสร้างข้อมูลขั้นสูง (บทที่ 20-22)"]
    direction TB
    C1["Arrays (บทที่ 20)<br/>[n]T, fixed size<br/>value type, compile-time  
size<br/>rarely used directly"]
    C2["Slices (บทที่ 21)<br/>[]T, dynamic size<br/>len, cap, append,  
copy<br/>backed by array, reference type"]
    C3["Maps (บทที่ 22)<br/>map[K]V, hash table<br/>make(), delete, comma ok  
idiom<br/>unordered, not safe for concurrent"]

    C1 --> C2
    C2 --> C3
end

subgraph ErrorHandling["4. การจัดการข้อผิดพลาด (บทที่ 23)"]
    direction TB
    D1["Error Interface<br/>error interface<br/>errors.New, fmt.Errorf"]
    D2["Error Wrapping<br/>%w, errors.Unwrap<br/>errors.Is,  
errors.As<br/>error chains"]
    D3["Best Practices<br/>check errors early<br/>don't ignore  
errors<br/>custom error types"]

    D1 --> D2
    D2 --> D3
end

subgraph Flow["ลำดับการเรียนรู้"]
    direction LR
    F1[จัดการโปรเจกต์] --> F2[เขียนทดสอบ] --> F3[ใช้โครงสร้างข้อมูล] --> F4[จัดการข้อผิดพลาด]
end

ProjectManagement --> Testing
Testing --> DataStructures
DataStructures --> ErrorHandling
Flow --> ProjectManagement

```

คำอธิบายภาษาไทย (แบบละเอียด)

ภาคที่ 3 ครอบคลุมบทที่ 17–23 โดยมีเนื้อหาแบ่งเป็น 4 ช่วงหลัก ตามแผนภาพด้านบน ซึ่งแสดงลำดับการเรียนรู้ที่ต่อเนื่องกัน

1. การจัดการโปรเจกต์ (บทที่ 17–18)

บทที่ 17: Go Modules – การจัดการโปรเจกต์สมัยใหม่

เป้าหมาย: จัดการ dependencies (แพ็คเกจภายนอก) และ versioning

Go Modules คืออะไร?

Go Modules เป็นระบบ dependency management อย่างเป็นทางการของ Go เริ่มตั้งแต่ Go 1.11 (stable ตั้งแต่ 1.16) แทนที่ GOPATH workspace แบบเดิม

โครงสร้างไฟล์

```
myproject/
├── go.mod          # ไฟล์หลัก ระบุ module name และ dependencies
├── go.sum          # checksum ของ dependencies (ความปลอดภัย)
├── main.go
└── internal/      # โค้ดเฉพาะของโปรเจกต์
```

คำสั่งพื้นฐาน

```
# เริ่มต้นโปรเจกต์ใหม่
go mod init github.com/username/myproject

# เพิ่ม dependency อัตโนมัติ (เมื่อ import ในโค้ด)
go mod tidy

# ดาวน์โหลด dependencies
go mod download

# แสดง dependency graph
go mod graph

# แก้ไข go.mod โดยตรง
go mod edit -require=github.com/gorilla/mux@v1.8.0

# ลบ unused dependencies
go mod tidy -v

# สร้าง vendor folder (copy dependencies เข้าโปรเจกต์)
go mod vendor
```

ตัวอย่าง go.mod

```
module github.com/username/myapp

go 1.21

require (
    github.com/gin-gonic/gin v1.9.1
    github.com/go-sql-driver/mysql v1.7.1
    gorm.io/gorm v1.25.5
)

require (
    github.com/bytedance/sonic v1.9.1 // indirect
    github.com/jinzhu/inflection v1.0.0 // indirect
)

replace github.com/old/package => github.com/new/package v1.0.0

exclude github.com/broken/package v1.2.3
```

คำอธิบายแต่ละส่วน:

- **module**: ชื่อของโปรเจกต์ (ปกติเป็น repository URL)
- **go**: เวอร์ชัน Go ที่ใช้
- **require**: dependencies ที่โปรเจกต์ต้องการ
- **indirect**: dependency ที่ถูกนำเข้ามาทางอ้อม
- **replace**: ใช้แทนที่แพ็คเกจด้วยเวอร์ชันหรือ source อื่น
- **exclude**: ห้ามใช้เวอร์ชันที่ระบุ

ตัวอย่างการใช้งานจริง

1. สร้างโปรเจกต์ใหม่

```
# สร้างโฟลเดอร์
mkdir my-rest-api
cd my-rest-api

# เริ่มต้น Go module
go mod init github.com/mycompany/my-rest-api

# สร้างไฟล์ main.go
cat > main.go << 'EOF'
package main

import (
    "fmt"
    "github.com/gin-gonic/gin"
```

```

)

func main() {
    r := gin.Default()
    r.GET("/ping", func(c *gin.Context) {
        c.JSON(200, gin.H{"message": "pong"})
    })
    r.Run() // listen and serve on 0.0.0.0:8080
}
EOF

# รัน go mod tidy จะดึง gin มาให้อัตโนมัติ
go mod tidy

# ดูผลลัพธ์ go.mod
cat go.mod

```

ผลลัพธ์ go.mod:

```

module github.com/mycompany/my-rest-api

go 1.21

require github.com/gin-gonic/gin v1.9.1

require (
    github.com/bytedance/sonic v1.9.1 // indirect
    github.com/chenzhuoyu/base64x v0.0.0-20221115062448-fe3a3abad311 //
indirect
    github.com/gabriel-vasile/mimetype v1.4.2 // indirect
    // ... indirect dependencies อื่นๆ
)

```

2. การอัปเดต dependency

```

# ดูเวอร์ชันล่าสุด
go list -u -m all

# อัปเดตแพ็คเกจเฉพาะ
go get -u github.com/gin-gonic/gin

# อัปเดตเป็นเวอร์ชันเฉพาะ
go get github.com/gin-gonic/gin@v1.9.0

# อัปเดตทั้งหมด
go get -u ./...

```

3. การใช้ replace สำหรับ local development

```
// go.mod
module github.com/mycompany/myapp

go 1.21

require (
    github.com/mycompany/mylib v1.0.0
)

// ใช้ local path แทน (ตอนพัฒนา)
replace github.com/mycompany/mylib => ../mylib
```

4. Semantic Import Versioning (SIV)

Go ใช้ major version ใน import path เมื่อ major version > 1

```
// v1 (ไม่ต้องระบุ /v1)
import "github.com/user/pkg"

// v2 ขึ้นไป ต้องระบุใน path
import "github.com/user/pkg/v2"
import "github.com/user/pkg/v3"
```

บทที่ 18: Go Module Proxies

เป้าหมาย: เพิ่มความเร็วและความปลอดภัยในการดาวน์โหลด dependencies

Go Module Proxy คืออะไร?

Go Module Proxy เป็นเซิร์ฟเวอร์ที่เก็บ cache ของ Go modules ช่วยให้:

- ดาวน์โหลด dependencies ได้เร็วขึ้น (cache ในประเทศ)
- ทำงานในองค์กรที่มี firewall
- ป้องกันการลบ module ออกจาก GitHub (sum.golang.org)
- ตรวจสอบความถูกต้องของ module

ตัวแปรสภาพแวดล้อมที่สำคัญ

```
# ตั้งค่า GOPROXY (ใช้ proxy หลายตัว)
export GOPROXY=https://proxy.golang.org,direct

# ใช้ private proxy
export GOPROXY=https://goproxy.io,https://proxy.golang.org,direct
```



```
# ตั้งค่า GOSUMDB (ตรวจสอบ checksum)
export GOSUMDB=sum.golang.org

# ตั้งค่า GOPRIVATE (private repositories)
export GOPRIVATE=github.com/mycompany/*,gitlab.com/private/*

# ปิด checksum สำหรับ private repo
export GONOSUMDB=github.com/mycompany/*
```

ตัวอย่างการใช้งานจริง

1. ตั้งค่า proxy สำหรับองค์กร

```
# .bashrc หรือ .zshrc
export GOPROXY=https://goproxy.company.com,https://proxy.golang.org,direct
export GOPRIVATE=github.com/mycompany/*
export GONOSUMDB=github.com/mycompany/*
export GOSUMDB=sum.golang.org
```

2. ใช้ Athens (self-hosted proxy)

Athens เป็น Go module proxy ที่สามารถติดตั้งเองได้

```
# docker-compose.yml
version: '3'
services:
  athens:
    image: gomods/athens:latest
    ports:
      - "3000:3000"
    environment:
      - ATHENS_STORAGE_TYPE=disk
      - ATHENS_DISK_STORAGE_ROOT=/var/lib/athens
    volumes:
      - ./athens-storage:/var/lib/athens
```

```
# เริ่มต้น Athens
docker-compose up -d

# ตั้งค่า GOPROXY
export GOPROXY=http://localhost:3000
```

3. การทำงานของ GOPROXY

1. go build/run/test
↓
2. ตรวจสอบ GOPROXY
↓
3. proxy.golang.org มี cache ไหม?
 - ├─ มี → ดาวน์โหลดจาก proxy
 - └─ ไม่มี → ดึงจาก VCS (GitHub) → เก็บ cache
↓
4. ตรวจสอบ GOSUMDB
↓
5. ดาวน์โหลด module

4. ปัญหาที่พบบ่อยและ解决方法

```
# ปัญหา: checksum mismatch สำหรับ private repo
# วิธีแก้: ตั้ง GONOSUMDB
export GONOSUMDB=github.com/private/*

# ปัญหา: proxy ไม่มี module (404)
# วิธีแก้: ใช้ direct
export GOPROXY=https://proxy.golang.org,direct

# ปัญหา: module ถูกลบจาก GitHub
# วิธีแก้: ใช้ proxy ที่มี cache (proxy.golang.org มี cache)
# หรือใช้ vendor
go mod vendor
```

2. การทดสอบหน่วย (บทที่ 19)

เป้าหมาย: เขียน test เพื่อให้โค้ดมีความมั่นใจและบำรุงรักษาง่าย

บทที่ 19: Unit Tests

โครงสร้างการทดสอบ

```
// calculator/calculator.go
package calculator

func Add(a, b int) int {
    return a + b
}

func Divide(a, b int) (int, error) {
    if b == 0 {
        return 0, ErrDivisionByZero
    }
}
```

```
    return a / b, nil
}
```

```
// calculator/calculator_test.go
package calculator

import (
    "testing"
)

// 1. Test function (ต้องขึ้นต้นด้วย Test)
func TestAdd(t *testing.T) {
    result := Add(2, 3)
    expected := 5

    if result != expected {
        t.Errorf("Add(2,3) = %d; want %d", result, expected)
    }
}

// 2. Table-driven tests (แนะนำ)
func TestDivide(t *testing.T) {
    tests := []struct {
        name      string
        a, b      int
        expected  int
        hasError  bool
    }{
        {"positive numbers", 10, 2, 5, false},
        {"negative numbers", -10, 2, -5, false},
        {"division by zero", 10, 0, 0, true},
        {"zero divided", 0, 5, 0, false},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result, err := Divide(tt.a, tt.b)

            if tt.hasError {
                if err == nil {
                    t.Errorf("expected error but got none")
                }
                return
            }

            if err != nil {
                t.Errorf("unexpected error: %v", err)
            }

            if result != tt.expected {
```

```

        t.Errorf("Divide(%d,%d) = %d; want %d",
            tt.a, tt.b, result, tt.expected)
    }
}

// 3. Helper function
func testHelper(t *testing.T, msg string) {
    t.Helper() // บรรทัดนี้บอกว่าเป็น helper (error จะแสดง caller)
    if msg == "" {
        t.Fatal("message cannot be empty")
    }
}

```

ตัวอย่างการใช้งานจริง

1. การทดสอบ HTTP Handler

```

// handler/user_handler.go
package handler

import (
    "encoding/json"
    "net/http"
)

type User struct {
    ID    int    `json:"id"`
    Name string `json:"name"`
}

type UserHandler struct {
    userService UserService
}

func (h *UserHandler) GetUser(w http.ResponseWriter, r *http.Request) {
    // รับ ID จาก URL
    id := r.URL.Query().Get("id")
    if id == "" {
        http.Error(w, "id required", http.StatusBadRequest)
        return
    }

    user, err := h.userService.GetUser(id)
    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }
}

```

```

w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(user)
}

```

```

// handler/user_handler_test.go
package handler

import (
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "testing"
)

// Mock UserService
type mockUserService struct {
    getUserFunc func(id string) (*User, error)
}

func (m *mockUserService) GetUser(id string) (*User, error) {
    return m.getUserFunc(id)
}

func TestGetUser(t *testing.T) {
    tests := []struct {
        name          string
        userID         string
        mockService    func() UserService
        expectedStatus int
        expectedUser   *User
    }{
        {
            name: "success",
            userID: "1",
            mockService: func() UserService {
                return &mockUserService{
                    getUserFunc: func(id string) (*User, error) {
                        return &User{ID: 1, Name: "John"}, nil
                    },
                },
            },
            expectedStatus: http.StatusOK,
            expectedUser:   &User{ID: 1, Name: "John"},
        },
        {
            name: "user not found",
            userID: "999",
            mockService: func() UserService {
                return &mockUserService{
                    getUserFunc: func(id string) (*User, error) {

```

```

        return nil, ErrUserNotFound
    },
    },
    },
    expectedStatus: http.StatusNotFound,
    expectedUser:   nil,
},
{
    name:           "missing id",
    userID:         "",
    mockService:    func() UserService { return &mockUserService{} },
    expectedStatus: http.StatusBadRequest,
    expectedUser:   nil,
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        // สร้าง handler
        handler := &UserHandler{
            userService: tt.mockService(),
        }

        // สร้าง request
        req := httptest.NewRequest("GET", "/user?id="+tt.userID, nil)
        w := httptest.NewRecorder()

        // เรียก handler
        handler.GetUser(w, req)

        // ตรวจสอบ status code
        if w.Code != tt.expectedStatus {
            t.Errorf("status = %d; want %d", w.Code, tt.expectedStatus)
        }

        // ตรวจสอบ response body
        if tt.expectedUser != nil {
            var user User
            json.NewDecoder(w.Body).Decode(&user)
            if user.ID != tt.expectedUser.ID {
                t.Errorf("user.ID = %d; want %d", user.ID,
tt.expectedUser.ID)
            }
        }
    })
}
}

```

2. การใช้ Subtests และ Parallel

```

func TestDatabaseQueries(t *testing.T) {
    // setup database (once)
    db := setupTestDB(t)
    defer db.Close()

    tests := []struct {
        name string
        query string
        expected int
    }{
        {"count users", "SELECT COUNT(*) FROM users", 10},
        {"count orders", "SELECT COUNT(*) FROM orders", 25},
    }

    for _, tt := range tests {
        tt := tt // capture range variable (Go 1.21+ ไม่ต้องแล้ว)
        t.Run(tt.name, func(t *testing.T) {
            t.Parallel() // รัน parallel

            var count int
            err := db.QueryRow(tt.query).Scan(&count)
            if err != nil {
                t.Fatal(err)
            }
            if count != tt.expected {
                t.Errorf("count = %d; want %d", count, tt.expected)
            }
        })
    }
}

```

3. Test Coverage

```

# ดู coverage ใน terminal
go test -cover

# สร้าง coverage profile
go test -coverprofile=coverage.out

# ดู coverage ใน browser
go tool cover -html=coverage.out

# ดูเฉพาะบาง package
go test -cover ./internal/...

# ตั้งค่า threshold (ใน CI)
go test -cover -coverprofile=coverage.out ./...
go tool cover -func=coverage.out | grep total
# total: (statements) 85.2%

```

4. Test Main (Setup/Teardown ทั้ง package)

```
// package_test.go
package mypackage

import (
    "os"
    "testing"
)

// TestMain รันก่อน test ทั้งหมด
func TestMain(m *testing.M) {
    // setup (รันก่อน test)
    setup()

    // รัน test
    code := m.Run()

    // teardown (รันหลัง test)
    teardown()

    os.Exit(code)
}

func setup() {
    // ตั้งค่า database connection, mock server, etc.
}

func teardown() {
    // clean up
}
```

5. การใช้ testdata folder

```
myproject/
├── internal/
│   └── parser/
│       ├── parser.go
│       ├── parser_test.go
│       └── testdata/          # โฟลเดอร์สำหรับ test
│           ├── valid.json
│           ├── invalid.json
│           └── sample.csv
```

```
func TestParseJSON(t *testing.T) {
    // อ่านไฟล์จาก testdata
```



```

data, err := os.ReadFile("testdata/valid.json")
if err != nil {
    t.Fatal(err)
}

result, err := ParseJSON(data)
if err != nil {
    t.Fatal(err)
}

// assert...
}

```

3. โครงสร้างข้อมูลขั้นสูง (บทที่ 20–22)

บทที่ 20: Arrays (อาเรย์)

เป้าหมาย: เข้าใจโครงสร้างข้อมูลแบบ fixed-size

ลักษณะสำคัญ

- **Fixed size:** ขนาดคงที่ กำหนดตอน compile time
- **Value type:** การ assign หรือส่งเป็น argument จะ copy ทั้ง array
- **Contiguous memory:** เก็บข้อมูลต่อเนื่องในหน่วยความจำ

ตัวอย่างการใช้งาน

```

package main

import "fmt"

func main() {
    // 1. ประกาศ array
    var arr1 [5]int           // [0 0 0 0 0]
    arr2 := [3]string{"a", "b", "c"} // [a b c]
    arr3 := [...]int{1, 2, 3}    // compiler นับขนาดอัตโนมัติ

    fmt.Println(arr1, arr2, arr3)

    // 2. เข้าถึงและแก้ไข
    arr1[0] = 10
    arr1[1] = 20
    fmt.Println(arr1[0]) // 10

    // 3. array เป็น value type (copy)
    original := [3]int{1, 2, 3}
    copy := original
    copy[0] = 100
}

```

```

fmt.Println(original) // [1 2 3] (ไม่เปลี่ยน)
fmt.Println(copy)     // [100 2 3]

// 4. วนลูป array
nums := [5]int{10, 20, 30, 40, 50}

// แบบ index
for i := 0; i < len(nums); i++ {
    fmt.Println(nums[i])
}

// แบบ range
for index, value := range nums {
    fmt.Printf("index=%d, value=%d\n", index, value)
}

// 5. ส่ง array เข้าฟังก์ชัน (copy)
func modifyArray(arr [3]int) {
    arr[0] = 999
}

data := [3]int{1, 2, 3}
modifyArray(data)
fmt.Println(data) // [1 2 3] (ไม่เปลี่ยน)

// 6. ส่ง pointer ของ array (rarely used)
func modifyArrayPtr(arr *[3]int) {
    arr[0] = 999
}

modifyArrayPtr(&data)
fmt.Println(data) // [999 2 3]
}

```

ข้อควรจำ

- **Arrays ใช้ไม่บ่อยใน Go** เพราะขนาดคงที่และ copy ข้อมูล
- ส่วนใหญ่ใช้ **Slices แทน** (บทถัดไป)
- ใช้ array เมื่อต้องการ:
 - ขนาดคงที่แน่นอน (เช่น 7 วันในสัปดาห์)
 - ประสิทธิภาพสูงมาก (stack allocation)
 - ทำงานกับระบบ low-level

บทที่ 21: Slices (สไลซ์)

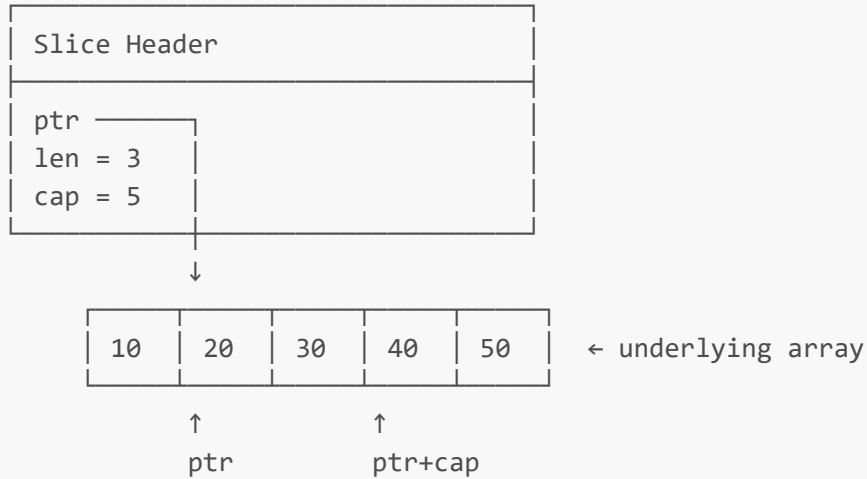
เป้าหมาย: เข้าใจโครงสร้างข้อมูล dynamic ที่ใช้บ่อยที่สุดใน Go

Slices คืออะไร?

Slice คือ **view** ที่ยึดหยุ่นของ array ประกอบด้วย 3 ส่วน:

- **pointer** → ชี้ไปยัง underlying array
- **len** → ความยาวปัจจุบัน
- **cap** → ความจุทั้งหมด

Slice Structure:



ตัวอย่างการใช้งาน

```
package main

import "fmt"

func main() {
    // 1. สร้าง slice
    var s1 []int           // nil slice (len=0, cap=0)
    s2 := []int{1, 2, 3}   // literal
    s3 := make([]int, 5)    // length=5, capacity=5
    s4 := make([]int, 5, 10) // length=5, capacity=10

    fmt.Println(s1, s2, s3, s4)

    // 2. append (เพิ่มสมาชิก)
    numbers := []int{1, 2, 3}
    numbers = append(numbers, 4) // [1,2,3,4]
    numbers = append(numbers, 5, 6) // [1,2,3,4,5,6]

    // append slice กับ slice
    more := []int{7, 8, 9}
    numbers = append(numbers, more...) // ... ใช้ unpack slice

    fmt.Println(numbers) // [1,2,3,4,5,6,7,8,9]

    // 3. len และ cap
    slice := make([]int, 3, 5)
```

```

fmt.Printf("len=%d, cap=%d\n", len(slice), cap(slice)) // len=3, cap=5

slice = append(slice, 100)
fmt.Printf("len=%d, cap=%d\n", len(slice), cap(slice)) // len=4, cap=5

slice = append(slice, 200, 300)
fmt.Printf("len=%d, cap=%d\n", len(slice), cap(slice)) // len=6, cap=10
(double)

// 4. Slicing (สร้าง slice ใหม่จาก slice)
original := []int{1, 2, 3, 4, 5}
sub := original[1:4] // [2,3,4]
fmt.Println(sub)

// sub ชี้ไปที่ underlying array เดียวกับ original
sub[0] = 999
fmt.Println(original) // [1,999,3,4,5] (เปลี่ยนด้วย!)

// 5. copy slice (deep copy)
src := []int{1, 2, 3}
dst := make([]int, len(src))
copy(dst, src)
dst[0] = 999
fmt.Println(src) // [1,2,3] (ไม่เปลี่ยน)
fmt.Println(dst) // [999,2,3]

// 6. การลบ element
// ลบ element ที่ index 2
s := []int{1, 2, 3, 4, 5}
index := 2
s = append(s[:index], s[index+1:]...)
fmt.Println(s) // [1,2,4,5]

// 7. การ insert element
s = []int{1, 2, 4, 5}
s = append(s[:2], append([]int{3}, s[2:]...)...)
fmt.Println(s) // [1,2,3,4,5]
}

// 8. ฟังก์ชันที่รับและคืน slice
func filter(numbers []int, predicate func(int) bool) []int {
    result := make([]int, 0, len(numbers))
    for _, n := range numbers {
        if predicate(n) {
            result = append(result, n)
        }
    }
    return result
}

```

ตัวอย่างการใช้งานจริง

1. Pagination (การแบ่งหน้า)

```
type PaginatedResult struct {
    Data      []User
    Total     int
    Page      int
    PerPage   int
    TotalPages int
}

func GetUsers(page, perPage int) (*PaginatedResult, error) {
    // ดึงข้อมูลทั้งหมด
    allUsers, err := userRepo.FindAll()
    if err != nil {
        return nil, err
    }

    total := len(allUsers)
    totalPages := (total + perPage - 1) / perPage

    // คำนวณ start และ end index
    start := (page - 1) * perPage
    end := start + perPage

    if start >= total {
        return &PaginatedResult{
            Data:      []User{},
            Total:      total,
            Page:      page,
            PerPage:   perPage,
            TotalPages: totalPages,
        }, nil
    }

    if end > total {
        end = total
    }

    // slice ข้อมูล
    return &PaginatedResult{
        Data:      allUsers[start:end],
        Total:      total,
        Page:      page,
        PerPage:   perPage,
        TotalPages: totalPages,
    }, nil
}
```

2. Stack (LIFO) ด้วย Slice

```

type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(item T) {
    s.items = append(s.items, item)
}

func (s *Stack[T]) Pop() (T, bool) {
    if len(s.items) == 0 {
        var zero T
        return zero, false
    }

    lastIndex := len(s.items) - 1
    item := s.items[lastIndex]
    s.items = s.items[:lastIndex]
    return item, true
}

func (s *Stack[T]) Peek() (T, bool) {
    if len(s.items) == 0 {
        var zero T
        return zero, false
    }
    return s.items[len(s.items)-1], true
}

```

3. Queue (FIFO) ด้วย Slice

```

type Queue[T any] struct {
    items []T
}

func (q *Queue[T]) Enqueue(item T) {
    q.items = append(q.items, item)
}

func (q *Queue[T]) Dequeue() (T, bool) {
    if len(q.items) == 0 {
        var zero T
        return zero, false
    }

    item := q.items[0]
    q.items = q.items[1:]
    return item, true
}

```

4. Performance Tip: Preallocate capacity

```
// ไม่ดี: ทำให้เกิดการ reallocation บ่อย
func badBuild() []int {
    var result []int
    for i := 0; i < 1000000; i++ {
        result = append(result, i) // reallocate หลายครั้ง
    }
    return result
}

// ดี: preallocate capacity
func goodBuild() []int {
    result := make([]int, 0, 1000000) // allocate ครั้งเดียว
    for i := 0; i < 1000000; i++ {
        result = append(result, i)
    }
    return result
}
```

บทที่ 22: Maps (แมพ)

เป้าหมาย: เข้าใจโครงสร้างข้อมูล key-value ที่ใช้บ่อย

Maps คืออะไร?

Map เป็น **hash table** ที่เก็บ key-value pairs

- Key ต้องเป็น comparable type (==, !=)
- Value สามารถเป็นชนิดใดก็ได้
- ไม่ปลอดภัยต่อ **concurrent access** (ต้องใช้ mutex หรือ sync.Map)

ตัวอย่างการใช้งาน

```
package main

import "fmt"

func main() {
    // 1. สร้าง map
    var m1 map[string]int // nil map (อ่านได้ แต่เขียนไม่ได้)
    m2 := make(map[string]int) // empty map
    m3 := map[string]int{ // map literal
        "apple": 5,
        "banana": 3,
    }
}
```

```

// nil map จะ panic ถ้าเขียน
// m1["key"] = 1 // panic!

// 2. เพิ่ม/แก้ไข
m2["apple"] = 10
m2["banana"] = 7
m2["orange"] = 4

fmt.Println(m2) // map[apple:10 banana:7 orange:4]

// 3. อ่านค่า
value := m2["apple"] // 10
value2 := m2["grape"] // 0 (zero value, ไม่มี key นี้)

// 4. comma ok idiom (ตรวจสอบว่ามี key หรือไม่)
if val, ok := m2["banana"]; ok {
    fmt.Printf("banana exists with value %d\n", val)
}

if _, ok := m2["grape"]; !ok {
    fmt.Println("grape does not exist")
}

// 5. ลบ key
delete(m2, "orange")
fmt.Println(m2) // map[apple:10 banana:7]

// 6. วนลูป map (order is random)
for key, value := range m2 {
    fmt.Printf("%s: %d\n", key, value)
}

// 7. วนลูปเฉพาะ key
for key := range m2 {
    fmt.Println(key)
}

// 8. map ของ struct
type User struct {
    Name string
    Age int
}

users := map[int]User{
    1: {Name: "Alice", Age: 30},
    2: {Name: "Bob", Age: 25},
}

// 9. map ของ map (nested)
matrix := map[string]map[string]int{
    "row1": {"col1": 1, "col2": 2},
    "row2": {"col1": 3, "col2": 4},
}

```



```
// 10. map กับ slice เป็น value
tags := map[string][]string{
    "golang": {"backend", "fast", "concurrent"},
    "python": {"ai", "ml", "easy"},
}
}
```

ตัวอย่างการใช้งานจริง

1. Cache (LRU cache แบบง่าย)

```
type Cache struct {
    store map[string]interface{}
    mu     sync.RWMutex
    ttl    time.Duration
}

func NewCache(ttl time.Duration) *Cache {
    c := &Cache{
        store: make(map[string]interface{}),
        ttl:   ttl,
    }

    // start cleanup goroutine
    go c.cleanup()

    return c
}

func (c *Cache) Set(key string, value interface{}) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.store[key] = value
}

func (c *Cache) Get(key string) (interface{}, bool) {
    c.mu.RLock()
    defer c.mu.RUnlock()
    val, ok := c.store[key]
    return val, ok
}

func (c *Cache) cleanup() {
    ticker := time.NewTicker(c.ttl)
    for range ticker.C {
        c.mu.Lock()
        // ลบ expired keys (สมมติมีเวลา expire)
        for k := range c.store {
            delete(c.store, k)
        }
    }
}
```

```

    }
    c.mu.Unlock()
}
}

```

2. Group By (การจัดกลุ่มข้อมูล)

```

type Order struct {
    ID        int
    UserID    int
    Amount    float64
    Status    string
}

func GroupOrdersByUser(orders []Order) map[int][]Order {
    result := make(map[int][]Order)

    for _, order := range orders {
        result[order.UserID] = append(result[order.UserID], order)
    }

    return result
}

func GroupOrdersByStatus(orders []Order) map[string][]Order {
    result := make(map[string][]Order)

    for _, order := range orders {
        result[order.Status] = append(result[order.Status], order)
    }

    return result
}

```

3. Set (ใช้ map แทน set)

```

type Set[T comparable] struct {
    items map[T]struct{}
}

func NewSet[T comparable]() *Set[T] {
    return &Set[T]{
        items: make(map[T]struct{}),
    }
}

func (s *Set[T]) Add(item T) {
    s.items[item] = struct{}{}
}

```

```

}

func (s *Set[T]) Remove(item T) {
    delete(s.items, item)
}

func (s *Set[T]) Contains(item T) bool {
    _, ok := s.items[item]
    return ok
}

func (s *Set[T]) Size() int {
    return len(s.items)
}

func (s *Set[T]) List() []T {
    result := make([]T, 0, len(s.items))
    for item := range s.items {
        result = append(result, item)
    }
    return result
}

// ใช้งาน
func main() {
    tags := NewSet[string]()
    tags.Add("golang")
    tags.Add("programming")
    tags.Add("golang") // duplicate (ไม่มีผล)

    fmt.Println(tags.Contains("golang")) // true
    fmt.Println(tags.Size())             // 2
    fmt.Println(tags.List())              // [golang programming]
}

```

4. Counter (นับความถี่)

```

func WordCount(text string) map[string]int {
    words := strings.Fields(strings.ToLower(text))
    counter := make(map[string]int)

    for _, word := range words {
        // ลบ punctuation
        word = strings.Trim(word, ". , ! ? ; : ")
        counter[word]++
    }

    return counter
}

// ใช้งาน

```

```

text := "Go is great. Go is fast. Go is concurrent!"
counts := WordCount(text)
// map[go:3 is:3 great:1 fast:1 concurrent:1]

```

5. Graph (กราฟ) ด้วย map

```

type Graph struct {
    nodes map[string][]string // adjacency list
}

func NewGraph() *Graph {
    return &Graph{
        nodes: make(map[string][]string),
    }
}

func (g *Graph) AddEdge(from, to string) {
    g.nodes[from] = append(g.nodes[from], to)
    g.nodes[to] = append(g.nodes[to], from) // undirected
}

func (g *Graph) BFS(start, target string) []string {
    visited := make(map[string]bool)
    queue := []string{start}
    parent := make(map[string]string)
    visited[start] = true

    for len(queue) > 0 {
        current := queue[0]
        queue = queue[1:]

        if current == target {
            // reconstruct path
            path := []string{}
            for node := target; node != ""; node = parent[node] {
                path = append([]string{node}, path...)
            }
            return path
        }

        for _, neighbor := range g.nodes[current] {
            if !visited[neighbor] {
                visited[neighbor] = true
                parent[neighbor] = current
                queue = append(queue, neighbor)
            }
        }
    }

    return nil
}

```

6. Map of functions

```
// Strategy pattern ด้วย map
type Operation func(a, b int) int

var operations = map[string]Operation{
    "add":      func(a, b int) int { return a + b },
    "subtract": func(a, b int) int { return a - b },
    "multiply": func(a, b int) int { return a * b },
    "divide":   func(a, b int) int { return a / b },
}

func calculate(op string, a, b int) (int, error) {
    if fn, ok := operations[op]; ok {
        return fn(a, b), nil
    }
    return 0, fmt.Errorf("unknown operation: %s", op)
}
```

4. การจัดการข้อผิดพลาด (บทที่ 23)

เป้าหมาย: เขียนโค้ดที่จัดการ error ได้อย่างถูกต้องและมีประสิทธิภาพ

บทที่ 23: Errors

Error Interface

```
// error interface (built-in)
type error interface {
    Error() string
}
```

ตัวอย่างการใช้งาน

```
package main

import (
    "errors"
    "fmt"
)

// 1. สร้าง error ด้วย errors.New
var ErrNotFound = errors.New("not found")
```

```

var ErrInvalidInput = errors.New("invalid input")

// 2. สร้าง error ด้วย fmt.Errorf
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero: %f / %f", a, b)
    }
    return a / b, nil
}

// 3. Custom error type
type ValidationError struct {
    Field    string
    Value    interface{}
    Message  string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("validation failed for %s (value: %v): %s",
        e.Field, e.Value, e.Message)
}

// 4. ฟังก์ชันที่ใช้ custom error
func validateUser(user User) error {
    if user.Name == "" {
        return &ValidationError{
            Field:    "name",
            Value:    user.Name,
            Message:  "name is required",
        }
    }
    if user.Age < 0 {
        return &ValidationError{
            Field:    "age",
            Value:    user.Age,
            Message:  "age must be positive",
        }
    }
    return nil
}

```

Error Wrapping (Go 1.13+)

```

package main

import (
    "errors"
    "fmt"
)

```

```
// ฟังก์ชันที่ wrap error
func readConfig(path string) error {
    data, err := os.ReadFile(path)
    if err != nil {
        // Wrap error ด้วย context
        return fmt.Errorf("read config file %s: %w", path, err)
    }
    // process data...
    return nil
}

func process() error {
    if err := readConfig("/etc/app/config.yaml"); err != nil {
        // Wrap เพิ่ม layer
        return fmt.Errorf("process failed: %w", err)
    }
    return nil
}

func main() {
    err := process()
    if err != nil {
        // ตรวจสอบด้วย errors.Is
        if errors.Is(err, os.ErrNotExist) {
            fmt.Println("config file does not exist")
        }

        // ตรวจสอบ type ด้วย errors.As
        var pathErr *os.PathError
        if errors.As(err, &pathErr) {
            fmt.Printf("path error: %s\n", pathErr.Path)
        }

        // ดู error chain
        fmt.Printf("full error: %v\n", err)
    }
}
```

ตัวอย่างการใช้งานจริง

1. การจัดการ error ใน HTTP handler

```
// domain errors
var (
    ErrUserNotFound      = errors.New("user not found")
    ErrInvalidPassword    = errors.New("invalid password")
    ErrEmailExists        = errors.New("email already exists")
)

// service layer
```

```

type UserService struct {
    repo UserRepository
}

func (s *UserService) Login(email, password string) (*User, error) {
    user, err := s.repo.FindByEmail(email)
    if err != nil {
        if errors.Is(err, ErrUserNotFound) {
            return nil, fmt.Errorf("login failed: %w", ErrUserNotFound)
        }
        return nil, fmt.Errorf("database error: %w", err)
    }

    if !user.VerifyPassword(password) {
        return nil, fmt.Errorf("login failed: %w", ErrInvalidPassword)
    }

    return user, nil
}

// handler layer
type UserHandler struct {
    service UserService
}

func (h *UserHandler) Login(w http.ResponseWriter, r *http.Request) {
    var req LoginRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "invalid request", http.StatusBadRequest)
        return
    }

    user, err := h.service.Login(req.Email, req.Password)
    if err != nil {
        // map error type
        switch {
        case errors.Is(err, ErrUserNotFound):
            http.Error(w, "user not found", http.StatusNotFound)
        case errors.Is(err, ErrInvalidPassword):
            http.Error(w, "invalid password", http.StatusUnauthorized)
        default:
            // log internal error
            log.Printf("login error: %v", err)
            http.Error(w, "internal server error",
http.StatusInternalServerError)
        }
        return
    }

    json.NewEncoder(w).Encode(user)
}

```


2. Retry pattern with error handling

```
func Retry(maxRetries int, fn func() error) error {
    var err error

    for i := 0; i < maxRetries; i++ {
        err = fn()
        if err == nil {
            return nil
        }

        // ตรวจสอบว่า error ที่เกิดขึ้น retry ได้หรือไม่
        var retryable *RetryableError
        if errors.As(err, &retryable) {
            delay := time.Duration(1<<i) * time.Second
            log.Printf("retry %d after %v: %v", i+1, delay, err)
            time.Sleep(delay)
            continue
        }

        // error ไม่สามารถ retry ได้
        return err
    }

    return fmt.Errorf("max retries exceeded: %w", err)
}

type RetryableError struct {
    Err error
}

func (e *RetryableError) Error() string {
    return fmt.Sprintf("retryable: %v", e.Err)
}

func (e *RetryableError) Unwrap() error {
    return e.Err
}

// ใช้งาน
err := Retry(3, func() error {
    resp, err := http.Get("https://api.example.com/data")
    if err != nil {
        return &RetryableError{Err: err} // network error, retry
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        if resp.StatusCode >= 500 {
            return &RetryableError{Err: fmt.Errorf("server error: %d",
resp.StatusCode)}}
        }
    }
})
```

```

        return fmt.Errorf("client error: %d", resp.StatusCode) // ❌ retry
    }

    return nil
})

```

3. Error grouping and handling

```

type ErrorGroup struct {
    errors []error
    mu     sync.Mutex
}

func (g *ErrorGroup) Add(err error) {
    g.mu.Lock()
    defer g.mu.Unlock()
    g.errors = append(g.errors, err)
}

func (g *ErrorGroup) Error() string {
    if len(g.errors) == 0 {
        return ""
    }

    messages := make([]string, len(g.errors))
    for i, err := range g.errors {
        messages[i] = err.Error()
    }
    return strings.Join(messages, "; ")
}

func (g *ErrorGroup) HasErrors() bool {
    g.mu.Lock()
    defer g.mu.Unlock()
    return len(g.errors) > 0
}

// ใช้งาน
func ValidateUser(user User) error {
    var errGroup ErrorGroup

    if user.Name == "" {
        errGroup.Add(errors.New("name is required"))
    }

    if !strings.Contains(user.Email, "@") {
        errGroup.Add(errors.New("invalid email"))
    }

    if user.Age < 18 {
        errGroup.Add(errors.New("age must be at least 18"))
    }
}

```

```

    }

    if errGroup.HasErrors() {
        return &errGroup
    }

    return nil
}

```

4. Panic recovery (เมื่อจำเป็น)

```

// สำหรับ service ที่ต้องรันตลอดเวลา
func RunWithRecovery(fn func()) {
    defer func() {
        if r := recover(); r != nil {
            // log panic
            log.Printf("panic recovered: %v\n%s", r, debug.Stack())

            // send alert
            sendAlert(fmt.Sprintf("panic: %v", r))
        }
    }()

    fn()
}

// ใช้กับ goroutine
go func() {
    RunWithRecovery(func() {
        // risky operation
        doSomething()
    })
}()

```

5. Best Practices สรุป

```

// ☒ DO: ตรวจสอบ error ทันที
data, err := readFile()
if err != nil {
    return fmt.Errorf("read file: %w", err)
}

// ☒ DON'T: ignore errors
data, _ := readFile()

// ☒ DO: ให้ context แก่ error
return fmt.Errorf("failed to process user %d: %w", userID, err)

```

```
// ❌ DON'T: return plain error without context
return err

// ✅ DO: ใช้ custom error types สำหรับ business logic
var ErrInsufficientFunds = errors.New("insufficient funds")

// ✅ DO: ใช้ errors.Is และ errors.As ในการตรวจสอบ
if errors.Is(err, ErrNotFound) {
    // handle not found
}

var valErr *ValidationError
if errors.As(err, &valErr) {
    fmt.Printf("field: %s\n", valErr.Field)
}
```

สรุปภาคที่ 3: ความสัมพันธ์ของเนื้อหา

```
flowchart LR
    subgraph Skills["ทักษะที่ได้"]
        S1[สร้างและจัดการ<br/>โปรเจกต์ Go]
        S2[เขียน Unit Tests<br/>และวัด coverage]
        S3[ใช้ Slices และ Maps<br/>จัดการข้อมูล]
        S4[จัดการ error<br/>อย่างมืออาชีพ]
    end

    subgraph Applications["การประยุกต์ใช้"]
        A1[โปรเจกต์ที่<br/>มีโครงสร้าง]
        A2[โค้ดที่มั่นใจ<br/>ผ่านการทดสอบ]
        A3[โครงสร้างข้อมูล<br/>ที่มีประสิทธิภาพ]
        A4[ระบบที่เสถียร<br/>debug ได้ง่าย]
    end

    Skills --> Applications
```

สิ่งที่ได้เรียนรู้

1. บทที่ 17-18 (Go Modules):

- จัดการ dependencies ด้วย go.mod
- ใช้ module proxy เพื่อความเร็วและความปลอดภัย
- จัดการ version และ private modules

2. บทที่ 19 (Testing):

- เขียน table-driven tests
- ใช้ mock และ stub
- วัด test coverage

3. บทที่ 20-22 (Data Structures):

- Array: fixed-size, value type
- Slice: dynamic, reference type, ใ้บ่อยที่สุด
- Map: key-value, hash table

4. บทที่ 23 (Error Handling):

- error interface และ error wrapping
- custom error types
- errors.Is และ errors.As
- retry pattern และ error grouping

พื้นฐานเหล่านี้จะนำไปใช้ใน **ภาคที่ 4** เพื่อพัฒนาแอปพลิเคชันเชิงปฏิบัติ เช่น HTTP server, concurrency, JSON handling, และการบันทึกข้อมูล

บทที่ 17: Go Modules - การจัดการโปรเจกต์สมัยใหม่

17.1 Go Modules คืออะไร?

Go Modules เป็นระบบจัดการ dependencies อย่างเป็นทางการ เริ่มตั้งแต่ Go 1.11 ทำให้ไม่ต้องพึ่งพา GOPATH

17.2 การเริ่มต้นโมดูล

```
go mod init example.com/myproject
```

สร้างไฟล์ **go.mod**:

```
module example.com/myproject  
  
go 1.21
```

17.3 การเพิ่ม dependencies

เมื่อเรา import แพคเกจภายนอกในโค้ดและรัน **go build** หรือ **go mod tidy** Go จะดึง dependencies และเพิ่มลงใน go.mod พร้อมสร้าง go.sum

```
go get github.com/gorilla/mux
```

17.4 go.mod

ประกอบด้วย:

- module path
- เวอร์ชัน Go

- require: dependencies และเวอร์ชัน
- replace: แทนที่โมดูลด้วย local path หรือเวอร์ชันอื่น
- exclude: ไม่ใช้บางเวอร์ชัน

17.5 go.sum

เก็บ checksum ของ dependencies เพื่อความปลอดภัยและการตรวจสอบความถูกต้อง

17.6 การอัปเดต dependencies

```
go get -u          # อัปเดตทุก dependency
go get -u ./...     # อัปเดตภายในโมดูล
go get github.com/gorilla/mux@v1.8.0 # อัปเดตเฉพาะ
go mod tidy         # ลบ dependencies ที่ไม่ใช่
```

17.7 vendor directory

สามารถสร้าง vendor folder สำหรับเก็บ dependencies ในโปรเจกต์ (สำหรับการ build ที่ไม่ต้องเชื่อมต่อ internet)

```
go mod vendor
```

แล้ว build ด้วย `go build -mod=vendor`

17.8 การทำงานกับ private repositories

ตั้งค่า GOPRIVATE:

```
go env -w GOPRIVATE=github.com/mycompany/*
```

บทที่ 18: Go Module Proxies

18.1 โมดูลพร็อกซีคืออะไร?

พร็อกซีทำหน้าที่เป็นตัวกลางในการให้บริการโมดูล Go ช่วยให้การดาวน์โหลด dependencies เร็วขึ้น และมั่นใจว่าโค้ดไม่ถูกแก้ไข

18.2 คำเริ่มต้น

Go 1.13+ ใช้ proxy <https://proxy.golang.org> เป็น default พร้อม fallback ไป direct

18.3 การตั้งค่า GOPROXY

```
go env -w GOPROXY=https://goproxy.io,direct
```

สามารถตั้งค่าเป็น `direct` เพื่อดึงโดยตรงจาก VCS

18.4 การใช้ proxy แบบ private

สำหรับ private repository ควรตั้ง `GOPRIVATE` เพื่อ bypass proxy

18.5 การตรวจสอบ checksum

Go ใช้ checksum database (`sum.golang.org`) ตรวจสอบ integrity

18.6 การปิดใช้งาน checksum สำหรับ private

```
go env -w GOSUMDB=off
```

บทที่ 19: การทดสอบหน่วย (Unit Tests)

19.1 การเขียน test

ไฟล์ test ต้องลงท้ายด้วย `_test.go` และฟังก์ชัน test ต้องขึ้นต้นด้วย `Test` และรับ `*testing.T`

```
// math_test.go
package math

import "testing"

func TestAdd(t *testing.T) {
    result := Add(2, 3)
    expected := 5
    if result != expected {
        t.Errorf("Add(2,3) = %d; want %d", result, expected)
    }
}
```

19.2 การรัน test

```
go test           # รัน test ใน current directory
go test ./...     # รันทั้งหมด
go test -v        # verbose output
go test -run TestAdd # รันเฉพาะฟังก์ชันที่ match
```

19.3 Table-driven tests

รูปแบบที่นิยมใน Go:

```
func TestAdd(t *testing.T) {
    tests := []struct {
        name string
        a, b int
        want int
    }{
        {"positive", 2, 3, 5},
        {"negative", -1, -2, -3},
        {"zero", 0, 0, 0},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            if got := Add(tt.a, tt.b); got != tt.want {
                t.Errorf("Add(%d, %d) = %d, want %d", tt.a, tt.b, got, tt.want)
            }
        })
    }
}
```

19.4 การใช้ subtests

`t.Run` ช่วยให้ test ทำงานแยก และสามารถร่นย่อได้

19.5 การทดสอบ HTTP

```
func TestHandler(t *testing.T) {
    req := httptest.NewRequest("GET", "/", nil)
    w := httptest.NewRecorder()
    handler(w, req)
    if w.Code != http.StatusOK {
        t.Errorf("expected 200, got %d", w.Code)
    }
}
```

19.6 การใช้ test fixtures

สามารถใช้ `TestMain` เพื่อ setup/teardown

```
func TestMain(m *testing.M) {
    // setup
    code := m.Run()
    // teardown
    os.Exit(code)
}
```


19.7 การครอบคลุม (coverage)

```
go test -cover
go test -coverprofile=coverage.out
go tool cover -html=coverage.out # ดูรายละเอียดใน browser
```

19.8 การใช้ mock

แนะนำให้ใช้ interface เพื่อให้สามารถเปลี่ยน implementation เป็น mock ได้ง่าย

บทที่ 20: อาร์เรย์ (Arrays)

20.1 อาร์เรย์คืออะไร?

อาร์เรย์คือชุดข้อมูลขนาดคงที่ของชนิดเดียวกัน

```
var arr [5]int           // array ขนาด 5, zero values
arr2 := [3]int{1,2,3}    // กำหนดค่า
arr3 := [...]int{1,2,3}  // ให้ compiler นับขนาด
```

20.2 การเข้าถึงและแก้ไข

```
arr[0] = 10
value := arr[0]
len(arr) // ความยาว
```

20.3 อาร์เรย์เป็นค่า (value type)

ใน Go อาร์เรย์เป็น value type เมื่อส่งเข้า function จะถูก copy ทั้งตัว

```
func modify(arr [3]int) {
    arr[0] = 100 // ไม่มีผลกับตัวแปรเดิม
}
```

20.4 การใช้ pointer กับอาร์เรย์

```
func modifyPtr(arr *[3]int) {
    arr[0] = 100 // แก้ไขตัวแปรเดิม
}
```

20.5 ข้อจำกัด

ขนาดอาร์เรย์เป็นส่วนหนึ่งของชนิด ดังนั้น `[3]int` และ `[4]int` เป็นคนละชนิดกัน
การส่งอาร์เรย์ขนาดใหญ่ไปยังฟังก์ชันอาจทำให้ประสิทธิภาพลดลง (copy ข้อมูล)

20.6 การวนลูป

```
for i := 0; i < len(arr); i++ {  
    fmt.Println(arr[i])  
}  
  
for i, v := range arr {  
    fmt.Println(i, v)  
}
```

20.7 อาร์เรย์แบบหลายมิติ

```
var matrix [2][3]int  
matrix[0][1] = 5
```

บทที่ 21: สไลซ์ (Slices)

21.1 สไลซ์คืออะไร?

สไลซ์เป็นโครงสร้างข้อมูลที่ยืดหยุ่นกว่าอาร์เรย์ มีขนาด dynamic จริงๆ แล้วสไลซ์คือ view ของอาร์เรย์

```
var s []int           // nil slice  
s2 := []int{1,2,3}    // slice literal  
s3 := make([]int, 5)   // length 5, capacity 5  
s4 := make([]int, 5, 10) // length 5, capacity 10
```

21.2 การเพิ่มสมาชิกด้วย append

```
s := []int{1,2}  
s = append(s, 3)      // [1,2,3]  
s = append(s, 4,5)    // [1,2,3,4,5]
```

21.3 การ slicing

```
arr := [5]int{1,2,3,4,5}
slice := arr[1:4] // [2,3,4]
```

slicing ใช้ syntax `[low:high]` (low inclusive, high exclusive) มี default: low = 0, high = len

21.4 capacity และ length

- `len(s)` : ความยาวปัจจุบัน
- `cap(s)` : ความจุ (จำนวน element ที่สามารถเก็บได้โดยไม่ต้อง allocate ใหม่)

เมื่อ append แล้วเกิน capacity Go จะ allocate array ใหม่ (ขนาดประมาณสองเท่า)

21.5 การ copy

```
src := []int{1,2,3}
dst := make([]int, len(src))
copy(dst, src)
```

copy คำนวณจำนวน element ที่คัดลอก

21.6 การลบ element

ไม่มีฟังก์ชันลบโดยตรง ใช้ slicing รวม:

```
// ลบ element index i
s = append(s[:i], s[i+1:]...)
```

21.7 การ prepend

```
s = append([]int{0}, s...)
```

21.8 slice ของ slice (multidimensional)

```
matrix := make([][]int, 3)
for i := range matrix {
    matrix[i] = make([]int, 3)
}
```

21.9 การส่ง slice ไปยังฟังก์ชัน

สไลซ์ส่งเป็น reference (pointer to underlying array) ดังนั้นการแก้ไข element มีผลต่อตัวแปรเดิม

21.10 ข้อควรระวัง: shared underlying array

เมื่อ slicing จาก slice เดิม อาจเกิดการแชร์อาร์เรย์ข้างใต้ ทำให้การแก้ไขกระทบกัน

บทที่ 22: แมพ (Maps)

22.1 แมพคืออะไร?

แมพคือโครงสร้างข้อมูลแบบ key-value (คล้าย dictionary) key type ต้อง comparable (==)

```
var m map[string]int // nil map, ใส่ค่าไม่ได้
m2 := make(map[string]int) // empty map
m3 := map[string]int{
    "one": 1,
    "two": 2,
}
```

22.2 การใช้งาน

```
m := make(map[string]int)
m["key"] = 42
value := m["key"]
delete(m, "key")
```

22.3 การตรวจสอบว่ามี key หรือไม่

```
val, ok := m["key"]
if ok {
    fmt.Println("found:", val)
} else {
    fmt.Println("not found")
}
```

22.4 การวนลูปผ่าน map

```
for key, value := range m {
    fmt.Println(key, value)
}
```

ลำดับการวนไม่แน่นอน (randomized)

22.5 Map เป็น reference type

การส่ง map ไปฟังก์ชันจะไม่ copy ข้อมูล การแก้ไขมีผลต่อต้นฉบับ

22.6 nil map

nil map ไม่สามารถเขียนได้ (จะ panic) ต้องใช้ make ก่อนเสมอ

22.7 การใช้ struct เป็น key

struct ที่ fields ทั้งหมด comparable สามารถเป็น key ได้

```
type Key struct {  
    A, B int  
}  
m := make(map[Key]string)  
m[Key{1,2}] = "value"
```

22.8 ประสิทธิภาพ

Map มีการ rehash เมื่อขนาดโตขึ้น ควรให้ hint ขนาดล่วงหน้าด้วย `make(map[K]V, hint)` เพื่อลดการ rehash

22.9 sync.Map

ถ้าต้องการใช้ map ใน concurrent environment ควรใช้ `sync.Map` หรือใช้ mutex

บทที่ 23: การจัดการข้อผิดพลาด (Errors)

23.1 Error ใน Go

Go ใช้ error interface ในการจัดการข้อผิดพลาด ไม่มี exception

```
type error interface {  
    Error() string  
}
```

ฟังก์ชันมักคืนค่า error เป็นค่าสุดท้าย

```
func doSomething() (result int, err error) {  
    if somethingWrong {  
        return 0, errors.New("something went wrong")  
    }  
    return 42, nil  
}
```

23.2 การสร้าง error

- `errors.New("message")`
- `fmt.Errorf("format %v", arg)`

23.3 การตรวจสอบ error

```
result, err := doSomething()
if err != nil {
    // จัดการ error
    fmt.Println("Error:", err)
    return
}
// ใช้ result
```

23.4 Custom error types

สามารถสร้าง struct ที่ implement error interface เพื่อเพิ่มข้อมูล

```
type ValidationError struct {
    Field string
    Value interface{}
}

func (e ValidationError) Error() string {
    return fmt.Sprintf("validation failed on %s: %v", e.Field, e.Value)
}
```

23.5 การตรวจสอบชนิดของ error

ใช้ type assertion หรือ `errors.As`

```
var ve ValidationError
if errors.As(err, &ve) {
    fmt.Println("Field:", ve.Field)
}
```

23.6 การห่อหุ้ม error (wrapping)

ใช้ `%w` ใน `fmt.Errorf` เพื่อ wrap error

```
if err != nil {
    return fmt.Errorf("failed to process: %w", err)
}
```

การ unwrap ด้วย `errors.Unwrap` หรือตรวจสอบด้วย `errors.Is`

23.7 errors.Is และ errors.As

- `errors.Is(err, target)` : ตรวจสอบว่า err หรือ err ที่ถูก wrap มี target หรือไม่
- `errors.As(err, &target)` : ดึง error เฉพาะชนิดออกมา

```
if errors.Is(err, sql.ErrNoRows) {  
    // handle no rows  
}
```

23.8 panic vs error

ใช้ error สำหรับกรณีที่คาดการณ์ได้ (เช่น ไฟล์ไม่พบ)

ใช้ panic สำหรับกรณีที่ไม่ควรเกิดขึ้น (programmer error) หรือไม่สามารถดำเนินการต่อได้

23.9 การใช้ defer กับ error

สามารถใช้ defer เพื่อทำ cleanup แม้มี error

ภาคที่ 4: การพัฒนาแอปพลิเคชันเชิงปฏิบัติ

(บทที่ 24–33)

แผนภาพแสดงความสัมพันธ์ของเนื้อหา

```
flowchart TB  
    subgraph Input["1. การรับข้อมูล (Input Layer)"]  
        direction TB  
        A1["HTTP Server (บทที่ 26)<br/>net/http, chi, gin<br/>routing, middleware, handlers"]  
        A2["JSON/XML Handling (บทที่ 25)<br/>encoding/json, xml<br/>Marshal/Unmarshal, tags"]  
        A3["Enum/Iota/Bitmask (บทที่ 27)<br/>iota, bit operations<br/>status flags, permissions"]  
        A4["Date & Time (บทที่ 28)<br/>time package, duration<br/>parsing, formatting, timezone"]  
        A1 --> A2  
        A2 --> A3  
        A3 --> A4  
    end  
  
    subgraph Processing["2. การประมวลผล (Processing Layer)"]  
        direction TB  
        B1["Anonymous Functions & Closure (บทที่ 24)<br/>func literals,
```

```

closures<br/>capturing variables, callbacks"]
    B2["Concurrency (บทที่ 30)<br/>goroutines, channels, select<br/>sync,
worker pools, patterns"]
    B3["Error Handling (from บทที่ 23)<br/>error wrapping, custom
errors<br/>panic recovery, retry"]

    B1 --> B2
    B2 --> B3
end

subgraph Storage["3. การจัดเก็บ (Storage Layer)"]
    direction TB
    C1["Files & Database (บทที่ 29)<br/>os, io, bufio<br/>SQL, migrations,
transactions"]
    C2["Logging (บทที่ 31)<br/>slog, zap, logrus<br/>structured logging,
levels"]

    C1 --> C2
end

subgraph Output["4. การส่งออก (Output Layer)"]
    direction TB
    D1["Templates (บทที่ 32)<br/>html/template, text/template<br/>layout,
partials, functions"]
    D2["Configuration (บทที่ 33)<br/>viper, env, flags<br/>multiple sources,
defaults"]

    D1 --> D2
end

Input --> Processing
Processing --> Storage
Storage --> Output

subgraph Example["ตัวอย่างแอปพลิเคชัน"]
    E1["REST API Server<br/>+ Authentication<br/>+ Database<br/>+ Logging"]
    E2["Web Application<br/>+ Templates<br/>+ Session<br/>+ Configuration"]
end

Output --> E1
Output --> E2

```

คำอธิบายภาษาไทย (แบบละเอียด)

ภาคที่ 4 ครอบคลุมบทที่ 24–33 โดยมีเนื้อหาแบ่งเป็น 4 ช่วงหลัก ตามแผนภาพด้านบน ซึ่งแสดงการไหลของข้อมูลจาก Input → Processing → Storage → Output

1. การรับข้อมูล (Input Layer)

บทที่ 24: ฟังก์ชันนิรนาม (Anonymous Functions) และ Closure

เป้าหมาย: เข้าใจการสร้างฟังก์ชันแบบไม่มีชื่อและการใช้งาน closure

Anonymous Functions

```
package main

import "fmt"

func main() {
    // 1. ประกาศ anonymous function และเรียกใช้ทันที
    func() {
        fmt.Println("Hello from anonymous")
    }()

    // 2. ประกาศและเก็บไว้ในตัวแปร
    add := func(a, b int) int {
        return a + b
    }
    result := add(3, 5)
    fmt.Println(result) // 8

    // 3. ส่ง anonymous function เป็น argument
    numbers := []int{1, 2, 3, 4, 5}

    // filter function
    evens := filter(numbers, func(n int) bool {
        return n%2 == 0
    })
    fmt.Println(evens) // [2, 4]

    // 4. คืนค่าเป็น function
    multiplier := createMultiplier(3)
    fmt.Println(multiplier(5)) // 15
    fmt.Println(multiplier(10)) // 30
}

func filter(nums []int, predicate func(int) bool) []int {
    result := make([]int, 0)
    for _, n := range nums {
        if predicate(n) {
            result = append(result, n)
        }
    }
    return result
}

func createMultiplier(factor int) func(int) int {
    return func(x int) int {
        return x * factor
    }
}
```

```
}  
}
```

Closures (การปิดล้อมตัวแปร)

```
package main  
  
import "fmt"  
  
func main() {  
    // 1. Closure จับตัวแปรภายนอก  
    counter := func() func() int {  
        count := 0  
        return func() int {  
            count++  
            return count  
        }  
    }()  
  
    fmt.Println(counter()) // 1  
    fmt.Println(counter()) // 2  
    fmt.Println(counter()) // 3  
  
    // 2. Closure ใน loop (ข้อควรระวัง!)  
    funcs := make([]func(), 3)  
  
    // ✗ Wrong: capture by reference  
    for i := 0; i < 3; i++ {  
        funcs[i] = func() {  
            fmt.Println(i) // i is captured by reference  
        }  
    }  
  
    for _, f := range funcs {  
        f() // prints 3, 3, 3 (all the same!)  
    }  
  
    // ✓ Correct: capture by value  
    funcs2 := make([]func(), 3)  
    for i := 0; i < 3; i++ {  
        i := i // create new variable  
        funcs2[i] = func() {  
            fmt.Println(i)  
        }  
    }  
  
    for _, f := range funcs2 {  
        f() // prints 0, 1, 2  
    }  
}
```

ตัวอย่างการใช้งานจริง

1. Middleware Pattern

```
// HTTP middleware with closure
type Middleware func(http.Handler) http.Handler

// Logger middleware
func Logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()

        // สร้าง response wrapper เพื่อ capture status
        rw := &responseWriter{ResponseWriter: w, status: http.StatusOK}

        next.ServeHTTP(rw, r)

        duration := time.Since(start)
        log.Printf("%s %s %d %v", r.Method, r.URL.Path, rw.status, duration)
    })
}

// Auth middleware
func Auth(requiredRole string) Middleware {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            token := r.Header.Get("Authorization")

            user, err := validateToken(token)
            if err != nil {
                http.Error(w, "unauthorized", http.StatusUnauthorized)
                return
            }

            if user.Role != requiredRole {
                http.Error(w, "forbidden", http.StatusForbidden)
                return
            }

            // ใส่ user info ลงใน context
            ctx := context.WithValue(r.Context(), "user", user)
            next.ServeHTTP(w, r.WithContext(ctx))
        })
    }
}

// Apply middlewares
func applyMiddlewares(handler http.Handler, middlewares ...Middleware)
http.Handler {
    for _, m := range middlewares {

```

```

        handler = m(handler)
    }
    return handler
}

// ใช้งาน
func main() {
    handler := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Write([]byte("Hello, World!"))
    })

    // chain middlewares
    wrapped := applyMiddlewares(
        handler,
        Logger,
        Auth("admin"),
    )

    http.ListenAndServe(":8080", wrapped)
}

```

2. Callback Pattern

```

// Event handler with callbacks
type EventHandler struct {
    onSuccess func(data interface{})
    onError   func(err error)
    onComplete func()
}

func (h *EventHandler) OnSuccess(fn func(interface{})) *EventHandler {
    h.onSuccess = fn
    return h
}

func (h *EventHandler) OnError(fn func(error)) *EventHandler {
    h.onError = fn
    return h
}

func (h *EventHandler) OnComplete(fn func()) *EventHandler {
    h.onComplete = fn
    return h
}

func (h *EventHandler) Execute(task func() (interface{}, error)) {
    go func() {
        defer func() {
            if h.onComplete != nil {
                h.onComplete()
            }
        }()
        task()
    }()
}

```

```

    }()

    result, err := task()
    if err != nil && h.onError != nil {
        h.onError(err)
        return
    }

    if h.onSuccess != nil {
        h.onSuccess(result)
    }
}()
}

// ใช้งาน
func main() {
    handler := &EventHandler{}

    handler.
        OnSuccess(func(data interface{}) {
            fmt.Printf("Success: %v\n", data)
        }).
        OnError(func(err error) {
            fmt.Printf("Error: %v\n", err)
        }).
        OnComplete(func() {
            fmt.Println("Task completed")
        }).
        Execute(func() (interface{}, error) {
            // simulate work
            time.Sleep(2 * time.Second)
            return map[string]string{"status": "ok"}, nil
        })

    time.Sleep(3 * time.Second)
}

```

3. Factory Pattern with Closure

```

// Database connection factory
type DBConfig struct {
    Host      string
    Port      int
    User      string
    Password  string
    Database  string
}

func NewDBConnection(config DBConfig) (*sql.DB, error) {
    dsn := fmt.Sprintf("%s:%s@tcp(%s:%d)/%s?charset=utf8mb4&parseTime=True&loc=Local",

```

```

        config.User, config.Password, config.Host, config.Port,
        config.Database)

    db, err := sql.Open("mysql", dsn)
    if err != nil {
        return nil, err
    }

    db.SetMaxOpenConns(25)
    db.SetMaxIdleConns(25)
    db.SetConnMaxLifetime(5 * time.Minute)

    return db, nil
}

// Repository factory with closure
type RepositoryFactory struct {
    db *sql.DB
}

func (f *RepositoryFactory) NewUserRepository() *UserRepository {
    return &UserRepository{db: f.db}
}

func (f *RepositoryFactory) NewOrderRepository() *OrderRepository {
    return &OrderRepository{db: f.db}
}

// ใช้งาน
func main() {
    config := DBConfig{
        Host:      "localhost",
        Port:      3306,
        User:      "root",
        Password:  "password",
        Database:  "myapp",
    }

    db, _ := NewDBConnection(config)
    factory := &RepositoryFactory{db: db}

    userRepo := factory.NewUserRepository()
    orderRepo := factory.NewOrderRepository()

    // use repositories...
}

```

บทที่ 25: การจัดการข้อมูล JSON และ XML

เป้าหมาย: แปลงข้อมูลระหว่าง Go struct กับ JSON/XML

JSON Handling

```
package main

import (
    "encoding/json"
    "fmt"
    "time"
)

// struct with JSON tags
type User struct {
    ID          int          `json:"id"`
    Name        string       `json:"name"`
    Email       string       `json:"email,omitempty"`
    Password    string       `json:"- "` // ไม่ถูก serialize
    CreatedAt   time.Time   `json:"created_at"`
    Tags        []string     `json:"tags,omitempty"`
    Metadata    map[string]interface{} `json:"metadata,omitempty"`
}

// custom marshaling
type CustomTime struct {
    time.Time
}

func (ct CustomTime) MarshalJSON() ([]byte, error) {
    return []byte(fmt.Sprintf("\"%s\"", ct.Format("2006-01-02 15:04:05"))), nil
}

func (ct *CustomTime) UnmarshalJSON(data []byte) error {
    str := string(data)
    str = str[1 : len(str)-1] // remove quotes
    t, err := time.Parse("2006-01-02 15:04:05", str)
    if err != nil {
        return err
    }
    ct.Time = t
    return nil
}

func main() {
    // 1. Marshal (struct -> JSON)
    user := User{
        ID:          1,
        Name:        "John Doe",
        Email:       "john@example.com",
        CreatedAt:   time.Now(),
        Tags:        []string{"golang", "developer"},
        Metadata:    map[string]interface{}{
            "role": "admin",
        },
    }
```

```

        "active": true,
    },
}

jsonData, err := json.Marshal(user)
if err != nil {
    panic(err)
}
fmt.Println(string(jsonData))

// 2. Marshal with indentation (pretty print)
prettyJSON, _ := json.MarshalIndent(user, "", " ")
fmt.Println(string(prettyJSON))

// 3. Unmarshal (JSON -> struct)
jsonString := `{"id":2,"name":"Jane Doe","email":"jane@example.com"}`
var user2 User
err = json.Unmarshal([]byte(jsonString), &user2)
if err != nil {
    panic(err)
}
fmt.Printf("%+v\n", user2)

// 4. Unmarshal to map
var data map[string]interface{}
err = json.Unmarshal([]byte(jsonString), &data)
fmt.Printf("name: %v\n", data["name"])

// 5. Streaming JSON (for large data)
decoder := json.NewDecoder(strings.NewReader(jsonString))
var user3 User
decoder.Decode(&user3)
fmt.Printf("%+v\n", user3)

// 6. Encoder for streaming output
encoder := json.NewEncoder(os.Stdout)
encoder.SetIndent("", " ")
encoder.Encode(user)
}

```

ตัวอย่างการใช้งานจริง

1. API Response Wrapper

```

type APIResponse struct {
    Success bool      `json:"success"`
    Data    interface{} `json:"data,omitempty"`
    Error   *APIError   `json:"error,omitempty"`
    Meta    *APIMeta    `json:"meta,omitempty"`
}

```



```

type APIError struct {
    Code    int    `json:"code"`
    Message string `json:"message"`
    Details string `json:"details,omitempty"`
}

type APIMeta struct {
    Page        int `json:"page"`
    PerPage     int `json:"per_page"`
    Total       int `json:"total"`
    TotalPages  int `json:"total_pages"`
}

func JSONResponse(w http.ResponseWriter, status int, data interface{}) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)

    response := APIResponse{
        Success: status >= 200 && status < 300,
        Data:    data,
    }

    json.NewEncoder(w).Encode(response)
}

func JSONError(w http.ResponseWriter, status int, code int, message string) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)

    response := APIResponse{
        Success: false,
        Error: &APIError{
            Code:    code,
            Message: message,
        },
    }

    json.NewEncoder(w).Encode(response)
}

// ใช้งาน
func GetUserHandler(w http.ResponseWriter, r *http.Request) {
    user, err := userService.GetUser(1)
    if err != nil {
        JSONError(w, http.StatusNotFound, 404, "User not found")
        return
    }

    JSONResponse(w, http.StatusOK, user)
}

```

2. JSON Validation with Struct Tags

```
type RegisterRequest struct {
    Username string `json:"username" validate:"required,min=3,max=50"`
    Email     string `json:"email" validate:"required,email"`
    Password  string `json:"password" validate:"required,min=8"`
    Age       int    `json:"age" validate:"gte=18,lte=99"`
    Phone     string `json:"phone" validate:"omitempty,e164"`
}

func (r *RegisterRequest) Validate() error {
    validate := validator.New()
    return validate.Struct(r)
}

// ใช้งาน
func RegisterHandler(w http.ResponseWriter, r *http.Request) {
    var req RegisterRequest

    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        JSONError(w, http.StatusBadRequest, 400, "Invalid JSON")
        return
    }

    if err := req.Validate(); err != nil {
        JSONError(w, http.StatusBadRequest, 400, err.Error())
        return
    }

    // process registration...
}
```

3. XML Handling

```
package main

import (
    "encoding/xml"
    "fmt"
)

// XML struct with tags
type Person struct {
    XMLName xml.Name `xml:"person"`
    ID      int    `xml:"id,attr"`
    Name     string `xml:"name"`
    Age      int    `xml:"age"`
    Address  Address `xml:"address"`
    Phones   []Phone `xml:"phones>phone"`
}
```

```

}

type Address struct {
    Street string `xml:"street"`
    City    string `xml:"city"`
    Country string `xml:"country"`
}

type Phone struct {
    Type string `xml:"type,attr"`
    Number string `xml:",chardata"`
}

func main() {
    // Marshal struct -> XML
    person := Person{
        ID: 1,
        Name: "John Doe",
        Age: 30,
        Address: Address{
            Street: "123 Main St",
            City: "Bangkok",
            Country: "Thailand",
        },
        Phones: []Phone{
            {Type: "home", Number: "02-123-4567"},
            {Type: "mobile", Number: "081-234-5678"},
        },
    }

    xmlData, _ := xml.MarshalIndent(person, "", " ")
    fmt.Println(string(xmlData))

    // Unmarshal XML -> struct
    xmlString := `<?xml version="1.0" encoding="UTF-8"?>
<person id="2">
  <name>Jane Doe</name>
  <age>25</age>
  <address>
    <street>456 Oak Ave</street>
    <city>Chiang Mai</city>
    <country>Thailand</country>
  </address>
  <phones>
    <phone type="mobile">082-987-6543</phone>
  </phones>
</person>`

    var person2 Person
    xml.Unmarshal([]byte(xmlString), &person2)
    fmt.Printf("%+v\n", person2)
}

```

บทที่ 26: พื้นฐานการสร้าง HTTP Server

เป้าหมาย: สร้าง HTTP server ที่มีประสิทธิภาพและปลอดภัย

Basic HTTP Server

```
package main

import (
    "context"
    "fmt"
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {
    // 1. Basic handler
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Hello, %s!", r.URL.Path[1:])
    })

    // 2. Different HTTP methods
    http.HandleFunc("/api/users", func(w http.ResponseWriter, r *http.Request)
{
        switch r.Method {
        case http.MethodGet:
            // get users
            w.Write([]byte("GET users"))
        case http.MethodPost:
            // create user
            w.Write([]byte("POST users"))
        case http.MethodPut:
            // update user
            w.Write([]byte("PUT users"))
        case http.MethodDelete:
            // delete user
            w.Write([]byte("DELETE users"))
        default:
            http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
        }
    })

    // 3. Server configuration
    server := &http.Server{
        Addr:      ":8080",
```

```

    ReadTimeout: 15 * time.Second,
    WriteTimeout: 15 * time.Second,
    IdleTimeout: 60 * time.Second,
    Handler:      nil, // use default ServeMux
}

// 4. Graceful shutdown
go func() {
    log.Printf("Server starting on %s", server.Addr)
    if err := server.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
        log.Fatalf("Server failed: %v", err)
    }
}()

// Wait for interrupt signal
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
log.Println("Shutting down server...")

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := server.Shutdown(ctx); err != nil {
    log.Fatalf("Server forced to shutdown: %v", err)
}

log.Println("Server exited")
}

```

Using chi Router

```

package main

import (
    "context"
    "encoding/json"
    "net/http"
    "time"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
    "github.com/go-chi/cors"
    "github.com/go-chi/httprate"
)

func main() {
    r := chi.NewRouter()

```

```

// Global middlewares
r.Use(middleware.Logger)           // log requests
r.Use(middleware.Recoverer)        // recover from panics
r.Use(middleware.RealIP)           // get real IP
r.Use(middleware.RequestID)        // add request ID
r.Use(middleware.Timeout(60 * time.Second)) // timeout

// CORS
r.Use(cors.Handler(cors.Options{
    AllowedOrigins:  []string{"https://*", "http://*"},
    AllowedMethods:  []string{"GET", "POST", "PUT", "DELETE", "OPTIONS"},
    AllowedHeaders:  []string{"Accept", "Authorization", "Content-Type"},
    ExposedHeaders:  []string{"Link"},
    AllowCredentials: false,
    MaxAge:          300,
}))

// Rate limiting
r.Use(httprate.LimitByIP(100, 1*time.Minute))

// Routes
r.Get("/", homeHandler)

r.Route("/api/v1", func(r chi.Router) {
    r.Get("/health", healthHandler)

    r.Route("/users", func(r chi.Router) {
        r.Get("/", listUsersHandler)
        r.Post("/", createUserHandler)

        r.Route("/{userID}", func(r chi.Router) {
            r.Use(userContext)
            r.Get("/", getUserHandler)
            r.Put("/", updateUserHandler)
            r.Delete("/", deleteUserHandler)
        })
    })
})

// Static files
r.Handle("/static/*", http.StripPrefix("/static/",
http.FileServer(http.Dir("./static"))))

// Start server
http.ListenAndServe(":8080", r)
}

// Middleware to load user from database
func userContext(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        userID := chi.URLParam(r, "userID")

        // Load user from database

```

```

        user, err := loadUser(userID)
        if err != nil {
            http.Error(w, "User not found", http.StatusNotFound)
            return
        }

        ctx := context.WithValue(r.Context(), "user", user)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

func getUserHandler(w http.ResponseWriter, r *http.Request) {
    user := r.Context().Value("user").(*User)
    json.NewEncoder(w).Encode(user)
}

```

ตัวอย่างการใช้งานจริง: REST API Server

```

// models/user.go
package models

import (
    "time"
    "golang.org/x/crypto/bcrypt"
)

type User struct {
    ID          int      `json:"id"`
    Username    string   `json:"username"`
    Email       string   `json:"email"`
    Password    string   `json:"-"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}

func (u *User) HashPassword() error {
    hashed, err := bcrypt.GenerateFromPassword([]byte(u.Password),
        bcrypt.DefaultCost)
    if err != nil {
        return err
    }
    u.Password = string(hashed)
    return nil
}

func (u *User) CheckPassword(password string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(u.Password), []byte(password))
    return err == nil
}

```

```
// handlers/user_handler.go
package handlers

import (
    "encoding/json"
    "net/http"
    "strconv"

    "github.com/go-chi/chi/v5"
    "github.com/go-playground/validator/v10"
)

type UserHandler struct {
    userService UserService
    validate     *validator.Validate
}

func NewUserHandler(userService UserService) *UserHandler {
    return &UserHandler{
        userService: userService,
        validate:     validator.New(),
    }
}

func (h *UserHandler) CreateUser(w http.ResponseWriter, r *http.Request) {
    var req CreateUserRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        JSONError(w, http.StatusBadRequest, "Invalid request body")
        return
    }

    if err := h.validate.Struct(req); err != nil {
        JSONError(w, http.StatusBadRequest, err.Error())
        return
    }

    user, err := h.userService.CreateUser(r.Context(), &req)
    if err != nil {
        JSONError(w, http.StatusInternalServerError, err.Error())
        return
    }

    JSONResponse(w, http.StatusCreated, user)
}

func (h *UserHandler) GetUser(w http.ResponseWriter, r *http.Request) {
    idStr := chi.URLParam(r, "id")
    id, err := strconv.Atoi(idStr)
    if err != nil {
        JSONError(w, http.StatusBadRequest, "Invalid user ID")
        return
    }
}
```



```

    user, err := h.userService.GetUserByID(r.Context(), id)
    if err != nil {
        JSONError(w, http.StatusNotFound, "User not found")
        return
    }

    JSONResponse(w, http.StatusOK, user)
}

func (h *UserHandler) UpdateUser(w http.ResponseWriter, r *http.Request) {
    idStr := chi.URLParam(r, "id")
    id, _ := strconv.Atoi(idStr)

    var req UpdateUserRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        JSONError(w, http.StatusBadRequest, "Invalid request body")
        return
    }

    user, err := h.userService.UpdateUser(r.Context(), id, &req)
    if err != nil {
        JSONError(w, http.StatusInternalServerError, err.Error())
        return
    }

    JSONResponse(w, http.StatusOK, user)
}

func (h *UserHandler) DeleteUser(w http.ResponseWriter, r *http.Request) {
    idStr := chi.URLParam(r, "id")
    id, _ := strconv.Atoi(idStr)

    if err := h.userService.DeleteUser(r.Context(), id); err != nil {
        JSONError(w, http.StatusInternalServerError, err.Error())
        return
    }

    w.WriteHeader(http.StatusNoContent)
}

func (h *UserHandler) ListUsers(w http.ResponseWriter, r *http.Request) {
    page, _ := strconv.Atoi(r.URL.Query().Get("page"))
    if page < 1 {
        page = 1
    }

    limit, _ := strconv.Atoi(r.URL.Query().Get("limit"))
    if limit < 1 || limit > 100 {
        limit = 20
    }

    users, total, err := h.userService.ListUsers(r.Context(), page, limit)
    if err != nil {

```

```

        JSONError(w, http.StatusInternalServerError, err.Error())
        return
    }

    response := &ListUsersResponse{
        Data:      users,
        Total:      total,
        Page:      page,
        PerPage:    limit,
        TotalPages: (total + limit - 1) / limit,
    }

    JSONResponse(w, http.StatusOK, response)
}

```

บทที่ 27: Enum, Iota และ Bitmask

เป้าหมาย: ใช้ `iota` สำหรับสร้าง enum และ bitmask

Basic Iota

```

package main

import "fmt"

// 1. Basic enum
type Weekday int

const (
    Sunday Weekday = iota // 0
    Monday                // 1
    Tuesday               // 2
    Wednesday             // 3
    Thursday              // 4
    Friday                // 5
    Saturday              // 6
)

// 2. Skip values
const (
    _ = iota // 0 (skip)
    KB = 1 << (10 * iota) // 1 << 10 = 1024
    MB                    // 1 << 20 = 1048576
    GB                    // 1 << 30 = 1073741824
    TB                    // 1 << 40 = 1099511627776
)

// 3. Enum with string representation
type Status int

```

```

const (
    StatusPending Status = iota
    StatusApproved
    StatusRejected
    StatusCancelled
)

func (s Status) String() string {
    return [...]string{"Pending", "Approved", "Rejected", "Cancelled"}[s]
}

// 4. Enum with JSON marshaling
type OrderStatus int

const (
    OrderStatusCreated OrderStatus = iota + 1 // start from 1
    OrderStatusPaid
    OrderStatusShipped
    OrderStatusDelivered
    OrderStatusCancelled
)

func (s OrderStatus) MarshalJSON() ([]byte, error) {
    return []byte(fmt.Sprintf(`"%s"`, s.String())), nil
}

func (s OrderStatus) String() string {
    return [...]string{"", "Created", "Paid", "Shipped", "Delivered",
    "Cancelled"}[s]
}

func main() {
    fmt.Println(Sunday)    // 0
    fmt.Println(Monday)   // 1
    fmt.Println(KB)        // 1024
    fmt.Println(StatusPending) // 0
    fmt.Println(StatusPending.String()) // "Pending"
}

```

Bitmask with Iota

```

package main

import "fmt"

// Permission flags (bitmask)
type Permission int

const (

```

```

PermRead   Permission = 1 << iota // 1 (001)
PermWrite                                     // 2 (010)
PermExecute                                     // 4 (100)
PermDelete                                     // 8 (1000)
PermAdmin   = PermRead | PermWrite | PermExecute | PermDelete // 15 (1111)
)

func (p Permission) String() string {
    if p == 0 {
        return "None"
    }

    var flags []string
    if p&PermRead != 0 {
        flags = append(flags, "Read")
    }
    if p&PermWrite != 0 {
        flags = append(flags, "Write")
    }
    if p&PermExecute != 0 {
        flags = append(flags, "Execute")
    }
    if p&PermDelete != 0 {
        flags = append(flags, "Delete")
    }

    return strings.Join(flags, "|")
}

func (p *Permission) Add(perm Permission) {
    *p |= perm
}

func (p *Permission) Remove(perm Permission) {
    *p &^= perm
}

func (p Permission) Has(perm Permission) bool {
    return p&perm != 0
}

// ตัวอย่างการใช้งาน
type User struct {
    Name      string
    Permissions Permission
}

func main() {
    user := User{
        Name:      "john",
        Permissions: PermRead | PermWrite,
    }
}

```

```

fmt.Println(user.Permissions) // Read|Write
fmt.Println(user.Permissions.Has(PermRead)) // true
fmt.Println(user.Permissions.Has(PermDelete)) // false

user.Permissions.Add(PermDelete)
fmt.Println(user.Permissions) // Read|Write|Delete

user.Permissions.Remove(PermWrite)
fmt.Println(user.Permissions) // Read|Delete
}

```

ตัวอย่างการใช้งานจริง

1. Order Status Management

```

type OrderStatus int

const (
    OrderStatusDraft OrderStatus = iota + 1
    OrderStatusPending
    OrderStatusConfirmed
    OrderStatusProcessing
    OrderStatusShipped
    OrderStatusDelivered
    OrderStatusCancelled
    OrderStatusRefunded
)

var orderStatusTransitions = map[OrderStatus][]OrderStatus{
    OrderStatusDraft:    {OrderStatusPending, OrderStatusCancelled},
    OrderStatusPending:  {OrderStatusConfirmed, OrderStatusCancelled},
    OrderStatusConfirmed: {OrderStatusProcessing, OrderStatusCancelled},
    OrderStatusProcessing: {OrderStatusShipped, OrderStatusCancelled},
    OrderStatusShipped:  {OrderStatusDelivered},
    OrderStatusDelivered: {OrderStatusRefunded},
    OrderStatusCancelled: {},
    OrderStatusRefunded: {},
}

func (s OrderStatus) CanTransitionTo(target OrderStatus) bool {
    allowed := orderStatusTransitions[s]
    for _, allowedStatus := range allowed {
        if allowedStatus == target {
            return true
        }
    }
    return false
}

func (s OrderStatus) IsFinal() bool {

```

```

        return s == OrderStatusDelivered ||
               s == OrderStatusCancelled ||
               s == OrderStatusRefunded
    }

    func (s OrderStatus) String() string {
        return [...]string{
            "", "Draft", "Pending", "Confirmed",
            "Processing", "Shipped", "Delivered",
            "Cancelled", "Refunded",
        }[s]
    }

    // ใช้งาน
    type Order struct {
        ID      int
        Status  OrderStatus
    }

    func (o *Order) TransitionTo(newStatus OrderStatus) error {
        if !o.Status.CanTransitionTo(newStatus) {
            return fmt.Errorf("cannot transition from %s to %s",
                o.Status, newStatus)
        }

        o.Status = newStatus
        return nil
    }

```

2. HTTP Status Code Group

```

type HTTPStatusGroup int

const (
    StatusGroupInfo HTTPStatusGroup = iota + 1
    StatusGroupSuccess
    StatusGroupRedirect
    StatusGroupClientError
    StatusGroupServerError
)

func GetStatusGroup(code int) HTTPStatusGroup {
    switch {
    case code >= 100 && code < 200:
        return StatusGroupInfo
    case code >= 200 && code < 300:
        return StatusGroupSuccess
    case code >= 300 && code < 400:
        return StatusGroupRedirect
    case code >= 400 && code < 500:
        return StatusGroupClientError
    }
}

```

```

    case code >= 500 && code < 600:
        return StatusGroupServerError
    default:
        return 0
}

}

func (g HTTPStatusGroup) String() string {
    return [...]string{
        "", "Informational", "Success",
        "Redirection", "Client Error", "Server Error",
    }[g]
}

```

บทที่ 28: วันที่และเวลา (Time)

เป้าหมาย: จัดการ date, time, duration, timezone อย่างถูกต้อง

Time Basics

```

package main

import (
    "fmt"
    "time"
)

func main() {
    // 1. Current time
    now := time.Now()
    fmt.Printf("Now: %v\n", now)
    fmt.Printf("Unix: %d\n", now.Unix())
    fmt.Printf("UnixNano: %d\n", now.UnixNano())

    // 2. Creating time
    t1 := time.Date(2024, time.December, 25, 10, 30, 0, 0, time.UTC)
    fmt.Println(t1)

    // 3. Parsing time
    t2, _ := time.Parse("2006-01-02 15:04:05", "2024-12-25 10:30:00")
    fmt.Println(t2)

    // 4. Formatting time
    fmt.Println(now.Format("2006-01-02 15:04:05"))
    fmt.Println(now.Format("Monday, 02 Jan 2006"))
    fmt.Println(now.Format(time.RFC3339))

    // 5. Time components
    fmt.Printf("Year: %d, Month: %d, Day: %d\n", now.Year(), now.Month(),

```

```

now.Day()
    fmt.Printf("Hour: %d, Minute: %d, Second: %d\n", now.Hour(), now.Minute(),
now.Second())
    fmt.Printf("Weekday: %s\n", now.Weekday())

// 6. Time arithmetic
tomorrow := now.Add(24 * time.Hour)
yesterday := now.Add(-24 * time.Hour)
oneWeekLater := now.AddDate(0, 0, 7)

fmt.Printf("Tomorrow: %v\n", tomorrow)
fmt.Printf("Yesterday: %v\n", yesterday)

// 7. Time difference
duration := tomorrow.Sub(now)
fmt.Printf("Duration: %v\n", duration)
fmt.Printf("Hours: %.2f\n", duration.Hours())
fmt.Printf("Minutes: %.2f\n", duration.Minutes())

// 8. Compare times
fmt.Println(now.Before(tomorrow)) // true
fmt.Println(now.After(yesterday)) // true
fmt.Println(now.Equal(now))       // true
}

```

Timezone Handling

```

package main

import (
    "fmt"
    "time"
)

func main() {
    // 1. Load location
    loc, _ := time.LoadLocation("Asia/Bangkok")
    nyLoc, _ := time.LoadLocation("America/New_York")

    // 2. Create time with location
    t := time.Date(2024, 12, 25, 10, 30, 0, 0, loc)
    fmt.Println(t)

    // 3. Convert between timezones
    tUTC := t.UTC()
    tNY := t.In(nyLoc)

    fmt.Printf("Bangkok: %v\n", t)
    fmt.Printf("UTC: %v\n", tUTC)
    fmt.Printf("New York: %v\n", tNY)
}

```



```

// 4. Parse with timezone
t2, _ := time.ParseInLocation(
    "2006-01-02 15:04:05",
    "2024-12-25 10:30:00",
    loc,
)
fmt.Println(t2)

// 5. Get current time in different timezone
nowBKK := time.Now().In(loc)
nowNY := time.Now().In(nyLoc)
fmt.Printf("Bangkok now: %v\n", nowBKK)
fmt.Printf("New York now: %v\n", nowNY)
}

```

Duration and Ticker

```

package main

import (
    "fmt"
    "time"
)

func main() {
    // 1. Duration parsing
    duration, _ := time.ParseDuration("1h30m")
    fmt.Println(duration) // 1h30m0s

    duration2, _ := time.ParseDuration("2h15m30s")
    fmt.Println(duration2.Hours()) // 2.2583333333333333
    fmt.Println(duration2.Minutes()) // 135.5

    // 2. Sleep
    fmt.Println("Sleeping...")
    time.Sleep(2 * time.Second)
    fmt.Println("Awake!")

    // 3. Timer (one-shot)
    timer := time.NewTimer(3 * time.Second)
    <-timer.C
    fmt.Println("Timer expired")

    // 4. Timer with stop
    timer2 := time.NewTimer(5 * time.Second)
    go func() {
        <-timer2.C
        fmt.Println("Timer expired")
    }()
}

```

```

time.Sleep(2 * time.Second)
if timer2.Stop() {
    fmt.Println("Timer stopped")
}

// 5. Ticker (repeating)
ticker := time.NewTicker(1 * time.Second)
done := make(chan bool)

go func() {
    for {
        select {
        case t := <-ticker.C:
            fmt.Printf("Tick at %v\n", t)
        case <-done:
            ticker.Stop()
            return
        }
    }
}()

time.Sleep(5 * time.Second)
done <- true
fmt.Println("Ticker stopped")

// 6. After (timeout pattern)
timeout := time.After(3 * time.Second)
select {
case <-timeout:
    fmt.Println("Timeout!")
}

// 7. AfterFunc
time.AfterFunc(2*time.Second, func() {
    fmt.Println("AfterFunc executed")
})

time.Sleep(3 * time.Second)
}

```

ตัวอย่างการใช้งานจริง

1. Date Range Validation

```

type DateRange struct {
    Start time.Time
    End   time.Time
}

```

```

func (r *DateRange) Validate() error {
    if r.Start.IsZero() {
        return errors.New("start date is required")
    }
    if r.End.IsZero() {
        return errors.New("end date is required")
    }
    if r.End.Before(r.Start) {
        return errors.New("end date must be after start date")
    }
    return nil
}

func (r *DateRange) Contains(t time.Time) bool {
    return (t.Equal(r.Start) || t.After(r.Start)) &&
        (t.Equal(r.End) || t.Before(r.End))
}

func (r *DateRange) Overlaps(other *DateRange) bool {
    return r.Start.Before(other.End) && r.End.After(other.Start)
}

func (r *DateRange) Duration() time.Duration {
    return r.End.Sub(r.Start)
}

// ใช้งาน
func GetBookingsForDateRange(db *sql.DB, start, end time.Time) ([]Booking,
error) {
    range := &DateRange{Start: start, End: end}
    if err := range.Validate(); err != nil {
        return nil, err
    }

    query := `SELECT id, user_id, start_time, end_time
              FROM bookings
              WHERE start_time >= ? AND end_time <= ?`

    rows, err := db.Query(query, start, end)
    // ... process rows
}

```

2. Rate Limiter with Time Window

```

type RateLimiter struct {
    limit      int
    window     time.Duration
    requests   map[string][]time.Time
    mu         sync.RWMutex
}

```

```

func NewRateLimiter(limit int, window time.Duration) *RateLimiter {
    return &RateLimiter{
        limit:    limit,
        window:   window,
        requests: make(map[string][]time.Time),
    }
}

func (rl *RateLimiter) Allow(key string) bool {
    rl.mu.Lock()
    defer rl.mu.Unlock()

    now := time.Now()
    cutoff := now.Add(-rl.window)

    // เก็บเฉพาะ requests ที่อยู่ใน window
    requests := rl.requests[key]
    valid := []time.Time{}
    for _, t := range requests {
        if t.After(cutoff) {
            valid = append(valid, t)
        }
    }

    if len(valid) >= rl.limit {
        return false
    }

    valid = append(valid, now)
    rl.requests[key] = valid
    return true
}

func (rl *RateLimiter) Cleanup() {
    rl.mu.Lock()
    defer rl.mu.Unlock()

    cutoff := time.Now().Add(-rl.window)
    for key, requests := range rl.requests {
        valid := []time.Time{}
        for _, t := range requests {
            if t.After(cutoff) {
                valid = append(valid, t)
            }
        }
        if len(valid) == 0 {
            delete(rl.requests, key)
        } else {
            rl.requests[key] = valid
        }
    }
}

```

```
// ใช้งาน
func main() {
    limiter := NewRateLimiter(100, time.Minute)

    // Cleanup every hour
    ticker := time.NewTicker(time.Hour)
    go func() {
        for range ticker.C {
            limiter.Cleanup()
        }
    }()

    // Use in handler
    http.HandleFunc("/api", func(w http.ResponseWriter, r *http.Request) {
        ip := r.RemoteAddr

        if !limiter.Allow(ip) {
            http.Error(w, "Rate limit exceeded", http.StatusTooManyRequests)
            return
        }

        // process request
    })
}
```

3. Schedule Job with Cron-like

```
type Scheduler struct {
    jobs    []*Job
    stopCh  chan struct{}
}

type Job struct {
    Name      string
    Schedule  string // "0 9 * * *" = every day at 9am
    Task      func()
    LastRun   time.Time
    NextRun   time.Time
}

func NewScheduler() *Scheduler {
    return &Scheduler{
        jobs:    []*Job{},
        stopCh:  make(chan struct{}),
    }
}

func (s *Scheduler) AddJob(job *Job) {
    s.jobs = append(s.jobs, job)
    s.calculateNextRun(job)
}
```

```

func (s *Scheduler) calculateNextRun(job *Job) {
    // Simplified cron parsing (use robfig/cron for production)
    now := time.Now()
    job.NextRun = now.Add(24 * time.Hour) // placeholder
}

func (s *Scheduler) Start() {
    ticker := time.NewTicker(1 * time.Minute)
    go func() {
        for {
            select {
            case <-ticker.C:
                s.checkJobs()
            case <-s.stopCh:
                ticker.Stop()
                return
            }
        }
    }()
}

func (s *Scheduler) checkJobs() {
    now := time.Now()
    for _, job := range s.jobs {
        if now.After(job.NextRun) {
            go job.Task()
            job.LastRun = now
            s.calculateNextRun(job)
        }
    }
}

func (s *Scheduler) Stop() {
    close(s.stopCh)
}

```

2. การประมวลผล (Processing Layer)

บทที่ 30: การทำงานพร้อมกัน (Concurrency)

เป้าหมาย: ใช้ goroutines และ channels ในการเขียนโปรแกรม concurrent

Goroutines Basics

```

package main

import (
    "fmt"

```

```

"sync"
"time"
)

func main() {
    // 1. Start a goroutine
    go func() {
        fmt.Println("Hello from goroutine")
    }()

    // 2. Wait for goroutine (bad way)
    time.Sleep(100 * time.Millisecond)

    // 3. Use WaitGroup
    var wg sync.WaitGroup

    for i := 1; i <= 5; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()
            fmt.Printf("Worker %d done\n", id)
        }(i)
    }

    wg.Wait()
    fmt.Println("All workers done")

    // 4. Goroutine with panic recovery
    var wg2 sync.WaitGroup
    wg2.Add(1)

    go func() {
        defer wg2.Done()
        defer func() {
            if r := recover(); r != nil {
                fmt.Printf("Recovered from panic: %v\n", r)
            }
        }()

        panic("something went wrong")
    }()

    wg2.Wait()
}

```

Channels

```

package main

import (

```

```

    "fmt"
    "time"
)

func main() {
    // 1. Unbuffered channel
    ch := make(chan int)

    go func() {
        ch <- 42 // send
    }()

    value := <-ch // receive
    fmt.Println(value)

    // 2. Buffered channel
    buffered := make(chan string, 3)
    buffered <- "one"
    buffered <- "two"
    buffered <- "three"
    // buffered <- "four" // would block

    fmt.Println(<-buffered)
    fmt.Println(<-buffered)
    fmt.Println(<-buffered)

    // 3. Channel with close
    ch2 := make(chan int, 5)

    go func() {
        for i := 0; i < 5; i++ {
            ch2 <- i
        }
        close(ch2)
    }()

    // Range over channel (auto stops when closed)
    for v := range ch2 {
        fmt.Println(v)
    }

    // 4. Select statement
    ch3 := make(chan string)
    ch4 := make(chan string)

    go func() {
        time.Sleep(1 * time.Second)
        ch3 <- "from channel 3"
    }()

    go func() {
        time.Sleep(2 * time.Second)
        ch4 <- "from channel 4"
    }()
}

```



```

}()

for i := 0; i < 2; i++ {
    select {
        case msg1 := <-ch3:
            fmt.Println(msg1)
        case msg2 := <-ch4:
            fmt.Println(msg2)
        case <-time.After(3 * time.Second):
            fmt.Println("timeout")
    }
}

// 5. Non-blocking channel operations
ch5 := make(chan int, 1)

select {
case ch5 <- 42:
    fmt.Println("Sent")
default:
    fmt.Println("Channel is full")
}

select {
case v := <-ch5:
    fmt.Printf("Received: %d\n", v)
default:
    fmt.Println("Channel is empty")
}
}

```

Worker Pool Pattern

```

package main

import (
    "context"
    "fmt"
    "sync"
    "time"
)

type Job struct {
    ID    int
    Data  interface{}
}

type Result struct {
    JobID    int
    Output   interface{}
}

```

```

    Error    error
}

type WorkerPool struct {
    numWorkers int
    jobQueue   chan Job
    resultQueue chan Result
    wg         sync.WaitGroup
    ctx        context.Context
    cancel     context.CancelFunc
}

func NewWorkerPool(numWorkers int, queueSize int) *WorkerPool {
    ctx, cancel := context.WithCancel(context.Background())

    return &WorkerPool{
        numWorkers: numWorkers,
        jobQueue:   make(chan Job, queueSize),
        resultQueue: make(chan Result, queueSize),
        ctx:       ctx,
        cancel:    cancel,
    }
}

func (wp *WorkerPool) Start(workerFunc func(Job) Result) {
    // Start workers
    for i := 0; i < wp.numWorkers; i++ {
        wp.wg.Add(1)
        go wp.worker(i, workerFunc)
    }

    // Wait for all workers to finish in background
    go func() {
        wp.wg.Wait()
        close(wp.resultQueue)
    }()
}

func (wp *WorkerPool) worker(id int, workerFunc func(Job) Result) {
    defer wp.wg.Done()

    for {
        select {
        case <-wp.ctx.Done():
            fmt.Printf("Worker %d stopping\n", id)
            return
        case job, ok := <-wp.jobQueue:
            if !ok {
                return
            }
            result := workerFunc(job)
            select {
            case wp.resultQueue <- result:

```

```

        case <-wp.ctx.Done():
            return
    }
}

func (wp *WorkerPool) Submit(job Job) {
    select {
    case wp.jobQueue <- job:
    case <-wp.ctx.Done():
    }
}

func (wp *WorkerPool) Results() <-chan Result {
    return wp.resultQueue
}

func (wp *WorkerPool) Stop() {
    wp.cancel()
    close(wp.jobQueue)
    wp.wg.Wait()
}

// ตัวอย่างการใช้งาน
func main() {
    // Create worker pool with 3 workers
    pool := NewWorkerPool(3, 100)

    // Define worker function
    workerFunc := func(job Job) Result {
        // Simulate work
        time.Sleep(time.Duration(job.ID) * 100 * time.Millisecond)

        return Result{
            JobID: job.ID,
            Output: fmt.Sprintf("Processed job %d", job.ID),
        }
    }

    pool.Start(workerFunc)

    // Submit 10 jobs
    for i := 1; i <= 10; i++ {
        pool.Submit(Job{ID: i, Data: fmt.Sprintf("data-%d", i)})
    }

    // Collect results
    for result := range pool.Results() {
        if result.Error != nil {
            fmt.Printf("Job %d failed: %v\n", result.JobID, result.Error)
        } else {
            fmt.Printf("Job %d: %s\n", result.JobID, result.Output)
        }
    }
}

```

```

    }
}

pool.Stop()
}

```

Pipeline Pattern

```

package main

import (
    "fmt"
    "strings"
)

func main() {
    // Pipeline: numbers -> square -> filter -> sum
    numbers := generate(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

    squared := square(numbers)
    filtered := filter(squared, func(n int) bool {
        return n%2 == 0 // even numbers
    })

    sum := sum(filtered)

    fmt.Println("Sum of squares of even numbers:", <-sum)
}

func generate(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        for _, n := range nums {
            out <- n
        }
        close(out)
    }()
    return out
}

func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        for n := range in {
            out <- n * n
        }
        close(out)
    }()
    return out
}

```

```

func filter(in <-chan int, predicate func(int) bool) <-chan int {
    out := make(chan int)
    go func() {
        for n := range in {
            if predicate(n) {
                out <- n
            }
        }
        close(out)
    }()
    return out
}

func sum(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        total := 0
        for n := range in {
            total += n
        }
        out <- total
        close(out)
    }()
    return out
}

```

Fan-Out / Fan-In Pattern

```

package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    // Generate numbers
    numbers := generateNumbers(100)

    // Fan-out: create multiple workers
    worker1 := squareWorker(numbers)
    worker2 := squareWorker(numbers)
    worker3 := squareWorker(numbers)

    // Fan-in: merge results from all workers
    results := merge(worker1, worker2, worker3)

    // Collect results
}

```

```

    for result := range results {
        fmt.Println(result)
    }
}

func generateNumbers(n int) <-chan int {
    out := make(chan int)
    go func() {
        for i := 1; i <= n; i++ {
            out <- i
        }
        close(out)
    }()
    return out
}

func squareWorker(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        for n := range in {
            time.Sleep(10 * time.Millisecond) // simulate work
            out <- n * n
        }
        close(out)
    }()
    return out
}

func merge(channels ...<-chan int) <-chan int {
    var wg sync.WaitGroup
    out := make(chan int)

    // Start a goroutine for each input channel
    output := func(c <-chan int) {
        for n := range c {
            out <- n
        }
        wg.Done()
    }

    wg.Add(len(channels))
    for _, c := range channels {
        go output(c)
    }

    // Close output channel when all workers done
    go func() {
        wg.Wait()
        close(out)
    }()

    return out
}

```

บทที่ 31: การบันทึกเหตุการณ์ (Logging)

เป้าหมาย: ใช้ structured logging สำหรับการ debug และ monitoring

Using slog (Go 1.21+)

```
package main

import (
    "context"
    "log/slog"
    "os"
    "time"
)

func main() {
    // 1. Basic logging
    slog.Info("Application started")
    slog.Warn("Low memory", "usage", 85)
    slog.Error("Database connection failed", "error", "timeout")

    // 2. Structured logging with attributes
    slog.Info("User logged in",
        "user_id", 123,
        "username", "john_doe",
        "ip", "192.168.1.1",
    )

    // 3. Log with group
    slog.Info("Request processed",
        slog.Group("request",
            "method", "GET",
            "path", "/api/users",
            "duration_ms", 150,
        ),
        slog.Group("response",
            "status", 200,
            "size", 1024,
        ),
    )

    // 4. Log with context
    ctx := context.WithValue(context.Background(), "request_id", "abc-123")
    slog.InfoContext(ctx, "Processing request", "user_id", 123)

    // 5. Set log level
    opts := &slog.HandlerOptions{
        Level: slog.LevelDebug,
```

```

}
handler := slog.NewJSONHandler(os.Stdout, opts)
logger := slog.New(handler)
slog.SetDefault(logger)

logger.Debug("Debug message", "detail", "only shown in debug mode")

// 6. Custom handler with fields
handler = slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level: slog.LevelInfo,
    ReplaceAttr: func(groups []string, a slog.Attr) slog.Attr {
        // Rename timestamp to time
        if a.Key == slog.TimeKey {
            a.Key = "timestamp"
        }
        return a
    },
})
slog.SetDefault(slog.New(handler))

slog.Info("With custom timestamp key", "value", 42)
}

```

Production Logging Setup

```

package logger

import (
    "io"
    "log/slog"
    "os"
    "time"
)

type Config struct {
    Level      string `yaml:"level"` // debug, info, warn, error
    Format     string `yaml:"format"` // json, text
    OutputPath string `yaml:"output"` // stdout, stderr, file path
}

func NewLogger(cfg Config) (*slog.Logger, error) {
    // Parse log level
    var level slog.Level
    switch cfg.Level {
    case "debug":
        level = slog.LevelDebug
    case "info":
        level = slog.LevelInfo
    case "warn":
        level = slog.LevelWarn
    }
}

```



```

case "error":
    level = slog.LevelError
default:
    level = slog.LevelInfo
}

// Setup output
var writer io.Writer
switch cfg.OutputPath {
case "stdout":
    writer = os.Stdout
case "stderr":
    writer = os.Stderr
default:
    file, err := os.OpenFile(cfg.OutputPath,
        os.O_CREATE|os.O_WRONLY|os.O_APPEND, 0644)
    if err != nil {
        return nil, err
    }
    writer = file
}

// Create handler
var handler slog.Handler
opts := &slog.HandlerOptions{
    Level: level,
    ReplaceAttr: func(groups []string, a slog.Attr) slog.Attr {
        // Add service name to all logs
        if a.Key == slog.TimeKey {
            a.Value = slog.StringValue(a.Value.Time().Format(time.RFC3339))
        }
        return a
    },
}

if cfg.Format == "json" {
    handler = slog.NewJSONHandler(writer, opts)
} else {
    handler = slog.NewTextHandler(writer, opts)
}

return slog.New(handler), nil
}

// Middleware for HTTP logging
func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()

        // Capture response status
        rw := &responseWriter{ResponseWriter: w, status: http.StatusOK}

        next.ServeHTTP(rw, r)
    })
}

```

```

        duration := time.Since(start)

        slog.Info("HTTP request",
            "method", r.Method,
            "path", r.URL.Path,
            "status", rw.status,
            "duration_ms", duration.Milliseconds(),
            "ip", r.RemoteAddr,
            "user_agent", r.UserAgent(),
        )
    })
}

type responseWriter struct {
    http.ResponseWriter
    status int
}

func (rw *responseWriter) WriteHeader(code int) {
    rw.status = code
    rw.ResponseWriter.WriteHeader(code)
}

```

Zap Logger (Alternative)

```

package main

import (
    "go.uber.org/zap"
    "go.uber.org/zap/zapcore"
)

func main() {
    // 1. Production logger
    logger, _ := zap.NewProduction()
    defer logger.Sync()

    logger.Info("Application started",
        zap.String("version", "1.0.0"),
        zap.Int("port", 8080),
    )

    logger.Error("Database error",
        zap.Error(fmt.Errorf("connection refused")),
        zap.String("host", "localhost"),
    )

    // 2. Development logger
    devLogger, _ := zap.NewDevelopment()
}

```

```

devLogger.Debug("Debug message", zap.String("key", "value"))

// 3. Custom logger
config := zap.Config{
    Level:      zap.NewAtomicLevelAt(zap.InfoLevel),
    Development: false,
    Encoding:   "json",
    EncoderConfig: zapcore.EncoderConfig{
        TimeKey:      "timestamp",
        LevelKey:     "level",
        NameKey:      "logger",
        CallerKey:    "caller",
        MessageKey:  "message",
        StacktraceKey: "stacktrace",
        LineEnding:  zapcore.DefaultLineEnding,
        EncodeLevel:  zapcore.LowercaseLevelEncoder,
        EncodeTime:   zapcore.ISO8601TimeEncoder,
        EncodeDuration: zapcore.SecondsDurationEncoder,
        EncodeCaller: zapcore.ShortCallerEncoder,
    },
    OutputPaths:      []string{"stdout", "/var/log/app.log"},
    ErrorOutputPaths: []string{"stderr"},
}

customLogger, _ := config.Build()
defer customLogger.Sync()

customLogger.Info("Custom logger",
    zap.String("service", "myapp"),
)

// 4. Sugared logger (less typed)
sugar := logger.Sugar()
sugar.Infow("User login",
    "user_id", 123,
    "username", "john",
)
}

```

บทที่ 32: เทมเพลต (Templates)

เป้าหมาย: สร้าง HTML และ text templates สำหรับ dynamic content

Text Templates

```

package main

import (
    "bytes"

```

```

"os"
"text/template"
)

func main() {
    // 1. Basic template
    tpl := template.Must(template.New("greeting").Parse("Hello, {{.Name}}!"))
    tpl.Execute(os.Stdout, map[string]string{"Name": "World"})

    // 2. Template with conditionals
    const emailTpl = `
Subject: Welcome {{.Name}}

{{if .IsVIP}}
Dear VIP Customer {{.Name}},
{{else}}
Dear {{.Name}},
{{end}}

Thank you for joining our platform.
{{if .Discount}}
Use code {{.Discount.Code}} for {{.Discount.Percent}}% off!
{{end}}

Best regards,
The Team
`

    data := struct {
        Name      string
        IsVIP      bool
        Discount *Discount
    }{
        Name:      "John Doe",
        IsVIP:     true,
        Discount: &Discount{
            Code:      "WELCOME20",
            Percent: 20,
        },
    }

    tpl = template.Must(template.New("email").Parse(emailTpl))
    tpl.Execute(os.Stdout, data)

    // 3. Template with loops
    const listTpl = `
<ul>
{{range .Items}}
    <li>{{.}}</li>
{{else}}
    <li>No items found</li>
{{end}}
</ul>

```

```

tpl = template.Must(template.New("list").Parse(listTpl))
tpl.Execute(os.Stdout, map[string][]string{
    "Items": {"Apple", "Banana", "Orange"},
})

// 4. Template with custom functions
funcMap := template.FuncMap{
    "upper": strings.ToUpper,
    "lower": strings.ToLower,
    "formatDate": func(t time.Time) string {
        return t.Format("2006-01-02")
    },
    "multiply": func(a, b int) int {
        return a * b
    },
}

const funcTpl = `
Uppercase: {{.Name | upper}}
Lowercase: {{.Name | lower}}
Date: {{.CreatedAt | formatDate}}
Total: {{multiply .Price .Quantity}}
`

tpl = template.Must(template.New("func").Funcs(funcMap).Parse(funcTpl))
tpl.Execute(os.Stdout, struct {
    Name      string
    CreatedAt time.Time
    Price      int
    Quantity   int
}{
    Name:      "John Doe",
    CreatedAt: time.Now(),
    Price:     100,
    Quantity:  3,
})
}

```

HTML Templates

```

package main

import (
    "html/template"
    "net/http"
)

type PageData struct {

```

```

    Title      string
    User       *User
    Products   []Product
    IsLoggedIn bool
    Flash      string
    CSRFToken  string
}

func renderTemplate(w http.ResponseWriter, tmplName string, data interface{}) {
    // Parse templates with layout
    tmpl := template.Must(template.ParseFiles(
        "templates/layout.html",
        "templates/"+tmplName+".html",
    ))

    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    tmpl.ExecuteTemplate(w, "layout", data)
}

// templates/layout.html
/*
<!DOCTYPE html>
<html>
<head>
    <title>{{.Title}} - My App</title>
    <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
    <header>
        {{template "navbar" .}}
    </header>

    <main>
        {{if .Flash}}
        <div class="alert">{{.Flash}}</div>
        {{end}}

        {{template "content" .}}
    </main>

    <footer>
        &copy; 2024 My App
    </footer>

    <script src="/static/js/app.js"></script>
</body>
</html>
*/

// templates/navbar.html
/*
{{define "navbar"}}
<nav>

```

```

    <a href="/">Home</a>
    {{if .IsLoggedIn}}
        <a href="/profile">Profile</a>
        <a href="/logout">Logout</a>
    {{else}}
        <a href="/login">Login</a>
        <a href="/register">Register</a>
    {{end}}
</nav>
{{end}}
*/

// templates/home.html
/*
{{define "content"}}
<h1>Welcome, {{.User.Name}}!</h1>

<div class="products">
    <h2>Our Products</h2>
    <div class="product-grid">
        {{range .Products}}
            <div class="product-card">
                <h3>{{.Name}}</h3>
                <p>{{.Description}}</p>
                <div class="price">${{.Price}}</div>
                <form method="POST" action="/cart/add">
                    <input type="hidden" name="csrf_token" value="{{$.CSRFToken}}">
                    <input type="hidden" name="product_id" value="{{.ID}}">
                    <input type="number" name="quantity" value="1" min="1">
                    <button type="submit">Add to Cart</button>
                </form>
            </div>
        {{end}}
    </div>
{{end}}
</div>
{{end}}
*/

```

Template Functions and Helpers

```

package main

import (
    "html/template"
    "strings"
    "time"
)

// Custom template functions
var templateFuncs = template.FuncMap{

```

```

// String helpers
"upper":      strings.ToUpper,
"lower":      strings.ToLower,
"title":      strings.Title,
"trim":       strings.TrimSpace,
"truncate":   truncate,

// Number helpers
"formatNumber": formatNumber,
"currency":     formatCurrency,

// Time helpers
"formatDate":   formatDate,
"formatTime":   formatTime,
"timeAgo":      timeAgo,

// Array helpers
"has":          contains,
"join":         strings.Join,
"first":        first,
"last":         last,

// HTML helpers
"safeHTML":     func(s string) template.HTML { return template.HTML(s) },
"safeJS":       func(s string) template.JS { return template.JS(s) },
}

func truncate(s string, length int) string {
    if len(s) <= length {
        return s
    }
    return s[:length] + "..."
}

func formatNumber(n int) string {
    return strconv.FormatInt(int64(n), 10)
}

func formatCurrency(amount float64) string {
    return fmt.Sprintf("%.2f", amount)
}

func formatDate(t time.Time) string {
    return t.Format("2006-01-02")
}

func formatTime(t time.Time) string {
    return t.Format("15:04:05")
}

func timeAgo(t time.Time) string {
    duration := time.Since(t)

```



```

switch {
case duration < time.Minute:
    return "just now"
case duration < time.Hour:
    return fmt.Sprintf("%d minutes ago", int(duration.Minutes()))
case duration < 24*time.Hour:
    return fmt.Sprintf("%d hours ago", int(duration.Hours()))
default:
    return fmt.Sprintf("%d days ago", int(duration.Hours()/24))
}
}

func contains(slice []string, item string) bool {
    for _, s := range slice {
        if s == item {
            return true
        }
    }
    return false
}

func first(slice []interface{}) interface{} {
    if len(slice) == 0 {
        return nil
    }
    return slice[0]
}

func last(slice []interface{}) interface{} {
    if len(slice) == 0 {
        return nil
    }
    return slice[len(slice)-1]
}

// Setup template with functions
func setupTemplates() *template.Template {
    return template.Must(template.New("").
        Funcs(templateFuncs).
        ParseGlob("templates/*.html"))
}

```

บทที่ 33: การจัดการค่า Configuration

เป้าหมาย: จัดการ configuration หลายแหล่ง (file, env, flags)

Using Viper

```

package config

import (
    "fmt"
    "strings"
    "time"

    "github.com/spf13/viper"
)

type Config struct {
    Server      ServerConfig `mapstructure:"server"`
    Database    DatabaseConfig `mapstructure:"database"`
    Redis       RedisConfig `mapstructure:"redis"`
    JWT         JWTConfig `mapstructure:"jwt"`
    Log         LogConfig `mapstructure:"log"`
    SMTP        SMTPConfig `mapstructure:"smtp"`
}

type ServerConfig struct {
    Port          int `mapstructure:"port"`
    Mode          string `mapstructure:"mode"`
    ReadTimeout   time.Duration `mapstructure:"read_timeout"`
    WriteTimeout  time.Duration `mapstructure:"write_timeout"`
}

type DatabaseConfig struct {
    Host          string `mapstructure:"host"`
    Port          int `mapstructure:"port"`
    User          string `mapstructure:"user"`
    Password      string `mapstructure:"password"`
    Name          string `mapstructure:"name"`
    SSLMode       string `mapstructure:"ssl_mode"`
    MaxOpenConns  int `mapstructure:"max_open_conns"`
    MaxIdleConns  int `mapstructure:"max_idle_conns"`
    ConnMaxLifetime time.Duration `mapstructure:"conn_max_lifetime"`
}

type RedisConfig struct {
    Addr          string `mapstructure:"addr"`
    Password      string `mapstructure:"password"`
    DB            int `mapstructure:"db"`
    PoolSize      int `mapstructure:"pool_size"`
}

type JWTConfig struct {
    Secret          string `mapstructure:"secret"`
    AccessExpiry    time.Duration `mapstructure:"access_expiry"`
    RefreshExpiry   time.Duration `mapstructure:"refresh_expiry"`
}

type LogConfig struct {

```

```

    Level string `mapstructure:"level"`
    Format string `mapstructure:"format"`
    Output string `mapstructure:"output"`
}

type SMTPConfig struct {
    Host      string `mapstructure:"host"`
    Port      int    `mapstructure:"port"`
    Username  string `mapstructure:"username"`
    Password  string `mapstructure:"password"`
    From      string `mapstructure:"from"`
}

func LoadConfig(configPath string) (*Config, error) {
    // Set defaults
    viper.SetDefault("server.port", 8080)
    viper.SetDefault("server.mode", "debug")
    viper.SetDefault("server.read_timeout", "15s")
    viper.SetDefault("server.write_timeout", "15s")
    viper.SetDefault("log.level", "info")
    viper.SetDefault("log.format", "json")
    viper.SetDefault("log.output", "stdout")

    // Config file
    viper.SetConfigName("config")
    viper.SetConfigType("yaml")
    viper.AddConfigPath(configPath)
    viper.AddConfigPath(".")
    viper.AddConfigPath("/etc/app/")

    // Environment variables
    viper.SetEnvKeyReplacer(strings.NewReplacer(".", "_"))

    // Read config file
    if err := viper.ReadInConfig(); err != nil {
        if _, ok := err.(viper.ConfigFileNotFoundError); !ok {
            return nil, fmt.Errorf("read config: %w", err)
        }
    }

    // Unmarshal
    var cfg Config
    if err := viper.Unmarshal(&cfg); err != nil {
        return nil, fmt.Errorf("unmarshal config: %w", err)
    }

    return &cfg, nil
}

```

Environment Variables

```

package config

import (
    "os"
    "strconv"
    "time"
)

type Config struct {
    // Server
    ServerPort int
    ServerMode string

    // Database
    DBHost    string
    DBPort    int
    DBUser    string
    DBPassword string
    DBName    string

    // Redis
    RedisAddr    string
    RedisPassword string
    RedisDB      int

    // JWT
    JWTSecret    string
    JWTAccessExpiry time.Duration
    JWTRefreshExpiry time.Duration
}

func LoadFromEnv() (*Config, error) {
    cfg := &Config{
        // Server defaults
        ServerPort: 8080,
        ServerMode: "debug",

        // Database defaults
        DBHost: "localhost",
        DBPort: 5432,

        // Redis defaults
        RedisAddr: "localhost:6379",
        RedisDB: 0,
    }

    // Server
    if port := os.Getenv("SERVER_PORT"); port != "" {
        if p, err := strconv.Atoi(port); err == nil {
            cfg.ServerPort = p
        }
    }

```

```

}

if mode := os.Getenv("SERVER_MODE"); mode != "" {
    cfg.ServerMode = mode
}

// Database
if host := os.Getenv("DB_HOST"); host != "" {
    cfg.DBHost = host
}

if port := os.Getenv("DB_PORT"); port != "" {
    if p, err := strconv.Atoi(port); err == nil {
        cfg.DBPort = p
    }
}

cfg.DBUser = os.Getenv("DB_USER")
cfg.DBPassword = os.Getenv("DB_PASSWORD")
cfg.DBName = os.Getenv("DB_NAME")

// Redis
if addr := os.Getenv("REDIS_ADDR"); addr != "" {
    cfg.RedisAddr = addr
}

cfg.RedisPassword = os.Getenv("REDIS_PASSWORD")

if db := os.Getenv("REDIS_DB"); db != "" {
    if d, err := strconv.Atoi(db); err == nil {
        cfg.RedisDB = d
    }
}

// JWT
cfg.JWTSecret = os.Getenv("JWT_SECRET")

if expiry := os.Getenv("JWT_ACCESS_EXPIRY"); expiry != "" {
    if d, err := time.ParseDuration(expiry); err == nil {
        cfg.JWTAccessExpiry = d
    }
} else {
    cfg.JWTAccessExpiry = 15 * time.Minute
}

if expiry := os.Getenv("JWT_REFRESH_EXPIRY"); expiry != "" {
    if d, err := time.ParseDuration(expiry); err == nil {
        cfg.JWTRefreshExpiry = d
    }
} else {
    cfg.JWTRefreshExpiry = 7 * 24 * time.Hour
}

```

```

// Validation
if cfg.JWTSecret == "" {
    return nil, fmt.Errorf("JWT_SECRET is required")
}

return cfg, nil
}

```

Command Line Flags

```

package main

import (
    "flag"
    "fmt"
    "time"
)

type Config struct {
    // Server
    ServerPort int
    ServerMode string

    // Database
    DBHost    string
    DBPort    int
    DBUser    string
    DBPassword string
    DBName    string

    // Redis
    RedisAddr    string
    RedisPassword string
    RedisDB      int

    // JWT
    JWTSecret      string
    JWTAccessExpiry time.Duration
    JWTRefreshExpiry time.Duration
}

func LoadFromFlags() *Config {
    cfg := &Config{}

    // Server flags
    flag.IntVar(&cfg.ServerPort, "port", 8080, "Server port")
    flag.StringVar(&cfg.ServerMode, "mode", "debug", "Server mode (debug/release)")

    // Database flags

```

```

flag.StringVar(&cfg.DBHost, "db-host", "localhost", "Database host")
flag.IntVar(&cfg.DBPort, "db-port", 5432, "Database port")
flag.StringVar(&cfg.DBUser, "db-user", "", "Database user")
flag.StringVar(&cfg.DBPassword, "db-password", "", "Database password")
flag.StringVar(&cfg.DBName, "db-name", "", "Database name")

// Redis flags
flag.StringVar(&cfg.RedisAddr, "redis-addr", "localhost:6379", "Redis
address")
flag.StringVar(&cfg.RedisPassword, "redis-password", "", "Redis password")
flag.IntVar(&cfg.RedisDB, "redis-db", 0, "Redis database")

// JWT flags
flag.StringVar(&cfg.JWTSecret, "jwt-secret", "", "JWT secret key")
flag.DurationVar(&cfg.JWTAccessExpiry, "jwt-access-expiry", 15*time.Minute,
"JWT access token expiry")
flag.DurationVar(&cfg.JWTRefreshExpiry, "jwt-refresh-expiry",
7*24*time.Hour, "JWT refresh token expiry")

flag.Parse()

return cfg
}

```

Complete Configuration Example

```

package config

import (
    "fmt"
    "strings"

    "github.com/spf13/viper"
)

// config.yaml
/*
server:
  port: 8080
  mode: release
  read_timeout: 15s
  write_timeout: 15s

database:
  host: localhost
  port: 5432
  user: postgres
  password: ${DB_PASSWORD}
  name: myapp
  ssl_mode: disable

```

```

max_open_conns: 25
max_idle_conns: 25
conn_max_lifetime: 5m

redis:
  addr: localhost:6379
  password: ${REDIS_PASSWORD}
  db: 0
  pool_size: 10

jwt:
  secret: ${JWT_SECRET}
  access_expiry: 15m
  refresh_expiry: 168h

log:
  level: info
  format: json
  output: stdout

smtp:
  host: smtp.gmail.com
  port: 587
  username: ${SMTP_USERNAME}
  password: ${SMTP_PASSWORD}
  from: noreply@myapp.com
*/

type Config struct {
    Server      ServerConfig `mapstructure:"server"`
    Database    DatabaseConfig `mapstructure:"database"`
    Redis       RedisConfig   `mapstructure:"redis"`
    JWT         JWTConfig     `mapstructure:"jwt"`
    Log         LogConfig     `mapstructure:"log"`
    SMTP        SMTPConfig    `mapstructure:"smtp"`
}

func InitConfig() (*Config, error) {
    // Set config file
    viper.SetConfigName("config")
    viper.SetConfigType("yaml")
    viper.AddConfigPath(".")
    viper.AddConfigPath("./config")
    viper.AddConfigPath("/etc/app")

    // Environment variables
    viper.SetEnvKeyReplacer(strings.NewReplacer(".", "_"))

    // Read config file
    if err := viper.ReadInConfig(); err != nil {
        return nil, fmt.Errorf("failed to read config: %w", err)
    }
}

```



```

// Expand environment variables in config
for _, key := range viper.AllKeys() {
    val := viper.GetString(key)
    if strings.Contains(val, "${") {
        expanded := os.ExpandEnv(val)
        viper.Set(key, expanded)
    }
}

var cfg Config
if err := viper.Unmarshal(&cfg); err != nil {
    return nil, fmt.Errorf("failed to unmarshal config: %w", err)
}

// Validate required fields
if cfg.JWT.Secret == "" {
    return nil, fmt.Errorf("JWT secret is required")
}

return &cfg, nil
}

```

สรุปภาคที่ 4: ตัวอย่างแอปพลิเคชันครบวงจร

```

// main.go - Complete application example
package main

import (
    "context"
    "log/slog"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
    "github.com/go-chi/cors"

    "myapp/config"
    "myapp/handlers"
    "myapp/logger"
    "myapp/repository"
    "myapp/service"
)

func main() {

```

```

// 1. Load configuration
cfg, err := config.InitConfig()
if err != nil {
    panic(err)
}

// 2. Setup logger
log, err := logger.NewLogger(cfg.Log)
if err != nil {
    panic(err)
}
slog.SetDefault(log)

slog.Info("Application starting", "config", cfg)

// 3. Connect to database
db, err := repository.NewPostgresDB(cfg.Database)
if err != nil {
    slog.Error("Failed to connect to database", "error", err)
    os.Exit(1)
}
defer db.Close()

// 4. Connect to Redis
redisClient, err := repository.NewRedisClient(cfg.Redis)
if err != nil {
    slog.Error("Failed to connect to Redis", "error", err)
    os.Exit(1)
}
defer redisClient.Close()

// 5. Initialize repositories
userRepo := repository.NewUserRepository(db)
sessionRepo := repository.NewSessionRepository(redisClient)

// 6. Initialize services
authService := service.NewAuthService(userRepo, sessionRepo, cfg.JWT)
userService := service.NewUserService(userRepo)

// 7. Initialize handlers
authHandler := handlers.NewAuthHandler(authService)
userHandler := handlers.NewUserHandler(userService)
healthHandler := handlers.NewHealthHandler(db, redisClient)

// 8. Setup router
r := chi.NewRouter()

// Global middlewares
r.Use(middleware.RequestID)
r.Use(middleware.RealIP)
r.Use(logger.HTTPMiddleware)
r.Use(middleware.Recoverer)
r.Use(middleware.Timeout(60 * time.Second))

```

```

r.Use(cors.Handler(cors.Options{
    AllowedOrigins:  []string{"*"},
    AllowedMethods:  []string{"GET", "POST", "PUT", "DELETE", "OPTIONS"},
    AllowedHeaders:  []string{"Accept", "Authorization", "Content-Type"},
    ExposedHeaders:  []string{"Link"},
    AllowCredentials: false,
    MaxAge:          300,
}))

// Health check
r.Get("/health", healthHandler.Health)
r.Get("/ready", healthHandler.Ready)
r.Get("/live", healthHandler.Live)

// Public routes
r.Post("/api/v1/auth/register", authHandler.Register)
r.Post("/api/v1/auth/login", authHandler.Login)
r.Post("/api/v1/auth/refresh", authHandler.Refresh)

// Protected routes
r.Group(func(r chi.Router) {
    r.Use(authHandler.AuthMiddleware)

    r.Get("/api/v1/users/me", userHandler.GetMe)
    r.Put("/api/v1/users/me", userHandler.UpdateMe)
    r.Post("/api/v1/auth/logout", authHandler.Logout)
})

// 9. Create server
server := &http.Server{
    Addr:      fmt.Sprintf(":%d", cfg.Server.Port),
    Handler:    r,
    ReadTimeout:  cfg.Server.ReadTimeout,
    WriteTimeout: cfg.Server.WriteTimeout,
    IdleTimeout:  120 * time.Second,
}

// 10. Start server
go func() {
    slog.Info("Server starting", "port", cfg.Server.Port)
    if err := server.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
        slog.Error("Server failed", "error", err)
        os.Exit(1)
    }
}()

// 11. Graceful shutdown
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

slog.Info("Shutting down server...")

```

```

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := server.Shutdown(ctx); err != nil {
    slog.Error("Server forced to shutdown", "error", err)
}

slog.Info("Server exited")
}

```

สิ่งที่ได้เรียนรู้ในภาคที่ 4:

1. การรับข้อมูล (**Input**): HTTP server, JSON/XML, Enum/Bitmask, Time handling
2. การประมวลผล (**Processing**): Anonymous functions, Closures, Concurrency patterns
3. การจัดเก็บ (**Storage**): Files, Database, Structured logging
4. การส่งออก (**Output**): HTML/Template, Configuration management

พื้นฐานเหล่านี้จะนำไปใช้ใน **ภาคที่ 5** เพื่อพัฒนาสู่การเป็น Go Developer มืออาชีพ ด้วยการวัดประสิทธิภาพ, profiling, context management, และ generics

บทที่ 24: ฟังก์ชันนิรนาม (Anonymous functions) และ Closure

24.1 ฟังก์ชันนิรนาม

ฟังก์ชันที่ไม่มีชื่อ สามารถกำหนดให้กับตัวแปรหรือส่งเป็น argument

```

func() {
    fmt.Println("anonymous")
}()

add := func(a, b int) int {
    return a + b
}
result := add(2,3)

```

24.2 Closure

Closure คือฟังก์ชันที่สามารถเข้าถึงตัวแปรที่อยู่นอกขอบเขตของมัน

```

func counter() func() int {
    count := 0
    return func() int {
        count++
        return count
    }
}

```

```
c := counter()
fmt.Println(c()) // 1
fmt.Println(c()) // 2
```

24.3 การใช้งานจริง

- สร้างฟังก์ชัน factory
- ใช้กับ callback (เช่น http.HandlerFunc)
- ใช้กับ goroutine (passing variables)

24.4 ข้อควรระวังเกี่ยวกับ closure ใน loop

```
func main() {
    funcs := []func(){}
    for i := 0; i < 3; i++ {
        funcs = append(funcs, func() {
            fmt.Println(i) // i เป็นตัวแปรเดียวกัน
        })
    }
    for _, f := range funcs {
        f() // output: 3 3 3
    }
}
```

วิธีแก้: ใช้ตัวแปร local copy

```
for i := 0; i < 3; i++ {
    i := i // shadow
    funcs = append(funcs, func() {
        fmt.Println(i)
    })
}
```

บทที่ 25: การจัดการข้อมูล JSON และ XML

25.1 JSON encoding

ใช้ struct tags เพื่อกำหนดชื่อ field ใน JSON

```
type Person struct {
    Name string `json:"name"`
    Age  int    `json:"age,omitempty"`
}
```

```
Email string `json:"-"` // ไม่รวมใน JSON
}
```

25.2 Marshal (Go -> JSON)

```
p := Person{Name: "John", Age: 30}
data, err := json.Marshal(p)
if err != nil {
    // handle
}
fmt.Println(string(data)) // {"name":"John","age":30}
```

ใช้ `MarshalIndent` เพื่อจัดรูปแบบ

25.3 Unmarshal (JSON -> Go)

```
jsonStr := `{"name":"John","age":30}`
var p Person
err := json.Unmarshal([]byte(jsonStr), &p)
```

25.4 การทำงานกับ dynamic JSON

ใช้ `map[string]interface{}` หรือ `interface{}`:

```
var data map[string]interface{}
json.Unmarshal([]byte(jsonStr), &data)
name := data["name"].(string)
```

25.5 การใช้ json.RawMessage

เพื่อการ decode แบบ lazy

25.6 XML

คล้ายกับ JSON แต่ใช้แท็ก `xml`:

```
type Person struct {
    XMLName xml.Name `xml:"person"`
    Name    string  `xml:"name"`
    Age     int     `xml:"age"`
}
data, _ := xml.Marshal(p)
```

บทที่ 26: พื้นฐานการสร้าง HTTP Server

26.1 HTTP Handler

`http.Handler` interface มี method `ServeHTTP(http.ResponseWriter, *http.Request)`

วิธีง่าย: ใช้ `http.HandleFunc`

```
func helloHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, %s!", r.URL.Path[1:])
}

func main() {
    http.HandleFunc("/", helloHandler)
    http.ListenAndServe(":8080", nil)
}
```

26.2 การใช้ ServeMux

```
mux := http.NewServeMux()
mux.HandleFunc("/hello", helloHandler)
mux.HandleFunc("/bye", byeHandler)
http.ListenAndServe(":8080", mux)
```

26.3 การอ่าน request data

- Query parameters: `r.URL.Query().Get("name")`
- Form data (POST): `r.ParseForm(); r.FormValue("name")`
- JSON body: `json.NewDecoder(r.Body).Decode(&data)`

26.4 การตั้งค่า header และ status

```
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusCreated)
```

26.5 Routing ขั้นสูง

ใช้ third-party เช่น `gorilla/mux` หรือ `chi`:

```
r := mux.NewRouter()
r.HandleFunc("/users/{id}", getUser).Methods("GET")
```

26.6 Middleware

ฟังก์ชันที่ห่อ handler เพื่อเพิ่ม logic (logging, auth)

```
func loggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        log.Println(r.Method, r.URL.Path)
        next.ServeHTTP(w, r)
    })
}

// ใช้
mux.Use(loggingMiddleware)
```

26.7 การสร้าง HTTP server แบบ custom

```
srv := &http.Server{
    Addr:           ":8080",
    Handler:        mux,
    ReadTimeout:   10 * time.Second,
    WriteTimeout:  10 * time.Second,
}
srv.ListenAndServe()
```

บทที่ 27: Enum, Iota และ Bitmask

27.1 Enum ใน Go

Go ไม่มี enum ในตัว แต่ใช้ const และ iota

```
type Weekday int

const (
    Sunday Weekday = iota
    Monday
    Tuesday
    Wednesday
    Thursday
    Friday
    Saturday
)
```

iota เริ่มที่ 0 และเพิ่มทีละ 1

27.2 การกำหนดค่าเริ่มต้นไม่ใช่ 0


```
const (
    _ = iota // ignore first
    KB = 1 << (10 * iota) // 1 << (10*1) = 1024
    MB // 1 << (10*2) = 1048576
    GB
)
```

27.3 Bitmask

ใช้ bitwise operator เพื่อรวม flags

```
type Permission uint8

const (
    Read Permission = 1 << iota // 1
    Write                // 2
    Execute              // 4
)

// รวม
perm := Read | Write // 3

// ตรวจสอบ
if perm&Read != 0 {
    fmt.Println("has read")
}
```

27.4 การสร้าง string representation

```
func (p Permission) String() string {
    var flags []string
    if p&Read != 0 {
        flags = append(flags, "Read")
    }
    // ...
    return strings.Join(flags, "|")
}
```

บทที่ 28: วันที่และเวลา

28.1 time.Time

```
t := time.Now()
fmt.Println(t) // 2024-01-15 10:30:00 +0000 UTC

// สร้างเวลาที่กำหนด
t2 := time.Date(2024, time.January, 15, 10, 30, 0, 0, time.UTC)
```

28.2 การจัดรูปแบบ

Go ใช้รูปแบบอ้างอิง: `Mon Jan 2 15:04:05 MST 2006`

```
t := time.Now()
fmt.Println(t.Format("2006-01-02 15:04:05"))
fmt.Println(t.Format("02/01/2006"))
```

28.3 การ parse string เป็น time

```
layout := "2006-01-02 15:04:05"
t, err := time.Parse(layout, "2024-01-15 14:30:00")
```

28.4 การคำนวณเวลา

```
t := time.Now()
t2 := t.Add(24 * time.Hour)
diff := t2.Sub(t) // 24h0m0s
```

28.5 time.Duration

```
d, _ := time.ParseDuration("1h30m")
time.Sleep(d)
```

28.6 Timer และ Ticker

- `time.After(d)` : คืน channel ที่จะรับค่าเมื่อครบเวลา
- `time.Ticker` : ส่งค่าทุกๆ ช่วงเวลา

```
ticker := time.NewTicker(1 * time.Second)
go func() {
    for range ticker.C {
        fmt.Println("tick")
    }
}
```

```
}()  
time.Sleep(5 * time.Second)  
ticker.Stop()
```

28.7 Timezone

ใช้ `time.LoadLocation("Asia/Bangkok")` เพื่อเปลี่ยน location

บทที่ 29: การจัดเก็บข้อมูล: ไฟล์และฐานข้อมูล

29.1 การอ่านไฟล์

```
data, err := os.ReadFile("file.txt")  
if err != nil {  
    log.Fatal(err)  
}  
fmt.Println(string(data))
```

29.2 การเขียนไฟล์

```
err := os.WriteFile("output.txt", []byte("hello"), 0644)
```

29.3 การใช้ bufio สำหรับอ่านทีละบรรทัด

```
file, err := os.Open("file.txt")  
if err != nil { ... }  
defer file.Close()  
  
scanner := bufio.NewScanner(file)  
for scanner.Scan() {  
    line := scanner.Text()  
    fmt.Println(line)  
}
```

29.4 การใช้ database/sql

```
import (  
    "database/sql"  
    _ "github.com/go-sql-driver/mysql"  
)  
  
db, err := sql.Open("mysql", "user:pass@tcp(127.0.0.1:3306)/dbname")
```

```
defer db.Close()

rows, err := db.Query("SELECT id, name FROM users")
for rows.Next() {
    var id int
    var name string
    rows.Scan(&id, &name)
}
```

29.5 การใช้ ORM (GORM)

```
import "gorm.io/gorm"

type User struct {
    gorm.Model
    Name string
    Age  int
}

db.Create(&User{Name: "John", Age: 30})
var user User
db.First(&user, 1)
```

29.6 การจัดการ connection pool

```
db.SetMaxOpenConns(25)
db.SetMaxIdleConns(25)
db.SetConnMaxLifetime(5 * time.Minute)
```

บทที่ 30: การทำงานพร้อมกัน (Concurrency)

30.1 Goroutine

Goroutine คือ lightweight thread จัดการโดย Go runtime

```
go func() {
    fmt.Println("hello from goroutine")
}()
```

30.2 Channel

Channel ใช้สื่อสารระหว่าง goroutine

```
ch := make(chan int)
go func() {
    ch <- 42 // ส่งค่า
}()
value := <-ch // รับค่า
```

30.3 Buffered channel

```
ch := make(chan int, 2) // buffer size 2
ch <- 1
ch <- 2
```

30.4 การใช้ select

รอหลาย channel

```
select {
case msg1 := <-ch1:
    fmt.Println(msg1)
case msg2 := <-ch2:
    fmt.Println(msg2)
case <-time.After(1 * time.Second):
    fmt.Println("timeout")
default:
    fmt.Println("no message")
}
```

30.5 Worker pool pattern

```
func worker(id int, jobs <-chan int, results chan<- int) {
    for j := range jobs {
        results <- j * 2
    }
}

jobs := make(chan int, 100)
results := make(chan int, 100)

for w := 1; w <= 3; w++ {
    go worker(w, jobs, results)
}

for j := 1; j <= 5; j++ {
    jobs <- j
}
```

```
close(jobs)

for r := 1; r <= 5; r++ {
    <-results
}
```

30.6 sync.WaitGroup

รอให้ goroutine ทั้งหมดทำงานเสร็จ

```
var wg sync.WaitGroup
for i := 0; i < 5; i++ {
    wg.Add(1)
    go func(i int) {
        defer wg.Done()
        fmt.Println(i)
    }(i)
}
wg.Wait()
```

30.7 Mutex

ป้องกัน race condition

```
var mu sync.Mutex
var counter int

mu.Lock()
counter++
mu.Unlock()
```

30.8 การตรวจจับ race condition

```
go run -race main.go
go test -race
```

30.9 Atomic operations

```
import "sync/atomic"
var counter int64
atomic.AddInt64(&counter, 1)
```

บทที่ 31: การบันทึกเหตุการณ์ (Logging)

31.1 log package พื้นฐาน

```
log.Println("Info message")
log.Printf("User %s logged in", username)
log.Fatal("fatal error") // log แล้ว os.Exit(1)
```

31.2 การตั้งค่า log flags

```
log.SetFlags(log.LstdFlags | log.Lshortfile)
// 2024/01/15 10:30:00 main.go:42: message
```

31.3 การสร้าง logger แยก

```
file, _ := os.OpenFile("app.log", os.O_CREATE|os.O_WRONLY|os.O_APPEND, 0666)
logger := log.New(file, "INFO: ", log.Ldate|log.Ltime)
logger.Println("application started")
```

31.4 การใช้ structured logging (log/slog) Go 1.21+

```
import "log/slog"

slog.Info("user login", "user", "john", "ip", "127.0.0.1")
slog.Error("database error", "error", err)
```

31.5 ระดับ log

สามารถตั้งระดับด้วย slog.SetLogLevel

31.6 การใช้ logging middleware ใน HTTP

```
func loggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        slog.Info("request",
            "method", r.Method,
            "path", r.URL.Path,
            "duration", time.Since(start),
        )
    })
}
```

```
    })  
}
```

บทที่ 32: เทมเพลต (Templates)

32.1 text/template และ html/template

Go มีแพ็คเกจสองตัว: `text/template` สำหรับข้อความทั่วไป, `html/template` สำหรับ HTML (มีการ escape อัตโนมัติ)

32.2 การใช้งานพื้นฐาน

```
tmpl, err := template.New("test").Parse("Hello {{.Name}}")  
data := struct{ Name string }{"John"}  
tmpl.Execute(os.Stdout, data)
```

32.3 การทำงานกับ struct และ map

```
type Person struct {  
    Name string  
    Age  int  
}  
data := Person{"John", 30}  
tmpl.Execute(w, data)
```

ใน template: `{{.Name}}` , `{{.Age}}`

32.4 คำสั่งใน template

- `{{if .Condition}} ... {{else}} ... {{end}}`
- `{{range .Items}} ... {{end}}`
- `{{with .Field}} ... {{end}}` เปลี่ยน context

32.5 ฟังก์ชันใน template

```
funcMap := template.FuncMap{  
    "toUpper": strings.ToUpper,  
}  
tmpl := template.New("test").Funcs(funcMap)  
tmpl.Parse(`{{. | toUpper}}`)
```

32.6 การใช้ template files


```
tpl := template.Must(template.ParseFiles("index.html"))
tpl.Execute(w, data)
```

32.7 การรวม template (partials)

```
tpl := template.Must(template.ParseGlob("templates/*.html"))
```

32.8 ความปลอดภัยของ html/template

html/template จะ escape อัดโนมัตเพื่อป้องกัน XSS ใช้ `{{. | safeHTML}}` เฉพาะเมื่อมั่นใจ

บทที่ 33: การจัดการค่า Configuration

33.1 การใช้ environment variables

```
import "os"

dbHost := os.Getenv("DB_HOST")
if dbHost == "" {
    dbHost = "localhost"
}
```

33.2 การใช้ flag package

```
import "flag"

var port = flag.Int("port", 8080, "server port")
flag.Parse()
fmt.Println("port:", *port)
```

33.3 การใช้ไฟล์ configuration (JSON/YAML)

```
type Config struct {
    Server struct {
        Port int `json:"port"`
    } `json:"server"`
}

data, _ := os.ReadFile("config.json")
var cfg Config
json.Unmarshal(data, &cfg)
```

33.4 การใช้ viper (popular library)

```
import "github.com/spf13/viper"

viper.SetConfigName("config")
viper.SetConfigType("yaml")
viper.AddConfigPath(".")
viper.ReadInConfig()

port := viper.GetInt("server.port")
```

33.5 การใช้ struct tags และการ validate

```
type Config struct {
    Port int `mapstructure:"port" validate:"required,min=1024,max=65535"`
}
```

33.6 การโอเวอร์ไรด์ค่า order (override)

ลำดับความสำคัญ: default -> file -> env -> flag

ภาคที่ 5: สู่การเป็นนักพัฒนา Go มืออาชีพ

(บทที่ 34–42)

แผนภาพแสดงความสัมพันธ์ของเนื้อหา

```
flowchart TB
    subgraph Performance["1. การวัดและปรับปรุงประสิทธิภาพ (Performance Optimization)"]
        direction TB
        A1["Benchmark (บทที่ 34)<br/>go test -bench<br/>benchmem, benchtime<br/>benchstat, comparing results"]
        A2["Profiling (บทที่ 36)<br/>pprof (CPU, Memory, Goroutine, Block, Mutex)<br/>go tool pprof, flame graphs"]
        A3["HTTP Client (บทที่ 35)<br/>connection pooling<br/>timeout, retry, transport"]
        A4["Context (บทที่ 37)<br/>deadline, cancellation<br/>request-scoped values, propagation"]
        A5["Generics (บทที่ 38)<br/>type parameters, constraints<br/>generic functions and types"]
        A1 --> A2
        A2 --> A3
    end
```

```

    A3 --> A4
    A4 --> A5
end

subgraph CodeDesign["2. การออกแบบโค้ด (Code Design)"]
    direction TB
    B1["OOP in Go (บทที่ 39)<br/>struct + methods vs classes<br/>composition over inheritance<br/>embedding, interfaces"]
    B2["Version Management (บทที่ 40)<br/>upgrading/downgrading Go<br/>go mod edit, go fix, go tool"]
    B3["Best Practices (บทที่ 41)<br/>idioms, naming conventions<br/>error handling patterns<br/>project structure, testing"]
end

subgraph Reference["3. เอกสารอ้างอิง (Reference)"]
    C1["Cheatsheet (บทที่ 42)<br/>commands, syntax, patterns<br/>quick reference, code snippets"]
end

Performance --> CodeDesign
CodeDesign --> Reference

subgraph Tools["เครื่องมือที่ใช้"]
    D1["go test -bench<br/>go test -benchmem"]
    D2["pprof<br/>go tool pprof"]
    D3["benchstat<br/>benchcmp"]
    D4["go fix<br/>gofmt, go vet"]
    D5["golangci-lint<br/>staticcheck"]
end

Performance --> Tools
CodeDesign --> Tools

```

คำอธิบายภาษาไทย (แบบละเอียด)

ภาคที่ 5 ครอบคลุมบทที่ 34-42 โดยมีเนื้อหาแบ่งเป็น 3 ช่วงหลัก ตามแผนภาพด้านบน ซึ่งจะเปลี่ยนผู้พัฒนาจากผู้ใช้ภาษาให้เป็นผู้ที่เข้าใจกลไกภายใน สามารถวิเคราะห์และปรับปรุงระบบให้มีประสิทธิภาพสูง พร้อมเขียนโค้ดที่บำรุงรักษาง่ายและยืดหยุ่น

1. การวัดและปรับปรุงประสิทธิภาพ (Performance Optimization)

บทที่ 34: การวัดประสิทธิภาพ (Benchmarks)

เป้าหมาย: เขียนและวิเคราะห์ benchmark เพื่อวัดประสิทธิภาพของโค้ด

พื้นฐานการเขียน Benchmark

```
// math/benchmark_test.go
package math
```

```

import (
    "testing"
)

// 1. Basic benchmark
func BenchmarkAdd(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Add(10, 20)
    }
}

// 2. Benchmark with different inputs
func BenchmarkAddLargeNumbers(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Add(1000000, 2000000)
    }
}

// 3. Benchmark with allocation tracking
func BenchmarkStringConcat(b *testing.B) {
    b.ReportAllocs() // report memory allocations

    for i := 0; i < b.N; i++ {
        s := ""
        for j := 0; j < 100; j++ {
            s += "a"
        }
    }
}

// 4. Benchmark with reset timer
func BenchmarkExpensiveSetup(b *testing.B) {
    // Setup that should not be measured
    data := make([]int, 1000000)
    for i := range data {
        data[i] = i
    }

    b.ResetTimer() // Reset benchmark timer

    for i := 0; i < b.N; i++ {
        processData(data)
    }
}

// 5. Sub-benchmarks
func BenchmarkSort(b *testing.B) {
    sizes := []int{100, 1000, 10000, 100000}

    for _, size := range sizes {
        b.Run(fmt.Sprintf("Size-%d", size), func(b *testing.B) {
            data := generateRandomSlice(size)

```

```

        b.ResetTimer()
        for i := 0; i < b.N; i++ {
            sort.Ints(data)
        }
    })
}

// 6. Parallel benchmark
func BenchmarkParallel(b *testing.B) {
    b.RunParallel(func(pb *testing.PB) {
        for pb.Next() {
            // Code to benchmark in parallel
            expensiveOperation()
        }
    })
}

```

การรัน Benchmark

```

# รัน benchmark ทั้งหมด
go test -bench=.

# รันเฉพาะ benchmark ที่ match pattern
go test -bench=Add

# ระบุจำนวนรอบ (default 1)
go test -bench=. -benchtime=10s

# วัด memory allocation
go test -bench=. -benchmem

# รัน benchmark และแสดง CPU profile
go test -bench=. -cpuprofile=cpu.prof

# รัน benchmark และแสดง memory profile
go test -bench=. -memprofile=mem.prof

# รัน benchmark และแสดง block profile
go test -bench=. -blockprofile=block.prof

# เปรียบเทียบผลลัพธ์ระหว่างสองเวอร์ชัน
go test -bench=. -count=5 > old.txt
# ... modify code ...
go test -bench=. -count=5 > new.txt
benchstat old.txt new.txt

```

ตัวอย่างการใช้งานจริง

1. Benchmarking String Concatenation vs StringBuilder

```
package benchmark

import (
    "strings"
    "testing"
)

// Bad: String concatenation in loop
func ConcatWithPlus(items []string) string {
    result := ""
    for _, item := range items {
        result += item
    }
    return result
}

// Good: Using strings.Builder
func ConcatWithBuilder(items []string) string {
    var builder strings.Builder
    builder.Grow(1024) // Pre-allocate capacity

    for _, item := range items {
        builder.WriteString(item)
    }
    return builder.String()
}

// Good: Using strings.Join
func ConcatWithJoin(items []string) string {
    return strings.Join(items, "")
}

func BenchmarkConcatWithPlus(b *testing.B) {
    items := make([]string, 100)
    for i := range items {
        items[i] = "a"
    }

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        ConcatWithPlus(items)
    }
}

func BenchmarkConcatWithBuilder(b *testing.B) {
    items := make([]string, 100)
    for i := range items {
        items[i] = "a"
    }
}
```

```

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        ConcatWithBuilder(items)
    }
}

func BenchmarkConcatWithJoin(b *testing.B) {
    items := make([]string, 100)
    for i := range items {
        items[i] = "a"
    }

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        ConcatWithJoin(items)
    }
}

```

ผลลัพธ์:

BenchmarkConcatWithPlus-8	10000	150000 ns/op	100000 B/op	99
allocs/op				
BenchmarkConcatWithBuilder-8	1000000	500 ns/op	1024 B/op	1
allocs/op				
BenchmarkConcatWithJoin-8	2000000	300 ns/op	1024 B/op	1
allocs/op				

2. Benchmarking Map Access Patterns

```

package benchmark

import (
    "testing"
)

type User struct {
    ID    int
    Name  string
    Age   int
}

// Using map[int]User (value)
func GetUserByIDValue(users map[int]User, id int) User {
    return users[id]
}

// Using map[int]*User (pointer)
func GetUserByIDPointer(users map[int]*User, id int) *User {

```

```

    return users[id]
}

func BenchmarkMapValue(b *testing.B) {
    users := make(map[int]User)
    for i := 0; i < 10000; i++ {
        users[i] = User{ID: i, Name: "User", Age: 30}
    }

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        _ = GetUserByIDValue(users, i%10000)
    }
}

func BenchmarkMapPointer(b *testing.B) {
    users := make(map[int]*User)
    for i := 0; i < 10000; i++ {
        users[i] = &User{ID: i, Name: "User", Age: 30}
    }

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        _ = GetUserByIDPointer(users, i%10000)
    }
}

```

3. Benchmarking JSON Serialization

```

package benchmark

import (
    "encoding/json"
    "testing"
)

type Person struct {
    Name    string `json:"name"`
    Age     int    `json:"age"`
    Email   string `json:"email"`
    Address string `json:"address"`
    Phone   string `json:"phone"`
    Tags    []string `json:"tags"`
}

func BenchmarkJSONMarshal(b *testing.B) {
    p := Person{
        Name:    "John Doe",
        Age:     30,
        Email:   "john@example.com",
        Address: "123 Main St, Bangkok",
    }
}

```



```

    Phone:    "081-234-5678",
    Tags:     []string{"golang", "developer", "backend"},
}

b.ResetTimer()
for i := 0; i < b.N; i++ {
    _, err := json.Marshal(p)
    if err != nil {
        b.Fatal(err)
    }
}

func BenchmarkJSONUnmarshal(b *testing.B) {
    data := []byte(`{"name":"John
Doe","age":30,"email":"john@example.com","address":"123 Main St,
Bangkok","phone":"081-234-5678","tags":["golang","developer","backend"]}`)

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        var p Person
        err := json.Unmarshal(data, &p)
        if err != nil {
            b.Fatal(err)
        }
    }
}

```

4. Benchmarking with benchstat

```

# สร้าง benchmark results
go test -bench=Concat -count=5 > concat_old.txt

# แก้ไขโค้ดให้ดีขึ้น

# สร้าง benchmark results ใหม่
go test -bench=Concat -count=5 > concat_new.txt

# เปรียบเทียบ
benchstat concat_old.txt concat_new.txt

```

ผลลัพธ์ benchstat:

name	old time/op	new time/op	delta
ConcatWithPlus-8	150µs ± 2%	150µs ± 1%	~ (p=0.841 n=5+5)
ConcatWithBuilder-8	500ns ± 1%	500ns ± 1%	~ (p=0.421 n=5+5)
ConcatWithJoin-8	300ns ± 1%	300ns ± 1%	~ (p=0.548 n=5+5)

name	old alloc/op	new alloc/op	delta
ConcatWithPlus-8	100kB ± 0%	100kB ± 0%	~ (all equal)
ConcatWithBuilder-8	1.02kB ± 0%	1.02kB ± 0%	~ (all equal)
ConcatWithJoin-8	1.02kB ± 0%	1.02kB ± 0%	~ (all equal)

name	old allocs/op	new allocs/op	delta
ConcatWithPlus-8	99.0 ± 0%	99.0 ± 0%	~ (all equal)
ConcatWithBuilder-8	1.00 ± 0%	1.00 ± 0%	~ (all equal)
ConcatWithJoin-8	1.00 ± 0%	1.00 ± 0%	~ (all equal)

บทที่ 36: การวิเคราะห์โปรไฟล์ (Program Profiling)

เป้าหมาย: ใช้ pprof เพื่อวิเคราะห์ CPU, Memory, Goroutine, และการบล็อก

การเปิดใช้งาน pprof

```
package main

import (
    "log"
    "net/http"
    _ "net/http/pprof" // Import for side effects
    "runtime"
)

func main() {
    // 1. Enable profiling endpoints
    go func() {
        log.Println(http.ListenAndServe("localhost:6060", nil))
    }()

    // 2. Start application
    // ... your application code
}
```

การเก็บ CPU Profile

```
package main

import (
    "os"
    "runtime/pprof"
    "time"
)

func main() {
```

```

// Create CPU profile file
f, err := os.Create("cpu.prof")
if err != nil {
    panic(err)
}
defer f.Close()

// Start CPU profiling
pprof.StartCPUProfile(f)
defer pprof.StopCPUProfile()

// Run code to profile
heavyComputation()
}

func heavyComputation() {
    // Simulate heavy computation
    for i := 0; i < 10000000; i++ {
        _ = i * i
    }
}

```

การเก็บ Memory Profile

```

package main

import (
    "os"
    "runtime/pprof"
)

func main() {
    // Create memory profile file
    f, err := os.Create("mem.prof")
    if err != nil {
        panic(err)
    }
    defer f.Close()

    // Run code to profile
    allocateMemory()

    // Write heap profile
    if err := pprof.WriteHeapProfile(f); err != nil {
        panic(err)
    }
}

func allocateMemory() {
    // Simulate memory allocation

```

```

data := make([]byte, 1024*1024*100) // 100 MB
for i := range data {
    data[i] = byte(i % 256)
}
_ = data
}

```

การวิเคราะห์ Profile ด้วย go tool pprof

```

# 1. วิเคราะห์ CPU profile
go tool pprof cpu.prof
# หรือ interactive mode
go tool pprof -http=:8080 cpu.prof

# 2. วิเคราะห์ Memory profile
go tool pprof mem.prof
go tool pprof -http=:8080 mem.prof

# 3. วิเคราะห์จาก running server
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30
go tool pprof http://localhost:6060/debug/pprof/heap

# 4. วิเคราะห์ goroutine
go tool pprof http://localhost:6060/debug/pprof/goroutine

# 5. วิเคราะห์ block
go tool pprof http://localhost:6060/debug/pprof/block

# 6. วิเคราะห์ mutex
go tool pprof http://localhost:6060/debug/pprof/mutex

```

คำสั่งใน pprof interactive mode

```

(pprof) top           # แสดง top functions
(pprof) top10         # แสดง top 10 functions
(pprof) list function # แสดง source code ของ function
(pprof) web           # แสดง flame graph (ต้องมี graphviz)
(pprof) peek          # แสดง call tree
(pprof) traces        # แสดง trace
(pprof) help          # แสดง help

```

ตัวอย่างการใช้งานจริง

1. CPU Profiling for API Handler

```

package main

import (
    "fmt"
    "log"
    "net/http"
    _ "net/http/pprof"
    "runtime"
    "time"
)

func main() {
    // Enable pprof endpoints
    go func() {
        log.Println(http.ListenAndServe("localhost:6060", nil))
    }()

    // Start application
    http.HandleFunc("/api/process", processHandler)
    http.HandleFunc("/api/compute", computeHandler)

    log.Fatal(http.ListenAndServe(":8080", nil))
}

func processHandler(w http.ResponseWriter, r *http.Request) {
    // CPU intensive operation
    result := 0
    for i := 0; i < 10000000; i++ {
        result += i * i
    }
    fmt.Fprintf(w, "Result: %d", result)
}

func computeHandler(w http.ResponseWriter, r *http.Request) {
    // Memory intensive operation
    data := make([]int, 1000000)
    for i := range data {
        data[i] = i
    }

    // Sort
    for i := 0; i < len(data); i++ {
        for j := i + 1; j < len(data); j++ {
            if data[i] > data[j] {
                data[i], data[j] = data[j], data[i]
            }
        }
    }

    fmt.Fprintf(w, "Computed %d items", len(data))
}

```

2. Generating Flame Graph

```
# เก็บ CPU profile 30 วินาที
go tool pprof -http=:8080 http://localhost:6060/debug/pprof/profile?seconds=30

# เก็บ memory profile และแสดง flame graph
go tool pprof -http=:8080 http://localhost:6060/debug/pprof/heap
```

3. Analyzing Goroutine Leaks

```
package main

import (
    "fmt"
    "net/http"
    _ "net/http/pprof"
    "time"
)

func main() {
    go func() {
        http.ListenAndServe("localhost:6060", nil)
    }()

    // Goroutine leak example
    for i := 0; i < 100; i++ {
        go leakyGoroutine()
    }

    time.Sleep(time.Hour)
}

func leakyGoroutine() {
    ch := make(chan int)
    go func() {
        // This goroutine never exits
        <-ch
    }()
    // ch never gets a value, goroutine leaks
}
```

ตรวจสอบ goroutine leak:

```
# ดู goroutine profile
go tool pprof http://localhost:6060/debug/pprof/goroutine

# ใน pprof interactive mode
```

```
(pprof) top
(pprof) list leakyGoroutine
```

4. Block Profile Analysis

```
package main

import (
    "net/http"
    _ "net/http/pprof"
    "runtime"
    "time"
)

func main() {
    // Enable block profiling
    runtime.SetBlockProfileRate(1)

    go func() {
        http.ListenAndServe("localhost:6060", nil)
    }()

    // Block example
    ch := make(chan int)
    go func() {
        time.Sleep(5 * time.Second)
        ch <- 42
    }()

    // This will block for 5 seconds
    <-ch

    select {}
}
```

5. Mutex Profile Analysis

```
package main

import (
    "net/http"
    _ "net/http/pprof"
    "runtime"
    "sync"
)

var mu sync.Mutex
var counter int
```

```

func main() {
    // Enable mutex profiling
    runtime.SetMutexProfileFraction(1)

    go func() {
        http.ListenAndServe("localhost:6060", nil)
    }()

    // Create mutex contention
    for i := 0; i < 10; i++ {
        go worker()
    }

    select {}
}

func worker() {
    for {
        mu.Lock()
        counter++
        mu.Unlock()
    }
}

```

บทที่ 35: สร้าง HTTP Client

เป้าหมาย: สร้าง HTTP client ที่มีประสิทธิภาพและเชื่อถือได้

Basic HTTP Client

```

package main

import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "time"
)

func main() {
    // 1. Basic GET request
    resp, err := http.Get("https://api.example.com/users")
    if err != nil {
        panic(err)
    }
}

```



```

defer resp.Body.Close()

body, _ := io.ReadAll(resp.Body)
fmt.Println(string(body))

// 2. POST request with JSON
user := map[string]interface{}{
    "name": "John Doe",
    "email": "john@example.com",
}

jsonData, _ := json.Marshal(user)

resp, err = http.Post(
    "https://api.example.com/users",
    "application/json",
    bytes.NewBuffer(jsonData),
)
defer resp.Body.Close()

// 3. Custom HTTP client with timeouts
client := &http.Client{
    Timeout: 10 * time.Second,
    Transport: &http.Transport{
        MaxIdleConns: 100,
        MaxIdleConnsPerHost: 10,
        IdleConnTimeout: 90 * time.Second,
        TLSHandshakeTimeout: 10 * time.Second,
    },
}

req, _ := http.NewRequest("GET", "https://api.example.com/users", nil)
req.Header.Set("Authorization", "Bearer token123")
req.Header.Set("User-Agent", "MyApp/1.0")

resp, err = client.Do(req)
defer resp.Body.Close()
}

```

Advanced HTTP Client with Retry

```

package client

import (
    "context"
    "fmt"
    "io"
    "net/http"
    "time"
)

```

```

type HTTPClient struct {
    client      *http.Client
    maxRetries  int
    retryDelay  time.Duration
    backoffFunc func(int) time.Duration
}

type Config struct {
    Timeout      time.Duration
    MaxRetries   int
    RetryDelay   time.Duration
    MaxIdleConns int
    IdleConnTimeout time.Duration
}

func NewHTTPClient(cfg Config) *HTTPClient {
    transport := &http.Transport{
        MaxIdleConns:      cfg.MaxIdleConns,
        MaxIdleConnsPerHost: cfg.MaxIdleConns / 2,
        IdleConnTimeout:    cfg.IdleConnTimeout,
        DisableCompression: false,
    }

    client := &http.Client{
        Timeout:   cfg.Timeout,
        Transport: transport,
    }

    return &HTTPClient{
        client:      client,
        maxRetries:  cfg.MaxRetries,
        retryDelay:  cfg.RetryDelay,
        backoffFunc: exponentialBackoff,
    }
}

func exponentialBackoff(attempt int) time.Duration {
    return time.Duration(1<<uint(attempt)) * time.Second
}

func (c *HTTPClient) Do(ctx context.Context, req *http.Request)
(*http.Response, error) {
    var lastErr error

    for attempt := 0; attempt <= c.maxRetries; attempt++ {
        // Clone request for each attempt
        reqClone := req.Clone(ctx)

        // Execute request
        resp, err := c.client.Do(reqClone)

        // Success
    }
}

```

```

    if err == nil && resp.StatusCode < 500 {
        return resp, nil
    }

    // Close body if exists
    if resp != nil {
        resp.Body.Close()
    }

    lastErr = err

    // Check if we should retry
    if !c.shouldRetry(err, resp) {
        break
    }

    // Wait before retry
    if attempt < c.maxRetries {
        delay := c.backoffFunc(attempt)
        select {
        case <-ctx.Done():
            return nil, ctx.Err()
        case <-time.After(delay):
        }
    }
}

return nil, fmt.Errorf("max retries exceeded: %w", lastErr)
}

func (c *HTTPClient) shouldRetry(err error, resp *http.Response) bool {
    if err != nil {
        // Network errors are retryable
        return true
    }

    if resp != nil {
        // Retry on server errors (5xx)
        if resp.StatusCode >= 500 && resp.StatusCode < 600 {
            return true
        }

        // Retry on rate limiting
        if resp.StatusCode == 429 {
            return true
        }
    }

    return false
}

// ใช้งาน
func main() {

```

```

cfg := Config{
    Timeout:      30 * time.Second,
    MaxRetries:   3,
    RetryDelay:   1 * time.Second,
    MaxIdleConns: 100,
    IdleConnTimeout: 90 * time.Second,
}

client := NewHTTPClient(cfg)
ctx := context.Background()

req, _ := http.NewRequestWithContext(ctx, "GET",
    "https://api.example.com/data", nil)
req.Header.Set("Authorization", "Bearer token")

resp, err := client.Do(ctx, req)
if err != nil {
    panic(err)
}
defer resp.Body.Close()

body, _ := io.ReadAll(resp.Body)
fmt.Println(string(body))
}

```

Circuit Breaker Pattern

```

package circuitbreaker

import (
    "context"
    "errors"
    "sync"
    "time"
)

type State int

const (
    StateClosed State = iota
    StateOpen
    StateHalfOpen
)

type CircuitBreaker struct {
    mu          sync.RWMutex
    state       State
    failureCount int
    lastFailureTime time.Time
}

```

```

    maxFailures      int
    timeout           time.Duration
    halfOpenMaxRequests int
    halfOpenRequests  int
}

func NewCircuitBreaker(maxFailures int, timeout time.Duration) *CircuitBreaker
{
    return &CircuitBreaker{
        state:      StateClosed,
        maxFailures: maxFailures,
        timeout:     timeout,
        halfOpenMaxRequests: 1,
    }
}

func (cb *CircuitBreaker) Call(fn func() error) error {
    if !cb.allow() {
        return errors.New("circuit breaker is open")
    }

    err := fn()

    cb.recordResult(err == nil)

    return err
}

func (cb *CircuitBreaker) allow() bool {
    cb.mu.Lock()
    defer cb.mu.Unlock()

    switch cb.state {
    case StateClosed:
        return true

    case StateOpen:
        if time.Since(cb.lastFailureTime) > cb.timeout {
            cb.state = StateHalfOpen
            cb.halfOpenRequests = 0
            return true
        }
        return false

    case StateHalfOpen:
        if cb.halfOpenRequests < cb.halfOpenMaxRequests {
            cb.halfOpenRequests++
            return true
        }
        return false

    default:
        return false
    }
}

```

```

    }
}

func (cb *CircuitBreaker) recordResult(success bool) {
    cb.mu.Lock()
    defer cb.mu.Unlock()

    if success {
        switch cb.state {
        case StateHalfOpen:
            cb.state = StateClosed
            cb.failureCount = 0
        case StateOpen:
            // Should not happen
        default:
            cb.failureCount = 0
        }
        return
    }

    // Failure
    cb.failureCount++
    cb.lastFailureTime = time.Now()

    if cb.state == StateHalfOpen {
        cb.state = StateOpen
    } else if cb.state == StateClosed && cb.failureCount >= cb.maxFailures {
        cb.state = StateOpen
    }
}

// ใช้งาน
func main() {
    cb := NewCircuitBreaker(5, 30*time.Second)

    for i := 0; i < 20; i++ {
        err := cb.Call(func() error {
            // Make HTTP request
            resp, err := http.Get("https://api.example.com/data")
            if err != nil {
                return err
            }
            defer resp.Body.Close()

            if resp.StatusCode >= 500 {
                return errors.New("server error")
            }

            return nil
        })

        if err != nil {
            fmt.Printf("Request %d failed: %v\n", i, err)

```

```

    } else {
        fmt.Printf("Request %d succeeded\n", i)
    }

    time.Sleep(1 * time.Second)
}
}

```

Connection Pool Management

```

package client

import (
    "crypto/tls"
    "net"
    "net/http"
    "time"
)

func CreateOptimizedClient() *http.Client {
    // Custom transport for optimal performance
    transport := &http.Transport{
        // Connection pooling
        MaxIdleConns:    100,
        MaxIdleConnsPerHost: 10,
        MaxConnsPerHost:  20,
        IdleConnTimeout:  90 * time.Second,

        // Connection establishment
        DialContext: (&net.Dialer{
            Timeout:   30 * time.Second,
            KeepAlive: 30 * time.Second,
        }).DialContext,

        // TLS configuration
        TLSHandshakeTimeout: 10 * time.Second,
        TLSClientConfig: &tls.Config{
            MinVersion: tls.VersionTLS12,
        },

        // Response handling
        DisableCompression: false,
        DisableKeepAlives:  false,

        // Expect header handling
        ExpectContinueTimeout: 1 * time.Second,

        // Response header timeout
        ResponseHeaderTimeout: 10 * time.Second,
    }
}

```

```

return &http.Client{
    Transport: transport,
    Timeout:   30 * time.Second,
    CheckRedirect: func(req *http.Request, via []*http.Request) error {
        if len(via) >= 10 {
            return http.ErrUseLastResponse
        }
        return nil
    },
}
}

```

บทที่ 37: การจัดการ Context

เป้าหมาย: ใช้ context สำหรับ timeout, cancellation, และ request-scoped values

Context Basics

```

package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    // 1. Background context (root context)
    ctx := context.Background()

    // 2. With cancel
    ctx, cancel := context.WithCancel(ctx)
    defer cancel()

    // 3. With timeout
    ctx, cancel = context.WithTimeout(ctx, 5*time.Second)
    defer cancel()

    // 4. With deadline
    ctx, cancel = context.WithDeadline(ctx, time.Now().Add(5*time.Second))
    defer cancel()

    // 5. With value
    ctx = context.WithValue(ctx, "user_id", 123)
    ctx = context.WithValue(ctx, "request_id", "abc-123")

    // 6. Check cancellation
    select {

```



```

    case <-ctx.Done():
        fmt.Println("Context cancelled:", ctx.Err())
    default:
        fmt.Println("Context still active")
}

// 7. Pass context to function
processRequest(ctx)
}

func processRequest(ctx context.Context) {
    // Extract values
    userID := ctx.Value("user_id")
    requestID := ctx.Value("request_id")

    fmt.Printf("Processing request %v for user %v\n", requestID, userID)

    // Check for cancellation
    select {
    case <-ctx.Done():
        fmt.Println("Request cancelled:", ctx.Err())
        return
    default:
    }

    // Do work with timeout
    done := make(chan bool)
    go func() {
        // Simulate work
        time.Sleep(2 * time.Second)
        done <- true
    }()

    select {
    case <-done:
        fmt.Println("Work completed")
    case <-ctx.Done():
        fmt.Println("Work cancelled:", ctx.Err())
    }
}

```

Context in HTTP Server

```

package main

import (
    "context"
    "fmt"
    "net/http"
    "time"

```

```

)

type key int

const (
    userKey key = iota
    requestIDKey
)

// Middleware to add request ID
func requestIDMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        requestID := r.Header.Get("X-Request-ID")
        if requestID == "" {
            requestID = generateRequestID()
        }

        ctx := context.WithValue(r.Context(), requestIDKey, requestID)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// Middleware to add timeout
func timeoutMiddleware(timeout time.Duration) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            ctx, cancel := context.WithTimeout(r.Context(), timeout)
            defer cancel()

            // Create channel for handler completion
            done := make(chan bool)

            go func() {
                next.ServeHTTP(w, r.WithContext(ctx))
                done <- true
            }()

            select {
            case <-done:
                // Handler completed
            case <-ctx.Done():
                // Timeout occurred
                http.Error(w, "Request timeout", http.StatusGatewayTimeout)
                return
            }
        })
    }
}

// Middleware to add user info
func authMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
    })
}

```

```

        // Validate token and get user
        user, err := validateToken(token)
        if err != nil {
            http.Error(w, "Unauthorized", http.StatusUnauthorized)
            return
        }

        ctx := context.WithValue(r.Context(), userKey, user)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// Handler with context
func userHandler(w http.ResponseWriter, r *http.Request) {
    // Extract values from context
    user := r.Context().Value(userKey).(*User)
    requestID := r.Context().Value(requestIDKey).(string)

    // Use context for database operations
    ctx := r.Context()

    // Query database with timeout
    data, err := fetchUserData(ctx, user.ID)
    if err != nil {
        if ctx.Err() == context.DeadlineExceeded {
            http.Error(w, "Database timeout", http.StatusGatewayTimeout)
        } else {
            http.Error(w, "Internal error", http.StatusInternalServerError)
        }
        return
    }

    fmt.Fprintf(w, "User: %+v, Request ID: %s\n", data, requestID)
}

func fetchUserData(ctx context.Context, userID int) (*UserData, error) {
    // Use context in database query
    rows, err := db.QueryContext(ctx, "SELECT * FROM users WHERE id = $1",
    userID)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    // Process rows...
    return &UserData{}, nil
}

```

Context in Database Operations

```

package repository

import (
    "context"
    "database/sql"
    "fmt"
    "time"
)

type UserRepository struct {
    db *sql.DB
}

func (r *UserRepository) GetUser(ctx context.Context, id int) (*User, error) {
    // Create context with timeout for this query
    ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel()

    var user User

    // Execute query with context
    row := r.db.QueryRowContext(ctx, "SELECT id, name, email FROM users WHERE
id = $1", id)

    err := row.Scan(&user.ID, &user.Name, &user.Email)
    if err != nil {
        if err == sql.ErrNoRows {
            return nil, ErrNotFound
        }
        return nil, err
    }

    return &user, nil
}

func (r *UserRepository) GetUsers(ctx context.Context, ids []int) ([]*User,
error) {
    // Create context with timeout
    ctx, cancel := context.WithTimeout(ctx, 10*time.Second)
    defer cancel()

    // Use context with query
    query := "SELECT id, name, email FROM users WHERE id = ANY($1)"

    rows, err := r.db.QueryContext(ctx, query, ids)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var users []*User
    for rows.Next() {

```

```

    var user User
    if err := rows.Scan(&user.ID, &user.Name, &user.Email); err != nil {
        return nil, err
    }
    users = append(users, &user)
}

return users, rows.Err()
}

// Transaction with context
func (r *UserRepository) TransferMoney(ctx context.Context, fromID, toID int,
amount float64) error {
    // Start transaction
    tx, err := r.db.BeginTx(ctx, &sql.TxOptions{
        Isolation: sql.LevelSerializable,
    })
    if err != nil {
        return err
    }
    defer tx.Rollback()

    // Withdraw from source
    _, err = tx.ExecContext(ctx,
        "UPDATE accounts SET balance = balance - $1 WHERE id = $2",
        amount, fromID)
    if err != nil {
        return err
    }

    // Deposit to destination
    _, err = tx.ExecContext(ctx,
        "UPDATE accounts SET balance = balance + $1 WHERE id = $2",
        amount, toID)
    if err != nil {
        return err
    }

    return tx.Commit()
}

```

Context with External APIs

```

package client

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"

```

```

    "time"
)

type APIClient struct {
    client *http.Client
    baseURL string
}

func NewAPIClient(baseURL string) *APIClient {
    return &APIClient{
        client: &http.Client{
            Timeout: 30 * time.Second,
        },
        baseURL: baseURL,
    }
}

func (c *APIClient) GetUser(ctx context.Context, id int) (*User, error) {
    // Create request with context
    url := fmt.Sprintf("%s/users/%d", c.baseURL, id)
    req, err := http.NewRequestWithContext(ctx, "GET", url, nil)
    if err != nil {
        return nil, err
    }

    // Add headers
    req.Header.Set("Accept", "application/json")

    // Execute request
    resp, err := c.client.Do(req)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()

    // Check status
    if resp.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("API returned status %d", resp.StatusCode)
    }

    // Decode response
    var user User
    if err := json.NewDecoder(resp.Body).Decode(&user); err != nil {
        return nil, err
    }

    return &user, nil
}

// Parallel requests with context
func (c *APIClient) GetMultipleUsers(ctx context.Context, ids []int) ([]*User, error) {
    results := make([]*User, len(ids))

```

```

errors := make([]error, len(ids))

// Create context with timeout
ctx, cancel := context.WithTimeout(ctx, 10*time.Second)
defer cancel()

// Channel for results
type result struct {
    index int
    user  *User
    err   error
}

resultCh := make(chan result, len(ids))

// Launch goroutines for each request
for i, id := range ids {
    go func(idx, userID int) {
        user, err := c.GetUser(ctx, userID)
        resultCh <- result{index: idx, user: user, err: err}
    }(i, id)
}

// Collect results
for i := 0; i < len(ids); i++ {
    select {
    case res := <-resultCh:
        results[res.index] = res.user
        errors[res.index] = res.err
    case <-ctx.Done():
        return nil, ctx.Err()
    }
}

// Check for errors
for _, err := range errors {
    if err != nil {
        return nil, err
    }
}

return results, nil
}

```

บทที่ 38: Generics – การเขียนโค้ดแบบยืดหยุ่น

เป้าหมาย: ใช้ type parameters เพื่อสร้างโค้ดที่ reuse ได้และ type-safe

Basic Generics

```

package main

import (
    "fmt"
    "constraints"
)

// 1. Generic function
func Identity[T any](x T) T {
    return x
}

// 2. Generic function with constraints
func Min[T constraints.Ordered](a, b T) T {
    if a < b {
        return a
    }
    return b
}

// 3. Generic type
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(item T) {
    s.items = append(s.items, item)
}

func (s *Stack[T]) Pop() (T, bool) {
    if len(s.items) == 0 {
        var zero T
        return zero, false
    }

    item := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return item, true
}

func (s *Stack[T]) Peek() (T, bool) {
    if len(s.items) == 0 {
        var zero T
        return zero, false
    }
    return s.items[len(s.items)-1], true
}

// 4. Generic with multiple type parameters
type Pair[K, V any] struct {
    Key    K
    Value  V
}

```



```

}

func (p Pair[K, V]) String() string {
    return fmt.Sprintf("(%v, %v)", p.Key, p.Value)
}

func main() {
    // Using generic functions
    fmt.Println(Identity(42))
    fmt.Println(Identity("hello"))

    fmt.Println(Min(10, 20))
    fmt.Println(Min(3.14, 2.71))

    // Using generic types
    intStack := Stack[int]{}
    intStack.Push(1)
    intStack.Push(2)
    val, _ := intStack.Pop()
    fmt.Println(val) // 2

    stringStack := Stack[string]{}
    stringStack.Push("hello")
    stringStack.Push("world")

    // Using Pair
    p1 := Pair[int, string]{Key: 1, Value: "one"}
    p2 := Pair[string, bool]{Key: "active", Value: true}

    fmt.Println(p1)
    fmt.Println(p2)
}

```

Advanced Generic Constraints

```

package main

import (
    "fmt"
    "math"
)

// 1. Custom constraint
type Number interface {
    ~int | ~int64 | ~float64 | ~float32
}

func Sum[T Number](values []T) T {
    var sum T
    for _, v := range values {

```

```

        sum += v
    }
    return sum
}

// 2. Interface constraint
type Stringer interface {
    String() string
}

func Print[T Stringer](t T) {
    fmt.Println(t.String())
}

// 3. Comparable constraint
func Contains[T comparable](slice []T, item T) bool {
    for _, v := range slice {
        if v == item {
            return true
        }
    }
    return false
}

// 4. Type constraints with methods
type Container[T any] interface {
    Add(item T)
    Get(index int) T
    Size() int
}

type SliceContainer[T any] struct {
    items []T
}

func (c *SliceContainer[T]) Add(item T) {
    c.items = append(c.items, item)
}

func (c *SliceContainer[T]) Get(index int) T {
    return c.items[index]
}

func (c *SliceContainer[T]) Size() int {
    return len(c.items)
}

// 5. Generic with type assertion
type JSONValue interface {
    ~string | ~int | ~float64 | ~bool | []any | map[string]any
}

func ToJSON[T JSONValue](v T) string {

```

```

    // Implementation would use type switch
    return fmt.Sprintf("%v", v)
}

// 6. Generic function with type parameters in methods
type Transformable[T any] interface {
    Transform(func(T) T) T
}

type NumberWrapper[T Number] struct {
    Value T
}

func (n NumberWrapper[T]) Transform(f func(T) T) T {
    return f(n.Value)
}

```

ตัวอย่างการใช้งานจริง

1. Generic Repository

```

package repository

import (
    "context"
    "database/sql"
    "fmt"
)

type Repository[T any] interface {
    Get(ctx context.Context, id int) (*T, error)
    Create(ctx context.Context, entity *T) error
    Update(ctx context.Context, entity *T) error
    Delete(ctx context.Context, id int) error
    List(ctx context.Context, limit, offset int) ([]*T, error)
}

type GORMRepository[T any] struct {
    db *gorm.DB
}

func NewGORMRepository[T any](db *gorm.DB) *GORMRepository[T] {
    return &GORMRepository[T]{db: db}
}

func (r *GORMRepository[T]) Get(ctx context.Context, id int) (*T, error) {
    var entity T
    result := r.db.WithContext(ctx).First(&entity, id)
    if result.Error != nil {
        return nil, result.Error
    }
}

```

```

    }
    return &entity, nil
}

func (r *GORMRepository[T]) Create(ctx context.Context, entity *T) error {
    return r.db.WithContext(ctx).Create(entity).Error
}

func (r *GORMRepository[T]) Update(ctx context.Context, entity *T) error {
    return r.db.WithContext(ctx).Save(entity).Error
}

func (r *GORMRepository[T]) Delete(ctx context.Context, id int) error {
    var entity T
    return r.db.WithContext(ctx).Delete(&entity, id).Error
}

func (r *GORMRepository[T]) List(ctx context.Context, limit, offset int) ([]*T,
error) {
    var entities []*T
    result := r.db.WithContext(ctx).
        Limit(limit).
        Offset(offset).
        Find(&entities)
    return entities, result.Error
}

// ใช้งาน
type User struct {
    ID    int    `gorm:"primaryKey"`
    Name  string `gorm:"size:100"`
    Email string `gorm:"uniqueIndex;size:100"`
}

type Product struct {
    ID    int    `gorm:"primaryKey"`
    Name  string `gorm:"size:100"`
    Price float64 `gorm:"type:decimal(10,2)"`
}

func main() {
    db := connectDB()

    userRepo := NewGORMRepository[User](db)
    productRepo := NewGORMRepository[Product](db)

    ctx := context.Background()

    // Use user repository
    user, _ := userRepo.Get(ctx, 1)

    // Use product repository

```

```
    products, _ := productRepo.List(ctx, 10, 0)
}
```

2. Generic Cache

```
package cache

import (
    "context"
    "encoding/json"
    "time"

    "github.com/redis/go-redis/v9"
)

type Cache[T any] struct {
    client *redis.Client
    prefix string
    ttl    time.Duration
}

func NewCache[T any](client *redis.Client, prefix string, ttl time.Duration)
*Cache[T] {
    return &Cache[T]{
        client: client,
        prefix: prefix,
        ttl:    ttl,
    }
}

func (c *Cache[T]) key(id string) string {
    return fmt.Sprintf("%s:%s", c.prefix, id)
}

func (c *Cache[T]) Get(ctx context.Context, id string) (*T, error) {
    data, err := c.client.Get(ctx, c.key(id)).Bytes()
    if err != nil {
        return nil, err
    }

    var result T
    if err := json.Unmarshal(data, &result); err != nil {
        return nil, err
    }

    return &result, nil
}

func (c *Cache[T]) Set(ctx context.Context, id string, value *T) error {
    data, err := json.Marshal(value)
    if err != nil {

```

```

        return err
    }

    return c.client.Set(ctx, c.key(id), data, c.ttl).Err()
}

func (c *Cache[T]) Delete(ctx context.Context, id string) error {
    return c.client.Del(ctx, c.key(id)).Err()
}

// ใช้งาน
type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email"`
}

func main() {
    rdb := redis.NewClient(&redis.Options{
        Addr: "localhost:6379",
    })

    userCache := NewCache[User](rdb, "user", 10*time.Minute)

    ctx := context.Background()

    // Get user from cache
    user, err := userCache.Get(ctx, "123")
    if err == nil {
        fmt.Printf("User from cache: %+v\n", user)
    } else {
        // Get from database
        user = getUserFromDB(123)
        userCache.Set(ctx, "123", user)
    }
}

```

3. Generic Worker Pool

```

package worker

import (
    "context"
    "sync"
)

type Job[T any] struct {
    ID    string
    Data  T
    Result chan Result[T]
}

```

```

type Result[T any] struct {
    JobID string
    Value T
    Error error
}

type WorkerPool[T any, R any] struct {
    numWorkers int
    jobQueue   chan Job[T]
    resultQueue chan Result[R]
    workerFunc func(Job[T]) Result[R]
    wg         sync.WaitGroup
    ctx         context.Context
    cancel      context.CancelFunc
}

func NewWorkerPool[T any, R any](numWorkers int, queueSize int, workerFunc
func(Job[T]) Result[R]) *WorkerPool[T, R] {
    ctx, cancel := context.WithCancel(context.Background())

    return &WorkerPool[T, R]{
        numWorkers: numWorkers,
        jobQueue:   make(chan Job[T], queueSize),
        resultQueue: make(chan Result[R], queueSize),
        workerFunc: workerFunc,
        ctx:        ctx,
        cancel:     cancel,
    }
}

func (wp *WorkerPool[T, R]) Start() {
    for i := 0; i < wp.numWorkers; i++ {
        wp.wg.Add(1)
        go wp.worker()
    }

    go func() {
        wp.wg.Wait()
        close(wp.resultQueue)
    }()
}

func (wp *WorkerPool[T, R]) worker() {
    defer wp.wg.Done()

    for {
        select {
        case <-wp.ctx.Done():
            return
        case job, ok := <-wp.jobQueue:
            if !ok {
                return
            }

```

```

    }

    result := wp.workerFunc(job)

    select {
    case wp.resultQueue <- result:
    case <-wp.ctx.Done():
        return
    }
}

}

func (wp *WorkerPool[T, R]) Submit(job Job[T]) {
    select {
    case wp.jobQueue <- job:
    case <-wp.ctx.Done():
    }
}

func (wp *WorkerPool[T, R]) Results() <-chan Result[R] {
    return wp.resultQueue
}

func (wp *WorkerPool[T, R]) Stop() {
    wp.cancel()
    close(wp.jobQueue)
    wp.wg.Wait()
}

// ใช้งาน
func main() {
    // Define worker function
    workerFunc := func(job Job[string]) Result[int] {
        // Process job
        length := len(job.Data)
        return Result[int]{
            JobID: job.ID,
            Value: length,
        }
    }

    // Create worker pool
    pool := NewWorkerPool[string, int](5, 100, workerFunc)
    pool.Start()

    // Submit jobs
    for i := 0; i < 100; i++ {
        job := Job[string]{
            ID:    fmt.Sprintf("job-%d", i),
            Data:  fmt.Sprintf("data-%d", i),
        }
        pool.Submit(job)
    }
}

```



```

    }

    // Collect results
    for result := range pool.Results() {
        fmt.Printf("Job %s result: %d\n", result.JobID, result.Value)
    }

    pool.Stop()
}

```

2. การออกแบบโค้ด (Code Design)

บทที่ 39: Go กับกระบวนทัศน์ OOP?

เป้าหมาย: เข้าใจว่า Go จัดการกับ OOP อย่างไร (struct, method, interface, composition)

Struct และ Method (แทน Class)

```

package main

import "fmt"

// 1. Struct (แทน class)
type Rectangle struct {
    Width  float64
    Height float64
}

// 2. Constructor pattern
func NewRectangle(width, height float64) *Rectangle {
    return &Rectangle{
        Width:  width,
        Height: height,
    }
}

// 3. Methods (value receiver)
func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

// 4. Methods (pointer receiver)
func (r *Rectangle) Scale(factor float64) {
    r.Width *= factor
    r.Height *= factor
}

// 5. Inheritance via composition (ไม่ใช่ inheritance แบบ OOP)
type ColoredRectangle struct {

```

```

    Rectangle // Embedded struct (composition)
    Color      string
}

func (c ColoredRectangle) Describe() string {
    return fmt.Sprintf("Rectangle with color %s, area %.2f",
        c.Color, c.Area())
}

// 6. Polymorphism via interfaces
type Shape interface {
    Area() float64
    Perimeter() float64
}

type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func (c Circle) Perimeter() float64 {
    return 2 * math.Pi * c.Radius
}

// 7. Interface composition
type Drawable interface {
    Draw() string
}

type ShapeWithDraw interface {
    Shape
    Drawable
}

func main() {
    // Create objects
    rect := NewRectangle(10, 5)
    fmt.Printf("Area: %.2f\n", rect.Area())

    rect.Scale(2)
    fmt.Printf("After scale: %.2f x %.2f\n", rect.Width, rect.Height)

    // Composition
    colored := ColoredRectangle{
        Rectangle: Rectangle{Width: 10, Height: 5},
        Color:      "red",
    }
    fmt.Println(colored.Describe())

    // Polymorphism

```

```

shapes := []Shape{
    Rectangle{Width: 10, Height: 5},
    Circle{Radius: 7},
}

for _, shape := range shapes {
    fmt.Printf("Area: %.2f, Perimeter: %.2f\n",
        shape.Area(), shape.Perimeter())
}
}

```

Composition over Inheritance

```

package main

import "fmt"

// 1. Base struct (ไม่ใช่ parent class)
type Animal struct {
    Name string
    Age  int
}

func (a Animal) Speak() string {
    return "Some sound"
}

func (a Animal) Info() string {
    return fmt.Sprintf("Name: %s, Age: %d", a.Name, a.Age)
}

// 2. Composition via embedding
type Dog struct {
    Animal    // Embedded (has-a relationship)
    Breed string
}

// Override method (not override, just new method)
func (d Dog) Speak() string {
    return "Woof!"
}

// 3. Composition with interface
type Flyer interface {
    Fly() string
}

type Swimmer interface {
    Swim() string
}

```

```

type Duck struct {
    Name string
}

func (d Duck) Fly() string {
    return "Flying"
}

func (d Duck) Swim() string {
    return "Swimming"
}

// 4. Composing multiple behaviors
type SuperDuck struct {
    Duck
    Flyer
    Swimmer
}

// 5. Dependency injection (better than inheritance)
type Database interface {
    Save(data string) error
}

type Logger interface {
    Log(message string)
}

type UserService struct {
    db      Database
    logger  Logger
}

func NewUserService(db Database, logger Logger) *UserService {
    return &UserService{
        db:      db,
        logger: logger,
    }
}

func (s *UserService) CreateUser(name string) error {
    s.logger.Log(fmt.Sprintf("Creating user: %s", name))
    return s.db.Save(name)
}

// 6. Functional options pattern (OOP alternative)
type Server struct {
    host      string
    port      int
    timeout   time.Duration
    maxConn   int
}

```

```

type Option func(*Server)

func WithHost(host string) Option {
    return func(s *Server) {
        s.host = host
    }
}

func WithPort(port int) Option {
    return func(s *Server) {
        s.port = port
    }
}

func WithTimeout(timeout time.Duration) Option {
    return func(s *Server) {
        s.timeout = timeout
    }
}

func NewServer(opts ...Option) *Server {
    // Default values
    s := &Server{
        host:    "localhost",
        port:     8080,
        timeout:  30 * time.Second,
        maxConn:  100,
    }

    // Apply options
    for _, opt := range opts {
        opt(s)
    }

    return s
}

```

Interface Design Principles

```

package main

import "fmt"

// 1. Small interfaces (Interface Segregation Principle)
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {

```

```

    Write(p []byte) (n int, err error)
}

type Closer interface {
    Close() error
}

// 2. Composed interfaces
type ReadWriter interface {
    Reader
    Writer
}

type ReadWriteCloser interface {
    Reader
    Writer
    Closer
}

// 3. Accept interfaces, return structs
type User struct {
    ID    int
    Name  string
}

type UserRepository interface {
    GetByID(id int) (*User, error)
    Save(user *User) error
}

// Implementation returns struct, not interface
type PostgresUserRepository struct {
    db *sql.DB
}

func NewPostgresUserRepository(db *sql.DB) *PostgresUserRepository {
    return &PostgresUserRepository{db: db}
}

func (r *PostgresUserRepository) GetByID(id int) (*User, error) {
    // implementation
    return &User{}, nil
}

func (r *PostgresUserRepository) Save(user *User) error {
    // implementation
    return nil
}

// 4. Interface for testing
type MockUserRepository struct {
    users map[int]*User
}

```

```
func (m *MockUserRepository) GetByID(id int) (*User, error) {
    if user, ok := m.users[id]; ok {
        return user, nil
    }
    return nil, ErrNotFound
}
```

บทที่ 40: การอัปเดตหรือดาวน์โหลดเวอร์ชัน Go

เป้าหมาย: จัดการเวอร์ชัน Go ในโปรเจกต์อย่างถูกต้อง

การอัปเดต Go เวอร์ชัน

```
# 1. ตรวจสอบเวอร์ชันปัจจุบัน
go version

# 2. ดาวน์โหลดเวอร์ชันใหม่
# ไปที่ https://go.dev/dl/
# หรือใช้ brew (macOS)
brew upgrade go

# 3. ตรวจสอบเวอร์ชันใหม่
go version

# 4. อัปเดตโมดูล
go mod edit -go=1.22

# 5. อัปเดต dependencies
go get -u ./...

# 6. ใช้ go fix สำหรับการเปลี่ยนแปลง API
go fix ./...

# 7. ตรวจสอบโค้ด
go vet ./...

# 8. รันทดสอบ
go test ./...
```

การจัดการหลายเวอร์ชันด้วย gvm

```
# 1. ติดตั้ง gvm
bash < <(curl -s -S -L
https://raw.githubusercontent.com/moovweb/gvm/master/binscripts/gvm-installer)
```

```
# 2. รายการเวอร์ชันที่สามารถติดตั้งได้
gvm listall

# 3. ติดตั้งเวอร์ชันต่างๆ
gvm install go1.20
gvm install go1.21
gvm install go1.22

# 4. ใช้เวอร์ชันเฉพาะ
gvm use go1.22 --default

# 5. สร้าง environment สำหรับโปรเจกต์
gvm pkgset create myproject
gvm pkgset use myproject
```

การอัปเดตโค้ดสำหรับ Go เวอร์ชันใหม่

```
// Go 1.18+ (Generics)
// Old code (before 1.18)
func MinInt(a, b int) int {
    if a < b {
        return a
    }
    return b
}

func MinFloat(a, b float64) float64 {
    if a < b {
        return a
    }
    return b
}

// New code with generics
func Min[T constraints.Ordered](a, b T) T {
    if a < b {
        return a
    }
    return b
}

// Go 1.21+ (slog)
// Old code (using log)
log.Printf("User %d logged in", userID)

// New code with slog
slog.Info("User logged in", "user_id", userID)

// Go 1.22+ (range over integers)
// Old code
```



```

for i := 0; i < 10; i++ {
    fmt.Println(i)
}

// New code (Go 1.22+)
for i := range 10 {
    fmt.Println(i)
}

```

Compatibility Check

```

// ใช้ build tags สำหรับเวอร์ชันเฉพาะ
// +build go1.22

package main

import "fmt"

func main() {
    // โค้ดที่ใช้เฉพาะ Go 1.22+
    for i := range 10 {
        fmt.Println(i)
    }
}

```

```

// ใช้ runtime.Version() ตรวจสอบ
package main

import (
    "fmt"
    "runtime"
    "strconv"
    "strings"
)

func main() {
    version := runtime.Version()
    fmt.Printf("Go version: %s\n", version)

    // ตรวจสอบ major version
    if strings.HasPrefix(version, "go1.22") {
        fmt.Println("Using Go 1.22+ features")
    }
}

// ใช้ build constraints
//go:build go1.22
package main

```

บทที่ 41: คำแนะนำในการออกแบบโค้ดที่ดี

เป้าหมาย: เรียนรู้ best practices และ idioms ของ Go

Naming Conventions

```
// 1. Package names (lowercase, short)
package user    // good
package userService // bad (too long)

// 2. Exported names (PascalCase)
type UserService struct {} // exported
type userRepository struct {} // unexported

// 3. Un-exported names (camelCase)
var dbConnection *sql.DB
func getUserByID(id int) {}

// 4. Acronyms (keep case)
type HTTPClient struct {} // good
type HttpClient struct {} // bad

// 5. Interface naming
type Reader interface {} // -er suffix
type Writer interface {}
type Stringer interface {}

// 6. Getters (no Get prefix)
type User struct {
    name string
}

func (u *User) Name() string { // good
    return u.name
}

func (u *User) GetName() string { // bad (in Go)
    return u.name
}
```

Error Handling Patterns

```
// 1. Check errors immediately
func readFile(filename string) ([]byte, error) {
    data, err := os.ReadFile(filename)
    if err != nil {
```

```

        return nil, fmt.Errorf("read file %s: %w", filename, err)
    }
    return data, nil
}

// 2. Wrap errors with context
func processRequest(req *http.Request) error {
    user, err := getUser(req.Context(), req.Header.Get("User-ID"))
    if err != nil {
        return fmt.Errorf("get user: %w", err)
    }

    if err := validateUser(user); err != nil {
        return fmt.Errorf("validate user %d: %w", user.ID, err)
    }

    return nil
}

// 3. Custom error types
type ValidationError struct {
    Field    string
    Value    interface{}
    Message  string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("validation failed for %s: %s", e.Field, e.Message)
}

// 4. Sentinel errors
var (
    ErrNotFound      = errors.New("not found")
    ErrUnauthorized  = errors.New("unauthorized")
    ErrInvalidInput  = errors.New("invalid input")
)

// 5. Error handling in defer
func doSomething() (err error) {
    f, err := os.Open("file.txt")
    if err != nil {
        return err
    }
    defer func() {
        if closeErr := f.Close(); closeErr != nil && err == nil {
            err = closeErr
        }
    }()

    // process file
    return nil
}

```

Code Organization

```
// 1. Project structure
/*
myapp/
├── cmd/
│   ├── api/
│   │   └── main.go
│   └── worker/
│       └── main.go
├── internal/
│   ├── config/
│   ├── domain/
│   │   ├── user/
│   │   │   ├── entity.go
│   │   │   ├── repository.go
│   │   │   └── service.go
│   │   └── order/
│   ├── infrastructure/
│   │   ├── db/
│   │   ├── cache/
│   │   └── queue/
│   └── interfaces/
│       ├── http/
│       └── grpc/
├── pkg/
│   ├── logger/
│   └── utils/
├── test/
├── go.mod
└── go.sum
*/

// 2. Function ordering in files
package user

import ()

// Constants
const ()

// Variables
var ()

// Types
type Service struct {}

// Constructor
func NewService() *Service {
    return &Service{}
}
```

```

}

// Public methods
func (s *Service) GetUser(id int) (*User, error) {}

// Private methods
func (s *Service) validateUser(user *User) error {}

// Helper functions
func generateID() string {}

```

Concurrency Patterns

```

// 1. Use channels for communication
func worker(jobs <-chan int, results chan<- int) {
    for job := range jobs {
        results <- process(job)
    }
}

// 2. Use sync.WaitGroup for coordination
func processBatch(items []Item) {
    var wg sync.WaitGroup
    for _, item := range items {
        wg.Add(1)
        go func(item Item) {
            defer wg.Done()
            process(item)
        }(item)
    }
    wg.Wait()
}

// 3. Use context for cancellation
func queryWithTimeout(ctx context.Context, query string) error {
    ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel()

    return db.QueryContext(ctx, query)
}

// 4. Don't start goroutines without knowing when to stop
type Worker struct {
    stop chan struct{}
    wg    sync.WaitGroup
}

func (w *Worker) Start() {
    w.wg.Add(1)
    go func() {

```

```

        defer w.wg.Done()
        for {
            select {
            case <-w.stop:
                return
            default:
                // do work
            }
        }
    }()
}

func (w *Worker) Stop() {
    close(w.stop)
    w.wg.Wait()
}

```

Performance Tips

```

// 1. Preallocate slices
// Bad
var result []int
for i := 0; i < 1000000; i++ {
    result = append(result, i)
}

// Good
result := make([]int, 0, 1000000)
for i := 0; i < 1000000; i++ {
    result = append(result, i)
}

// 2. Use strings.Builder for concatenation
// Bad
s := ""
for i := 0; i < 1000; i++ {
    s += "a"
}

// Good
var builder strings.Builder
builder.Grow(1000)
for i := 0; i < 1000; i++ {
    builder.WriteString("a")
}
s := builder.String()

// 3. Use sync.Pool for temporary objects
var bufferPool = sync.Pool{
    New: func() interface{} {

```

```

        return new(bytes.Buffer)
    },
}

func process() {
    buf := bufferPool.Get().(*bytes.Buffer)
    defer bufferPool.Put(buf)
    buf.Reset()

    // use buf
}

// 4. Avoid unnecessary allocations
// Bad
func getID() string {
    return fmt.Sprintf("%d", id)
}

// Good
var idBuffer [20]byte
func getID() string {
    return strconv.Itoa(id)
}

```

3. เอกสารอ้างอิง (Reference)

บทที่ 42: ซีทชีต (Cheatsheet)

Go Commands

```

# Development
go mod init <module>      # Initialize module
go mod tidy               # Add missing dependencies
go mod vendor             # Create vendor directory
go build                  # Build binary
go run <file>             # Run program
go test ./...             # Run tests
go test -bench .          # Run benchmarks
go test -cover            # Test coverage
go fmt ./...              # Format code
go vet ./...              # Static analysis
go generate ./...         # Generate code

# Tools
go get <package>          # Download package
go install <package>      # Install binary
go list -m all            # List dependencies
go mod graph              # Show dependency graph
go mod why <package>      # Why is package needed

```

```
# Profiling
go test -cpuprofile cpu.prof
go test -memprofile mem.prof
go test -blockprofile block.prof
go tool pprof cpu.prof
```

Common Patterns

```
// Type declarations
type MyInt int
type User struct {
    ID    int    `json:"id"`
    Name string `json:"name"`
}
type Reader interface {
    Read(p []byte) (n int, err error)
}

// Error handling
if err != nil {
    return fmt.Errorf("operation failed: %w", err)
}

// Defer
defer func() {
    if r := recover(); r != nil {
        log.Printf("recovered: %v", r)
    }
}()

// Channels
ch := make(chan int, 10)
go func() { ch <- 42 }()
value := <-ch

// Select
select {
case <-ch1:
    // handle
case <-ch2:
    // handle
case <-time.After(5 * time.Second):
    // timeout
default:
    // non-blocking
}

// Context
ctx := context.Background()
```



```
ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
defer cancel()
```

Useful Functions

```
// Strings
strings.Contains(s, substr)
strings.HasPrefix(s, prefix)
strings.HasSuffix(s, suffix)
strings.Join(slice, sep)
strings.Split(s, sep)
strings.ReplaceAll(s, old, new)
strings.ToLower(s)
strings.ToUpper(s)
strings.TrimSpace(s)

// Slices
len(slice)
cap(slice)
append(slice, elem...)
copy(dst, src)
make([]T, len, cap)

// Maps
len(map)
make(map[K]V)
delete(map, key)
val, ok := map[key]

// Time
time.Now()
time.Parse(layout, value)
t.Format(layout)
t.Add(duration)
t.Sub(t2)
time.Sleep(duration)
time.After(duration)

// JSON
json.Marshal(v)
json.Unmarshal(data, &v)
json.NewEncoder(w).Encode(v)
json.NewDecoder(r).Decode(&v)
```

HTTP Server Patterns

```
// Basic server
srv := &http.Server{
    Addr:           ":8080",
    Handler:        handler,
    ReadTimeout:    15 * time.Second,
    WriteTimeout:   15 * time.Second,
}
srv.ListenAndServe()

// Graceful shutdown
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
srv.Shutdown(ctx)

// Middleware
func Logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        log.Printf("%s %s", r.Method, r.URL.Path)
        next.ServeHTTP(w, r)
    })
}
```

สรุปภาคที่ 5: เส้นทางสู่มืออาชีพ

```
flowchart LR
    subgraph Stage1["ขั้นที่ 1: วัดผล"]
        A1[Benchmark]
        A2[Profiling]
    end

    subgraph Stage2["ขั้นที่ 2: ปรับปรุง"]
        B1[Optimize HTTP Client]
        B2[Use Context]
        B3[Apply Generics]
    end

    subgraph Stage3["ขั้นที่ 3: ออกแบบ"]
        C1[Apply Go Idioms]
        C2[Composition over Inheritance]
        C3[Small Interfaces]
    end

    subgraph Stage4["ขั้นที่ 4: เอกสาร"]
        D1[Cheatsheet]
    end
```

```
D2[Code Review Checklist]
end

Stage1 --> Stage2
Stage2 --> Stage3
Stage3 --> Stage4
```

สิ่งที่ได้เรียนรู้ในภาคที่ 5:

1. การวัดประสิทธิภาพ (บทที่ 34, 36):

- เขียน benchmark และวิเคราะห์ผล
- ใช้ pprof สำหรับ CPU, Memory, Goroutine profiling
- เปรียบเทียบประสิทธิภาพระหว่างเวอร์ชัน

2. การปรับแต่ง (บทที่ 35, 37, 38):

- สร้าง HTTP client ที่มีประสิทธิภาพ
- ใช้ context สำหรับ timeout และ cancellation
- ใช้ generics สำหรับโค้ดที่ reuse ได้

3. การออกแบบ (บทที่ 39, 40, 41):

- เข้าใจ OOP ใน Go (struct, method, interface, composition)
- จัดการเวอร์ชัน Go
- ใช้ best practices และ idioms

4. เอกสารอ้างอิง (บทที่ 42):

- Cheatsheet สำหรับการพัฒนารวดเร็ว

พื้นฐานเหล่านี้จะนำไปใช้ใน ภาคที่ 6 เพื่อเรียนรู้เครื่องมือและไลบรารียอดนิยม เช่น chi, viper, cobra, zap, GORM

บทที่ 34: การวัดประสิทธิภาพ (Benchmarks)

34.1 การเขียน benchmark

ไฟล์ `_test.go` ฟังก์ชันเริ่มต้นด้วย `Benchmark` รับ `*testing.B`

```
func BenchmarkAdd(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Add(1, 2)
    }
}
```

34.2 การรัน benchmark

```

go test -bench=.          # รันทั้งหมด
go test -bench=Add        # รันเฉพาะ Add
go test -bench=. -benchmem # แสดง memory allocation

```

34.3 การ reset timer

```

func BenchmarkSomething(b *testing.B) {
    setup()
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        // code to benchmark
    }
}

```

34.4 การเปรียบเทียบประสิทธิภาพ

ใช้ `benchstat` tool

34.5 การเขียน benchmark ที่มีการทำงานที่แตกต่างกัน

```

func BenchmarkAdd(b *testing.B) {
    tests := []struct{
        name string
        a,b int
    }{
        {"small", 1,2},
        {"large", 1000000, 2000000},
    }
    for _, tt := range tests {
        b.Run(tt.name, func(b *testing.B) {
            for i := 0; i < b.N; i++ {
                Add(tt.a, tt.b)
            }
        })
    }
}

```

บทที่ 35: สร้าง HTTP Client

35.1 การใช้ http.Get

```

resp, err := http.Get("https://api.example.com/data")
if err != nil { ... }

```

```
defer resp.Body.Close()
body, _ := io.ReadAll(resp.Body)
```

35.2 การสร้าง custom client

```
client := &http.Client{
    Timeout: 10 * time.Second,
    Transport: &http.Transport{
        MaxIdleConns: 10,
        IdleConnTimeout: 30 * time.Second,
    },
}
resp, err := client.Get(url)
```

35.3 การส่ง POST request พร้อม JSON

```
data := map[string]interface{}{"name": "John"}
jsonData, _ := json.Marshal(data)

req, _ := http.NewRequest("POST", url, bytes.NewBuffer(jsonData))
req.Header.Set("Content-Type", "application/json")

resp, err := client.Do(req)
```

35.4 การจัดการ cookies

```
jar, _ := cookiejar.New(nil)
client := &http.Client{Jar: jar}
```

35.5 การใช้ context กับ request

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
req := req.WithContext(ctx)
resp, err := client.Do(req)
```

35.6 การ retry logic

การใช้ library เช่น github.com/avast/retry-go

บทที่ 36: การวิเคราะห์โปรไฟล์ (Program Profiling)

36.1 การใช้ pprof

Go มี built-in profiler สำหรับ CPU, memory, goroutine, block, mutex

36.2 การเปิด pprof endpoint ใน HTTP server

```
import _ "net/http/pprof"

func main() {
    go func() {
        log.Println(http.ListenAndServe("localhost:6060", nil))
    }()
    // ...
}
```

36.3 การเก็บ CPU profile

```
f, _ := os.Create("cpu.prof")
pprof.StartCPUProfile(f)
defer pprof.StopCPUProfile()
// run code
```

36.4 การวิเคราะห์ profile

```
go tool pprof -http=:8080 cpu.prof
```

36.5 การตรวจสอบ memory allocation

```
go test -memprofile mem.prof -bench .
```

36.6 การวิเคราะห์ goroutine

```
curl http://localhost:6060/debug/pprof/goroutine?debug=1
```

บทที่ 37: การจัดการ Context

37.1 Context คืออะไร?

Context ใช้ส่งค่า deadline, cancellation, และ request-scoped values ผ่านขอบเขตของ API

37.2 การสร้าง context

```
ctx := context.Background() // empty context, root
ctx := context.TODO()      // เมื่อยังไม่แน่ใจ
```

37.3 Context with cancellation

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()
// ถ้าเรียก cancel() จะทำให้ ctx.Done() ถูกปิด
```

37.4 Context with timeout

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
```

37.5 Context with deadline

```
deadline := time.Now().Add(5*time.Second)
ctx, cancel := context.WithDeadline(context.Background(), deadline)
```

37.6 การตรวจสอบ context

```
select {
case <-ctx.Done():
    fmt.Println("context cancelled:", ctx.Err())
    return
default:
    // do work
}
```

37.7 การส่งค่าใน context (ไม่ค่อยใช้มาก)

```
type key string
ctx := context.WithValue(context.Background(), key("userID"), 123)
userID := ctx.Value(key("userID")).(int)
```

37.8 การใช้ context ใน HTTP handler

```
func handler(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()
    // ใช้ ctx ในการทำงาน
}
```

บทที่ 38: Generics - การเขียนโค้ดแบบยืดหยุ่น

38.1 Generics ใน Go 1.18+

Generics ช่วยให้เขียนฟังก์ชันหรือชนิดข้อมูลที่ทำงานกับหลายชนิดได้ โดยไม่ต้องใช้ interface{} และ type assertion

38.2 ฟังก์ชัน generic

```
func Map[T any](s []T, f func(T) T) []T {
    result := make([]T, len(s))
    for i, v := range s {
        result[i] = f(v)
    }
    return result
}

// เรียกใช้
nums := []int{1,2,3}
doubled := Map(nums, func(x int) int { return x * 2 })
```

38.3 Type constraints

```
func Sum[T int | float64](a, b T) T {
    return a + b
}
```

38.4 การใช้ interface เป็น constraint

```
type Number interface {
    int | int64 | float64
}

func Max[T Number](a, b T) T {
    if a > b {
        return a
    }
    return b
}
```


38.5 Generic types (structs)

```
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(item T) {
    s.items = append(s.items, item)
}

func (s *Stack[T]) Pop() T {
    item := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return item
}
```

38.6 constraints แพคเกจ

Go มีแพคเกจ `constraints` (experimental) สำหรับ constraints ทั่วไป

บทที่ 39: Go กับกระบวนทัศน์ OOP?

39.1 Go ไม่ใช่ภาษา OOP แบบดั้งเดิม

Go ไม่มี class, inheritance, polymorphism ผ่าน method overriding แต่ใช้ composition และ interface

39.2 Encapsulation

ใช้ package visibility (ตัวพิมพ์เล็ก/ใหญ่)

39.3 Polymorphism

ใช้ interface แทน inheritance

```
type Shape interface {
    Area() float64
}

type Circle struct{ Radius float64 }
func (c Circle) Area() float64 { return math.Pi * c.Radius * c.Radius }

type Rectangle struct{ W, H float64 }
func (r Rectangle) Area() float64 { return r.W * r.H }

func PrintArea(s Shape) {
```

```
    fmt.Println(s.Area())
}
```

39.4 Composition over inheritance

```
type Engine struct{}
func (e Engine) Start() { ... }

type Car struct {
    Engine // embedded
    Wheels int
}
// Car สามารถเรียก Start() ได้โดยตรง
```

39.5 การ reuse โค้ด

ใช้ function, interface, embedding แทน inheritance

39.6 ข้อดีของแนวทางนี้

- ลด coupling
- เข้าใจง่าย
- หลีกเลี่ยงปัญหา "diamond problem"

บทที่ 40: การอัปเดตหรือดาวน์เกรดเวอร์ชัน Go

40.1 การติดตั้งหลายเวอร์ชัน

ใช้ `go install golang.org/dl/go1.20@latest` แล้ว `go1.20 download`

40.2 การเปลี่ยนเวอร์ชัน (Linux/macOS)

ใช้ `go get golang.org/dl/go1.21.0` แล้ว `go1.21.0 download` หรือใช้ `gvm` (Go Version Manager)

40.3 การอัปเดต go.mod

```
go mod edit -go=1.21
go mod tidy
```

40.4 การตรวจสอบ compatibility

ใช้ `go fix` เพื่ออัปเดตโค้ดอัตโนมัติ

40.5 การทดสอบกับเวอร์ชันใหม่

ใช้ CI/CD เพื่อทดสอบหลายเวอร์ชัน

40.6 การดาวน์เกรด

ถ้าต้องการกลับไปใช้เวอร์ชันเก่า ให้เปลี่ยน PATH หรือใช้ version manager

บทที่ 41: คำแนะนำในการออกแบบโค้ดที่ดี

41.1 Go idioms

- **Keep it simple:** หลีกเลี่ยงความซับซ้อนที่ไม่จำเป็น
- **Use interfaces only when necessary:** อย่าสร้าง interface ก่อน แต่ให้สร้างเมื่อมีความจำเป็น
- **Accept interfaces, return structs:** ฟังก์ชันควรรับ interface, คืน concrete type
- **Use zero values:** ใช้ zero value ให้เป็นประโยชน์
- **Prefer composition over inheritance**

41.2 การตั้งชื่อ

- **Package names:** สั้น, เป็นคำเดียว, ตัวพิมพ์เล็ก
- **Variables:** ใช้ชื่อสั้นภายในขอบเขตเล็ก, ชื่อยาวในขอบเขตใหญ่
- **Getters:** ไม่ใช่ `Get` prefix เช่น `Name()` แทน `GetName()`

41.3 การจัดการ error

- ตรวจสอบ error ทันที
- ห่อ error ด้วยข้อมูลเพิ่มเติม (context)
- อย่าใช้ panic สำหรับการจัดการ error ปกติ

41.4 การจัดโครงสร้างโปรเจกต์

```
/cmd/           # executables
/internal/      # private code
/pkg/          # public library code
/api/          # API definitions
```

41.5 การเขียน documentation

- ใช้ comment ก่อนฟังก์ชัน/ชนิด
- ตัวอย่างใน comment จะปรากฏใน `go doc`

41.6 การใช้ linter

```
golangci-lint run
```

41.7 การทำ code review

- ตรวจสอบ error handling
- ดู race conditions
- ตรวจสอบการใช้ goroutine และ channel

บทที่ 42: ซีทชีท (Cheatsheet)

42.1 คำสั่ง Go ที่ใช้บ่อย

```
go mod init <module>      # เริ่มต้นโมดูล
go mod tidy                # จัด dependencies
go build                   # คอมไพล์
go run <file>              # รันโดยตรง
go test ./...              # ทดสอบทั้งหมด
go fmt ./...               # จัดรูปแบบโค้ด
go vet ./...               # ตรวจสอบข้อผิดพลาดที่อาจเกิดขึ้น
go get <pkg>@<version>    # ดาวน์โหลดแพคเกจ
go install <pkg>@latest    # ติดตั้ง binary
go doc <pkg>               # ดู documentation
```

42.2 ชนิดข้อมูล

Type	Description
<code>bool</code>	true/false
<code>string</code>	immutable sequence
<code>int, int8, int16, int32, int64</code>	signed integers
<code>uint, uint8(byte), uint16, uint32, uint64</code>	unsigned integers
<code>float32, float64</code>	floating point
<code>complex64, complex128</code>	complex numbers
<code>rune</code>	alias for int32 (Unicode)

42.3 การประกาศตัวแปร

```
var name string
var age int = 30
name := "John"           // short declaration (inside function)
var x, y int = 1, 2
var (                     // block
    a string
    b int
)
```

42.4 โครงสร้างควบคุม

```
// if
if x > 0 { ... } else if x < 0 { ... } else { ... }

// for
for i := 0; i < 10; i++ { ... }
for condition { ... }
for { ... break }
```

```
// switch
switch value {
case 1:
    ...
default:
    ...
}
```

42.5 slice และ map

```
// slice
s := []int{1,2,3}
s = append(s, 4)
s2 := make([]int, 5, 10)
copy(dst, src)

// map
m := make(map[string]int)
m["key"] = 1
delete(m, "key")
val, ok := m["key"]
```

42.6 channel

```
ch := make(chan int)
ch <- 42
val := <-ch
close(ch)
```

42.7 การทำงานกับไฟล์

```
data, _ := os.ReadFile("file.txt")
os.WriteFile("out.txt", []byte("data"), 0644)
```

42.8 JSON

```
type T struct {
    Name string `json:"name"`
}
data, _ := json.Marshal(t)
json.Unmarshal(data, &t)
```

42.9 HTTP

```
http.HandleFunc("/", handler)
http.ListenAndServe(":8080", nil)
```

42.10 Context

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
```

ภาคที่ 6: เครื่องมือและไลบรารียอดนิยม

(บทที่ 43–45)

แผนภาพ Data Flow (Flowchart TB)

```
flowchart TB
    subgraph Entry["จุดเริ่มต้น (Entry Points)"]
        A1["CLI Application<br/>cobra"]
        A2["HTTP Server<br/>chi"]
        A3["Configuration<br/>viper"]
    end

    subgraph Core["แกนหลัก (Core Layer)"]
        B1["Routing & Middleware<br/>chi"]
        B2["Business Logic<br/>Custom Handlers"]
        B3["Validation<br/>go-playground/validator"]
        B4["Authentication<br/>golang-jwt/jwt"]
    end

    subgraph DataLayer["ชั้นข้อมูล (Data Layer)"]
        C1["ORM<br/>GORM"]
        C2["Database<br/>PostgreSQL/MySQL"]
        C3["Migration<br/>AutoMigrate / golang-migrate"]
    end
```

```

    C4["Query Builder<br/>GORM"]
end

subgraph Output["ชั้นส่งออก (Output Layer)"]
    D1["Logging<br/>zap"]
    D2["Email<br/>gomail + hermes"]
    D3["Response<br/>JSON/XML"]
end

subgraph External["ระบบภายนอก (External Systems)"]
    E1["SMTP Server<br/>Gmail / SendGrid"]
    E2["Database Server"]
    E3["File System"]
end

A1 --> B1
A2 --> B1
A3 --> B1
B1 --> B2
B2 --> B3
B3 --> B4
B4 --> C1
C1 --> C2
C1 --> C3
C1 --> C4
C2 --> E2
C3 --> E2
B2 --> D1
B2 --> D2
D2 --> E1
B2 --> D3
D3 --> A2

subgraph Tools["เครื่องมือพัฒนา (Development Tools)"]
    F1["air<br/>(hot reload)"]
    F2["golangci-lint<br/>(linter)"]
    F3["dlv<br/>(debugger)"]
    F4["go mod<br/>(dependency)"]
end

A1 -.-> F1
A2 -.-> F1
B1 -.-> F2
B2 -.-> F2
C1 -.-> F3
A1 -.-> F4
A2 -.-> F4

```

คำอธิบายภาษาไทย (แบบละเอียด)

ภาคที่ 6 ครอบคลุมบทที่ 43–45 โดยนำเสนอเครื่องมือและไลบรารียอดนิยมที่ใช้ในงานพัฒนา Go ระดับ Production แต่ละตัว มีหน้าที่และตำแหน่งในสถาปัตยกรรมของแอปพลิเคชัน ดังแสดงในแผนภาพ Data Flow ด้านบน

1. chi – เราเตอร์และมิดเดิลแวร์ (บทที่ 43)

บทบาท: รับ HTTP request, ส่งต่อผ่าน middleware chain, จับคู่ route, และส่งไปยัง handler ที่เหมาะสม

โครงสร้างข้อมูล: chi ใช้ `chi.Router` ซึ่งเป็น interface ที่สืบทอดจาก `http.Handler` มีความสามารถในการเพิ่ม middleware, สร้าง route group, และจับคู่ pattern เช่น `/users/{id}`

ตัวอย่างการใช้งานจริง:

```
package main

import (
    "context"
    "encoding/json"
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
    "github.com/go-chi/cors"
    "github.com/go-chi/httprate"
)

type User struct {
    ID    int    `json:"id"`
    Name string `json:"name"`
}

var users = map[int]User{
    1: {ID: 1, Name: "John"},
}

func main() {
    r := chi.NewRouter()

    // Global middleware
    r.Use(middleware.Logger)
    r.Use(middleware.Recoverer)
    r.Use(middleware.RealIP)
    r.Use(middleware.Timeout(60 * time.Second))

    // CORS
    r.Use(cors.Handler(cors.Options{
        AllowedOrigins: []string{"*"},
    }))
}
```



```

    AllowedMethods: []string{"GET", "POST", "PUT", "DELETE", "OPTIONS"},
    AllowedHeaders: []string{"Accept", "Authorization", "Content-Type"},
    AllowCredentials: false,
    MaxAge:          300,
}))

// Rate limiting
r.Use(httprate.LimitByIP(100, 1*time.Minute))

// Public routes
r.Get("/health", func(w http.ResponseWriter, r *http.Request) {
    w.Write([]byte("OK"))
})

// API v1
r.Route("/api/v1", func(r chi.Router) {
    r.Get("/users", listUsers)
    r.Get("/users/{id}", getUser)
    r.Post("/users", createUser)
})

// Start server with graceful shutdown
srv := &http.Server{
    Addr:           ":8080",
    Handler:        r,
    ReadTimeout:   15 * time.Second,
    WriteTimeout:  15 * time.Second,
}

go func() {
    log.Println("Server starting on :8080")
    if err := srv.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
        log.Fatalf("Server failed: %v", err)
    }
}()

quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
log.Println("Shutting down...")

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
srv.Shutdown(ctx)
}

func listUsers(w http.ResponseWriter, r *http.Request) {
    json.NewEncoder(w).Encode(users)
}

func getUser(w http.ResponseWriter, r *http.Request) {
    id := chi.URLParam(r, "id")

```

```

    // ... implementation
}

func createUser(w http.ResponseWriter, r *http.Request) {
    // ... implementation
}

```

2. viper – การจัดการ Configuration (บทที่ 43)

บทบาท: โหลดค่าตั้งค่าจากหลายแหล่ง (ไฟล์, environment variables, flags) และให้ API สำหรับอ่านค่า

โครงสร้างข้อมูล: viper ใช้ `map[string]interface{}` ภายใน เก็บค่าจากทุกแหล่ง โดยมีลำดับความสำคัญ: flags > env > config file > defaults

ตัวอย่างการใช้งานจริง:

```

package config

import (
    "fmt"
    "strings"
    "time"

    "github.com/spf13/viper"
)

type Config struct {
    Server    ServerConfig
    Database  DatabaseConfig
    JWT       JWTConfig
}

type ServerConfig struct {
    Port int    `mapstructure:"port"`
    Mode string `mapstructure:"mode"`
}

type DatabaseConfig struct {
    Host      string `mapstructure:"host"`
    Port      int    `mapstructure:"port"`
    User      string `mapstructure:"user"`
    Password  string `mapstructure:"password"`
    Name      string `mapstructure:"name"`
}

type JWTConfig struct {
    Secret string          `mapstructure:"secret"`
    Expiry time.Duration `mapstructure:"expiry"`
}

```

```

func Load() (*Config, error) {
    viper.SetConfigName("config")
    viper.SetConfigType("yaml")
    viper.AddConfigPath(".")
    viper.AddConfigPath("./config")

    viper.SetAutomaticEnv()
    viper.SetEnvKeyReplacer(strings.NewReplacer(".", "_"))

    // Defaults
    viper.SetDefault("server.port", 8080)
    viper.SetDefault("server.mode", "debug")
    viper.SetDefault("jwt.expiry", "15m")

    if err := viper.ReadInConfig(); err != nil {
        if _, ok := err.(viper.ConfigFileNotFoundError); !ok {
            return nil, fmt.Errorf("read config: %w", err)
        }
    }

    var cfg Config
    if err := viper.Unmarshal(&cfg); err != nil {
        return nil, fmt.Errorf("unmarshal: %w", err)
    }
    return &cfg, nil
}

```

ไฟล์ config.yaml:

```

server:
  port: 9090
database:
  host: localhost
  port: 5432
  user: postgres
  password: ${DB_PASSWORD} # จาก environment
  name: myapp
jwt:
  secret: ${JWT_SECRET}

```

3. cobra – การสร้าง CLI (บทที่ 43)

บทบาท: จัดโครงสร้างคำสั่ง (commands), flags, arguments, และ generate help/documentation

โครงสร้างข้อมูล: แต่ละคำสั่งคือ `cobra.Command` ที่มี fields เช่น Use, Short, Run, Flags

ตัวอย่างการใช้งานจริง:

```

package cmd

import (
    "fmt"
    "os"

    "github.com/spf13/cobra"
)

var rootCmd = &cobra.Command{
    Use:   "myapp",
    Short: "MyApp CLI",
    Long:  `A sample CLI application built with cobra.`,
    Run: func(cmd *cobra.Command, args []string) {
        fmt.Println("Hello from MyApp")
    },
}

var serveCmd = &cobra.Command{
    Use:   "serve",
    Short: "Start the HTTP server",
    RunE: func(cmd *cobra.Command, args []string) error {
        port, _ := cmd.Flags().GetInt("port")
        fmt.Printf("Starting server on :%d\n", port)
        // start server...
        return nil
    },
}

func init() {
    serveCmd.Flags().IntP("port", "p", 8080, "port to listen on")
    rootCmd.AddCommand(serveCmd)
}

func Execute() {
    if err := rootCmd.Execute(); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
}

```

```

// main.go
package main

import "myapp/cmd"

func main() {
    cmd.Execute()
}

```

4. zap – Structured Logging (บทที่ 43)

บทบาท: บันทึก log แบบมีโครงสร้าง (JSON) ด้วยประสิทธิภาพสูง รองรับระดับความรุนแรง และการแยก output

โครงสร้างข้อมูล: zap ใช้ `zap.Logger` ซึ่งมี methods เช่น `Info`, `Error` พร้อม fields ที่กำหนดด้วย `zap.String`, `zap.Int` เป็นต้น

ตัวอย่างการใช้งานจริง:

```
package logger

import (
    "os"

    "go.uber.org/zap"
    "go.uber.org/zap/zapcore"
)

var Log *zap.Logger
var Sugar *zap.SugaredLogger

func Init(level string, output string) error {
    var lvl zapcore.Level
    switch level {
    case "debug":
        lvl = zapcore.DebugLevel
    case "info":
        lvl = zapcore.InfoLevel
    case "warn":
        lvl = zapcore.WarnLevel
    case "error":
        lvl = zapcore.ErrorLevel
    default:
        lvl = zapcore.InfoLevel
    }

    encoderConfig := zapcore.EncoderConfig{
        TimeKey:        "time",
        LevelKey:       "level",
        NameKey:        "logger",
        CallerKey:      "caller",
        MessageKey:     "msg",
        StacktraceKey:  "stacktrace",
        LineEnding:     zapcore.DefaultLineEnding,
        EncodeLevel:    zapcore.LowercaseLevelEncoder,
        EncodeTime:     zapcore.ISO8601TimeEncoder,
        EncodeDuration: zapcore.SecondsDurationEncoder,
        EncodeCaller:   zapcore.ShortCallerEncoder,
    }
}
```

```

var ws zapcore.WriteSyncer
if output == "stdout" {
    ws = zapcore.AddSync(os.Stdout)
} else {
    // file output
    ws = zapcore.AddSync(&lumberjack.Logger{
        Filename:  output,
        MaxSize:   100,
        MaxBackups: 3,
        MaxAge:    28,
        Compress:  true,
    })
}

core := zapcore.NewCore(
    zapcore.NewJSONEncoder(encoderConfig),
    ws,
    lvl,
)
Log = zap.New(core, zap.AddCaller(), zap.AddStacktrace(zapcore.ErrorLevel))
Sugar = Log.Sugar()
return nil
}

```

การใช้งานใน HTTP middleware:

```

func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        rw := &responseWriter{ResponseWriter: w, status: http.StatusOK}
        next.ServeHTTP(rw, r)
        duration := time.Since(start)

        Log.Info("HTTP request",
            zap.String("method", r.Method),
            zap.String("path", r.URL.Path),
            zap.Int("status", rw.status),
            zap.Duration("duration", duration),
            zap.String("ip", r.RemoteAddr),
        )
    })
}

```

5. GORM – ORM ทรงพลัง (บทที่ 44)

บทบาท: แมป struct กับตารางฐานข้อมูล, จัดการ CRUD, transactions, associations, และ migrations

โครงสร้างข้อมูล: ใช้ `gorm.DB` เป็น session สำหรับ query, `gorm.Model` สำหรับ fields พื้นฐาน (ID, CreatedAt, UpdatedAt, DeletedAt)

ตัวอย่างการใช้งานจริง (model และ repository):

```
// models/user.go
package models

import (
    "time"

    "gorm.io/gorm"
)

type User struct {
    ID          uint          `gorm:"primaryKey" json:"id"`
    Username    string         `gorm:"size:50;uniqueIndex;not null"
    json:"username"`
    Email       string         `gorm:"size:100;uniqueIndex;not null"
    json:"email"`
    Password    string         `gorm:"size:255;not null" json:"-`
    CreatedAt   time.Time      `json:"created_at"`
    UpdatedAt   time.Time      `json:"updated_at"`
    DeletedAt   gorm.DeletedAt `gorm:"index" json:"-`
}

// repositories/user_repo.go
package repositories

import (
    "context"
    "errors"

    "gorm.io/gorm"
    "myapp/models"
)

type UserRepository struct {
    db *gorm.DB
}

func NewUserRepository(db *gorm.DB) *UserRepository {
    return &UserRepository{db: db}
}

func (r *UserRepository) Create(ctx context.Context, user *models.User) error {
    return r.db.WithContext(ctx).Create(user).Error
}

func (r *UserRepository) GetByID(ctx context.Context, id uint) (*models.User,
    error) {
```

```

var user models.User
err := r.db.WithContext(ctx).First(&user, id).Error
if errors.Is(err, gorm.ErrRecordNotFound) {
    return nil, nil
}
return &user, err
}

func (r *UserRepository) Update(ctx context.Context, user *models.User) error {
    return r.db.WithContext(ctx).Save(user).Error
}

```

การใช้งาน **transaction** พร้อม **locking**:

```

func (r *OrderRepository) CreateOrder(ctx context.Context, order *models.Order)
error {
    return r.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        // Lock product rows
        for _, item := range order.Items {
            var product models.Product
            if err := tx.Clauses(clause.Locking{Strength:
"UPDATE"}).First(&product, item.ProductID).Error; err != nil {
                return err
            }
            if product.Stock < item.Quantity {
                return errors.New("insufficient stock")
            }
            product.Stock -= item.Quantity
            if err := tx.Save(&product).Error; err != nil {
                return err
            }
        }
        if err := tx.Create(order).Error; err != nil {
            return err
        }
        return nil
    })
}

```

6. golang + hermes – การส่งอีเมล (บทที่ 45)

บทบาท: สร้างและส่งอีเมลแบบ HTML ที่สวยงาม พร้อม fallback text

โครงสร้างข้อมูล:

- `golang.Dialer` สำหรับเชื่อมต่อ SMTP
- `golang.Message` สำหรับสร้างอีเมล (From, To, Subject, Body, Attachments)
- `hermes.Hermes` สำหรับสร้าง HTML email จาก template

ตัวอย่างการใช้งานจริง:

```
package email

import (
    "bytes"
    "fmt"
    "html/template"
    "time"

    "github.com/matcornic/hermes/v2"
    "gopkg.in/gomail.v2"
)

type Config struct {
    SMTPHost      string
    SMTPPort      int
    SMTPUser      string
    SMTPPassword  string
    FromEmail     string
    FromName      string
    AppURL        string
    AppName       string
    AppLogo       string
}

type EmailService struct {
    dialer *gomail.Dialer
    hermes hermes.Hermes
    from   string
}

func New(cfg Config) *EmailService {
    dialer := gomail.NewDialer(cfg.SMTPHost, cfg.SMTPPort, cfg.SMTPUser,
    cfg.SMTPPassword)

    h := hermes.Hermes{
        Product: hermes.Product{
            Name: cfg.AppName,
            Link: cfg.AppURL,
            Logo: cfg.AppLogo,
        },
        Theme: new(hermes.Flat),
    }

    return &EmailService{
        dialer: dialer,
        hermes: h,
        from:   fmt.Sprintf("%s <%s>", cfg.FromName, cfg.FromEmail),
    }
}
```

```

func (s *EmailService) SendWelcome(to, name, verifyURL string) error {
    email := hermes.Email{
        Body: hermes.Body{
            Name: name,
            Intros: []string{
                fmt.Sprintf("Welcome to %s!", s.hermes.Product.Name),
                "We're excited to have you on board.",
            },
            Actions: []hermes.Action{
                {
                    Instructions: "Please verify your email address:",
                    Button: hermes.Button{
                        Color: "#22BC66",
                        Text: "Verify Email",
                        Link: verifyURL,
                    },
                },
            },
            Outros: []string{
                "If you didn't create an account, ignore this email.",
            },
        },
    }

    html, err := s.hermes.GenerateHTML(email)
    if err != nil {
        return err
    }
    text := fmt.Sprintf("Welcome %s! Please verify: %s", name, verifyURL)

    return s.send(to, "Welcome to "+s.hermes.Product.Name, html, text)
}

func (s *EmailService) send(to, subject, htmlBody, textBody string) error {
    m := gmail.NewMessage()
    m.SetHeader("From", s.from)
    m.SetHeader("To", to)
    m.SetHeader("Subject", subject)
    m.SetBody("text/plain", textBody)
    m.AddAlternative("text/html", htmlBody)

    return s.dialer.DialAndSend(m)
}

```

การใช้งานใน handler:

```

func (h *AuthHandler) Register(w http.ResponseWriter, r *http.Request) {
    // ... create user ...
    verifyURL := fmt.Sprintf("%s/verify?token=%s", h.config.AppURL, token)
    if err := h.emailService.SendWelcome(user.Email, user.Name, verifyURL); err
    != nil {

```

```

    log.Printf("Failed to send welcome email: %v", err)
    // ไม่ fail การ register
}
// ...
}

```

เทมเพลตและตัวอย่างโค้ดเพิ่มเติม

เทมเพลตโครงสร้างโปรเจกต์สำหรับภาคนี้

```

myapp/
├── cmd/
│   ├── api/
│   │   └── main.go      # ใช้ chi
│   └── cli/
│       └── main.go      # ใช้ cobra
├── internal/
│   ├── config/
│   │   └── config.go    # viper
│   ├── handlers/
│   │   └── user.go
│   ├── models/
│   │   └── user.go
│   ├── repository/
│   │   └── user.go      # GORM
│   └── logger/
│       └── logger.go    # zap
├── pkg/
│   └── email/
│       └── email.go      # gmail + hermes
├── config.yaml
├── go.mod
└── go.sum

```

Checklist การเลือกใช้เครื่องมือ

เครื่องมือ	เมื่อการใช้	ทางเลือก
chi	ต้องการ router น้ำหนักเบา เร็ว รองรับ middleware มาตรฐาน	gin, echo
viper	ต้องการ config ที่ยืดหยุ่น (file, env, flags)	envconfig, koanf
cobra	สร้าง CLI ที่มีหลายคำสั่ง	urfave/cli
zap	ต้องการ logging ที่เร็วมาก ใช้ structured	slog, logrus
GORM	ต้องการ ORM ที่ครบฟีเจอร์, associations	sqlx, sqlc
gmail + hermes	ต้องการส่งอีเมล HTML สวยงาม	sendgrid, mailgun

สรุป

ภาคที่ 6 นำเสนอชุดเครื่องมือที่จำเป็นสำหรับการพัฒนาแอปพลิเคชัน Go ในระดับ Production:

- **chi** จัดการ HTTP routing และ middleware อย่างมีประสิทธิภาพ
- **viper** จัดการ configuration จากหลายแหล่ง
- **cobra** สร้าง CLI ที่มีโครงสร้างชัดเจน
- **zap** บันทึก structured logs ด้วยความเร็วสูง
- **GORM** จัดการฐานข้อมูลผ่าน ORM ที่ทรงพลัง
- **gomail + hermes** สร้างและส่งอีเมล HTML ที่สวยงาม

เครื่องมือเหล่านี้ทำงานสอดคล้องกันตามแผนภาพ Data Flow ที่แสดงไว้ ช่วยให้การพัฒนาระบบ มีประสิทธิภาพ และบำรุงรักษาง่าย

บทที่ 43: chi, viper, cobra, zap และเครื่องมือสำคัญ

43.1 chi – เราเตอร์และมิดเดิลแวร์

chi เป็น lightweight router ที่มีประสิทธิภาพสูง รองรับ middleware และเข้ากันได้กับ net/http มาตรฐาน

การติดตั้ง

```
go get github.com/go-chi/chi/v5
```

ตัวอย่างพื้นฐาน

```
package main

import (
    "net/http"
    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
)

func main() {
    r := chi.NewRouter()

    // ใช้ middleware พื้นฐาน
    r.Use(middleware.Logger)
    r.Use(middleware.Recoverer)

    // กำหนด routes
    r.Get("/", func(w http.ResponseWriter, r *http.Request) {
        w.Write([]byte("Hello World"))
    })

    // route พร้อมพารามิเตอร์
```

```

r.Get("/users/{id}", getUser)

// group routes
r.Route("/api", func(r chi.Router) {
    r.Get("/users", listUsers)
    r.Post("/users", createUser)

    // sub-router พร้อม middleware เฉพาะ
    r.Route("/admin", func(r chi.Router) {
        r.Use(adminOnly)
        r.Get("/dashboard", adminDashboard)
    })
})

http.ListenAndServe(":8080", r)
}

func getUser(w http.ResponseWriter, r *http.Request) {
    id := chi.URLParam(r, "id")
    w.Write([]byte("User ID: " + id))
}

```

มิดเดิลแวร์ที่ใช้บ่อย

- `middleware.Logger` – บันทึก request
- `middleware.Recoverer` – จับ panic
- `middleware.Timeout` – กำหนด timeout
- `middleware.Compress` – บีบอัด response
- `middleware.RealIP` – ดึง IP จริงจาก proxy

43.2 viper – การจัดการ configuration

`viper` รองรับหลายรูปแบบ (JSON, YAML, ENV, flags) และสามารถโหลดจากไฟล์, environment variables, หรือ remote system

การติดตั้ง

```
go get github.com/spf13/viper
```

ตัวอย่างการใช้งาน

```

package config

import (
    "log"
    "github.com/spf13/viper"
)

```

```

type Config struct {
    Server    ServerConfig
    Database  DatabaseConfig
    Redis     RedisConfig
    JWT       JWTConfig
}

type ServerConfig struct {
    Port int
    Mode string
}

type DatabaseConfig struct {
    Host      string
    Port      int
    User      string
    Password  string
    Name      string
}

func LoadConfig() (*Config, error) {
    viper.SetConfigName("config")           // ชื่อไฟล์ (ไม่รวมนามสกุล)
    viper.SetConfigType("yaml")             // yaml, json, toml, etc.
    viper.AddConfigPath(".")                // path ที่ค้นหา
    viper.AddConfigPath("/etc/app/")
    viper.SetConfigFile("")                 // อ่านจาก environment variables

    // กำหนดค่า default
    viper.SetDefault("server.port", 8080)
    viper.SetDefault("server.mode", "debug")

    // อ่านไฟล์
    if err := viper.ReadInConfig(); err != nil {
        log.Printf("Config file not found: %v", err)
    }

    var cfg Config
    if err := viper.Unmarshal(&cfg); err != nil {
        return nil, err
    }

    return &cfg, nil
}

```

ไฟล์ config.yaml ตัวอย่าง

```
server:
  port: 8080
  mode: release
```

```
database:
  host: localhost
  port: 3306
  user: root
  password: secret
  name: mydb

redis:
  addr: localhost:6379
  password: ""
  db: 0

jwt:
  secret: "your-secret-key"
  access_expiry: 15m
  refresh_expiry: 7d
```

43.3 cobra – การสร้าง CLI

[cobra](#) เป็นไลบรารีสำหรับสร้าง command-line application รองรับ commands, flags, และ subcommands

การติดตั้ง

```
go get -u github.com/spf13/cobra/cobra
```

การเริ่มต้นโปรเจกต์

```
cobra init --pkg-name mycli
```

ตัวอย่างการเพิ่ม command

```
// cmd/root.go
package cmd

import (
    "fmt"
    "os"
    "github.com/spf13/cobra"
)

var rootCmd = &cobra.Command{
    Use:   "mycli",
    Short: "My CLI application",
    Long:  "A sample CLI built with cobra",
    Run:   func(cmd *cobra.Command, args []string) {
```

```

        fmt.Println("Hello from CLI")
    },
}

// cmd/serve.go
var serveCmd = &cobra.Command{
    Use:   "serve",
    Short: "Start the server",
    Run: func(cmd *cobra.Command, args []string) {
        port, _ := cmd.Flags().GetInt("port")
        fmt.Printf("Starting server on port %d\n", port)
    },
}

func init() {
    serveCmd.Flags().IntP("port", "p", 8080, "port to listen on")
    rootCmd.AddCommand(serveCmd)
}

func Execute() {
    if err := rootCmd.Execute(); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
}

```

การใช้งาน

```
go run main.go serve --port 3000
```

43.4 validator – การตรวจสอบข้อมูล

[validator](#) ใช้ struct tags ในการกำหนด validation rules

การติดตั้ง

```
go get github.com/go-playground/validator/v10
```

ตัวอย่าง

```

package main

import (
    "fmt"
    "github.com/go-playground/validator/v10"

```



```

)

type RegisterRequest struct {
    Name      string `validate:"required,min=3,max=50"`
    Email      string `validate:"required,email"`
    Password   string `validate:"required,min=8"`
    Age        int    `validate:"gte=18,lte=99"`
}

func main() {
    validate := validator.New()

    req := RegisterRequest{
        Name:      "Jo",
        Email:      "invalid",
        Password:   "short",
        Age:        16,
    }

    err := validate.Struct(req)
    if err != nil {
        for _, err := range err.(validator.ValidationErrors) {
            fmt.Printf("Field %s failed on tag %s\n", err.Field(), err.Tag())
        }
    }
}

```

Tags ที่ใช้บ่อย

- **required** – ต้องมีค่า
- **email** – รูปแบบ email
- **min, max** – ขนาดต่ำสุด/สูงสุด
- **gte, lte** – มากกว่าหรือเท่ากับ, น้อยกว่าหรือเท่ากับ
- **oneof=red blue** – ต้องเป็นหนึ่งในค่าที่กำหนด
- **uuid** – ต้องเป็น UUID
- **url** – ต้องเป็น URL

43.5 zap – structured logging

zap เป็น logger ที่มีความเร็วสูงและรองรับ structured logging

การติดตั้ง

```
go get go.uber.org/zap
```

ตัวอย่าง

```

package logger

import (
    "go.uber.org/zap"
)

var Log *zap.Logger

func InitLogger(mode string) {
    var err error
    if mode == "production" {
        Log, err = zap.NewProduction()
    } else {
        Log, err = zap.NewDevelopment()
    }
    if err != nil {
        panic(err)
    }
    defer Log.Sync()
}

func main() {
    InitLogger("development")
    defer Log.Sync()

    Log.Info("User logged in",
        zap.String("user", "john"),
        zap.Int("id", 123),
        zap.Duration("duration", time.Second*2),
    )

    Log.Error("Database error",
        zap.Error(err),
        zap.String("query", "SELECT * FROM users"),
    )
}

```

43.6 air – hot-reload

air ใช้สำหรับ reload อัตโนมัติเมื่อไฟล์เปลี่ยนแปลง เหมาะสำหรับการพัฒนา

การติดตั้ง

```
go install github.com/cosmtrek/air@latest
```

การใช้งาน

สร้างไฟล์ `.air.toml` ใน root ของโปรเจกต์ (หรือใช้ default) แล้วรัน:

```
air
```

ตัวอย่าง .air.toml

```
root = "."
tmp_dir = "tmp"

[build]
  cmd = "go build -o ./tmp/main ."
  bin = "./tmp/main"
  include_ext = ["go", "tpl", "tmpl", "html"]
  exclude_dir = ["assets", "tmp", "vendor"]
  delay = 1000
  stop_on_error = true
  send_interrupt = false
  kill_delay = 500
```

บทที่ 44: GORM – ORM ทรงพลังสำหรับ Go

44.1 บทนำ

GORM (Go Object Relational Mapping) เป็น ORM ที่ได้รับความนิยมสูงสุดในภาษา Go ช่วยลดความซับซ้อนในการจัดการฐานข้อมูล ด้วยการแมป struct กับตารางโดยอัตโนมัติ พร้อมฟังก์ชัน CRUD ที่ใช้งานง่าย การจัดการ transaction แบบมีระบบ และฟีเจอร์ขั้นสูงอย่าง query cache, queue processor

44.2 การติดตั้งและการตั้งค่าพื้นฐาน

```
go get -u gorm.io/gorm
go get -u gorm.io/driver/postgres // หรือ driver อื่นตามที่ใช้
```

ตัวอย่างการเชื่อมต่อ PostgreSQL

```
package main

import (
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
    "log"
)

func main() {
    dsn := "host=localhost user=gorm password=gorm dbname=gorm port=5432
    sslmode=disable TimeZone=Asia/Bangkok"
```

```

db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})
if err != nil {
    log.Fatal("failed to connect database")
}

// ใช้ db ต่อไป
}

```

44.3 กำหนด Model (Entity)

```

type User struct {
    ID          uint          `gorm:"primaryKey"`
    Name        string        `gorm:"size:100;not null"`
    Email       string        `gorm:"uniqueIndex;size:100;not null"`
    Age         int           `gorm:"default:0"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
    DeletedAt   gorm.DeletedAt `gorm:"index"`
}

```

44.4 CRUD Operations

Create

```

user := User{Name: "สมชาย", Email: "somchai@example.com", Age: 30}
result := db.Create(&user) // สร้าง record ใหม่
fmt.Println(user.ID)      // คัดค่า ID ที่ถูกสร้าง
fmt.Println(result.Error)  // error ถ้ามี

```

Read

```

// ดึง record แรกที่ตรงเงื่อนไข
var user User
db.First(&user, 1) // by primary key
db.First(&user, "email = ?", "somchai@example.com")

// ดึงทั้งหมด
var users []User
db.Find(&users)

// พร้อมเงื่อนไข
db.Where("age > ?", 20).Find(&users)
db.Where(&User{Name: "สมชาย"}).Find(&users)

```

Update

```
// อัปเดต single column
db.Model(&user).Update("Name", "สมชาย ใหม่")

// อัปเดตหลาย columns ด้วย struct (ไม่สนใจ zero values)
db.Model(&user).Updates(User{Name: "สมชาย ใหม่", Age: 31})

// อัปเดตหลาย columns ด้วย map
db.Model(&user).Updates(map[string]interface{}{"name": "สมชาย ใหม่", "age": 31})
```

Delete

```
// soft delete (ถ้ามี gorm.DeletedAt)
db.Delete(&user, 1)

// hard delete
db.Unscoped().Delete(&user, 1)
```

44.5 SessionFactory

Session factory เป็นรูปแบบการสร้าง `*gorm.DB` ที่มี configuration คงที่ (เช่น logging, connection pool) และสามารถสร้าง session ใหม่สำหรับแต่ละ request หรือ transaction

ตัวอย่าง session factory

```
package db

import (
    "gorm.io/gorm"
    "gorm.io/gorm/logger"
    "log"
    "time"
)

type SessionFactory struct {
    db *gorm.DB
}

func NewSessionFactory(dsn string) (*SessionFactory, error) {
    // ตั้งค่า logger และ connection pool
    newLogger := logger.New(
        log.New(os.Stdout, "\r\n", log.LstdFlags),
        logger.Config{
            SlowThreshold: time.Second,
            LogLevel:      logger.Info,
            Colorful:      true,
        },
    )
```

```

db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{
    Logger: newLogger,
    NowFunc: func() time.Time { return time.Now().UTC() },
})
if err != nil {
    return nil, err
}

// ตั้งค่า connection pool
sqlDB, _ := db.DB()
sqlDB.SetMaxIdleConns(10)
sqlDB.SetMaxOpenConns(100)
sqlDB.SetConnMaxLifetime(time.Hour)

return &SessionFactory{db: db}, nil
}

// NewSession คืนค่า session ใหม่สำหรับการทำ transaction หรือ query แบบแยก context
func (sf *SessionFactory) NewSession() *gorm.DB {
    return sf.db.Session(&gorm.Session{})
}

```

การใช้งาน

```

factory, _ := db.NewSessionFactory(dsn)

// สร้าง session ใหม่สำหรับ request นี้
session := factory.NewSession()
var user User
session.First(&user, 1)

// เมื่อต้องการ transaction
session.Transaction(func(tx *gorm.DB) error {
    // ใช้ tx แทน session
    return nil
})

```

44.6 การใช้ GORM Transaction เพื่อ Rollback

GORM มีฟังก์ชัน `db.Transaction` ที่ช่วยให้เราสามารถรวมหลายคำสั่ง SQL ไว้ใน transaction เดียวกันได้ โดยหากฟังก์ชันที่ส่งเข้าไปคืนค่า `error` GORM จะทำการ rollback โดยอัตโนมัติ ถ้าคืน `nil` จะ commit

ตัวอย่างการใช้งาน Transaction

```

func PlaceOrder(db *gorm.DB, userID uint, items []CartItem) error {
    // เริ่ม transaction
    return db.Transaction(func(tx *gorm.DB) error {

```

```

// 1. คำนวณราคารวม และตรวจสอบสต็อกพร้อม lock
var total float64
for _, item := range items {
    var stock Stock
    // Lock แถว stock เพื่อป้องกัน race condition
    if err := tx.Clauses(clause.Locking{Strength: "UPDATE"}).
        Where("product_id = ?", item.ProductID).
        First(&stock).Error; err != nil {
        return err // สินค้าไม่มีในระบบ
    }
    if stock.Quantity < item.Quantity {
        return errors.New("สินค้าไม่พอ")
    }
    // หักสต็อก (จะบันทึกภายหลัง)
    stock.Quantity -= item.Quantity
    if err := tx.Save(&stock).Error; err != nil {
        return err
    }
    // คำนวณราคารวม
    total += getPrice(item.ProductID) * float64(item.Quantity)
}

// 2. สร้าง order
order := Order{UserID: userID, Total: total}
if err := tx.Create(&order).Error; err != nil {
    return err
}

// 3. สร้าง receipt
receipt := Receipt{OrderID: order.ID, Amount: total}
if err := tx.Create(&receipt).Error; err != nil {
    return err
}

// ทุกอย่างสำเร็จ -> commit อัตโนมัติ
return nil
})
}

```

44.7 Query Cache

GORM เองไม่มี query cache ในตัว แต่สามารถใช้ plugin หรือจัดการเองผ่าน Redis หรือ memory cache

ตัวอย่างการใช้ Redis cache กับ GORM

```

import (
    "context"
    "encoding/json"
    "github.com/go-redis/redis/v8"
    "gorm.io/gorm"

```

```

    "time"
)

type CachedDB struct {
    db      *gorm.DB
    cache *redis.Client
}

func (c *CachedDB) FirstWithCache(dest interface{}, conds ...interface{}) error {
    // สร้าง cache key จากเงื่อนไข
    key := fmt.Sprintf("query:%v", conds)

    // พยายามอ่านจาก cache
    val, err := c.cache.Get(context.Background(), key).Result()
    if err == nil {
        // พบใน cache
        return json.Unmarshal([]byte(val), dest)
    }

    // ไม่พบใน cache, query ฐานข้อมูล
    if err := c.db.First(dest, conds...).Error; err != nil {
        return err
    }

    // บันทึกผลลัพธ์ลง cache (serialize)
    data, _ := json.Marshal(dest)
    c.cache.Set(context.Background(), key, data, 5*time.Minute)
    return nil
}

```

บทที่ 45: การส่งอีเมลด้วย gomail และ hermes

45.1 gomail – การส่งอีเมล

[gomail](#) เป็นไลบรารีที่ใช้งานง่ายสำหรับ SMTP

การติดตั้ง

```
go get gopkg.in/gomail.v2
```

ตัวอย่าง

```

package mail

import (
    "gopkg.in/gomail.v2"

```



```

)

type Mailer struct {
    dialer *gomail.Dialer
    from   string
}

func NewMailer(host string, port int, user, pass, from string) *Mailer {
    return &Mailer{
        dialer: gomail.NewDialer(host, port, user, pass),
        from:   from,
    }
}

func (m *Mailer) Send(to, subject, body string) error {
    msg := gomail.NewMessage()
    msg.SetHeader("From", m.from)
    msg.SetHeader("To", to)
    msg.SetHeader("Subject", subject)
    msg.SetBody("text/html", body)

    return m.dialer.DialAndSend(msg)
}

```

45.2 hermes – สร้าง HTML email ที่สวยงาม

[hermes](#) ใช้สร้าง email template แบบ responsive

การติดตั้ง

```
go get github.com/matcornic/hermes/v2
```

ตัวอย่าง

```

package email

import (
    "github.com/matcornic/hermes/v2"
)

type EmailGenerator struct {
    h hermes.Hermes
}

func NewEmailGenerator(appURL, appName string) *EmailGenerator {
    h := hermes.Hermes{
        Product: hermes.Product{
            Name: appName,

```

```

        Link: appURL,
        Logo: appURL + "/logo.png",
    },
}
return &EmailGenerator{h: h}
}

func (g *EmailGenerator) WelcomeEmail(name, verifyURL string) (string, error) {
    email := hermes.Email{
        Body: hermes.Body{
            Name: name,
            Intros: []string{
                "Welcome to our platform!",
            },
            Actions: []hermes.Action{
                {
                    Instructions: "Please click below to verify your email
address:",
                    Button: hermes.Button{
                        Color: "#22BC66",
                        Text: "Verify Email",
                        Link: verifyURL,
                    },
                },
            },
            Outros: []string{
                "If you didn't sign up, you can ignore this email.",
            },
        },
    }

    return g.h.GenerateHTML(email)
}

```

ภาคที่ 7: การออกแบบสถาปัตยกรรมและ Workflow

(บทที่ 46–48)

แผนภาพ Data Flow (Flowchart TB)

```

flowchart TB
    subgraph Delivery["Delivery Layer (Interfaces)"]
        D1["HTTP Handlers<br>chi/gin handlers"]
        D2["Middleware<br>Auth, Logger, CORS"]
        D3["DTOs & Validation<br>request/response structs"]
    end

    subgraph Usecase["Usecase Layer (Business Logic)"]

```

```

    U1["Use Case Implementations<br/>RegisterUser, CreateOrder"]
    U2["Domain Services<br/>TransferService"]
    U3["Repository Interfaces<br/>UserRepository, OrderRepository"]
end

subgraph Repository["Repository Layer (Data Access)"]
    R1["Repository Implementations<br/>PostgresUserRepo, RedisCache"]
    R2["External API Clients<br/>PaymentGateway, SMS"]
    R3["Cache Layer<br/>Redis"]
end

subgraph Infrastructure["Infrastructure Layer"]
    I1["Database<br/>PostgreSQL, MySQL"]
    I2["Message Broker<br/>RabbitMQ, Kafka"]
    I3["External Systems<br/>SMS, Email"]
end

subgraph Models["Models (Domain Entities)"]
    M1["Entities<br/>User, Order, Product"]
    M2["Value Objects<br/>Email, Money, Address"]
    M3["Domain Events<br/>UserRegistered, OrderPlaced"]
end

Delivery --> Usecase
Usecase --> Repository
Repository --> Infrastructure
Models -.-> Delivery
Models -.-> Usecase
Models -.-> Repository

subgraph Blueprint["Production Blueprint"]
    B1["cmd/<br/>api/, worker/"]
    B2["internal/<br/>core/, application/, infrastructure/"]
    B3["pkg/<br/>shared utils"]
    B4["configs/, migrations/, deploy/"]
end

subgraph Workflow["Development Workflow"]
    W1["Task List Template"]
    W2["Checklist Template"]
    W3["Feature Development Flow<br/>Analyze → Design → Implement → Test →
Deploy"]
end

Blueprint --> Delivery
Blueprint --> Usecase
Blueprint --> Repository
Workflow --> Blueprint

```

คำอธิบายภาษาไทย (แบบละเอียด)

ภาคที่ 7 ประกอบด้วย 3 บทหลัก:

- บทที่ 46: **Clean Architecture** และโครงสร้างโปรเจกต์
- บทที่ 47: **Blueprint** สำหรับโปรเจกต์ Go ระดับ **Production**
- บทที่ 48: การออกแบบ **Workflow** และ **Task Management**

ทั้งสามบทเชื่อมโยงกันเพื่อให้ผู้อ่านสามารถออกแบบและพัฒนาแอปพลิเคชันที่มีสถาปัตยกรรมแข็งแกร่ง พร้อมขยาย และมีกระบวนการพัฒนาที่เป็นระบบ

บทที่ 46: Clean Architecture และโครงสร้างโปรเจกต์

หลักการ Clean Architecture

Clean Architecture แบ่งระบบออกเป็นชั้น ๆ ที่มีความรับผิดชอบชัดเจน โดยมีกฎสำคัญคือ **Dependency Rule**: dependencies ต้องชี้เข้าหาศูนย์กลาง (จากภายนอกเข้าสู่ภายใน) เท่านั้น ชั้นในสุดคือ **Entities** (Domain Models) ซึ่งไม่ควรขึ้นกับอะไรภายนอก

ชั้นต่าง ๆ (จากนอกสุดเข้าสู่ในสุด):

1. Delivery (Interface Adapters)

- รับคำขอจากภายนอก (HTTP, gRPC, CLI)
- แปลงข้อมูลเป็น DTO
- เรียก Usecase
- ส่ง Response กลับ

2. Usecase (Application Business Rules)

- จัดลำดับการทำงาน (orchestration)
- ใช้ Repository Interfaces
- ไม่ขึ้นกับรายละเอียดของฐานข้อมูลหรือเฟรมเวิร์ก

3. Repository (Interface)

- กำหนด contract สำหรับการเข้าถึงข้อมูล
- ถูก Implement ในชั้น Infrastructure

4. Entities (Domain Models)

- โมเดลหลักทางธุรกิจ (User, Order)
- Value Objects (Email, Money)
- Domain Events

5. Infrastructure

- การ Implement จริงของ Repository (GORM, SQL)
- การเชื่อมต่อกับ External Services
- Cache, Queue, etc.

ตัวอย่างโครงสร้างโปรเจกต์ตาม Clean Architecture

```

myapp/
├── cmd/
│   └── api/
│       └── main.go                # entry point
├── internal/
│   ├── config/                  # โหลด configuration
│   ├── domain/                  # Entities, Value Objects, Events
│   │   ├── user/
│   │   │   ├── entity.go
│   │   │   ├── value_objects.go
│   │   │   ├── events.go
│   │   │   └── repository.go    # interface
│   │   └── order/
│   │       └── ...
│   ├── application/             # Usecases
│   │   ├── user/
│   │   │   ├── register.go
│   │   │   ├── login.go
│   │   │   └── dto.go
│   │   └── order/
│   │       └── ...
│   ├── infrastructure/          # Implementations
│   │   ├── persistence/
│   │   │   ├── gorm/
│   │   │   │   ├── user_repo.go
│   │   │   │   └── models.go
│   │   │   └── redis/
│   │   ├── queue/
│   │   └── email/
│   └── interfaces/              # Delivery
│       ├── http/
│       │   ├── handlers/
│       │   ├── middleware/
│       │   └── routes.go
│       └── cli/
├── pkg/                          # reusable packages
├── configs/
├── migrations/
└── go.mod

```

ตัวอย่างโค้ดของแต่ละชั้น

1. Domain Layer (Entity)

```

// internal/domain/user/entity.go
package user

import (
    "errors"

```

```

    "time"
)

type User struct {
    id          uint
    email       Email
    password    Password
    name        string
    isActive    bool
    createdAt   time.Time
    updatedAt   time.Time
    events      []DomainEvent
}

func NewUser(email, password, name string) (*User, error) {
    emailVO, err := NewEmail(email)
    if err != nil {
        return nil, err
    }
    passwordVO, err := NewPassword(password)
    if err != nil {
        return nil, err
    }
    return &User{
        id:          generateID(),
        email:       *emailVO,
        password:    *passwordVO,
        name:        name,
        isActive:    true,
        createdAt:   time.Now(),
        updatedAt:   time.Now(),
        events:      []DomainEvent{},
    }, nil
}

// Getters
func (u *User) ID() uint      { return u.id }
func (u *User) Email() string { return u.email.String() }
func (u *User) Name() string  { return u.name }
func (u *User) IsActive() bool { return u.isActive }

// Business logic
func (u *User) Deactivate() error {
    if !u.isActive {
        return errors.New("user already inactive")
    }
    u.isActive = false
    u.updatedAt = time.Now()
    u.addDomainEvent(NewUserDeactivatedEvent(u.id))
    return nil
}

func (u *User) addDomainEvent(event DomainEvent) {

```

```

    u.events = append(u.events, event)
}

func (u *User) Events() []DomainEvent { return u.events }
func (u *User) ClearEvents()           { u.events = nil }

```

2. Repository Interface

```

// internal/domain/user/repository.go
package user

import "context"

type Repository interface {
    Save(ctx context.Context, user *User) error
    FindByID(ctx context.Context, id uint) (*User, error)
    FindByEmail(ctx context.Context, email string) (*User, error)
    Update(ctx context.Context, user *User) error
    Delete(ctx context.Context, id uint) error
}

```

3. Application Layer (Usecase)

```

// internal/application/user/register.go
package user

import (
    "context"
    "fmt"

    "myapp/internal/domain/user"
)

type RegisterUseCase struct {
    userRepo user.Repository
    eventBus EventBus
}

type RegisterInput struct {
    Email    string `json:"email"`
    Password string `json:"password"`
    Name     string `json:"name"`
}

type RegisterOutput struct {
    ID      uint   `json:"id"`
    Email   string `json:"email"`
}

```

```

    Name string `json:"name"`
}

func NewRegisterUseCase(repo user.Repository, bus EventBus) *RegisterUseCase {
    return &RegisterUseCase{
        userRepo: repo,
        eventBus: bus,
    }
}

func (uc *RegisterUseCase) Execute(ctx context.Context, input RegisterInput)
(*RegisterOutput, error) {
    // ตรวจสอบ email ซ้ำ
    existing, _ := uc.userRepo.FindByEmail(ctx, input.Email)
    if existing != nil {
        return nil, fmt.Errorf("email already registered")
    }

    // สร้าง User entity
    u, err := user.NewUser(input.Email, input.Password, input.Name)
    if err != nil {
        return nil, err
    }

    // บันทึก
    if err := uc.userRepo.Save(ctx, u); err != nil {
        return nil, err
    }

    // Dispatch domain events
    for _, event := range u.Events() {
        uc.eventBus.Publish(event)
    }

    return &RegisterOutput{
        ID:      u.ID(),
        Email:   u.Email(),
        Name:    u.Name(),
    }, nil
}

```

4. Infrastructure Layer (Repository Implementation with GORM)

```

// internal/infrastructure/persistence/gorm/user_repo.go
package gormrepo

import (
    "context"

    "gorm.io/gorm"

```



```

    "myapp/internal/domain/user"
)

type userRepository struct {
    db *gorm.DB
}

func NewUserRepository(db *gorm.DB) user.Repository {
    return &userRepository{db: db}
}

func (r *userRepository) Save(ctx context.Context, u *user.User) error {
    model := &UserModel{
        ID:      u.ID(),
        Email:   u.Email(),
        Name:    u.Name(),
        IsActive: u.IsActive(),
        CreatedAt: u.CreatedAt,
        UpdatedAt: u.UpdatedAt,
    }
    return r.db.WithContext(ctx).Create(model).Error
}

// ... other methods

```

5. Delivery Layer (HTTP Handler)

```

// internal/interfaces/http/handlers/user.go
package handlers

import (
    "encoding/json"
    "net/http"

    "myapp/internal/application/user"
)

type UserHandler struct {
    registerUC *user.RegisterUseCase
}

func NewUserHandler(registerUC *user.RegisterUseCase) *UserHandler {
    return &UserHandler{registerUC: registerUC}
}

func (h *UserHandler) Register(w http.ResponseWriter, r *http.Request) {
    var input user.RegisterInput
    if err := json.NewDecoder(r.Body).Decode(&input); err != nil {
        http.Error(w, "Invalid request", http.StatusBadRequest)
        return
    }
}

```

```

}

output, err := h.registerUC.Execute(r.Context(), input)
if err != nil {
    http.Error(w, err.Error(), http.StatusInternalServerError)
    return
}

w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusCreated)
json.NewEncoder(w).Encode(output)
}

```

บทที่ 47: Blueprint สำหรับโปรเจกต์ Go ระดับ Production

โครงสร้างโฟลเดอร์หลัก (Blueprint)

```

project-root/
├── cmd/                                # Executable entrypoints
│   ├── api/                           # REST API server
│   │   └── main.go
│   └── worker/                         # Background worker
│       └── main.go
├── internal/                           # Private code
│   ├── config/                         # Configuration loading (viper)
│   ├── domain/                         # Domain models (DDD)
│   │   ├── user/
│   │   │   ├── entity.go
│   │   │   ├── value_objects.go
│   │   │   ├── repository.go        # interface
│   │   │   └── events.go
│   │   └── order/
│   ├── application/                   # Usecases
│   │   ├── user/
│   │   │   ├── register.go
│   │   │   ├── login.go
│   │   │   └── dto.go
│   │   └── order/
│   ├── infrastructure/                # Implementations
│   │   ├── persistence/
│   │   │   ├── gorm/
│   │   │   │   ├── user_repo.go
│   │   │   │   └── models.go
│   │   │   └── redis/
│   │   ├── queue/                     # RabbitMQ, Kafka
│   │   ├── cache/
│   │   └── email/
│   └── interfaces/                    # Delivery
│       └── http/

```

```

├── handlers/
├── middleware/
├── routes.go
└── cli/
├── pkg/                                # Reusable packages
├──   ├── logger/                       # zap wrapper
├──   ├── jwt/
├──   ├── validator/
├──   └── errors/
├── api/                                # API definitions (OpenAPI, Protobuf)
├──   └── openapi.yaml
├── configs/                            # Configuration files
├──   ├── config.yaml
├──   └── config.example.yaml
├── migrations/                         # Database migrations
├──   └── 001_create_users_table.sql
├── scripts/                           # Build, deployment scripts
├── test/                              # Integration tests
├──   └── integration/
├── deploy/                            # Docker, k8s manifests
├──   ├── docker-compose.yaml
├──   └── k8s/
├── go.mod
└── go.sum

```

คำอธิบายเพิ่มเติม

- **cmd/**: มีไฟล์ `main.go` สำหรับแต่ละ executable (api, worker, cli) โดยปกติจะมีแค่ 2-3 ตัว
- **internal/**: โค้ดที่ไม่อยากให้ package อื่น import ได้ (Go compiler จะบังคับ)
 - **domain/**: แบ่งตาม bounded context (user, order) ภายในประกอบด้วย entity, value objects, repository interface, domain events
 - **application/**: use cases แต่ละตัว (RegisterUser, CreateOrder) และ DTOs
 - **infrastructure/**: การ implement repository (GORM, sqlx), cache (Redis), queue (RabbitMQ), email, etc.
 - **interfaces/**: HTTP handlers, middleware, routes; CLI commands
- **pkg/**: โค้ดที่ reusable ระหว่างโปรเจกต์ต่าง ๆ (เช่น logger, jwt, validator)
- **api/**: OpenAPI/Swagger specs หรือ protobuf
- **configs/**: ไฟล์คอนฟิก (YAML) และตัวอย่าง
- **migrations/**: SQL scripts สำหรับ migration (อาจใช้ `golang-migrate` หรือ `atlas`)
- **test/**: integration tests ที่ใช้ testcontainers หรือ environment จริง
- **deploy/**: docker-compose และ Kubernetes YAMLs

บทที่ 48: การออกแบบ Workflow และ Task Management

1. Workflow การพัฒนา Feature ใหม่

flowchart LR

A[Analyze
Requirement] --> B[Design
Domain Models]

B --> C[Implement Domain
Entities, Interfaces]

C --> D[Implement Infrastructure
Repository, Cache]

D --> E[Implement Usecase
Business Logic]

E --> F[Implement Handler
HTTP/CLI]

F --> G[Write Tests
Unit, Integration]

G --> H[Document & Review]

H --> I[Deploy]

2. Task List Template

Phase 1: Analysis & Design

- ☐ เข้าใจ requirement, user stories
- ☐ ระบุ bounded context และ aggregate
- ☐ ออกแบบ entity, value objects
- ☐ กำหนด repository interface methods
- ☐ ระบุ domain events
- ☐ ออกแบบ DTOs (request/response)

Phase 2: Implementation

- ☐ สร้าง entity struct และ business methods
- ☐ สร้าง repository interface
- ☐ สร้าง usecase struct และ method
- ☐ สร้าง HTTP handler
- ☐ ลงทะเบียน routes
- ☐ เขียน unit tests (domain, usecase with mocks)
- ☐ เขียน integration tests (repository with testcontainers)

Phase 3: Infrastructure

- ☐ Implement repository (GORM)
- ☐ สร้าง migration scripts
- ☐ ตั้งค่า connection pool
- ☐ Implement cache (ถ้าจำเป็น)
- ☐ ทดสอบ repository

Phase 4: Documentation & Review

- ☐ อัปเดต API docs (Swagger)
- ☐ รัน linter (`golangci-lint run`)
- ☐ รัน test coverage (`go test -cover`), ตั้งเป้า >80%
- ☐ รัน race detector (`go test -race`)
- ☐ Code review

Phase 5: Deployment

- ☐ สร้าง Docker image
- ☐ อัปเดต docker-compose (ถ้ามี)
- ☐ Deploy to staging
- ☐ ทดสอบ end-to-end
- ☐ Deploy to production

3. Checklist Template

Code Quality

- ☐ All exported functions have comments
- ☐ No unused imports/variables (`go vet`)
- ☐ Code formatted with `go fmt`
- ☐ Error handling explicit (no ignored errors)
- ☐ No panic in library code (only main/init)
- ☐ Context passed as first parameter
- ☐ Interfaces are small and focused
- ☐ No global state except configuration

Security

- ☐ Passwords hashed with bcrypt
- ☐ JWT secret loaded from environment
- ☐ CORS properly configured
- ☐ Input validation on all endpoints
- ☐ SQL injection prevented (parameterized queries)
- ☐ No sensitive data in logs
- ☐ HTTPS enforced in production
- ☐ Rate limiting on auth endpoints

Performance

- ☐ Database indexes on frequently queried columns
- ☐ Use of caching (Redis) for hot data
- ☐ Connection pools configured for DB and Redis
- ☐ Avoid N+1 queries (use Preload in GORM)
- ☐ Benchmarks for critical paths

Testing

- ☐ Unit tests cover business logic (usecase)
- ☐ Repository tests with testcontainers
- ☐ HTTP handler tests with httptest
- ☐ Mock external dependencies
- ☐ Race condition tests with `-race`
- ☐ Coverage >80%

Deployment

- ☐ Configurable via environment variables
- ☐ Graceful shutdown implemented
- ☐ Health check endpoints (/health, /ready)
- ☐ Logging to stdout (for container)
- ☐ Docker image built with non-root user
- ☐ Secrets not baked into image
- ☐ Database migration runs automatically on startup
- ☐ Readiness and liveness probes configured
- ☐ Monitoring metrics exposed (Prometheus)

ตัวอย่างการนำไปใช้จริง: การเพิ่มโมดูล "Product"

สมมติเราต้องเพิ่มฟีเจอร์จัดการสินค้า (Product) ในแอปพลิเคชัน e-commerce

ขั้นตอนตาม Workflow

1. **Analyze:** Product มี fields: id, name, description, price, stock, status (active/inactive), created_at, updated_at. Use cases: create, get, list, update, delete, search.
2. **Design:**
 - Entity **Product** (id, name, price, stock, ...) with business methods like **ReduceStock(quantity), IncreaseStock(quantity), IsAvailable()**.
 - Repository interface: **Save, FindByID, FindAll, Update, Delete, Search**.
 - Use cases: **CreateProduct, GetProduct, UpdateProduct, DeleteProduct, ListProducts, SearchProducts**.
 - DTOs: **CreateProductRequest, ProductResponse, ListProductsRequest**, etc.

3. Implement Domain:

```
// internal/domain/product/entity.go
type Product struct {
    id      uint
    name    string
    price   Money
    stock   int
    status  ProductStatus
    createdAt time.Time
    updatedAt time.Time
}
// business methods...
```

4. Implement Infrastructure:

- GORM model with tags
- Repository implementation using GORM

5. Implement Usecase:

- `CreateProductUseCase` that validates and saves product

6. Implement Handler:

- HTTP handlers with validation

7. Write Tests:

- Unit tests for entity business logic
- Usecase tests with mock repository
- Repository tests with testcontainers

8. Document & Review:

- Update OpenAPI spec
- Run linter and coverage

9. Deploy:

- Create migration for `products` table
- Update Docker image

ผลลัพธ์: โครงสร้างไฟล์ใหม่

```
internal/
├── domain/
│   ├── product/
│   │   ├── entity.go
│   │   ├── repository.go
│   │   └── events.go
│   └── application/
│       ├── product/
│       │   ├── create.go
│       │   ├── get.go
│       │   ├── list.go
│       │   ├── update.go
│       │   ├── delete.go
│       │   └── dto.go
│       └── infrastructure/
│           ├── persistence/
│           │   └── gorm/
│           │       ├── product_repo.go
│           │       └── product_model.go
│           └── interfaces/
│               ├── http/
│               │   └── handlers/
│               │       └── product.go
```

สรุป

ภาคที่ 7 ได้เสนอแนวทางในการออกแบบสถาปัตยกรรมและกระบวนการพัฒนาสำหรับโปรเจกต์ Go ระดับ Production:

- **Clean Architecture** ช่วยแยกความรับผิดชอบและทำให้โค้ดทดสอบง่าย
- **Blueprint** ที่นำเสนอเป็นโครงสร้างเริ่มต้นที่ผ่านการพิสูจน์แล้วจากโปรเจกต์จริง
- **Workflow และ Task Management** ช่วยให้ทีมงานเป็นระบบ ลดความเสี่ยง และเพิ่มคุณภาพ

การนำหลักการเหล่านี้ไปใช้จะช่วยให้ออปพลิเคชันมีความยืดหยุ่น บำรุงรักษาง่าย และสามารถขยายขนาดได้ตามความต้องการทางธุรกิจ

บทที่ 46: Clean Architecture และโครงสร้างโปรเจกต์

46.1 โครงสร้างโปรเจกต์แบบ Clean Architecture

โครงสร้างที่แบ่งเป็น 3 ชั้นหลัก:

- **Delivery** – รับและส่งข้อมูล (HTTP handlers, gRPC, CLI)
- **Usecase** – business logic (interactors)
- **Repository** – การเข้าถึงข้อมูล (database, external API)

นอกจากนี้ยังมี **Models** (Entities) ที่ใช้ร่วมกันทุกชั้น

โครงสร้างโฟลเดอร์ตัวอย่าง

```
├── cmd/
│   └── api/
│       └── main.go
├── internal/
│   ├── config/                # การตั้งค่า
│   ├── delivery/
│   │   ├── http/             # HTTP handlers
│   │   │   ├── handler.go
│   │   │   ├── middleware.go
│   │   └── routes.go
│   ├── cli/                   # (optional) CLI commands
│   ├── models/                # entities / DTOs
│   ├── repository/           # implementations
│   │   ├── user_repo.go
│   │   ├── user_repo_mock.go (สำหรับ test)
│   │   └── redis/             # redis implementation
│   └── usecase/               # business logic
│       ├── user_usecase.go
│       └── auth_usecase.go
├── pkg/                       # reusable packages
│   ├── jwt/
│   ├── mail/
│   └── redis/
├── go.mod
└── config.yaml
```


46.2 Models (Entities)

models/user.go

```
package models

import "time"

type User struct {
    ID          uint      `json:"id"`
    Name        string    `json:"name"`
    Email       string    `json:"email"`
    Password    string    `json:"- "` // ไม่ส่งกลับใน JSON
    IsVerified  bool      `json:"is_verified"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}

// สำหรับการ register
type RegisterRequest struct {
    Name        string `json:"name" validate:"required,min=3"`
    Email       string `json:"email" validate:"required,email"`
    Password    string `json:"password" validate:"required,min=8"`
}

// สำหรับ login
type LoginRequest struct {
    Email       string `json:"email" validate:"required,email"`
    Password    string `json:"password" validate:"required"`
}
```

46.3 Repository Interface

internal/repository/user_repo.go

```
package repository

import (
    "context"
    "your-project/internal/models"
)

type UserRepository interface {
    Create(ctx context.Context, user *models.User) error
    GetByID(ctx context.Context, id uint) (*models.User, error)
    GetByEmail(ctx context.Context, email string) (*models.User, error)
    Update(ctx context.Context, user *models.User) error
    Delete(ctx context.Context, id uint) error
}
```

46.4 Usecase

internal/usecase/user_usecase.go

```
package usecase

import (
    "context"
    "errors"
    "golang.org/x/crypto/bcrypt"
    "your-project/internal/models"
    "your-project/internal/repository"
)

type UserUsecase interface {
    Register(ctx context.Context, req *models.RegisterRequest) (*models.User, error)
    GetUserByID(ctx context.Context, id uint) (*models.User, error)
}

type userUsecase struct {
    userRepo repository.UserRepository
}

func NewUserUsecase(userRepo repository.UserRepository) UserUsecase {
    return &userUsecase{userRepo: userRepo}
}

func (u *userUsecase) Register(ctx context.Context, req *models.RegisterRequest) (*models.User, error) {
    // ตรวจสอบว่ามี email ซ้ำหรือไม่
    existing, _ := u.userRepo.GetByEmail(ctx, req.Email)
    if existing != nil {
        return nil, errors.New("email already registered")
    }

    // Hash password
    hashed, err := bcrypt.GenerateFromPassword([]byte(req.Password),
    bcrypt.DefaultCost)
    if err != nil {
        return nil, err
    }

    user := &models.User{
        Name:      req.Name,
        Email:     req.Email,
        Password:  string(hashed),
        IsVerified: false,
    }
}
```

```

    if err := u.userRepo.Create(ctx, user); err != nil {
        return nil, err
    }

    return user, nil
}

```

46.5 Delivery (HTTP)

internal/delivery/http/handler.go

```

package http

import (
    "encoding/json"
    "net/http"
    "your-project/internal/usecase"
    "your-project/internal/models"
    "github.com/go-chi/chi/v5"
    "github.com/go-playground/validator/v10"
)

type UserHandler struct {
    userUsecase usecase.UserUsecase
    validate    *validator.Validate
}

func NewUserHandler(userUsecase usecase.UserUsecase) *UserHandler {
    return &UserHandler{
        userUsecase: userUsecase,
        validate:    validator.New(),
    }
}

func (h *UserHandler) Register(w http.ResponseWriter, r *http.Request) {
    var req models.RegisterRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid request", http.StatusBadRequest)
        return
    }

    if err := h.validate.Struct(req); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    user, err := h.userUsecase.Register(r.Context(), &req)
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }
}

```

```

}

w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusCreated)
json.NewEncoder(w).Encode(user)
}

```

internal/delivery/http/routes.go

```

package http

import (
    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
)

func SetupRoutes(userHandler *UserHandler, authHandler *AuthHandler) *chi.Mux {
    r := chi.NewRouter()

    r.Use(middleware.Logger)
    r.Use(middleware.Recoverer)

    r.Post("/api/register", userHandler.Register)
    r.Post("/api/login", authHandler.Login)

    // Protected routes
    r.Group(func(r chi.Router) {
        r.Use(authHandler.AuthMiddleware)
        r.Get("/api/users/{id}", userHandler.GetUser)
        // ...
    })

    return r
}

```

บทที่ 47: Blueprint สำหรับโปรเจกต์ Go ระดับ Production

47.1 โครงสร้างโฟลเดอร์หลัก

```

project-root/
├─ cmd/           # จุดเริ่มต้นของโปรแกรม (executable)
├─ internal/      # โค้ดเฉพาะของแอปพลิเคชัน (ไม่ถูก import จากภายนอก)
├─ pkg/          # โค้ดที่นำกลับไปใช้ได้ (reusable) ในโปรเจกต์อื่น
├─ api/          # ไฟล์ที่เกี่ยวข้องกับ API (เช่น Swagger)
├─ configs/      # ไฟล์ configuration
├─ deploy/       # Docker, Kubernetes files
├─ migrations/   # SQL migration files

```

```
└─ scripts/          # สคริปต์ช่วยพัฒนา
└─ test/             # integration tests
```

47.2 ชั้นการทำงานภายใน **internal/**

apps/ – จดรวม dependencies และกำหนด routing

- **bootstrap/injection/** – ใช้สำหรับ wire dependencies (repository, service, handler) ด้วย dependency injection แบบ manual
- **router/v1/** – กำหนด routes สำหรับ API version 1 แยก public/protected

core/ – ชั้นโดเมนของแอปพลิเคชัน (แบ่งตาม domain module)

แต่ละ module (เช่น **auth**, **user**, **iot**) มีโครงสร้างย่อย:

- **entity/** – entity หลัก (struct พร้อม behavior) และ value objects
- **repository/** – interface สำหรับ repository
- **service/** – interface สำหรับ service และการ implement business logic
- **dto/** – data transfer objects (request/response)
- **model/** – model ที่ใช้กับฐานข้อมูล (optional)
- **handler/** – HTTP handlers (รับ request, เรียก service, ส่ง response)
- **routes.go** – ลงทะเบียน routes ของ module นี้

platform/ – ชั้น infrastructure

- **config/** – โหลด configuration (viper)
- **db/** – การเชื่อมต่อ PostgreSQL, Redis
- **cache/** – ฟังก์ชันช่วยสำหรับ Redis cache
- **queue/** – message queue (Redis pub/sub) + worker pool + dead letter queue
- **logger/** – structured logger (slog) + middleware

transport/ – ชั้น delivery

- **middleware/** – middleware ต่างๆ (CORS, rate limit, auth, logging, recovery, security headers)
- **httpx/** – utilities สำหรับ response, request, validation
- **utils/** – helper functions (context, pagination)

47.3 การเพิ่มโมดูลใหม่ (Feature)

สมมติต้องการเพิ่มโมดูล **product**:

1. สร้างโครงสร้างใน **internal/core/product/**:

- **entity/product.go** – entity และ behavior
- **repository/repository.go** – interface สำหรับ repository
- **service/service.go** – interface สำหรับ service + implementation
- **dto/product_dto.go** – request/response structs
- **handler/product_handler.go** – HTTP handlers
- **routes.go** – routes สำหรับ product

2. เพิ่ม repository implementation ใน `internal/platform/db/postgres/`
 3. เพิ่ม service implementation ใน `internal/core/product/service/service_impl.go`
 4. เพิ่ม handler ใน `internal/core/product/handler/`
 5. ลงทะเบียน dependencies ใน `internal/apps/app/bootstrap/injection/`
 6. เพิ่ม routes ใน `internal/apps/app/router/v1/protected_routes.go`
 7. อย่าลืมเพิ่ม migrations ถ้ามีการเปลี่ยนแปลง schema
-

บทที่ 48: การออกแบบ Workflow และ Task Management

48.1 Workflow การพัฒนา Feature ใหม่

1. **Analyze** – ทำความเข้าใจ requirement, ระบุ domain models, use cases
2. **Design** – ออกแบบ entities, value objects, repository interface, service interface
3. **Implement Domain** – เขียน entity, repository interface, service interface ใน `internal/core/<module>`
4. **Implement Infrastructure** – เขียน repository implementation (GORM), cache, queue (ถ้าจำเป็น)
5. **Implement Service** – เขียน business logic ใน service implementation
6. **Implement Handler** – เขียน HTTP handlers, ตรวจสอบ input validation, ใช้ service
7. **Register Routes** – ลงทะเบียน routes ใน router
8. **Test** – เขียน unit tests (domain, service), integration tests (handler)
9. **Document** – อัปเดต API docs (Swagger) ถ้ามี

48.2 Task List Template

Phase 1: Domain Design

- ☐ ระบุ domain model (entity, value objects)
- ☐ กำหนด invariants (business rules)
- ☐ ระบุ use cases (methods in service)
- ☐ กำหนด events (ถ้ามี)
- ☐ ออกแบบ repository interface (methods)
- ☐ ออกแบบ DTOs (request/response)

Phase 2: Implementation

- ☐ สร้าง entity struct และ behavior methods
- ☐ สร้าง repository interface
- ☐ สร้าง service interface
- ☐ สร้าง DTO structs
- ☐ เขียน unit tests สำหรับ entity
- ☐ เขียน unit tests สำหรับ service (mock repository)

Phase 3: Infrastructure

- ☐ สร้าง repository implementation (GORM)
- ☐ สร้าง migration file (ถ้ามี)
- ☐ ตั้งค่า Redis cache (ถ้าจำเป็น)
- ☐ ตั้งค่า message queue (ถ้าจำเป็น)
- ☐ ทดสอบ repository ด้วย integration test

Phase 4: Delivery

- ☐ สร้าง HTTP handlers
- ☐ เพิ่ม input validation (go-playground/validator)
- ☐ สร้าง routes
- ☐ ลงทะเบียน dependencies ใน injection
- ☐ ทดสอบ handler ด้วย httptest

Phase 5: Integration & Documentation

- ☐ ทดสอบ end-to-end ด้วย curl/Postman
- ☐ อัปเดต Swagger docs (ถ้ามี)
- ☐ อัปเดต README (ถ้าจำเป็น)
- ☐ รัน linter (`golangci-lint run`) และแก้ไข warnings

Phase 6: Review & Deploy

- ☐ Code review
- ☐ ตรวจสอบ performance (ถ้ามีการ query มาก)
- ☐ รัน test coverage (`go test -cover`), ควร > 80%
- ☐ รัน race detector (`go test -race`)
- ☐ Deploy to staging
- ☐ ทดสอบใน staging
- ☐ Deploy to production

48.3 Checklist Template

Code Quality Checklist

- ☐ All exported functions have comments
- ☐ No unused imports or variables (go vet)
- ☐ Code formatted with go fmt
- ☐ Error handling is explicit (no ignored errors)
- ☐ No use of panic in library code (only in main/init)
- ☐ Context is passed as first parameter
- ☐ Interfaces are small and focused
- ☐ No global state except configuration

Security Checklist

- ☐ Passwords hashed with bcrypt

- ☐ JWT secret loaded from environment, not hardcoded
- ☐ Refresh tokens stored in Redis, not in DB
- ☐ Access token short-lived ($\leq 15\text{min}$)
- ☐ CORS configured properly (allow only trusted origins)
- ☐ Input validation on all endpoints
- ☐ SQL injection prevented by using parameterized queries (GORM)
- ☐ No sensitive data in logs
- ☐ HTTPS enforced in production
- ☐ Rate limiting on auth endpoints

Performance Checklist

- ☐ Database indexes created on frequently queried columns
- ☐ User data cached in Redis
- ☐ Connection pools configured for DB and Redis
- ☐ Use of goroutines for non-blocking tasks (email sending)
- ☐ Avoid N+1 queries (use Preload in GORM)
- ☐ Benchmarks for critical paths

Testing Checklist

- ☐ Unit tests cover business logic (usecase)
- ☐ Repository tests with testcontainers or in-memory DB
- ☐ HTTP handler tests with httptest
- ☐ Mock external dependencies (Redis, Mailer)
- ☐ Race condition tests with `-race` flag
- ☐ Test coverage $> 80\%$

Deployment Checklist

- ☐ Configurable via environment variables
- ☐ Graceful shutdown (wait for existing requests)
- ☐ Health check endpoint
- ☐ Logging to stdout (for container)
- ☐ Docker image built with non-root user
- ☐ Secrets not baked into image
- ☐ Database migration runs automatically on startup (or separate step)
- ☐ Readiness and liveness probes configured
- ☐ Monitoring (Prometheus metrics) exposed

ภาคที่ 8: Domain-Driven Design (DDD) กับ Go

(บทที่ 49–51)

แผนภาพ Data Flow (Flowchart TB)


```

flowchart TB
    subgraph Ubiquitous["Ubiquitous Language (ภาษาร่วม)"]
        U1["Domain Experts & Developers<br/>ใช้คำศัพท์เดียวกัน"]
    end

    subgraph BoundedContext["Bounded Contexts (บริบทที่จำกัด)"]
        BC1["Order Context<br/>Order, OrderItem, Payment"]
        BC2["User Context<br/>User, Profile, Authentication"]
        BC3["Inventory Context<br/>Product, Stock, Warehouse"]
    end

    subgraph DomainModel["Domain Model (โมเดลโดเมน)"]
        direction TB
        DM1["Entities<br/>- มี identity<br/>- mutable"]
        DM2["Value Objects<br/>- ไม่มี identity<br/>- immutable"]
        DM3["Aggregates<br/>- กลุ่มของ entities/value objects<br/>- Aggregate Root เป็นทางเข้า"]
        DM4["Domain Events<br/>- เหตุการณ์สำคัญในโดเมน<br/>- ใช้สื่อสารระหว่าง aggregates"]
        DM5["Repositories<br/>- abstraction สำหรับ aggregate roots<br/>- ซ่อน infrastructure"]
        DM6["Domain Services<br/>- business logic ที่ไม่เหมาะกับ entity/value object"]
    end

    subgraph Application["Application Layer (Use Cases)"]
        A1["Commands<br/>PlaceOrder, RegisterUser"]
        A2["Queries<br/>GetOrder, ListUsers"]
        A3["DTOs<br/>Request/Response objects"]
        A4["Event Handlers<br/>- สنج domain events<br/>- ทำงาน async"]
    end

    subgraph Infrastructure["Infrastructure Layer"]
        I1["Persistence<br/>PostgreSQL, GORM"]
        I2["Event Store<br/>PostgreSQL JSONB, EventStoreDB"]
        I3["Message Broker<br/>RabbitMQ, Kafka"]
        I4["Cache<br/>Redis"]
    end

    Ubiquitous --> DomainModel
    DomainModel --> Application
    Application --> Infrastructure

    subgraph EventStorming["Event Storming (Workshop)"]
        ES1["Domain Events<br/>(orange)"]
        ES2["Commands<br/>(blue)"]
        ES3["Aggregates<br/>(yellow)"]
        ES4["Policies / Rules<br/>(purple)"]
        ES5["Read Models<br/>(green)"]
        ES6["External Systems<br/>(pink)"]
    end

```

```
subgraph CQRS["CQRS (Command Query Responsibility Segregation)"]
  C1["Command Model<br/>- Write side<br/>- Domain logic, consistency"]
  C2["Query Model<br/>- Read side<br/>- Optimized for queries,
denormalized"]
  C3["Event Sourcing<br/>- Store events as source of truth<br/>- Rebuild
state from events"]
  C4["Projections<br/>- Build read models from events"]
end

EventStorming --> DomainModel
CQRS --> Application
CQRS --> Infrastructure
```

คำอธิบายภาษาไทย (แบบละเอียด)

ภาคที่ 8 ครอบคลุมบทที่ 49–51 โดยมีเนื้อหาเกี่ยวกับ **Domain-Driven Design (DDD)** และการนำไปใช้ในภาษา Go:

- **บทที่ 49:** หลักการ DDD และการนำไปใช้ใน Go
- **บทที่ 50:** Aggregates, Event Storming และ CQRS
- **บทที่ 51:** การออกแบบบริการด้วย Go-DDD

DDD เป็นแนวทางในการออกแบบซอฟต์แวร์ที่เน้นการสร้างโมเดลที่สะท้อนความรู้ความเข้าใจ (domain knowledge) อย่างแท้จริง โดยใช้ภาษากลาง (Ubiquitous Language) ร่วมกันระหว่างนักพัฒนาและผู้เชี่ยวชาญโดเมน

บทที่ 49: หลักการ DDD และการนำไปใช้ใน Go

1. หลักการสำคัญของ DDD

- **Ubiquitous Language (ภาษาร่วม):** สร้างภาษากลางที่ใช้ร่วมกันระหว่างนักพัฒนาและผู้เชี่ยวชาญโดเมน ใช้ศัพท์เดียวกันในโค้ด, การสนทนา, และเอกสาร
- **Bounded Context (บริบทที่จำกัด):** แบ่งโดเมนขนาดใหญ่ออกเป็นบริบทย่อยที่มีขอบเขตชัดเจน แต่ละ Bounded Context มีโมเดลของตัวเองและภาษาร่วมของตัวเอง
- **Entities:** วัตถุที่มีเอกลักษณ์ (identity) และสามารถเปลี่ยนแปลงได้ (mutable) เช่น **User**, **Order**
- **Value Objects:** วัตถุที่ไม่มีเอกลักษณ์ในตัวเอง ถูกกำหนดโดยคุณสมบัติ (immutable) เช่น **Address**, **Money**
- **Aggregates:** กลุ่มของ Entities และ Value Objects ที่ถูกจัดการเป็นหน่วยเดียวกัน มี Aggregate Root เป็นตัวควบคุมความสอดคล้องของข้อมูล
- **Domain Events:** เหตุการณ์สำคัญในโดเมนที่เกิดขึ้น เช่น **OrderPlaced**, **UserRegistered**
- **Repositories:** ให้ abstraction ในการเข้าถึง aggregate roots ซ่อนรายละเอียดของแหล่งข้อมูล
- **Domain Services:** ใช้สำหรับ logic ที่ไม่เหมาะจะอยู่ใน entity หรือ value object เช่น **TransferService** ที่โอนเงินระหว่างบัญชี

2. การนำ DDD ไปใช้ใน Go

Go ไม่มี class และ inheritance แต่ใช้ **struct + method** และ **interface** เพื่อสร้าง abstraction การสืบทอดผ่าน **composition** (embedding) แทน

2.1 Entities และ Value Objects

```
// internal/domain/user/entity.go
package user

import (
    "errors"
    "time"
)

// Entity: User (มี identity)
type User struct {
    id          uint
    email       Email      // Value Object
    password    Password  // Value Object
    name        string
    isActive    bool
    createdAt   time.Time
    updatedAt   time.Time
    events      []DomainEvent
}

// Constructor (factory method)
func NewUser(email, password, name string) (*User, error) {
    emailVO, err := NewEmail(email)
    if err != nil {
        return nil, err
    }
    passwordVO, err := NewPassword(password)
    if err != nil {
        return nil, err
    }
    return &User{
        id:          generateID(),
        email:       *emailVO,
        password:    *passwordVO,
        name:        name,
        isActive:    true,
        createdAt:   time.Now(),
        updatedAt:   time.Now(),
        events:      []DomainEvent{},
    }, nil
}

// Getters (exported)
func (u *User) ID() uint      { return u.id }
func (u *User) Email() string { return u.email.String() }
func (u *User) Name() string  { return u.name }
func (u *User) IsActive() bool { return u.isActive }

// Business method
```

```

func (u *User) Deactivate() error {
    if !u.isActive {
        return errors.New("user already inactive")
    }
    u.isActive = false
    u.updatedAt = time.Now()
    u.addDomainEvent(NewUserDeactivatedEvent(u.id))
    return nil
}

func (u *User) addDomainEvent(event DomainEvent) {
    u.events = append(u.events, event)
}

func (u *User) Events() []DomainEvent { return u.events }
func (u *User) ClearEvents()          { u.events = nil }

```

```

// internal/domain/user/value_objects.go
package user

import (
    "regexp"
    "strings"

    "golang.org/x/crypto/bcrypt"
)

// Value Object: Email (immutable)
type Email struct {
    value string
}

func NewEmail(email string) (*Email, error) {
    email = strings.TrimSpace(email)
    if !isValidEmail(email) {
        return nil, errors.New("invalid email format")
    }
    return &Email{value: email}, nil
}

func (e Email) String() string { return e.value }

// Value Object: Password (immutable, hashed)
type Password struct {
    hash string
}

func NewPassword(plain string) (*Password, error) {
    if len(plain) < 8 {
        return nil, errors.New("password must be at least 8 characters")
    }

```

```

    }
    hash, err := bcrypt.GenerateFromPassword([]byte(plain), bcrypt.DefaultCost)
    if err != nil {
        return nil, err
    }
    return &Password{hash: string(hash)}, nil
}

func (p *Password) Compare(plain string) bool {
    return bcrypt.CompareHashAndPassword([]byte(p.hash), []byte(plain)) == nil
}

```

2.2 Aggregates and Aggregate Root

```

// internal/domain/order/aggregate.go
package order

import (
    "errors"
    "time"
)

// Aggregate Root: Order
type Order struct {
    id            uint
    customerID    uint
    items         []OrderItem // slice of value objects
    status        OrderStatus
    total         Money
    createdAt     time.Time
    updatedAt     time.Time
    events        []DomainEvent
}

func NewOrder(customerID uint) *Order {
    o := &Order{
        id:            generateID(),
        customerID:    customerID,
        items:         []OrderItem{},
        status:        StatusDraft,
        total:         Money{},
        createdAt:     time.Now(),
        updatedAt:     time.Now(),
    }
    o.addDomainEvent(NewOrderCreatedEvent(o.id, customerID))
    return o
}

func (o *Order) AddItem(productID uint, name string, price Money, quantity int)
error {

```

```

    if o.status != StatusDraft {
        return errors.New("cannot add item to non-draft order")
    }
    if quantity <= 0 {
        return errors.New("quantity must be positive")
    }
    // Create value object
    item := NewOrderItem(productID, name, price, quantity)
    o.items = append(o.items, item)
    o.recalculateTotal()
    o.updatedAt = time.Now()
    o.addDomainEvent(NewOrderItemAddedEvent(o.id, productID, quantity))
    return nil
}

func (o *Order) Submit() error {
    if len(o.items) == 0 {
        return errors.New("cannot submit empty order")
    }
    if o.status != StatusDraft {
        return errors.New("order already submitted")
    }
    o.status = StatusSubmitted
    o.updatedAt = time.Now()
    o.addDomainEvent(NewOrderSubmittedEvent(o.id))
    return nil
}

func (o *Order) recalculateTotal() {
    var sum int64
    for _, item := range o.items {
        sum += item.Total().Cents()
    }
    o.total = NewMoneyFromCents(sum)
}

// Repository interface for Order aggregate root
type OrderRepository interface {
    Save(ctx context.Context, order *Order) error
    FindByID(ctx context.Context, id uint) (*Order, error)
    FindByCustomer(ctx context.Context, customerID uint) ([]*Order, error)
}

```

2.3 Domain Events

```

// internal/domain/order/events.go
package order

import "time"

```

```

type DomainEvent interface {
    OccurredAt() time.Time
}

type OrderCreatedEvent struct {
    OrderID      uint      `json:"order_id"`
    CustomerID   uint      `json:"customer_id"`
    Occurred     time.Time `json:"occurred_at"`
}

func NewOrderCreatedEvent(orderID, customerID uint) OrderCreatedEvent {
    return OrderCreatedEvent{
        OrderID:      orderID,
        CustomerID:   customerID,
        Occurred:     time.Now(),
    }
}

func (e OrderCreatedEvent) OccurredAt() time.Time { return e.Occurred }

type OrderSubmittedEvent struct {
    OrderID      uint      `json:"order_id"`
    Occurred     time.Time `json:"occurred_at"`
}

func NewOrderSubmittedEvent(orderID uint) OrderSubmittedEvent {
    return OrderSubmittedEvent{
        OrderID:      orderID,
        Occurred:     time.Now(),
    }
}

func (e OrderSubmittedEvent) OccurredAt() time.Time { return e.Occurred }

```

2.4 Domain Services

```

// internal/domain/order/transfer_service.go
package order

type TransferService interface {
    Transfer(ctx context.Context, fromID, toID uint, amount Money) error
}

// Implementation in infrastructure layer
type BankTransferService struct {
    client *http.Client
    // ...
}

func (s *BankTransferService) Transfer(ctx context.Context, fromID, toID uint,

```

```

amount Money) error {
    // Call external bank API
    // ...
}

```

2.5 Repositories (interface ใน domain, implementation ใน infrastructure)

```

// internal/domain/order/repository.go (interface)
package order

import "context"

type OrderRepository interface {
    Save(ctx context.Context, order *Order) error
    FindByID(ctx context.Context, id uint) (*Order, error)
}

```

```

// internal/infrastructure/persistence/gorm/order_repo.go (implementation)
package gormrepo

import (
    "context"
    "encoding/json"

    "gorm.io/gorm"
    "myapp/internal/domain/order"
)

type orderRepository struct {
    db *gorm.DB
}

func NewOrderRepository(db *gorm.DB) order.OrderRepository {
    return &orderRepository{db: db}
}

// GORM model for persistence
type OrderModel struct {
    ID          uint          `gorm:"primaryKey"`
    CustomerID  uint          `gorm:"index"`
    Status      string        `gorm:"size:20"`
    TotalCents  int64
    Items       json.RawMessage `gorm:"type:jsonb"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
}

func (r *orderRepository) Save(ctx context.Context, o *order.Order) error {

```



```
// Convert domain Order to OrderModel
itemsJSON, _ := json.Marshal(o.Items()) // assume items are serializable
model := &OrderModel{
    ID:          o.ID(),
    CustomerID: o.CustomerID(),
    Status:      o.Status().String(),
    TotalCents: o.Total().Cents(),
    Items:      itemsJSON,
    CreatedAt:  o.CreatedAt(),
    UpdatedAt:  o.UpdatedAt(),
}
return r.db.WithContext(ctx).Save(model).Error
}
```

บทที่ 50: Aggregates, Event Storming และ CQRS

1. Aggregates (กลุ่มวัตถุที่มีความสอดคล้อง)

Aggregate คือกลุ่มของ Entities และ Value Objects ที่ถูกยึดเข้าด้วยกันโดย Aggregate Root ซึ่งเป็นตัวเดียวที่อนุญาตให้เข้าถึงหรือแก้ไขวัตถุอื่นภายในกลุ่มจากภายนอก

หลักการ:

- Aggregate Root มี identity และเป็นจุดเดียวที่ภายนอกเข้าถึงได้
- การเปลี่ยนแปลงใด ๆ ภายใน Aggregate ต้องทำผ่าน Root เท่านั้น
- ภายใน Aggregate เดียวกันต้องมีความสอดคล้องกันในพื้นที่ (consistency boundary)
- ระหว่าง Aggregates ควรใช้ eventual consistency ผ่าน Domain Events

2. Event Storming (เทคนิคค้นพบโดเมน)

Event Storming เป็นเวิร์กช็อปแบบมีส่วนร่วมที่ช่วยให้ทีม (นักพัฒนา, นักธุรกิจ, ผู้เชี่ยวชาญโดเมน) ระดมและเข้าใจโดเมนผ่านเหตุการณ์สำคัญที่เกิดขึ้นในระบบ

สัญลักษณ์ทั่วไป:

- **สีส้ม** – Domain Events (สิ่งที่เกิดขึ้นแล้ว) เช่น **OrderPlaced**, **PaymentReceived**
- **สีน้ำเงิน** – Commands (คำสั่งที่ทำให้เกิดเหตุการณ์) เช่น **PlaceOrder**, **RefundPayment**
- **สีเหลือง** – Aggregates (กลุ่มข้อมูลที่ถูกคำสั่งเรียก) เช่น **Order**, **Customer**
- **สีม่วง** – Policies / Rules (กฎที่ทำงานอัตโนมัติ)
- **สีเขียว** – Read Models (ข้อมูลที่แสดงผล)
- **สีชมพู** – External Systems (ระบบภายนอก)

3. CQRS (Command Query Responsibility Segregation)

CQRS แยกโมเดลการ **เขียน** (Command) และ **อ่าน** (Query) ออกจากกัน ทำให้สามารถปรับแต่งให้เหมาะสมกับแต่ละฝั่งได้

ตัวอย่างโครงสร้าง **CQRS** ใน **Go**:

```

// Command side (write model)
type PlaceOrderCommand struct {
    CustomerID uint
    Items      []OrderItemDTO
}

type PlaceOrderHandler struct {
    orderRepo order.OrderRepository
    eventBus  EventBus
}

func (h *PlaceOrderHandler) Handle(ctx context.Context, cmd PlaceOrderCommand)
error {
    // Create order aggregate
    ord := order.NewOrder(cmd.CustomerID)
    for _, item := range cmd.Items {
        ord.AddItem(item.ProductID, item.Name, item.Price, item.Quantity)
    }
    if err := ord.Submit(); err != nil {
        return err
    }
    if err := h.orderRepo.Save(ctx, ord); err != nil {
        return err
    }
    // Publish events
    for _, evt := range ord.Events() {
        h.eventBus.Publish(evt)
    }
    return nil
}

// Query side (read model)
type OrderReadModel struct {
    ID          uint
    CustomerID  uint
    Status      string
    Total       float64
    Items       []OrderItemReadModel
}

type OrderQueryHandler struct {
    db *sql.DB // หรือ repository อ่านอย่างเดียว
}

func (h *OrderQueryHandler) GetOrder(ctx context.Context, id uint)
(*OrderReadModel, error) {
    var order OrderReadModel
    // query จาก read database (อาจเป็นตารางที่ denormalized)
    // ...
    return &order, nil
}

```

Event Sourcing + Projections

Event Sourcing คือการเก็บ event ทั้งหมดเป็น source of truth แล้วสร้าง state ปัจจุบันจากการ replay events

```
// Event store
type EventStore interface {
    Save(ctx context.Context, streamID string, events []DomainEvent,
        expectedVersion int) error
    Load(ctx context.Context, streamID string) ([]DomainEvent, error)
}

// Projection: สร้าง read model จาก events
type OrderProjection struct {
    db *sql.DB
}

func (p *OrderProjection) HandleEvent(event DomainEvent) {
    switch e := event.(type) {
    case OrderCreatedEvent:
        p.db.Exec("INSERT INTO order_read_models (id, customer_id, status,
            total, created_at) VALUES ($1, $2, $3, $4, $5)",
            e.OrderID, e.CustomerID, "Draft", 0, e.Occurred)
    case OrderItemAddedEvent:
        p.db.Exec("INSERT INTO order_item_read_models (order_id, product_id,
            quantity, price) VALUES ($1, $2, $3, $4)",
            e.OrderID, e.ProductID, e.Quantity, e.Price)
        p.db.Exec("UPDATE order_read_models SET total = total + $1 WHERE id =
            $2", e.Price*float64(e.Quantity), e.OrderID)
    case OrderSubmittedEvent:
        p.db.Exec("UPDATE order_read_models SET status = 'Submitted' WHERE id =
            $1", e.OrderID)
    }
}
```

บทที่ 51: การออกแบบบริการด้วย Go-DDD

โครงสร้างโปรเจกต์แบบ DDD เต็มรูปแบบ

```
myapp/
├── cmd/
│   └── api/
│       └── main.go
├── internal/
│   ├── config/           # configuration
│   ├── domain/          # domain layer
│   │   └── user/
│   │       ├── entity.go
│   │       └── value_objects.go
```

```

├── repository.go
├── events.go
├── service.go (domain services)
├── order/
│   ├── aggregate.go
│   ├── repository.go
│   └── events.go
├── shared/          # shared value objects
├── application/     # use cases
│   ├── user/
│   │   ├── register.go
│   │   ├── login.go
│   │   └── dto.go
│   └── order/
│       ├── place_order.go
│       ├── get_order.go
│       └── dto.go
├── infrastructure/  # implementations
│   ├── persistence/
│   │   ├── gorm/
│   │   │   ├── user_repo.go
│   │   │   └── order_repo.go
│   │   └── event_store/
│   ├── bus/         # event bus
│   └── mail/
├── interfaces/      # delivery
│   ├── http/
│   │   ├── handlers/
│   │   ├── middleware/
│   │   └── routes.go
│   └── cli/
├── pkg/              # reusable packages
│   ├── logger/
│   └── errors/
├── migrations/
└── go.mod

```

ตัวอย่าง Use Case (Application Layer)

```

// internal/application/order/place_order.go
package order

import (
    "context"

    "myapp/internal/domain/order"
    "myapp/internal/domain/user"
)

type PlaceOrderUseCase struct {

```

```

    orderRepo order.OrderRepository
    userRepo  user.UserRepository
    eventBus  EventBus
}

type PlaceOrderInput struct {
    UserID uint
    Items  []OrderItemInput
}

type OrderItemInput struct {
    ProductID uint
    Name      string
    Price     float64
    Quantity  int
}

type PlaceOrderOutput struct {
    OrderID uint
    Status  string
}

func NewPlaceOrderUseCase(
    orderRepo order.OrderRepository,
    userRepo  user.UserRepository,
    eventBus  EventBus,
) *PlaceOrderUseCase {
    return &PlaceOrderUseCase{
        orderRepo: orderRepo,
        userRepo:  userRepo,
        eventBus:  eventBus,
    }
}

func (uc *PlaceOrderUseCase) Execute(ctx context.Context, input
PlaceOrderInput) (*PlaceOrderOutput, error) {
    // 1. Validate user exists and is active
    u, err := uc.userRepo.FindByID(ctx, input.UserID)
    if err != nil {
        return nil, fmt.Errorf("user not found: %w", err)
    }
    if !u.IsActive() {
        return nil, errors.New("inactive user cannot place order")
    }

    // 2. Create order aggregate
    ord := order.NewOrder(input.UserID)
    for _, item := range input.Items {
        money := order.NewMoney(item.Price)
        ord.AddItem(item.ProductID, item.Name, money, item.Quantity)
    }

    // 3. Submit order

```

```

    if err := ord.Submit(); err != nil {
        return nil, err
    }

    // 4. Persist
    if err := uc.orderRepo.Save(ctx, ord); err != nil {
        return nil, err
    }

    // 5. Dispatch events
    for _, evt := range ord.Events() {
        uc.eventBus.Publish(evt)
    }

    return &PlaceOrderOutput{
        OrderID: ord.ID(),
        Status:  ord.Status().String(),
    }, nil
}

```

Dependency Injection (Wire)

```

// internal/wire/wire.go
package wire

import (
    "myapp/internal/application/order"
    "myapp/internal/domain/order"
    "myapp/internal/infrastructure/persistence/gorm"
    "myapp/internal/interfaces/http/handlers"
    "myapp/pkg/eventbus"
)

func InitializeOrderAPI(db *gorm.DB, bus eventbus.EventBus)
*handlers.OrderHandler {
    orderRepo := gorm.NewOrderRepository(db)
    userRepo := gorm.NewUserRepository(db)

    placeOrderUC := order.NewPlaceOrderUseCase(orderRepo, userRepo, bus)
    getOrderUC := order.NewGetOrderUseCase(orderRepo)

    return handlers.NewOrderHandler(placeOrderUC, getOrderUC)
}

```

สรุป

ภาคที่ 8 ได้แนะนำหลักการ Domain-Driven Design และการนำไปใช้ใน Go อย่างเป็นรูปธรรม:

- **หลักการ DDD:** Ubiquitous Language, Bounded Context, Entities, Value Objects, Aggregates, Repositories, Domain Services, Domain Events
- **Event Storming:** เทคนิคการวิเคราะห์โดเมนผ่านเหตุการณ์
- **CQRS:** แยก command และ query models, event sourcing, projections

การนำ DDD มาปรับใช้ใน Go ช่วยให้โค้ดสะท้อนภาษารธุรกิจ ทดสอบง่าย และปรับเปลี่ยน infrastructure ได้โดยไม่กระทบโดเมน

บทที่ 49: หลักการ DDD และการนำไปใช้ใน Go

49.1 Domain-Driven Design (DDD) คืออะไร?

Domain-Driven Design เป็นแนวทางการออกแบบซอฟต์แวร์ที่เน้นการสร้างโมเดลที่สะท้อนความรู้ความเข้าใจ (domain knowledge) อย่างแท้จริง โดยมีหลักการสำคัญคือการทำให้ออฟต์แวร์สอดคล้องกับความต้องการทางธุรกิจผ่านการร่วมมือกันระหว่างนักพัฒนาและผู้เชี่ยวชาญในโดเมน (domain experts)

49.2 หลักการสำคัญของ DDD

1. Ubiquitous Language (ภาษาร่วม)

- สร้างภาษากลางที่ใช้ร่วมกันระหว่างนักพัฒนาและผู้เชี่ยวชาญโดเมน
- ใช้ศัพท์เดียวกันในโค้ด, การสนทนา, และเอกสาร
- ลดความเข้าใจผิดและเพิ่มความสอดคล้อง

2. Bounded Context (บริบทที่จำกัด)

- แบ่งโดเมนขนาดใหญ่ออกเป็นบริบทย่อยที่มีขอบเขตชัดเจน
- แต่ละ Bounded Context มีโมเดลของตัวเองและภาษาร่วมของตัวเอง
- ลดความซับซ้อนและความขัดแย้งของโมเดล

3. Entities และ Value Objects

- **Entity:** วัตถุที่มีเอกลักษณ์ (identity) และสามารถเปลี่ยนแปลงได้ (mutable) เช่น **User**, **Order**
- **Value Object:** วัตถุที่ไม่มีเอกลักษณ์ในตัวเอง ถูกกำหนดโดยคุณสมบัติ (immutable) เช่น **Address**, **Money**

4. Aggregates

- กลุ่มของ Entities และ Value Objects ที่ถูกจัดการเป็นหน่วยเดียวกัน
- มี Aggregate Root (รฐ) เป็นตัวควบคุมความสอดคล้องของข้อมูล
- เช่น **Order** (aggregate root) ประกอบด้วย **OrderItem** (entity) และ **Address** (value object)

5. Domain Events

- เหตุการณ์สำคัญในโดเมนที่เกิดขึ้น เช่น **OrderPlaced**, **UserRegistered**
- ใช้ในการสื่อสารระหว่าง aggregates หรือระหว่าง bounded contexts

6. Repositories

- ให้ abstraction ในการเข้าถึง aggregate roots
- ซ่อนรายละเอียดของแหล่งข้อมูล (database, cache)

7. Domain Services

- ใช้สำหรับ logic ที่ไม่เหมาะจะอยู่ใน entity หรือ value object
- เช่น **TransferService** ที่โอนเงินระหว่างบัญชี

49.3 การนำ DDD ไปใช้กับ Go: ข้อแนะนำเฉพาะภาษา

1. จัดโครงสร้างโปรเจกต์ตามโมดูล

```
/cmd
  /myapp
    main.go
/internal
  /domain
    /order
      order.go (entity, value objects)
      repository.go (interface)
      events.go
  /application
    order_service.go
  /infrastructure
    /persistence
      order_repo_mysql.go
    /bus
      event_bus_kafka.go
  /presentation
    /http
      order_handler.go
/pkg
  /shared
    errors.go, uuid.go, etc.
```

2. ใช้ interface เพื่อ Dependency Inversion

- Application layer รับ domain interface
- Infrastructure ถูก inject ผ่าน constructor
- ใช้ **wire** (Google Wire) หรือ manual DI สำหรับการประกอบ dependencies

3. จัดการ Transaction

- นิยมใช้ **Unit of Work** pattern: application service เริ่ม transaction ผ่าน interface

4. Domain Events

- ใช้ channel หรือ event bus ภายใน memory ก่อน แล้วค่อยเพิ่ม infrastructure

5. Value Objects กับ immutability

- ใช้ struct พร้อม private fields และ constructor functions
- เปรียบเทียบด้วย **==** หรือ implement **Equals** method

บทที่ 50: Aggregates, Event Storming และ CQRS

50.1 Aggregate (กลุ่มวัตถุที่มีความสอดคล้อง)

Aggregate คือกลุ่มของวัตถุ (Entities + Value Objects) ที่ถูกยึดเข้าด้วยกันโดย **Aggregate Root** (รูท) ซึ่งเป็นตัวเดียวที่อนุญาตให้เข้าถึงหรือแก้ไขวัตถุอื่นภายในกลุ่มจากภายนอก การออกแบบ Aggregate ช่วยรักษาความถูกต้องของข้อมูล (invariants) และลดความซับซ้อนในการจัดการธุรกรรม

หลักการสำคัญ:

- **Aggregate Root** มี identity และเป็นจุดเดียวที่ภายนอกเข้าถึงได้
- การเปลี่ยนแปลงใด ๆ ภายใน Aggregate ต้องทำผ่าน Root เท่านั้น
- ภายใน Aggregate เดียวกันต้องมีความสอดคล้องกันในพื้นที่ (consistency boundary)
- ระหว่าง Aggregates ควรใช้ **eventual consistency** ผ่าน Domain Events

ตัวอย่างใน Go:

```
// aggregate root: Order
type Order struct {
    id      OrderID
    status  OrderStatus
    items   []OrderItem // value object
    total   Money
}

func (o *Order) AddItem(product Product, quantity int) error {
    if o.status != Draft {
        return errors.New("cannot add item to non-draft order")
    }
    // invariant: total must not exceed limit
    newTotal := o.total.Add(product.Price.Mul(quantity))
    if newTotal.GreaterThan(MaxOrderTotal) {
        return errors.New("order total exceeds limit")
    }
    o.items = append(o.items, NewOrderItem(product, quantity))
    o.total = newTotal
    o.addDomainEvent(OrderItemAdded{OrderID: o.id, ProductID: product.ID})
    return nil
}

// repository interface รับเฉพาะ aggregate root
type OrderRepository interface {
    Save(order *Order) error
    FindByID(id OrderID) (*Order, error)
}
```

50.2 Event Storming (เทคนิคค้นพบโดเมน)

Event Storming เป็นเวิร์กช็อปแบบมีส่วนร่วมที่ช่วยให้ทีม (นักพัฒนา, นักธุรกิจ, ผู้เชี่ยวชาญโดเมน) ระบุและเข้าใจโดเมนผ่านเหตุการณ์สำคัญที่เกิดขึ้นในระบบ โดยใช้โน้ตสีต่าง ๆ บนกระดาน

สัญลักษณ์ทั่วไป:

- **สีส้ม** – Domain Events (สิ่งที่เกิดขึ้นแล้ว) เช่น `OrderPlaced`, `PaymentReceived`
- **สีน้ำเงิน** – Commands (คำสั่งที่ทำให้เกิดเหตุการณ์) เช่น `PlaceOrder`, `RefundPayment`
- **สีเหลือง** – Aggregates (กลุ่มข้อมูลที่ถูกคำสั่งเรียก) เช่น `Order`, `Customer`
- **สีม่วง** – External Systems / Policies (ระบบภายนอกหรือกฎ)
- **สีเขียว** – Read Models / Views (ข้อมูลที่แสดงผล)

ขั้นตอนคร่าว ๆ:

1. ระบุ Domain Events (อดีต) โดยเรียงตามลำดับเวลา
2. ระบุ Commands ที่ทำให้เกิดเหตุการณ์นั้น
3. จับคู่ Command กับ Aggregate (ผู้รับผิดชอบ)
4. เพิ่ม Policies / Rules และ External Systems
5. ระบุ Read Models ที่จำเป็นต่อการแสดงผล

Event Storming นำไปสู่การกำหนด **Bounded Context** และ **Aggregates** ที่ชัดเจน ก่อนเริ่มเขียนโค้ด

50.3 CQRS (Command Query Responsibility Segregation)

CQRS แยกโมเดลการ **เขียน** (Command) และ **อ่าน** (Query) ออกจากกัน ทำให้สามารถปรับแต่งให้เหมาะสมกับแต่ละฝั่งได้ เช่น ใช้ฐานข้อมูลแบบ Event Sourcing สำหรับ Command และฐานข้อมูลแบบ Materialized View สำหรับ Query

ใน Go สามารถจัดโครงสร้างได้ดังนี้:

แยก **Command** และ **Query Models**

```
// Command models (เขียน)
type PlaceOrderCommand struct {
    OrderID string
    Items []OrderItemDTO
}

// Query models (อ่าน)
type OrderView struct {
    OrderID string
    Status string
    TotalPrice float64
    Items []OrderItemView
}
```

Command Handlers

```

type PlaceOrderHandler struct {
    repo      domain.OrderRepository
    eventBus   domain.EventBus
}

func (h *PlaceOrderHandler) Handle(ctx context.Context, cmd PlaceOrderCommand)
error {
    order, err := domain.NewOrder(cmd.OrderID)
    if err != nil {
        return err
    }
    for _, item := range cmd.Items {
        order.AddItem(item.ProductID, item.Quantity)
    }
    if err := h.repo.Save(order); err != nil {
        return err
    }
    // Publish events for projection
    for _, event := range order.Events() {
        h.eventBus.Publish(event)
    }
    return nil
}

```

Query Handlers (อ่านจาก Read Database)

```

type OrderQueryHandler struct {
    db *sql.DB // หรือ ORM
}

func (h *OrderQueryHandler) GetOrder(ctx context.Context, orderID string)
(*OrderView, error) {
    var view OrderView
    err := h.db.QueryRowContext(ctx, "SELECT ... FROM order_views WHERE id =
?", orderID).Scan(&view)
    return &view, err
}

```

Projection (สร้าง Read Model จาก Events)

```

type OrderProjection struct {
    db *sql.DB
}

func (p *OrderProjection) HandleEvent(event domain.DomainEvent) {
    switch e := event.(type) {
    case OrderPlaced:

```

```

        p.db.Exec("INSERT INTO order_views (id, status, total) VALUES (?, ?,
?)", e.OrderID, "Placed", e.Total)
    case OrderItemAdded:
        p.db.Exec("INSERT INTO order_items_view ...")
    }
}

```

50.4 การนำ CQRS ไปใช้ใน Go อย่างมีประสิทธิภาพ

- **ใช้ Interface ในการแยก:** Command handlers, Query handlers, Projection handlers แต่ละตัวเป็น struct ที่ implements interface ต่างกัน ทำให้ test และ replace ได้ง่าย
- **Event Store:** ใน Go สามารถใช้ database เช่น PostgreSQL พร้อม JSONB เก็บ events
- **Read Database:** อาจใช้ฐานข้อมูลแยก (SQL, NoSQL) หรือ cache เช่น Redis สำหรับการ query ที่รวดเร็ว
- **Concurrency:** Goroutine + channel ใช้ในการประมวลผล projection แบบ asynchronous

ข้อควรระวัง:

- CQRS เพิ่มความซับซ้อน เหมาะสำหรับระบบที่ต้องการความยืดหยุ่นสูง (microservices, high scalability)
- ไม่จำเป็นต้องใช้ Event Sourcing เสมอไป CQRS สามารถแยกโมเดลอ่าน-เขียนโดยใช้ฐานข้อมูลเดียวกันได้ (แต่แยกตารางหรือ schema)
- การจัดการ eventual consistency ต้องออกแบบ UX ให้เหมาะสม

บทที่ 51: การออกแบบบริการด้วย Go-DDD

51.1 โครงสร้างโปรเจกต์แบบ DDD เต็มรูปแบบ

```

project/
├── cmd/
│   └── api/
│       └── main.go                # entry point
├── internal/
│   ├── domain/                   # ชั้นโดเมน (core business logic)
│   │   ├── user/
│   │   │   ├── entity.go         # User entity
│   │   │   ├── value_objects.go  # Email, Password, etc.
│   │   │   ├── repository.go     # interface
│   │   │   ├── service.go        # domain services
│   │   │   └── events.go         # domain events
│   │   └── order/
│   │       └── ...
│   └── shared/                   # shared value objects
├── application/                  # ชั้นแอปพลิเคชัน (use cases)
│   ├── user/
│   │   ├── register.go          # use case
│   │   ├── login.go
│   │   └── dto.go                # input/output DTOs
│   └── order/
│       └── ...

```

```

├── infrastructure/           # ชั้นโครงสร้างพื้นฐาน
│   ├── persistence/
│   │   ├── gorm/
│   │   │   ├── user_repo.go # implementation
│   │   │   └── models.go    # GORM models
│   │   └── redis/
│   ├── mail/
│   ├── bus/                 # event bus
│   └── interfaces/         # ชั้นติดต่อกับภายนอก
│       ├── http/
│       │   ├── handlers/
│       │   ├── middleware/
│       │   └── routes.go
│       └── cli/
├── pkg/                     # reusable packages
└── go.mod

```

51.2 การสร้าง Domain Layer

1. Entities (domain/user/entity.go)

```

package user

import (
    "time"
    "github.com/google/uuid"
)

type User struct {
    id          uuid.UUID
    email       Email          // value object
    password    Password       // value object
    name        string
    isVerified  bool
    createdAt   time.Time
    updatedAt   time.Time
    events      []DomainEvent
}

// Constructor
func NewUser(email, password, name string) (*User, error) {
    emailVO, err := NewEmail(email)
    if err != nil {
        return nil, err
    }
    passwordVO, err := NewPassword(password)
    if err != nil {
        return nil, err
    }
}

```

```

    user := &User{
        id:          uuid.New(),
        email:        *emailVO,
        password:     *passwordVO,
        name:         name,
        isVerified:   false,
        createdAt:    time.Now(),
        updatedAt:    time.Now(),
        events:       []DomainEvent{},
    }

    user.addDomainEvent(NewUserRegisteredEvent(user.id, user.email.String()))

    return user, nil
}

// Getters (exported)
func (u *User) ID() uuid.UUID      { return u.id }
func (u *User) Email() Email       { return u.email }
func (u *User) Name() string       { return u.name }
func (u *User) IsVerified() bool   { return u.isVerified }
func (u *User) Events() []DomainEvent { return u.events }
func (u *User) ClearEvents()       { u.events = nil }

// Behavior methods
func (u *User) Verify() {
    u.isVerified = true
    u.updatedAt = time.Now()
    u.addDomainEvent(NewUserVerifiedEvent(u.id))
}

func (u *User) ChangePassword(old, new string) error {
    if err := u.password.Compare(old); err != nil {
        return ErrInvalidPassword
    }
    newPassword, err := NewPassword(new)
    if err != nil {
        return err
    }
    u.password = *newPassword
    u.updatedAt = time.Now()
    return nil
}

func (u *User) addDomainEvent(event DomainEvent) {
    u.events = append(u.events, event)
}

```

2. Value Objects (domain/user/value_objects.go)

```

package user

import (
    "regexp"
    "errors"
    "golang.org/x/crypto/bcrypt"
)

type Email struct {
    value string
}

func NewEmail(email string) (*Email, error) {
    // validate email format
    emailRegex := regexp.MustCompile(`^[a-z0-9._%+\-]+@[a-z0-9.\-]+\.[a-z]{2,}$`)
    if !emailRegex.MatchString(email) {
        return nil, errors.New("invalid email format")
    }
    return &Email{value: email}, nil
}

func (e Email) String() string { return e.value }

type Password struct {
    hash string
}

func NewPassword(plain string) (*Password, error) {
    if len(plain) < 8 {
        return nil, errors.New("password must be at least 8 characters")
    }
    hash, err := bcrypt.GenerateFromPassword([]byte(plain), bcrypt.DefaultCost)
    if err != nil {
        return nil, err
    }
    return &Password{hash: string(hash)}, nil
}

func (p *Password) Compare(plain string) error {
    return bcrypt.CompareHashAndPassword([]byte(p.hash), []byte(plain))
}

```

3. Domain Events (domain/user/events.go)

```

package user

import (
    "time"
    "github.com/google/uuid"

```

```

)

type DomainEvent interface {
    OccurredAt() time.Time
}

type UserRegisteredEvent struct {
    UserID    uuid.UUID `json:"user_id"`
    Email     string    `json:"email"`
    Occurred  time.Time `json:"occurred_at"`
}

func (e UserRegisteredEvent) OccurredAt() time.Time { return e.Occurred }

func NewUserRegisteredEvent(userID uuid.UUID, email string) UserRegisteredEvent {
    return UserRegisteredEvent{
        UserID:    userID,
        Email:     email,
        Occurred:  time.Now(),
    }
}

type UserVerifiedEvent struct {
    UserID    uuid.UUID `json:"user_id"`
    Occurred  time.Time `json:"occurred_at"`
}

func NewUserVerifiedEvent(userID uuid.UUID) UserVerifiedEvent {
    return UserVerifiedEvent{
        UserID:    userID,
        Occurred:  time.Now(),
    }
}

```

51.3 การสร้าง Application Layer

Use Case (application/user/register.go)

```

package user

import (
    "context"
    "your-project/internal/domain/user"
    "your-project/internal/infrastructure/bus"
)

type RegisterUseCase struct {
    userRepo user.Repository
    eventBus *bus.EventBus
}

```



```

}

type RegisterInput struct {
    Email    string `json:"email" validate:"required,email"`
    Password string `json:"password" validate:"required,min=8"`
    Name     string `json:"name" validate:"required"`
}

type RegisterOutput struct {
    ID      string `json:"id"`
    Email   string `json:"email"`
    Name    string `json:"name"`
}

func NewRegisterUseCase(repo user.Repository, eventBus *bus.EventBus)
*RegisterUseCase {
    return &RegisterUseCase{
        userRepo: repo,
        eventBus: eventBus,
    }
}

func (uc *RegisterUseCase) Execute(ctx context.Context, input RegisterInput)
(*RegisterOutput, error) {
    // 1. ตรวจสอบว่า email ซ้ำหรือไม่
    emailVO, _ := user.NewEmail(input.Email)
    existing, _ := uc.userRepo.FindByEmail(ctx, *emailVO)
    if existing != nil {
        return nil, ErrEmailAlreadyExists
    }

    // 2. สร้าง User entity
    newUser, err := user.NewUser(input.Email, input.Password, input.Name)
    if err != nil {
        return nil, err
    }

    // 3. บันทึกผ่าน repository
    if err := uc.userRepo.Save(ctx, newUser); err != nil {
        return nil, err
    }

    // 4. Dispatch domain events
    for _, event := range newUser.Events() {
        uc.eventBus.Publish(event)
    }
    newUser.ClearEvents()

    // 5. ส่ง output
    return &RegisterOutput{
        ID:      newUser.ID().String(),
        Email:   newUser.Email().String(),
        Name:    newUser.Name(),
    }, nil
}

```

```
    }, nil  
}
```

51.4 การสร้าง Infrastructure Layer

Event Bus (infrastructure/bus/event_bus.go)

```
package bus  
  
import (  
    "context"  
    "sync"  
    "your-project/internal/domain/user"  
)  
  
type EventHandler func(context.Context, user.DomainEvent) error  
  
type EventBus struct {  
    handlers map[string][]EventHandler  
    mu       sync.RWMutex  
}  
  
func NewEventBus() *EventBus {  
    return &EventBus{  
        handlers: make(map[string][]EventHandler),  
    }  
}  
  
func (b *EventBus) Subscribe(eventName string, handler EventHandler) {  
    b.mu.Lock()  
    defer b.mu.Unlock()  
    b.handlers[eventName] = append(b.handlers[eventName], handler)  
}  
  
func (b *EventBus) Publish(event user.DomainEvent) {  
    b.mu.RLock()  
    handlers := b.handlers[eventName(event)]  
    b.mu.RUnlock()  
  
    for _, h := range handlers {  
        go h(context.Background(), event) // async  
    }  
}  
  
func eventName(event user.DomainEvent) string {  
    switch event.(type) {  
    case user.UserRegisteredEvent:  
        return "UserRegistered"  
    case user.UserVerifiedEvent:  
        return "UserVerified"  
    }
```

```

    default:
        return "Unknown"
    }
}

```

51.5 สรุปประโยชน์ของการใช้ DDD ร่วมกับ Go

- **ความชัดเจนของโดเมน:** โค้ดสะท้อนภาษารธุรกิจ (Ubiquitous Language)
- **การแยกหน้าที่:** แต่ละ layer มีความรับผิดชอบชัดเจน ลด coupling
- **ทดสอบง่าย:** domain layer ไม่ขึ้นกับ infrastructure สามารถ unit test ด้วย mock
- **ปรับเปลี่ยน infrastructure ได้:** เปลี่ยนฐานข้อมูลหรือ event bus โดยไม่กระทบโดเมน
- **Go เหมาะสม:** struct, interface, package system ช่วยให้จัดระเบียบตาม bounded context ได้ดี และการทำงาน concurrency ผ่าน goroutine ช่วยให้จัดการ domain events ได้มีประสิทธิภาพ

ภาคที่ 9: การผสานระบบภายนอกและคุณลักษณะเสริม (Advanced Integrations)

ภาคที่ 9: การผสานระบบภายนอก (บทที่ 52–58)

แผนภาพ Data Flow (Flowchart TB)

```

flowchart TB
    subgraph External["ระบบภายนอก (External Systems)"]
        direction LR
        IoT["IoT Sensors"] -->|MQTT| MQTT
        User["ผู้ใช้"] -->|HTTP Request| API
    end

    subgraph Core["ระบบหลัก Go"]
        API["HTTP API / WebSocket"]
        Redis["(Redis)"]
        Rabbit["(RabbitMQ)"]
        Influx["(InfluxDB)"]
    end

    subgraph Processing["การประมวลผล"]
        Workers["Worker Pool"]
        Cache["Cache Layer"]
        Stream["Event Stream"]
    end

    subgraph Notifications["การแจ้งเตือน"]
        SMS["SMS Gateway"]
        LINE["LINE Notify"]
        Discord["Discord Webhook"]
    end

```

```

subgraph Clients["ไคลเอนต์"]
  Browser[เว็บเบราว์เซอร์]
  Mobile[แอปมือถือ]
end

%% Data flow
MQTT -->|Telemetry| Influx
API -->|Read/Write| Redis
API -->|Publish Event| Rabbit
Rabbit -->|Consume| Workers
Workers -->|Store Metrics| Influx
Workers -->|Update Cache| Redis
Influx -->|Query| API
Redis -->|Cached Data| API

API -->|WebSocket| Browser
API -->|WebSocket| Mobile

Workers -->|Alert| SMS
Workers -->|Alert| LINE
Workers -->|Alert| Discord
Influx -->|Threshold Exceeded| Workers

```

คำอธิบายภาษาไทย (แบบละเอียด)

ภาคที่ 9 ครอบคลุมบทที่ 52–58 โดยนำเสนอการผสมผสานระบบภายนอกที่พบได้บ่อยในโลกแห่งความจริง:

- **Redis (บทที่ 52):** Cache, Message Queue
- **RabbitMQ (บทที่ 53):** Message Broker มาตรฐานองค์กร
- **MQTT (บทที่ 54):** IoT และระบบเรียลไทม์
- **InfluxDB (บทที่ 55):** Time-Series Database
- **WebSocket (บทที่ 56):** การสื่อสารแบบ real-time
- **SMS และ LINE Notify (บทที่ 57):** การแจ้งเตือนผ่านมือถือ
- **Discord Webhook (บทที่ 58):** การแจ้งเตือนใน Discord

บทที่ 52: Redis สำหรับ Cache และ Message Queue

Redis คืออะไร?

Redis (Remote Dictionary Server) เป็นฐานข้อมูลแบบ in-memory ที่รวดเร็วสูง นิยมใช้สำหรับ caching, session storage, message queue (Pub/Sub, List), และ real-time analytics

การติดตั้งและเชื่อมต่อ

```
go get github.com/redis/go-redis/v9
```

```

package cache

import (
    "context"
    "encoding/json"
    "time"

    "github.com/redis/go-redis/v9"
)

var rdb *redis.Client

func InitRedis(addr, password string, db int) {
    rdb = redis.NewClient(&redis.Options{
        Addr:      addr,
        Password: password,
        DB:        db,
    })
}

func GetRedis() *redis.Client {
    return rdb
}

```

1. Cache-aside Pattern

```

// GetUserWithCache
func GetUserWithCache(ctx context.Context, userID int) (*User, error) {
    key := fmt.Sprintf("user:%d", userID)

    // 1. Try to get from Redis
    val, err := rdb.Get(ctx, key).Result()
    if err == nil {
        var user User
        if err := json.Unmarshal([]byte(val), &user); err == nil {
            return &user, nil
        }
    }

    // 2. Cache miss: query database
    user, err := db.GetUserByID(ctx, userID)
    if err != nil {
        return nil, err
    }

    // 3. Store in cache (async)
    go func() {
        data, _ := json.Marshal(user)
        rdb.Set(context.Background(), key, data, 10*time.Minute)
    }()
}

```

```

    }()

    return user, nil
}

```

2. Redis Pub/Sub (Message Queue)

Publisher

```

func PublishOrderEvent(ctx context.Context, orderID int) error {
    event := map[string]interface{}{
        "event": "order_created",
        "order_id": orderID,
    }
    data, _ := json.Marshal(event)
    return rdb.Publish(ctx, "orders", data).Err()
}

```

Subscriber

```

func SubscribeOrders() {
    pubsub := rdb.Subscribe(context.Background(), "orders")
    defer pubsub.Close()

    ch := pubsub.Channel()
    for msg := range ch {
        var event map[string]interface{}
        json.Unmarshal([]byte(msg.Payload), &event)
        // process event
        log.Printf("Received event: %v", event)
    }
}

```

3. Redis List as Queue (Producer-Consumer)

Producer (LPush)

```

func EnqueueJob(job interface{}) error {
    data, _ := json.Marshal(job)
    return rdb.LPush(context.Background(), "jobs", data).Err()
}

```

Consumer (BRPop)

```

func Worker() {
    for {
        result, err := rdb.BRPop(context.Background(), 0, "jobs").Result()
        if err != nil {
            continue
        }
        // result[1] is the value
        var job Job
        json.Unmarshal([]byte(result[1]), &job)
        processJob(job)
    }
}

```

บทที่ 53: RabbitMQ – Message Broker มาตรฐานองค์กร

RabbitMQ คืออะไร?

RabbitMQ เป็น message broker ที่ใช้โปรโตคอล AMQP รองรับ routing ที่ซับซ้อน (direct, topic, fanout exchanges), message persistence, และ reliability features

การติดตั้งและเชื่อมต่อ

```
go get github.com/rabbitmq/amqp091-go
```

```

package queue

import (
    "context"
    "log"

    amqp "github.com/rabbitmq/amqp091-go"
)

type RabbitMQ struct {
    conn *amqp.Connection
    ch   *amqp.Channel
}

func NewRabbitMQ(url string) (*RabbitMQ, error) {
    conn, err := amqp.Dial(url)
    if err != nil {
        return nil, err
    }
    ch, err := conn.Channel()
    if err != nil {
        conn.Close()
    }
}

```

```

        return nil, err
    }
    return &RabbitMQ{conn: conn, ch: ch}, nil
}

func (r *RabbitMQ) Close() {
    r.ch.Close()
    r.conn.Close()
}

```

1. Simple Queue (Work Queue)

Producer

```

func (r *RabbitMQ) Publish(queueName string, body []byte) error {
    q, err := r.ch.QueueDeclare(
        queueName, // name
        true,      // durable
        false,     // delete when unused
        false,     // exclusive
        false,     // no-wait
        nil,      // arguments
    )
    if err != nil {
        return err
    }

    return r.ch.Publish(
        "", // exchange
        q.Name, // routing key
        false, // mandatory
        false, // immediate
        amqp.Publishing{
            ContentType: "application/json",
            Body:        body,
            DeliveryMode: amqp.Persistent, // make message persistent
        },
    )
}

```

Consumer

```

func (r *RabbitMQ) Consume(queueName string, handler func([]byte) error) error {
    q, err := r.ch.QueueDeclare(queueName, true, false, false, false, nil)
    if err != nil {
        return err
    }
}

```



```

// prefetch 1 (fair dispatch)
r.ch.Qos(1, 0, false)

msgs, err := r.ch.Consume(
    q.Name, // queue
    "",     // consumer
    false,  // auto-ack (we'll ack manually)
    false,  // exclusive
    false,  // no-local
    false,  // no-wait
    nil,    // args
)
if err != nil {
    return err
}

go func() {
    for d := range msgs {
        if err := handler(d.Body); err != nil {
            // if error, reject and requeue
            d.Nack(false, true)
        } else {
            d.Ack(false)
        }
    }
}()
return nil
}

```

2. Publish/Subscribe (Fanout Exchange)

Producer

```

func (r *RabbitMQ) PublishFanout(exchangeName string, body []byte) error {
    err := r.ch.ExchangeDeclare(
        exchangeName, // name
        "fanout",      // type
        true,           // durable
        false,          // auto-deleted
        false,          // internal
        false,          // no-wait
        nil,           // arguments
    )
    if err != nil {
        return err
    }

    return r.ch.Publish(
        exchangeName, // exchange

```

```

        "", // routing key (ignored for fanout)
        false,
        false,
        amqp.Publishing{
            ContentType: "application/json",
            Body: body,
        },
    )
}

```

Consumer

```

func (r *RabbitMQ) SubscribeFanout(exchangeName string, handler func([]byte)
error) error {
    err := r.ch.ExchangeDeclare(exchangeName, "fanout", true, false, false,
false, nil)
    if err != nil {
        return err
    }

    q, err := r.ch.QueueDeclare(
        "", // name (auto-generated)
        false, // durable
        false, // delete when unused
        true, // exclusive (temporary)
        false, // no-wait
        nil,
    )
    if err != nil {
        return err
    }

    err = r.ch.QueueBind(q.Name, "", exchangeName, false, nil)
    if err != nil {
        return err
    }

    msgs, err := r.ch.Consume(q.Name, "", true, false, false, false, nil)
    if err != nil {
        return err
    }

    go func() {
        for d := range msgs {
            handler(d.Body)
        }
    }()
    return nil
}

```

3. Routing (Direct Exchange) และ Topic Exchange

คล้ายกันแต่ใช้ exchange type ต่างกันและ routing key มี pattern สำหรับ topic

บทที่ 54: MQTT สำหรับ IoT และระบบเรียลไทม์

MQTT คืออะไร?

MQTT (Message Queuing Telemetry Transport) เป็นโปรโตคอล lightweight สำหรับการสื่อสารแบบ publish/subscribe เหมาะสำหรับ IoT, mobile apps, และระบบที่ต้องการการส่งข้อมูลแบบ real-time ด้วยแบนด์วิดท์ต่ำ

การติดตั้งและเชื่อมต่อ

```
go get github.com/eclipse/paho.mqtt.golang
```

```
package mqtt

import (
    "fmt"
    "log"
    "time"

    mqtt "github.com/eclipse/paho.mqtt.golang"
)

type MQTTClient struct {
    client mqtt.Client
}

func NewMQTTClient(broker, clientID, username, password string) (*MQTTClient, error) {
    opts := mqtt.NewClientOptions()
    opts.AddBroker(broker)
    opts.SetClientID(clientID)
    if username != "" {
        opts.SetUsername(username)
        opts.SetPassword(password)
    }
    opts.SetDefaultPublishHandler(func(client mqtt.Client, msg mqtt.Message) {
        log.Printf("Received: %s from topic: %s", msg.Payload(), msg.Topic())
    })
    opts.SetAutoReconnect(true)
    opts.SetConnectTimeout(10 * time.Second)

    client := mqtt.NewClient(opts)
    if token := client.Connect(); token.Wait() && token.Error() != nil {
        return nil, token.Error()
    }
}
```

```

    return &MQTTClient{client: client}, nil
}

func (m *MQTTClient) Publish(topic string, qos byte, retained bool, payload
[]byte) error {
    token := m.client.Publish(topic, qos, retained, payload)
    token.Wait()
    return token.Error()
}

func (m *MQTTClient) Subscribe(topic string, qos byte, handler
mqtt.MessageHandler) error {
    token := m.client.Subscribe(topic, qos, handler)
    token.Wait()
    return token.Error()
}

func (m *MQTTClient) Close() {
    m.client.Disconnect(250)
}

```

ตัวอย่าง: IoT Sensor Data Collector

```

func main() {
    mqttClient, err := NewMQTTClient("tcp://localhost:1883", "go_collector",
    "", "")
    if err != nil {
        panic(err)
    }
    defer mqttClient.Close()

    // Subscribe to all sensor data topics
    err = mqttClient.Subscribe("sensors/+/data", 0, func(client mqtt.Client,
    msg mqtt.Message) {
        var data SensorData
        if err := json.Unmarshal(msg.Payload(), &data); err != nil {
            log.Printf("Invalid JSON: %v", err)
            return
        }
        // Save to InfluxDB or process
        fmt.Printf("Received from %s: temp=%.2f, humidity=%.2f\n",
            data.DeviceID, data.Temp, data.Humidity)
    })
    if err != nil {
        panic(err)
    }

    // Keep the program running
    select {}
}

```

บทที่ 55: InfluxDB – Time-Series Database

InfluxDB คืออะไร?

InfluxDB เป็นฐานข้อมูล time-series ที่ออกแบบมาเพื่อจัดเก็บและสืบค้นข้อมูลที่มี timestamp เช่น ข้อมูลเซ็นเซอร์, metrics, logs มีประสิทธิภาพสูงและมี query language เฉพาะ (Flux หรือ InfluxQL)

การติดตั้งและเชื่อมต่อ

```
go get github.com/influxdata/influxdb-client-go/v2
```

```
package influx

import (
    "context"
    "time"

    influxdb2 "github.com/influxdata/influxdb-client-go/v2"
    "github.com/influxdata/influxdb-client-go/v2/api"
)

type InfluxDB struct {
    client    influxdb2.Client
    writeAPI  api.WriteAPIBlocking
    queryAPI  api.QueryAPI
    org       string
    bucket    string
}

func NewInfluxDB(url, token, org, bucket string) *InfluxDB {
    client := influxdb2.NewClient(url, token)
    writeAPI := client.WriteAPIBlocking(org, bucket)
    queryAPI := client.QueryAPI(org)
    return &InfluxDB{
        client:    client,
        writeAPI:  writeAPI,
        queryAPI:  queryAPI,
        org:       org,
        bucket:    bucket,
    }
}

func (i *InfluxDB) WritePoint(measurement string, tags map[string]string,
    fields map[string]interface{}, t time.Time) error {
    p := influxdb2.NewPoint(measurement, tags, fields, t)
    return i.writeAPI.WritePoint(context.Background(), p)
}
```

```

}

func (i *InfluxDB) Query(flux string) ([]api.FluxRecord, error) {
    result, err := i.queryAPI.Query(context.Background(), flux)
    if err != nil {
        return nil, err
    }
    var records []api.FluxRecord
    for result.Next() {
        records = append(records, result.Record())
    }
    return records, result.Err()
}

```

ตัวอย่างการใช้งาน

```

// Write sensor data
influx.WritePoint("temperature",
    map[string]string{"device": "sensor1", "location": "room101"},
    map[string]interface{}{"value": 25.6},
    time.Now(),
)

// Query average temperature for last hour
flux := `from(bucket:"mybucket")
    |> range(start: -1h)
    |> filter(fn: (r) => r._measurement == "temperature")
    |> filter(fn: (r) => r.device == "sensor1")
    |> aggregateWindow(every: 5m, fn: mean)`
records, err := influx.Query(flux)

```

บทที่ 56: WebSocket และ Socket.IO

WebSocket พื้นฐาน

WebSocket เป็นโปรโตคอลที่ช่วยให้ client-server สื่อสารแบบ full-duplex แทน TCP เหมาะสำหรับ real-time applications เช่น chat, live notifications

การติดตั้ง gorilla/websocket

```
go get github.com/gorilla/websocket
```

ตัวอย่าง WebSocket Server

```

package websocket

import (
    "log"
    "net/http"
    "sync"

    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { return true },
}

type Client struct {
    conn *websocket.Conn
    send chan []byte
    hub *Hub
}

type Hub struct {
    clients    map[*Client]bool
    broadcast  chan []byte
    register   chan *Client
    unregister chan *Client
    mu         sync.RWMutex
}

func NewHub() *Hub {
    return &Hub{
        clients:    make(map[*Client]bool),
        broadcast:  make(chan []byte),
        register:   make(chan *Client),
        unregister: make(chan *Client),
    }
}

func (h *Hub) Run() {
    for {
        select {
        case client := <-h.register:
            h.mu.Lock()
            h.clients[client] = true
            h.mu.Unlock()
        case client := <-h.unregister:
            h.mu.Lock()
            if _, ok := h.clients[client]; ok {
                delete(h.clients, client)
                close(client.send)
            }
            h.mu.Unlock()
        case message := <-h.broadcast:

```

```

        h.mu.RLock()
        for client := range h.clients {
            select {
            case client.send <- message:
            default:
                close(client.send)
                delete(h.clients, client)
            }
        }
        h.mu.RUnlock()
    }
}

func (c *Client) readPump() {
    defer func() {
        c.hub.unregister <- c
        c.conn.Close()
    }()
    for {
        _, msg, err := c.conn.ReadMessage()
        if err != nil {
            break
        }
        c.hub.broadcast <- msg
    }
}

func (c *Client) writePump() {
    defer c.conn.Close()
    for {
        select {
        case msg, ok := <-c.send:
            if !ok {
                c.conn.WriteMessage(websocket.CloseMessage, []byte{})
                return
            }
            if err := c.conn.WriteMessage(websocket.TextMessage, msg); err !=
nil {
                return
            }
        }
    }
}

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { return true },
}

func ServeWs(hub *Hub, w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        log.Println(err)
    }
}

```



```

        return
    }
    client := &Client{conn: conn, send: make(chan []byte, 256), hub: hub}
    hub.register <- client

    go client.writePump()
    go client.readPump()
}

```

การใช้งานใน main

```

hub := NewHub()
go hub.Run()
http.HandleFunc("/ws", func(w http.ResponseWriter, r *http.Request) {
    ServeWs(hub, w, r)
})

```

บทที่ 57: การส่ง SMS และ LINE Notify

1. ส่ง SMS ผ่าน Twilio (ตัวอย่าง)

```

package sms

import (
    "github.com/twilio/twilio-go"
    twilioApi "github.com/twilio/twilio-go/rest/api/v2010"
)

type TwilioSender struct {
    client      *twilio.RestClient
    fromNumber string
}

func NewTwilioSender(accountSid, authToken, fromNumber string) *TwilioSender {
    client := twilio.NewRestClientWithParams(twilio.ClientParams{
        Username: accountSid,
        Password: authToken,
    })
    return &TwilioSender{client: client, fromNumber: fromNumber}
}

func (t *TwilioSender) Send(to, message string) error {
    params := &twilioApi.CreateMessageParams{}
    params.SetTo(to)
    params.SetFrom(t.fromNumber)
    params.SetBody(message)
}

```

```
_, err := t.client.Api.CreateMessage(params)
return err
}
```

2. LINE Notify – การแจ้งเตือนผ่าน LINE

LINE Notify ให้บริการแจ้งเตือนฟรี โดยต้องสร้าง token ที่ <https://notify-bot.line.me/>

```
package linenotify

import (
    "bytes"
    "fmt"
    "io"
    "mime/multipart"
    "net/http"
    "os"
)

type LINE struct {
    token string
}

func NewLINE(token string) *LINE {
    return &LINE{token: token}
}

func (l *LINE) SendMessage(message string) error {
    url := "https://notify-api.line.me/api/notify"
    data := []byte(fmt.Sprintf("message=%s", message))
    req, err := http.NewRequest("POST", url, bytes.NewBuffer(data))
    if err != nil {
        return err
    }
    req.Header.Set("Authorization", "Bearer "+l.token)
    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")

    client := &http.Client{}
    resp, err := client.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("LINE Notify returned status %d", resp.StatusCode)
    }
    return nil
}

func (l *LINE) SendImage(message, imagePath string) error {
```

```

var b bytes.Buffer
w := multipart.NewWriter(&b)

w.WriteField("message", message)

file, err := os.Open(imagePath)
if err != nil {
    return err
}
defer file.Close()

part, err := w.CreateFormFile("imageFile", imagePath)
if err != nil {
    return err
}
io.Copy(part, file)
w.Close()

req, err := http.NewRequest("POST", "https://notify-
api.line.me/api/notify", &b)
if err != nil {
    return err
}
req.Header.Set("Authorization", "Bearer "+l.token)
req.Header.Set("Content-Type", w.FormDataContentType())

client := &http.Client{}
resp, err := client.Do(req)
if err != nil {
    return err
}
defer resp.Body.Close()
if resp.StatusCode != http.StatusOK {
    return fmt.Errorf("LINE Notify returned status %d", resp.StatusCode)
}
return nil
}

```

บทที่ 58: Discord Webhook สำหรับแจ้งเตือน

Discord Webhook คืออะไร?

Discord Webhook เป็นวิธีส่งข้อความอัตโนมัติไปยัง channel ใน Discord โดยไม่ต้องใช้ bot token เหมาะสำหรับการแจ้งเตือนจากระบบ (เช่น deployment status, error logs)

การส่งข้อความผ่าน Go

```
package discord
```

```

import (
    "bytes"
    "encoding/json"
    "fmt"
    "net/http"
)

type Webhook struct {
    url string
}

type WebhookPayload struct {
    Content  string `json:"content,omitempty"`
    Username string `json:"username,omitempty"`
    AvatarURL string `json:"avatar_url,omitempty"`
    Embeds   []Embed `json:"embeds,omitempty"`
}

type Embed struct {
    Title      string `json:"title,omitempty"`
    Description string `json:"description,omitempty"`
    Color      int    `json:"color,omitempty"` // decimal color
    Fields     []Field `json:"fields,omitempty"`
    Timestamp  string `json:"timestamp,omitempty"`
}

type Field struct {
    Name   string `json:"name"`
    Value  string `json:"value"`
    Inline bool   `json:"inline,omitempty"`
}

func NewWebhook(url string) *Webhook {
    return &Webhook{url: url}
}

func (w *Webhook) SendMessage(content string) error {
    payload := WebhookPayload{Content: content}
    return w.send(payload)
}

func (w *Webhook) SendEmbed(embed Embed) error {
    payload := WebhookPayload{Embeds: []Embed{embed}}
    return w.send(payload)
}

func (w *Webhook) send(payload WebhookPayload) error {
    data, err := json.Marshal(payload)
    if err != nil {
        return err
    }
    resp, err := http.Post(w.url, "application/json", bytes.NewBuffer(data))
    if err != nil {

```

```

        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK && resp.StatusCode !=
http.StatusNoContent {
        return fmt.Errorf("discord webhook returned status %d",
resp.StatusCode)
    }
    return nil
}

```

ตัวอย่างการแจ้งเตือน Deployment

```

webhook := discord.NewWebhook("https://discord.com/api/webhooks/...")

// Simple message
webhook.SendMessage("🚀 Deployment started: v1.2.3")

// Embed with details
embed := discord.Embed{
    Title:      "Deployment Successful",
    Description: "Version v1.2.3 has been deployed to production.",
    Color:      0x00ff00, // green
    Fields: []discord.Field{
        {Name: "Environment", Value: "Production", Inline: true},
        {Name: "Version", Value: "v1.2.3", Inline: true},
        {Name: "Deployed By", Value: "CI/CD", Inline: false},
    },
    Timestamp: time.Now().Format(time.RFC3339),
}
webhook.SendEmbed(embed)

```

สรุป

ภาคที่ 9 ได้นำเสนอการผสานระบบภายนอกที่จำเป็นสำหรับแอปพลิเคชันสมัยใหม่:

ระบบ	การใช้งานหลัก	ตัวอย่าง
Redis	Cache, Message Queue	Caching user data, job queue
RabbitMQ	Message Broker	Async email sending, order processing
MQTT	IoT Communication	Sensor data collection
InfluxDB	Time-Series Database	Metrics storage, telemetry
WebSocket	Real-time Communication	Chat, live notifications
SMS	Mobile alerts	OTP, transaction alerts

ระบบ	การใช้งานหลัก	ตัวอย่าง
LINE Notify	Instant messaging	System alerts, notifications
Discord	Team communication	Deployment status, error logs

การรวมระบบเหล่านี้เข้ากับ Go application ช่วยให้สามารถสร้างแอปพลิเคชันที่มีความสามารถครบครัน รองรับการทำงานแบบ real-time, event-driven, และการแจ้งเตือนหลากหลายช่องทาง

บทที่ 52: Redis สำหรับ Cache และ Message Queue

Redis เป็นฐานข้อมูลแบบ in-memory ที่มีความเร็วสูง รองรับโครงสร้างข้อมูลที่หลากหลาย เช่น Strings, Hashes, Lists, Sets, Sorted Sets, Streams และ Pub/Sub ทำให้สามารถนำไปใช้ได้ทั้งเป็น Cache และ Message Queue

52.1 การติดตั้งและเชื่อมต่อกับ Redis ใน Go

```
go get github.com/redis/go-redis/v9
```

```
package main

import (
    "context"
    "fmt"
    "time"

    "github.com/redis/go-redis/v9"
)

func main() {
    // สร้าง client
    rdb := redis.NewClient(&redis.Options{
        Addr:      "localhost:6379",
        Password: "", // no password set
        DB:        0, // use default DB
        PoolSize:  10, // connection pool size
    })

    ctx := context.Background()

    // ทดสอบเชื่อมต่อ
    pong, err := rdb.Ping(ctx).Result()
    if err != nil {
        panic(err)
    }
    fmt.Println("Redis connected:", pong)
}
```

52.2 Redis เป็น Cache (CRUD)

```
type Cache struct {
    client *redis.Client
    ttl    time.Duration
}

func NewCache(client *redis.Client, ttl time.Duration) *Cache {
    return &Cache{client: client, ttl: ttl}
}

func (c *Cache) Set(ctx context.Context, key string, value interface{}) error {
    data, err := json.Marshal(value)
    if err != nil {
        return err
    }
    return c.client.Set(ctx, key, data, c.ttl).Err()
}

func (c *Cache) Get(ctx context.Context, key string, dest interface{}) error {
    data, err := c.client.Get(ctx, key).Bytes()
    if err != nil {
        if err == redis.Nil {
            return ErrCacheMiss
        }
        return err
    }
    return json.Unmarshal(data, dest)
}

func (c *Cache) Delete(ctx context.Context, key string) error {
    return c.client.Del(ctx, key).Err()
}

var ErrCacheMiss = errors.New("cache: key not found")
```

การใช้ Cache Pattern (Cache-Aside)

```
func GetUser(ctx context.Context, cache *Cache, repo UserRepository, id string)
(*User, error) {
    var user User
    err := cache.Get(ctx, "user:"+id, &user)
    if err == nil {
        return &user, nil
    }
    if err != ErrCacheMiss {
        return nil, err
    }
}
```

```

// cache miss → query DB
user, err := repo.FindByID(ctx, id)
if err != nil {
    return nil, err
}

// เก็บ cache (แบบ async เพื่อไม่ให้กระทบ latency)
go cache.Set(context.Background(), "user:"+id, user)
return &user, nil
}

```

52.3 Redis Pub/Sub (Message Queue แบบเบา)

Redis Pub/Sub เหมาะสำหรับการส่งข้อความระหว่าง components โดยไม่ต้องมี broker ที่หนักหน่วง

```

// Publisher
func publishMessage(rdb *redis.Client, channel string, message string) error {
    return rdb.Publish(context.Background(), channel, message).Err()
}

// Subscriber
func subscribeChannel(rdb *redis.Client, channel string, handler func(string)) {
    pubsub := rdb.Subscribe(context.Background(), channel)
    defer pubsub.Close()

    ch := pubsub.Channel()
    for msg := range ch {
        handler(msg.Payload)
    }
}

```

52.4 Redis Streams (Message Queue ที่มีความทนทาน)

Redis Streams ให้คุณสมบัติคล้าย Kafka: การเก็บข้อความ, consumer groups, acknowledgment

```

// ส่งข้อความเข้าสู่ stream
func addToStream(rdb *redis.Client, stream string, values
map[string]interface{}) (string, error) {
    return rdb.XAdd(context.Background(), &redis.XAddArgs{
        Stream: stream,
        Values: values,
    }).Result()
}

// อ่านข้อความจาก stream (consumer group)
func readFromStream(rdb *redis.Client, stream, group, consumer string) error {
    // สร้าง consumer group ถ้ายังไม่มี
}

```



```

rdb.XGroupCreateMkStream(context.Background(), stream, group, "0").Err()

for {
    streams, err := rdb.XReadGroup(context.Background(),
&redis.XReadGroupArgs{
        Group:    group,
        Consumer: consumer,
        Streams:  []string{stream, ">"},
        Count:    10,
        Block:    0,
    }).Result()

    if err != nil {
        return err
    }

    for _, stream := range streams {
        for _, message := range stream.Messages {
            // ประมวลผลข้อความ
            fmt.Printf("Message ID: %s, Values: %v\n", message.ID,
message.Values)

            // ack ว่าได้ประมวลผลแล้ว
            rdb.XAck(context.Background(), stream, group,
message.ID).Result()
        }
    }
}

```

บทที่ 53: RabbitMQ – Message Broker มาตรฐานองค์กร

RabbitMQ เป็น message broker ที่รองรับ routing ที่ยืดหยุ่น (exchanges, queues, bindings) เหมาะกับระบบที่ต้องการความน่าเชื่อถือสูง

53.1 การติดตั้งและเชื่อมต่อ

```
go get github.com/rabbitmq/amqp091-go
```

```

package main

import (
    "log"
    "github.com/rabbitmq/amqp091-go"
)

func main() {

```

```

conn, err := amqp091.Dial("amqp://guest:guest@localhost:5672/")
if err != nil {
    log.Fatal(err)
}
defer conn.Close()

ch, err := conn.Channel()
if err != nil {
    log.Fatal(err)
}
defer ch.Close()
}

```

53.2 การส่งและรับข้อความ (Simple Queue)

```

// ประกาศ queue
q, err := ch.QueueDeclare(
    "hello", // name
    false,   // durable
    false,   // delete when unused
    false,   // exclusive
    false,   // no-wait
    nil,     // arguments
)

// ส่งข้อความ
err = ch.Publish(
    "", // exchange
    q.Name, // routing key
    false, // mandatory
    false, // immediate
    amqp091.Publishing{
        ContentType: "text/plain",
        Body: []byte("Hello World"),
    },
)

// รับข้อความ
msgs, err := ch.Consume(
    q.Name, // queue
    "",     // consumer
    true,   // auto-ack
    false,  // exclusive
    false,  // no-local
    false,  // no-wait
    nil,    // args
)

for msg := range msgs {

```

```
    log.Printf("Received: %s", msg.Body)
}
```

53.3 Work Queues (Task Distribution)

```
// ประกาศ durable queue
q, err := ch.QueueDeclare(
    "task_queue", // name
    true,         // durable
    false,        // delete when unused
    false,        // exclusive
    false,        // no-wait
    nil,
)

// ส่งข้อความแบบ persistent
err = ch.Publish(
    "", // exchange
    q.Name, // routing key
    false, // mandatory
    false,
    amqp091.Publishing{
        DeliveryMode: amqp091.Persistent,
        ContentType:  "text/plain",
        Body:         []byte(body),
    },
)

// Consumer: ตั้ง prefetch count เพื่อกระจายงานอย่างยุติธรรม
err = ch.Qos(
    1, // prefetch count
    0, // prefetch size
    false, // global
)
```

53.4 Publish/Subscribe ด้วย Fanout Exchange

```
// ประกาศ exchange แบบ fanout
err = ch.ExchangeDeclare(
    "logs", // name
    "fanout", // type
    true,    // durable
    false,   // auto-deleted
    false,   // internal
    false,   // no-wait
    nil,
)
```

```
// ส่งข้อความไปยัง exchange
err = ch.Publish(
    "logs", // exchange
    "",     // routing key
    false,  // mandatory
    false,
    amqp091.Publishing{
        ContentType: "text/plain",
        Body:        []byte("log message"),
    },
)

// สร้าง queue ขั้วคราวและ bind กับ exchange
q, err := ch.QueueDeclare(
    "", // name (auto-generate)
    false, // durable
    false, // delete when unused
    true,  // exclusive
    false, // no-wait
    nil,
)
err = ch.QueueBind(
    q.Name, // queue name
    "",     // routing key
    "logs", // exchange
    false,
    nil,
)
)
```

53.5 Routing (Direct Exchange) และ Topics

```
// Direct exchange
err = ch.ExchangeDeclare(
    "direct_logs", // name
    "direct",      // type
    true, false, false, false, nil,
)

// ส่งโดยระบุ routing key (เช่น "info", "warning")
err = ch.Publish(
    "direct_logs",
    "error",
    false, false,
    amqp091.Publishing{Body: []byte("error log")},
)

// รับเฉพาะ routing key ที่สนใจ
err = ch.QueueBind(q.Name, "error", "direct_logs", false, nil)

// Topic exchange (ใช้ pattern)
```

```

err = ch.ExchangeDeclare(
    "topic_logs",
    "topic",
    true, false, false, false, nil,
)
// routing key pattern: "*.critical", "kern.*", "#"
err = ch.QueueBind(q.Name, "*.critical", "topic_logs", false, nil)

```

53.6 การทำ RPC (Remote Procedure Call) ด้วย RabbitMQ

```

// Client: สร้าง callback queue และส่ง request
corrId := randomString(32)
replyTo := q.Name // queue ที่ client สร้างไว้

err = ch.Publish(
    "", // exchange
    "rpc_queue", // routing key
    false, false,
    amqp091.Publishing{
        ContentType: "text/plain",
        CorrelationId: corrId,
        ReplyTo:      replyTo,
        Body:          []byte("10"),
    },
)

// Client รับ response ผ่าน reply queue
for msg := range msgs {
    if msg.CorrelationId == corrId {
        // response มาถึง
    }
}

// Server: รับ request และส่ง response
for msg := range msgs {
    body := string(msg.Body)
    // คำนวณผล
    response := fmt.Sprintf("Result: %s", body)

    err = ch.Publish(
        "", // exchange
        msg.ReplyTo, // routing key ไปยัง client's reply queue
        false, false,
        amqp091.Publishing{
            ContentType: "text/plain",
            CorrelationId: msg.CorrelationId,
            Body:          []byte(response),
        },
    )
}

```

```
msg.Ack(false)
}
```

บทที่ 54: MQTT สำหรับ IoT และระบบเรียลไทม์

MQTT เป็น lightweight messaging protocol เหมาะสำหรับอุปกรณ์ IoT, เซ็นเซอร์, และระบบที่มีแบนด์วิดท์จำกัด

54.1 การติดตั้งและเชื่อมต่อ MQTT Broker (เช่น Mosquitto)

```
go get github.com/eclipse/paho.mqtt.golang
```

```
package main

import (
    "fmt"
    "log"
    "time"

    mqtt "github.com/eclipse/paho.mqtt.golang"
)

func main() {
    opts := mqtt.NewClientOptions()
    opts.AddBroker("tcp://localhost:1883")
    opts.SetClientID("go-mqtt-client")
    opts.SetUsername("user")
    opts.SetPassword("pass")
    opts.SetDefaultPublishHandler(func(client mqtt.Client, msg mqtt.Message) {
        fmt.Printf("Received: %s from %s\n", msg.Payload(), msg.Topic())
    })

    client := mqtt.NewClient(opts)
    if token := client.Connect(); token.Wait() && token.Error() != nil {
        log.Fatal(token.Error())
    }
    defer client.Disconnect(250)

    // Subscribe
    token := client.Subscribe("sensor/temperature", 1, nil)
    token.Wait()

    // Publish
    token = client.Publish("sensor/temperature", 1, false, "25.6")
    token.Wait()

    time.Sleep(5 * time.Second)
}
```

54.2 QoS Levels

- **0** – At most once (fire and forget)
- **1** – At least once (อาจ duplicate)
- **2** – Exactly once (สูงสุด)

```
client.Publish("topic", 1, false, payload) // QoS 1
```

54.3 Retained Messages

ข้อความที่ถูกเก็บไว้ที่ broker และจะส่งให้ subscriber ใหม่ทันทีที่ subscribe

```
// ส่ง retained message
client.Publish("status/device", 1, true, "online")

// subscriber ใหม่จะได้รับ "online" ทันที
```

54.4 Last Will and Testament (LWT)

ข้อความที่ broker จะส่งเมื่อ client disconnect โดยไม่ clean

```
opts.SetWill("device/status", "offline", 1, true)
```

54.5 ตัวอย่าง: IoT Gateway ที่รับข้อมูลจากเซ็นเซอร์และส่งไปยัง Kafka หรือ InfluxDB

```
func main() {
    // MQTT subscriber
    client := connectMQTT()
    client.Subscribe("sensors+/data", 1, func(c mqtt.Client, msg mqtt.Message) {
        // แยก topic เช่น sensors/temperature/data
        parts := strings.Split(msg.Topic(), "/")
        sensorType := parts[1]

        // แปลง payload เป็น float
        value, _ := strconv.ParseFloat(string(msg.Payload), 64)

        // บันทึกลง InfluxDB
        writeToInfluxDB(sensorType, value)

        // ส่งไป Kafka สำหรับ analytics
        sendToKafka(sensorType, value)
    })
}
```

```
    })  
    select {}  
}
```

บทที่ 55: InfluxDB – Time-Series Database

InfluxDB เป็นฐานข้อมูลที่ออกแบบมาเฉพาะสำหรับ time-series data (IoT metrics, monitoring data, sensor readings)

55.1 การเชื่อมต่อและเขียนข้อมูล (InfluxDB 2.x)

```
go get github.com/influxdata/influxdb-client-go/v2
```

```
package main  
  
import (  
    "context"  
    "fmt"  
    "time"  
  
    influxdb2 "github.com/influxdata/influxdb-client-go/v2"  
)  
  
func main() {  
    client := influxdb2.NewClient("http://localhost:8086", "my-token")  
    defer client.Close()  
  
    writeAPI := client.WriteAPIBlocking("my-org", "my-bucket")  
  
    // สร้าง point  
    p := influxdb2.NewPoint(  
        "temperature",  
        map[string]string{"sensor": "sensor1", "unit": "celsius"},  
        map[string]interface{}{"value": 25.6},  
        time.Now(),  
    )  
  
    // เขียน  
    err := writeAPI.WritePoint(context.Background(), p)  
    if err != nil {  
        fmt.Println("Write error:", err)  
    }  
}
```

55.2 การ query ข้อมูล


```

queryAPI := client.QueryAPI("my-org")

fluxQuery := `
    from(bucket:"my-bucket")
    |> range(start: -1h)
    |> filter(fn: (r) => r._measurement == "temperature")
    |> filter(fn: (r) => r.sensor == "sensor1")
    |> aggregateWindow(every: 1m, fn: mean)
`

result, err := queryAPI.Query(context.Background(), fluxQuery)
if err == nil {
    for result.Next() {
        record := result.Record()
        fmt.Printf("Time: %s, Value: %v\n", record.Time(), record.Value())
    }
}

```

55.3 ตัวอย่าง: ระบบ monitor CPU และ Memory

```

type MetricsCollector struct {
    client influxdb2.Client
    org     string
    bucket  string
}

func NewMetricsCollector(url, token, org, bucket string) *MetricsCollector {
    return &MetricsCollector{
        client: influxdb2.NewClient(url, token),
        org:    org,
        bucket: bucket,
    }
}

func (m *MetricsCollector) RecordCPU(usage float64) error {
    writeAPI := m.client.WriteAPIBlocking(m.org, m.bucket)
    p := influxdb2.NewPoint(
        "cpu",
        map[string]string{"host": "server01"},
        map[string]interface{}{"usage": usage},
        time.Now(),
    )
    return writeAPI.WritePoint(context.Background(), p)
}

func (m *MetricsCollector) RecordMemory(used, total uint64) error {
    writeAPI := m.client.WriteAPIBlocking(m.org, m.bucket)
    p := influxdb2.NewPoint(
        "memory",

```

```

        map[string]string{"host": "server01"},
        map[string]interface{}{"used_bytes": used, "total_bytes": total},
        time.Now(),
    )
    return writeAPI.WritePoint(context.Background(), p)
}

```

บทที่ 56: WebSocket และ Socket.IO

WebSocket ช่วยให้มี connection แบบ full-duplex เหมาะกับ real-time feature เช่น chat, notifications, live dashboard

56.1 WebSocket พื้นฐานด้วย Gorilla WebSocket

```
go get github.com/gorilla/websocket
```

```

package main

import (
    "log"
    "net/http"

    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { return true },
}

var clients = make(map[*websocket.Conn]bool)
var broadcast = make(chan []byte)

func wsHandler(w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        log.Print(err)
        return
    }
    defer conn.Close()
    clients[conn] = true

    for {
        _, msg, err := conn.ReadMessage()
        if err != nil {
            delete(clients, conn)
            break
        }
    }
}

```

```

        broadcast <- msg
    }
}

func handleMessages() {
    for {
        msg := <-broadcast
        for client := range clients {
            err := client.WriteMessage(websocket.TextMessage, msg)
            if err != nil {
                client.Close()
                delete(clients, client)
            }
        }
    }
}

func main() {
    http.HandleFunc("/ws", wsHandler)
    go handleMessages()
    http.ListenAndServe(":8080", nil)
}

```

56.2 Client-side in Go

```

func main() {
    c, _, err := websocket.DefaultDialer.Dial("ws://localhost:8080/ws", nil)
    if err != nil {
        log.Fatal(err)
    }
    defer c.Close()

    go func() {
        for {
            _, msg, err := c.ReadMessage()
            if err != nil {
                log.Println("read error:", err)
                return
            }
            fmt.Printf("Received: %s\n", msg)
        }
    }()

    for {
        msg := bufio.NewReader(os.Stdin)
        text, _ := msg.ReadString('\n')
        c.WriteMessage(websocket.TextMessage, []byte(text))
    }
}

```

56.3 Socket.IO (Alternative)

Socket.IO ช่วยให้การจัดการ connection ง่ายขึ้น (reconnection, fallback)

```
go get github.com/googollee/go-socket.io
```

```
server, _ := socketio.NewServer(nil)

server.OnConnect("/", func(s socketio.Conn) error {
    s.SetContext("")
    s.Join("room")
    return nil
})

server.OnEvent("/", "chat message", func(s socketio.Conn, msg string) {
    s.Emit("chat message", "echo: "+msg)
})

server.OnDisconnect("/", func(s socketio.Conn, reason string) {
    log.Println("disconnected:", reason)
})

http.Handle("/socket.io/", server)
http.ListenAndServe(":8080", nil)
```

บทที่ 57: การส่ง SMS และ LINE Notify

57.1 ส่ง SMS ผ่าน Twilio

```
go get github.com/twilio/twilio-go
```

```
import (
    "github.com/twilio/twilio-go"
    "github.com/twilio/twilio-go/rest/api/v2010"
)

func sendSMS(accountSid, authToken, from, to, body string) error {
    client := twilio.NewRestClientWithParams(twilio.ClientParams{
        Username: accountSid,
        Password: authToken,
    })
    params := &api.CreateMessageParams{}
    params.SetTo(to)
```

```

params.SetFrom(from)
params.SetBody(body)

_, err := client.Api.CreateMessage(params)
return err
}

```

57.2 LINE Notify (ง่ายที่สุด)

LINE Notify เป็นบริการแจ้งเตือนฟรี สามารถส่งข้อความผ่าน HTTP API

ขั้นตอน:

1. ลงทะเบียนที่ notify-bot.line.me
2. สร้าง token สำหรับกลุ่มหรือตัวคุณเอง

```

func sendLineNotify(token, message string) error {
    url := "https://notify-api.line.me/api/notify"
    payload := strings.NewReader("message=" + url.QueryEscape(message))
    req, _ := http.NewRequest("POST", url, payload)
    req.Header.Set("Authorization", "Bearer "+token)
    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")

    client := &http.Client{}
    resp, err := client.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("LINE Notify returned status %d", resp.StatusCode)
    }
    return nil
}

```

57.3 ส่งข้อความผ่าน LINE Official Account (Messaging API)

```
go get github.com/line/line-bot-sdk-go/v7/linebot
```

```

func main() {
    bot, err := linebot.New("YOUR_CHANNEL_SECRET", "YOUR_CHANNEL_ACCESS_TOKEN")
    if err != nil {
        log.Fatal(err)
    }
}

```

```

// ส่งข้อความ push
_, err = bot.PushMessage("USER_ID", linebot.NewTextMessage("Hello")).Do()
if err != nil {
    log.Fatal(err)
}

// หรือ reply ต่อ request
http.HandleFunc("/callback", func(w http.ResponseWriter, r *http.Request) {
    events, err := bot.ParseRequest(r)
    if err != nil {
        if err == linebot.ErrInvalidSignature {
            w.WriteHeader(400)
        } else {
            w.WriteHeader(500)
        }
        return
    }
    for _, event := range events {
        if event.Type == linebot.EventTypeMessage {
            if _, ok := event.Message.(*linebot.TextMessage); ok {
                bot.ReplyMessage(event.ReplyToken,
                    linebot.NewTextMessage("Hello!")).Do()
            }
        }
    }
})
http.ListenAndServe(":8080", nil)
}

```

บทที่ 58: Discord Webhook สำหรับแจ้งเตือน

Discord Webhook ใช้ส่งข้อความแจ้งเตือนเข้า channel ได้ง่าย โดยไม่ต้องสร้าง bot

58.1 การสร้าง Webhook ใน Discord

1. เปิด Discord channel ที่ต้องการ
2. คลิก Edit Channel → Integrations → Webhooks → Create Webhook
3. คัดลอก URL ที่ได้

58.2 ส่งข้อความด้วย Go

```

type DiscordWebhook struct {
    URL      string
    Content  string `json:"content"`
    Username string `json:"username,omitempty"`
    Embeds  []Embed `json:"embeds,omitempty"`
}

type Embed struct {

```

```

    Title      string `json:"title"`
    Description string `json:"description"`
    Color      int    `json:"color"`
    Fields     []Field `json:"fields"`
}

type Field struct {
    Name    string `json:"name"`
    Value   string `json:"value"`
    Inline  bool   `json:"inline,omitempty"`
}

func SendDiscordWebhook(url string, content string) error {
    data := DiscordWebhook{Content: content}
    payload, _ := json.Marshal(data)

    resp, err := http.Post(url, "application/json", bytes.NewBuffer(payload))
    if err != nil {
        return err
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusNoContent && resp.StatusCode !=
http.StatusOK {
        return fmt.Errorf("discord webhook returned status %d",
resp.StatusCode)
    }
    return nil
}

```

58.3 การส่งข้อความแบบมี Embed

```

func SendAlert(title, description string, color int) {
    embed := Embed{
        Title:      title,
        Description: description,
        Color:      color, // 0xff0000 = red
    }
    data := DiscordWebhook{
        Username: "Alert Bot",
        Embeds:   []Embed{embed},
    }
    // ...
}

```

ภาคที่ 10: เหมเพลต กระบวนการพัฒนา และตัวอย่างโค้ด

ภาคที่ 10: เทมเพลต กระบวนการพัฒนา และตัวอย่างโค้ด (บทที่ 59–63)

แผนภาพ Data Flow (Flowchart TB)

```
flowchart TB
    subgraph FullStack["บทที่ 59: Full-stack Example"]
        direction TB
        FS1["Frontend (React/HTML)<br/>- User interface<br/>- WebSocket client"]
        FS2["Backend (Go API)<br/>- REST endpoints<br/>- WebSocket hub<br/>- Business logic"]
        FS3["Database (PostgreSQL)<br/>- Users, Orders<br/>- GORM models"]
        FS4["Cache (Redis)<br/>- Session store<br/>- Rate limiting"]
        FS5["Message Queue (RabbitMQ)<br/>- Async tasks<br/>- Email sending"]
    end

    subgraph Templates["บทที่ 60-61: Templates & Checklists"]
        TT1["Task List Template<br/>- Phase-based checklist<br/>- Feature development"]
        TT2["Checklist Template<br/>- Code quality<br/>- Security<br/>- Performance<br/>- Testing<br/>- Deployment"]
    end

    subgraph Workflow["บทที่ 62: Workflow Diagram"]
        WF1["Feature Development Flow<br/>Analyze → Design → Implement → Test → Deploy"]
        WF2["Authentication Flow<br/>Login → JWT → Redis refresh"]
        WF3["Order Processing Flow<br/>Place Order → Stock Check → Payment → Notification"]
    end

    subgraph Config["บทที่ 63: mop Config"]
        C1["Configuration Management<br/>- YAML files<br/>- Environment variables<br/>- Defaults"]
        C2["Load & Validate<br/>- viper<br/>- Struct tags"]
    end

    FullStack --> Templates
    FullStack --> Workflow
    FullStack --> Config
    Templates --> Workflow
    Config --> FullStack
```

คำอธิบายภาษาไทย (แบบละเอียด)

ภาคที่ 10 ครอบคลุมบทที่ 59–63 โดยรวบรวมเทมเพลต กระบวนการพัฒนา และตัวอย่างโค้ดครบวงจรที่สามารถนำไปปรับใช้ในโปรเจกต์จริงได้ทันที

- **บทที่ 59:** ตัวอย่างโค้ดครบวงจร (Full-stack Example) – สร้างแอปพลิเคชัน REST API + WebSocket + Database + Queue + Cache
- **บทที่ 60:** Task List Template – เทมเพลตสำหรับติดตามงานในแต่ละเฟส
- **บทที่ 61:** Checklist Template – รายการตรวจสอบคุณภาพโค้ดและกระบวนการ
- **บทที่ 62:** แผนภาพการทำงาน (Workflow Diagram) – แผนภาพ Mermaid สำหรับอธิบายการทำงานของระบบ
- **บทที่ 63:** mop Config – การจัดการ Configuration ด้วย viper และ environment variables

บทที่ 59: ตัวอย่างโค้ดครบวงจร (Full-stack Example)

1. โครงสร้างโปรเจกต์ (อ้างอิง Clean Architecture + DDD)

```
myapp/
├── cmd/
│   └── api/
│       └── main.go                # entry point
├── internal/
│   ├── config/                   # viper config
│   ├── domain/                  # entities, value objects, events
│   │   ├── user/
│   │   │   ├── entity.go
│   │   │   ├── repository.go
│   │   │   └── events.go
│   │   └── order/
│   ├── application/             # use cases
│   │   ├── user/
│   │   │   ├── register.go
│   │   │   ├── login.go
│   │   │   └── dto.go
│   │   └── order/
│   ├── infrastructure/          # implementations
│   │   ├── persistence/
│   │   │   ├── gorm/
│   │   │   │   ├── user_repo.go
│   │   │   │   └── order_repo.go
│   │   │   └── redis/
│   │   ├── queue/               # RabbitMQ
│   │   ├── email/              # gmail
│   │   └── interfaces/          # delivery
│   │       ├── http/
│   │       │   ├── handlers/
│   │       │   ├── middleware/
│   │       │   └── routes.go
│   │       └── websocket/
│   │           └── hub.go
├── pkg/
│   └── logger/                  # zap wrapper
```

```
|   └─ jwt/  
└─ configs/  
    └─ config.yaml  
└─ migrations/  
└─ go.mod  
└─ go.sum
```

2. ตัวอย่างโค้ดหลัก (main.go)

```
package main  
  
import (  
    "context"  
    "log"  
    "net/http"  
    "os"  
    "os/signal"  
    "syscall"  
    "time"  
  
    "myapp/internal/config"  
    "myapp/internal/infrastructure/persistence/gorm"  
    "myapp/internal/infrastructure/queue"  
    "myapp/internal/infrastructure/email"  
    "myapp/internal/interfaces/http/handlers"  
    "myapp/internal/interfaces/http/middleware"  
    "myapp/internal/interfaces/websocket"  
    "myapp/pkg/logger"  
    "github.com/go-chi/chi/v5"  
)  
  
func main() {  
    // 1. Load configuration  
    cfg, err := config.Load()  
    if err != nil {  
        panic(err)  
    }  
  
    // 2. Setup logger  
    logger.Init(cfg.Log)  
    defer logger.Sync()  
  
    // 3. Connect to database  
    db, err := gorm.Connect(cfg.Database)  
    if err != nil {  
        logger.Fatal("Failed to connect to database", "error", err)  
    }  
  
    // 4. Connect to Redis  
    redisClient, err := redis.Connect(cfg.Redis)
```

```

if err != nil {
    logger.Fatal("Failed to connect to Redis", "error", err)
}

// 5. Connect to RabbitMQ
rabbit, err := queue.Connect(cfg.RabbitMQ)
if err != nil {
    logger.Fatal("Failed to connect to RabbitMQ", "error", err)
}
defer rabbit.Close()

// 6. Setup email service
emailService := email.NewService(cfg.SMTP)

// 7. Initialize repositories
userRepo := gorm.NewUserRepository(db)
orderRepo := gorm.NewOrderRepository(db)
sessionRepo := redis.NewSessionRepository(redisClient)

// 8. Initialize use cases
registerUC := user.NewRegisterUseCase(userRepo, emailService)
loginUC := user.NewLoginUseCase(userRepo, sessionRepo, cfg.JWT)
placeOrderUC := order.NewPlaceOrderUseCase(orderRepo, userRepo, rabbit)

// 9. Initialize handlers
userHandler := handlers.NewUserHandler(registerUC, loginUC)
orderHandler := handlers.NewOrderHandler(placeOrderUC)

// 10. WebSocket hub
wsHub := websocket.NewHub()
go wsHub.Run()

// 11. Setup router
r := chi.NewRouter()
r.Use(middleware.Logger)
r.Use(middleware.Recoverer)
r.Use(middleware.CORS)

// Public routes
r.Post("/api/register", userHandler.Register)
r.Post("/api/login", userHandler.Login)
r.Get("/ws", wsHub.ServeWS)

// Protected routes (require JWT)
r.Group(func(r chi.Router) {
    r.Use(middleware.Auth(sessionRepo, cfg.JWT))
    r.Get("/api/me", userHandler.Me)
    r.Post("/api/orders", orderHandler.PlaceOrder)
})

// 12. Start server
srv := &http.Server{
    Addr:      ":" + cfg.Server.Port,

```

```

    Handler:      r,
    ReadTimeout:  15 * time.Second,
    WriteTimeout: 15 * time.Second,
}

go func() {
    logger.Info("Server starting", "port", cfg.Server.Port)
    if err := srv.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
        logger.Fatal("Server failed", "error", err)
    }
}()

// 13. Graceful shutdown
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

logger.Info("Shutting down server...")
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
srv.Shutdown(ctx)
}

```

3. ตัวอย่าง Use Case: Place Order

```

// internal/application/order/place_order.go
package order

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"

    "myapp/internal/domain/order"
    "myapp/internal/domain/user"
    "myapp/internal/infrastructure/queue"
)

type PlaceOrderUseCase struct {
    orderRepo order.Repository
    userRepo  user.Repository
    queue     *queue.RabbitMQ
}

type PlaceOrderInput struct {
    UserID uint
    Items  []OrderItemInput
}

```

```

type OrderItemInput struct {
    ProductID uint
    Name       string
    Price      float64
    Quantity   int
}

func NewPlaceOrderUseCase(orderRepo order.Repository, userRepo user.Repository,
queue *queue.RabbitMQ) *PlaceOrderUseCase {
    return &PlaceOrderUseCase{
        orderRepo: orderRepo,
        userRepo:   userRepo,
        queue:      queue,
    }
}

func (uc *PlaceOrderUseCase) Execute(ctx context.Context, input
PlaceOrderInput) (*order.Order, error) {
    // 1. Validate user exists and is active
    u, err := uc.userRepo.FindByID(ctx, input.UserID)
    if err != nil || u == nil {
        return nil, errors.New("user not found")
    }
    if !u.IsActive() {
        return nil, errors.New("inactive user cannot place order")
    }

    // 2. Create order aggregate
    ord := order.NewOrder(input.UserID)
    for _, item := range input.Items {
        money := order.NewMoney(item.Price)
        ord.AddItem(item.ProductID, item.Name, money, item.Quantity)
    }

    // 3. Submit order (domain logic)
    if err := ord.Submit(); err != nil {
        return nil, err
    }

    // 4. Persist
    if err := uc.orderRepo.Save(ctx, ord); err != nil {
        return nil, err
    }

    // 5. Publish domain event to queue (async processing)
    event := order.OrderSubmittedEvent{
        OrderID: ord.ID(),
        UserID:  ord.CustomerID(),
        Total:   ord.Total().Float64(),
    }
    data, _ := json.Marshal(event)
    uc.queue.Publish("order.events", data)
}

```

```
    return ord, nil
}
```

4. ตัวอย่าง WebSocket Hub

```
// internal/interfaces/websocket/hub.go
package websocket

import (
    "net/http"
    "sync"

    "github.com/gorilla/websocket"
)

type Hub struct {
    clients    map[*Client]bool
    broadcast  chan []byte
    register   chan *Client
    unregister chan *Client
    mu         sync.RWMutex
}

func NewHub() *Hub {
    return &Hub{
        clients:    make(map[*Client]bool),
        broadcast:  make(chan []byte),
        register:   make(chan *Client),
        unregister: make(chan *Client),
    }
}

func (h *Hub) Run() {
    for {
        select {
        case client := <-h.register:
            h.mu.Lock()
            h.clients[client] = true
            h.mu.Unlock()
        case client := <-h.unregister:
            h.mu.Lock()
            if _, ok := h.clients[client]; ok {
                delete(h.clients, client)
                close(client.send)
            }
            h.mu.Unlock()
        case msg := <-h.broadcast:
            h.mu.RLock()
            for client := range h.clients {

```

```

        select {
        case client.send <- msg:
        default:
            close(client.send)
            delete(h.clients, client)
        }
    }
    h.mu.RUnlock()
}
}

func (h *Hub) ServeWS(w http.ResponseWriter, r *http.Request) {
    upgrader := websocket.Upgrader{CheckOrigin: func(r *http.Request) bool {
return true }}
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        return
    }
    client := &Client{hub: h, conn: conn, send: make(chan []byte, 256)}
    h.register <- client

    go client.writePump()
    go client.readPump()
}

```

บทที่ 60: Task List Template

เทมเพลตสำหรับการติดตามงานพัฒนา feature ใหม่ (อ้างอิงจากบทที่ 48)

```

## Feature: [ชื่อฟีเจอร์]
**Owner:** [ชื่อผู้รับผิดชอบ]
**Due:** [วันที่]

### Phase 1: Analysis & Design
- [ ] ระบุ requirement และ user stories
- [ ] ระบุ bounded context และ aggregate root
- [ ] ออกแบบ entity, value objects
- [ ] กำหนด repository interface methods
- [ ] ระบุ domain events
- [ ] ออกแบบ DTOs (request/response)
- [ ] สร้าง sequence diagram (ถ้ามี)

### Phase 2: Domain Implementation
- [ ] สร้าง entity struct และ business methods
- [ ] สร้าง value objects
- [ ] สร้าง repository interface
- [ ] สร้าง domain events
- [ ] เขียน unit tests สำหรับ domain logic

```

```

### Phase 3: Infrastructure Implementation
- [ ] Implement repository (GORM/sqlx)
- [ ] สร้าง migration scripts
- [ ] ตั้งค่า connection pool
- [ ] Implement cache (Redis) ถ้าจำเป็น
- [ ] ทดสอบ repository (integration tests)

### Phase 4: Application Layer
- [ ] สร้าง use case structs
- [ ] เขียน business logic orchestration
- [ ] สร้าง DTOs (input/output)
- [ ] เขียน unit tests สำหรับ use cases (mock repository)

### Phase 5: Delivery Layer
- [ ] สร้าง HTTP handlers
- [ ] เพิ่ม input validation (go-playground/validator)
- [ ] เพิ่ม middleware (auth, logging) ถ้าจำเป็น
- [ ] ลงทะเบียน routes
- [ ] ทดสอบ handlers (httptest)

### Phase 6: Documentation & Review
- [ ] อัปเดต OpenAPI/Swagger
- [ ] รัน golangci-lint และแก้ไข warnings
- [ ] รัน go test -cover (target >80%)
- [ ] รัน go test -race
- [ ] Code review

### Phase 7: Deployment
- [ ] สร้าง migration สำหรับ production
- [ ] อัปเดต docker-compose (ถ้ามี)
- [ ] Deploy to staging
- [ ] ทดสอบ end-to-end
- [ ] Deploy to production

```

บทที่ 61: Checklist Template

รายการตรวจสอบคุณภาพสำหรับการเตรียม release

Code Quality

- ☐ All exported functions have comments (godoc)
- ☐ No unused imports or variables (go vet)
- ☐ Code formatted with go fmt
- ☐ Error handling explicit (no ignored errors)
- ☐ No panic in library code (only main/init)
- ☐ Context passed as first parameter in all functions
- ☐ Interfaces are small and focused
- ☐ No global state except configuration

Security

- ☐ Passwords hashed with bcrypt
- ☐ JWT secret loaded from environment, not hardcoded
- ☐ Refresh tokens stored in Redis, not in DB
- ☐ Access token short-lived ($\leq 15\text{min}$)
- ☐ CORS configured properly (allow only trusted origins)
- ☐ Input validation on all endpoints
- ☐ SQL injection prevented (use parameterized queries / GORM)
- ☐ No sensitive data in logs
- ☐ HTTPS enforced in production
- ☐ Rate limiting on auth endpoints

Performance

- ☐ Database indexes created on frequently queried columns
- ☐ User data cached in Redis where appropriate
- ☐ Connection pools configured for DB and Redis
- ☐ Use of goroutines for non-blocking tasks (email, etc.)
- ☐ Avoid N+1 queries (use Preload in GORM)
- ☐ Benchmarks for critical paths

Testing

- ☐ Unit tests cover business logic (usecase)
- ☐ Repository tests with testcontainers or in-memory DB
- ☐ HTTP handler tests with httptest
- ☐ Mock external dependencies (Redis, Mailer)
- ☐ Race condition tests with `-race` flag
- ☐ Test coverage $> 80\%$

Deployment

- ☐ Configurable via environment variables
- ☐ Graceful shutdown implemented
- ☐ Health check endpoint (`/health`, `/ready`)
- ☐ Logging to stdout (for container)
- ☐ Docker image built with non-root user
- ☐ Secrets not baked into image
- ☐ Database migration runs automatically on startup
- ☐ Readiness and liveness probes configured
- ☐ Monitoring (Prometheus metrics) exposed

บทที่ 62: แผนภาพการทำงาน (Workflow Diagram)

1. Feature Development Workflow

```

graph LR
    A[Analyze<br/>Requirement] --> B[Design<br/>Domain Models]
    B --> C[Implement Domain<br/>Entities, Interfaces]
    C --> D[Implement Infrastructure<br/>Repository, Cache]
    D --> E[Implement Usecase<br/>Business Logic]
    E --> F[Implement Handler<br/>HTTP/CLI]
    F --> G[Write Tests<br/>Unit, Integration]
    G --> H[Document & Review]
    H --> I[Deploy]

```

2. Authentication Flow (JWT + Redis Refresh)

```

sequenceDiagram
    participant Client
    participant API
    participant DB
    participant Redis

    Client->>API: POST /login (email, password)
    API->>DB: Validate credentials
    DB-->>API: User data
    API->>API: Generate access & refresh tokens
    API->>Redis: Store refresh token (user_id, expiry)
    API-->>Client: Tokens

    Client->>API: GET /protected (access token)
    API->>API: Validate access token
    API-->>Client: Response

    Client->>API: POST /refresh (refresh token)
    API->>Redis: Verify refresh token
    Redis-->>API: Valid
    API->>API: Generate new access token
    API-->>Client: New access token

```

3. Order Processing Flow (Async with Queue)

```

sequenceDiagram
    participant User
    participant API
    participant DB
    participant Queue
    participant Worker
    participant Email

    User->>API: POST /orders (items)
    API->>DB: Create order (status = pending)

```

```
DB-->>API: Order ID
API->>Queue: Publish order_created event
API-->>User: Order created (processing)

Queue->>Worker: Consume event
Worker->>DB: Check stock, calculate total
Worker->>DB: Update order status (processing)
Worker->>Queue: Publish payment_required event
Worker->>Email: Send order confirmation
Email-->>User: Email notification
```

บทที่ 63: mop Config – การจัดการ Configuration

1. ไฟล์ config.yaml ตัวอย่าง

```
# configs/config.yaml
server:
  port: 8080
  mode: release
  read_timeout: 15s
  write_timeout: 15s

database:
  host: localhost
  port: 5432
  user: postgres
  password: ${DB_PASSWORD}
  name: myapp
  ssl_mode: disable
  max_open_conns: 25
  max_idle_conns: 25
  conn_max_lifetime: 5m

redis:
  addr: localhost:6379
  password: ${REDIS_PASSWORD}
  db: 0
  pool_size: 10

jwt:
  secret: ${JWT_SECRET}
  access_expiry: 15m
  refresh_expiry: 168h

log:
  level: info
  format: json
  output: stdout
```

```

rabbitmq:
  url: amqp://${RABBITMQ_USER}:${RABBITMQ_PASSWORD}@localhost:5672/
  exchange: order_events

smtp:
  host: smtp.gmail.com
  port: 587
  username: ${SMTP_USERNAME}
  password: ${SMTP_PASSWORD}
  from: noreply@myapp.com

```

2. ฟังก์ชันโหลด Configuration ด้วย viper (Go)

```

// internal/config/config.go
package config

import (
    "fmt"
    "strings"
    "time"

    "github.com/spf13/viper"
)

type Config struct {
    Server      ServerConfig `mapstructure:"server"`
    Database    DatabaseConfig `mapstructure:"database"`
    Redis       RedisConfig `mapstructure:"redis"`
    JWT         JWTConfig `mapstructure:"jwt"`
    Log         LogConfig `mapstructure:"log"`
    RabbitMQ    RabbitMQConfig `mapstructure:"rabbitmq"`
    SMTP        SMTPConfig `mapstructure:"smtp"`
}

type ServerConfig struct {
    Port          int `mapstructure:"port"`
    Mode          string `mapstructure:"mode"`
    ReadTimeout   time.Duration `mapstructure:"read_timeout"`
    WriteTimeout  time.Duration `mapstructure:"write_timeout"`
}

type DatabaseConfig struct {
    Host          string `mapstructure:"host"`
    Port          int `mapstructure:"port"`
    User          string `mapstructure:"user"`
    Password      string `mapstructure:"password"`
    Name          string `mapstructure:"name"`
    SSLMode       string `mapstructure:"ssl_mode"`
    MaxOpenConns  int `mapstructure:"max_open_conns"`
    MaxIdleConns  int `mapstructure:"max_idle_conns"`
}

```

```

    ConnMaxLifetime time.Duration `mapstructure:"conn_max_lifetime"`
}

type RedisConfig struct {
    Addr      string `mapstructure:"addr"`
    Password  string `mapstructure:"password"`
    DB        int    `mapstructure:"db"`
    PoolSize  int    `mapstructure:"pool_size"`
}

type JWTConfig struct {
    Secret      string `mapstructure:"secret"`
    AccessExpiry time.Duration `mapstructure:"access_expiry"`
    RefreshExpiry time.Duration `mapstructure:"refresh_expiry"`
}

type LogConfig struct {
    Level  string `mapstructure:"level"`
    Format string `mapstructure:"format"`
    Output string `mapstructure:"output"`
}

type RabbitMQConfig struct {
    URL      string `mapstructure:"url"`
    Exchange string `mapstructure:"exchange"`
}

type SMTPConfig struct {
    Host      string `mapstructure:"host"`
    Port      int    `mapstructure:"port"`
    Username  string `mapstructure:"username"`
    Password  string `mapstructure:"password"`
    From      string `mapstructure:"from"`
}

func Load() (*Config, error) {
    viper.SetConfigName("config")
    viper.SetConfigType("yaml")
    viper.AddConfigPath(".")
    viper.AddConfigPath("./configs")
    viper.AddConfigPath("/etc/app")

    viper.SetEnvKeyReplacer(strings.NewReplacer(".", "_"))

    // Defaults
    viper.SetDefault("server.port", 8080)
    viper.SetDefault("server.mode", "debug")
    viper.SetDefault("server.read_timeout", "15s")
    viper.SetDefault("server.write_timeout", "15s")
    viper.SetDefault("log.level", "info")
    viper.SetDefault("log.format", "json")
    viper.SetDefault("log.output", "stdout")
}

```

```

if err := viper.ReadInConfig(); err != nil {
    if _, ok := err.(viper.ConfigFileNotFoundError); !ok {
        return nil, fmt.Errorf("failed to read config: %w", err)
    }
}

// Expand environment variables in config values
for _, key := range viper.AllKeys() {
    val := viper.GetString(key)
    if strings.Contains(val, "${") {
        expanded := os.ExpandEnv(val)
        viper.Set(key, expanded)
    }
}

var cfg Config
if err := viper.Unmarshal(&cfg); err != nil {
    return nil, fmt.Errorf("failed to unmarshal config: %w", err)
}

// Validate required fields
if cfg.JWT.Secret == "" {
    return nil, fmt.Errorf("JWT secret is required")
}

return &cfg, nil
}

```

3. การใช้งานใน main.go

```

cfg, err := config.Load()
if err != nil {
    log.Fatal("Failed to load config:", err)
}
log.Printf("Server starting on port %d", cfg.Server.Port)

```

สรุป

ภาคที่ 10 ได้รวบรวมองค์ประกอบสุดท้ายที่ทำให้โปรเจกต์ Go ระดับ Production พร้อมใช้งาน:

- **Full-stack Example** (บทที่ 59) แสดงการทำงานร่วมกันของ Clean Architecture, WebSocket, Queue, และ Cache
- **Task List Template** (บทที่ 60) ช่วยที่วางแผนและติดตามงานพัฒนา feature ใหม่
- **Checklist Template** (บทที่ 61) ตรวจสอบคุณภาพโค้ด ความปลอดภัย และกระบวนการ deploy
- **Workflow Diagram** (บทที่ 62) อธิบายลำดับการทำงานด้วยแผนภาพ Mermaid
- **mop Config** (บทที่ 63) จัดการ configuration อย่างมืออาชีพด้วย viper

คู่มือนี้ครอบคลุมทุกขั้นตอนตั้งแต่การเริ่มต้นจนถึงการนำไปใช้งานจริง พร้อมเทมเพลตและโค้ดตัวอย่างที่สามารถนำไปปรับใช้ได้ทันที

บทที่ 59: ตัวอย่างโค้ดครบวงจร (Full-stack Example)

59.1 โครงสร้างโปรเจกต์ Full-stack

```
fullstack/
├── cmd/
│   └── api/
│       └── main.go
├── internal/
│   ├── config/
│   ├── domain/
│   │   ├── user/
│   │   └── task/
│   ├── application/
│   │   ├── user/
│   │   └── task/
│   ├── infrastructure/
│   │   ├── persistence/
│   │   ├── cache/
│   │   ├── queue/
│   │   └── websocket/
│   └── interfaces/
│       ├── http/
│       │   ├── handlers/
│       │   ├── middleware/
│       │   └── routes.go
│       └── websocket/
├── pkg/
│   ├── logger/
│   └── utils/
├── web/
│   ├── index.html
│   └── app.js
├── go.mod
├── go.sum
└── config.yaml
```

59.2 ตัวอย่าง: API Server + WebSocket + Redis Pub/Sub

main.go

```
package main

import (
    "context"
    "log"
```

```

"net/http"
"os"
"os/signal"
"syscall"
"time"

"github.com/go-chi/chi/v5"
"github.com/go-chi/chi/v5/middleware"
"github.com/gorilla/websocket"
"github.com/redis/go-redis/v9"
)

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { return true },
}

func main() {
    // Redis client
    rdb := redis.NewClient(&redis.Options{
        Addr: "localhost:6379",
    })

    // Chi router
    r := chi.NewRouter()
    r.Use(middleware.Logger)
    r.Use(middleware.Recoverer)

    // Serve static files
    r.Handle("/static/*", http.StripPrefix("/static/",
    http.FileServer(http.Dir("./web"))))

    // WebSocket endpoint
    r.Get("/ws", func(w http.ResponseWriter, r *http.Request) {
        conn, err := upgrader.Upgrade(w, r, nil)
        if err != nil {
            log.Print("upgrade failed:", err)
            return
        }
        defer conn.Close()

        // Subscribe to Redis channel
        pubsub := rdb.Subscribe(context.Background(), "notifications")
        defer pubsub.Close()

        // Goroutine for receiving messages from Redis and sending to WebSocket
        go func() {
            for msg := range pubsub.Channel() {
                conn.WriteMessage(websocket.TextMessage, []byte(msg.Payload))
            }
        }()

        // Keep connection alive
        for {

```



```

        if _, _, err := conn.ReadMessage(); err != nil {
            break
        }
    }
})

// API endpoint that publishes to Redis
r.Post("/publish", func(w http.ResponseWriter, r *http.Request) {
    message := r.FormValue("message")
    if message == "" {
        http.Error(w, "message required", http.StatusBadRequest)
        return
    }
    rdb.Publish(context.Background(), "notifications", message)
    w.Write([]byte("ok"))
})

srv := &http.Server{
    Addr:    ":8080",
    Handler: r,
}

go func() {
    log.Println("Server starting on :8080")
    if err := srv.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
        log.Fatal(err)
    }
}()

quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
log.Println("Shutting down...")
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
srv.Shutdown(ctx)
}

```

web/index.html

```

<!DOCTYPE html>
<html>
<head>
    <title>WebSocket Demo</title>
</head>
<body>
    <input id="message" type="text" placeholder="Enter message">
    <button onclick="publish()">Send</button>
    <div id="messages"></div>

```

```

<script>
  const ws = new WebSocket("ws://localhost:8080/ws");
  ws.onmessage = (event) => {
    const div = document.createElement("div");
    div.textContent = event.data;
    document.getElementById("messages").appendChild(div);
  };
  function publish() {
    const msg = document.getElementById("message").value;
    fetch("/publish", {
      method: "POST",
      headers: {"Content-Type": "application/x-www-form-urlencoded"},
      body: "message=" + encodeURIComponent(msg)
    });
  }
</script>
</body>
</html>

```

บทที่ 60: Task List Template

(เนื้อหาเดิมจาก Task List Template ในภาคที่ 7-8 สามารถนำมาใช้ได้ แต่สามารถเพิ่มเติมหรือปรับให้เข้ากับภาคนี้ได้)

60.1 Task List Template สำหรับการพัฒนา Feature ใหม่ (ปรับปรุง)

```

# Feature: [ชื่อ Feature]
## Owner: [ชื่อผู้รับผิดชอบ]
## Due Date: [วันที่กำหนดส่ง]

---

## Phase 1: Domain Design (1-2 วัน)
- [ ] ระบุ domain model (entity, value objects)
- [ ] กำหนด invariants (business rules)
- [ ] ระบุ use cases
- [ ] กำหนด domain events (ถ้ามี)
- [ ] ออกแบบ repository interface
- [ ] ออกแบบ DTOs (request/response)

## Phase 2: Implementation (2-4 วัน)
- [ ] สร้าง entity struct และ behavior methods
- [ ] สร้าง value objects
- [ ] สร้าง repository interface
- [ ] สร้าง service interface
- [ ] สร้าง DTOs
- [ ] เขียน unit tests สำหรับ entity
- [ ] เขียน unit tests สำหรับ service (mock repository)

## Phase 3: Infrastructure (1-2 วัน)

```

- [] สร้าง repository implementation (GORM)
- [] สร้าง migration file
- [] ตั้งค่า Redis cache (ถ้าจำเป็น)
- [] ตั้งค่า message queue (ถ้าจำเป็น)
- [] ทดสอบ repository ด้วย integration test

Phase 4: Delivery (1 วัน)

- [] สร้าง HTTP handlers
- [] เพิ่ม input validation
- [] สร้าง routes
- [] ลงทะเบียน dependencies ใน injection
- [] ทดสอบ handler ด้วย httptest

Phase 5: Integration & Documentation (0.5-1 วัน)

- [] ทดสอบ end-to-end
- [] อัปเดต Swagger docs
- [] อัปเดต README
- [] รัน linter และแก้ไข warnings

Phase 6: Review & Deploy (0.5 วัน)

- [] Code review
- [] ตรวจสอบ performance
- [] รัน test coverage (เป้า >80%)
- [] รัน race detector
- [] Deploy to staging
- [] ทดสอบใน staging
- [] Deploy to production

****หมายเหตุ:**** _____

บทที่ 61: Checklist Template

(ปรับปรุงให้ครอบคลุมการทำงานกับระบบภายนอก)

61.1 Code Quality Checklist

Code Quality Checklist

Documentation

- [] All exported functions have comments
- [] Package has package-level documentation
- [] Complex logic has inline comments

Code Style

- [] Code formatted with `go fmt`
- [] No unused imports/variables (`go vet`)
- [] Consistent naming convention

Error Handling

- [] All errors handled explicitly
- [] Errors wrapped with context (`%w`)
- [] No panic in library code

Concurrency

- [] Goroutines have lifecycle management
- [] Channels closed appropriately
- [] No race conditions (`go test -race`)
- [] Context passed as first parameter

Performance

- [] Slice pre-allocated when size known
- [] String concatenation uses `strings.Builder`
- [] Database queries have indexes

Security

- [] Input validation on all external inputs
- [] No hardcoded secrets
- [] Passwords hashed with bcrypt
- [] SQL injection prevented
- [] CORS properly configured

Testing

- [] Unit tests cover business logic
- [] Table-driven tests used
- [] Mock external dependencies
- [] Test coverage >80%

External Integrations

- [] Connection pools configured (Redis, DB)
- [] Timeout and retry configured for external APIs
- [] Circuit breaker implemented for critical services
- [] Graceful handling of connection loss (reconnect)

บทที่ 62: แผนภาพการทำงาน (Workflow Diagram)

62.1 Workflow การพัฒนา Feature ใหม่ (ภาพรวม)

```

flowchart LR
    A[Analyze<br/>Requirement] --> B[Design<br/>Domain]
    B --> C[Implement<br/>Domain]
    C --> D[Implement<br/>Infrastructure]
    D --> E[Implement<br/>Application]
    E --> F[Implement<br/>Delivery]
    F --> G[Test]
    G --> H[Review & Deploy]
  
```

62.2 Workflow การทำงานของ Full-stack Application

```
sequenceDiagram
    participant User
    participant WebUI
    participant API
    participant Cache
    participant DB
    participant Queue
    participant Worker

    User->>WebUI: Click "Create Task"
    WebUI->>API: POST /tasks
    API->>Cache: Check user session
    Cache-->>API: OK
    API->>DB: Insert task
    DB-->>API: task id
    API->>Queue: Publish "task.created"
    API-->>WebUI: 201 Created
    WebUI-->>User: Show success

    Queue->>Worker: Consume "task.created"
    Worker->>DB: Update task status
    Worker->>API: Notify via WebSocket
    API->>WebUI: Push notification
    WebUI-->>User: Real-time update
```

62.3 แผนภาพการสื่อสารระหว่างระบบภายนอก

```
flowchart LR
    subgraph GoApp[Go Application]
        A1[HTTP Handler]
        A2[Use Case]
        A3[Repository]
    end

    subgraph External[External Systems]
        B1[Redis<br/>Cache + Pub/Sub]
        B2[RabbitMQ<br/>Message Queue]
        B3[PostgreSQL<br/>Main DB]
        B4[InfluxDB<br/>Metrics]
        B5[Twilio<br/>SMS]
        B6[LINE<br/>Notify]
    end

    A1 --> A2
    A2 --> A3
    A3 --> B3
    A2 --> B1
    A2 --> B2
    A2 --> B4
```

```
A2 --> B5
A2 --> B6
B2 --> Worker
Worker --> B3
```

บทที่ 63: mop Config – การจัดการ Configuration

63.1 ไฟล์ configuration ตัวอย่าง (config.yaml)

```
# config/config.yaml
app:
  name: "myapp"
  version: "1.0.0"

server:
  port: 8080
  mode: release
  read_timeout: 15s
  write_timeout: 15s

database:
  host: localhost
  port: 5432
  user: postgres
  password: ${DB_PASSWORD} # ใช้ environment variable
  name: mydb
  sslmode: disable
  max_open_conns: 25
  max_idle_conns: 25
  conn_max_lifetime: 5m

redis:
  addr: localhost:6379
  password: ${REDIS_PASSWORD}
  db: 0
  pool_size: 10
  ttl: 10m

rabbitmq:
  url: amqp://guest:guest@localhost:5672/
  exchange: "events"
  queue: "task_queue"

mqtt:
  broker: tcp://localhost:1883
  client_id: "go-mqtt-client"
  username: ${MQTT_USER}
  password: ${MQTT_PASS}
```

```

influxdb:
  url: http://localhost:8086
  token: ${INFLUX_TOKEN}
  org: "myorg"
  bucket: "metrics"

twilio:
  account_sid: ${TWILIO_ACCOUNT_SID}
  auth_token: ${TWILIO_AUTH_TOKEN}
  from: "+1234567890"

line_notify:
  token: ${LINE_NOTIFY_TOKEN}

discord_webhook:
  url: ${DISCORD_WEBHOOK_URL}

log:
  level: info
  format: json
  output: stdout

```

63.2 โหลด Configuration ด้วย Viper (รองรับ Environment Variables)

```

package config

import (
    "fmt"
    "strings"
    "time"

    "github.com/spf13/viper"
)

type Config struct {
    App          AppConfig
    Server       ServerConfig
    Database     DatabaseConfig
    Redis        RedisConfig
    RabbitMQ     RabbitMQConfig
    MQTT         MQTTConfig
    InfluxDB     InfluxDBConfig
    Twilio       TwilioConfig
    LineNotify   LineNotifyConfig
    Discord      DiscordConfig
    Log          LogConfig
}

// struct definitions...

```

```

func Load() (*Config, error) {
    viper.SetConfigName("config")
    viper.SetConfigType("yaml")
    viper.AddConfigPath(".")
    viper.AddConfigPath("./config")
    viper.AddConfigPath("/etc/app")

    // Environment variables override
    viper.SetEnvKeyReplacer(strings.NewReplacer(".", "_"))

    // Defaults
    viper.SetDefault("server.port", 8080)
    viper.SetDefault("server.mode", "debug")
    viper.SetDefault("log.level", "info")
    viper.SetDefault("log.format", "json")
    viper.SetDefault("log.output", "stdout")

    if err := viper.ReadInConfig(); err != nil {
        if _, ok := err.(viper.ConfigFileNotFoundError); !ok {
            return nil, fmt.Errorf("read config: %w", err)
        }
    }

    var cfg Config
    if err := viper.Unmarshal(&cfg); err != nil {
        return nil, fmt.Errorf("unmarshal: %w", err)
    }

    return &cfg, nil
}

```

63.3 ตัวอย่างการใช้งานใน main.go

```

func main() {
    cfg, err := config.Load()
    if err != nil {
        log.Fatalf("Failed to load config: %v", err)
    }

    // ใช้ config
    db := connectDB(cfg.Database)
    redis := connectRedis(cfg.Redis)
    mqtt := connectMQTT(cfg.MQTT)
    // ...

    // ส่งการแจ้งเตือนผ่าน Discord
    if cfg.Discord.URL != "" {
        sendDiscordWebhook(cfg.Discord.URL, "App started")
    }
}

```



```
}  
}
```

สรุปภาคที่ 9–10

ในภาคที่ 9 เราได้สำรวจการผสาน Go กับระบบภายนอกที่พบได้บ่อยในโลกจริง: Redis (cache/queue), RabbitMQ, MQTT, InfluxDB, WebSocket, SMS/LINE, Discord Webhook แต่ละตัวมีไลบรารี Go ที่พร้อมใช้งาน และสามารถนำไปสร้างระบบที่มีประสิทธิภาพสูงได้

ภาคที่ 10 นำเสนอเทมเพลตและเครื่องมือช่วยในการพัฒนา: ตัวอย่างโค้ดครบวงจร, Task List Template, Checklist Template, แผนภาพ workflow และการจัดการ configuration อย่างเป็นระบบ ช่วยให้ทีมพัฒนาทำงานได้เป็นระเบียบและลดความผิดพลาด

ด้วยองค์ความรู้ทั้งหมดในคู่มือนี้ ผู้อ่านสามารถเริ่มพัฒนาแอปพลิเคชัน Go ตั้งแต่ระดับพื้นฐานไปจนถึงระดับ enterprise และสามารถปรับใช้สถาปัตยกรรมที่เหมาะสมกับความต้องการทางธุรกิจได้อย่างมั่นใจ

คู่มือภาษา Go ฉบับ นำไปทำงาน

ครอบคลุมทุกมิติ ตั้งแต่พื้นฐานสู่สถาปัตยกรรมระดับองค์กร พร้อมแผนภาพและโค้ดตัวอย่างที่รันได้จริง

Version 3.0 – เมษายน 2026

บทนำ

ความเป็นมาของคู่มือ

ในยุคที่ซอฟต์แวร์มีความซับซ้อนมากขึ้นเรื่อย ๆ ภาษาโปรแกรมมิ่งที่เรียบง่าย มีประสิทธิภาพสูง และสามารถจัดการกับการทำงานพร้อมกันได้ดี กลายเป็นสิ่งที่นักพัฒนาต้องการอย่างยิ่ง ภาษา Go (หรือ Golang) ถือกำเนิดขึ้นจากความต้องการของ engineers ที่ Google ซึ่งเผชิญกับความท้าทายในการพัฒนาและบำรุงรักษาระบบขนาดใหญ่ที่มีการทำงานพร้อมกันสูง พวกเขาต้องการภาษาใหม่ที่เหมาะสมความรวดเร็วในการทำงานของภาษา C ความง่ายของภาษา Python และความสามารถในการจัดการ concurrency ที่ดีขึ้น

คู่มือเล่มนี้เกิดจากความตั้งใจที่จะรวบรวมองค์ความรู้เกี่ยวกับภาษา Go ตั้งแต่ระดับพื้นฐานจนถึงระดับมืออาชีพ ครอบคลุมทั้งไวยากรณ์พื้นฐาน การจัดการโปรเจกต์ด้วย Go Modules การทดสอบหน่วย การทำงานพร้อมกัน (concurrency) ไปจนถึงการออกแบบสถาปัตยกรรมระดับ Production และการประยุกต์ใช้ Domain-Driven Design (DDD) ร่วมกับ Go

วัตถุประสงค์

คู่มือนี้ถูกออกแบบให้เป็น **ทั้งตำราเรียนและคู่มืออ้างอิง** โดยเน้นให้ผู้อ่านสามารถนำไปประยุกต์ใช้ได้ทันที ตั้งแต่การติดตั้ง การเขียนโปรแกรมพื้นฐาน ไปจนถึงการออกแบบสถาปัตยกรรมแบบ Clean Architecture และ Domain-Driven Design (DDD) รวมถึงการเชื่อมต่อกับระบบภายนอกที่พบได้บ่อยในโลกแห่งความจริง

สิ่งที่คุณจะได้รับ

- ความเข้าใจภาษา Go อย่างลึกซึ้ง ตั้งแต่ไวยากรณ์จนถึง concurrency
- แนวทางการจัดโครงสร้างโปรเจกต์สำหรับการผลิตจริง

- เทคนิคการทดสอบหน่วย (Unit Test) และการวัดประสิทธิภาพ
- รูปแบบสถาปัตยกรรม Clean Architecture + DDD + CQRS
- การผสาน Redis, RabbitMQ, MQTT, InfluxDB, WebSocket, SMS, LINE Notify, Discord
- เทมเพลตและ checklist ที่ช่วยให้ทีมทำงานเป็นระบบ
- แผนภาพ (draw.io) สำหรับอธิบายโครงสร้างและกระบวนการทำงาน
- โค้ดตัวอย่างที่สามารถนำไปรันทดสอบได้จริง

กลุ่มเป้าหมาย

- **ผู้เริ่มต้น** ที่ต้องการเรียนรู้ภาษา Go ตั้งแต่ศูนย์
- **นักพัฒนาที่เปลี่ยนภาษา** จากภาษาอื่นมาสู่ Go
- **นักพัฒนาที่ต้องการยกระดับ** สู่การเป็น Go Developer มืออาชีพ
- **สถาปนิกซอฟต์แวร์** ที่สนใจการออกแบบระบบด้วย Go

วิธีการอ่าน

- หากยังไม่เคยเขียน Go มาก่อน ให้เริ่มจาก **ภาคที่ 1-3** เพื่อทำความเข้าใจพื้นฐาน
- หากต้องการออกแบบแอปพลิเคชันทันที ให้ข้ามไป **ภาคที่ 7-8** เพื่อศึกษา Clean Architecture และ DDD
- หากต้องการเชื่อมต่อกับระบบอื่น (ฐานข้อมูล time-series, message queue, IoT) ให้ดู **ภาคที่ 9**

บทนิยาม (Glossary)

ความหมายของคำศัพท์เฉพาะทาง

คำศัพท์	คำอธิบาย
Go (Golang)	ภาษาโปรแกรมมิ่งที่พัฒนาโดย Google เปิดตัวในปี 2009 ออกแบบมาเพื่อการพัฒนา software ที่มีประสิทธิภาพสูง จัดการ concurrency ได้ดี และมีไวยากรณ์ที่เรียบง่าย
Goroutine	เธรดขนาดเล็กที่ถูกจัดการโดย Go runtime ใช้สำหรับการทำงานแบบ concurrent การสร้างทำได้โดยใช้ 키워ด go หน้าฟังก์ชัน
Channel	โครงสร้างข้อมูลที่ใช้ในการสื่อสารระหว่าง goroutine ช่วยให้ส่งข้อมูลระหว่างกันได้ อย่างปลอดภัย
Compiler	โปรแกรมที่แปลงซอร์สโค้ดภาษา Go ให้เป็นไฟล์ binary ที่เครื่องสามารถรันได้โดยตรง
Go Modules	ระบบจัดการ dependencies อย่างเป็นทางการของ Go เริ่มใช้ตั้งแต่ Go 1.11 ทำให้ไม่ต้องพึ่งพา GOPATH อีกต่อไป
Interface	ชนิดข้อมูลที่กำหนดชุดของ method signatures ชนิดใดก็ตามที่มี method ครบตามที่กำหนด จะถือว่า implement interface นั้นโดยอัตโนมัติ
Struct	ชนิดข้อมูลที่ใช้รวมฟิลด์หลาย ๆ ชนิดเข้าด้วยกัน คล้ายกับ class ในภาษาอื่น แต่ไม่มี method ในตัว
Pointer	ตัวแปรที่เก็บ address ของตัวแปรอื่น ใช้ & เพื่อ address และ * เพื่อ dereference
Defer	คำสั่งที่ใช้เลื่อนการทำงานของฟังก์ชันออกไปจนกว่าฟังก์ชันรอบนอกจะจบการทำงาน ใช้สำหรับ cleanup หรือพักการ

คำศัพท์	คำอธิบาย
Panic / Recover	Panic คือการหยุดการทำงานปกติของโปรแกรม Recover ใช้ใน defer เพื่อจับ panic และควบคุมการทำงานต่อ
Clean Architecture	สถาปัตยกรรมซอฟต์แวร์ที่แบ่งเป็น 3 ชั้นหลัก: Delivery (รับส่งข้อมูล), Usecase (business logic), Repository (การเข้าถึงข้อมูล)
DDD (Domain-Driven Design)	แนวทางการออกแบบซอฟต์แวร์ที่เน้นการสร้างโมเดลที่สะท้อนความรู้ความเข้าใจทางธุรกิจ (domain knowledge) อย่างแท้จริง
Aggregate	กลุ่มของ Entities และ Value Objects ที่ถูกจัดการเป็นหน่วยเดียวกัน มี Aggregate Root เป็นตัวควบคุมความสอดคล้องของข้อมูล
CQRS (Command Query Responsibility Segregation)	รูปแบบการออกแบบที่แยกโมเดลการเขียน (Command) และการอ่าน (Query) ออกจากกัน
Ubiquitous Language	ภาษากลางที่ใช้ร่วมกันระหว่างนักพัฒนาและผู้เชี่ยวชาญโดเมน ใช้ศัพท์เดียวกันในโค้ด, การสนทนา, และเอกสาร
Bounded Context	การแบ่งโดเมนขนาดใหญ่ออกเป็นบริบทย่อยที่มีขอบเขตชัดเจน แต่ละบริบทมีโมเดลและภาษาร่วมของตัวเอง
Repository Pattern	การสร้าง abstraction ชั้นกลางระหว่าง business logic และแหล่งข้อมูล
Middleware	ฟังก์ชันที่ห่อ handler เพื่อเพิ่ม logic เช่น logging, auth, rate limit
Value Object	วัตถุที่ไม่มีเอกลักษณ์เฉพาะตัว ถูกกำหนดโดยค่าของมัน (immutable)
Domain Event	เหตุการณ์สำคัญในโดเมนที่เกิดขึ้นระหว่างการทำงานของระบบ

🧠 การออกแบบคู่มือ

ปรัชญาการออกแบบ

คู่มือนี้ถูกออกแบบโดยยึดหลักการเรียนรู้แบบ **"เรียนรู้จากการปฏิบัติ" (Learning by Doing)** เน้นหาจุดลำดับจากง่ายไปยาก เริ่มจากพื้นฐานที่จำเป็นต่อการเริ่มต้นเขียนโปรแกรม ไปจนถึงหัวข้อขั้นสูงที่นักพัฒนามืออาชีพต้องรู้

โครงสร้างการเรียนรู้

ระดับที่ 1: พื้นฐาน

- └─ ความรู้เบื้องต้นเกี่ยวกับคอมพิวเตอร์และการเขียนโปรแกรม
- └─ ทำความรู้จักกับ Go
- └─ การติดตั้งและเตรียมสภาพแวดล้อม
- └─ เขียนโปรแกรมแรก

ระดับที่ 2: พื้นฐานภาษา

- └─ ตัวแปรและชนิดข้อมูล
- └─ คำสั่งควบคุม
- └─ ฟังก์ชัน
- └─ พอยน์เตอร์

└─ โครงสร้างข้อมูลพื้นฐาน (array, slice, map)

ระดับที่ 3: การพัฒนาแอปพลิเคชัน

└─ การจัดการข้อผิดพลาด
└─ การทำงานกับไฟล์และฐานข้อมูล
└─ HTTP Server/Client
└─ การทำงานพร้อมกัน (concurrency)

ระดับที่ 4: เครื่องมือและไลบรารี

└─ Go Modules
└─ การทดสอบ
└─ การวัดประสิทธิภาพ
└─ ไลบรารียอดนิยม

ระดับที่ 5: การออกแบบสถาปัตยกรรม

└─ Clean Architecture
└─ DDD (Domain-Driven Design)
└─ CQRS และ Event Sourcing

รูปแบบการเรียนรู้แต่ละบท

แต่ละบทในคู่มือมีโครงสร้างที่สอดคล้องกัน:

1. **บทนำ** - อธิบายว่าบทนี้เกี่ยวกับอะไร และทำไมถึงสำคัญ
2. **เนื้อหาหลัก** - อธิบายแนวคิดและทฤษฎี พร้อมตัวอย่างโค้ดประกอบ
3. **ตัวอย่างการประยุกต์ใช้** - กรณีศึกษา หรือการนำไปใช้จริง
4. **ข้อควรระวัง** - ปัญหาที่พบบ่อยและวิธีแก้ไข
5. **แบบฝึกหัด** (สำหรับบทที่เหมาะสม) - เพื่อทบทวนความเข้าใจ

รูปแบบโค้ดตัวอย่าง

โค้ดตัวอย่างในคู่มือใช้รูปแบบที่สอดคล้องกับ Go idiom:

- ใช้ **gofmt** ในการจัดรูปแบบ
- มีการอธิบายบรรทัดสำคัญด้วย comment
- แสดงทั้งการทำงานที่ถูกต้องและข้อผิดพลาดที่พบบ่อย

```
// รูปแบบโค้ดตัวอย่าง
func Example() {
    // การทำงานที่ถูกต้อง
    result, err := doSomething()
    if err != nil {
        // การจัดการ error
        log.Printf("error: %v", err)
        return
    }
    // ใช้ result
}
```

๕ การออกแบบ Workflow

Workflow การเรียนรู้ภาษา Go

```
flowchart TD
    subgraph Phase1[ระยะที่ 1: ศึกษาทฤษฎี]
        A1[อ่านเนื้อหา] --> A2[ทำความเข้าใจแนวคิด]
        A2 --> A3[จดบันทึก]
    end

    subgraph Phase2[ระยะที่ 2: ลงมือปฏิบัติ]
        B1[เขียนโค้ดตามตัวอย่าง] --> B2[ดัดแปลงและทดลอง]
        B2 --> B3[สร้างโปรเจกต์เล็กๆ]
    end

    subgraph Phase3[ระยะที่ 3: ทดสอบและแก้ไข]
        C1[รันโปรแกรม] --> C2[ตรวจสอบผลลัพธ์]
        C2 --> C3[แก้ไขข้อผิดพลาด]
        C3 --> C4[ปรับปรุงประสิทธิภาพ]
    end

    subgraph Phase4[ระยะที่ 4: ทบทวนและสรุป]
        D1[สรุป] --> D2[สร้างเอกสารอ้างอิง]
        D2 --> D3[แชร์ความรู้กับทีม]
    end

    Phase1 --> Phase2 --> Phase3 --> Phase4
```

Workflow การพัฒนาโปรเจกต์ Go

```
flowchart LR
    subgraph Analysis[วิเคราะห์]
        A1[รับ Requirement]
        A2[วิเคราะห์ Domain]
        A3[ระบุ Use Cases]
    end

    subgraph Design[ออกแบบ]
        B1[ออกแบบสถาปัตยกรรม]
        B2[กำหนด Interface]
        B3[ออกแบบ Database]
        B4[เลือก Tools]
    end

    subgraph Development[พัฒนา]
        C1[เขียน Domain Layer]
        C2[เขียน Application Layer]
        C3[เขียน Infrastructure]
    end
```

```

    C4[เขียน Interface]
end

subgraph Testing[ทดสอบ]
    D1[Unit Test]
    D2[Integration Test]
    D3[E2E Test]
    D4[Performance Test]
end

subgraph Deploy[นำส่ง]
    E1[Build]
    E2[Deploy to Staging]
    E3[Deploy to Production]
    E4[Monitor]
end

Analysis --> Design --> Development --> Testing --> Deploy

```

Workflow การเพิ่ม Feature ใหม่

```

flowchart TD
    subgraph Phase1[Phase 1: Domain Design]
        A1[ระบุ domain model]
        A2[กำหนด invariants]
        A3[ระบุ use cases]
        A4[กำหนด domain events]
        A5[ออกแบบ repository interface]
        A6[ออกแบบ DTOs]
    end

    subgraph Phase2[Phase 2: Implementation]
        B1[สร้าง entity struct]
        B2[สร้าง value objects]
        B3[สร้าง repository interface]
        B4[สร้าง service interface]
        B5[สร้าง DTOs]
        B6[เขียน unit tests - entity]
        B7[เขียน unit tests - service]
    end

    subgraph Phase3[Phase 3: Infrastructure]
        C1[สร้าง repository implementation]
        C2[สร้าง migration file]
        C3[ตั้งค่า Redis cache]
        C4[ตั้งค่า message queue]
        C5[ทดสอบ repository]
    end

    subgraph Phase4[Phase 4: Delivery]

```

```

    D1[สร้าง HTTP handlers]
    D2[เพิ่ม input validation]
    D3[สร้าง routes]
    D4[ลงทะเบียน dependencies]
    D5[ทดสอบ handler]
end

subgraph Phase5[Phase 5: Integration]
    E1[ทดสอบ end-to-end]
    E2[อัปเดต documentation]
    E3[รัน linter]
end

subgraph Phase6[Phase 6: Review & Deploy]
    F1[Code review]
    F2[ตรวจสอบ performance]
    F3[รัน test coverage]
    F4[รัน race detector]
    F5[Deploy to staging]
    F6[Deploy to production]
end

Phase1 --> Phase2 --> Phase3 --> Phase4 --> Phase5 --> Phase6

```

แผนภาพสถาปัตยกรรม Clean Architecture

```

flowchart TB
    subgraph Interface[Interface Layer]
        HTTP[HTTP Handlers]
        CLI[CLI Commands]
        GRPC[gRPC Services]
    end

    subgraph Application[Application Layer]
        UC1[Use Case 1]
        UC2[Use Case 2]
        UC3[Use Case 3]
    end

    subgraph Domain[Domain Layer]
        ENT[Entities]
        VO[Value Objects]
        REPO[Repository Interfaces]
        EVENTS[Domain Events]
    end

    subgraph Infrastructure[Infrastructure Layer]
        DB[Database]
        CACHE[Redis Cache]
        MQ[Message Queue]
    end

```

```

EMAIL[Email Service]
end

Interface --> Application
Application --> Domain
Infrastructure -.-> REPO
Infrastructure -.-> EVENTS

```

แผนภาพ DDD Bounded Context

```

flowchart LR
    subgraph Context1[User Context]
        U1[User Entity]
        U2[Email VO]
        U3[Password VO]
        U4[User Repository]
    end

    subgraph Context2[Order Context]
        O1[Order Aggregate]
        O2[OrderItem Entity]
        O3[Address VO]
        O4[Order Repository]
    end

    subgraph Context3[Payment Context]
        P1[Payment Entity]
        P2[Transaction VO]
        P3[Payment Repository]
    end

    Context1 <--> Context2
    Context2 <--> Context3

```

แผนภาพ CQRS และ Event Sourcing

```

flowchart TD
    subgraph Command[Command Side]
        CMD[Command Handler]
        AGG[Aggregate Root]
        EVENT[Domain Events]
        STORE[Event Store]
    end

    subgraph Query[Query Side]
        QRY[Query Handler]
        READ[Read Database]
        VIEW[Projection View]
    end

```



```

end

subgraph Sync[Eventual Consistency]
    PROJ[Projection]
    BUS[Event Bus]
end

CMD --> AGG
AGG --> EVENT
EVENT --> STORE
EVENT --> BUS
BUS --> PROJ
PROJ --> READ
QRY --> READ
QRY --> VIEW

```

Task List Template

Template สำหรับการพัฒนา Feature ใหม่

```

# Feature: [ชื่อ Feature]
## Owner: [ชื่อผู้รับผิดชอบ]
## Due Date: [วันที่กำหนดส่ง]

---

## Phase 1: Domain Design

### Tasks
- [ ] **T1.1** ระบุ domain model (entity, value objects)
  - [ ] ระบุ entity: _____
  - [ ] ระบุ value objects: _____
- [ ] **T1.2** กำหนด invariants (business rules)
  - [ ] Rule 1: _____
  - [ ] Rule 2: _____
- [ ] **T1.3** ระบุ use cases
  - [ ] Use case 1: _____
  - [ ] Use case 2: _____
- [ ] **T1.4** กำหนด domain events (ถ้ามี)
  - [ ] Event 1: _____
  - [ ] Event 2: _____
- [ ] **T1.5** ออกแบบ repository interface
  - [ ] Method: _____
  - [ ] Method: _____
- [ ] **T1.6** ออกแบบ DTOs (request/response)
  - [ ] Request: _____
  - [ ] Response: _____

**หมายเหตุ:** _____

```

Phase 2: Implementation

Tasks

- [] ****T2.1**** สร้าง entity struct และ behavior methods
 - [] File: `internal/domain/[module]/entity.go`
 - [] Constructor: `New[Entity]()`
 - [] Methods: _____
- [] ****T2.2**** สร้าง value objects
 - [] File: `internal/domain/[module]/value\_objects.go`
 - [] VO 1: _____
- [] ****T2.3**** สร้าง repository interface
 - [] File: `internal/domain/[module]/repository.go`
- [] ****T2.4**** สร้าง service interface
 - [] File: `internal/domain/[module]/service.go`
- [] ****T2.5**** สร้าง DTO structs
 - [] File: `internal/application/[module]/dto.go`
- [] ****T2.6**** เขียน unit tests สำหรับ entity
 - [] File: `internal/domain/[module]/entity\_test.go`
 - [] Test cases: _____
- [] ****T2.7**** เขียน unit tests สำหรับ service (mock repository)
 - [] File: `internal/application/[module]/[usecase]\_test.go`
 - [] Mock repository implementation

****หมายเหตุ:**** _____

Phase 3: Infrastructure

Tasks

- [] ****T3.1**** สร้าง repository implementation
 - [] File: `internal/infrastructure/persistence/gorm/[module]\_repo.go`
 - [] Implement interface methods
- [] ****T3.2**** สร้าง migration file
 - [] File: `migrations/[timestamp]\_create\_[table]\_table.sql`
 - [] Up migration: _____
 - [] Down migration: _____
- [] ****T3.3**** ตั้งค่า Redis cache (ถ้าจำเป็น)
 - [] Cache key pattern: _____
 - [] TTL: _____
- [] ****T3.4**** ตั้งค่า message queue (ถ้าจำเป็น)
 - [] Topic/Queue name: _____
 - [] Consumer implementation
- [] ****T3.5**** ทดสอบ repository ด้วย integration test
 - [] File: `internal/infrastructure/persistence/gorm/[module]\_repo\_test.go`
 - [] Use testcontainers or in-memory DB

****หมายเหตุ:**** _____

Phase 4: Delivery

Tasks

- [] ****T4.1**** สร้าง HTTP handlers
 - [] File: `internal/interfaces/http/handlers/[module]\_handler.go`
 - [] Handler methods: _____
- [] ****T4.2**** เพิ่ม input validation
 - [] Validation tags: _____
 - [] Custom validator (ถ้ามี): _____
- [] ****T4.3**** สร้าง routes
 - [] File: `internal/interfaces/http/routes.go`
 - [] Routes: _____
- [] ****T4.4**** ลงทะเบียน dependencies ใน injection
 - [] File: `internal/apps/app/bootstrap/injection/wire.go`
 - [] Update provider set
- [] ****T4.5**** ทดสอบ handler ด้วย httpstest
 - [] File: `internal/interfaces/http/handlers/[module]\_handler\_test.go`
 - [] Test cases: _____

****หมายเหตุ:**** _____

Phase 5: Integration & Documentation

Tasks

- [] ****T5.1**** ทดสอบ end-to-end
 - [] curl/Postman collection: _____
 - [] Test scenarios: _____
- [] ****T5.2**** อัปเดต Swagger docs
 - [] File: `api/swagger.yaml` or `docs/docs.go`
 - [] Annotations: _____
- [] ****T5.3**** อัปเดต README
 - [] Add feature description
 - [] Update API examples
- [] ****T5.4**** รัน linter และแก้ไข warnings
 - [] Command: `golangci-lint run ./...`
 - [] Issues fixed: _____

****หมายเหตุ:**** _____

Phase 6: Review & Deploy

Tasks

- [] ****T6.1**** Code review
 - [] PR created: _____
 - [] Reviewers: _____
 - [] Comments addressed
- [] ****T6.2**** ตรวจสอบ performance
 - [] Benchmark: _____

- [] Query optimization: _____
- [] ****T6.3**** รัน test coverage
 - [] Command: `go test -cover ./...`
 - [] Coverage: _____% (target >80%)
- [] ****T6.4**** รัน race detector
 - [] Command: `go test -race ./...`
 - [] Issues found: _____
- [] ****T6.5**** Deploy to staging
 - [] Date: _____
 - [] Version: _____
- [] ****T6.6**** ทดสอบใน staging
 - [] Smoke test passed
 - [] Regression test passed
- [] ****T6.7**** Deploy to production
 - [] Date: _____
 - [] Version: _____
 - [] Monitoring checked

****หมายเหตุ:**** _____

Summary

- ****Total Tasks:**** ____ / ____ completed
- ****Blockers:**** _____
- ****Next Steps:**** _____

☒ Checklist Template

Code Quality Checklist

Code Quality Checklist

Documentation

- [] All exported functions have comments (godoc format)
- [] Package has package-level documentation comment
- [] Complex logic has inline comments explaining "why"
- [] README updated with relevant information

Code Style

- [] Code formatted with `go fmt` or `gofmt`
- [] No unused imports or variables (`go vet` passed)
- [] Consistent naming convention (camelCase, PascalCase)
- [] No magic numbers (use constants)
- [] Line length < 120 characters (preferably)

Error Handling

- [] All errors are handled explicitly (no `\_` ignoring)

- [] Errors are wrapped with context (``fmt.Errorf("...: %w", err)``)
- [] No panic in library code (only in main/init for fatal errors)
- [] Custom error types used when appropriate
- [] Error messages are descriptive and actionable

Concurrency

- [] Goroutines have proper lifecycle management
- [] Channels are closed appropriately
- [] No race conditions (``go test -race`` passed)
- [] `sync.Mutex` used correctly (Lock/Unlock pairs)
- [] Context passed as first parameter for cancellation

Performance

- [] No unnecessary allocations in hot paths
- [] Slice pre-allocated when size known (``make([]T, 0, capacity)``)
- [] String concatenation uses ``strings.Builder`` for large operations
- [] Database queries have appropriate indexes
- [] No N+1 queries

Security

- [] Input validation on all external inputs
- [] SQL injection prevented (use parameterized queries)
- [] No hardcoded secrets or credentials
- [] Sensitive data not logged
- [] Passwords hashed with `bcrypt` (not stored in plaintext)
- [] JWT secrets loaded from environment
- [] CORS configured properly (allow only trusted origins)

Testing

- [] Unit tests cover business logic
- [] Table-driven tests used for multiple scenarios
- [] Edge cases tested (nil, empty, boundary values)
- [] Mock external dependencies
- [] Test coverage > 80%

Project Structure

- [] Follows standard Go project layout
- [] Packages have single responsibility
- [] No circular dependencies
- [] Internal packages used for private code
- [] Go modules properly configured

Dependencies

- [] `go.mod` has only required dependencies
- [] `go.sum` is committed
- [] ``go mod tidy`` run before commit
- [] No unused dependencies

Version Control

- [] Commit messages follow convention (feat, fix, docs, etc.)
- [] No debug code (`fmt.Println`, `log.Println`) in production code
- [] No commented out code
- [] `.gitignore` properly configured

Reviewer Notes

- [] Code reviewed by at least one other developer
- [] All review comments addressed

****Status:**** [] Ready for merge | [] Changes requested | [] Approved

****Reviewer:**** _____

****Date:**** _____

Deployment Checklist

Deployment Checklist

Pre-Deployment (Staging)

Code Readiness

- [] All tests passing (`go test ./...`)
- [] Race detector passed (`go test -race ./...`)
- [] Linter passed (`golangci-lint run ./...`)
- [] Build successful (`go build ./...`)
- [] All PRs merged and approved

Configuration

- [] Environment variables verified
- [] Configuration files updated for staging
- [] Feature flags configured
- [] Third-party service credentials verified

Database

- [] Migration scripts reviewed
- [] Migrations tested in staging environment
- [] Rollback plan documented
- [] Backup created before migration

Infrastructure

- [] Container images built and tagged
- [] Kubernetes/ deployment files updated
- [] Resource limits configured
- [] Health check endpoints configured
- [] Monitoring and alerting configured

Security

- [] Security scan passed
- [] No secrets in code or config
- [] TLS certificates valid

Staging Deployment

Deployment Steps

- [] Deploy to staging environment
- [] Verify pod/container health
- [] Run smoke tests
- [] Run integration tests
- [] Verify logs for errors
- [] Load testing (if required)

Validation

- [] Feature works as expected
- [] No regression in existing features
- [] Performance meets baseline
- [] Error handling works
- [] Monitoring shows expected metrics

Pre-Production (Final Check)

Business Approval

- [] Product owner sign-off
- [] QA sign-off
- [] Security sign-off
- [] Documentation updated

Rollback Plan

- [] Rollback procedure documented
- [] Database rollback plan ready
- [] Previous version image available
- [] Rollback tested

Communication

- [] Release notes prepared
- [] Stakeholders notified
- [] Support team informed

Production Deployment

Deployment Steps

- [] Schedule maintenance window (if required)
- [] Create production backup
- [] Deploy with canary/blue-green strategy
- [] Monitor deployment progress
- [] Verify health checks
- [] Run post-deployment tests

Post-Deployment

- [] Monitor logs for errors (15 min)
- [] Verify key metrics
- [] Check user feedback channels

```

- [ ] Update status page (if applicable)
- [ ] Announce successful deployment

#### Rollback Trigger Conditions
- [ ] Error rate > 1%
- [ ] Critical feature broken
- [ ] Security incident detected
- [ ] Performance degradation > 50%

---

### Post-Deployment

#### Cleanup
- [ ] Remove old images (if applicable)
- [ ] Clean up temporary resources
- [ ] Update documentation with new version

#### Monitoring
- [ ] Monitor for 24 hours
- [ ] Review error logs daily for 1 week
- [ ] Check resource utilization

#### Retrospective
- [ ] Deployment time recorded
- [ ] Issues encountered documented
- [ ] Improvements identified for next deployment

---

**Deployment Status:** [ ] Success | [ ] Failed | [ ] Rolled back
**Deployed by:** _____
**Date:** _____
**Version:** _____

```

ตัวอย่างโค้ด

ตัวอย่าง 1: Clean Architecture - User Registration (Full Example)

โครงสร้างโปรเจกต์

```

user-service/
├── cmd/
│   └── api/
│       └── main.go
├── internal/
│   ├── domain/
│   │   └── user/
│   │       ├── entity.go
│   │       └── value_objects.go

```



```

├── repository.go
├── application/
│   ├── user/
│   │   ├── register.go
│   │   └── dto.go
│   ├── infrastructure/
│   │   ├── persistence/
│   │   │   └── gorm/
│   │   │       └── user_repo.go
│   └── interfaces/
│       ├── http/
│       │   ├── handlers/
│       │   │   └── user_handler.go
│       └── routes.go
├── go.mod
└── go.sum

```

go.mod

```

module user-service

go 1.21

require (
    github.com/go-chi/chi/v5 v5.0.10
    github.com/go-playground/validator/v10 v10.15.5
    github.com/google/uuid v1.3.1
    github.com/lib/pq v1.10.9
    golang.org/x/crypto v0.14.0
    gorm.io/driver/postgres v1.5.3
    gorm.io/gorm v1.25.5
)

```

Domain Layer - Entity

internal/domain/user/entity.go

```

package user

import (
    "errors"
    "time"

    "github.com/google/uuid"
)

// User represents the core domain entity

```

```

type User struct {
    id          uuid.UUID
    email       Email
    password    Password
    name        string
    isVerified  bool
    createdAt   time.Time
    updatedAt   time.Time
}

// NewUser creates a new User entity
func NewUser(email, password, name string) (*User, error) {
    emailVO, err := NewEmail(email)
    if err != nil {
        return nil, err
    }

    passwordVO, err := NewPassword(password)
    if err != nil {
        return nil, err
    }

    if name == "" {
        return nil, errors.New("name is required")
    }

    now := time.Now()
    return &User{
        id:          uuid.New(),
        email:       *emailVO,
        password:    *passwordVO,
        name:        name,
        isVerified:  false,
        createdAt:   now,
        updatedAt:   now,
    }, nil
}

// Reconstruct reconstructs a User from persistence (for repository)
func Reconstruct(id uuid.UUID, email Email, passwordHash string, name string,
isVerified bool, createdAt, updatedAt time.Time) (*User, error) {
    password, err := NewPasswordFromHash(passwordHash)
    if err != nil {
        return nil, err
    }

    return &User{
        id:          id,
        email:       email,
        password:    *password,
        name:        name,
        isVerified:  isVerified,
        createdAt:   createdAt,

```

```

        updatedAt: updatedAt,
    }, nil
}

// Getters
func (u *User) ID() uuid.UUID    { return u.id }
func (u *User) Email() Email     { return u.email }
func (u *User) Name() string     { return u.name }
func (u *User) IsVerified() bool { return u.isVerified }
func (u *User) CreatedAt() time.Time { return u.createdAt }
func (u *User) UpdatedAt() time.Time { return u.updatedAt }
func (u *User) PasswordHash() string { return u.password.Hash() }

// Behaviors
func (u *User) Verify() {
    u.isVerified = true
    u.updatedAt = time.Now()
}

func (u *User) ChangePassword(oldPlain, newPlain string) error {
    if err := u.password.Compare(oldPlain); err != nil {
        return ErrInvalidPassword
    }

    newPassword, err := NewPassword(newPlain)
    if err != nil {
        return err
    }

    u.password = *newPassword
    u.updatedAt = time.Now()
    return nil
}

```

internal/domain/user/value_objects.go

```

package user

import (
    "errors"
    "regexp"

    "golang.org/x/crypto/bcrypt"
)

// Email is a value object
type Email struct {
    value string
}

// NewEmail creates a new Email value object

```

```

func NewEmail(email string) (*Email, error) {
    emailRegex := regexp.MustCompile(`^[a-z0-9._%+\-]+\@[a-z0-9.\-]+\.[a-z]{2,}$`)
    if !emailRegex.MatchString(email) {
        return nil, errors.New("invalid email format")
    }
    return &Email{value: email}, nil
}

func (e Email) String() string { return e.value }

// Password is a value object containing hashed password
type Password struct {
    hash string
}

// NewPassword creates a new Password from plain text
func NewPassword(plain string) (*Password, error) {
    if len(plain) < 8 {
        return nil, errors.New("password must be at least 8 characters")
    }

    hash, err := bcrypt.GenerateFromPassword([]byte(plain), bcrypt.DefaultCost)
    if err != nil {
        return nil, err
    }

    return &Password{hash: string(hash)}, nil
}

// NewPasswordFromHash creates a Password from existing hash
func NewPasswordFromHash(hash string) (*Password, error) {
    return &Password{hash: hash}, nil
}

// Compare checks if plain password matches the hash
func (p *Password) Compare(plain string) error {
    return bcrypt.CompareHashAndPassword([]byte(p.hash), []byte(plain))
}

func (p *Password) Hash() string { return p.hash }

```

internal/domain/user/repository.go

```

package user

import (
    "context"

    "github.com/google/uuid"
)

```

```
// Repository defines the interface for user data access
type Repository interface {
    Create(ctx context.Context, user *User) error
    FindByID(ctx context.Context, id uuid.UUID) (*User, error)
    FindByEmail(ctx context.Context, email Email) (*User, error)
    Update(ctx context.Context, user *User) error
    Delete(ctx context.Context, id uuid.UUID) error
}

// Domain errors
var (
    ErrUserNotFound      = errors.New("user not found")
    ErrEmailAlreadyUsed = errors.New("email already used")
    ErrInvalidPassword  = errors.New("invalid password")
)
```

Application Layer

internal/application/user/dto.go

```
package user

// RegisterInput represents the data needed to register a user
type RegisterInput struct {
    Email    string `json:"email" validate:"required,email"`
    Password string `json:"password" validate:"required,min=8"`
    Name     string `json:"name" validate:"required"`
}

// RegisterOutput represents the response after registration
type RegisterOutput struct {
    ID      string `json:"id"`
    Email   string `json:"email"`
    Name    string `json:"name"`
}

// LoginInput represents the data needed to login
type LoginInput struct {
    Email    string `json:"email" validate:"required,email"`
    Password string `json:"password" validate:"required"`
}

// LoginOutput represents the response after login
type LoginOutput struct {
    AccessToken  string `json:"access_token"`
    RefreshToken string `json:"refresh_token"`
    ExpiresIn    int64  `json:"expires_in"`
}
```

```
package user

import (
    "context"
    "errors"

    "user-service/internal/domain/user"
)

var ErrEmailAlreadyExists = errors.New("email already exists")

// RegisterUseCase handles user registration
type RegisterUseCase struct {
    userRepo user.Repository
}

// NewRegisterUseCase creates a new RegisterUseCase
func NewRegisterUseCase(repo user.Repository) *RegisterUseCase {
    return &RegisterUseCase{userRepo: repo}
}

// Execute performs user registration
func (uc *RegisterUseCase) Execute(ctx context.Context, input RegisterInput)
(*RegisterOutput, error) {
    // Check if email already exists
    emailVO, err := user.NewEmail(input.Email)
    if err != nil {
        return nil, err
    }

    existing, err := uc.userRepo.FindByEmail(ctx, *emailVO)
    if err != nil && err != user.ErrUserNotFound {
        return nil, err
    }
    if existing != nil {
        return nil, ErrEmailAlreadyExists
    }

    // Create new user
    newUser, err := user.NewUser(input.Email, input.Password, input.Name)
    if err != nil {
        return nil, err
    }

    // Save to repository
    if err := uc.userRepo.Create(ctx, newUser); err != nil {
        return nil, err
    }

    return &RegisterOutput{
```

```

        ID:    newUser.ID().String(),
        Email: newUser.Email().String(),
        Name:  newUser.Name(),
    }, nil
}

```

Infrastructure Layer

internal/infrastructure/persistence/gorm/user_repo.go

```

package gorm

import (
    "context"
    "errors"
    "time"

    "github.com/google/uuid"
    "gorm.io/gorm"

    "user-service/internal/domain/user"
)

// UserModel represents the database model
type UserModel struct {
    ID          string    `gorm:"primaryKey;type:uuid"`
    Email       string    `gorm:"uniqueIndex;size:100;not null"`
    Password    string    `gorm:"not null"`
    Name        string    `gorm:"size:100;not null"`
    IsVerified  bool      `gorm:"default:false"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
    DeletedAt   gorm.DeletedAt `gorm:"index"`
}

func (UserModel) TableName() string {
    return "users"
}

// UserRepository implements user.Repository using GORM
type UserRepository struct {
    db *gorm.DB
}

// NewUserRepository creates a new UserRepository
func NewUserRepository(db *gorm.DB) *UserRepository {
    return &UserRepository{db: db}
}

// Create inserts a new user

```

```

func (r *UserRepository) Create(ctx context.Context, u *user.User) error {
    model := &UserModel{
        ID:          u.ID().String(),
        Email:        u.Email().String(),
        Password:     u.PasswordHash(),
        Name:         u.Name(),
        IsVerified:   u.IsVerified(),
        CreatedAt:    u.CreatedAt(),
        UpdatedAt:    u.UpdatedAt(),
    }
    return r.db.WithContext(ctx).Create(model).Error
}

// FindByID finds a user by ID
func (r *UserRepository) FindByID(ctx context.Context, id uuid.UUID)
(*user.User, error) {
    var model UserModel
    err := r.db.WithContext(ctx).Where("id = ?",
id.String()).First(&model).Error
    if err != nil {
        if errors.Is(err, gorm.ErrRecordNotFound) {
            return nil, user.ErrUserNotFound
        }
        return nil, err
    }
    return r.toDomain(&model)
}

// FindByEmail finds a user by email
func (r *UserRepository) FindByEmail(ctx context.Context, email user.Email)
(*user.User, error) {
    var model UserModel
    err := r.db.WithContext(ctx).Where("email = ?",
email.String()).First(&model).Error
    if err != nil {
        if errors.Is(err, gorm.ErrRecordNotFound) {
            return nil, user.ErrUserNotFound
        }
        return nil, err
    }
    return r.toDomain(&model)
}

// Update updates an existing user
func (r *UserRepository) Update(ctx context.Context, u *user.User) error {
    model := &UserModel{
        ID:          u.ID().String(),
        Email:        u.Email().String(),
        Password:     u.PasswordHash(),
        Name:         u.Name(),
        IsVerified:   u.IsVerified(),
        UpdatedAt:    u.UpdatedAt(),
    }
}

```



```

        return r.db.WithContext(ctx).Save(model).Error
    }

    // Delete soft deletes a user
    func (r *UserRepository) Delete(ctx context.Context, id uuid.UUID) error {
        return r.db.WithContext(ctx).Where("id = ?",
            id.String()).Delete(&UserModel{}).Error
    }

    func (r *UserRepository) toDomain(m *UserModel) (*user.User, error) {
        id, err := uuid.Parse(m.ID)
        if err != nil {
            return nil, err
        }

        email, err := user.NewEmail(m.Email)
        if err != nil {
            return nil, err
        }

        return user.Reconstruct(
            id,
            *email,
            m.Password,
            m.Name,
            m.IsVerified,
            m.CreatedAt,
            m.UpdatedAt,
        )
    }
}

```

Interface Layer

internal/interfaces/http/handlers/user_handler.go

```

package handlers

import (
    "encoding/json"
    "net/http"

    "github.com/go-playground/validator/v10"

    "user-service/internal/application/user"
)

// UserHandler handles HTTP requests for user operations
type UserHandler struct {
    registerUC *user.RegisterUseCase
    validate   *validator.Validate
}

```

```

}

// NewUserHandler creates a new UserHandler
func NewUserHandler(registerUC *user.RegisterUseCase) *UserHandler {
    return &UserHandler{
        registerUC: registerUC,
        validate:    validator.New(),
    }
}

// Register handles POST /api/register
func (h *UserHandler) Register(w http.ResponseWriter, r *http.Request) {
    var req user.RegisterInput
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid request body", http.StatusBadRequest)
        return
    }

    if err := h.validate.Struct(req); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    output, err := h.registerUC.Execute(r.Context(), req)
    if err != nil {
        switch err {
        case user.ErrEmailAlreadyExists:
            http.Error(w, "Email already registered", http.StatusConflict)
        default:
            http.Error(w, "Internal server error",
http.StatusInternalServerError)
        }
        return
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(output)
}

// HealthCheck handles GET /health
func (h *UserHandler) HealthCheck(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
}

```

internal/interfaces/http/routes.go

```
package http
```

```

import (
    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"

    "user-service/internal/interfaces/http/handlers"
)

// SetupRoutes configures all HTTP routes
func SetupRoutes(userHandler *handlers.UserHandler) *chi.Mux {
    r := chi.NewRouter()

    // Middleware
    r.Use(middleware.Logger)
    r.Use(middleware.Recoverer)
    r.Use(middleware.RequestID)
    r.Use(middleware.RealIP)

    // Health check
    r.Get("/health", userHandler.HealthCheck)

    // API routes
    r.Route("/api", func(r chi.Router) {
        r.Post("/register", userHandler.Register)
    })

    return r
}

```

Main Entry Point

cmd/api/main.go

```

package main

import (
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "gorm.io/driver/postgres"
    "gorm.io/gorm"
    "gorm.io/gorm/logger"

    "user-service/internal/application/user"
    "user-service/internal/domain/user"
    gormRepo "user-service/internal/infrastructure/persistence/gorm"
    "user-service/internal/interfaces/http/handlers"
)

```

```

    "user-service/internal/interfaces/http"
)

func main() {
    // Database connection
    dsn := "host=localhost user=postgres password=postgres dbname=userdb
port=5432 sslmode=disable TimeZone=Asia/Bangkok"
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{
        Logger: logger.Default.LogMode(logger.Info),
    })
    if err != nil {
        log.Fatal("Failed to connect to database:", err)
    }

    // Auto migrate (development only)
    if err := db.AutoMigrate(&gormRepo.UserModel{}); err != nil {
        log.Fatal("Failed to migrate database:", err)
    }

    // Dependency injection
    userRepo := gormRepo.NewUserRepository(db)
    registerUC := user.NewRegisterUseCase(userRepo)
    userHandler := handlers.NewUserHandler(registerUC)

    // Setup routes
    router := http.SetupRoutes(userHandler)

    // Create server
    srv := &http.Server{
        Addr:         ":8080",
        Handler:      router,
        ReadTimeout:  15 * time.Second,
        WriteTimeout: 15 * time.Second,
        IdleTimeout:  60 * time.Second,
    }

    // Graceful shutdown
    go func() {
        log.Println("Starting server on :8080")
        if err := srv.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
            log.Fatalf("Server failed: %v", err)
        }
    }()

    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
    <-quit
    log.Println("Shutting down server...")

    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()

```

```

    if err := srv.Shutdown(ctx); err != nil {
        log.Fatalf("Server forced to shutdown: %v", err)
    }

    log.Println("Server exited")
}

```

ตัวอย่าง 2: Worker Pool Pattern

```

package main

import (
    "context"
    "fmt"
    "sync"
    "time"
)

// Job represents a unit of work
type Job struct {
    ID      int
    Payload string
}

// Result represents the outcome of processing a job
type Result struct {
    JobID  int
    Output string
    Error  error
}

// WorkerPool manages a pool of workers for concurrent job processing
type WorkerPool struct {
    numWorkers  int
    jobQueue    chan Job
    resultQueue chan Result
    wg          sync.WaitGroup
    ctx         context.Context
    cancel      context.CancelFunc
}

// NewWorkerPool creates a new worker pool
func NewWorkerPool(numWorkers int, queueSize int) *WorkerPool {
    ctx, cancel := context.WithCancel(context.Background())
    return &WorkerPool{
        numWorkers: numWorkers,
        jobQueue:   make(chan Job, queueSize),
        resultQueue: make(chan Result, queueSize),
        ctx:        ctx,
        cancel:     cancel,
    }
}

```

```

    }
}

// Start launches the worker pool
func (wp *WorkerPool) Start() {
    for i := 0; i < wp.numWorkers; i++ {
        wp.wg.Add(1)
        go wp.worker(i)
    }
}

// worker processes jobs from the queue
func (wp *WorkerPool) worker(id int) {
    defer wp.wg.Done()
    for {
        select {
        case <-wp.ctx.Done():
            fmt.Printf("Worker %d stopping\n", id)
            return
        case job, ok := <-wp.jobQueue:
            if !ok {
                return
            }
            result := wp.processJob(job)
            select {
            case wp.resultQueue <- result:
            case <-wp.ctx.Done():
                return
            }
        }
    }
}

// processJob handles a single job
func (wp *WorkerPool) processJob(job Job) Result {
    // Simulate processing time
    time.Sleep(100 * time.Millisecond)

    // Process the job
    output := fmt.Sprintf("Processed job %d with payload: %s", job.ID,
job.Payload)

    return Result{
        JobID: job.ID,
        Output: output,
        Error: nil,
    }
}

// Submit adds a job to the queue
func (wp *WorkerPool) Submit(job Job) bool {
    select {
    case wp.jobQueue <- job:

```

```

        return true
    case <-wp.ctx.Done():
        return false
    default:
        return false
    }
}

// Results returns a channel for consuming results
func (wp *WorkerPool) Results() <-chan Result {
    return wp.resultQueue
}

// Stop gracefully shuts down the worker pool
func (wp *WorkerPool) Stop() {
    wp.cancel()
    close(wp.jobQueue)
    wp.wg.Wait()
    close(wp.resultQueue)
}

func main() {
    // Create worker pool with 5 workers
    pool := NewWorkerPool(5, 100)
    pool.Start()

    // Create context with timeout
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    // Submit 50 jobs
    go func() {
        for i := 0; i < 50; i++ {
            job := Job{
                ID:      i,
                Payload: fmt.Sprintf("data-%d", i),
            }
            if !pool.Submit(job) {
                fmt.Printf("Failed to submit job %d\n", i)
            }
        }
    }()

    // Collect results
    results := pool.Results()
    processed := 0

    for {
        select {
        case <-ctx.Done():
            fmt.Println("Timeout reached")
            pool.Stop()
            fmt.Printf("Processed: %d jobs\n", processed)

```

```

        return
    case result, ok := <-results:
        if !ok {
            fmt.Println("All jobs processed")
            fmt.Printf("Processed: %d jobs\n", processed)
            return
        }
        processed++
        fmt.Printf("Result: %s\n", result.Output)
    }
}

```

ตัวอย่าง 3: Redis Cache Integration

```

package cache

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "github.com/redis/go-redis/v9"
)

// RedisCache provides caching functionality using Redis
type RedisCache struct {
    client *redis.Client
    ttl    time.Duration
}

// NewRedisCache creates a new Redis cache client
func NewRedisCache(addr, password string, db int, ttl time.Duration)
*RedisCache {
    client := redis.NewClient(&redis.Options{
        Addr:      addr,
        Password: password,
        DB:        db,
    })

    return &RedisCache{
        client: client,
        ttl:    ttl,
    }
}

// Set stores a value in cache
func (c *RedisCache) Set(ctx context.Context, key string, value interface{})
error {

```



```

    data, err := json.Marshal(value)
    if err != nil {
        return fmt.Errorf("failed to marshal value: %w", err)
    }

    return c.client.Set(ctx, key, data, c.ttl).Err()
}

// Get retrieves a value from cache
func (c *RedisCache) Get(ctx context.Context, key string, dest interface{}) error {
    data, err := c.client.Get(ctx, key).Bytes()
    if err != nil {
        if err == redis.Nil {
            return ErrCacheMiss
        }
        return fmt.Errorf("failed to get from cache: %w", err)
    }

    if err := json.Unmarshal(data, dest); err != nil {
        return fmt.Errorf("failed to unmarshal cached data: %w", err)
    }

    return nil
}

// Delete removes a value from cache
func (c *RedisCache) Delete(ctx context.Context, key string) error {
    return c.client.Del(ctx, key).Err()
}

// Exists checks if a key exists in cache
func (c *RedisCache) Exists(ctx context.Context, key string) (bool, error) {
    result, err := c.client.Exists(ctx, key).Result()
    if err != nil {
        return false, err
    }
    return result > 0, nil
}

// ErrCacheMiss indicates that the requested key was not found
var ErrCacheMiss = fmt.Errorf("cache miss")

// Usage example
func Example() {
    ctx := context.Background()
    cache := NewRedisCache("localhost:6379", "", 0, 10*time.Minute)

    // Store user data
    user := map[string]interface{}{
        "id":    123,
        "name":  "John Doe",
        "email": "john@example.com",
    }

```

```

    }

    if err := cache.Set(ctx, "user:123", user); err != nil {
        fmt.Printf("Cache set error: %v\n", err)
    }

    // Retrieve user data
    var cachedUser map[string]interface{}
    if err := cache.Get(ctx, "user:123", &cachedUser); err != nil {
        if err == ErrCacheMiss {
            fmt.Println("User not found in cache")
        } else {
            fmt.Printf("Cache get error: %v\n", err)
        }
    } else {
        fmt.Printf("Cached user: %v\n", cachedUser)
    }
}

```

ตัวอย่าง 4: Generic Repository Pattern (Go 1.18+)

```

package repository

import (
    "context"

    "gorm.io/gorm"
)

// Entity defines the interface that all entities must implement
type Entity interface {
    GetID() string
}

// Repository is a generic repository interface
type Repository[T Entity] interface {
    Create(ctx context.Context, entity T) error
    GetByID(ctx context.Context, id string) (T, error)
    Update(ctx context.Context, entity T) error
    Delete(ctx context.Context, id string) error
    Find(ctx context.Context, query Query) ([]T, error)
}

// Query represents search criteria
type Query struct {
    Filters map[string]interface{}
    Limit   int
    Offset  int
    OrderBy string
}

```

```

// GormRepository is a generic GORM implementation
type GormRepository[T Entity] struct {
    db *gorm.DB
}

// NewGormRepository creates a new generic GORM repository
func NewGormRepository[T Entity](db *gorm.DB) *GormRepository[T] {
    return &GormRepository[T]{db: db}
}

// Create inserts a new entity
func (r *GormRepository[T]) Create(ctx context.Context, entity T) error {
    return r.db.WithContext(ctx).Create(entity).Error
}

// GetByID retrieves an entity by ID
func (r *GormRepository[T]) GetByID(ctx context.Context, id string) (T, error) {
    {
        var entity T
        err := r.db.WithContext(ctx).Where("id = ?", id).First(&entity).Error
        return entity, err
    }
}

// Update updates an existing entity
func (r *GormRepository[T]) Update(ctx context.Context, entity T) error {
    return r.db.WithContext(ctx).Save(entity).Error
}

// Delete removes an entity by ID
func (r *GormRepository[T]) Delete(ctx context.Context, id string) error {
    var entity T
    return r.db.WithContext(ctx).Where("id = ?", id).Delete(&entity).Error
}

// Find finds entities matching the query
func (r *GormRepository[T]) Find(ctx context.Context, query Query) ([]T, error) {
    {
        var entities []T
        db := r.db.WithContext(ctx)

        for field, value := range query.Filters {
            db = db.Where(field+" = ?", value)
        }

        if query.Limit > 0 {
            db = db.Limit(query.Limit)
        }
        if query.Offset > 0 {
            db = db.Offset(query.Offset)
        }
        if query.OrderBy != "" {
            db = db.Order(query.OrderBy)
        }
    }
}

```

```

    }

    err := db.Find(&entities).Error
    return entities, err
}

// Example entity
type Product struct {
    ID    string `gorm:"primaryKey"`
    Name  string `gorm:"size:100;not null"`
    Price float64 `gorm:"not null"`
}

func (p Product) GetID() string { return p.ID }

// Usage example
func ExampleUsage() {
    db, _ := gorm.Open(postgres.Open(dsn), &gorm.Config{})
    productRepo := NewGormRepository[Product](db)

    ctx := context.Background()

    // Create
    product := Product{ID: "1", Name: "Laptop", Price: 999.99}
    productRepo.Create(ctx, product)

    // Find
    products, _ := productRepo.Find(ctx, Query{
        Filters: map[string]interface{}{"name": "Laptop"},
        Limit:   10,
    })

    // Get by ID
    found, _ := productRepo.GetByID(ctx, "1")
}

```

mop Config – การจัดการ Configuration

ไฟล์ config/config.yaml

```

# config/config.yaml
server:
  port: 8080
  mode: release
  read_timeout: 15s
  write_timeout: 15s

database:
  host: localhost

```

```

port: 5432
user: postgres
password: postgres
name: userdb
sslmode: disable
max_open_conns: 25
max_idle_conns: 25
conn_max_lifetime: 5m

redis:
  addr: localhost:6379
  password: ""
  db: 0
  pool_size: 10
  ttl: 10m

jwt:
  secret: "your-secret-key-change-in-production"
  access_expiry: 15m
  refresh_expiry: 168h # 7 days

smtp:
  host: smtp.gmail.com
  port: 587
  username: your-email@gmail.com
  password: your-app-password
  from: your-email@gmail.com

log:
  level: info
  format: json
  output: stdout

```

ไฟล์ config/config.go

```

package config

import (
    "fmt"
    "time"

    "github.com/spf13/viper"
)

// Config holds all application configuration
type Config struct {
    Server      ServerConfig    `mapstructure:"server"`
    Database    DatabaseConfig  `mapstructure:"database"`
    Redis       RedisConfig     `mapstructure:"redis"`
    JWT         JWTConfig       `mapstructure:"jwt"`
}

```

```

SMTP      SMTPConfig      `mapstructure:"smtp"`
Log        LogConfig       `mapstructure:"log"`
}

// ServerConfig holds HTTP server configuration
type ServerConfig struct {
    Port      int      `mapstructure:"port"`
    Mode      string   `mapstructure:"mode"`
    ReadTimeout time.Duration `mapstructure:"read_timeout"`
    WriteTimeout time.Duration `mapstructure:"write_timeout"`
}

// DatabaseConfig holds database configuration
type DatabaseConfig struct {
    Host      string   `mapstructure:"host"`
    Port      int      `mapstructure:"port"`
    User      string   `mapstructure:"user"`
    Password  string   `mapstructure:"password"`
    Name      string   `mapstructure:"name"`
    SSLMode   string   `mapstructure:"sslmode"`
    MaxOpenConns int     `mapstructure:"max_open_conns"`
    MaxIdleConns int     `mapstructure:"max_idle_conns"`
    ConnMaxLifetime time.Duration `mapstructure:"conn_max_lifetime"`
}

// RedisConfig holds Redis configuration
type RedisConfig struct {
    Addr      string   `mapstructure:"addr"`
    Password  string   `mapstructure:"password"`
    DB        int      `mapstructure:"db"`
    PoolSize  int      `mapstructure:"pool_size"`
    TTL       time.Duration `mapstructure:"ttl"`
}

// JWTConfig holds JWT configuration
type JWTConfig struct {
    Secret      string   `mapstructure:"secret"`
    AccessExpiry time.Duration `mapstructure:"access_expiry"`
    RefreshExpiry time.Duration `mapstructure:"refresh_expiry"`
}

// SMTPConfig holds email configuration
type SMTPConfig struct {
    Host      string `mapstructure:"host"`
    Port      int    `mapstructure:"port"`
    Username  string `mapstructure:"username"`
    Password  string `mapstructure:"password"`
    From      string `mapstructure:"from"`
}

// LogConfig holds logging configuration
type LogConfig struct {
    Level string `mapstructure:"level"`
}

```

```

    Format string `mapstructure:"format"`
    Output string `mapstructure:"output"`
}

// LoadConfig loads configuration from file and environment variables
func LoadConfig(configPath string) (*Config, error) {
    viper.SetConfigFile(configPath)
    viper.SetConfigType("yaml")

    // Set default values
    setDefaults()

    // Read config file
    if err := viper.ReadInConfig(); err != nil {
        return nil, fmt.Errorf("failed to read config file: %w", err)
    }

    // Bind environment variables
    viper.SetEnvPrefix("app")
    viper.AutomaticEnv()

    // Unmarshal into struct
    var cfg Config
    if err := viper.Unmarshal(&cfg); err != nil {
        return nil, fmt.Errorf("failed to unmarshal config: %w", err)
    }

    return &cfg, nil
}

func setDefaults() {
    viper.SetDefault("server.port", 8080)
    viper.SetDefault("server.mode", "debug")
    viper.SetDefault("server.read_timeout", "15s")
    viper.SetDefault("server.write_timeout", "15s")

    viper.SetDefault("database.max_open_conns", 25)
    viper.SetDefault("database.max_idle_conns", 25)
    viper.SetDefault("database.conn_max_lifetime", "5m")

    viper.SetDefault("redis.pool_size", 10)
    viper.SetDefault("redis.ttl", "10m")

    viper.SetDefault("jwt.access_expiry", "15m")
    viper.SetDefault("jwt.refresh_expiry", "168h")

    viper.SetDefault("log.level", "info")
    viper.SetDefault("log.format", "json")
    viper.SetDefault("log.output", "stdout")
}

// GetDSN returns PostgreSQL connection string
func (c *DatabaseConfig) GetDSN() string {
    return fmt.Sprintf(

```

```

        "host=%s port=%d user=%s password=%s dbname=%s sslmode=%s
        TimeZone=Asia/Bangkok",
        c.Host, c.Port, c.User, c.Password, c.Name, c.SSLMode,
    )
}

// GetRedisAddr returns Redis address
func (c *RedisConfig) GetAddr() string {
    return c.Addr
}

```

ไฟล์ main.go ที่ใช้ config

```

package main

import (
    "log"
    "user-service/internal/config"
)

func main() {
    // Load configuration
    cfg, err := config.LoadConfig("config/config.yaml")
    if err != nil {
        log.Fatalf("Failed to load config: %v", err)
    }

    // Use configuration
    log.Printf("Server starting on port %d", cfg.Server.Port)
    log.Printf("Database: %s", cfg.Database.GetDSN())
}

```

GORM CRUD กับฐานข้อมูลหลายประเภท: PostgreSQL, MySQL, MongoDB, InfluxDB

แผนภาพ Data Flow (Flowchart TB)

```

graph TD
    subgraph App ["Go Application"]
        A1["Business Logic / Use Cases"]
        A2["GORM Models / Structs"]
        A3["Repository Layer"]
    end

    subgraph ORM ["ORM Layer (GORM)"]
        A2
        A3
    end

```



```

    G1["GORM ORM<br/>- Auto Migration<br/>- CRUD Operations<br/>-
Associations<br/>- Hooks"]
    end

    subgraph SQL["SQL Databases"]
        Pg[(PostgreSQL)]
        My[(MySQL)]
    end

    subgraph NoSQL["NoSQL / Time-Series"]
        Mg[(MongoDB)]
        Inf[(InfluxDB)]
    end

    subgraph Drivers["Database Drivers"]
        D1["pgx / lib/pq"]
        D2["go-sql-driver/mysql"]
        D3["mongo-go-driver"]
        D4["influxdb-client-go"]
    end

    A1 --> A2
    A2 --> A3
    A3 --> G1

    G1 --> D1
    D1 --> Pg

    G1 --> D2
    D2 --> My

    A3 --> D3
    D3 --> Mg

    A3 --> D4
    D4 --> Inf

```

คำอธิบายภาษาไทย (แบบละเอียด)

ใน Go การทำ CRUD กับฐานข้อมูลสามารถใช้ไลบรารีที่เหมาะสมกับแต่ละประเภท:

- **GORM** – ORM ที่ทรงพลัง เหมาะกับฐานข้อมูล SQL (PostgreSQL, MySQL, SQLite, SQL Server)
- **MongoDB** – ใช้ official driver (go.mongodb.org/mongo-driver) สำหรับ document database
- **InfluxDB** – ใช้ official client (github.com/influxdata/influxdb-client-go) สำหรับ time-series database

แผนภาพด้านบนแสดงการแยกชั้น: **Application** ใช้ **Repository Layer** ซึ่งเรียกใช้ ORM หรือ driver โดยตรง ตามชนิดของฐานข้อมูล

1. PostgreSQL + GORM

การติดตั้ง

```
go get -u gorm.io/gorm
go get -u gorm.io/driver/postgres
```

ตัวอย่าง Model และ CRUD

```
package models

import (
    "time"
    "gorm.io/gorm"
)

type User struct {
    ID          uint           `gorm:"primaryKey" json:"id"`
    Username    string         `gorm:"size:50;uniqueIndex;not null" json:"username"`
    Email       string         `gorm:"size:100;uniqueIndex;not null" json:"email"`
    Password    string         `gorm:"size:255;not null" json:"-"`
    CreatedAt   time.Time      `json:"created_at"`
    UpdatedAt   time.Time      `json:"updated_at"`
    DeletedAt   gorm.DeletedAt `gorm:"index" json:"-"`
}
```

```
package repository

import (
    "context"
    "errors"
    "gorm.io/gorm"
    "myapp/models"
)

type UserRepository struct {
    db *gorm.DB
}

func NewUserRepository(db *gorm.DB) *UserRepository {
    return &UserRepository{db: db}
}

// Create
func (r *UserRepository) Create(ctx context.Context, user *models.User) error {
```

```

        return r.db.WithContext(ctx).Create(user).Error
    }

// Read by ID
func (r *UserRepository) GetByID(ctx context.Context, id uint) (*models.User,
error) {
    var user models.User
    err := r.db.WithContext(ctx).First(&user, id).Error
    if errors.Is(err, gorm.ErrRecordNotFound) {
        return nil, nil
    }
    return &user, err
}

// Read by email
func (r *UserRepository) GetByEmail(ctx context.Context, email string)
(*models.User, error) {
    var user models.User
    err := r.db.WithContext(ctx).Where("email = ?", email).First(&user).Error
    if errors.Is(err, gorm.ErrRecordNotFound) {
        return nil, nil
    }
    return &user, err
}

// Update
func (r *UserRepository) Update(ctx context.Context, user *models.User) error {
    return r.db.WithContext(ctx).Save(user).Error
}

// Delete (soft delete)
func (r *UserRepository) Delete(ctx context.Context, id uint) error {
    return r.db.WithContext(ctx).Delete(&models.User{}, id).Error
}

// List with pagination
func (r *UserRepository) List(ctx context.Context, page, limit int)
([]*models.User, int64, error) {
    var users []*models.User
    var total int64

    offset := (page - 1) * limit
    query := r.db.WithContext(ctx).Model(&models.User{})

    if err := query.Count(&total).Error; err != nil {
        return nil, 0, err
    }

    err := query.Offset(offset).Limit(limit).Find(&users).Error
    return users, total, err
}

```

การเชื่อมต่อ

```
import (  
    "gorm.io/driver/postgres"  
    "gorm.io/gorm"  
)  
  
func ConnectPostgres(dsn string) (*gorm.DB, error) {  
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})  
    if err != nil {  
        return nil, err  
    }  
    // Auto migrate  
    db.AutoMigrate(&models.User{})  
    return db, nil  
}
```

DSN format: host=localhost user=gorm password=gorm dbname=gorm port=5432
sslmode=disable TimeZone=Asia/Bangkok

2. MySQL + GORM

การติดตั้ง

```
go get -u gorm.io/driver/mysql
```

ตัวอย่าง Model และ CRUD (คล้ายกับ PostgreSQL)

Model เหมือนกัน เพียงแต่ driver ต่างกัน

```
import (  
    "gorm.io/driver/mysql"  
    "gorm.io/gorm"  
)  
  
func ConnectMySQL(dsn string) (*gorm.DB, error) {  
    db, err := gorm.Open(mysql.Open(dsn), &gorm.Config{})  
    if err != nil {  
        return nil, err  
    }  
    db.AutoMigrate(&models.User{})  
    return db, nil  
}
```

DSN format: `user:pass@tcp(127.0.0.1:3306)/dbname?charset=utf8mb4&parseTime=True&loc=Local`

ความแตกต่างที่ควรระวัง

- **Auto-increment:** MySQL ใช้ `AUTO_INCREMENT`, PostgreSQL ใช้ `SERIAL` หรือ `BIGSERIAL` – GORM จัดการให้อัตโนมัติ
- **JSON type:** PostgreSQL รองรับ JSONB โดยตรง, MySQL ต้องใช้ `json` (เวอร์ชัน 5.7+) – GORM จะ map เป็น string หรือ struct ตาม tags
- **Case sensitivity:** ชื่อตารางและคอลัมน์ใน MySQL บน Windows/Unix อาจ sensitive ขึ้นอยู่กับ configuration

3. MongoDB (NoSQL Document Database)

MongoDB ไม่ใช่ GORM แต่ใช้ official driver `mongo-go-driver`

การติดตั้ง

```
go get go.mongodb.org/mongo-driver/mongo
go get go.mongodb.org/mongo-driver/bson
```

ตัวอย่าง Model และ CRUD

```
package models

import (
    "time"
    "go.mongodb.org/mongo-driver/bson/primitive"
)

type User struct {
    ID          primitive.ObjectID `bson:"_id,omitempty" json:"id"`
    Username    string              `bson:"username" json:"username"`
    Email       string              `bson:"email" json:"email"`
    Password    string              `bson:"password" json:"-"`
    CreatedAt   time.Time           `bson:"created_at" json:"created_at"`
    UpdatedAt   time.Time           `bson:"updated_at" json:"updated_at"`
}
```

```
package repository

import (
    "context"
    "time"

    "go.mongodb.org/mongo-driver/bson"
```

```

    "go.mongodb.org/mongo-driver/bson/primitive"
    "go.mongodb.org/mongo-driver/mongo"
    "go.mongodb.org/mongo-driver/mongo/options"
)

type UserRepository struct {
    collection *mongo.Collection
}

func NewUserRepository(db *mongo.Database) *UserRepository {
    return &UserRepository{
        collection: db.Collection("users"),
    }
}

// Create
func (r *UserRepository) Create(ctx context.Context, user *models.User) error {
    user.ID = primitive.NewObjectID()
    user.CreatedAt = time.Now()
    user.UpdatedAt = time.Now()
    _, err := r.collection.InsertOne(ctx, user)
    return err
}

// Read by ID
func (r *UserRepository) GetByID(ctx context.Context, id string) (*models.User, error) {
    objID, err := primitive.ObjectIDFromHex(id)
    if err != nil {
        return nil, err
    }
    var user models.User
    err = r.collection.FindOne(ctx, bson.M{"_id": objID}).Decode(&user)
    if err == mongo.ErrNoDocuments {
        return nil, nil
    }
    return &user, err
}

// Read by email
func (r *UserRepository) GetByEmail(ctx context.Context, email string) (*models.User, error) {
    var user models.User
    err := r.collection.FindOne(ctx, bson.M{"email": email}).Decode(&user)
    if err == mongo.ErrNoDocuments {
        return nil, nil
    }
    return &user, err
}

// Update
func (r *UserRepository) Update(ctx context.Context, user *models.User) error {
    user.UpdatedAt = time.Now()

```

```

    filter := bson.M{"_id": user.ID}
    update := bson.M{"$set": user}
    _, err := r.collection.UpdateOne(ctx, filter, update)
    return err
}

// Delete
func (r *UserRepository) Delete(ctx context.Context, id string) error {
    objID, err := primitive.ObjectIDFromHex(id)
    if err != nil {
        return err
    }
    _, err = r.collection.DeleteOne(ctx, bson.M{"_id": objID})
    return err
}

// List with pagination
func (r *UserRepository) List(ctx context.Context, page, limit int)
([]*models.User, int64, error) {
    var users []*models.User
    skip := int64((page - 1) * limit)
    limit64 := int64(limit)

    opts := options.Find().SetSkip(skip).SetLimit(limit64)
    cursor, err := r.collection.Find(ctx, bson.M{}, opts)
    if err != nil {
        return nil, 0, err
    }
    defer cursor.Close(ctx)

    if err = cursor.All(ctx, &users); err != nil {
        return nil, 0, err
    }

    total, err := r.collection.CountDocuments(ctx, bson.M{})
    return users, total, err
}

```

การเชื่อมต่อ

```

import (
    "context"
    "go.mongodb.org/mongo-driver/mongo"
    "go.mongodb.org/mongo-driver/mongo/options"
)

func ConnectMongo(uri string) (*mongo.Database, error) {
    client, err := mongo.Connect(context.Background(),
    options.Client().ApplyURI(uri))
    if err != nil {

```

```

        return nil, err
    }
    // Ping to verify connection
    if err = client.Ping(context.Background(), nil); err != nil {
        return nil, err
    }
    return client.Database("myapp"), nil
}

```

URI format: `mongodb://localhost:27017`

4. InfluxDB (Time-Series Database)

InfluxDB ใช้ client เฉพาะ `influxdb-client-go`

การติดตั้ง

```
go get github.com/influxdata/influxdb-client-go/v2
```

ตัวอย่าง Model และ CRUD

InfluxDB ไม่มีแนวคิดของ "document" เหมือน MongoDB แต่เป็น **measurement + tags + fields** เหมาะสำหรับข้อมูลอนุกรมเวลา (เช่น อุณหภูมิ, ความชื้น)

```

package repository

import (
    "context"
    "time"

    influxdb2 "github.com/influxdata/influxdb-client-go/v2"
    "github.com/influxdata/influxdb-client-go/v2/api"
)

type SensorData struct {
    DeviceID string
    Location string
    Temp     float64
    Humidity float64
    Timestamp time.Time
}

type InfluxRepository struct {
    client    influxdb2.Client
    writeAPI  api.WriteAPIBlocking
    queryAPI  api.QueryAPI
    org       string
}

```



```

    bucket    string
}

func NewInfluxRepository(url, token, org, bucket string) (*InfluxRepository,
error) {
    client := influxdb2.NewClient(url, token)
    writeAPI := client.WriteAPIBlocking(org, bucket)
    queryAPI := client.QueryAPI(org)

    // Test connection
    _, err := queryAPI.Query(context.Background(), `buckets()`)
    if err != nil {
        return nil, err
    }

    return &InfluxRepository{
        client:    client,
        writeAPI:  writeAPI,
        queryAPI:  queryAPI,
        org:       org,
        bucket:    bucket,
    }, nil
}

// Create (Write point)
func (r *InfluxRepository) WriteSensorData(ctx context.Context, data
SensorData) error {
    p := influxdb2.NewPoint(
        "sensor_data",
        map[string]string{
            "device_id": data.DeviceID,
            "location": data.Location,
        },
        map[string]interface{}{
            "temperature": data.Temp,
            "humidity":   data.Humidity,
        },
        data.Timestamp,
    )
    return r.writeAPI.WritePoint(ctx, p)
}

// Read (Query) - example: get average temperature per device for last hour
func (r *InfluxRepository) GetAverageTemp(ctx context.Context, deviceID string,
duration time.Duration) (float64, error) {
    start := time.Now().Add(-duration)
    flux := `from(bucket:"` + r.bucket + `")
        |> range(start: ` + start.Format(time.RFC3339) + `)
        |> filter(fn: (r) => r._measurement == "sensor_data")
        |> filter(fn: (r) => r.device_id == "` + deviceID + `")
        |> filter(fn: (r) => r._field == "temperature")
        |> mean()`

```

```

    result, err := r.queryAPI.Query(ctx, flux)
    if err != nil {
        return 0, err
    }

    var avg float64
    for result.Next() {
        record := result.Record()
        if v, ok := record.Value().(float64); ok {
            avg = v
        }
    }
    return avg, result.Err()
}

// Delete (hard delete) - InfluxDB does not support single point deletion
// easily; usually you drop data by time range.
func (r *InfluxRepository) DeleteByTimeRange(ctx context.Context, start, stop
time.Time) error {
    deleteAPI := r.client.DeleteAPI()
    return deleteAPI.Delete(ctx, r.org, r.bucket, start, stop, "")
}

// Close
func (r *InfluxRepository) Close() {
    r.client.Close()
}

```

การเชื่อมต่อ

```

func main() {
    repo, err := NewInfluxRepository(
        "http://localhost:8086",
        "my-token",
        "my-org",
        "my-bucket",
    )
    if err != nil {
        panic(err)
    }
    defer repo.Close()
}

```

สรุปตารางเปรียบเทียบ

ฐานข้อมูล	ไดรเวอร์ / ORM	CRUD วิธีหลัก	เหมาะกับงาน
-----------	----------------	---------------	-------------

ฐานข้อมูล	ไดรเวอร์ / ORM	CRUD วิธีหลัก	เหมาะกับงาน
PostgreSQL	GORM + gorm.io/driver/postgres	Create, First, Where, Save, Delete	ระบบที่มีความสัมพันธ์ซับซ้อน ต้องการ ACID
MySQL	GORM + gorm.io/driver/mysql	เช่นเดียวกับ PostgreSQL	Web application ขนาดเล็กถึงกลาง
MongoDB	mongo-go-driver	InsertOne, FindOne, UpdateOne, DeleteOne	Document store, schema-less, scaling สูง
InfluxDB	influxdb-client-go	WritePoint, Query (Flux)	Time-series data, IoT, metrics

ข้อแนะนำในการเลือกใช้:

- หากต้องการ ORM ที่ครบฟีเจอ์และใช้ SQL → **GORM + PostgreSQL/MySQL**
- หากต้องการความยืดหยุ่นของ schema และ scale แนวนอน → **MongoDB**
- หากต้องการเก็บข้อมูลอนุกรมเวลา (เช่น ค่าเซ็นเซอร์) → **InfluxDB**

เทมเพลตและตัวอย่างโค้ดเพิ่มเติม

Repository Interface แบบรวมสำหรับทุกฐานข้อมูล (เพื่อให้ application layer ไม่ขึ้นกับ DB)

```
type UserRepository interface {
    Create(ctx context.Context, user *User) error
    GetByID(ctx context.Context, id string) (*User, error)
    GetByEmail(ctx context.Context, email string) (*User, error)
    Update(ctx context.Context, user *User) error
    Delete(ctx context.Context, id string) error
    List(ctx context.Context, page, limit int) ([]*User, int64, error)
}
```

จากนั้น implement สำหรับแต่ละฐานข้อมูลตามตัวอย่างที่ให้ไว้

การทดสอบด้วย Mock (ตัวอย่างสำหรับ GORM)

```
// ใช้ gorm.io/driver/sqlite สำหรับทดสอบ
func TestUserRepository(t *testing.T) {
    db, _ := gorm.Open(sqlite.Open("file::memory:?cache=shared"),
        &gorm.Config{})
    db.AutoMigrate(&models.User{})
    repo := NewUserRepository(db)

    // ทดสอบ Create
    user := &models.User{Username: "test", Email: "test@example.com", Password:
        "pass"}
    err := repo.Create(context.Background(), user)
```

```
assert.NoError(t, err)
assert.NotZero(t, user.ID)
}
```

แผนภาพและตัวอย่างโค้ดนี้ครอบคลุมการทำ CRUD กับฐานข้อมูลหลัก 4 ประเภทที่ใช้อยู่ใน Go โดยเน้นการใช้ GORM สำหรับ SQL และไดรเวอร์เฉพาะสำหรับ NoSQL/Time-series พร้อมแนวทางการแยก repository layer เพื่อให้แอปพลิเคชันสามารถเปลี่ยนฐานข้อมูลได้ง่ายในอนาคต

📌 สรุป

คู่มือภาษา Go ฉบับสมบูรณ์นี้ครอบคลุมเนื้อหาตั้งแต่พื้นฐานภาษา Go ไปจนถึงการออกแบบสถาปัตยกรรมระดับองค์กรและการผสานระบบภายนอกที่ใช้ในโลกแห่งความเป็นจริง

จุดเด่นของคู่มือ

1. **ครบถ้วนสมบูรณ์** - ครอบคลุมทุกหัวข้อตั้งแต่พื้นฐานจนถึงขั้นสูง
2. **แผนภาพประกอบ** - มีแผนภาพในรูปแบบ Mermaid/draw.io สำหรับอธิบายโครงสร้างและกระบวนการทำงาน
3. **โค้ดที่รันได้จริง** - ตัวอย่างโค้ดทั้งหมดสามารถนำไปรันทดสอบได้
4. **เทมเพลตและ Checklist** - เครื่องมือช่วยในการพัฒนาและการนำเสนอซอฟต์แวร์
5. **แนวทางปฏิบัติ** - สอดแทรก best practices และข้อควรระวัง

การนำคู่มือไปใช้

- **ผู้เริ่มต้น** ควรศึกษาเนื้อหาตามลำดับตั้งแต่ภาคที่ 1 ถึงภาคที่ 3
- **นักพัฒนาที่มีประสบการณ์** สามารถข้ามไปยังภาคที่ 7-9 เพื่อศึกษา Clean Architecture, DDD และการผสานระบบภายนอก
- **ทีมพัฒนา** สามารถนำ Task List Template และ Checklist Template ไปปรับใช้ในการทำงาน

คู่มือภาษา Go ฉบับ นำไปทำงานจริง

ครอบคลุมทุกมิติ ตั้งแต่พื้นฐานสู่สถาปัตยกรรมระดับองค์กร พร้อมแผนภาพและโค้ดตัวอย่างที่รันได้จริง

Version 3.0 – เมษายน 2026

📖 บทนำ

ความเป็นมาของคู่มือ

ในยุคที่ซอฟต์แวร์มีความซับซ้อนมากขึ้นเรื่อย ๆ ภาษาโปรแกรมมิ่งที่เรียบง่าย มีประสิทธิภาพสูง และสามารถจัดการกับการทำงานพร้อมกันได้ดี กลายเป็นสิ่งที่นักพัฒนาต้องการอย่างยิ่ง ภาษา Go (หรือ Golang) ถือกำเนิดขึ้นจากความต้องการของ engineers ที่ Google ซึ่งเผชิญกับความท้าทายในการพัฒนาและบำรุงรักษาระบบขนาดใหญ่ที่มีการทำงานพร้อมกันสูง พวกเขาต้องการภาษาใหม่ที่เหมาะสมความรวดเร็วในการทำงานของภาษา C ความง่ายของภาษา Python และความสามารถในการจัดการ concurrency ที่ดีขึ้น

คู่มือเล่มนี้เกิดจากความตั้งใจที่จะรวบรวมองค์ความรู้เกี่ยวกับภาษา Go ตั้งแต่ระดับพื้นฐานจนถึงระดับมืออาชีพ ครอบคลุมทั้งไวยากรณ์พื้นฐาน การจัดการโปรเจกต์ด้วย Go Modules การทดสอบหน่วย การทำงานพร้อมกัน (concurrency) ไปจนถึงการออกแบบสถาปัตยกรรมระดับ Production และการประยุกต์ใช้ Domain-Driven Design (DDD) ร่วมกับ Go

วัตถุประสงค์

คู่มือนี้ถูกออกแบบให้เป็น **ทั้งตำราเรียนและคู่มืออ้างอิง** โดยเน้นให้ผู้อ่านสามารถนำไปประยุกต์ใช้ได้ทันที ตั้งแต่การติดตั้ง การเขียนโปรแกรมพื้นฐาน ไปจนถึงการออกแบบสถาปัตยกรรมแบบ Clean Architecture และ Domain-Driven Design (DDD) รวมถึงการเชื่อมต่อกับระบบภายนอกที่พบได้บ่อยในโลกแห่งความจริง

สิ่งที่คุณจะได้รับ

- ความเข้าใจภาษา Go อย่างลึกซึ้ง ตั้งแต่ไวยากรณ์จนถึง concurrency
- แนวทางการจัดโครงสร้างโปรเจกต์สำหรับการผลิตจริง
- เทคนิคการทดสอบหน่วย (Unit Test) และการวัดประสิทธิภาพ
- รูปแบบสถาปัตยกรรม Clean Architecture + DDD + CQRS
- การผสาน Redis, RabbitMQ, MQTT, InfluxDB, WebSocket, SMS, LINE Notify, Discord
- เทมเพลตและ checklist ที่ช่วยให้ทีมงานเป็นระบบ
- แผนภาพ (draw.io) สำหรับอธิบายโครงสร้างและกระบวนการทำงาน
- โค้ดตัวอย่างที่สามารถนำไปรันทดสอบได้จริง

กลุ่มเป้าหมาย

- **ผู้เริ่มต้น** ที่ต้องการเรียนรู้ภาษา Go ตั้งแต่ศูนย์
- **นักพัฒนาที่เปลี่ยนภาษา** จากภาษาอื่นมาสู่ Go
- **นักพัฒนาที่ต้องการยกระดับ** สู่การเป็น Go Developer มืออาชีพ
- **สถาปนิกซอฟต์แวร์** ที่สนใจการออกแบบระบบด้วย Go

วิธีการอ่าน

- หากยังไม่เคยเขียน Go มาก่อน ให้เริ่มจาก **ภาคที่ 1-3** เพื่อทำความเข้าใจพื้นฐาน
- หากต้องการออกแบบแอปพลิเคชันทันที ให้ข้ามไป **ภาคที่ 7-8** เพื่อศึกษา Clean Architecture และ DDD
- หากต้องการเชื่อมต่อกับระบบอื่น (ฐานข้อมูล time-series, message queue, IoT) ให้ดู **ภาคที่ 9**

บทนิยาม (Glossary)

ความหมายของคำศัพท์เฉพาะทาง

คำศัพท์	คำอธิบาย
Go (Golang)	ภาษาโปรแกรมมิ่งที่พัฒนาโดย Google เปิดตัวในปี 2009 ออกแบบมาเพื่อการพัฒนา software ที่มีประสิทธิภาพสูง จัดการ concurrency ได้ดี และมีไวยากรณ์ที่เรียบง่าย
Goroutine	เธรดขนาดเล็กที่ถูกจัดการโดย Go runtime ใช้สำหรับการทำงานแบบ concurrent การสร้างทำได้โดยใช้ 키워ด <code>go</code> หน้าฟังก์ชัน
Channel	โครงสร้างข้อมูลที่ใช้ในการสื่อสารระหว่าง goroutine ช่วยให้ส่งข้อมูลระหว่างกันได้ อย่างปลอดภัย

คำศัพท์	คำอธิบาย
Compiler	โปรแกรมที่แปลงซอร์สโค้ดภาษา Go ให้เป็นไฟล์ binary ที่เครื่องสามารถรันได้โดยตรง
Go Modules	ระบบจัดการ dependencies อย่างเป็นทางการของ Go เริ่มใช้ตั้งแต่ Go 1.11 ทำให้ไม่ต้องพึ่งพา GOPATH อีกต่อไป
Interface	ชนิดข้อมูลที่กำหนดชุดของ method signatures ชนิดใดก็ตามที่มี method ครบตามที่กำหนด จะถือว่า implement interface นั้นโดยอัตโนมัติ
Struct	ชนิดข้อมูลที่ใช้รวมฟิลด์หลาย ๆ ชนิดเข้าด้วยกัน คล้ายกับ class ในภาษาอื่น แต่ไม่มี method ในตัว
Pointer	ตัวแปรที่เก็บ address ของตัวแปรอื่น ใช้ & เพื่อ address และ * เพื่อ dereference
Defer	คำสั่งที่ใช้เลื่อนการทำงานของฟังก์ชันออกไปจนกว่าฟังก์ชันรอบนอกจะจบการทำงาน ใช้สำหรับ cleanup หรือพักการ
Panic / Recover	Panic คือการหยุดการทำงานปกติของโปรแกรม Recover ใช้ใน defer เพื่อจับ panic และควบคุมการทำงานต่อ
Clean Architecture	สถาปัตยกรรมซอฟต์แวร์ที่แบ่งเป็น 3 ชั้นหลัก: Delivery (รับส่งข้อมูล), Usecase (business logic), Repository (การเข้าถึงข้อมูล)
DDD (Domain-Driven Design)	แนวทางการออกแบบซอฟต์แวร์ที่เน้นการสร้างโมเดลที่สะท้อนความรู้ความเข้าใจทางธุรกิจ (domain knowledge) อย่างแท้จริง
Aggregate	กลุ่มของ Entities และ Value Objects ที่ถูกจัดการเป็นหน่วยเดียวกัน มี Aggregate Root เป็นตัวควบคุมความสอดคล้องของข้อมูล
CQRS (Command Query Responsibility Segregation)	รูปแบบการออกแบบที่แยกโมเดลการเขียน (Command) และการอ่าน (Query) ออกจากกัน
Ubiquitous Language	ภาษากลางที่ใช้ร่วมกันระหว่างนักพัฒนาและผู้เชี่ยวชาญโดเมน ใช้ศัพท์เดียวกันในโค้ด, การสนทนา, และเอกสาร
Bounded Context	การแบ่งโดเมนขนาดใหญ่ออกเป็นบริบทย่อยที่มีขอบเขตชัดเจน แต่ละบริบทมีโมเดลและภาษาร่วมของตัวเอง
Repository Pattern	การสร้าง abstraction ชั้นกลางระหว่าง business logic และแหล่งข้อมูล
Middleware	ฟังก์ชันที่ห่อ handler เพื่อเพิ่ม logic เช่น logging, auth, rate limit
Value Object	วัตถุที่ไม่มีเอกลักษณ์เฉพาะตัว ถูกกำหนดโดยค่าของมัน (immutable)
Domain Event	เหตุการณ์สำคัญในโดเมนที่เกิดขึ้นระหว่างการทำงานของระบบ

📖 สารบัญ

ภาคที่ 1: ปฐมบทกับการเขียนโปรแกรม

- บทที่ 1: ความรู้เบื้องต้นเกี่ยวกับการเขียนโปรแกรมคอมพิวเตอร์
- บทที่ 2: รู้จักกับภาษา Go

- **บทที่ 3:** พื้นฐานการใช้งาน Terminal
- **บทที่ 4:** เตรียมสภาพแวดล้อมสำหรับพัฒนา
- **บทที่ 5:** สร้างแอปพลิเคชันแรกของคุณ

ภาคที่ 2: พื้นฐานภาษาและโครงสร้างข้อมูล

- **บทที่ 6:** ระบบเลขฐานสองและฐานสิบ
- **บทที่ 7:** เลขฐานสิบหก, ฐานแปด, ASCII, UTF8, Unicode และ Runes
- **บทที่ 8:** ตัวแปร, ค่าคงที่ และชนิดข้อมูลพื้นฐาน
- **บทที่ 9:** คำสั่งควบคุมการทำงาน
- **บทที่ 10:** ฟังก์ชัน
- **บทที่ 11:** แพคเกจและการนำเข้า
- **บทที่ 12:** การเริ่มต้นทำงานของแพคเกจ
- **บทที่ 13:** การสร้างชนิดข้อมูลใหม่ (Types)
- **บทที่ 14:** เมธอด (Methods)
- **บทที่ 15:** พอยน์เตอร์ (Pointer)
- **บทที่ 16:** อินเทอร์เฟซ (Interfaces)

ภาคที่ 3: การจัดการโปรเจกต์และโครงสร้างข้อมูลขั้นสูง

- **บทที่ 17:** Go Modules - การจัดการโปรเจกต์สมัยใหม่
- **บทที่ 18:** Go Module Proxies
- **บทที่ 19:** การทดสอบหน่วย (Unit Tests)
- **บทที่ 20:** อาร์เรย์ (Arrays)
- **บทที่ 21:** สไลซ์ (Slices)
- **บทที่ 22:** แมพ (Maps)
- **บทที่ 23:** การจัดการข้อผิดพลาด (Errors)

ภาคที่ 4: การพัฒนาแอปพลิเคชันเชิงปฏิบัติ

- **บทที่ 24:** ฟังก์ชันนิรนาม (Anonymous functions) และ Closure
- **บทที่ 25:** การจัดการข้อมูล JSON และ XML
- **บทที่ 26:** พื้นฐานการสร้าง HTTP Server
- **บทที่ 27:** Enum, iota และ Bitmask
- **บทที่ 28:** วันที่และเวลา
- **บทที่ 29:** การจัดเก็บข้อมูล: ไฟล์และฐานข้อมูล
- **บทที่ 30:** การทำงานพร้อมกัน (Concurrency)
- **บทที่ 31:** การบันทึกเหตุการณ์ (Logging)
- **บทที่ 32:** เทมเพลต (Templates)
- **บทที่ 33:** การจัดการค่า Configuration

ภาคที่ 5: สู่การเป็นนักพัฒนา Go มืออาชีพ

- **บทที่ 34:** การวัดประสิทธิภาพ (Benchmarks)
- **บทที่ 35:** สร้าง HTTP Client
- **บทที่ 36:** การวิเคราะห์โปรไฟล์ (Program Profiling)
- **บทที่ 37:** การจัดการ Context

- บทที่ 38: Generics - การเขียนโค้ดแบบยืดหยุ่น
- บทที่ 39: Go กับกระบวนทัศน์ OOP?
- บทที่ 40: การอัปเดตหรือดาวน์โหลดไครดเวอร์ชัน Go
- บทที่ 41: คำแนะนำในการออกแบบโค้ดที่ดี
- บทที่ 42: ชีทสรุป (Cheatsheet)

ภาคที่ 6: เครื่องมือและไลบรารียอดนิยม

- บทที่ 43: chi, viper, cobra, zap และเครื่องมือสำคัญ
- บทที่ 44: GORM – ORM ทรงพลังสำหรับ Go
- บทที่ 45: การส่งอีเมลด้วย gmail และ hermes

ภาคที่ 7: การออกแบบสถาปัตยกรรมและ Workflow

- บทที่ 46: Clean Architecture และโครงสร้างโปรเจกต์
- บทที่ 47: Blueprint สำหรับโปรเจกต์ Go ระดับ Production
- บทที่ 48: การออกแบบ Workflow และ Task Management

ภาคที่ 8: Domain-Driven Design (DDD) กับ Go

- บทที่ 49: หลักการ DDD และการนำไปใช้ใน Go
- บทที่ 50: Aggregates, Event Storming และ CQRS
- บทที่ 51: การออกแบบบริการด้วย Go-DDD

ภาคที่ 9: การผสานระบบภายนอกและคุณสมบัติเสริม (Advanced Integrations)

- บทที่ 52: Redis สำหรับ Cache และ Message Queue
- บทที่ 53: RabbitMQ – Message Broker มาตรฐานองค์กร
- บทที่ 54: MQTT สำหรับ IoT และระบบเรียลไทม์
- บทที่ 55: InfluxDB – Time-Series Database
- บทที่ 56: WebSocket และ Socket.IO
- บทที่ 57: การส่ง SMS และ LINE Notify
- บทที่ 58: Discord Webhook สำหรับแจ้งเตือน

ภาคที่ 10: เทมเพลต กระบวนการพัฒนา และตัวอย่างโค้ด

- บทที่ 59: ตัวอย่างโค้ดครบวงจร (Full-stack Example)
- บทที่ 60: Task List Template
- บทที่ 61: Checklist Template
- บทที่ 62: แผนภาพการทำงาน (Workflow Diagram)
- บทที่ 63: mop Config – การจัดการ Configuration

🧐 การออกแบบคู่มือ

ปรัชญาการออกแบบ

คู่มือนี้ถูกออกแบบโดยยึดหลักการเรียนรู้แบบ **"เรียนรู้จากการปฏิบัติ" (Learning by Doing)** เนื้อหาถูกจัดลำดับจากง่ายไปยาก เริ่มจากพื้นฐานที่จำเป็นต่อการเริ่มต้นเขียนโปรแกรม ไปจนถึงหัวข้อขั้นสูงที่นักพัฒนามืออาชีพต้องรู้

โครงสร้างการเรียนรู้

ระดับที่ 1: พื้นฐาน

- ความรู้เบื้องต้นเกี่ยวกับคอมพิวเตอร์และการเขียนโปรแกรม
- ทำความรู้จักกับ Go
- การติดตั้งและเตรียมสภาพแวดล้อม
- เขียนโปรแกรมแรก

ระดับที่ 2: พื้นฐานภาษา

- ตัวแปรและชนิดข้อมูล
- คำสั่งควบคุม
- ฟังก์ชัน
- พอยน์เตอร์
- โครงสร้างข้อมูลพื้นฐาน (array, slice, map)

ระดับที่ 3: การพัฒนาแอปพลิเคชัน

- การจัดการข้อผิดพลาด
- การทำงานกับไฟล์และฐานข้อมูล
- HTTP Server/Client
- การทำงานพร้อมกัน (concurrency)

ระดับที่ 4: เครื่องมือและไลบรารี

- Go Modules
- การทดสอบ
- การวัดประสิทธิภาพ
- ไลบรารียอดนิยม

ระดับที่ 5: การออกแบบสถาปัตยกรรม

- Clean Architecture
- DDD (Domain-Driven Design)
- CQRS และ Event Sourcing

รูปแบบการเรียนรู้แต่ละบท

แต่ละบทในคู่มือมีโครงสร้างที่สอดคล้องกัน:

1. **บทนำ** - อธิบายว่าบทนี้เกี่ยวกับอะไร และทำไมถึงสำคัญ
2. **เนื้อหาหลัก** - อธิบายแนวคิดและทฤษฎี พร้อมตัวอย่างโค้ดประกอบ
3. **ตัวอย่างการประยุกต์ใช้** - กรณีศึกษา หรือการนำไปใช้จริง
4. **ข้อควรระวัง** - ปัญหาที่พบบ่อยและวิธีแก้ไข
5. **แบบฝึกหัด** (สำหรับบทที่เหมาะสม) - เพื่อทบทวนความเข้าใจ

รูปแบบโค้ดตัวอย่าง

โค้ดตัวอย่างในคู่มือใช้รูปแบบที่สอดคล้องกับ Go idiom:

- ใช้ `gofmt` ในการจัดรูปแบบ
- มีการอธิบายบรรทัดสำคัญด้วย comment
- แสดงทั้งการทำงานที่ถูกต้องและข้อผิดพลาดที่พบบ่อย

```
// รูปแบบโค้ดตัวอย่าง
func Example() {
    // การทำงานที่ถูกต้อง
    result, err := doSomething()
    if err != nil {
        // การจัดการ error
        log.Printf("error: %v", err)
        return
    }
    // ใช้ result
}
```

๕ การออกแบบ Workflow

Workflow การเรียนรู้ภาษา Go

```
flowchart TD
    subgraph Phase1[ระยะที่ 1: ศึกษาทฤษฎี]
        A1[อ่านเนื้อหา] --> A2[ทำความเข้าใจแนวคิด]
        A2 --> A3[จดบันทึก]
    end

    subgraph Phase2[ระยะที่ 2: ลงมือปฏิบัติ]
        B1[เขียนโค้ดตามตัวอย่าง] --> B2[ดัดแปลงและทดลอง]
        B2 --> B3[สร้างโปรเจกต์เล็กๆ]
    end

    subgraph Phase3[ระยะที่ 3: ทดสอบและแก้ไข]
        C1[รันโปรแกรม] --> C2[ตรวจสอบผลลัพธ์]
        C2 --> C3[แก้ไขข้อผิดพลาด]
        C3 --> C4[ปรับปรุงประสิทธิภาพ]
    end

    subgraph Phase4[ระยะที่ 4: ทบทวนและสรุป]
        D1[สรุป] --> D2[สร้างเอกสารอ้างอิง]
        D2 --> D3[แชร์ความรู้กับทีม]
    end

    Phase1 --> Phase2 --> Phase3 --> Phase4
```

Workflow การพัฒนาโปรเจกต์ Go

```
flowchart LR
    subgraph Analysis[วิเคราะห์]
        A1[รับ Requirement]
        A2[วิเคราะห์ Domain]
```

```

    A3[ระบุ Use Cases]
end

subgraph Design[ออกแบบ]
    B1[ออกแบบสถาปัตยกรรม]
    B2[กำหนด Interface]
    B3[ออกแบบ Database]
    B4[เลือก Tools]
end

subgraph Development[พัฒนา]
    C1[เขียน Domain Layer]
    C2[เขียน Application Layer]
    C3[เขียน Infrastructure]
    C4[เขียน Interface]
end

subgraph Testing[ทดสอบ]
    D1[Unit Test]
    D2[Integration Test]
    D3[E2E Test]
    D4[Performance Test]
end

subgraph Deploy[นำส่ง]
    E1[Build]
    E2[Deploy to Staging]
    E3[Deploy to Production]
    E4[Monitor]
end

Analysis --> Design --> Development --> Testing --> Deploy

```

Workflow การเพิ่ม Feature ใหม่

```

flowchart TD
    subgraph Phase1[Phase 1: Domain Design]
        A1[ระบุ domain model]
        A2[กำหนด invariants]
        A3[ระบุ use cases]
        A4[กำหนด domain events]
        A5[ออกแบบ repository interface]
        A6[ออกแบบ DTOs]
    end

    subgraph Phase2[Phase 2: Implementation]
        B1[สร้าง entity struct]
        B2[สร้าง value objects]
        B3[สร้าง repository interface]
        B4[สร้าง service interface]
    end

```

```

    B5[สร้าง DTOs]
    B6[เขียน unit tests - entity]
    B7[เขียน unit tests - service]
end

subgraph Phase3[Phase 3: Infrastructure]
    C1[สร้าง repository implementation]
    C2[สร้าง migration file]
    C3[ตั้งค่า Redis cache]
    C4[ตั้งค่า message queue]
    C5[ทดสอบ repository]
end

subgraph Phase4[Phase 4: Delivery]
    D1[สร้าง HTTP handlers]
    D2[เพิ่ม input validation]
    D3[สร้าง routes]
    D4[ลงทะเบียน dependencies]
    D5[ทดสอบ handler]
end

subgraph Phase5[Phase 5: Integration]
    E1[ทดสอบ end-to-end]
    E2[อัปเดต documentation]
    E3[รัน linter]
end

subgraph Phase6[Phase 6: Review & Deploy]
    F1[Code review]
    F2[ตรวจสอบ performance]
    F3[รัน test coverage]
    F4[รัน race detector]
    F5[Deploy to staging]
    F6[Deploy to production]
end

Phase1 --> Phase2 --> Phase3 --> Phase4 --> Phase5 --> Phase6

```

แผนภาพสถาปัตยกรรม Clean Architecture

```

flowchart TB
    subgraph Interface[Interface Layer]
        HTTP[HTTP Handlers]
        CLI[CLI Commands]
        GRPC[gRPC Services]
    end

    subgraph Application[Application Layer]
        UC1[Use Case 1]
        UC2[Use Case 2]
    end

```

```

    UC3[Use Case 3]
end

subgraph Domain[Domain Layer]
    ENT[Entities]
    VO[Value Objects]
    REPO[Repository Interfaces]
    EVENTS[Domain Events]
end

subgraph Infrastructure[Infrastructure Layer]
    DB[Database]
    CACHE[Redis Cache]
    MQ[Message Queue]
    EMAIL[Email Service]
end

Interface --> Application
Application --> Domain
Infrastructure -.-> REPO
Infrastructure -.-> EVENTS

```

แผนภาพ DDD Bounded Context

```

flowchart LR
    subgraph Context1[User Context]
        U1[User Entity]
        U2[Email VO]
        U3[Password VO]
        U4[User Repository]
    end

    subgraph Context2[Order Context]
        O1[Order Aggregate]
        O2[OrderItem Entity]
        O3[Address VO]
        O4[Order Repository]
    end

    subgraph Context3[Payment Context]
        P1[Payment Entity]
        P2[Transaction VO]
        P3[Payment Repository]
    end

    Context1 <--> Context2
    Context2 <--> Context3

```

แผนภาพ CQRS และ Event Sourcing

```

flowchart TD
    subgraph Command[Command Side]
        CMD[Command Handler]
        AGG[Aggregate Root]
        EVENT[Domain Events]
        STORE[Event Store]
    end

    subgraph Query[Query Side]
        QRY[Query Handler]
        READ[Read Database]
        VIEW[Projection View]
    end

    subgraph Sync[Eventual Consistency]
        PROJ[Projection]
        BUS[Event Bus]
    end

    CMD --> AGG
    AGG --> EVENT
    EVENT --> STORE
    EVENT --> BUS
    BUS --> PROJ
    PROJ --> READ
    QRY --> READ
    QRY --> VIEW

```

Task List Template

Template สำหรับการพัฒนา Feature ใหม่

```

# Feature: [ชื่อ Feature]
## Owner: [ชื่อผู้รับผิดชอบ]
## Due Date: [วันที่กำหนดส่ง]

---

## Phase 1: Domain Design

### Tasks
- [ ] **T1.1** ระบุ domain model (entity, value objects)
  - [ ] ระบุ entity: _____
  - [ ] ระบุ value objects: _____
- [ ] **T1.2** กำหนด invariants (business rules)
  - [ ] Rule 1: _____
  - [ ] Rule 2: _____
- [ ] **T1.3** ระบุ use cases

```

- [] Use case 1: _____
- [] Use case 2: _____
- [] ****T1.4**** กำหนด domain events (ถ้ามี)
 - [] Event 1: _____
 - [] Event 2: _____
- [] ****T1.5**** ออกแบบ repository interface
 - [] Method: _____
 - [] Method: _____
- [] ****T1.6**** ออกแบบ DTOs (request/response)
 - [] Request: _____
 - [] Response: _____

****หมายเหตุ:**** _____

Phase 2: Implementation

Tasks

- [] ****T2.1**** สร้าง entity struct และ behavior methods
 - [] File: `internal/domain/[module]/entity.go`
 - [] Constructor: `New[Entity]()`
 - [] Methods: _____
- [] ****T2.2**** สร้าง value objects
 - [] File: `internal/domain/[module]/value\_objects.go`
 - [] VO 1: _____
- [] ****T2.3**** สร้าง repository interface
 - [] File: `internal/domain/[module]/repository.go`
- [] ****T2.4**** สร้าง service interface
 - [] File: `internal/domain/[module]/service.go`
- [] ****T2.5**** สร้าง DTO structs
 - [] File: `internal/application/[module]/dto.go`
- [] ****T2.6**** เขียน unit tests สำหรับ entity
 - [] File: `internal/domain/[module]/entity\_test.go`
 - [] Test cases: _____
- [] ****T2.7**** เขียน unit tests สำหรับ service (mock repository)
 - [] File: `internal/application/[module]/[usecase]\_test.go`
 - [] Mock repository implementation

****หมายเหตุ:**** _____

Phase 3: Infrastructure

Tasks

- [] ****T3.1**** สร้าง repository implementation
 - [] File: `internal/infrastructure/persistence/gorm/[module]\_repo.go`
 - [] Implement interface methods
- [] ****T3.2**** สร้าง migration file
 - [] File: `migrations/[timestamp]\_create\_[table]\_table.sql`
 - [] Up migration: _____
 - [] Down migration: _____

- [] ****T3.3**** ตั้งค่า Redis cache (ถ้าจำเป็น)
 - [] Cache key pattern: _____
 - [] TTL: _____
- [] ****T3.4**** ตั้งค่า message queue (ถ้าจำเป็น)
 - [] Topic/Queue name: _____
 - [] Consumer implementation
- [] ****T3.5**** ทดสอบ repository ด้วย integration test
 - [] File: `internal/infrastructure/persistence/gorm/[module]\_repo\_test.go`
 - [] Use testcontainers or in-memory DB

****หมายเหตุ:**** _____

Phase 4: Delivery

Tasks

- [] ****T4.1**** สร้าง HTTP handlers
 - [] File: `internal/interfaces/http/handlers/[module]\_handler.go`
 - [] Handler methods: _____
- [] ****T4.2**** เพิ่ม input validation
 - [] Validation tags: _____
 - [] Custom validator (ถ้ามี): _____
- [] ****T4.3**** สร้าง routes
 - [] File: `internal/interfaces/http/routes.go`
 - [] Routes: _____
- [] ****T4.4**** ลงทะเบียน dependencies ใน injection
 - [] File: `internal/apps/app/bootstrap/injection/wire.go`
 - [] Update provider set
- [] ****T4.5**** ทดสอบ handler ด้วย httptest
 - [] File: `internal/interfaces/http/handlers/[module]\_handler\_test.go`
 - [] Test cases: _____

****หมายเหตุ:**** _____

Phase 5: Integration & Documentation

Tasks

- [] ****T5.1**** ทดสอบ end-to-end
 - [] curl/Postman collection: _____
 - [] Test scenarios: _____
- [] ****T5.2**** อัปเดต Swagger docs
 - [] File: `api/swagger.yaml` or `docs/docs.go`
 - [] Annotations: _____
- [] ****T5.3**** อัปเดต README
 - [] Add feature description
 - [] Update API examples
- [] ****T5.4**** รัน linter และแก้ไข warnings
 - [] Command: `golangci-lint run ./...`
 - [] Issues fixed: _____

****หมายเหตุ:**** _____

Phase 6: Review & Deploy

Tasks

- [] ****T6.1**** Code review
 - [] PR created: _____
 - [] Reviewers: _____
 - [] Comments addressed
- [] ****T6.2**** ตรวจสอบ performance
 - [] Benchmark: _____
 - [] Query optimization: _____
- [] ****T6.3**** รัน test coverage
 - [] Command: `go test -cover ./...`
 - [] Coverage: _____% (target >80%)
- [] ****T6.4**** รัน race detector
 - [] Command: `go test -race ./...`
 - [] Issues found: _____
- [] ****T6.5**** Deploy to staging
 - [] Date: _____
 - [] Version: _____
- [] ****T6.6**** ทดสอบใน staging
 - [] Smoke test passed
 - [] Regression test passed
- [] ****T6.7**** Deploy to production
 - [] Date: _____
 - [] Version: _____
 - [] Monitoring checked

****หมายเหตุ:**** _____

Summary

- ****Total Tasks:**** ____ / ____ completed
- ****Blockers:**** _____
- ****Next Steps:**** _____

☒ Checklist Template

Code Quality Checklist

Code Quality Checklist

Documentation

- [] All exported functions have comments (godoc format)

- [] Package has package-level documentation comment
- [] Complex logic has inline comments explaining "why"
- [] README updated with relevant information

Code Style

- [] Code formatted with `go fmt` or `gofmt`
- [] No unused imports or variables (`go vet` passed)
- [] Consistent naming convention (camelCase, PascalCase)
- [] No magic numbers (use constants)
- [] Line length < 120 characters (preferably)

Error Handling

- [] All errors are handled explicitly (no `\_` ignoring)
- [] Errors are wrapped with context (`fmt.Errorf("...: %w", err)`)
- [] No panic in library code (only in main/init for fatal errors)
- [] Custom error types used when appropriate
- [] Error messages are descriptive and actionable

Concurrency

- [] Goroutines have proper lifecycle management
- [] Channels are closed appropriately
- [] No race conditions (`go test -race` passed)
- [] `sync.Mutex` used correctly (Lock/Unlock pairs)
- [] Context passed as first parameter for cancellation

Performance

- [] No unnecessary allocations in hot paths
- [] Slice pre-allocated when size known (`make([]T, 0, capacity)`)
- [] String concatenation uses `strings.Builder` for large operations
- [] Database queries have appropriate indexes
- [] No N+1 queries

Security

- [] Input validation on all external inputs
- [] SQL injection prevented (use parameterized queries)
- [] No hardcoded secrets or credentials
- [] Sensitive data not logged
- [] Passwords hashed with bcrypt (not stored in plaintext)
- [] JWT secrets loaded from environment
- [] CORS configured properly (allow only trusted origins)

Testing

- [] Unit tests cover business logic
- [] Table-driven tests used for multiple scenarios
- [] Edge cases tested (nil, empty, boundary values)
- [] Mock external dependencies
- [] Test coverage > 80%

Project Structure

- [] Follows standard Go project layout
- [] Packages have single responsibility
- [] No circular dependencies
- [] Internal packages used for private code

```

- [ ] Go modules properly configured

### Dependencies
- [ ] go.mod has only required dependencies
- [ ] go.sum is committed
- [ ] `go mod tidy` run before commit
- [ ] No unused dependencies

### Version Control
- [ ] Commit messages follow convention (feat, fix, docs, etc.)
- [ ] No debug code (fmt.Println, log.Println) in production code
- [ ] No commented out code
- [ ] .gitignore properly configured

### Reviewer Notes
- [ ] Code reviewed by at least one other developer
- [ ] All review comments addressed

---
**Status:** [ ] Ready for merge | [ ] Changes requested | [ ] Approved
**Reviewer:** _____
**Date:** _____

```

Deployment Checklist

```

## Deployment Checklist

### Pre-Deployment (Staging)

#### Code Readiness
- [ ] All tests passing (`go test ./...`)
- [ ] Race detector passed (`go test -race ./...`)
- [ ] Linter passed (`golangci-lint run ./...`)
- [ ] Build successful (`go build ./...`)
- [ ] All PRs merged and approved

#### Configuration
- [ ] Environment variables verified
- [ ] Configuration files updated for staging
- [ ] Feature flags configured
- [ ] Third-party service credentials verified

#### Database
- [ ] Migration scripts reviewed
- [ ] Migrations tested in staging environment
- [ ] Rollback plan documented
- [ ] Backup created before migration

#### Infrastructure
- [ ] Container images built and tagged

```

- [] Kubernetes/ deployment files updated
- [] Resource limits configured
- [] Health check endpoints configured
- [] Monitoring and alerting configured

Security

- [] Security scan passed
- [] No secrets in code or config
- [] TLS certificates valid

Staging Deployment

Deployment Steps

- [] Deploy to staging environment
- [] Verify pod/container health
- [] Run smoke tests
- [] Run integration tests
- [] Verify logs for errors
- [] Load testing (if required)

Validation

- [] Feature works as expected
- [] No regression in existing features
- [] Performance meets baseline
- [] Error handling works
- [] Monitoring shows expected metrics

Pre-Production (Final Check)

Business Approval

- [] Product owner sign-off
- [] QA sign-off
- [] Security sign-off
- [] Documentation updated

Rollback Plan

- [] Rollback procedure documented
- [] Database rollback plan ready
- [] Previous version image available
- [] Rollback tested

Communication

- [] Release notes prepared
- [] Stakeholders notified
- [] Support team informed

Production Deployment

```

#### Deployment Steps
- [ ] Schedule maintenance window (if required)
- [ ] Create production backup
- [ ] Deploy with canary/blue-green strategy
- [ ] Monitor deployment progress
- [ ] Verify health checks
- [ ] Run post-deployment tests

#### Post-Deployment
- [ ] Monitor logs for errors (15 min)
- [ ] Verify key metrics
- [ ] Check user feedback channels
- [ ] Update status page (if applicable)
- [ ] Announce successful deployment

#### Rollback Trigger Conditions
- [ ] Error rate > 1%
- [ ] Critical feature broken
- [ ] Security incident detected
- [ ] Performance degradation > 50%

---

### Post-Deployment

#### Cleanup
- [ ] Remove old images (if applicable)
- [ ] Clean up temporary resources
- [ ] Update documentation with new version

#### Monitoring
- [ ] Monitor for 24 hours
- [ ] Review error logs daily for 1 week
- [ ] Check resource utilization

#### Retrospective
- [ ] Deployment time recorded
- [ ] Issues encountered documented
- [ ] Improvements identified for next deployment

---

**Deployment Status:** [ ] Success | [ ] Failed | [ ] Rolled back
**Deployed by:** _____
**Date:** _____
**Version:** _____

```

ตัวอย่างโค้ด

ตัวอย่าง 1: Clean Architecture - User Registration (Full Example)

โครงสร้างโปรเจกต์

```
user-service/  
├── cmd/  
│   └── api/  
│       └── main.go  
├── internal/  
│   ├── domain/  
│   │   └── user/  
│   │       ├── entity.go  
│   │       ├── value_objects.go  
│   │       └── repository.go  
│   ├── application/  
│   │   └── user/  
│   │       ├── register.go  
│   │       └── dto.go  
│   ├── infrastructure/  
│   │   └── persistence/  
│   │       └── gorm/  
│   │           └── user_repo.go  
│   └── interfaces/  
│       └── http/  
│           ├── handlers/  
│           │   └── user_handler.go  
│           └── routes.go  
├── go.mod  
└── go.sum
```

go.mod

```
module user-service  
  
go 1.21  
  
require (  
    github.com/go-chi/chi/v5 v5.0.10  
    github.com/go-playground/validator/v10 v10.15.5  
    github.com/google/uuid v1.3.1  
    github.com/lib/pq v1.10.9  
    golang.org/x/crypto v0.14.0  
    gorm.io/driver/postgres v1.5.3  
    gorm.io/gorm v1.25.5  
)
```

Domain Layer - Entity

internal/domain/user/entity.go

```

package user

import (
    "errors"
    "time"

    "github.com/google/uuid"
)

// User represents the core domain entity
type User struct {
    id          uuid.UUID
    email       Email
    password    Password
    name        string
    isVerified  bool
    createdAt   time.Time
    updatedAt   time.Time
}

// NewUser creates a new User entity
func NewUser(email, password, name string) (*User, error) {
    emailVO, err := NewEmail(email)
    if err != nil {
        return nil, err
    }

    passwordVO, err := NewPassword(password)
    if err != nil {
        return nil, err
    }

    if name == "" {
        return nil, errors.New("name is required")
    }

    now := time.Now()
    return &User{
        id:          uuid.New(),
        email:       *emailVO,
        password:    *passwordVO,
        name:        name,
        isVerified:  false,
        createdAt:   now,
        updatedAt:   now,
    }, nil
}

// Reconstruct reconstructs a User from persistence (for repository)
func Reconstruct(id uuid.UUID, email Email, passwordHash string, name string,
    isVerified bool, createdAt, updatedAt time.Time) (*User, error) {

```

```

password, err := NewPasswordFromHash(passwordHash)
if err != nil {
    return nil, err
}

return &User{
    id:          id,
    email:       email,
    password:    *password,
    name:        name,
    isVerified:  isVerified,
    createdAt:   createdAt,
    updatedAt:   updatedAt,
}, nil
}

// Getters
func (u *User) ID() uuid.UUID      { return u.id }
func (u *User) Email() Email      { return u.email }
func (u *User) Name() string      { return u.name }
func (u *User) IsVerified() bool  { return u.isVerified }
func (u *User) CreatedAt() time.Time { return u.createdAt }
func (u *User) UpdatedAt() time.Time { return u.updatedAt }
func (u *User) PasswordHash() string { return u.password.Hash() }

// Behaviors
func (u *User) Verify() {
    u.isVerified = true
    u.updatedAt = time.Now()
}

func (u *User) ChangePassword(oldPlain, newPlain string) error {
    if err := u.password.Compare(oldPlain); err != nil {
        return ErrInvalidPassword
    }

    newPassword, err := NewPassword(newPlain)
    if err != nil {
        return err
    }

    u.password = *newPassword
    u.updatedAt = time.Now()
    return nil
}

```

internal/domain/user/value_objects.go

```

package user

import (

```



```

    "errors"
    "regexp"

    "golang.org/x/crypto/bcrypt"
)

// Email is a value object
type Email struct {
    value string
}

// NewEmail creates a new Email value object
func NewEmail(email string) (*Email, error) {
    emailRegex := regexp.MustCompile(`^[a-z0-9._%+\-]+\@[a-z0-9.\-]+\.[a-z]{2,}$`)
    if !emailRegex.MatchString(email) {
        return nil, errors.New("invalid email format")
    }
    return &Email{value: email}, nil
}

func (e Email) String() string { return e.value }

// Password is a value object containing hashed password
type Password struct {
    hash string
}

// NewPassword creates a new Password from plain text
func NewPassword(plain string) (*Password, error) {
    if len(plain) < 8 {
        return nil, errors.New("password must be at least 8 characters")
    }

    hash, err := bcrypt.GenerateFromPassword([]byte(plain), bcrypt.DefaultCost)
    if err != nil {
        return nil, err
    }

    return &Password{hash: string(hash)}, nil
}

// NewPasswordFromHash creates a Password from existing hash
func NewPasswordFromHash(hash string) (*Password, error) {
    return &Password{hash: hash}, nil
}

// Compare checks if plain password matches the hash
func (p *Password) Compare(plain string) error {
    return bcrypt.CompareHashAndPassword([]byte(p.hash), []byte(plain))
}

func (p *Password) Hash() string { return p.hash }

```

internal/domain/user/repository.go

```
package user

import (
    "context"

    "github.com/google/uuid"
)

// Repository defines the interface for user data access
type Repository interface {
    Create(ctx context.Context, user *User) error
    FindByID(ctx context.Context, id uuid.UUID) (*User, error)
    FindByEmail(ctx context.Context, email Email) (*User, error)
    Update(ctx context.Context, user *User) error
    Delete(ctx context.Context, id uuid.UUID) error
}

// Domain errors
var (
    ErrUserNotFound      = errors.New("user not found")
    ErrEmailAlreadyUsed = errors.New("email already used")
    ErrInvalidPassword  = errors.New("invalid password")
)
```

Application Layer

internal/application/user/dto.go

```
package user

// RegisterInput represents the data needed to register a user
type RegisterInput struct {
    Email    string `json:"email" validate:"required,email"`
    Password string `json:"password" validate:"required,min=8"`
    Name     string `json:"name" validate:"required"`
}

// RegisterOutput represents the response after registration
type RegisterOutput struct {
    ID      string `json:"id"`
    Email   string `json:"email"`
    Name    string `json:"name"`
}

// LoginInput represents the data needed to login
```

```

type LoginInput struct {
    Email    string `json:"email" validate:"required,email"`
    Password string `json:"password" validate:"required"`
}

// LoginOutput represents the response after login
type LoginOutput struct {
    AccessToken string `json:"access_token"`
    RefreshToken string `json:"refresh_token"`
    ExpiresIn    int64  `json:"expires_in"`
}

```

internal/application/user/register.go

```

package user

import (
    "context"
    "errors"

    "user-service/internal/domain/user"
)

var ErrEmailAlreadyExists = errors.New("email already exists")

// RegisterUseCase handles user registration
type RegisterUseCase struct {
    userRepo user.Repository
}

// NewRegisterUseCase creates a new RegisterUseCase
func NewRegisterUseCase(repo user.Repository) *RegisterUseCase {
    return &RegisterUseCase{userRepo: repo}
}

// Execute performs user registration
func (uc *RegisterUseCase) Execute(ctx context.Context, input RegisterInput) (*RegisterOutput, error) {
    // Check if email already exists
    emailVO, err := user.NewEmail(input.Email)
    if err != nil {
        return nil, err
    }

    existing, err := uc.userRepo.FindByEmail(ctx, *emailVO)
    if err != nil && err != user.ErrUserNotFound {
        return nil, err
    }
    if existing != nil {
        return nil, ErrEmailAlreadyExists
    }
}

```

```

// Create new user
newUser, err := user.NewUser(input.Email, input.Password, input.Name)
if err != nil {
    return nil, err
}

// Save to repository
if err := uc.userRepo.Create(ctx, newUser); err != nil {
    return nil, err
}

return &RegisterOutput{
    ID:    newUser.ID().String(),
    Email: newUser.Email().String(),
    Name:  newUser.Name(),
}, nil
}

```

Infrastructure Layer

internal/infrastructure/persistence/gorm/user_repo.go

```

package gorm

import (
    "context"
    "errors"
    "time"

    "github.com/google/uuid"
    "gorm.io/gorm"

    "user-service/internal/domain/user"
)

// UserModel represents the database model
type UserModel struct {
    ID          string    `gorm:"primaryKey;type:uuid"`
    Email       string    `gorm:"uniqueIndex;size:100;not null"`
    Password    string    `gorm:"not null"`
    Name        string    `gorm:"size:100;not null"`
    IsVerified  bool      `gorm:"default:false"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
    DeletedAt   gorm.DeletedAt `gorm:"index"`
}

func (UserModel) TableName() string {
    return "users"
}

```

```

}

// UserRepository implements user.Repository using GORM
type UserRepository struct {
    db *gorm.DB
}

// NewUserRepository creates a new UserRepository
func NewUserRepository(db *gorm.DB) *UserRepository {
    return &UserRepository{db: db}
}

// Create inserts a new user
func (r *UserRepository) Create(ctx context.Context, u *user.User) error {
    model := &UserModel{
        ID:          u.ID().String(),
        Email:       u.Email().String(),
        Password:    u.PasswordHash(),
        Name:        u.Name(),
        IsVerified:  u.IsVerified(),
        CreatedAt:   u.CreatedAt(),
        UpdatedAt:   u.UpdatedAt(),
    }
    return r.db.WithContext(ctx).Create(model).Error
}

// FindByID finds a user by ID
func (r *UserRepository) FindByID(ctx context.Context, id uuid.UUID)
(*user.User, error) {
    var model UserModel
    err := r.db.WithContext(ctx).Where("id = ?",
id.String()).First(&model).Error
    if err != nil {
        if errors.Is(err, gorm.ErrRecordNotFound) {
            return nil, user.ErrUserNotFound
        }
        return nil, err
    }
    return r.toDomain(&model)
}

// FindByEmail finds a user by email
func (r *UserRepository) FindByEmail(ctx context.Context, email user.Email)
(*user.User, error) {
    var model UserModel
    err := r.db.WithContext(ctx).Where("email = ?",
email.String()).First(&model).Error
    if err != nil {
        if errors.Is(err, gorm.ErrRecordNotFound) {
            return nil, user.ErrUserNotFound
        }
        return nil, err
    }
}

```

```

        return r.toDomain(&model)
    }

    // Update updates an existing user
    func (r *UserRepository) Update(ctx context.Context, u *user.User) error {
        model := &UserModel{
            ID:          u.ID().String(),
            Email:        u.Email().String(),
            Password:     u.PasswordHash(),
            Name:         u.Name(),
            IsVerified:   u.IsVerified(),
            UpdatedAt:    u.UpdatedAt(),
        }
        return r.db.WithContext(ctx).Save(model).Error
    }

    // Delete soft deletes a user
    func (r *UserRepository) Delete(ctx context.Context, id uuid.UUID) error {
        return r.db.WithContext(ctx).Where("id = ?",
            id.String()).Delete(&UserModel{}).Error
    }

    func (r *UserRepository) toDomain(m *UserModel) (*user.User, error) {
        id, err := uuid.Parse(m.ID)
        if err != nil {
            return nil, err
        }

        email, err := user.NewEmail(m.Email)
        if err != nil {
            return nil, err
        }

        return user.Reconstruct(
            id,
            *email,
            m.Password,
            m.Name,
            m.IsVerified,
            m.CreatedAt,
            m.UpdatedAt,
        )
    }
}

```

Interface Layer

internal/interfaces/http/handlers/user_handler.go

```
package handlers
```

```

import (
    "encoding/json"
    "net/http"

    "github.com/go-playground/validator/v10"

    "user-service/internal/application/user"
)

// UserHandler handles HTTP requests for user operations
type UserHandler struct {
    registerUC *user.RegisterUseCase
    validate   *validator.Validate
}

// NewUserHandler creates a new UserHandler
func NewUserHandler(registerUC *user.RegisterUseCase) *UserHandler {
    return &UserHandler{
        registerUC: registerUC,
        validate:   validator.New(),
    }
}

// Register handles POST /api/register
func (h *UserHandler) Register(w http.ResponseWriter, r *http.Request) {
    var req user.RegisterInput
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid request body", http.StatusBadRequest)
        return
    }

    if err := h.validate.Struct(req); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    output, err := h.registerUC.Execute(r.Context(), req)
    if err != nil {
        switch err {
        case user.ErrEmailAlreadyExists:
            http.Error(w, "Email already registered", http.StatusConflict)
        default:
            http.Error(w, "Internal server error",
http.StatusInternalServerError)
        }
        return
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(output)
}

```

```
// HealthCheck handles GET /health
func (h *UserHandler) HealthCheck(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
}
```

internal/interfaces/http/routes.go

```
package http

import (
    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"

    "user-service/internal/interfaces/http/handlers"
)

// SetupRoutes configures all HTTP routes
func SetupRoutes(userHandler *handlers.UserHandler) *chi.Mux {
    r := chi.NewRouter()

    // Middleware
    r.Use(middleware.Logger)
    r.Use(middleware.Recoverer)
    r.Use(middleware.RequestID)
    r.Use(middleware.RealIP)

    // Health check
    r.Get("/health", userHandler.HealthCheck)

    // API routes
    r.Route("/api", func(r chi.Router) {
        r.Post("/register", userHandler.Register)
    })

    return r
}
```

Main Entry Point

cmd/api/main.go

```
package main

import (
    "log"
    "net/http"
)
```



```

"os"
"os/signal"
"syscall"
"time"

"gorm.io/driver/postgres"
"gorm.io/gorm"
"gorm.io/gorm/logger"

"user-service/internal/application/user"
"user-service/internal/domain/user"
gormRepo "user-service/internal/infrastructure/persistence/gorm"
"user-service/internal/interfaces/http/handlers"
"user-service/internal/interfaces/http"
)

func main() {
    // Database connection
    dsn := "host=localhost user=postgres password=postgres dbname=userdb
port=5432 sslmode=disable TimeZone=Asia/Bangkok"
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{
        Logger: logger.Default.LogMode(logger.Info),
    })
    if err != nil {
        log.Fatal("Failed to connect to database:", err)
    }

    // Auto migrate (development only)
    if err := db.AutoMigrate(&gormRepo.UserModel{}); err != nil {
        log.Fatal("Failed to migrate database:", err)
    }

    // Dependency injection
    userRepo := gormRepo.NewUserRepository(db)
    registerUC := user.NewRegisterUseCase(userRepo)
    userHandler := handlers.NewUserHandler(registerUC)

    // Setup routes
    router := http.SetupRoutes(userHandler)

    // Create server
    srv := &http.Server{
        Addr:         ":8080",
        Handler:      router,
        ReadTimeout:  15 * time.Second,
        WriteTimeout: 15 * time.Second,
        IdleTimeout:  60 * time.Second,
    }

    // Graceful shutdown
    go func() {
        log.Println("Starting server on :8080")
        if err := srv.ListenAndServe(); err != nil && err !=

```

```

http.ErrServerClosed {
    log.Fatalf("Server failed: %v", err)
}
]()

quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
log.Println("Shutting down server...")

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
    log.Fatalf("Server forced to shutdown: %v", err)
}

log.Println("Server exited")
}

```

ตัวอย่าง 2: Worker Pool Pattern

```

package main

import (
    "context"
    "fmt"
    "sync"
    "time"
)

// Job represents a unit of work
type Job struct {
    ID      int
    Payload string
}

// Result represents the outcome of processing a job
type Result struct {
    JobID  int
    Output string
    Error  error
}

// WorkerPool manages a pool of workers for concurrent job processing
type WorkerPool struct {
    numWorkers int
    jobQueue   chan Job
    resultQueue chan Result
    wg         sync.WaitGroup
}

```

```

    ctx      context.Context
    cancel    context.CancelFunc
}

// NewWorkerPool creates a new worker pool
func NewWorkerPool(numWorkers int, queueSize int) *WorkerPool {
    ctx, cancel := context.WithCancel(context.Background())
    return &WorkerPool{
        numWorkers:  numWorkers,
        jobQueue:     make(chan Job, queueSize),
        resultQueue:  make(chan Result, queueSize),
        ctx:          ctx,
        cancel:       cancel,
    }
}

// Start launches the worker pool
func (wp *WorkerPool) Start() {
    for i := 0; i < wp.numWorkers; i++ {
        wp.wg.Add(1)
        go wp.worker(i)
    }
}

// worker processes jobs from the queue
func (wp *WorkerPool) worker(id int) {
    defer wp.wg.Done()
    for {
        select {
        case <-wp.ctx.Done():
            fmt.Printf("Worker %d stopping\n", id)
            return
        case job, ok := <-wp.jobQueue:
            if !ok {
                return
            }
            result := wp.processJob(job)
            select {
            case wp.resultQueue <- result:
            case <-wp.ctx.Done():
                return
            }
        }
    }
}

// processJob handles a single job
func (wp *WorkerPool) processJob(job Job) Result {
    // Simulate processing time
    time.Sleep(100 * time.Millisecond)

    // Process the job
    output := fmt.Sprintf("Processed job %d with payload: %s", job.ID,

```

```

job.Payload)

    return Result{
        JobID:  job.ID,
        Output: output,
        Error:  nil,
    }
}

// Submit adds a job to the queue
func (wp *WorkerPool) Submit(job Job) bool {
    select {
    case wp.jobQueue <- job:
        return true
    case <-wp.ctx.Done():
        return false
    default:
        return false
    }
}

// Results returns a channel for consuming results
func (wp *WorkerPool) Results() <-chan Result {
    return wp.resultQueue
}

// Stop gracefully shuts down the worker pool
func (wp *WorkerPool) Stop() {
    wp.cancel()
    close(wp.jobQueue)
    wp.wg.Wait()
    close(wp.resultQueue)
}

func main() {
    // Create worker pool with 5 workers
    pool := NewWorkerPool(5, 100)
    pool.Start()

    // Create context with timeout
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    // Submit 50 jobs
    go func() {
        for i := 0; i < 50; i++ {
            job := Job{
                ID:      i,
                Payload: fmt.Sprintf("data-%d", i),
            }
            if !pool.Submit(job) {
                fmt.Printf("Failed to submit job %d\n", i)
            }
        }
    }()
}

```

```

    }
}()

// Collect results
results := pool.Results()
processed := 0

for {
    select {
    case <-ctx.Done():
        fmt.Println("Timeout reached")
        pool.Stop()
        fmt.Printf("Processed: %d jobs\n", processed)
        return
    case result, ok := <-results:
        if !ok {
            fmt.Println("All jobs processed")
            fmt.Printf("Processed: %d jobs\n", processed)
            return
        }
        processed++
        fmt.Printf("Result: %s\n", result.Output)
    }
}
}

```

ตัวอย่าง 3: Redis Cache Integration

```

package cache

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "github.com/redis/go-redis/v9"
)

// RedisCache provides caching functionality using Redis
type RedisCache struct {
    client *redis.Client
    ttl    time.Duration
}

// NewRedisCache creates a new Redis cache client
func NewRedisCache(addr, password string, db int, ttl time.Duration)
*RedisCache {
    client := redis.NewClient(&redis.Options{
        Addr:      addr,

```

```

        Password: password,
        DB:      db,
    })

    return &RedisCache{
        client: client,
        ttl:    ttl,
    }
}

// Set stores a value in cache
func (c *RedisCache) Set(ctx context.Context, key string, value interface{}) error {
    data, err := json.Marshal(value)
    if err != nil {
        return fmt.Errorf("failed to marshal value: %w", err)
    }

    return c.client.Set(ctx, key, data, c.ttl).Err()
}

// Get retrieves a value from cache
func (c *RedisCache) Get(ctx context.Context, key string, dest interface{}) error {
    data, err := c.client.Get(ctx, key).Bytes()
    if err != nil {
        if err == redis.Nil {
            return ErrCacheMiss
        }
        return fmt.Errorf("failed to get from cache: %w", err)
    }

    if err := json.Unmarshal(data, dest); err != nil {
        return fmt.Errorf("failed to unmarshal cached data: %w", err)
    }

    return nil
}

// Delete removes a value from cache
func (c *RedisCache) Delete(ctx context.Context, key string) error {
    return c.client.Del(ctx, key).Err()
}

// Exists checks if a key exists in cache
func (c *RedisCache) Exists(ctx context.Context, key string) (bool, error) {
    result, err := c.client.Exists(ctx, key).Result()
    if err != nil {
        return false, err
    }
    return result > 0, nil
}

```

```
// ErrCacheMiss indicates that the requested key was not found
var ErrCacheMiss = fmt.Errorf("cache miss")

// Usage example
func Example() {
    ctx := context.Background()
    cache := NewRedisCache("localhost:6379", "", 0, 10*time.Minute)

    // Store user data
    user := map[string]interface{}{
        "id": 123,
        "name": "John Doe",
        "email": "john@example.com",
    }

    if err := cache.Set(ctx, "user:123", user); err != nil {
        fmt.Printf("Cache set error: %v\n", err)
    }

    // Retrieve user data
    var cachedUser map[string]interface{}
    if err := cache.Get(ctx, "user:123", &cachedUser); err != nil {
        if err == ErrCacheMiss {
            fmt.Println("User not found in cache")
        } else {
            fmt.Printf("Cache get error: %v\n", err)
        }
    } else {
        fmt.Printf("Cached user: %v\n", cachedUser)
    }
}
```

ตัวอย่าง 4: Generic Repository Pattern (Go 1.18+)

```
package repository

import (
    "context"

    "gorm.io/gorm"
)

// Entity defines the interface that all entities must implement
type Entity interface {
    GetID() string
}

// Repository is a generic repository interface
type Repository[T Entity] interface {
    Create(ctx context.Context, entity T) error
}
```

```

    GetByID(ctx context.Context, id string) (T, error)
    Update(ctx context.Context, entity T) error
    Delete(ctx context.Context, id string) error
    Find(ctx context.Context, query Query) ([]T, error)
}

// Query represents search criteria
type Query struct {
    Filters map[string]interface{}
    Limit   int
    Offset  int
    OrderBy string
}

// GormRepository is a generic GORM implementation
type GormRepository[T Entity] struct {
    db *gorm.DB
}

// NewGormRepository creates a new generic GORM repository
func NewGormRepository[T Entity](db *gorm.DB) *GormRepository[T] {
    return &GormRepository[T]{db: db}
}

// Create inserts a new entity
func (r *GormRepository[T]) Create(ctx context.Context, entity T) error {
    return r.db.WithContext(ctx).Create(entity).Error
}

// GetByID retrieves an entity by ID
func (r *GormRepository[T]) GetByID(ctx context.Context, id string) (T, error) {
    var entity T
    err := r.db.WithContext(ctx).Where("id = ?", id).First(&entity).Error
    return entity, err
}

// Update updates an existing entity
func (r *GormRepository[T]) Update(ctx context.Context, entity T) error {
    return r.db.WithContext(ctx).Save(entity).Error
}

// Delete removes an entity by ID
func (r *GormRepository[T]) Delete(ctx context.Context, id string) error {
    var entity T
    return r.db.WithContext(ctx).Where("id = ?", id).Delete(&entity).Error
}

// Find finds entities matching the query
func (r *GormRepository[T]) Find(ctx context.Context, query Query) ([]T, error) {
    var entities []T
    db := r.db.WithContext(ctx)

```



```

    for field, value := range query.Filters {
        db = db.Where(field+" = ?", value)
    }

    if query.Limit > 0 {
        db = db.Limit(query.Limit)
    }
    if query.Offset > 0 {
        db = db.Offset(query.Offset)
    }
    if query.OrderBy != "" {
        db = db.Order(query.OrderBy)
    }

    err := db.Find(&entities).Error
    return entities, err
}

// Example entity
type Product struct {
    ID    string `gorm:"primaryKey"`
    Name  string `gorm:"size:100;not null"`
    Price float64 `gorm:"not null"`
}

func (p Product) GetID() string { return p.ID }

// Usage example
func ExampleUsage() {
    db, _ := gorm.Open(postgres.Open(dsn), &gorm.Config{})
    productRepo := NewGormRepository[Product](db)

    ctx := context.Background()

    // Create
    product := Product{ID: "1", Name: "Laptop", Price: 999.99}
    productRepo.Create(ctx, product)

    // Find
    products, _ := productRepo.Find(ctx, Query{
        Filters: map[string]interface{}{"name": "Laptop"},
        Limit:   10,
    })

    // Get by ID
    found, _ := productRepo.GetByID(ctx, "1")
}

```

mop Config – การจัดการ Configuration

ไฟล์ config/config.yaml

```
# config/config.yaml
server:
  port: 8080
  mode: release
  read_timeout: 15s
  write_timeout: 15s

database:
  host: localhost
  port: 5432
  user: postgres
  password: postgres
  name: userdb
  sslmode: disable
  max_open_conns: 25
  max_idle_conns: 25
  conn_max_lifetime: 5m

redis:
  addr: localhost:6379
  password: ""
  db: 0
  pool_size: 10
  ttl: 10m

jwt:
  secret: "your-secret-key-change-in-production"
  access_expiry: 15m
  refresh_expiry: 168h # 7 days

smtp:
  host: smtp.gmail.com
  port: 587
  username: your-email@gmail.com
  password: your-app-password
  from: your-email@gmail.com

log:
  level: info
  format: json
  output: stdout
```

ไฟล์ config/config.go

```
package config

import (
```

```

    "fmt"
    "time"

    "github.com/spf13/viper"
)

// Config holds all application configuration
type Config struct {
    Server      ServerConfig `mapstructure:"server"`
    Database    DatabaseConfig `mapstructure:"database"`
    Redis       RedisConfig `mapstructure:"redis"`
    JWT         JWTConfig `mapstructure:"jwt"`
    SMTP        SMTPConfig `mapstructure:"smtp"`
    Log         LogConfig `mapstructure:"log"`
}

// ServerConfig holds HTTP server configuration
type ServerConfig struct {
    Port          int `mapstructure:"port"`
    Mode          string `mapstructure:"mode"`
    ReadTimeout   time.Duration `mapstructure:"read_timeout"`
    WriteTimeout  time.Duration `mapstructure:"write_timeout"`
}

// DatabaseConfig holds database configuration
type DatabaseConfig struct {
    Host          string `mapstructure:"host"`
    Port          int `mapstructure:"port"`
    User          string `mapstructure:"user"`
    Password      string `mapstructure:"password"`
    Name          string `mapstructure:"name"`
    SSLMode       string `mapstructure:"sslmode"`
    MaxOpenConns  int `mapstructure:"max_open_conns"`
    MaxIdleConns  int `mapstructure:"max_idle_conns"`
    ConnMaxLifetime time.Duration `mapstructure:"conn_max_lifetime"`
}

// RedisConfig holds Redis configuration
type RedisConfig struct {
    Addr          string `mapstructure:"addr"`
    Password      string `mapstructure:"password"`
    DB            int `mapstructure:"db"`
    PoolSize      int `mapstructure:"pool_size"`
    TTL           time.Duration `mapstructure:"ttl"`
}

// JWTConfig holds JWT configuration
type JWTConfig struct {
    Secret         string `mapstructure:"secret"`
    AccessExpiry   time.Duration `mapstructure:"access_expiry"`
    RefreshExpiry  time.Duration `mapstructure:"refresh_expiry"`
}

```

```
// SMTPConfig holds email configuration
type SMTPConfig struct {
    Host      string `mapstructure:"host"`
    Port      int    `mapstructure:"port"`
    Username  string `mapstructure:"username"`
    Password  string `mapstructure:"password"`
    From      string `mapstructure:"from"`
}

// LogConfig holds logging configuration
type LogConfig struct {
    Level  string `mapstructure:"level"`
    Format string `mapstructure:"format"`
    Output string `mapstructure:"output"`
}

// LoadConfig loads configuration from file and environment variables
func LoadConfig(configPath string) (*Config, error) {
    viper.SetConfigFile(configPath)
    viper.SetConfigType("yaml")

    // Set default values
    setDefaults()

    // Read config file
    if err := viper.ReadInConfig(); err != nil {
        return nil, fmt.Errorf("failed to read config file: %w", err)
    }

    // Bind environment variables
    viper.SetEnvPrefix("app")
    viper.AutomaticEnv()

    // Unmarshal into struct
    var cfg Config
    if err := viper.Unmarshal(&cfg); err != nil {
        return nil, fmt.Errorf("failed to unmarshal config: %w", err)
    }

    return &cfg, nil
}

func setDefaults() {
    viper.SetDefault("server.port", 8080)
    viper.SetDefault("server.mode", "debug")
    viper.SetDefault("server.read_timeout", "15s")
    viper.SetDefault("server.write_timeout", "15s")

    viper.SetDefault("database.max_open_conns", 25)
    viper.SetDefault("database.max_idle_conns", 25)
    viper.SetDefault("database.conn_max_lifetime", "5m")

    viper.SetDefault("redis.pool_size", 10)
    viper.SetDefault("redis.ttl", "10m")
}
```

```

    viper.SetDefault("jwt.access_expiry", "15m")
    viper.SetDefault("jwt.refresh_expiry", "168h")

    viper.SetDefault("log.level", "info")
    viper.SetDefault("log.format", "json")
    viper.SetDefault("log.output", "stdout")
}

// GetDSN returns PostgreSQL connection string
func (c *DatabaseConfig) GetDSN() string {
    return fmt.Sprintf(
        "host=%s port=%d user=%s password=%s dbname=%s sslmode=%s
        TimeZone=Asia/Bangkok",
        c.Host, c.Port, c.User, c.Password, c.Name, c.SSLMode,
    )
}

// GetRedisAddr returns Redis address
func (c *RedisConfig) GetAddr() string {
    return c.Addr
}

```

ไฟล์ main.go ที่ใช้ config

```

package main

import (
    "log"
    "user-service/internal/config"
)

func main() {
    // Load configuration
    cfg, err := config.LoadConfig("config/config.yaml")
    if err != nil {
        log.Fatalf("Failed to load config: %v", err)
    }

    // Use configuration
    log.Printf("Server starting on port %d", cfg.Server.Port)
    log.Printf("Database: %s", cfg.Database.GetDSN())
}

```

GORM CRUD กับฐานข้อมูลหลายประเภท: PostgreSQL, MySQL, MongoDB, InfluxDB

แผนภาพ Data Flow (Flowchart TB)

```
flowchart TB
    subgraph App["Go Application"]
        A1["Business Logic / Use Cases"]
        A2["GORM Models / Structs"]
        A3["Repository Layer"]
    end

    subgraph ORM["ORM Layer (GORM)"]
        G1["GORM ORM<br/>- Auto Migration<br/>- CRUD Operations<br/>- Associations<br/>- Hooks"]
    end

    subgraph SQL["SQL Databases"]
        Pg["(PostgreSQL)"]
        My["(MySQL)"]
    end

    subgraph NoSQL["NoSQL / Time-Series"]
        Mg["(MongoDB)"]
        Inf["(InfluxDB)"]
    end

    subgraph Drivers["Database Drivers"]
        D1["pgx / lib/pq"]
        D2["go-sql-driver/mysql"]
        D3["mongo-go-driver"]
        D4["influxdb-client-go"]
    end

    A1 --> A2
    A2 --> A3
    A3 --> G1

    G1 --> D1
    D1 --> Pg

    G1 --> D2
    D2 --> My

    A3 --> D3
    D3 --> Mg

    A3 --> D4
    D4 --> Inf
```

คำอธิบายภาษาไทย (แบบละเอียด)

ใน Go การทำ CRUD กับฐานข้อมูลสามารถใช้ไลบรารีที่เหมาะสมกับแต่ละประเภท:

- **GORM** – ORM ที่ทรงพลัง เหมาะกับฐานข้อมูล SQL (PostgreSQL, MySQL, SQLite, SQL Server)
- **MongoDB** – ใช้ official driver (go.mongodb.org/mongo-driver) สำหรับ document database
- **InfluxDB** – ใช้ official client (github.com/influxdata/influxdb-client-go) สำหรับ time-series database

แผนภาพด้านล่างแสดงการแยกชั้น: **Application** ใช้ **Repository Layer** ซึ่งเรียกใช้ ORM หรือ driver โดยตรง ตามชนิดของฐานข้อมูล

1. PostgreSQL + GORM

การติดตั้ง

```
go get -u gorm.io/gorm
go get -u gorm.io/driver/postgres
```

ตัวอย่าง Model และ CRUD

```
package models

import (
    "time"
    "gorm.io/gorm"
)

type User struct {
    ID          uint           `gorm:"primaryKey" json:"id"`
    Username    string         `gorm:"size:50;uniqueIndex;not null" json:"username"`
    Email       string         `gorm:"size:100;uniqueIndex;not null" json:"email"`
    Password    string         `gorm:"size:255;not null" json:"-"`
    CreatedAt   time.Time      `json:"created_at"`
    UpdatedAt   time.Time      `json:"updated_at"`
    DeletedAt   gorm.DeletedAt `gorm:"index" json:"-"`
}
```

```
package repository

import (
    "context"
    "errors"
    "gorm.io/gorm"
    "myapp/models"
)
```

```

)

type UserRepository struct {
    db *gorm.DB
}

func NewUserRepository(db *gorm.DB) *UserRepository {
    return &UserRepository{db: db}
}

// Create
func (r *UserRepository) Create(ctx context.Context, user *models.User) error {
    return r.db.WithContext(ctx).Create(user).Error
}

// Read by ID
func (r *UserRepository) GetByID(ctx context.Context, id uint) (*models.User, error) {
    var user models.User
    err := r.db.WithContext(ctx).First(&user, id).Error
    if errors.Is(err, gorm.ErrRecordNotFound) {
        return nil, nil
    }
    return &user, err
}

// Read by email
func (r *UserRepository) GetByEmail(ctx context.Context, email string) (*models.User, error) {
    var user models.User
    err := r.db.WithContext(ctx).Where("email = ?", email).First(&user).Error
    if errors.Is(err, gorm.ErrRecordNotFound) {
        return nil, nil
    }
    return &user, err
}

// Update
func (r *UserRepository) Update(ctx context.Context, user *models.User) error {
    return r.db.WithContext(ctx).Save(user).Error
}

// Delete (soft delete)
func (r *UserRepository) Delete(ctx context.Context, id uint) error {
    return r.db.WithContext(ctx).Delete(&models.User{}, id).Error
}

// List with pagination
func (r *UserRepository) List(ctx context.Context, page, limit int) ([]*models.User, int64, error) {
    var users []*models.User
    var total int64

```



```

offset := (page - 1) * limit
query := r.db.WithContext(ctx).Model(&models.User{})

if err := query.Count(&total).Error; err != nil {
    return nil, 0, err
}

err := query.Offset(offset).Limit(limit).Find(&users).Error
return users, total, err
}

```

การเชื่อมต่อ

```

import (
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
)

func ConnectPostgres(dsn string) (*gorm.DB, error) {
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})
    if err != nil {
        return nil, err
    }
    // Auto migrate
    db.AutoMigrate(&models.User{})
    return db, nil
}

```

DSN format: host=localhost user=gorm password=gorm dbname=gorm port=5432
 sslmode=disable TimeZone=Asia/Bangkok

2. MySQL + GORM

การติดตั้ง

```
go get -u gorm.io/driver/mysql
```

ตัวอย่าง Model และ CRUD (คล้ายกับ PostgreSQL)

Model เหมือนกัน เพียงแต่ driver ต่างกัน

```

import (
    "gorm.io/driver/mysql"
    "gorm.io/gorm"
)

```

```
)

func ConnectMySQL(dsn string) (*gorm.DB, error) {
    db, err := gorm.Open(mysql.Open(dsn), &gorm.Config{})
    if err != nil {
        return nil, err
    }
    db.AutoMigrate(&models.User{})
    return db, nil
}
```

DSN format: `user:pass@tcp(127.0.0.1:3306)/dbname?charset=utf8mb4&parseTime=True&loc=Local`

ความแตกต่างที่ควรระวัง

- **Auto-increment:** MySQL ใช้ `AUTO_INCREMENT`, PostgreSQL ใช้ `SERIAL` หรือ `BIGSERIAL` – GORM จัดการให้อัตโนมัติ
- **JSON type:** PostgreSQL รองรับ JSONB โดยตรง, MySQL ต้องใช้ `json` (เวอร์ชัน 5.7+) – GORM จะ map เป็น string หรือ struct ตาม tags
- **Case sensitivity:** ชื่อตารางและคอลัมน์ใน MySQL บน Windows/Unix อาจ sensitive ขึ้นอยู่กับ configuration

3. MongoDB (NoSQL Document Database)

MongoDB ไม่ใช่ GORM แต่ใช้ official driver `mongo-go-driver`

การติดตั้ง

```
go get go.mongodb.org/mongo-driver/mongo
go get go.mongodb.org/mongo-driver/bson
```

ตัวอย่าง Model และ CRUD

```
package models

import (
    "time"
    "go.mongodb.org/mongo-driver/bson/primitive"
)

type User struct {
    ID          primitive.ObjectID `bson:"_id,omitempty" json:"id"`
    Username    string              `bson:"username" json:"username"`
    Email       string              `bson:"email" json:"email"`
    Password    string              `bson:"password" json:"-"`
    CreatedAt   time.Time           `bson:"created_at" json:"created_at"`
}
```

```
UpdatedAt time.Time          `bson:"updated_at" json:"updated_at"`
}
```

```
package repository

import (
    "context"
    "time"

    "go.mongodb.org/mongo-driver/bson"
    "go.mongodb.org/mongo-driver/bson/primitive"
    "go.mongodb.org/mongo-driver/mongo"
    "go.mongodb.org/mongo-driver/mongo/options"
)

type UserRepository struct {
    collection *mongo.Collection
}

func NewUserRepository(db *mongo.Database) *UserRepository {
    return &UserRepository{
        collection: db.Collection("users"),
    }
}

// Create
func (r *UserRepository) Create(ctx context.Context, user *models.User) error {
    user.ID = primitive.NewObjectID()
    user.CreatedAt = time.Now()
    user.UpdatedAt = time.Now()
    _, err := r.collection.InsertOne(ctx, user)
    return err
}

// Read by ID
func (r *UserRepository) GetByID(ctx context.Context, id string) (*models.User, error) {
    objID, err := primitive.ObjectIDFromHex(id)
    if err != nil {
        return nil, err
    }
    var user models.User
    err = r.collection.FindOne(ctx, bson.M{"_id": objID}).Decode(&user)
    if err == mongo.ErrNoDocuments {
        return nil, nil
    }
    return &user, err
}

// Read by email
```

```

func (r *UserRepository) GetByEmail(ctx context.Context, email string)
(*models.User, error) {
    var user models.User
    err := r.collection.FindOne(ctx, bson.M{"email": email}).Decode(&user)
    if err == mongo.ErrNoDocuments {
        return nil, nil
    }
    return &user, err
}

// Update
func (r *UserRepository) Update(ctx context.Context, user *models.User) error {
    user.UpdatedAt = time.Now()
    filter := bson.M{"_id": user.ID}
    update := bson.M{"$set": user}
    _, err := r.collection.UpdateOne(ctx, filter, update)
    return err
}

// Delete
func (r *UserRepository) Delete(ctx context.Context, id string) error {
    objID, err := primitive.ObjectIDFromHex(id)
    if err != nil {
        return err
    }
    _, err = r.collection.DeleteOne(ctx, bson.M{"_id": objID})
    return err
}

// List with pagination
func (r *UserRepository) List(ctx context.Context, page, limit int)
([]*models.User, int64, error) {
    var users []*models.User
    skip := int64((page - 1) * limit)
    limit64 := int64(limit)

    opts := options.Find().SetSkip(skip).SetLimit(limit64)
    cursor, err := r.collection.Find(ctx, bson.M{}, opts)
    if err != nil {
        return nil, 0, err
    }
    defer cursor.Close(ctx)

    if err = cursor.All(ctx, &users); err != nil {
        return nil, 0, err
    }

    total, err := r.collection.CountDocuments(ctx, bson.M{})
    return users, total, err
}

```

```
import (  
    "context"  
    "go.mongodb.org/mongo-driver/mongo"  
    "go.mongodb.org/mongo-driver/mongo/options"  
)  
  
func ConnectMongo(uri string) (*mongo.Database, error) {  
    client, err := mongo.Connect(context.Background(),  
options.Client().ApplyURI(uri))  
    if err != nil {  
        return nil, err  
    }  
    // Ping to verify connection  
    if err = client.Ping(context.Background(), nil); err != nil {  
        return nil, err  
    }  
    return client.Database("myapp"), nil  
}
```

URI format: `mongodb://localhost:27017`

4. InfluxDB (Time-Series Database)

InfluxDB ใช้ client เฉพาะ `influxdb-client-go`

การติดตั้ง

```
go get github.com/influxdata/influxdb-client-go/v2
```

ตัวอย่าง Model และ CRUD

InfluxDB ไม่มีแนวคิดของ "document" เหมือน MongoDB แต่เป็น **measurement + tags + fields** เหมาะสำหรับข้อมูลอนุกรมเวลา (เช่น อุณหภูมิ, ความชื้น)

```
package repository  
  
import (  
    "context"  
    "time"  
  
    influxdb2 "github.com/influxdata/influxdb-client-go/v2"  
    "github.com/influxdata/influxdb-client-go/v2/api"  
)
```

```

type SensorData struct {
    DeviceID string
    Location string
    Temp     float64
    Humidity float64
    Timestamp time.Time
}

type InfluxRepository struct {
    client    influxdb2.Client
    writeAPI  api.WriteAPIBlocking
    queryAPI  api.QueryAPI
    org       string
    bucket    string
}

func NewInfluxRepository(url, token, org, bucket string) (*InfluxRepository,
error) {
    client := influxdb2.NewClient(url, token)
    writeAPI := client.WriteAPIBlocking(org, bucket)
    queryAPI := client.QueryAPI(org)

    // Test connection
    _, err := queryAPI.Query(context.Background(), `buckets()`)
    if err != nil {
        return nil, err
    }

    return &InfluxRepository{
        client:    client,
        writeAPI:  writeAPI,
        queryAPI:  queryAPI,
        org:       org,
        bucket:    bucket,
    }, nil
}

// Create (Write point)
func (r *InfluxRepository) WriteSensorData(ctx context.Context, data
SensorData) error {
    p := influxdb2.NewPoint(
        "sensor_data",
        map[string]string{
            "device_id": data.DeviceID,
            "location": data.Location,
        },
        map[string]interface{}{
            "temperature": data.Temp,
            "humidity":    data.Humidity,
        },
        data.Timestamp,
    )
    return r.writeAPI.WritePoint(ctx, p)
}

```

```

}

// Read (Query) - example: get average temperature per device for last hour
func (r *InfluxRepository) GetAverageTemp(ctx context.Context, deviceID string,
duration time.Duration) (float64, error) {
    start := time.Now().Add(-duration)
    flux := `from(bucket:"` + r.bucket + `")
        |> range(start: ` + start.Format(time.RFC3339) + `)
        |> filter(fn: (r) => r._measurement == "sensor_data")
        |> filter(fn: (r) => r.device_id == "` + deviceID + `")
        |> filter(fn: (r) => r._field == "temperature")
        |> mean()`

    result, err := r.queryAPI.Query(ctx, flux)
    if err != nil {
        return 0, err
    }

    var avg float64
    for result.Next() {
        record := result.Record()
        if v, ok := record.Value().(float64); ok {
            avg = v
        }
    }
    return avg, result.Err()
}

// Delete (hard delete) - InfluxDB does not support single point deletion
easily; usually you drop data by time range.
func (r *InfluxRepository) DeleteByTimeRange(ctx context.Context, start, stop
time.Time) error {
    deleteAPI := r.client.DeleteAPI()
    return deleteAPI.Delete(ctx, r.org, r.bucket, start, stop, "")
}

// Close
func (r *InfluxRepository) Close() {
    r.client.Close()
}

```

การเชื่อมต่อ

```

func main() {
    repo, err := NewInfluxRepository(
        "http://localhost:8086",
        "my-token",
        "my-org",
        "my-bucket",
    )
}

```

```

    if err != nil {
        panic(err)
    }
    defer repo.Close()
}

```

สรุปตารางเปรียบเทียบ

ฐานข้อมูล	ไดรเวอร์ / ORM	CRUD วิธีหลัก	เหมาะกับงาน
PostgreSQL	GORM + gorm.io/driver/postgres	Create, First, Where, Save, Delete	ระบบที่มีความสัมพันธ์ซับซ้อน ต้องการ ACID
MySQL	GORM + gorm.io/driver/mysql	เช่นเดียวกับ PostgreSQL	Web application ขนาดเล็กถึงกลาง
MongoDB	mongo-go-driver	InsertOne, FindOne, UpdateOne, DeleteOne	Document store, schema-less, scaling สูง
InfluxDB	influxdb-client-go	WritePoint, Query (Flux)	Time-series data, IoT, metrics

ข้อแนะนำในการเลือกใช้:

- หากต้องการ ORM ที่ครบฟีเจอร์และใช้ SQL → **GORM + PostgreSQL/MySQL**
- หากต้องการความยืดหยุ่นของ schema และ scale แนวนอน → **MongoDB**
- หากต้องการเก็บข้อมูลอนุกรมเวลา (เช่น ค่าเซ็นเซอร์) → **InfluxDB**

เทมเพลตและตัวอย่างโค้ดเพิ่มเติม

Repository Interface แบบรวมสำหรับทุกฐานข้อมูล (เพื่อให้ application layer ไม่ขึ้นกับ DB)

```

type UserRepository interface {
    Create(ctx context.Context, user *User) error
    GetByID(ctx context.Context, id string) (*User, error)
    GetByEmail(ctx context.Context, email string) (*User, error)
    Update(ctx context.Context, user *User) error
    Delete(ctx context.Context, id string) error
    List(ctx context.Context, page, limit int) ([]*User, int64, error)
}

```

จากนั้น implement สำหรับแต่ละฐานข้อมูลตามตัวอย่างที่ให้ไว้

การทดสอบด้วย Mock (ตัวอย่างสำหรับ GORM)


```
// ใช้ gorm.io/driver/sqlite สำหรับทดสอบ
func TestUserRepository(t *testing.T) {
    db, _ := gorm.Open(sqlite.Open("file::memory:?cache=shared"),
    &gorm.Config{})
    db.AutoMigrate(&models.User{})
    repo := NewUserRepository(db)

    // ทดสอบ Create
    user := &models.User{Username: "test", Email: "test@example.com", Password:
    "pass"}
    err := repo.Create(context.Background(), user)
    assert.NoError(t, err)
    assert.NotZero(t, user.ID)
}
```

Golang Live-reload เมื่อ code เปลี่ยนแปลง

บทนำ

ในการพัฒนาแอปพลิเคชันด้วย Go (Golang) โดยเฉพาะที่เป็น Web Server หรือ REST API การแก้ไขโค้ดแต่ละครั้งจำเป็นต้องหยุดเซิร์ฟเวอร์, รัน `go build` ใหม่ แล้วเริ่มเซิร์ฟเวอร์อีกครั้ง ซึ่งกระบวนการนี้ซ้ำซากและลดประสิทธิภาพของนักพัฒนา

Live-reload คือเทคนิคที่ช่วยให้แอปพลิเคชันสามารถรีสตาร์ทตัวเองโดยอัตโนมัติเมื่อตรวจพบการเปลี่ยนแปลงของไฟล์ต้นฉบับ นักพัฒนาจะเห็นผลลัพธ์ทันทีโดยไม่ต้องกดคำสั่งใด ๆ ทำให้การพัฒนาเร็วขึ้นและสนุกขึ้น

บทความนี้จะอธิบายความหมาย, วิธีการตั้งค่า, และออกแบบเวิร์กโฟลว์สำหรับ Live-reload ใน Go

บทนิยาม

ศัพท์	คำอธิบาย
Live-reload	การที่เซิร์ฟเวอร์หรือแอปพลิเคชันทำการรีสตาร์ทตัวเองเมื่อมีโค้ดเปลี่ยนแปลง โดยไม่ต้องอาศัยผู้ใช้กด restart
Hot reload	การอัปเดตโค้ดที่กำลังทำงานอยู่โดยไม่ต้องหยุดกระบวนการ (harder in Go)
File watcher	ตัวตรวจสอบการเปลี่ยนแปลงของไฟล์ในระบบ เช่น <code>fsnotify</code>
Process manager	ตัวควบคุมการรันและรีสตาร์ทแอปพลิเคชัน เช่น <code>Air</code> , <code>Fresh</code> , <code>reflex</code>

เครื่องมือที่นิยมใช้ใน Go ecosystem:

- **Air** (แนะนำ) – รองรับ live reload พร้อม config ยืดหยุ่น
- **Fresh** – เรียบง่าย ใช้ง่าย
- **reflex** – ใช้กับคำสั่ง shell
- **CompileDaemon** – ทำงานร่วมกับ `go build`

คู่มือ (Manual)

เราจะใช้ **Air** เป็นหลักเพราะติดตั้งง่ายและมีฟีเจอร์ครบ

1. ติดตั้ง Air

```
go install github.com/air-verse/air@latest
```

ตรวจสอบว่า `$GOPATH/bin` อยู่ใน PATH แล้ว

2. สร้างไฟล์ configuration (`.air.toml`)

ในโฟลเดอร์โปรเจกต์ของคุณ ให้รัน:

```
air init
```

จะได้ไฟล์ `.air.toml` พร้อมค่าเริ่มต้น ปรับแต่งได้ดังนี้

```
# .air.toml
root = "."
tmp_dir = "tmp"

[build]
  args_bin = []
  bin = "./tmp/main"
  cmd = "go build -o ./tmp/main ."
  delay = 1000
  exclude_dir = ["assets", "tmp", "vendor", "log"]
  exclude_file = []
  exclude_regex = ["_test.go"]
  exclude_unchanged = false
  follow_symlink = false
  full_bin = ""
  include_dir = []
  include_ext = ["go", "tpl", "tmpl", "html"]
  include_file = []
  kill_delay = "0s"
  log = "build-errors.log"
  send_interrupt = false
  stop_on_error = true

[color]
  app = ""
  build = "yellow"
  main = "magenta"
  runner = "green"
  watcher = "cyan"
```

```
[log]
  main_only = false
  time = false

[misc]
  clean_on_exit = false

[screen]
  clear_on_rebuild = true
```

3. รันแอปด้วย Air

ในโปรเจกต์ที่ใช้ `main.go`:

```
air
```

ตัวอย่าง `main.go` พื้นฐาน:

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Hello, live reload!")
    })
    http.ListenAndServe(":8080", nil)
}
```

เมื่อคุณแก้ไข `main.go` หรือไฟล์ `.html` ใด ๆ ที่ระบุไว้ใน `include_ext` Air จะทำการ:

- Build โปรเจกต์ใหม่
- หยุดกระบวนการเก่า
- รันกระบวนการใหม่

4. ใช้กับ Web Browser (Frontend Live Reload)

Air จัดการฝั่ง Go backend เท่านั้น ถ้าต้องการให้หน้าเว็บ reload อัตโนมัติเมื่อเทมเพลต HTML หรือไฟล์ CSS เปลี่ยน ให้ใช้เครื่องมือ frontend เช่น **BrowserSync** หรือเพิ่ม WebSocket ใน Go:

ตัวอย่างแบบง่าย: ใช้ `air` ร่วมกับ `air + browser-sync`

```
# terminal 1
air

# terminal 2
browser-sync start --proxy "localhost:8080" --files "templates/*.html"
```

หรือเพิ่ม middleware ใน Go ที่ inject script สำหรับ WebSocket reload (advanced)

5. ทางเลือกอื่น (Fresh, reflex)

Fresh (ติดตั้ง: `go install github.com/pilu/fresh@latest`)

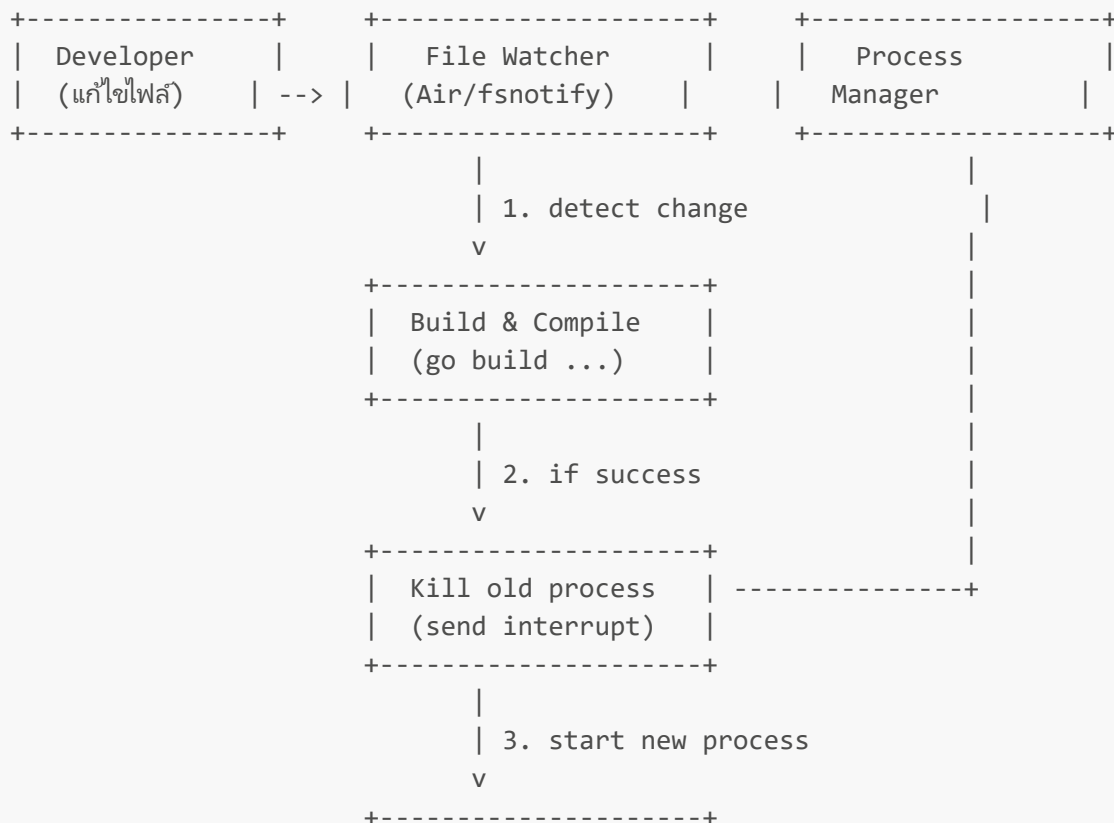
```
fresh
```

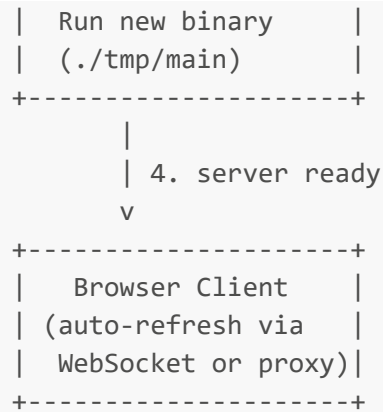
reflex (ติดตั้งผ่าน brew หรือ binary)

```
reflex -s -r '\.go$' -- sh -c 'go build -o app && ./app'
```

ออกแบบ Workflow

ด้านล่างคือเวิร์กโฟลว์ของ Live-reload ที่สมบูรณ์สำหรับ Go Web Application





คำอธิบายแต่ละขั้น

1. **File Watcher** ตรวจสอบการเปลี่ยนแปลงในไฟล์ `.go`, `.html`, `.tpl` ฯลฯ
2. เมื่อมีการเปลี่ยนแปลง → ทำงาน `go build` ไปยังไดเรกทอรีชั่วคราว (เช่น `tmp/`)
3. ถ้า build สำเร็จ → ส่งสัญญาณ `os.Interrupt` หรือ `SIGKILL` ไปยังกระบวนการเก่า
4. รันไบนารีใหม่จาก `tmp/` พร้อมกับอาร์กิวเมนต์เดิม
5. **Optional** ส่งสัญญาณไปยัง browser (ผ่าน WebSocket หรือ SSE) ให้รีเฟรชหน้า
6. นักพัฒนาเห็นผลทันที โดยไม่ต้องกด restart เอง

การจัดการทรัพยากร (Port binding, connections)

- ต้องแน่ใจว่า process เก่าปล่อย port ก่อนที่ process ใหม่จะ bind (ใช้ `delay` หรือ `kill_delay`)
- ใช้ `send_interrupt = true` เพื่อให้ Go server จัดการ graceful shutdown

```

// graceful shutdown snippet
srv := &http.Server{Addr: ":8080", Handler: handler}
go srv.ListenAndServe()

c := make(chan os.Signal, 1)
signal.Notify(c, os.Interrupt, syscall.SIGTERM)
<-c
srv.Shutdown(context.Background())

```

ตัวอย่าง workflow จริง (ใช้ Air)

```

$ cd myproject
$ air

  _/ _\  | | | _|
/_/--\  |_| |_| \_ , built with Go

watching .
!exclude tmp
building...

```

```
running...  
listening on :8080
```

แก้ไข `main.go` → Air จะแสดง:

```
building...  
running...
```

เซิร์ฟเวอร์ restart โดยอัตโนมัติ พร้อมให้บริการทันที

สรุป

Live-reload เป็นเครื่องมือสำคัญที่ช่วยเพิ่มความเร็วในการพัฒนา Go โดยเฉพาะกับเว็บแอปพลิเคชัน เราแนะนำให้ใช้ **Air** เพราะตั้งค่าง่าย, config ได้ละเอียด, และมี community แข็งแกร่ง การออกแบบเวิร์กโฟลว์ที่เหมาะสมจะรวมทั้ง file watcher, auto build, process management และ optional browser refresh เพื่อประสบการณ์การพัฒนาที่ลื่นไหล

แผนภาพและตัวอย่างโค้ดนี้ครอบคลุมการทำ CRUD กับฐานข้อมูลหลัก 4 ประเภทที่ใช้บ่อยใน Go โดยเน้นการใช้ GORM สำหรับ SQL และไดรเวอร์เฉพาะสำหรับ NoSQL/Time-series พร้อมแนวทางการแยก repository layer เพื่อให้แอปพลิเคชันสามารถเปลี่ยนฐานข้อมูลได้ง่ายในอนาคต

🔗 สรุป

คู่มือภาษา Go ฉบับสมบูรณ์นี้ครอบคลุมเนื้อหาตั้งแต่พื้นฐานภาษา Go ไปจนถึงการออกแบบสถาปัตยกรรมระดับองค์กรและการผสานระบบภายนอกที่ใช้ในโลกแห่งความเป็นจริง

จุดเด่นของคู่มือ

1. **ครบถ้วนสมบูรณ์** - ครอบคลุมทุกหัวข้อตั้งแต่พื้นฐานจนถึงขั้นสูง
2. **แผนภาพประกอบ** - มีแผนภาพในรูปแบบ Mermaid/draw.io สำหรับอธิบายโครงสร้างและกระบวนการทำงาน
3. **โค้ดที่รันได้จริง** - ตัวอย่างโค้ดทั้งหมดสามารถนำไปรันทดสอบได้
4. **เทมเพลตและ Checklist** - เครื่องมือช่วยในการพัฒนาและการนำเสนอซอฟต์แวร์
5. **แนวทางปฏิบัติ** - สอดแทรก best practices และข้อควรระวัง

การนำคู่มือไปใช้

- **ผู้เริ่มต้น** ควรศึกษาเนื้อหาตามลำดับตั้งแต่ภาคที่ 1 ถึงภาคที่ 3
- **นักพัฒนาที่มีประสบการณ์** สามารถข้ามไปยังภาคที่ 7-9 เพื่อศึกษา Clean Architecture, DDD และการผสานระบบภายนอก
- **ทีมพัฒนา** สามารถนำ Task List Template และ Checklist Template ไปปรับใช้ในการทำงาน

ผู้เขียน: คงนคร จันทะคุณ
อีเมล: kongnakornjantakun@gmail.com
วันที่: เมษายน 2026

** ประวัติของผู้เขียน:**

เส้นทางนักพัฒนาซอฟต์แวร์: จากโค้ดบรรทัดแรกสู่ผู้นำทางเทคนิค

ทุกเส้นทางของนักพัฒนาซอฟต์แวร์มักเริ่มต้นจากความอยากรู้อยากเห็น และสำหรับผม เส้นทางนั้นเริ่มขึ้นเมื่อราว 15 ปีก่อน ด้วยพื้นฐานด้านวิทยาการคอมพิวเตอร์จากมหาวิทยาลัยราชภัฏพิบูลสงคราม ก่อนจะขยับขยายขอบเขตความรู้ไปสู่ศาสตร์อีกด้านอย่างนิติศาสตร์จากมหาวิทยาลัยสุโขทัยธรรมมาธิราช และประกาศนียบัตรทนายความจากสภานายความ ความรู้ด้านกฎหมายนี้เองที่กลายเป็นจุดแข็งสำคัญ ช่วยให้ผมมองเห็นภาพรวมของธุรกิจ เข้าใจข้อกำหนด และสื่อสารกับผู้ใช้งานได้อย่างมีประสิทธิภาพมากขึ้นในเวลาต่อมา

จุดเริ่มต้นของนักปฏิบัติ

ปี 2549 ผมเริ่มต้นเส้นทางสายไอทีในฐานะ IT Support และ PHP Web Developer ที่บริษัท คอนโทรล ดาต้า (ประเทศไทย) จำกัด ช่วงเวลานั้นเป็นช่วงของการเรียนรู้พื้นฐานอย่างแท้จริง ทั้งการดูแล Data Center การให้บริการลูกค้า และการพัฒนาเว็บไซต์ด้วย PHP ซึ่งเป็นรากฐานสำคัญที่ทำให้เข้าใจทั้งด้านเทคนิคและความต้องการของผู้ใช้ไปพร้อมกัน

ก้าวสู่โลกของเนื้อหาคุณภาพ

ปี 2552 ผมได้ร่วมงานกับบริษัท สยามสปอร์ต ซินดิเคท จำกัด (มหาชน) ในตำแหน่ง Senior Developer นี่คือช่วงเวลาที่ได้พัฒนาทักษะการเขียน PHP และจัดการฐานข้อมูล MySQL, MSSQL ในการขับเคลื่อนเว็บไซต์ข่าวกีฬาชื่อดังอย่าง siamsport.co.th ซึ่งต้องรองรับผู้เข้าชมจำนวนมากและต้องการประสิทธิภาพสูง การดูแลระบบ การทดสอบซอฟต์แวร์ และการฝึกอบรมผู้ใช้ในช่วงนี้ ทำให้เข้าใจความสำคัญของซอฟต์แวร์ที่เสถียรและใช้งานง่าย

ขยายขอบเขตกับองค์กรใหญ่

ปี 2559 ถือเป็นอีกหนึ่งจุดเปลี่ยนสำคัญ เมื่อผมเข้าร่วมงานกับบริษัท ทรู คอร์ปอเรชั่น จำกัด (มหาชน) ในตำแหน่ง Senior Full Stack Developer ที่นี่ผมได้ขยายขีดความสามารถจากการใช้ PHP ไปสู่ Node.js, TypeScript และการจัดการฐานข้อมูลที่หลากหลาย ทั้ง MySQL, MSSQL รวมถึงการใช้ ORM อย่าง TypeORM อย่างมืออาชีพ

ผลงานสำคัญในช่วงนี้คือการมีส่วนร่วมพัฒนาและปรับปรุงระบบการศึกษาระดับประเทศอย่าง connected.org และ trueplookpanya.com ซึ่งเป็นแพลตฟอร์มที่เข้าถึงผู้ใช้งานจำนวนมาก การได้ร่วมกับทีมออกแบบกระบวนการทางธุรกิจ วางแผนพัฒนา ทำ UAT และให้การสนับสนุนผู้ใช้ เปิดมุมมองว่าการพัฒนาซอฟต์แวร์ไม่ได้จบที่การเขียนโค้ด แต่ต้องทำให้ผู้ใช้ใช้งานได้จริงและเกิดประโยชน์สูงสุด

พลิกโฉมด้วยเทคโนโลยีใหม่

ปัจจุบัน ผมดำรงตำแหน่ง Technical Lead / Full Stack Software Engineer ที่บริษัท ซีทีซี โกลบอล (ประเทศไทย) จำกัด บทบาทนี้ท้าทายกว่าเดิมมาก เพราะนอกจากจะต้องพัฒนาระบบด้วย Node.js, TypeScript, Python แล้ว ยังต้องวางสถาปัตยกรรมซอฟต์แวร์ กำหนดมาตรฐานการเขียนโค้ด และดูแลทีมพัฒนา ทั้ง Developer, Application Support และ DevOps

สิ่งที่ทำให้ช่วงเวลานี้พิเศษคือการได้นำเทคโนโลยีสมัยใหม่มาประยุกต์ใช้อย่างจริงจัง:

- **ระบบอัตโนมัติ** ด้วย n8n และ UiPath RPA ช่วยลดงานซ้ำซ้อนและเพิ่มประสิทธิภาพการทำงาน
- **Generative AI** ผสาน OpenAI API เข้ากับแอปพลิเคชันเพื่อสร้างฟีเจอร์อัจฉริยะ
- **CI/CD Pipeline** ตั้งค่า Jenkins, GitHub Actions พร้อม Quality Gates (SonarQube, ESLint, Test Coverage) เพื่อให้การ deploy มั่นใจและปลอดภัย
- **ระบบ Monitoring** ด้วย Grafana และ ELK Stack เพื่อให้เห็นภาพรวมของระบบแบบ Real-time

ความสำเร็จที่ภูมิใจ

ตลอดเส้นทางที่ผ่านมา มีผลงานหลายอย่างที่สร้างความภูมิใจ:

-- **PoC <http://cmoniot.trueddns.com:52160> ที่นำ IoT มาติดตามอุณหภูมิ น้ำรั่ว และควัน พร้อม Dashboard แจ้งเตือนแบบ Real-time

- **การเป็นวิทยากรและที่ปรึกษา** ถ่ายทอดความรู้ด้าน DevOps, การเขียนโค้ดคุณภาพ, Design Thinking และ Business Model Canvas ให้ทีมพัฒนาทั้งภายในและภายนอกองค์กร
- **การลดข้อผิดพลาดในการ deploy** จากการนำ Branch Protection และ CI/CD มาบังคับใช้ เพิ่มความมั่นใจให้ทีม

จุดแข็งที่สร้างความแตกต่าง

นอกเหนือจากทักษะทางเทคนิคที่หลากหลาย จุดแข็งที่ทำให้การทำงานประสบความสำเร็จคือ:

- **การสื่อสารที่มีประสิทธิภาพ** สามารถอธิบายเรื่องเทคนิคให้คนที่ไม่ใช่สายเทคนิคเข้าใจ และทำงานร่วมกับทีมธุรกิจได้อย่างราบรื่น
- **การแก้ปัญหาอย่างเป็นระบบ** วิเคราะห์หาสาเหตุที่แท้จริง และเสนอแนวทางที่เหมาะสม ไม่ใช่แค่แก้ปัญหาที่ปลายเหตุ
- **การเรียนรู้เร็วและปรับตัวเก่ง** ติดตามเทคโนโลยีใหม่อย่างสม่ำเสมอ และพร้อมนำมาประยุกต์ใช้เมื่อเห็นว่าจะเกิดประโยชน์
- **ภาวะผู้นำและการทำงานเป็นทีม** ให้คำปรึกษา สนับสนุนสมาชิก และสร้างบรรยากาศที่ทุกคนเติบโตไปด้วยกัน

สายธารแห่งการเรียนรู้ที่ไม่สิ้นสุด

ความมุ่งมั่นในการพัฒนาตนเองสะท้อนผ่านการเรียนคอร์สออนไลน์มากมาย ตั้งแต่ Data Science, Machine Learning, Docker & Kubernetes ไปจนถึง DevOps และการพัฒนา Full Stack ด้วย Angular & Nest JS รวมถึงการอบรม Business Model Canvas, Design Thinking, Agile & Scrum จาก True Digital Academy และ General Assembly

เส้นทางกว่าสิบห้าปีที่ผ่านมา สอนให้รู้ว่าการพัฒนาซอฟต์แวร์ที่ดีไม่ได้มีแค่ทักษะการเขียนโค้ด แต่ต้องเข้าใจธุรกิจ มีวิสัยทัศน์ในการออกแบบระบบ มีทักษะการนำทีม และที่สำคัญคือต้องมีความอยากรู้อยากเรียนรู้อย่างต่อเนื่อง ผมเชื่อว่าทุกโครงการ ทุกบรรทัดโค้ดที่เขียนขึ้น ล้วนเป็นส่วนหนึ่งในการขับเคลื่อนองค์กรและสร้างประสบการณ์ที่ดีให้ผู้ใช้ และนั่นคือความภาคภูมิใจของนักพัฒนาทุกคน